SQL Masterclass: Data Engineering & Analytics

Personal Study Notes & Technical Reference — by **Youssef Hany**

Instructor: Data with Baraa | **Duration:** 30 Hours

> 📝 **Summary**
> This note acts as my comprehensive study guide and practical reference manual. It documents my journey mastering SQL for big data environments—bridging the gap between raw data extraction and optimized data architecture. It contains syntax rules, performance tuning strategies, architectural blueprints, and real-world project code (ETL pipelines, Exploratory Data Analysis, and Advanced Analytics).

https://github.com/Youssef-Hany1/SQL-Course

Table of Contents

Table of Contents
Module 1: Introduction to SQL
- Database (DB)
- SQL (Structured Query Language)
- DBMS (Database Management System)
- Server
- Database Types
- SQL Database
- SQL Commands

Module 2: SELECT Queries
- Components of SQL
- SELECT & FROM
- WHERE
- ORDER BY
- GROUP BY
- ALIAS (AS)
- HAVING
- Filter your data
- DISTINCT
- TOP

- Coding & Execution Order

Module 3: Data Definition Language (DDL)

- CREATE
- ALTER
- DROP

Module 4: Data Manipulation Language (DML)

- INSERT
- UPDATE
- DELETE

Module 5: Filtering Data

- WHERE Operators
- Comparison Operators
- Logical Operators
- Range Operator
- Membership Operator
- Search Operator

Module 6: SQL Joins

- Combining Data
- What is SQL JOINs
- When to use SQL JOINs
- JOIN Types
- JOIN Code
- NO JOIN
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- LEFT ANTI JOIN
- RIGHT ANTI JOIN
- FULL ANTI JOIN
- CROSS JOIN
- How to choose between JOIN Types?
- Multi-Table JOIN
- Multi-Table JOIN Example

Module 7: SQL SET Operators

- What is SET Operators
- Combining Data
- Syntax
- Rules
- UNION
- UNION ALL
- UNION ALL vs UNION
- EXCEPT (MINUS)
- INTERSECT
- Use Cases
- Notes

- Summary

Module 8: String Functions

- SQL Functions
- String Functions
- CONCAT
- UPPER
- LOWER
- TRIM
- REPLACE
- LEN
- LEFT
- RIGHT
- SUBSTRING

Module 9: Numeric Functions

- String Functions
- ROUND
- ABS

Module 10: Date & Time Functions

- What is Date & Time
- Date & Time Functions
- DAY, MONTH, YEAR
- DATEPART
- DATENAME
- DATETRUNC
- EOMONTH
- Notes
- Part Extraction
- **Introduction to Formatting & Casting**
- Formatting
- Casting
- FORMAT
- CONVERT
- CAST
- FORMAT vs CONVERT vs CAST
- DATEADD
- DATEDIFF
- ISDATE

Module 11: NULL Functions

- Introduction to NULLs
- NULL Functions
- ISNULL
- COALESCE
- ISNULL vs COALESCE
- Handling NULLs Cases (ISNULL & COALESCE)
- NULLIF
- IS NULL

- IS NOT NULL
- Use Cases (IS NULL & IS NOT NULL)
- NULL vs Empty String vs Blank Space
- Handling NULLs - Data Policies
- Summary

Module 12: CASE WHEN Statement

- Introduction to CASE Statements
- How It Works?
- **Use Case – Categorizing Data**
- CASE Rules
- **Use Case – Mapping Values**
- Quick Form
- **Use Case – Handling NULLs**
- **Use Case – Conditional Aggregation**
- Summary

Module 13: Window Functions Basics

- Aggregate Functions
- WINDOW vs GROUP BY
- **Syntax of Window Functions**
- Types of Window Functions
- Window Expression
- Window PARTITION BY
- Window ORDER BY
- Window Frame
- COMPACT FRAME
- Window Functions Rules
- Summary

Module 14: Window Aggregate Functions

- **Introduction to Window Aggregate Functions**
- Syntax
- COUNT
- SUM
- AVG
- MIN / MAX
- Comparison Use Cases
- **Use Case: Running & Rolling Total**
- **Use Case: Moving Average**
- Summary

Module 15: Window Ranking Functions

- **Introduction to Ranking Window Functions**
- Syntax
- ROW_NUMBER
- RANK
- DENSE_RANK
- ROW_NUMBER vs RANK vs DENSE_RANK
- ROW_NUMBER USE CASE: TOP-N ANALYSIS

ROW_NUMBER USE CASE: BOTTOM-N ANALYSIS
ROW_NUMBER USE CASE: GENERATE UNIQUE IDS
ROW_NUMBER USE CASE: IDENTIFY DUPLICATES
NTILE
NTILE USE CASE: DATA SEGMENTATION
NTILE USE CASE: EQUALIZING LOAD
CUME_DIST
PERCENT_RANK
CUME_DIST vs PERCENT_RANK
Summary
Module 16: Window Value Functions
Introduction to Window Value Functions
LEAD / LAG
LEAD/LAG USE CASE: Month-Over-Month (MoM) ANALYSIS
LEAD/LAG USE CASE: CUSTOMER RETENTION ANALYSIS
LEAD/LAG USE CASE: TIME GAP ANALYSIS
FIRST_VALUE / LAST_VALUE
FIRST_VALUE/LAST_VALUE USE CASE: COMPARE TO EXTREMES
Summary
SQL WINDOW USE CASES
Module 17: Subqueries
Advanced SQL Techniques
Simplified Database Architecture
SUBQUERY
How Subquery Works
Why Subqueries
Subquery Categories
Subquery Result Types
Subquery in FROM Clause
How Database Execute Subqueries
Subquery in SELECT Clause
Subquery in JOIN Clause
Subquery in WHERE Clause
Subquery using IN Operator
Subquery using ALL & ANY Operator
Non-Correlated | Correlated Subqueries
Subquery using EXISTS Operator
Subquery vs JOINs
Summary
Module 18: Common Table Expression (CTE)
Why CTE ?
How Database Executes CTE
CTE Types
Non-Recursive CTE
Standalone CTE
Multiple Standalone CTE

- Nested CTE
- CTE Best Practices
- Recursive CTE
- Summary

Module 19: Views

- Database Structure
- 3 Levels Architecture
- What is View?
- VIEW vs TABLE
- VIEWS USE CASE: Central Complex Query Logic
- VIEW vs CTE
- CREATE View
- DROP View
- ALTER View
- How Database Executes Views
- VIEWS USE CASE: Hide Complexity
- VIEWS USE CASE: Data Security
- VIEWS USE CASE: Flexibility & Dynamic
- VIEWS USE CASE: Multi-Language
- VIEWS USE CASE: Virtual Data Marts in Data Warehouse
- Summary

Module 20: CTAS & TEMP Tables

- Database Table
- Table Types
- CTAS vs CREATE/INSERT
- CTAS vs VIEWS
- CREATE / INSERT Syntax
- CTAS Syntax
- How to Refresh CTAS
- CTAS USE CASE: Optimize Performance
- CTAS USE CASE: Creating a Snapshot
- CTAS USE CASE: Physical Data Marts in Data Warehouse
- Temporary Tables
- Temporary Tables Syntax
- Example
- How Database Execute TEMP Tables
- TEMP USE CASE: Intermediate Results
- SQL Tables Summary
- Compare Advanced SQL Techniques
- SQL Project Big Picture

Module 21: Stored Procedures

- Stored Procedure vs Query
- Stored Procedure vs Coding with Python
- Stored Procedure Syntax
- Stored Procedure Parameters
- Stored Procedure Multi-Queries

- Stored Procedure Variables
- Stored Procedure Control Flow (IF ELSE)
- Stored Procedure Error Handling (TRY & CATCH)
- Styling Stored Procedure

Module 22: Triggers

- What are Triggers
- Triggers Syntax
- TRIGGERS USE CASE: LOGGING

Module 23: Indexes

- Index Types
- Database Tables Without Indexes
- Heap Structure
- Clustered Index
- Clustered Index vs Heap Structure
- Non-Clustered Index
- Clustered vs Non-Clustered Index
- Clustered & Non-Clustered Index Syntax
- Composite Index
- Columnstore & Rowstore Index
- How SQL Builds Columnstore (The Process)
- Clustered Columnstore Index vs Non-Clustered Columnstore Index
- Columnstore vs Rowstore Index
- Columnstore Index Syntax
- Comparison (HEAP, COLUMNSTORE & ROWSTORE)
- Unique Index
- Unique Index Syntax
- Filtered Index
- Filtered Index Syntax
- Choose the Right Index
- Index Management & Monitoring
- Monitor Index Usage
- Monitor Missing Indexes
- Monitor Duplicate Indexes
- Update Statistics
- Monitor Fragmentations
- Execution Plan
- Execution Plan Basics
- Execution Plan: Heap vs Clustered Index
- Execution Plan: Non-Clustered Index
- Types of Scan
- Join Algorithms
- Execution Plan: Rowstore vs Columnstore
- Execution Plan Summary
- SQL Hints
- Indexes Summary (Indexing Strategy)

Module 24: Partitions

- Introduction to Partitions
- Building Partitions: #1 Create Partition Function
- Building Partitions: #2 Create Filegroups
- Building Partitions: #3 Create Data Files
- Building Partitions: #4 Create Partition Scheme
- Building Partitions: #5 Create Partitioned Table
- Building Partitions: #6 Insert Data
- Everything is Connected
- The Performance (Execution Plan)
- Partitioning & Indexes

Module 25: Performance Tips

- The Golden Rule
- Fetching Data
- Filtering Data
- Joining Data
- Aggregating Data
- Subqueries
- Creating Tables (DDL)
- Indexing
- Partitioning
- Summary

Module 26: AI & SQL

- When to use ChatGPT & Copilot
- ChatGPT Prompt
- Top Prompts For Developers
- Top Prompts For Learning

Module 27: SQL Data Warehouse Project

- SQL Projects
- **Data Warehouse (DWH)**
- ETL
- Data Architectures
- Design the Layers
- Separation of Concerns (SOC)
- Naming Conventions
- Git
- Bronze Layer
- Silver Layer
- Gold Layer
- Data Modeling
- Theory: Star Schema vs Snowflake Schema
- Theory: Dimensions vs Facts

Module 28: **Exploratory & Advanced Data Analytics Project**

- SQL Projects
- Dimensions vs Measures
- Data Analytics Project Roadmap
- Exploratory Data Analysis (EDA)

- Database Exploration
- Dimension Exploration
- Data Exploration
- Measure Exploration
- Magnitude Exploration
- Ranking Analysis

Advanced Data Analytics

- Change Over Time Analysis
- Cumulative Analysis
- Performance Analysis
- Part to Whole Analysis
- Data Segmentation
- Build Report

Module 1: Introduction to SQL

Database (DB)

- Stores data

SQL (Structured Query Language)

- Speaks to the database

DBMS (Database Management System)

- Manages all requests to the database
- Makes the priority which SQL must be executed first
- Manages the security whether SQL is allowed to be executed in the first place

Server

- Where database lives

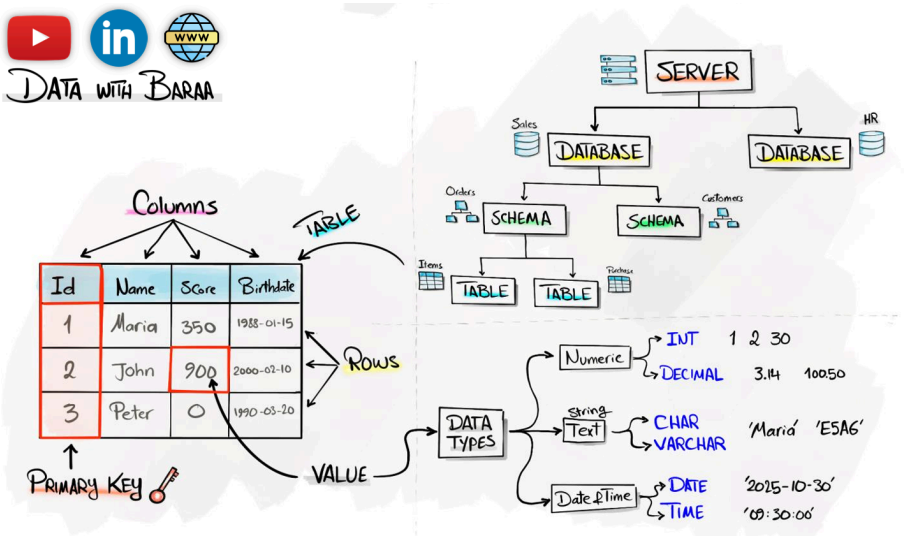
Database Types

- SQL
  - Relational (Microsoft SQL Server / MySQL / PostgreSQL)
- No SQL
  - Key-Value (Redis / Amazon DynamoDB)
  - Column-Based (Apache Cassandra / Amazon Redshift)
  - Graph (Neo4j)
  - Document (MongoDB)

SQL Database

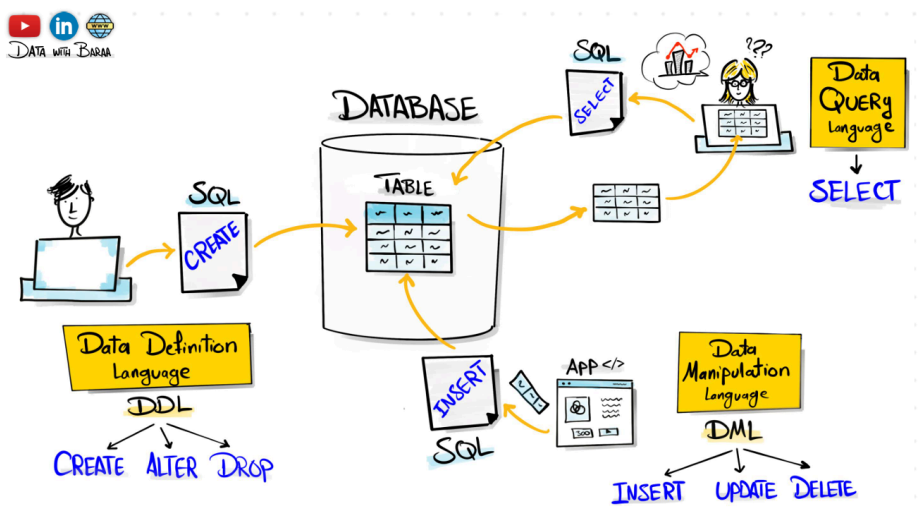
- Server contains databases
- Database contains Schemas
- Schema contains Tables
- Table contains Columns, Rows, Primary Key (unique identifier), VALUE
- VALUE: DATA TYPES

- Numeric
  - INT (1,2,15)
  - DECIMAL (3.14,8.5)
- Text (String)
  - CHAR ('Youssef')
  - VARCHAR ('E5A6')
- Date & Time
  - DATE ('2003-01-01')
  - TIME ('09:30:00')



SQL Commands

- Data Definition Language (DDL)
  - CREATE, ALTER, DROP
- Data Manipulation Language (DML)
  - INSERT, UPDATE, DELETE
- Data Query Language
  - SELECT



Module 2: SELECT Queries

Components of SQL

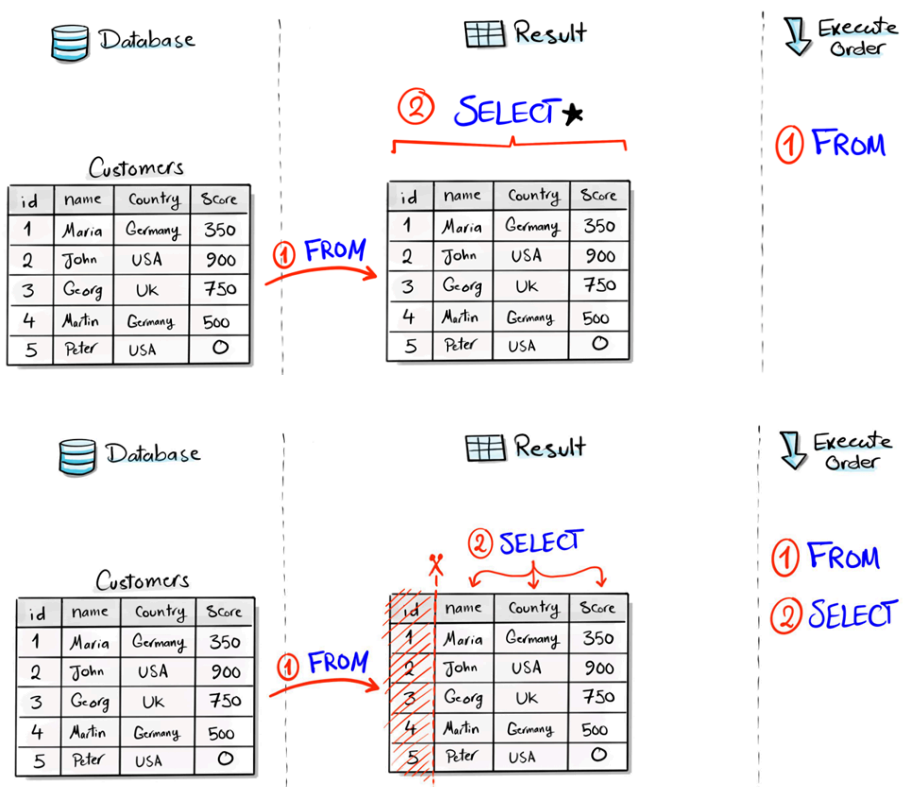
- Clauses
- Keywords
- Functions
- Identifiers

- Operators
- Values
- Comments

SELECT & FROM

- Syntax

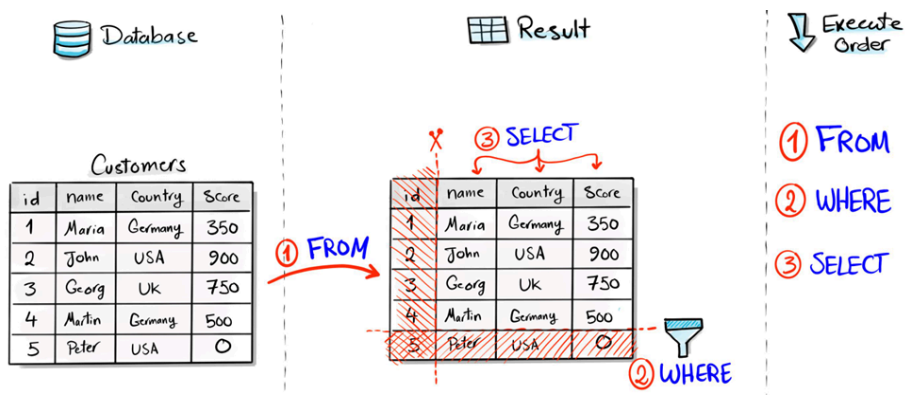
```sql
-- Everything
SELECT *
FROM table_name

-- Select Few Columns
SELECT
    Col_1,
    Col_2
FROM table_name
```



WHERE

- Filters your data based on a condition
- Syntax

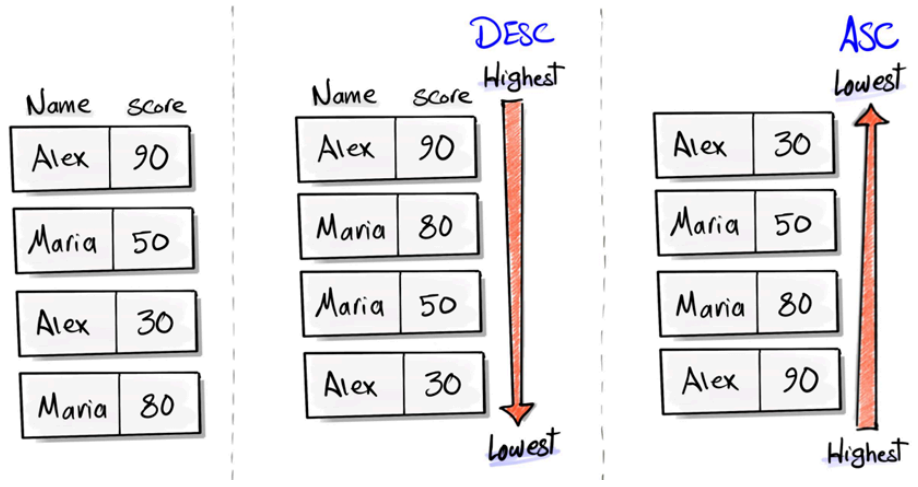
```sql
SELECT *
FROM table_name
WHERE Condition
```



ORDER BY

- Sorts your data

- ASC (Lowest → Highest)
- DESC (Highest → Lowest)

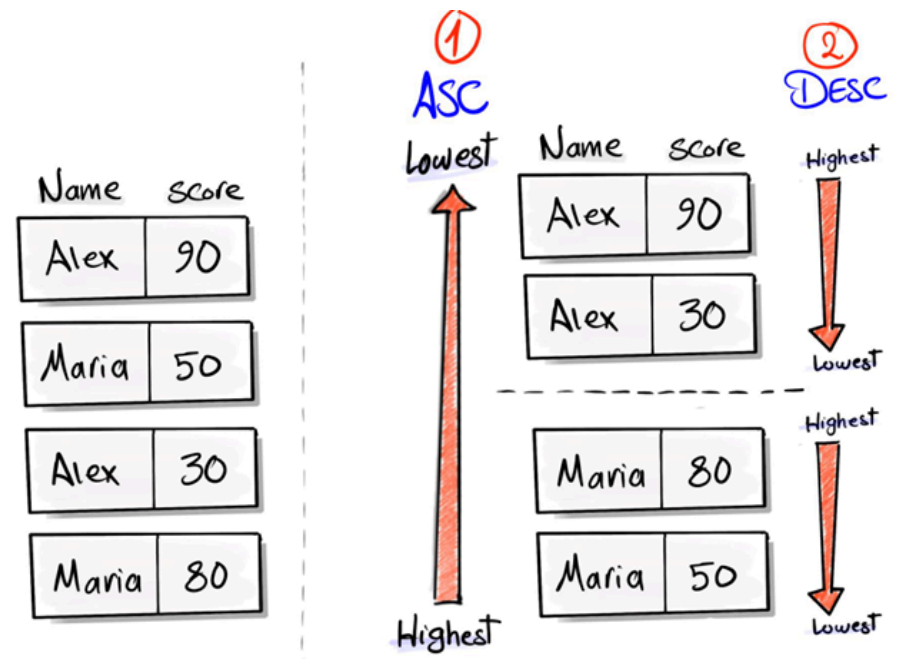


- Syntax

```sql
SELECT *
FROM table_name
ORDER BY Col ASC
```

  - Default → ASC
- Nested ORDER BY
  - Syntax

```sql
SELECT *
FROM table_name
ORDER BY
    Column_1 ASC,
    Column_2 DESC
```



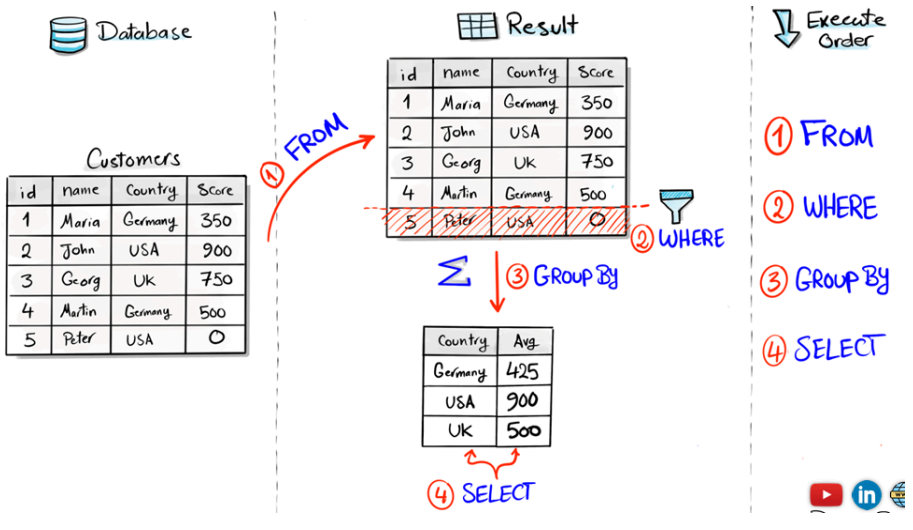
GROUP BY

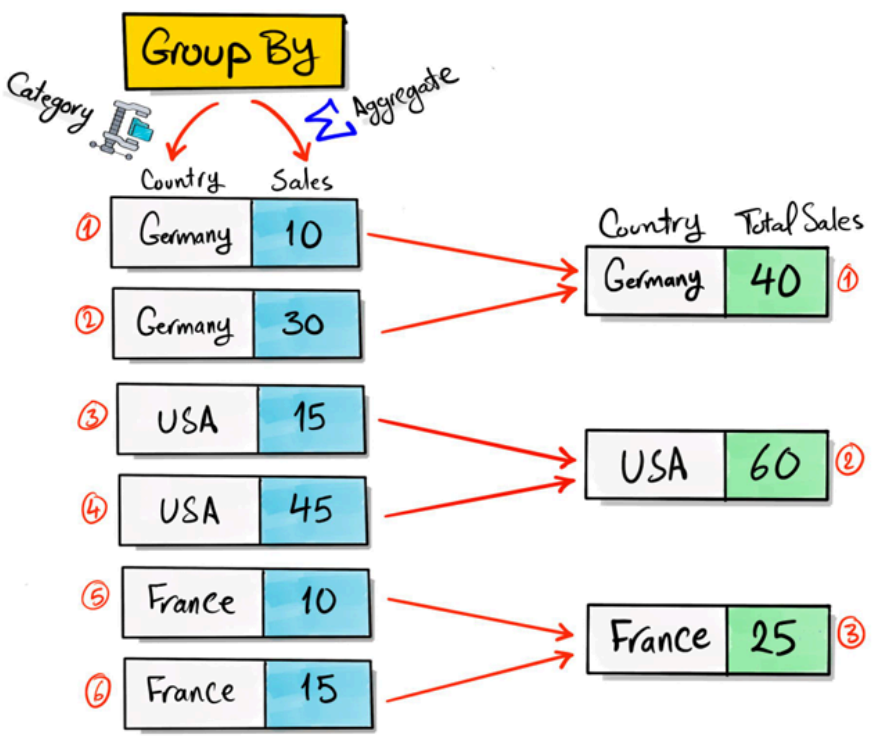
- Aggregates your data
  - Combines rows with the same value
  - Aggregates a column by another column
- Syntax

```sql
SELECT
    country,
    SUM(score) AS total_score,
    COUNT(id) AS total_customers
```

```
FROM table_name
GROUP BY country
```

- country → category
- SUM(score), COUNT(id) → aggregation





ALIAS (AS)

- Shorthand name (LABEL) assigned to a column or table in a query

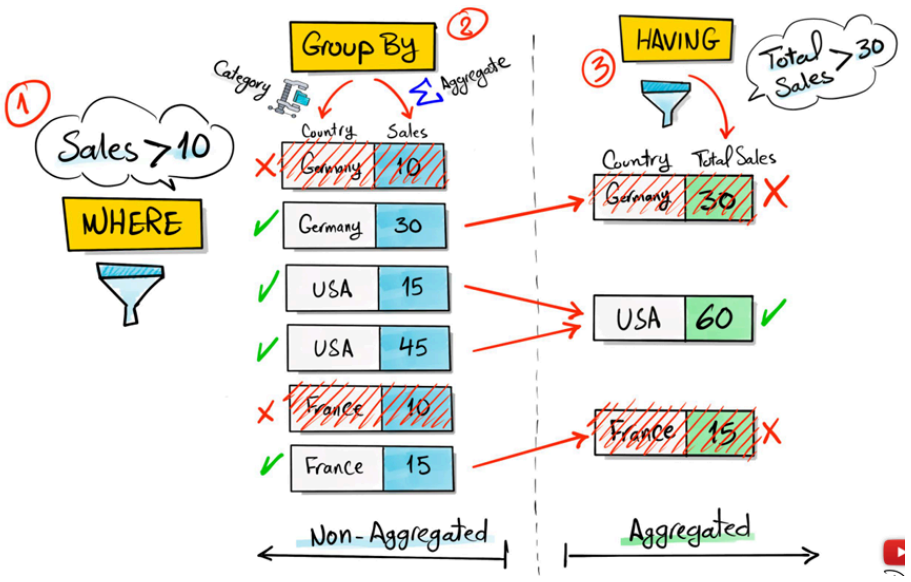
HAVING

- Filters data after aggregation
  - Can be used only with GROUP BY
- Syntax

```
SELECT
    country,
    SUM(score) AS total_score
FROM table_name
GROUP BY country
HAVING SUM(score) > 800 /*condition*/
```

Filter your data

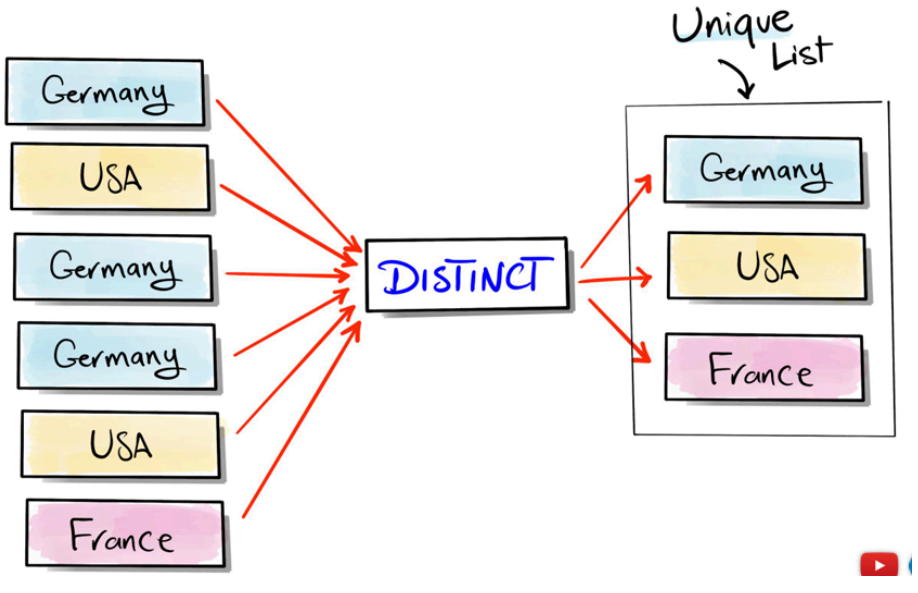
- Before aggregation → WHERE
- After aggregation → HAVING



DISTINCT

- Removes duplicates (repeated values)
    - Each value appears only once
- Syntax

```
SELECT DISTINCT country
FROM table_name
```



TOP

- Limits your data
    - Restrict the number of rows returned
- Syntax

```
SELECT TOP 3 *
FROM table_name
```

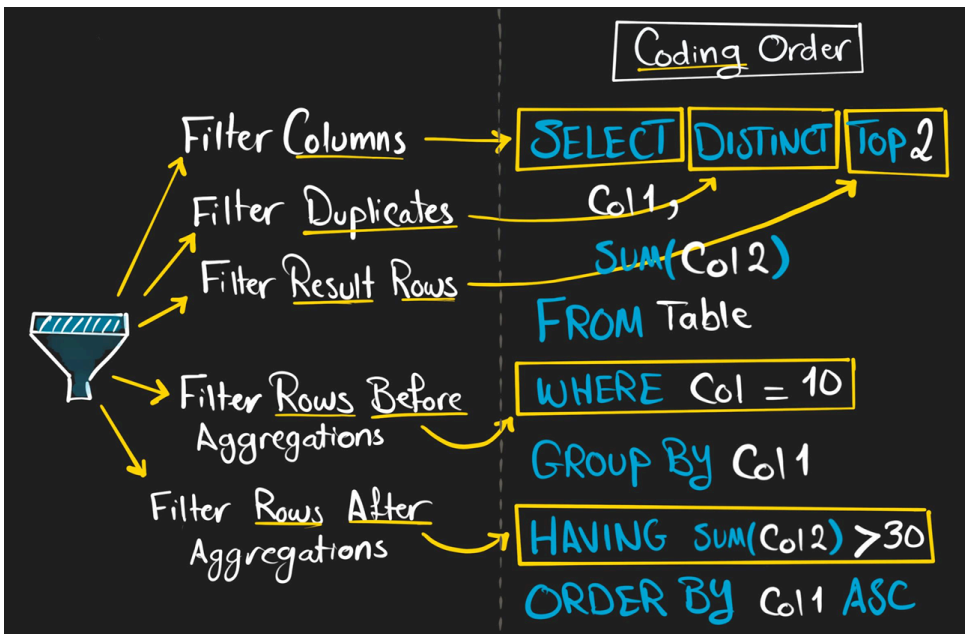
Coding & Execution Order

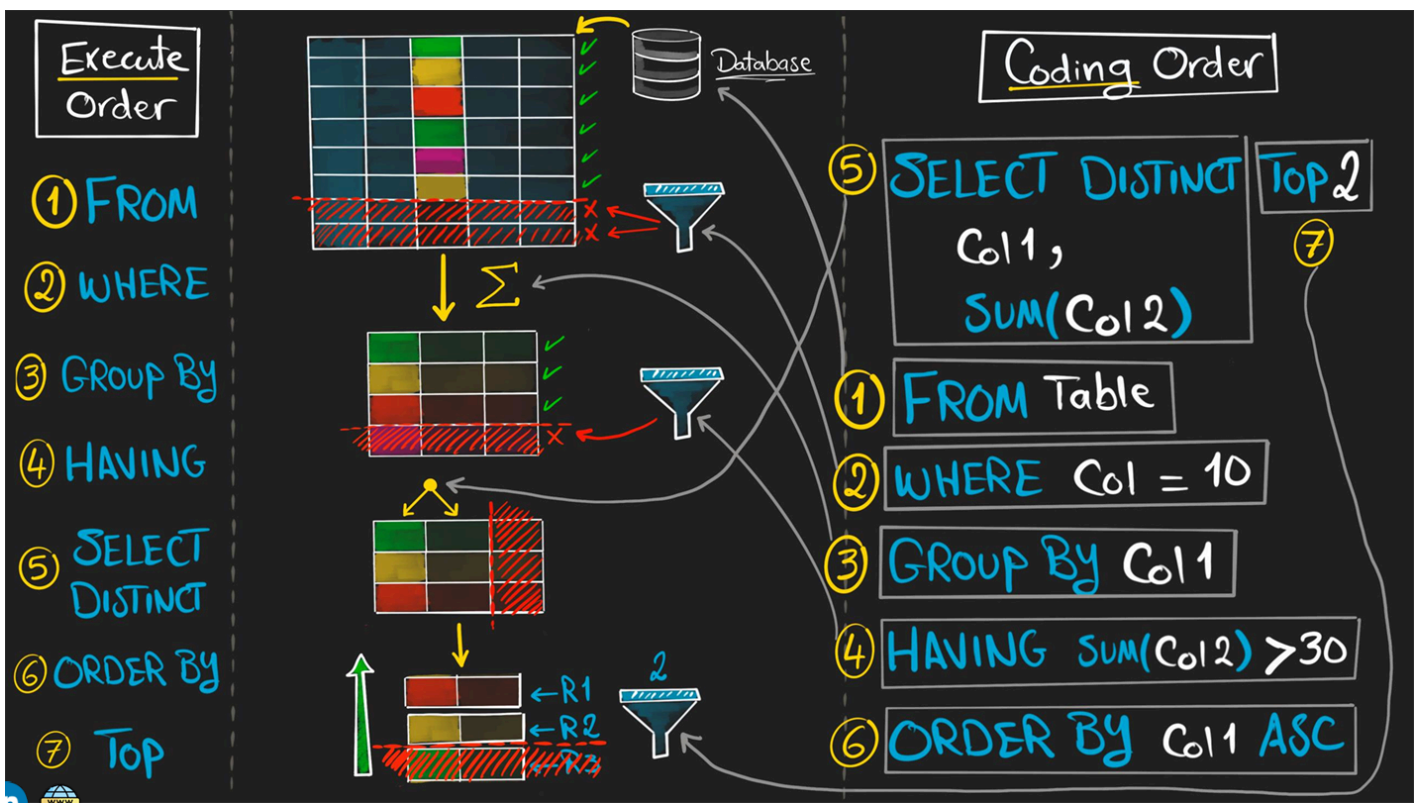
- Coding Order

```
SELECT DISTINCT TOP 2
    Col1,
    SUM(Col2)
FROM table_name
WHERE col = 10
GROUP BY Col1
```

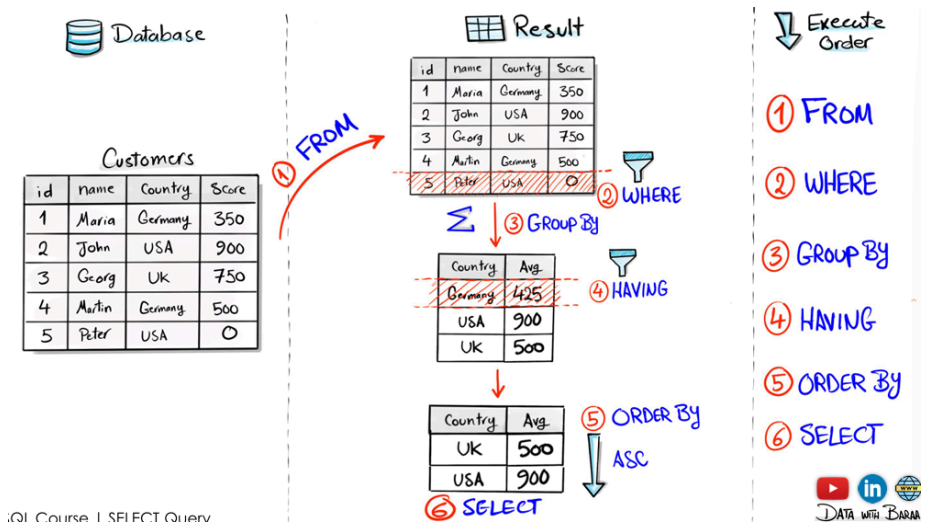
```
HAVING SUM(Col2) > 30
ORDER BY Col1 ASC
```

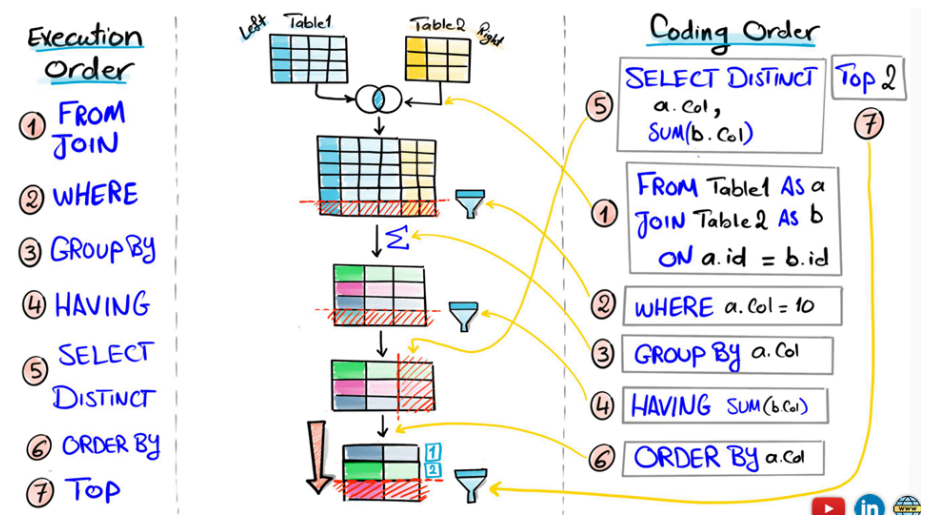
- Execution Order
  1. FROM
  2. WHERE
  3. GROUP BY
  4. HAVING
  5. SELECT DISTINCT
  6. ORDER BY
  7. TOP





MORE EXAMPLES





Module 3: Data Definition Language (DDL)

- Defines the structure of your database

CREATE

- Creates a table in your database
- Syntax

```
CREATE TABLE persons (
    id INT NOT NULL,
    person_name VARCHAR(50) NOT NULL,
    birth_date DATE,
    phone VARCHAR(15) NOT NULL
    CONSTRAINT pk_persons PRIMARY KEY (id)
)
```

    - Column Name Data Type Constraint

ALTER

- Changes the definition of your table
- Syntax

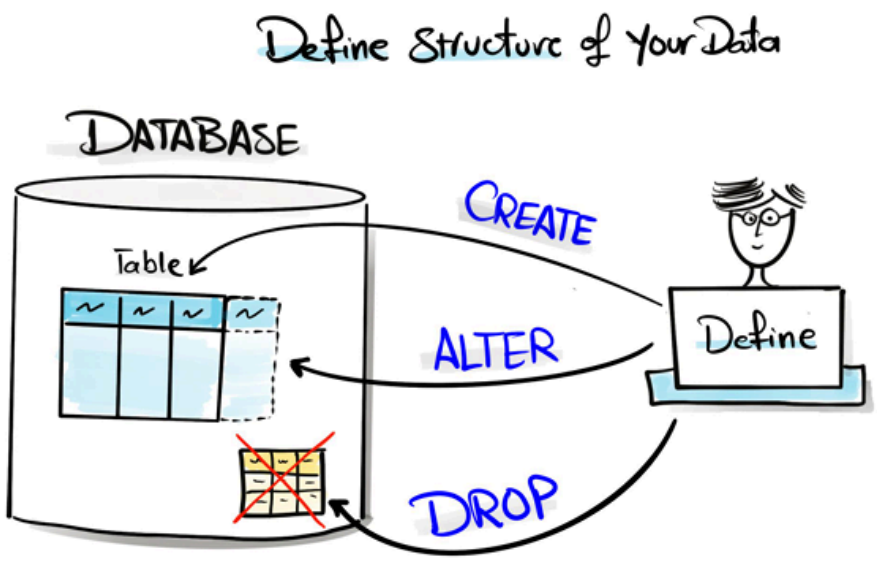
```
-- Adding
ALTER TABLE persons
ADD email VARCHAR(50) NOT NULL

-- Removing
ALTER TABLE persons
DROP COLUMN phone;
```

- Adding columns
    - The new columns are appended at the end of table by default

DROP

- Remove a table from your database
- Syntax

```
DROP TABLE persons;
```



Module 4: Data Manipulation Language (DML)

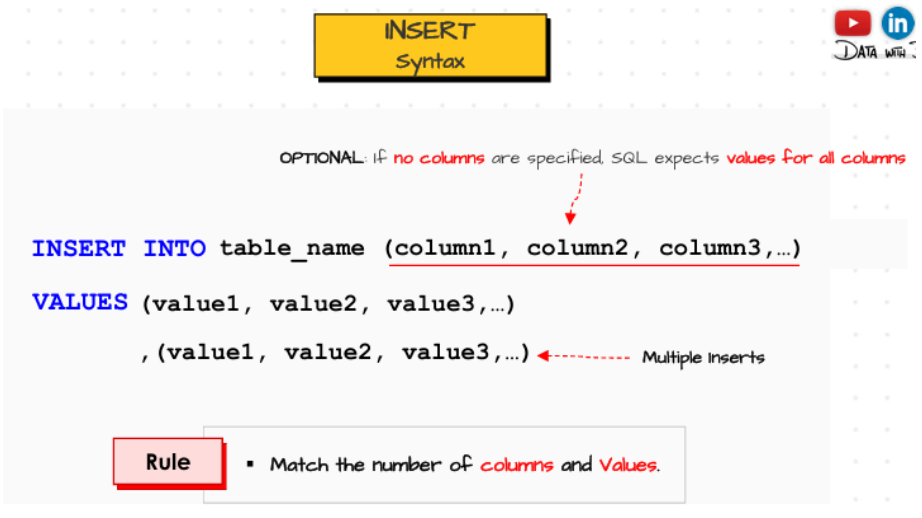
- Modify (Manipulate) your data

INSERT

- Adds a new data (rows) to your table
- Manual Entry (values) → INSERT
  - Syntax

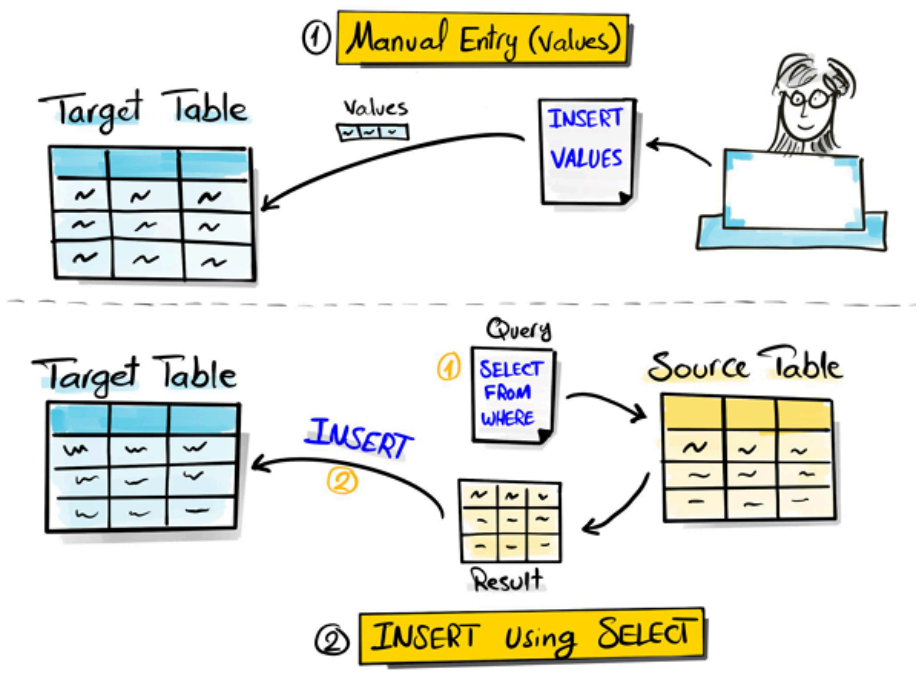
```
INSERT INTO table_name (col1, col2, col3, ..)
VALUES
    (val1, val2, val3, ..),
    (val1, val2, val3, ..)
```

  - Caution → Columns and values must be in the same order
  - Rules → Matching data types, column count & constraints
  - Note → You can skip the columns if you insert values for every column
  - Tip → Always list columns explicitly for clarity and maintainability
  - Note → Columns not included in INSERT become NULL (unless a default or constraint exists)



- INSERT using SELECT
  - Syntax

```sql
INSERT INTO persons (id, person_name, birth_date, phone)
SELECT
    id,
    first_name,
    NULL,
    'Unknown'
FROM customers
```

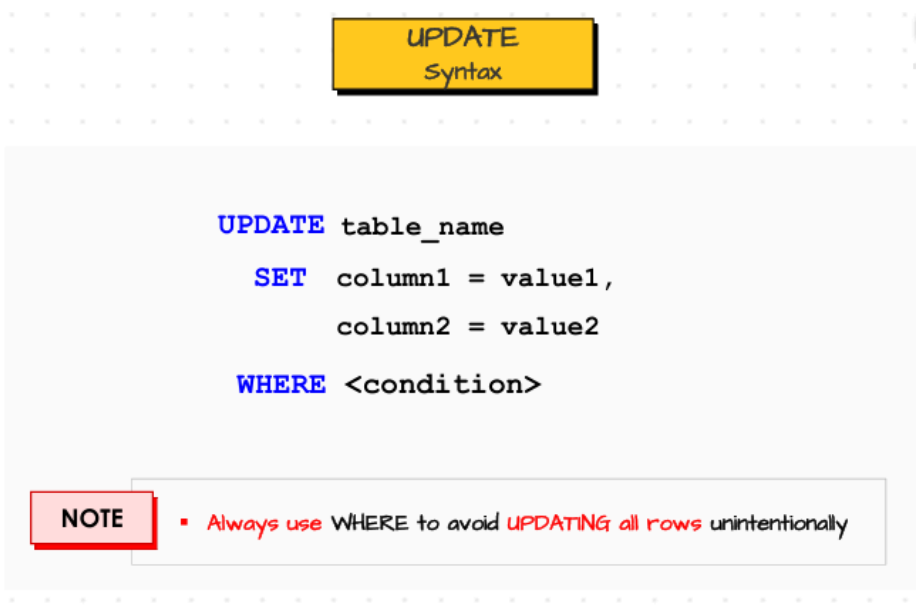


UPDATE

- Changes the data of already existing rows
- Syntax

```sql
UPDATE table_name
SET column1 = value1,
      column2 = value2
WHERE <condition>
```

- Caution → Without a WHERE, all rows will be updated!
- Best practice → Check with SELECT before running UPDATE to avoid updating the wrong data



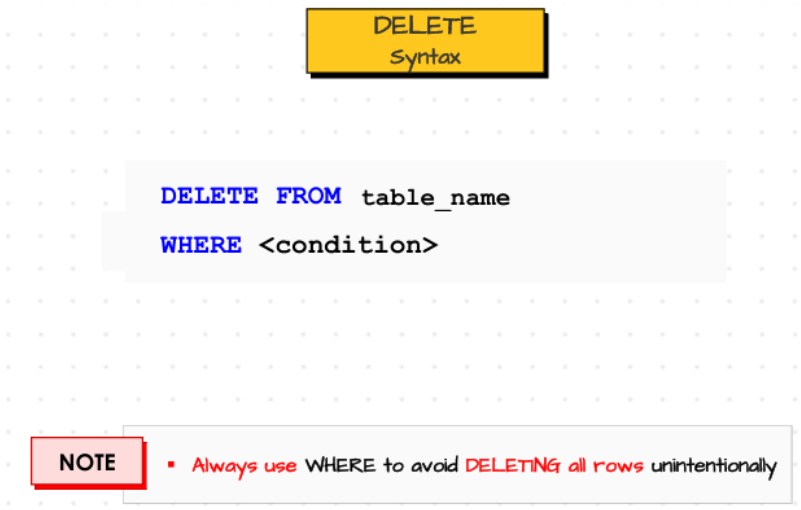
DELETE

- Removes already existing rows from your table
- Syntax

```
DELETE FROM table_name
WHERE <condition>
```

- Caution → Without a WHERE, all rows will be deleted!
- Best practice → Check with SELECT before running DELETE to avoid deleting the wrong data
- TRUNCATE
  - Clears the whole table at once without checking or logging
  - Syntax

```
TRUNCATE TABLE table_name
```



Module 5: Filtering Data

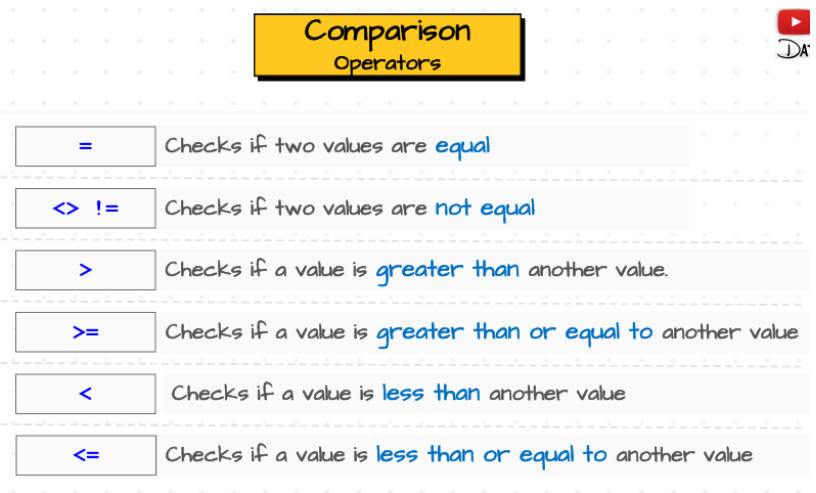
- Data Cleaning

WHERE Operators

- Comparison Operators
  - =
  - <> or =!
  - >
  - > =
  - <
  - < =
- Logical Operators
  - AND
  - OR
  - NOT
- Range Operator
  - BETWEEN
- Membership Operator
  - IN
  - NOT IN
- Search Operator
  - LIKE

Comparison Operators

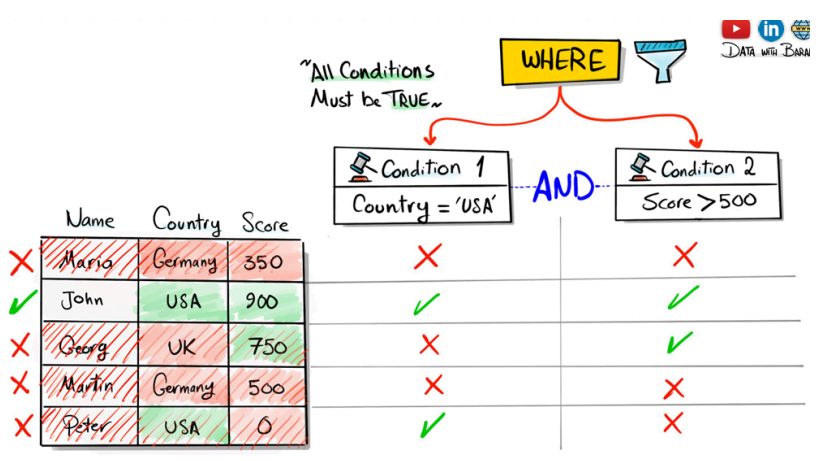
- Compare two things
- condition → expression operator expression
  - column1 = column2
    - first_name = last_name
  - column1 = value
    - first_name = 'Youssef'
  - function = value
    - UPPER(first_name) = 'YOUSSEF'
  - expression = value
    - price * quantity = 1000
  - subquery = value
    - (SELECT AVG(sales) FROM orders) = 1000



Logical Operators

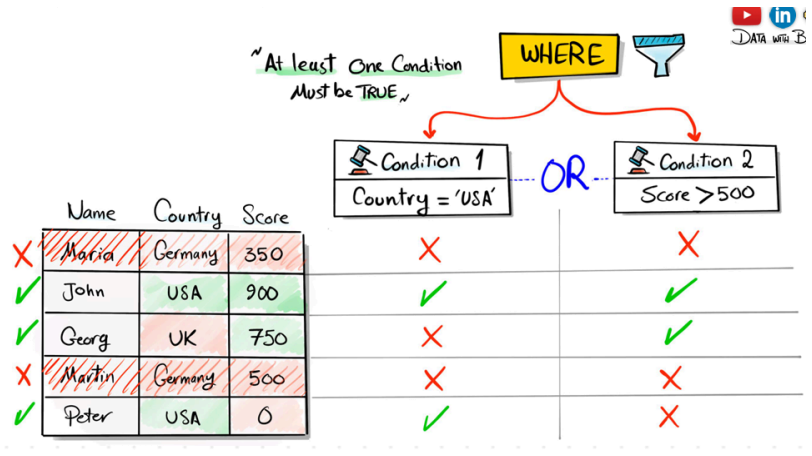
- AND
  - All conditions must be TRUE

```sql
WHERE country = 'USA' AND score > 500
```



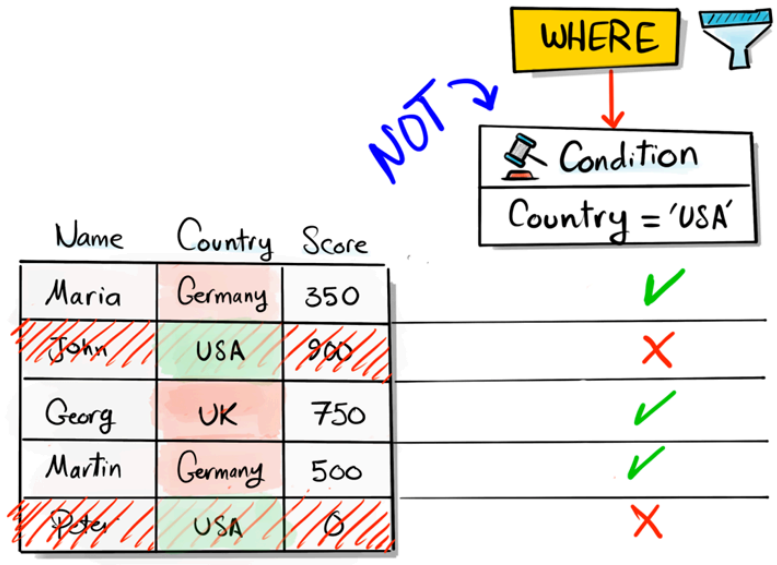
- OR
  - At least one condition must be TRUE

```sql
WHERE country = 'USA' OR score > 500
```



- NOT
  - Excludes matching values

```
WHERE NOT score < 500
```



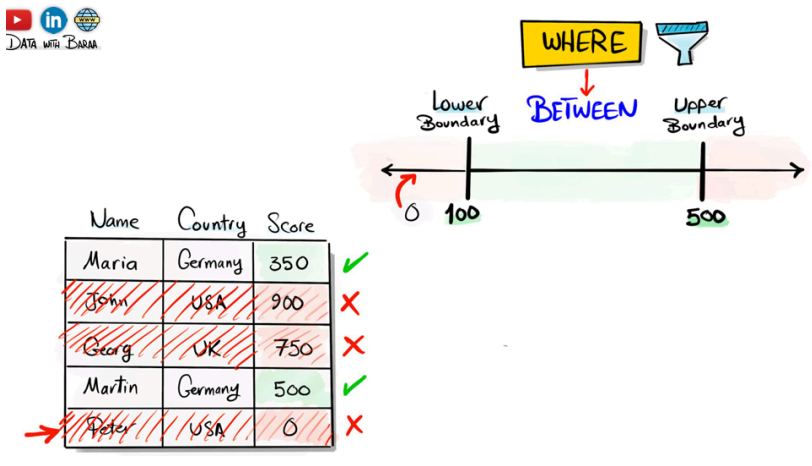
Range Operator

- BETWEEN
  - Check if a value is within a range
  - Boundaries are inclusive

```
WHERE score BETWEEN 100 AND 500
```

  - Tip → Explicit comparisons clearly show that both boundaries are included

```
WHERE score >= 100 AND score <= 500
```



Membership Operator

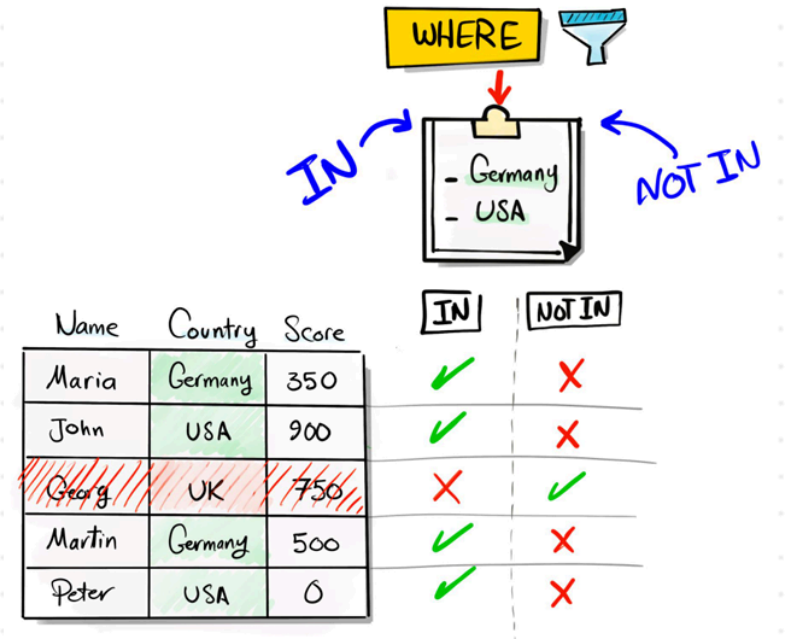
- IN
  - Check if a value exists in a list

- Tip → Use IN instead of OR for multiple values in the same column to simplify SQL

```sql
WHERE country IN ('Germany', 'USA')
```

- NOT IN
  - Check if a value does not exist in a list

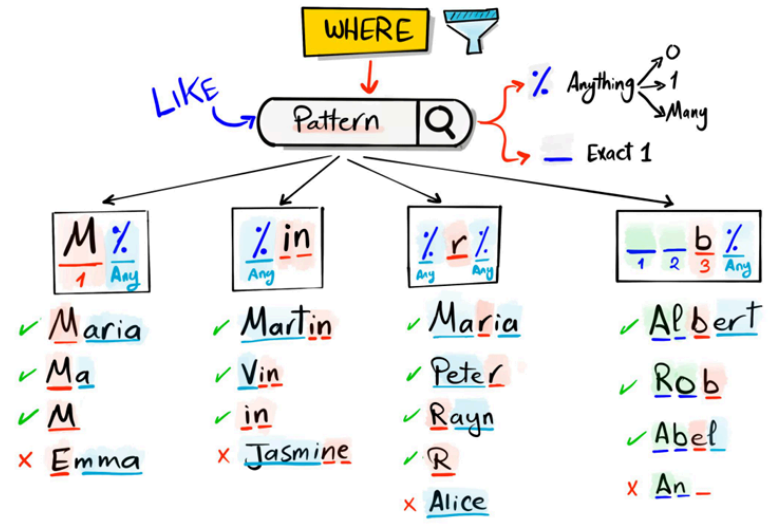
```sql
WHERE country NOT IN ('Germany', 'USA')
```



Search Operator

- LIKE
  - Search for a pattern in text

```sql
WHERE first_name LIKE 'M%';
-- OR
WHERE first_name LIKE '%n';
-- OR
WHERE first_name LIKE '%r%';
-- OR
WHERE first_name LIKE '__r%';
```

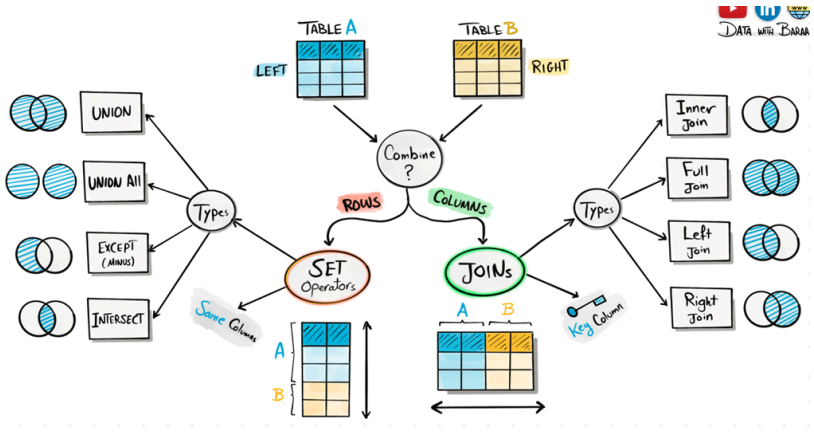


`AND`	All conditions must be TRUE
`OR`	At least one condition must be TRUE
`NOT`	(Reverse) Excludes matching values
`BETWEEN`	Check if a value is within a range
`IN`	Check if a value exists in a list
`LIKE`	Search for a pattern in text

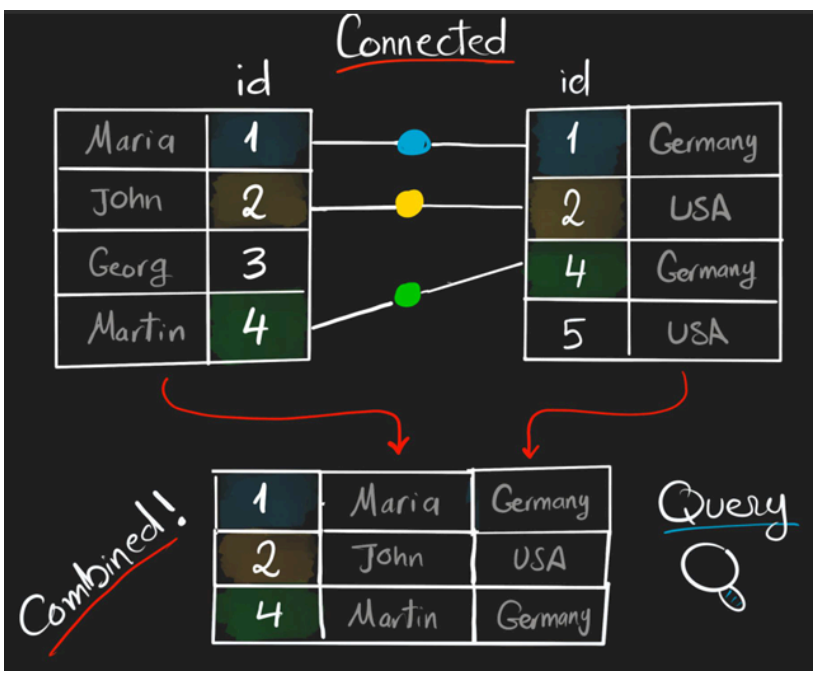
Module 6: SQL Joins

Combining Data

- Columns
- JOINs
  - Types
    - Inner Join
    - Full Join
    - Left Join
    - Right Join
  - Key Column



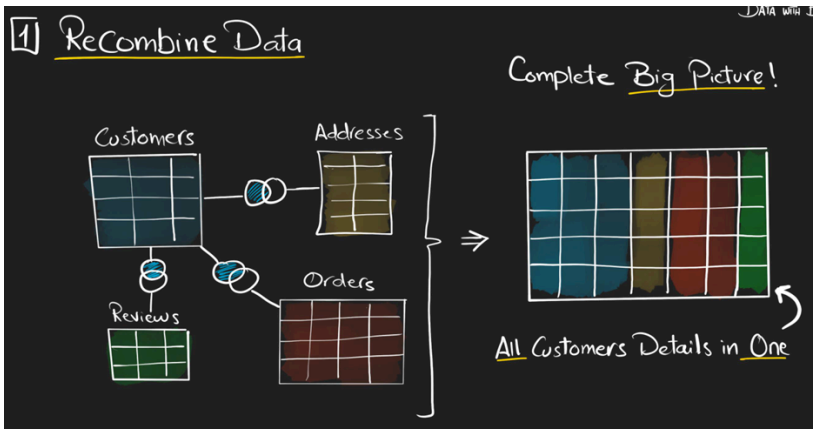
What is SQL JOINs

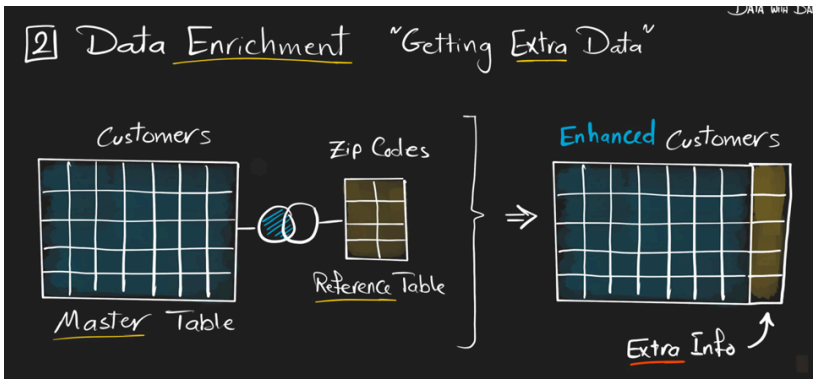


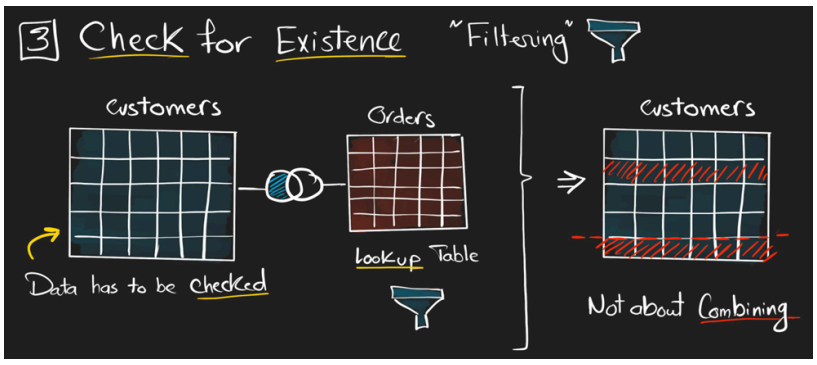
When to use SQL JOINs

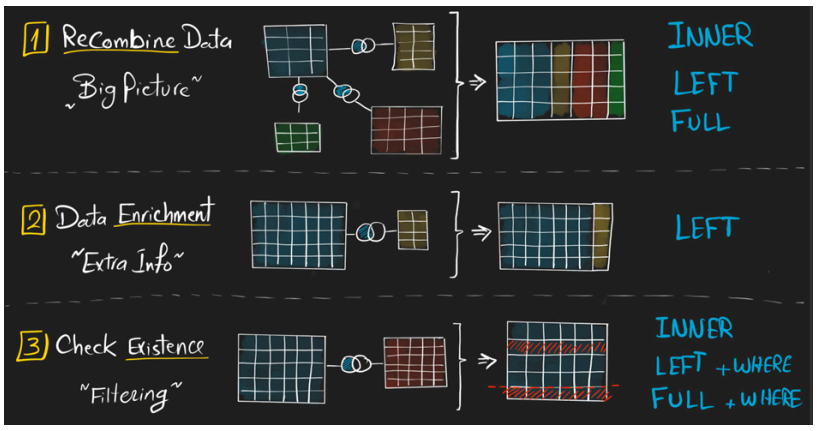
1. Recombine Data (Big Picture)

2. Data Enrichment (Getting Extra Data)
3. Check for Existence (Filtering)



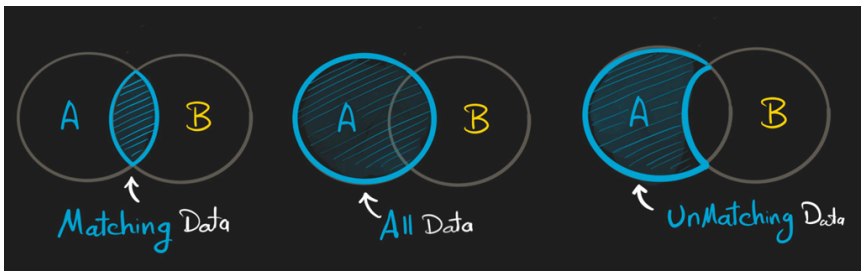




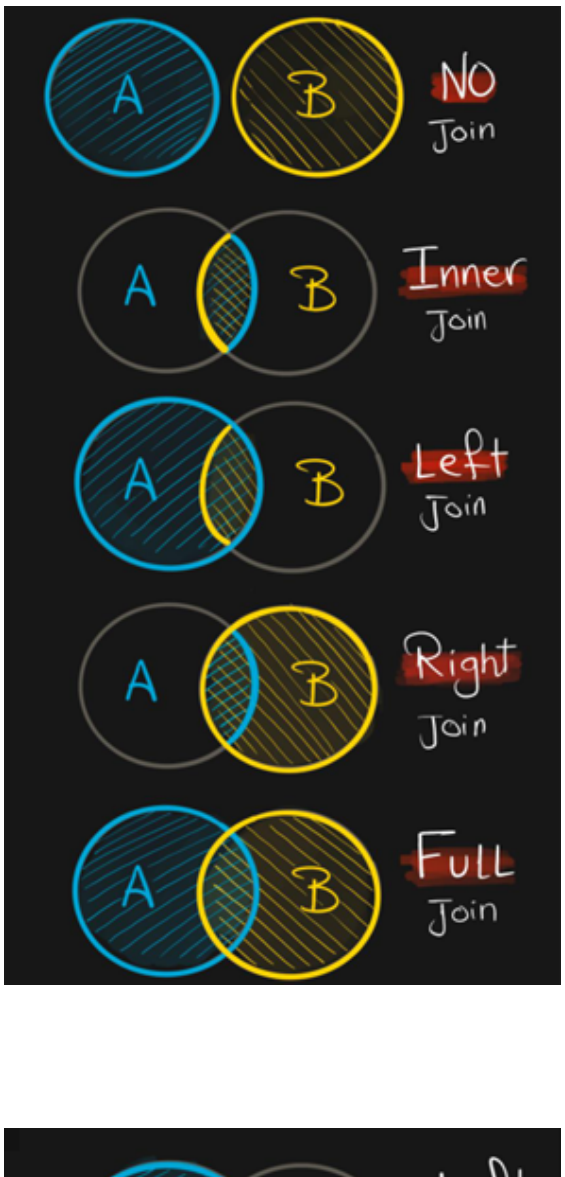


JOIN Types

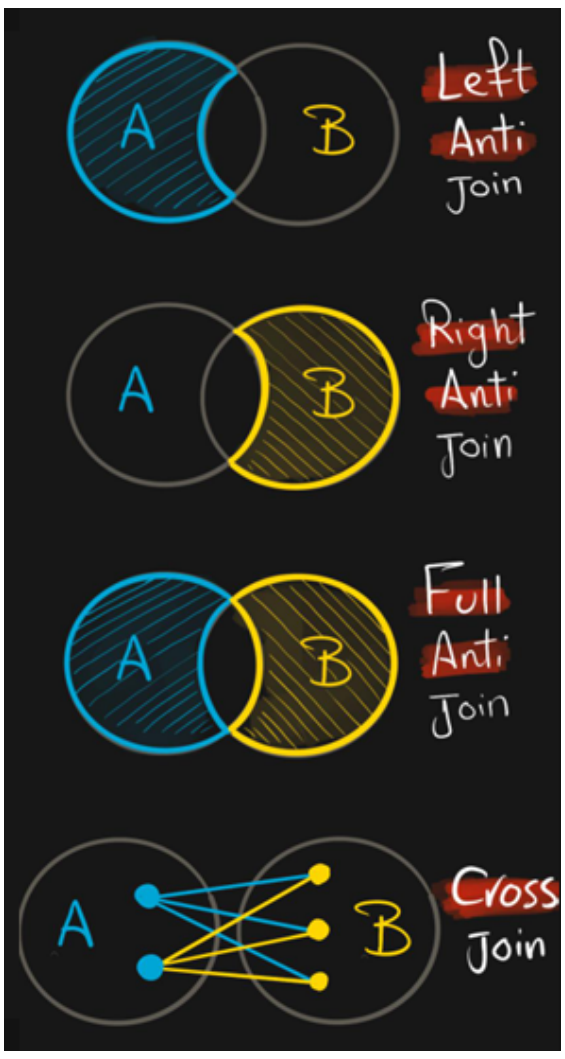
JOINs Possibilities



Basic JOIN Types



Advanced JOIN Types



JOIN Code

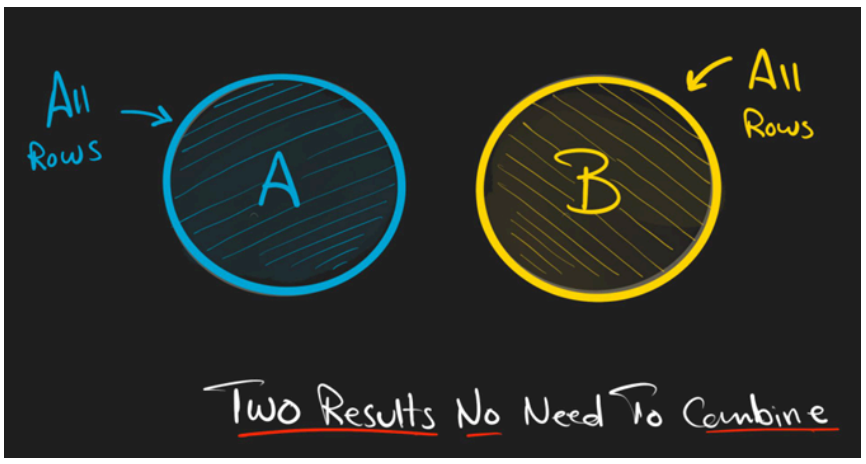
```
SELECT *
FROM A
<type> JOIN B
ON A.key = B.key; // ON <condition>
```

- default type → INNER
- Column Ambiguity → Add the table name before the column to avoid confusion in joins with same-named columns

NO JOIN

- Returns data from two tables without combining them

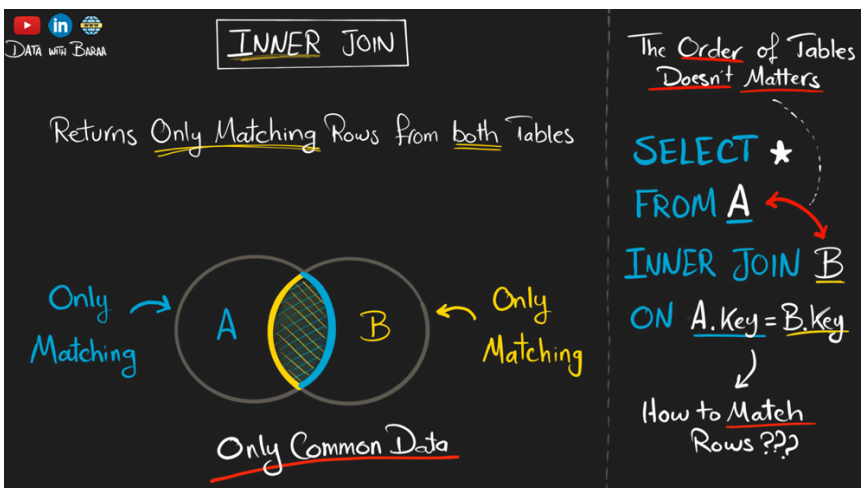
```sql
SELECT *
FROM customers;

SELECT *
FROM orders;
```



INNER JOIN

- Returns only matching rows from both tables

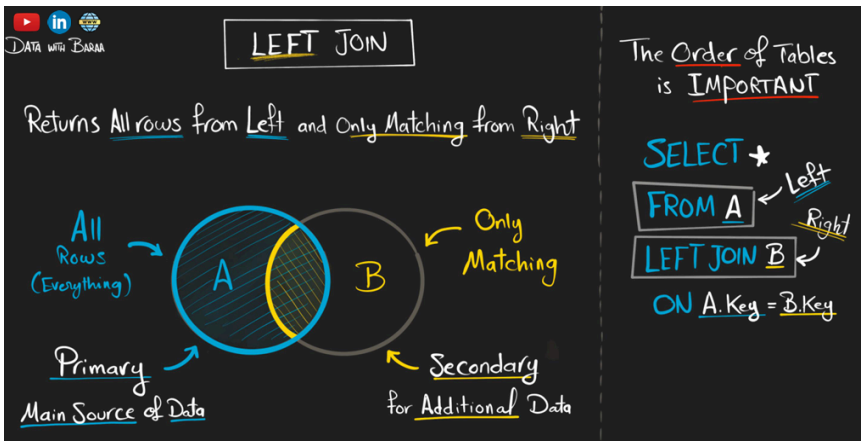
```sql
SELECT
    c.id,
    c.first_name,
    o.order_id,
    o.sales
FROM customers AS c
INNER JOIN orders AS o
ON c.id = o.customer_id;
```



LEFT JOIN

- Returns all rows from the left table and only matching from the right table

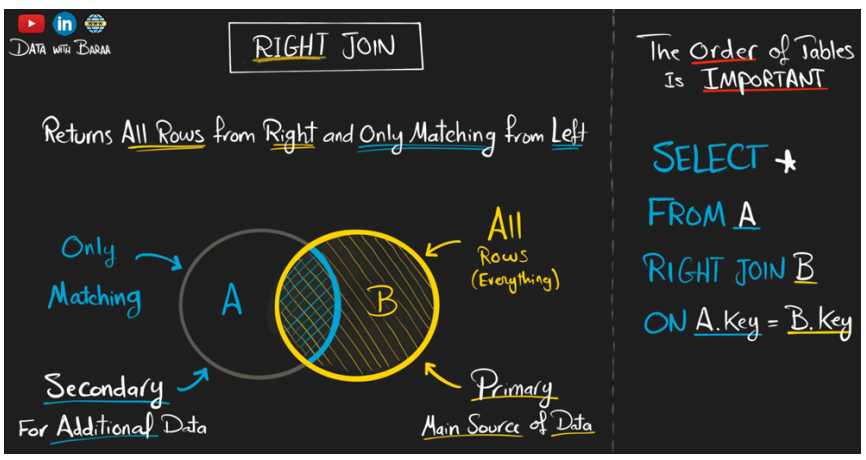
```sql
SELECT
    c.id,
    c.first_name,
    o.order_id,
    o.sales
FROM customers AS c
LEFT JOIN orders AS o
ON c.id = o.customer_id;
```



RIGHT JOIN

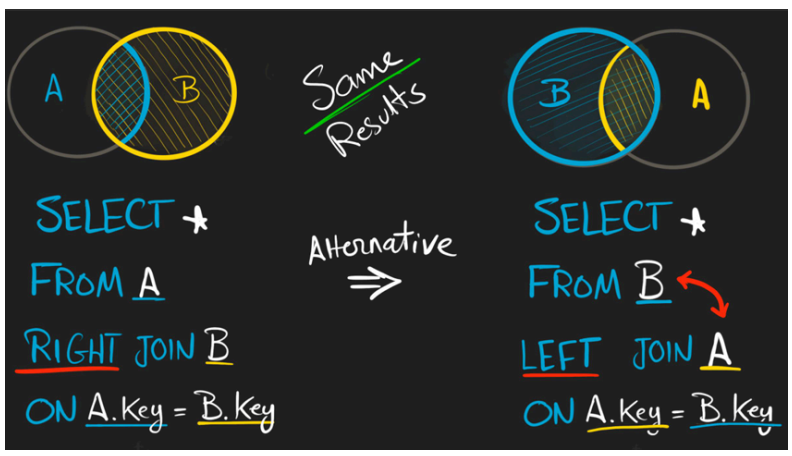
- Returns all rows from the right table and only matching from the left table

```sql
SELECT
    o.order_id,
    o.sales,
    c.id,
    c.first_name
FROM customers AS c
RIGHT JOIN orders AS o
ON c.id = o.customer_id;
```



- Alternative to RIGHT JOIN

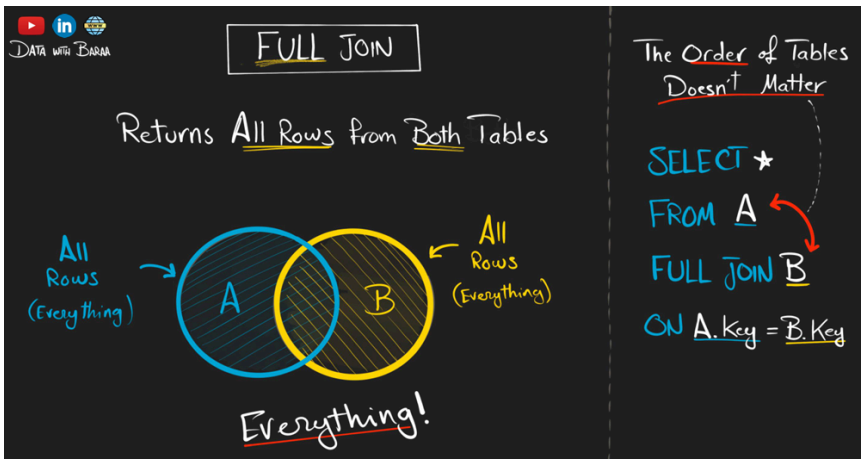
```sql
SELECT
    c.id,
    c.first_name,
    o.order_id,
    o.sales
FROM orders AS o
LEFT JOIN customers AS c
ON c.id = o.customer_id;
```



FULL JOIN

- Returns all rows from both tables

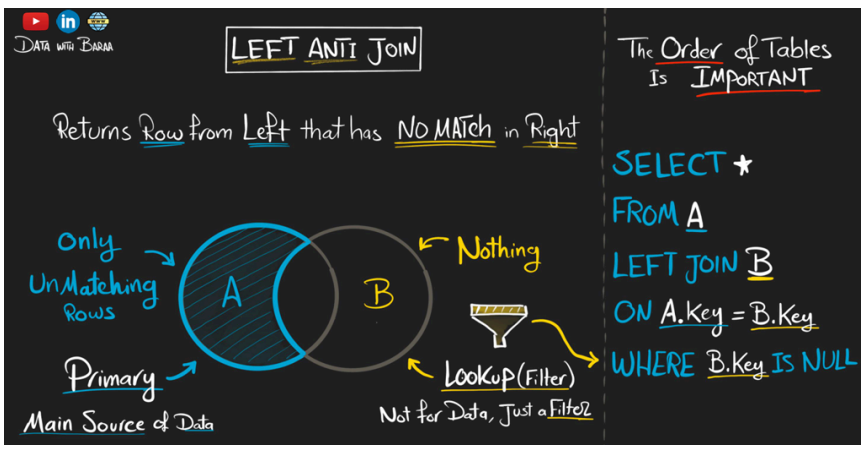
```sql
SELECT
    c.id,
    c.first_name,
    o.order_id,
    o.sales
FROM customers AS c
FULL JOIN orders AS o
ON c.id = o.customer_id;
```



LEFT ANTI JOIN

- Returns rows from the left table that has no match in the right table

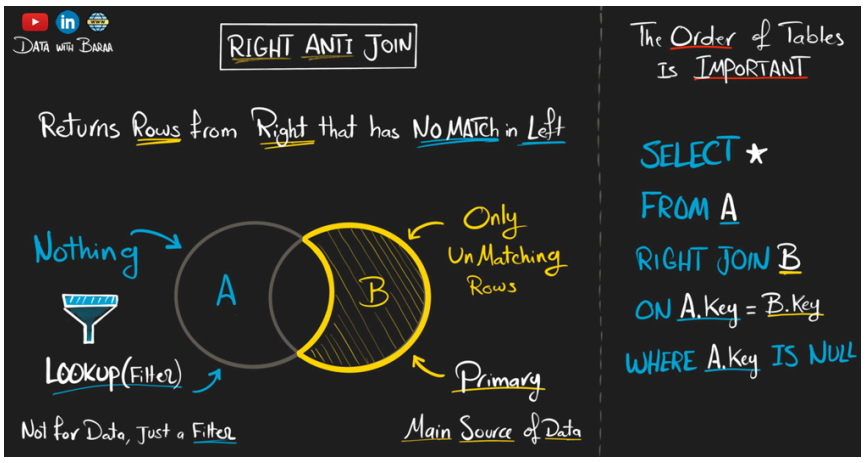
```sql
SELECT
    c.id,
    c.first_name
FROM customers AS c
LEFT JOIN orders AS o
ON c.id = o.customer_id
WHERE o.customer_id IS NULL;
```



RIGHT ANTI JOIN

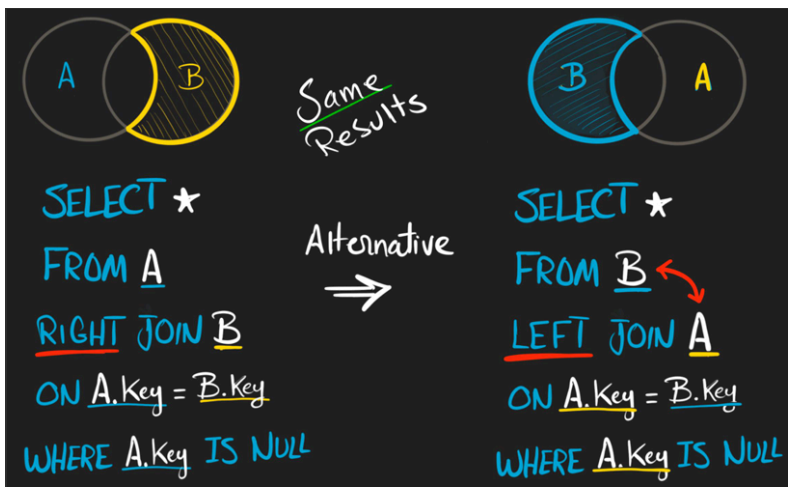
- Returns rows from the right table that has no match in the left table

```sql
SELECT
    o.order_id,
    o.sales
FROM customers AS c
RIGHT JOIN orders AS o
ON c.id = o.customer_id
WHERE c.id IS NULL;
```



- Alternative to RIGHT ANTI JOIN

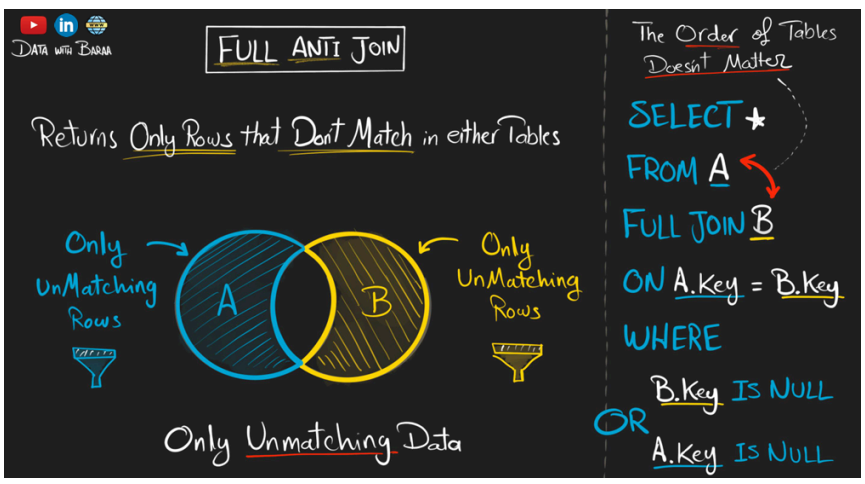
```sql
SELECT
    o.order_id,
    o.sales
FROM orders AS o
LEFT JOIN customers AS c
ON c.id = o.customer_id
WHERE c.id IS NULL;
```



FULL ANTI JOIN

- Returns only rows that don't match in either tables

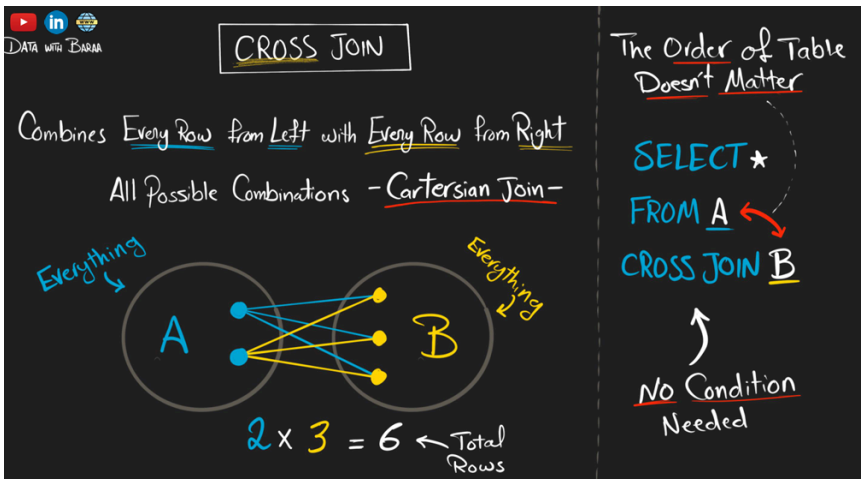
```sql
SELECT *
FROM orders AS o
FULL JOIN customers AS c
ON c.id = o.customer_id
WHERE o.customer_id IS NULL OR c.id IS NULL;
```



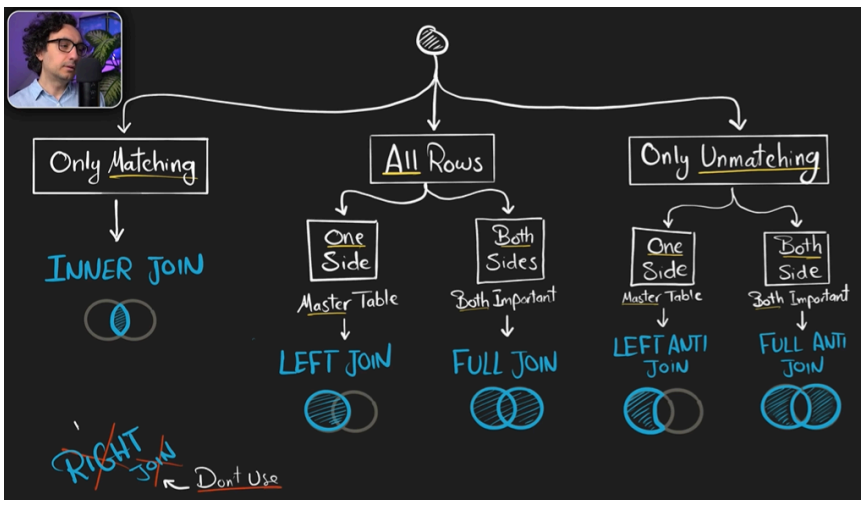
CROSS JOIN

- Combines every row from the left table with every row from the right table
- All Possible Combinations <Cartesian JOIN>

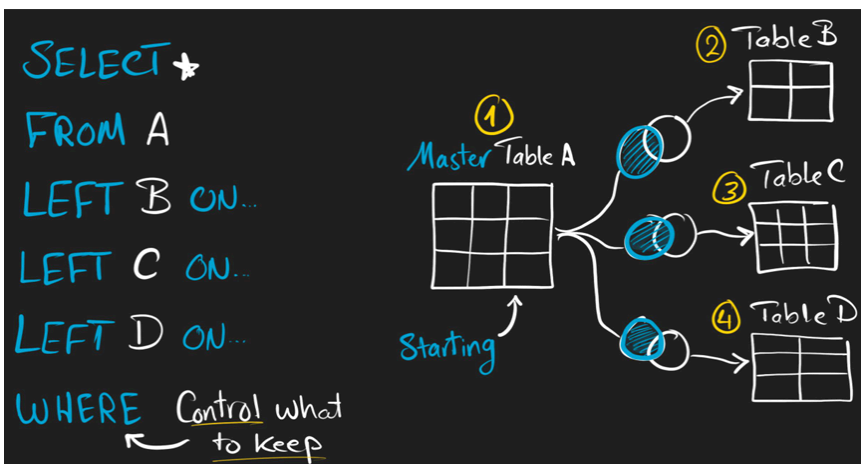
```sql
SELECT *
FROM customers
CROSS JOIN orders;
```

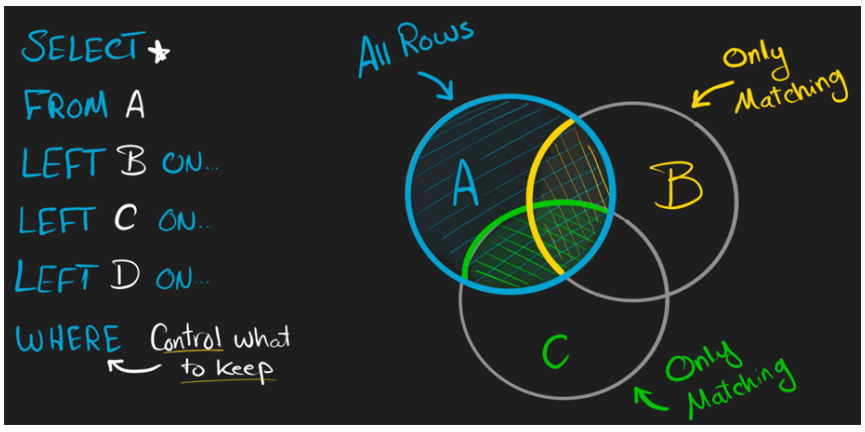


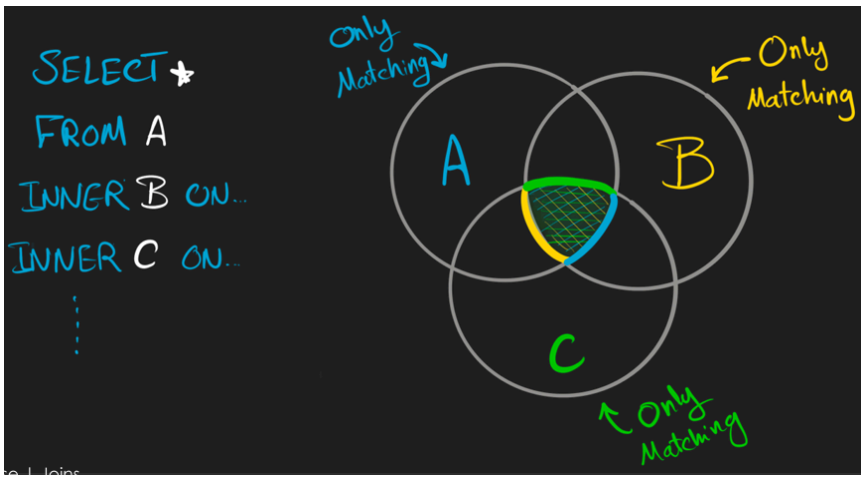
How to choose between JOIN Types?



Multi-Table JOIN

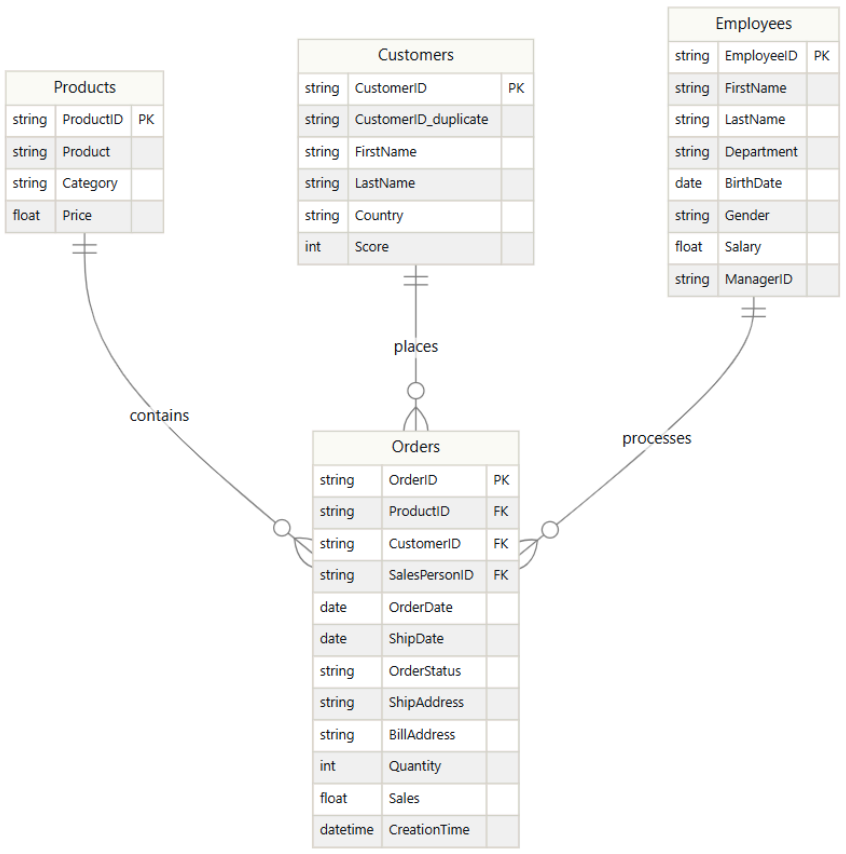






Multi-Table JOIN Example

- Using SalesDB, Retrieve a list of all orders, along with the related customer, product, and employee details.
- For each order, display:
  - Order ID
  - Customer's name
  - Product name
  - Sales amount
  - Product price
  - Salesperson's name




```sql
SELECT
    o.OrderID,
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    p.Product AS ProductName,
    o.Sales,
    p.Price,
    CONCAT(e.FirstName, ' ', e.LastName) AS SalesPersonName
FROM Sales.Orders AS o
LEFT JOIN Sales.Customers AS c
ON o.CustomerID = c.CustomerID
LEFT JOIN Sales.Products AS p
ON o.ProductID = p.ProductID
```

```sql
LEFT JOIN Sales.Employees AS e
ON o.SalesPersonID = e.EmployeeID;
```

	OrderID	CustomerName	ProductName	Sales	Price	SalesPersonName
1	1	Kevin Brown	Bottle	10	10	Mary
2	2	Mary	Tire	15	15	Mary
3	3	Jossef Goldberg	Bottle	20	10	Carol Baker
4	4	Jossef Goldberg	Gloves	60	30	Mary
5	5	Kevin Brown	Caps	25	25	Carol Baker
6	6	Mary	Caps	50	25	Carol Baker
7	7	Jossef Goldberg	Tire	30	15	Frank Lee
8	8	Mark Schwarz	Bottle	90	10	Mary
9	9	Kevin Brown	Bottle	20	10	Mary
10	10	Mary	Tire	60	15	Carol Baker

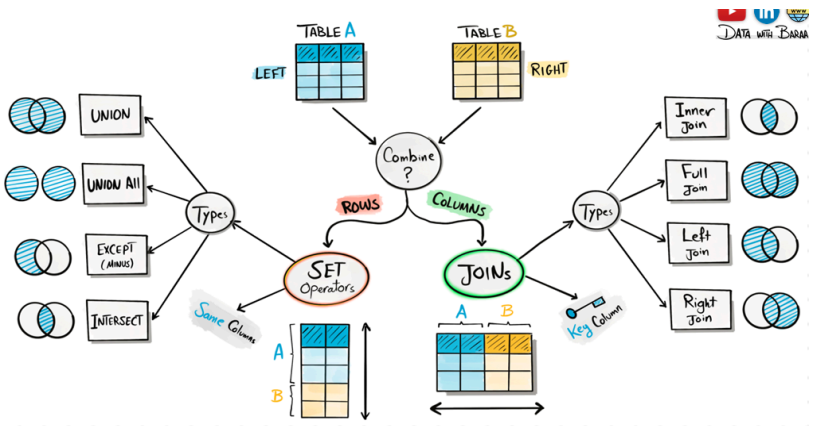
Module 7: SQL SET Operators

What is SET Operators

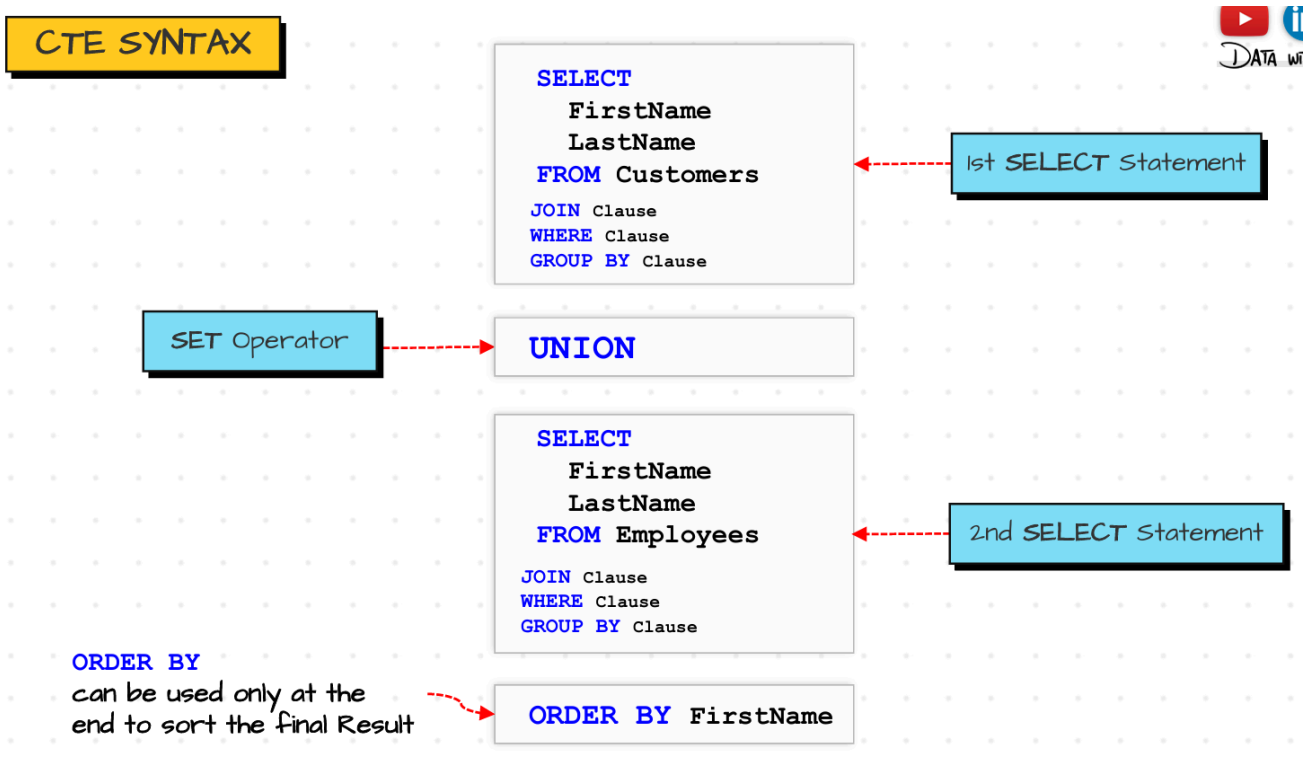
- Combine the results of multiple queries into a single result set

Combining Data

- Rows
  - SET Operators
    - Types
      - UNION
      - UNION ALL
      - EXCEPT (MINUS)
      - INTERSECT
    - Same Columns



Syntax



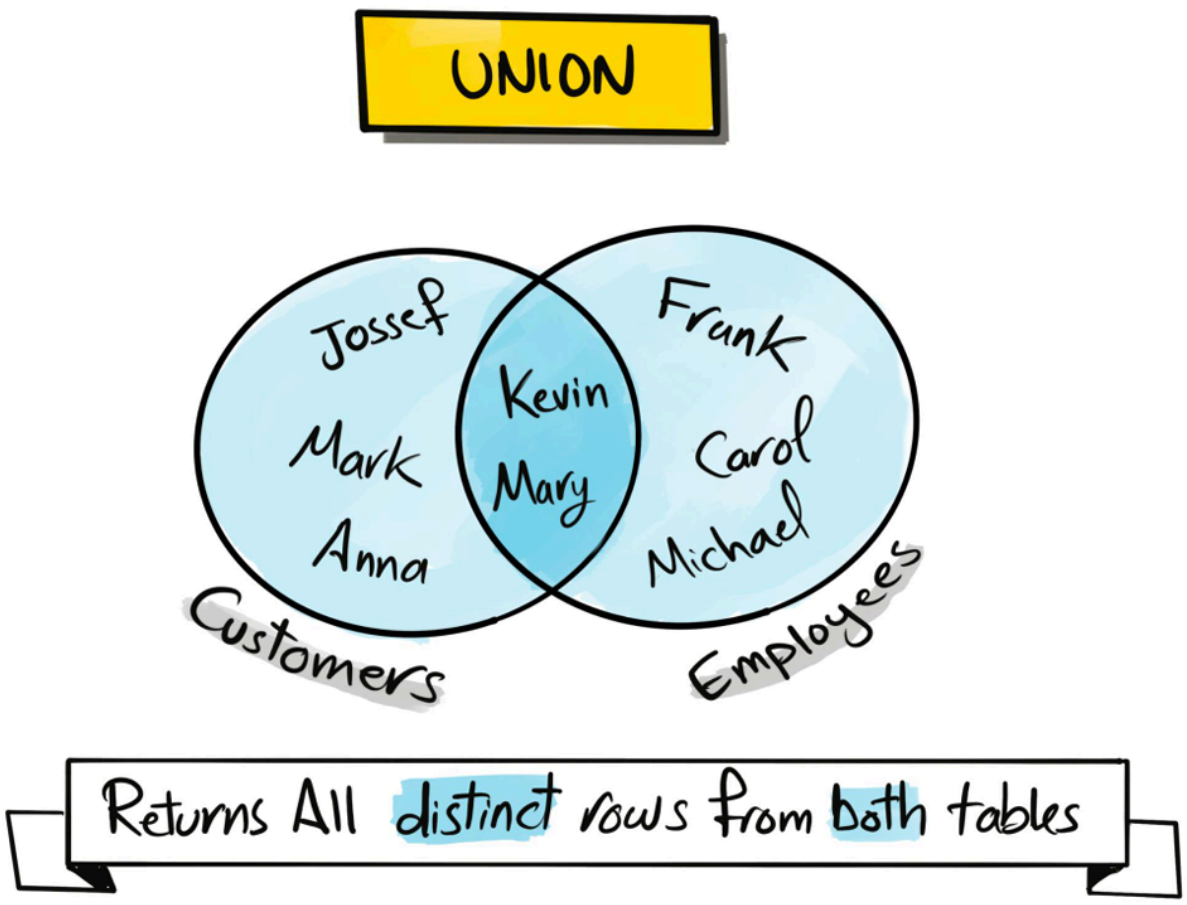
Rules

1. SQL CLAUSES
    a. SET Operator can be used almost in all clauses (WHERE, JOIN, GROUP BY, HAVING)
    b. ORDER BY is allowed only once at the end of query
2. NUMBER OF COLUMNS
    a. The number of columns in each query must be the same
3. DATA TYPES
    a. Data types of columns in each query must be compatible
4. ORDER OF COLUMNS
    a. The order of the columns in each query must be the same
5. COLUMN ALIASES
    a. The column names in the result set are determined by the column names specified in the first query
6. CORRECT COLUMNS
    a. Even if all rules are met and SQL shows no errors, the result may be incorrect
    b. Incorrect column selection leads to inaccurate results

UNION

- Returns all distinct unique rows from both queries
- Remove duplicate rows from the result
- The order of queries in a UNION operation does not affect the result

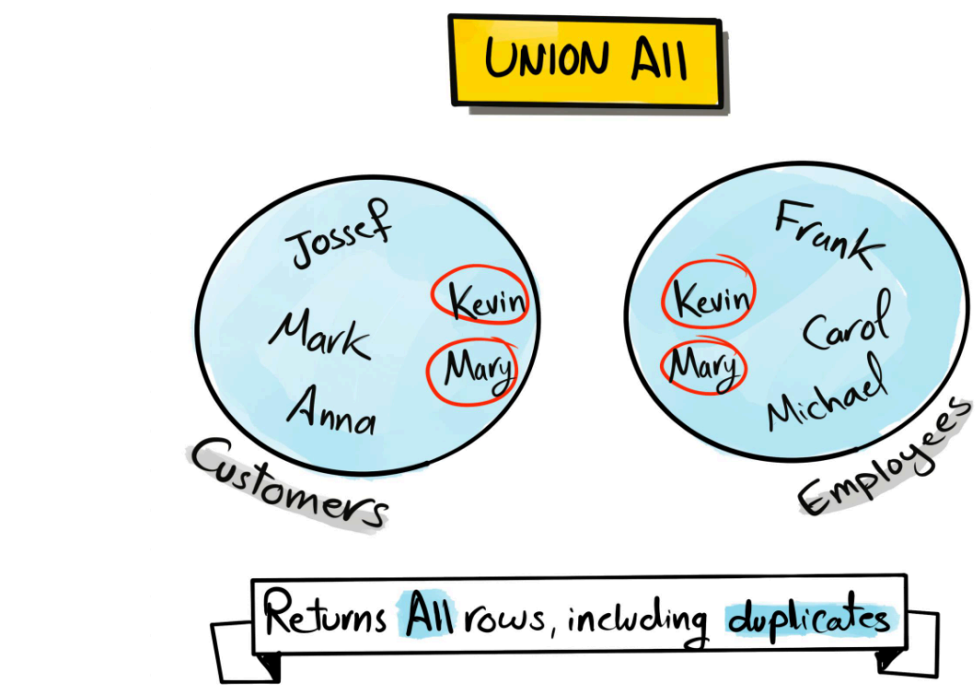
```sql
SELECT
    FirstName,
    LastName
FROM Sales.Customers
UNION
SELECT
    FirstName,
    LastName
FROM Sales.Employees;
```



UNION ALL

- Returns all rows from both queries, including duplicates
- The order of queries in a UNION operation does not affect the result

```sql
SELECT
    FirstName,
    LastName
FROM Sales.Customers
UNION ALL
SELECT
    FirstName,
    LastName
FROM Sales.Employees;
```



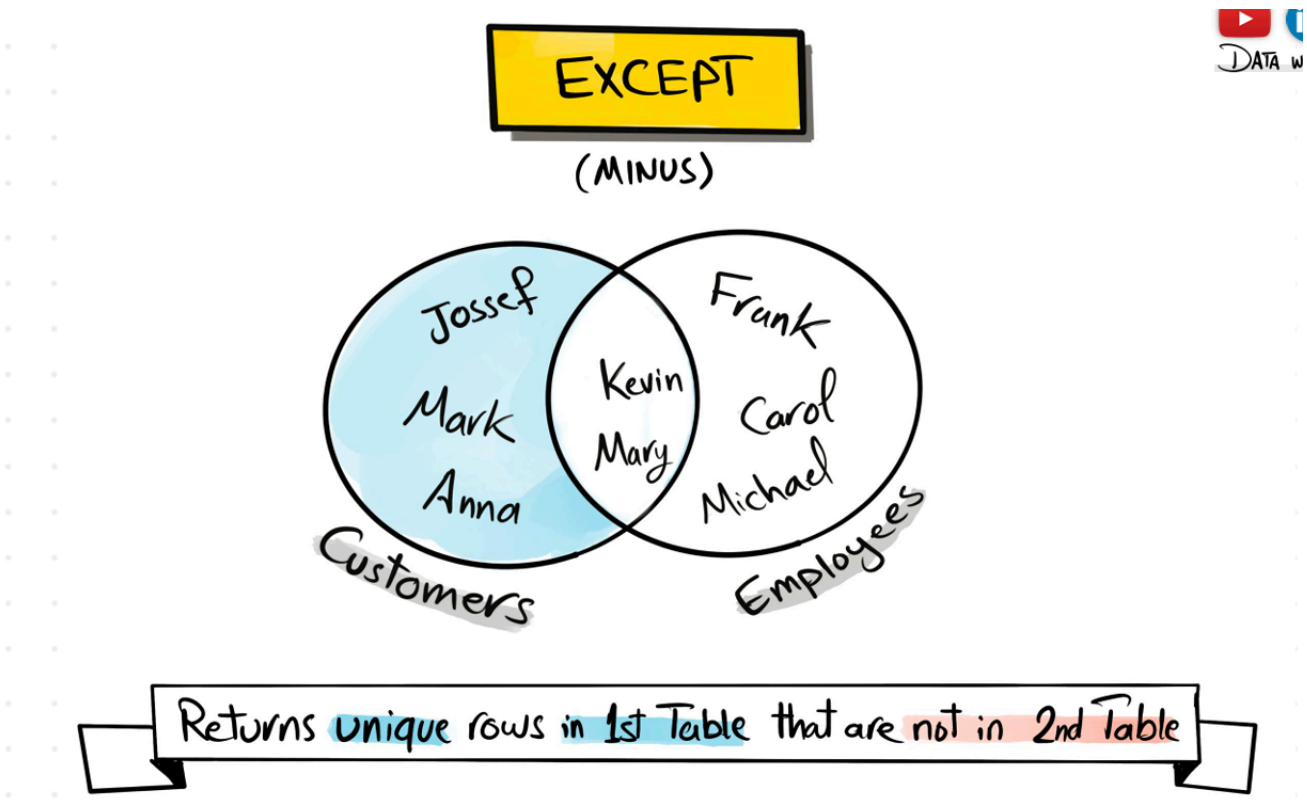
UNION ALL vs UNION

- UNION ALL is generally faster than UNION
- If you’re confident there are no duplicates, use UNION ALL
- Use UNION ALL to find duplicates and quality issues

EXCEPT (MINUS)

- Returns all distinct rows from the first query that are not found in the second query
- The order of queries in a EXCEPT does affect the result

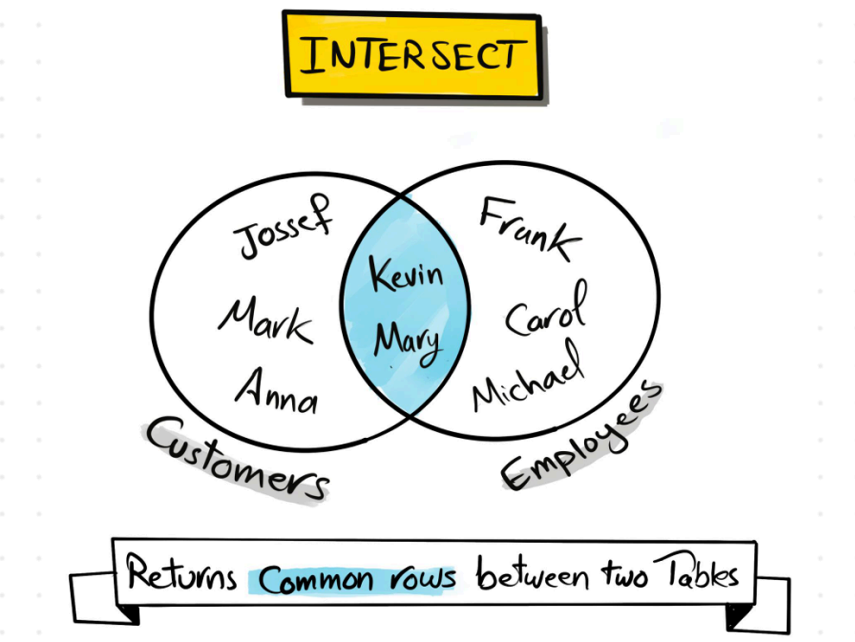
```sql
SELECT
    FirstName,
    LastName
FROM Sales.Customers
EXCEPT
SELECT
    FirstName,
    LastName
FROM Sales.Employees;
```



INTERSECT

- Returns only the rows that are common in both queries
- The order of queries in a INTERSECT does not affect the result

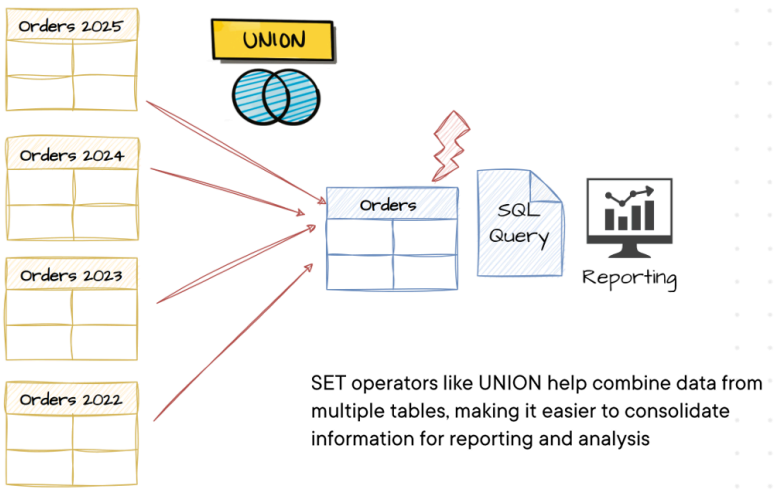
```sql
SELECT
    FirstName,
    LastName
FROM Sales.Customers
INTERSECT
SELECT
    FirstName,
    LastName
FROM Sales.Employees;
```



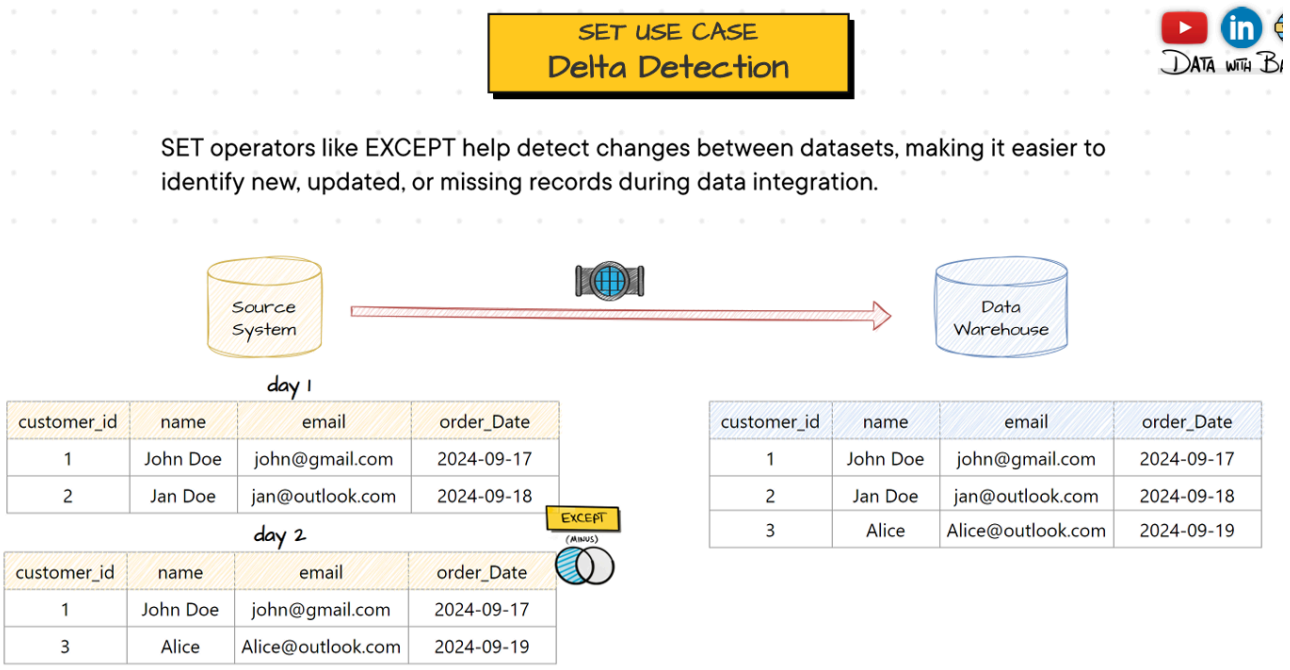
Use Cases

- Combining Information
  - Combine similar information before analyzing the data

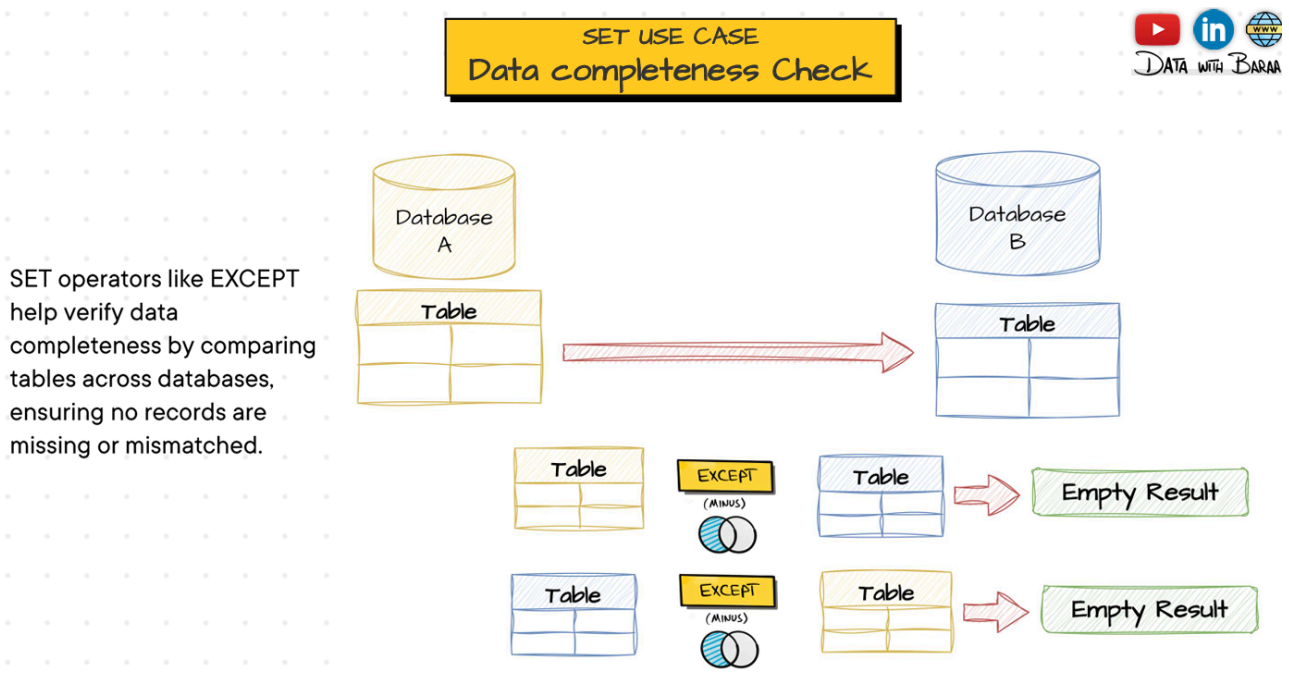
```sql
SELECT
    'Orders' AS SourceTable
    ,[OrderID]
    ,[ProductID]
    ,[CustomerID]
    ,[SalesPersonID]
    ,[OrderDate]
    ,[ShipDate]
    ,[OrderStatus]
    ,[ShipAddress]
    ,[BillAddress]
    ,[Quantity]
    ,[Sales]
    ,[CreationTime]
FROM Sales.Orders
UNION
SELECT
    'OrdersArchive' AS SourceTable
    ,[OrderID]
    ,[ProductID]
    ,[CustomerID]
    ,[SalesPersonID]
    ,[OrderDate]
    ,[ShipDate]
    ,[OrderStatus]
    ,[ShipAddress]
    ,[BillAddress]
    ,[Quantity]
    ,[Sales]
    ,[CreationTime]
FROM Sales.OrdersArchive
ORDER BY OrderID ASC;
```



- Delta Detection
  - Identifying the differences or changes (delta) between two batches of data



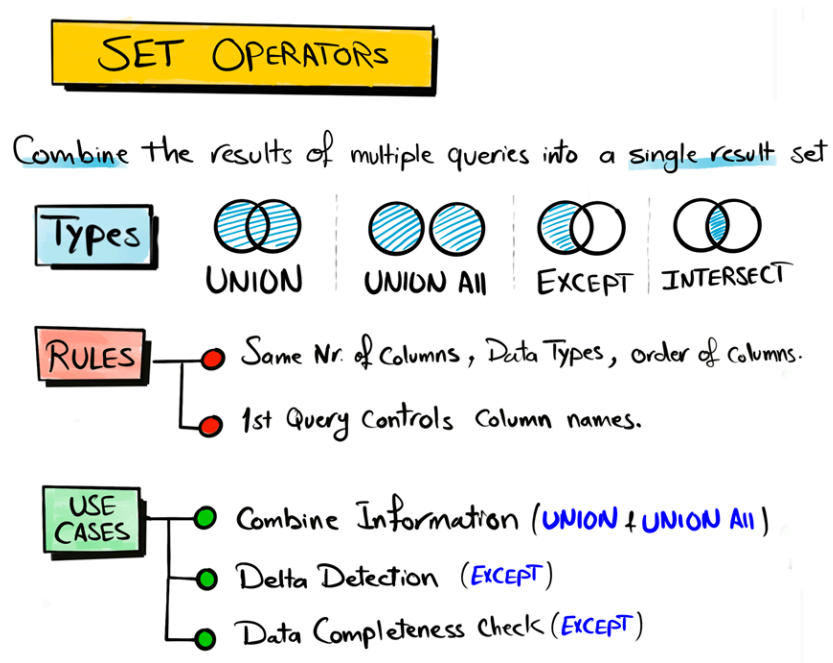
- Data Completeness Check
  - EXCEPT operator can be used to compare tables to detect discrepancies between databases



Notes

- BEST PRACTICES → Never use an asterisk (*) to combine tables; list needed columns instead
- SOURCE FLAG → Include additional column to indicate the source of each row

Summary



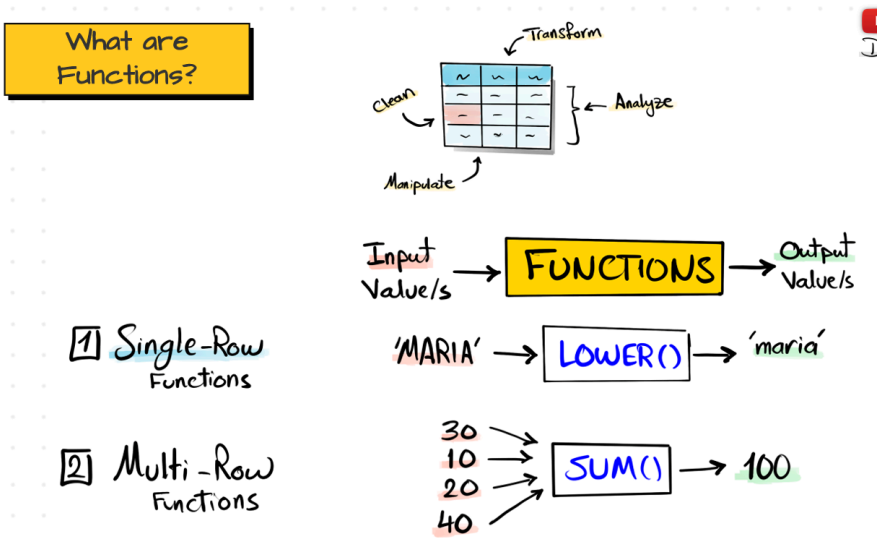
Module 8: String Functions

SQL Functions

Functions

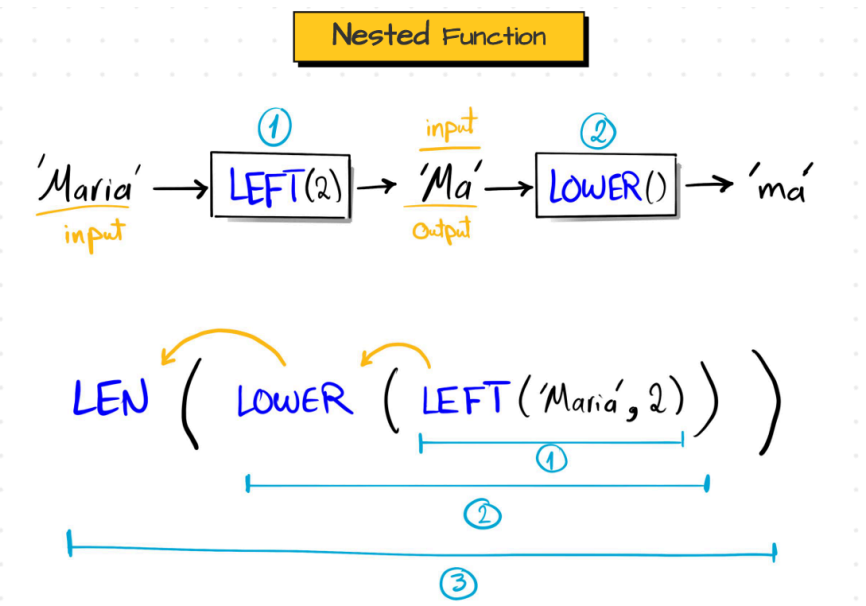
- A built-in SQL code:
  - accepts an input value
  - processes it
  - returns an output value

1. Single-Row Functions → LOWER()
2. Multi-Row Functions → SUM()



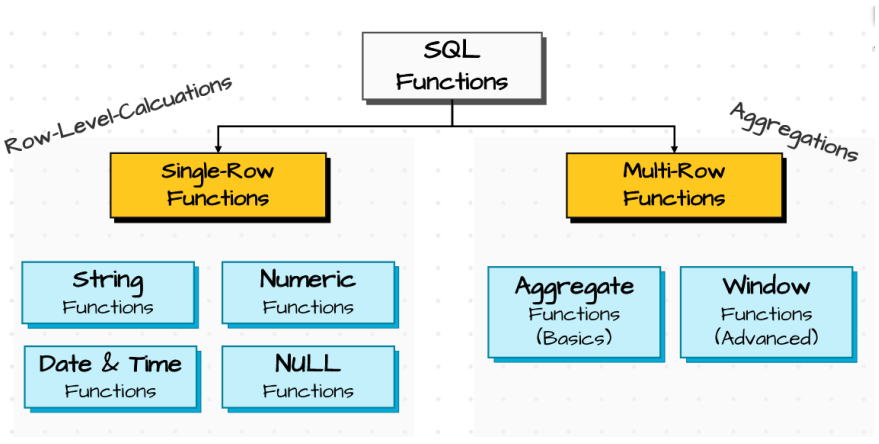
Nested Functions

- Function used inside another function



SQL Functions

1. Single-Row Functions
    a. String Functions
    b. Numeric Functions
    c. Date & Time Functions
    d. NULL Functions
2. Multi-Row Functions
    a. Aggregate Functions (Basics)
    b. Window Functions (Advanced)



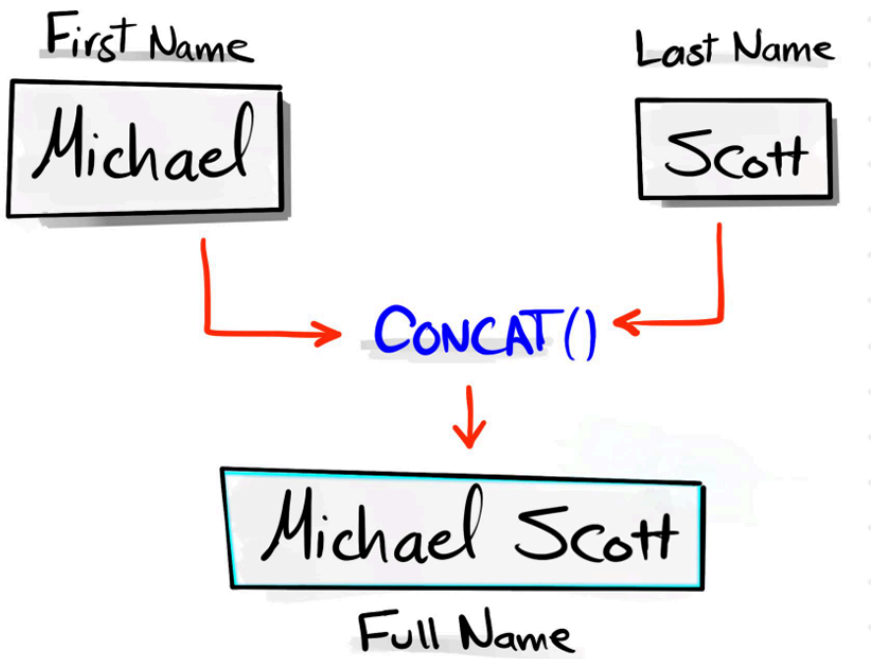
String Functions

- Manipulation
    - CONCAT
    - UPPER
    - LOWER
    - TRIM
    - REPLACE
- Calculation
    - LEN
- String Extraction
    - LEFT
    - RIGHT
    - SUBSTRING

CONCAT

- Combines multiple strings into one

```sql
SELECT CONCAT(first_name, '-', country) AS name_country
FROM customers;
```



UPPER

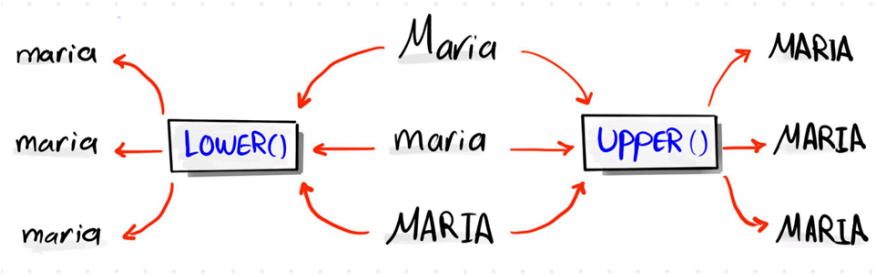
- Converts all characters to uppercase

```sql
SELECT UPPER(first_name) AS up_name
FROM customers;
```

LOWER

- Converts all characters to lowercase

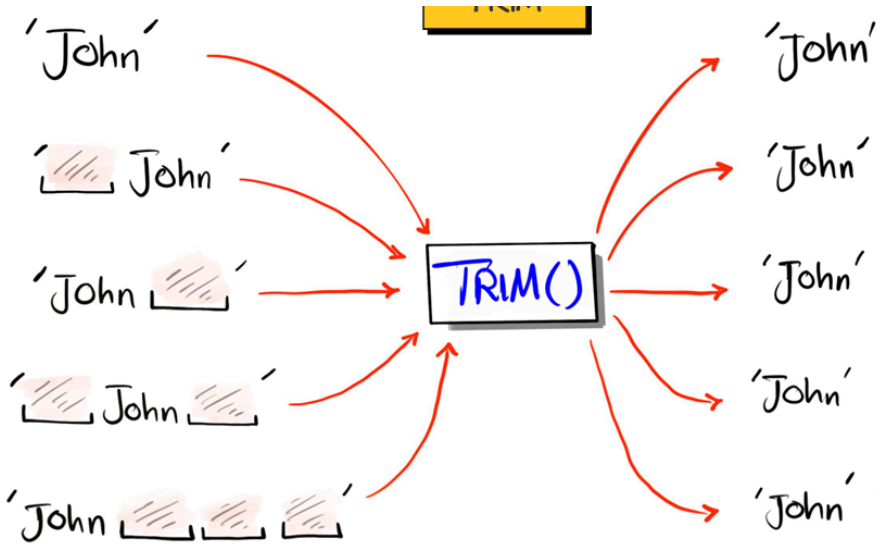
```sql
SELECT LOWER(first_name) AS low_name
FROM customers;
```



TRIM

- Removes Leading and Trailing spaces

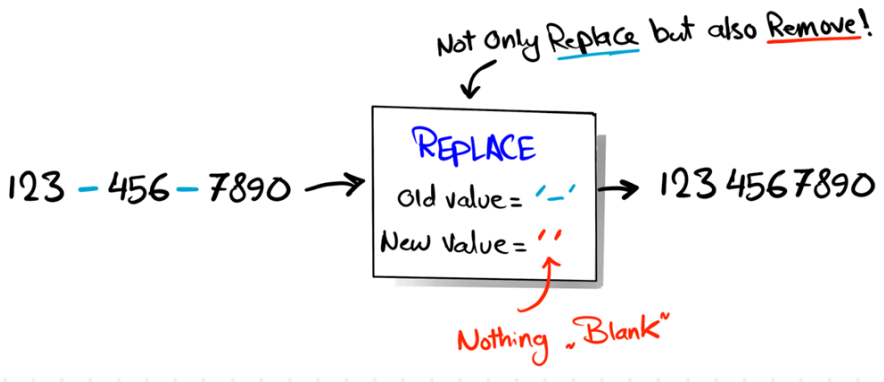
```sql
SELECT
    first_name,
    LEN(first_name) AS len_name,
    LEN(TRIM(first_name)) AS len_trim_name,
    LEN(first_name) - LEN(TRIM(first_name)) AS flag
FROM customers
-- WHERE LEN(first_name) != LEN(TRIM(first_name))
-- WHERE first_name != TRIM(first_name)
;
```



REPLACE

- Replaces specific character with a new character

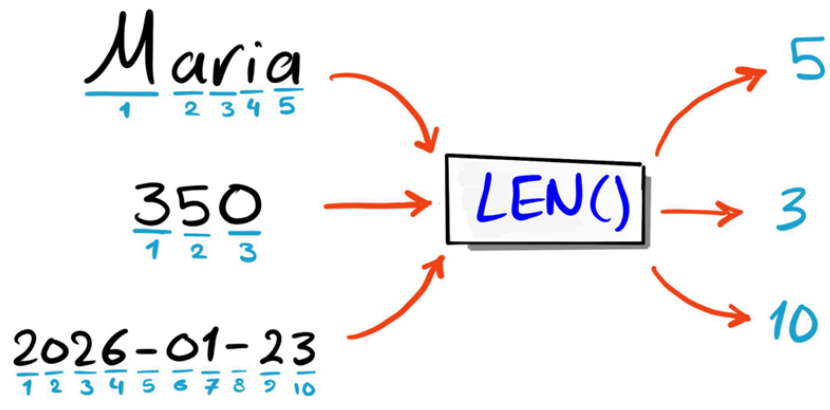
```
SELECT
    '123-456-7890' AS phone,
    REPLACE('123-456-7890', '-', '') AS clean_phone;

SELECT
    'report.txt' AS old_filename,
    REPLACE('report.txt', '.txt', '.csv') AS new_filename;
```



LEN

- Counts how many characters

```
SELECT
    first_name,
    LEN(first_name) AS len_name
FROM customers;
```



LEFT

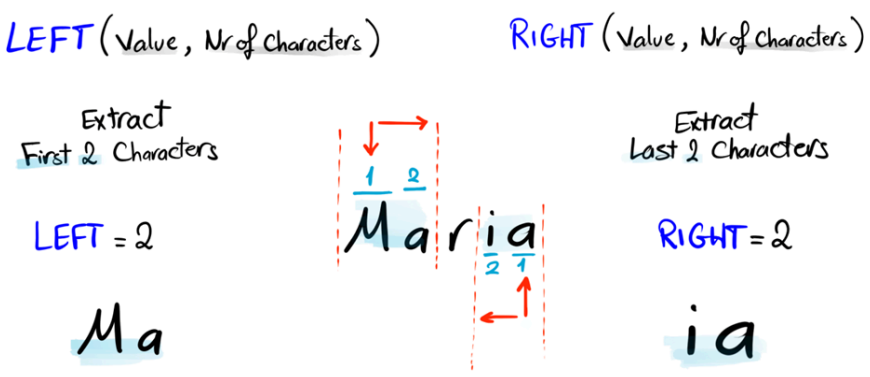
- Extracts specific Number of Characters from the start

```sql
SELECT
    first_name,
    LEFT(first_name, 2) AS first_2_char
FROM customers;
```

RIGHT

- Extracts specific Number of Characters from the end

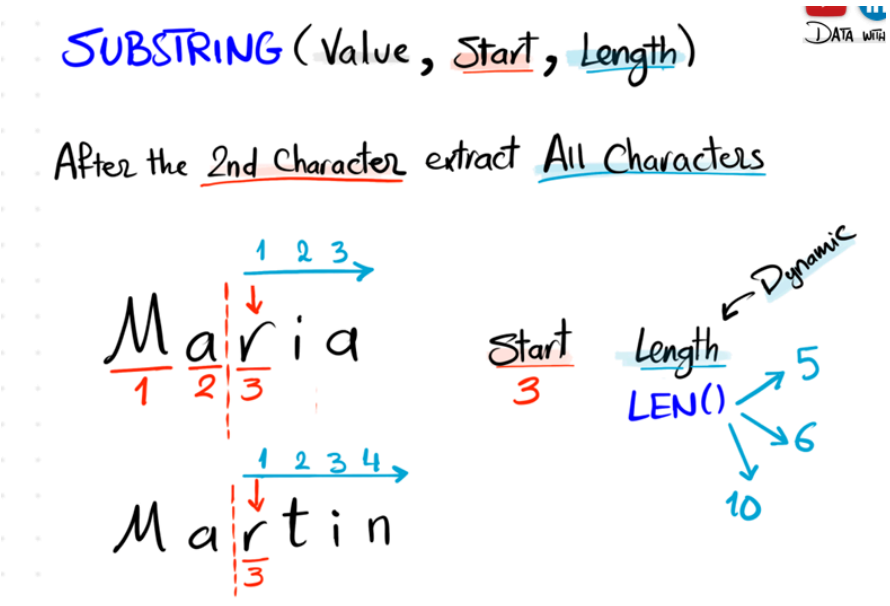
```sql
SELECT
    first_name,
    RIGHT(first_name, 2) AS last_2_char
FROM customers;
```

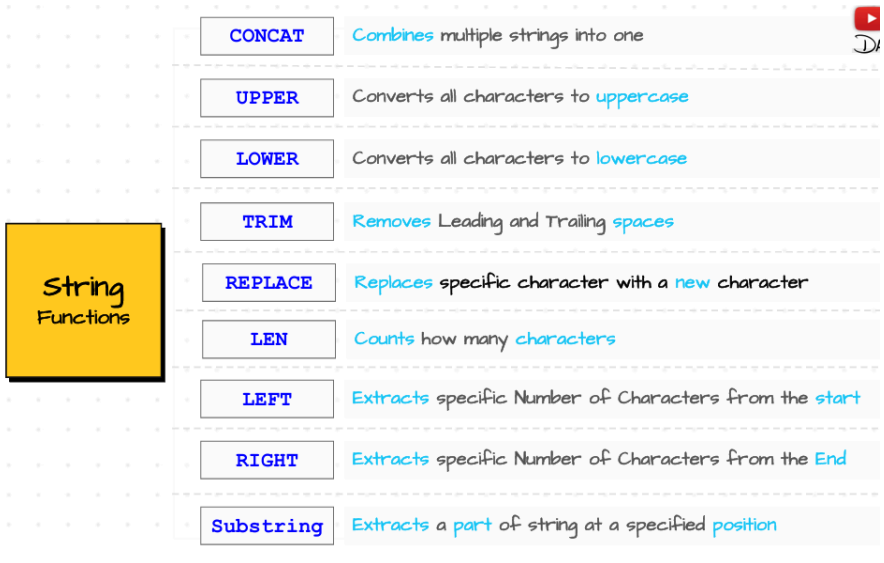


SUBSTRING

- Extracts a part of string at a specified position

```sql
SELECT
    first_name,
    SUBSTRING(first_name, 2, LEN(first_name)) AS sub_char
FROM customers;
```





Module 9: Numeric Functions

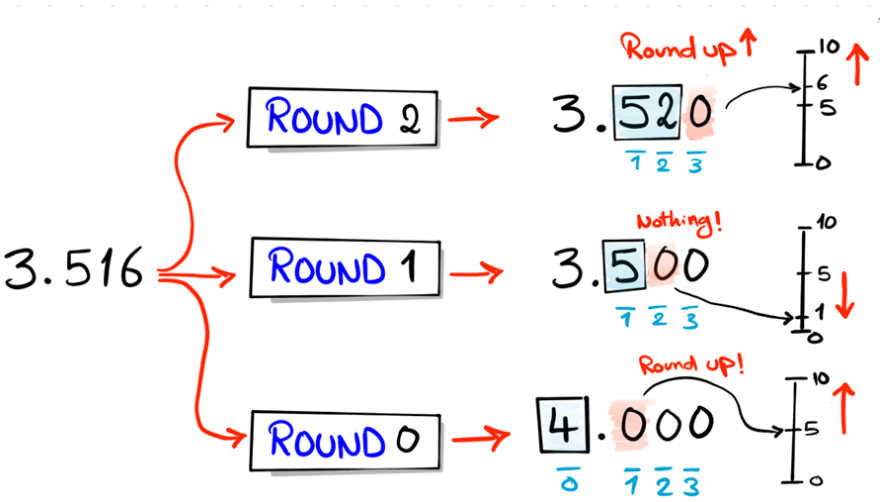
String Functions

- ROUND
- ABS

ROUND

```sql
SELECT
    3.516,
    ROUND(3.516, 2),
    ROUND(3.516, 1),
    ROUND(3.516, 0);
```

	number	round_2	round_1	round_0
1	3.516	3.520	3.500	4.000



ABS

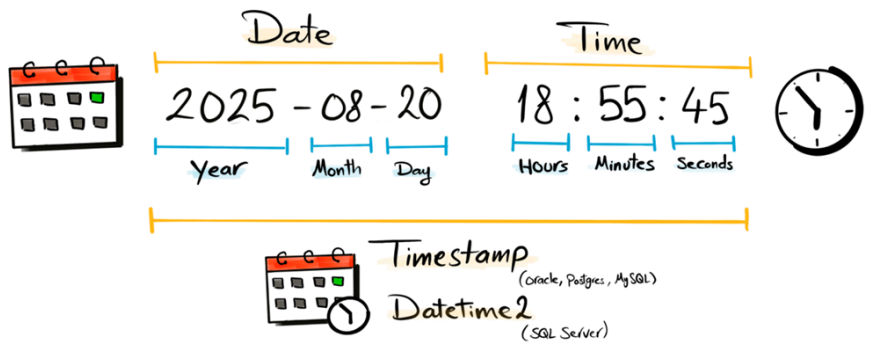
- Returns the absolute (positive) value of a number, removing any negative sign

```sql
SELECT
    -10 AS number,
    ABS(-10) AS absolute_1,
    ABS(10) AS absolute_2;
```

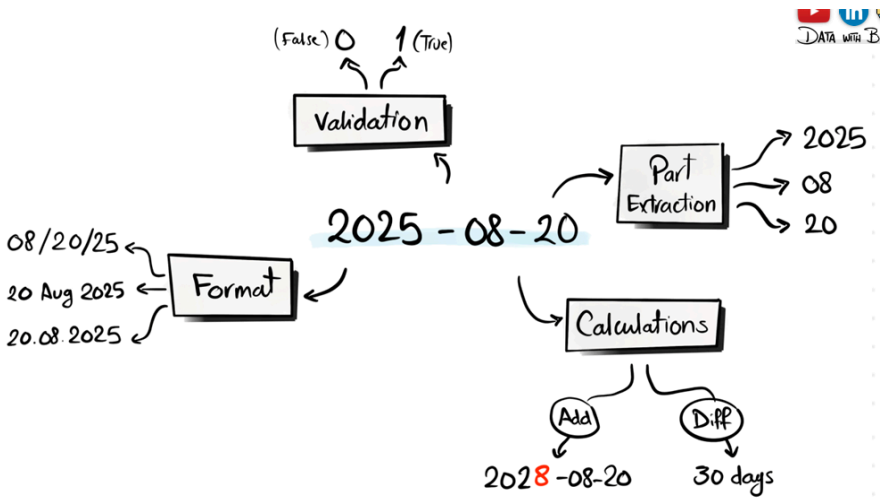
	number	absolute_1	absolute_2
1	-10	10	10

Module 10: Date & Time Functions

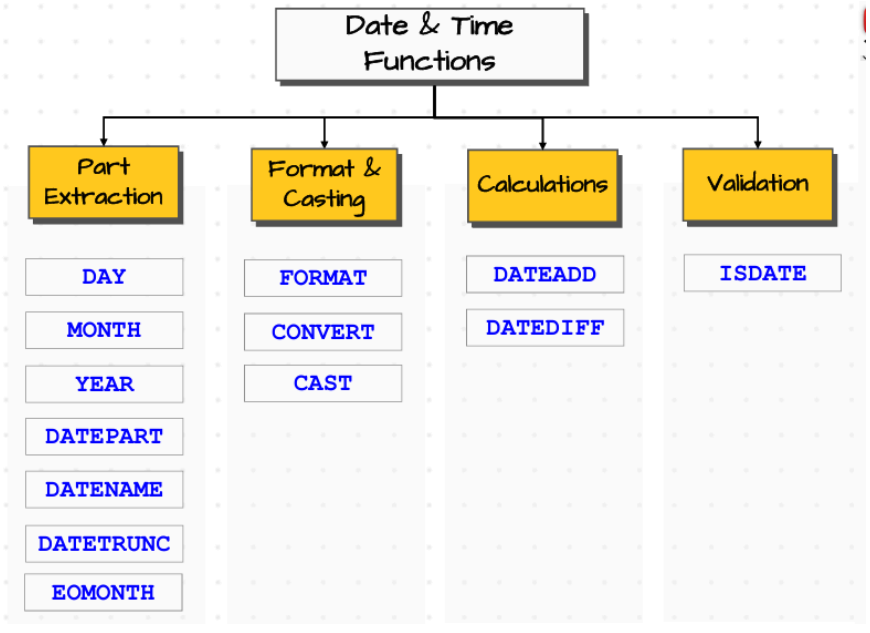
What is Date & Time



Date & Time Functions



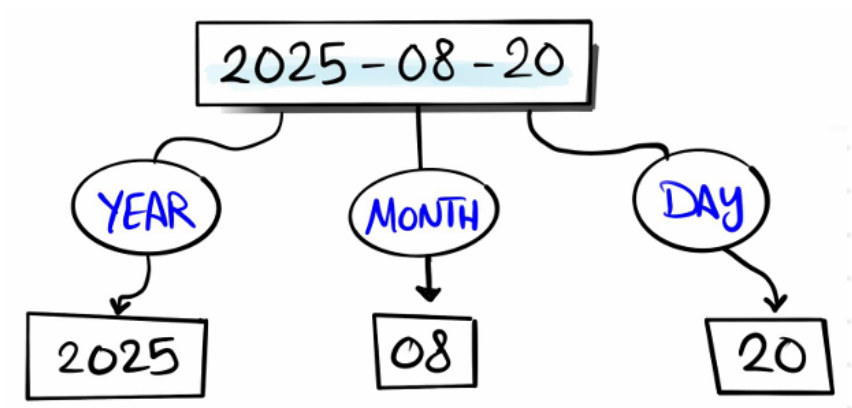
- Part Extraction
  - DAY
  - MONTH
  - YEAR
  - DATEPART
  - DATENAME
  - DATETRUNC
  - EOMONTH
- Format & Casting
  - FORMAT
  - CONVERT
  - CAST
- Calculations
  - DATEADD
  - DATEDIFF
- Validation
  - ISDATE



DAY, MONTH, YEAR

- DAY
  - Returns the day from a date
- MONTH
  - Returns the month from a date
- YEAR
  - Returns the year from a date

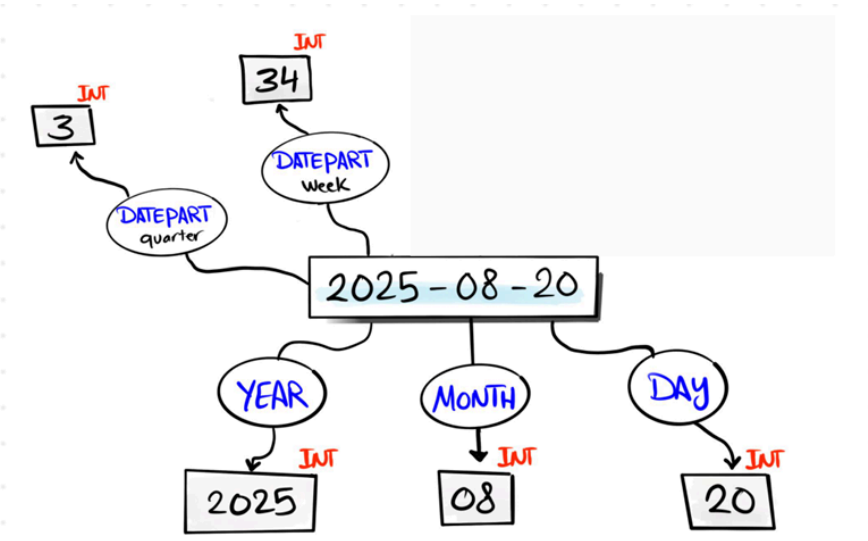
```sql
SELECT
    OrderID,
    CreationTime,
    YEAR(CreationTime) AS creationtime_year,
    MONTH(CreationTime) AS creationtime_month,
    DAY(CreationTime) AS creationtime_day
FROM Sales.Orders;
```



DATEPART

- Returns a specific part of a date as a number

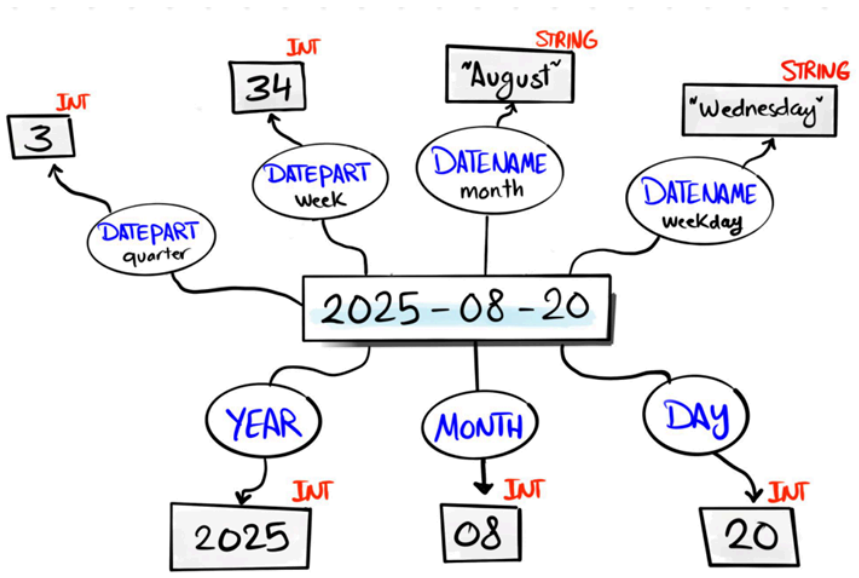
```sql
SELECT
    OrderID,
    CreationTime,
    DATEPART(year, CreationTime) AS Year_dp,
    DATEPART(month, CreationTime) AS Month_dp,
    DATEPART(day, CreationTime) AS Day_dp,
    DATEPART(hour, CreationTime) AS Hour_dp,
    DATEPART(week, CreationTime) AS Week_dp,
    DATEPART(quarter, CreationTime) AS Quarter_dp
FROM Sales.Orders;
```



DATENAME

- Returns the name of a specific part of a date

```
SELECT
    OrderID,
    CreationTime,
    DATENAME(month, CreationTime) AS Year_dn,
    DATENAME(weekday, CreationTime) AS Day_dn
FROM Sales.Orders;
```

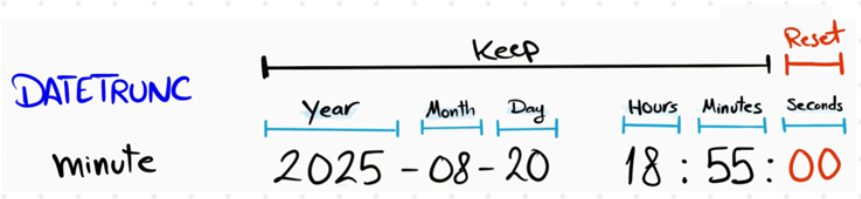


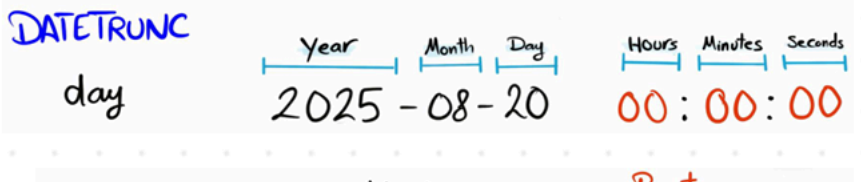
DATETRUNC

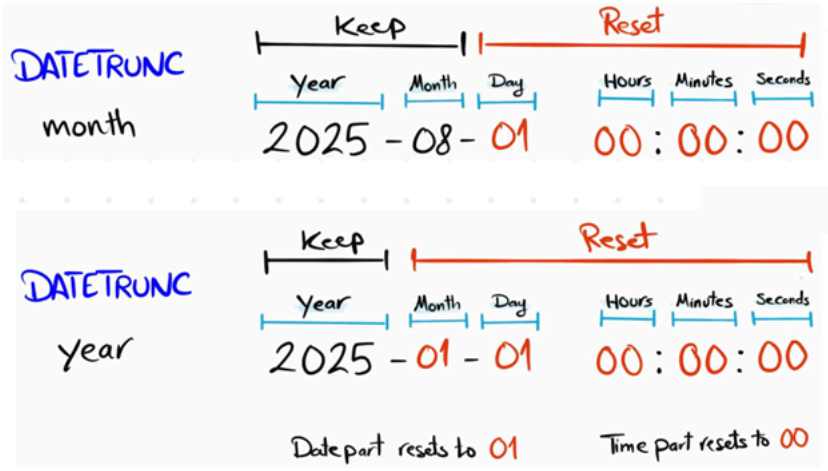
- Truncates the date to the specific part

```
SELECT
    OrderID,
    CreationTime,
    DATETRUNC(minute, CreationTime) AS Minute_dt,
    DATETRUNC(hour, CreationTime) AS Hour_dt,
    DATETRUNC(day, CreationTime) AS Day_dt,
    DATETRUNC(month, CreationTime) AS Month_dt,
    DATETRUNC(year, CreationTime) AS Year_dt
FROM Sales.Orders;
```

```
SELECT
    DATETRUNC(month,CreationTime) AS Creation,
    COUNT(*)
FROM Sales.Orders
GROUP BY DATETRUNC(month,CreationTime);
```



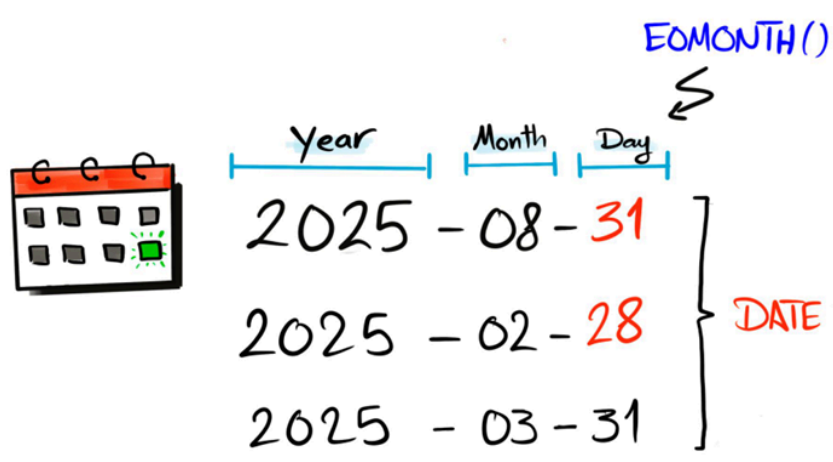




EOMONTH

- Returns the last day of a month

```sql
SELECT
    OrderID,
    CreationTime,
    EOMONTH(CreationTime) AS EndOfMonth,
    CAST(DATETRUNC(month,CreationTime) AS DATE) AS StartOfMonth
FROM Sales.Orders
```

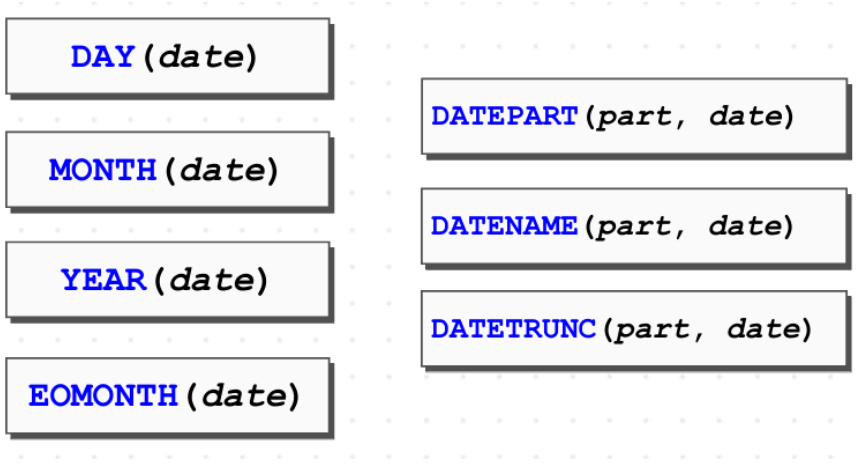


Notes

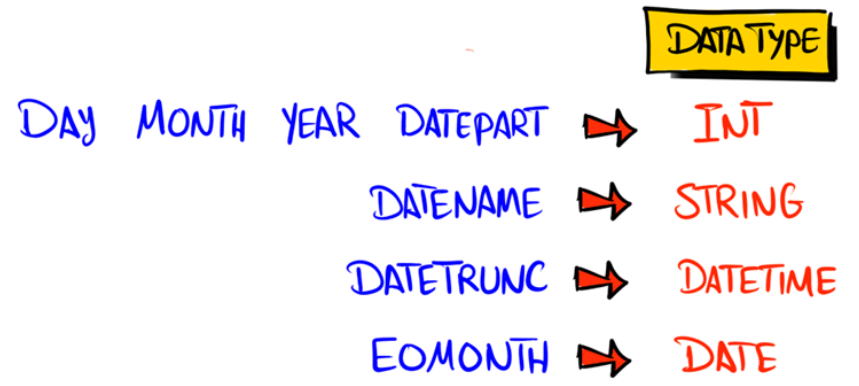
- Filtering data using an integer is faster than using a string
- Avoid using DATENAME for filtering data, instead use DATEPART

Part Extraction

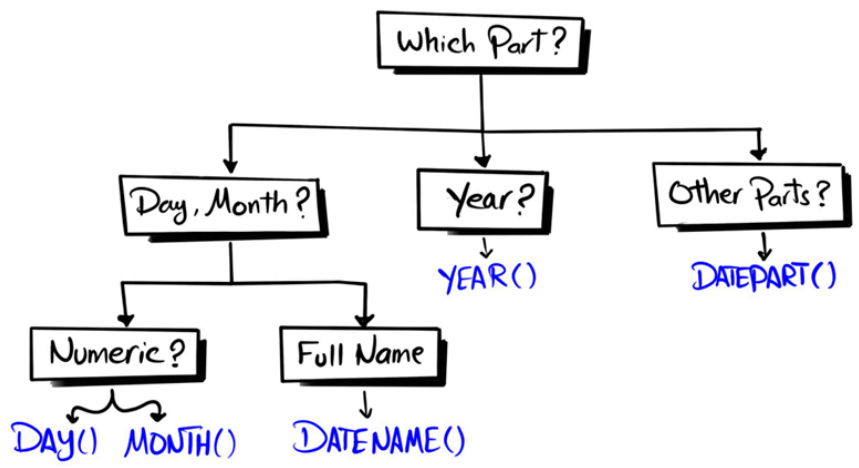
- Syntax

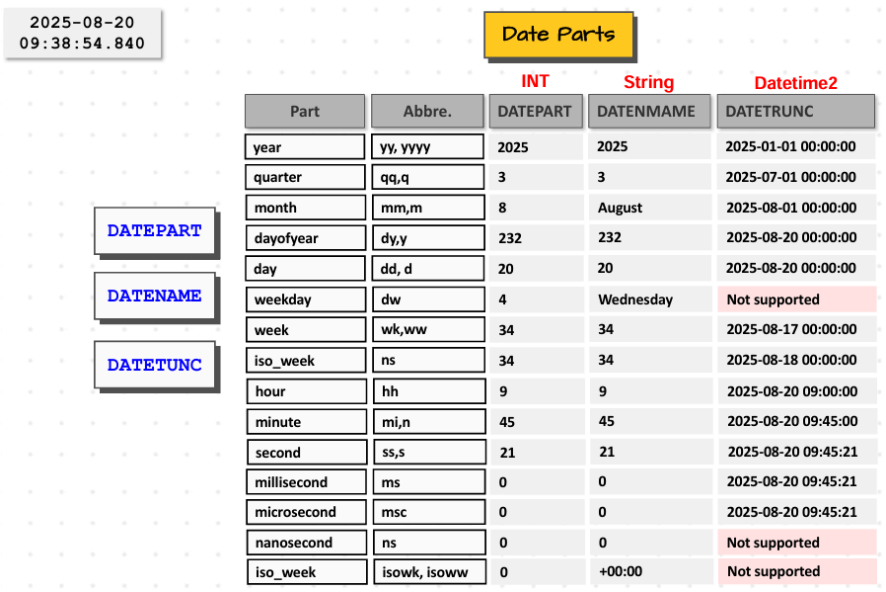


- Data Types



- Date Part

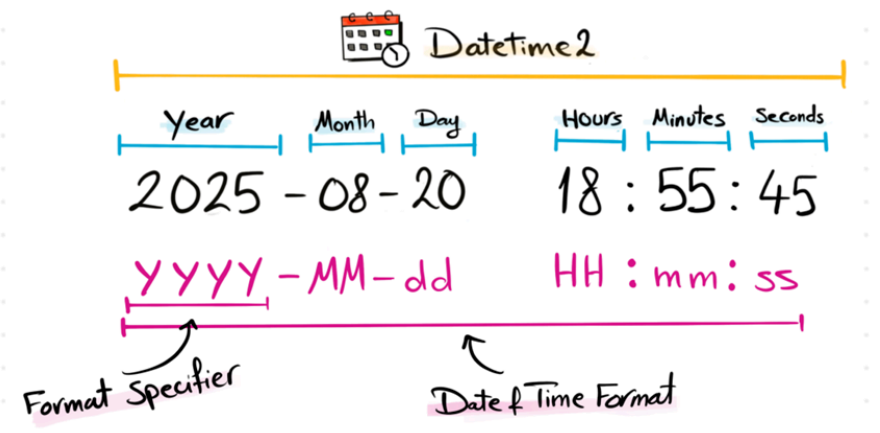




Date Parts

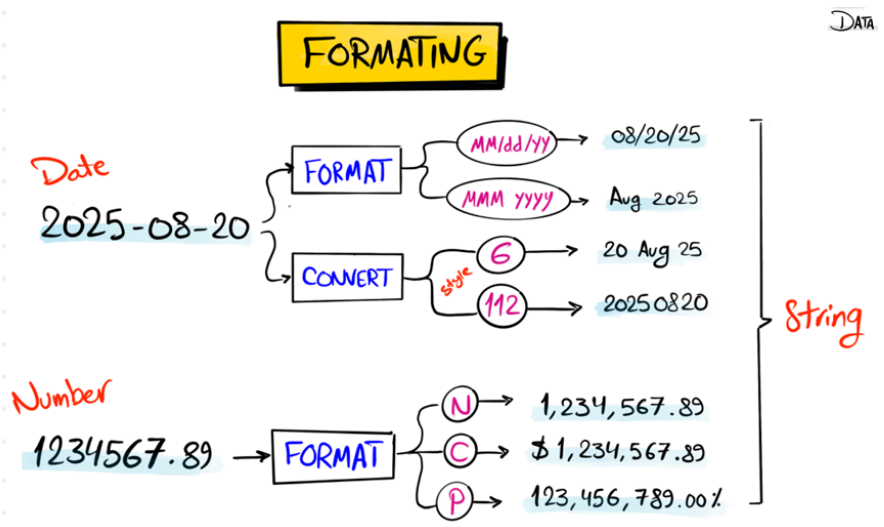
		INT	String	Datetime2
Part	Abbre.	DATEPART	DATENAME	DATETRUNC
year	yy, yyyy	2025	2025	2025-01-01 00:00:00
quarter	qq,q	3	3	2025-07-01 00:00:00
month	mm,m	8	August	2025-08-01 00:00:00
dayofyear	dy,y	232	232	2025-08-20 00:00:00
day	dd, d	20	20	2025-08-20 00:00:00
weekday	dw	4	Wednesday	Not supported
week	wk,ww	34	34	2025-08-17 00:00:00
iso_week	ns	34	34	2025-08-18 00:00:00
hour	hh	9	9	2025-08-20 09:00:00
minute	mi,n	45	45	2025-08-20 09:45:00
second	ss,s	21	21	2025-08-20 09:45:21
millisecond	ms	0	0	2025-08-20 09:45:21
microsecond	msc	0	0	2025-08-20 09:45:21
nanosecond	ns	0	0	Not supported
iso_week	isowk, isoww	0	+00:00	Not supported

Introduction to Formatting & Casting



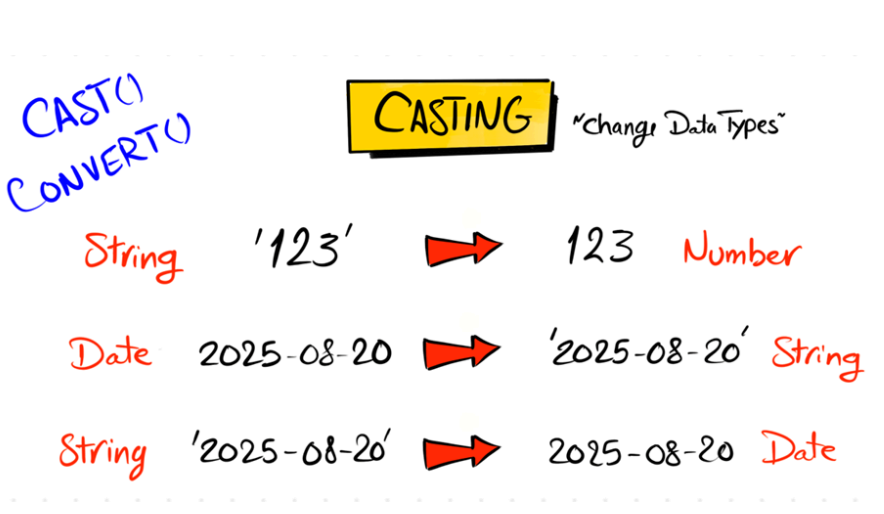
Formatting

- Changing the format of a value from one to another
- Changing how the data looks



Casting

- Changing the data type from one to another



FORMAT

- Formats a date or time value
- Syntax

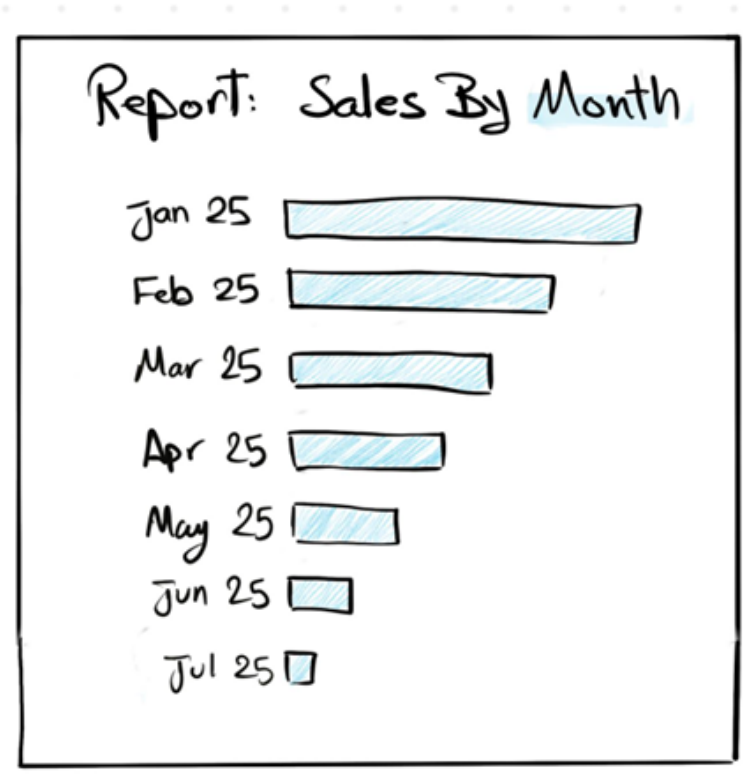
```
FORMAT(value, format [,culture])
--EX
```

```
FORMAT(OrderDate, 'dd/MM/yyyy')
FORMAT(OrderDate, 'dd/MM/yyyy', 'ja-JP')
FORMAT(1234.56, 'D', 'fr-FR')
```

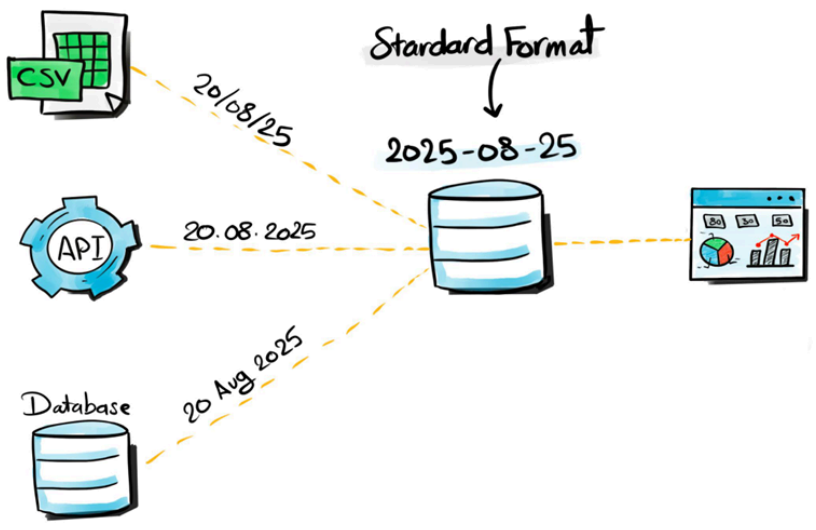
  - Default Culture = ‘en-US’

- Data Aggregations

```
SELECT
    FORMAT(OrderDate,'MMM yy') AS OrderDate,
    COUNT(*) AS NrOfOrders
FROM Sales.Orders
GROUP BY FORMAT(OrderDate,'MMM yy');
```



- Data Standardization



- All Formats

Format	Description	Result
D	Full day name	
d	Day of the month	8/20/2025
dd	Day of the month (two-digit)	20
ddd	Abbreviated day name	Wed
dddd	Full day name	Wednesday
M	Month number	44044
MM	Month number (two-digit)	8
MMM	Abbreviated month name	Aug
MMMM	Full month name	August
yy	Year (two-digit)	25
yyyy	Year (four-digit)	2025
hh	Hour (12-hour format, two-digit)	06
HH	Hour (24-hour format, two-digit)	18
m	Minutes	August 20
mm	Minutes (two-digit)	55
s	Seconds	2025-08-20T18:55:45
ss	Seconds (two-digit)	45
f	Fractional seconds (one digit)	Wednesday, August 20, 2025 6:55 PM
ff	Fractional seconds (two digits)	00
fff	Fractional seconds (three digits)	000
tt	AM/PM designator	PM

Format	Description	Query	Result
N	Numeric default	SELECT FORMAT(1234.56, 'N')	1,234.56
P	Percentage	SELECT FORMAT(1234.56, 'P')	123,456.00 %
C	Currency	SELECT FORMAT(1234.56, 'C')	$1,234.56
E	Scientific notation	SELECT FORMAT(1234.56, 'E')	1,23E+09
F	Fixed-point	SELECT FORMAT(1234.56, 'F')	1234.56
N0	Numeric no decimals	SELECT FORMAT(1234.56, 'N0')	1,235
N1	Numeric one decimal	SELECT FORMAT(1234.56, 'N1')	1,234.6
N2	Numeric two decimals	SELECT FORMAT(1234.56, 'N2')	1,234.56
N , de_DE	Numeric (German)	SELECT FORMAT(1234.56, 'N', 'de-DE')	1.234,56
N, en_US	Numeric (US)	SELECT FORMAT(1234.56, 'N', 'en-US')	1,234.56

```sql
SELECT
    OrderID,
    CreationTime,
    FORMAT(CreationTime, 'MM-dd-yyyy') USA_FORMAT,
    FORMAT(CreationTime, 'dd-MM-yyyy') EURO_FORMAT,
    FORMAT(CreationTime, 'dd') dd,
    FORMAT(CreationTime, 'ddd') ddd,
    FORMAT(CreationTime, 'dddd') dddd,
    FORMAT(CreationTime, 'MM') MM,
    FORMAT(CreationTime, 'MMM') MMM,
    FORMAT(CreationTime, 'MMMM') MMMM
FROM Sales.Orders;
```

```sql
/* Show Creation Time using the following format:
   Day Wed Jan Q1 2025 12:34:56 PM */

SELECT
    OrderID,
    CreationTime,
    'Day ' + FORMAT(CreationTime, 'ddd MMM') +
    ' Q' + DATENAME(quarter,CreationTime) + ' ' +
    FORMAT(CreationTime, 'yyyy hh:mm:ss tt') AS CustomeFormat
FROM Sales.Orders;
```

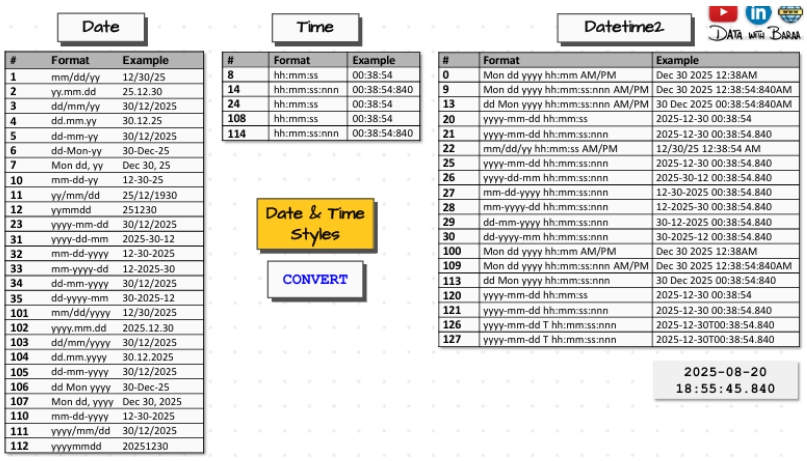
CONVERT

- Converts a date or time value to a different data type
- Formats the value
- Syntax

```sql
CONVERT(data_type, value [,style])
--EX
CONVERT(INT, '123')
CONVERT(VARCHAR, OrderDate, '34')
```

- Default Style = 0

- All Styles



#	Format	Example
1	mm/dd/yy	12/30/25
2	yy.mm.dd	25.12.30
3	dd/mm/yy	30/12/2025
4	dd.mm.yy	30.12.25
5	dd-mm-yy	30/12/2025
6	dd-Mon-yy	30-Dec-25
7	Mon dd, yy	Dec 30, 25
10	mm-dd-yy	12-30-25
11	yy/mm/dd	25/12/1930
12	yymmdd	251230
23	yyyy-mm-dd	30/12/2025
31	yyyy-dd-mm	2025-30-12
32	mm-dd-yyyy	12-30-2025
33	mm-yyyy-dd	12-2025-30
34	dd-mm-yyyy	30/12/2025
35	dd-yyyy-mm	30-2025-12
101	mm/dd/yyyy	12/30/2025
102	yyyy.mm.dd	2025.12.30
103	dd/mm/yyyy	30/12/2025
104	dd.mm.yyyy	30.12.2025
105	dd-mm-yyyy	30/12/2025
106	dd Mon yyyy	30-Dec-25
107	Mon dd, yyyy	Dec 30, 2025
110	mm-dd-yyyy	12-30-2025
111	yyyy/mm/dd	30/12/2025
112	yyyymmdd	20251230

#	Format	Example
8	hh:mm:ss	00:38:54
14	hh:mm:ss:nnn	00:38:54:840
24	hh:mm:ss	00:38:54
108	hh:mm:ss	00:38:54
114	hh:mm:ss:nnn	00:38:54:840

#	Format	Example
0	Mon dd yyyy hh:mm AM/PM	Dec 30 2025 12:38AM
9	Mon dd yyyy hh:mm:ss:nnn AM/PM	Dec 30 2025 12:38:54:840AM
13	dd Mon yyyy hh:mm:ss:nnn AM/PM	30 Dec 2025 00:38:54:840AM
20	yyyy-mm-dd hh:mm:ss	2025-12-30 00:38:54
21	yyyy-mm-dd hh:mm:ss:nnn	2025-12-30 00:38:54.840
22	mm/dd/yy hh:mm:ss AM/PM	12/30/25 12:38:54 AM
25	yyyy-mm-dd hh:mm:ss:nnn	2025-12-30 00:38:54.840
26	yyyy-dd-mm hh:mm:ss:nnn	2025-30-12 00:38:54.840
27	mm-dd-yyyy hh:mm:ss:nnn	12-30-2025 00:38:54.840
28	mm-yyyy-dd hh:mm:ss:nnn	12-2025-30 00:38:54.840
29	dd-mm-yyyy hh:mm:ss:nnn	30-12-2025 00:38:54.840
30	dd-yyyy-mm hh:mm:ss:nnn	30-2025-12 00:38:54.840
100	Mon dd yyyy hh:mm AM/PM	Dec 30 2025 12:38AM
109	Mon dd yyyy hh:mm:ss:nnn AM/PM	Dec 30 2025 12:38:54:840AM
113	dd Mon yyyy hh:mm:ss:nnn	30 Dec 2025 00:38:54:840
120	yyyy-mm-dd hh:mm:ss	2025-12-30 00:38:54
121	yyyy-mm-dd hh:mm:ss:nnn	2025-12-30 00:38:54.840
126	yyyy-mm-dd T hh:mm:ss:nnn	2025-12-30T00:38:54.840
127	yyyy-mm-dd T hh:mm:ss:nnn	2025-12-30T00:38:54.840

```sql
SELECT
CONVERT(INT, '123') AS [String to Int CONVERT],
CONVERT(DATE, '2005-11-06') AS [String to Date CONVERT];
```

```sql
SELECT
    CreationTime,
    CONVERT(DATE, CreationTime) AS [Datetime to Date CONVERT],
    CONVERT(VARCHAR, CreationTime, 32) AS [USA Std. Style:32],
    CONVERT(VARCHAR, CreationTime, 34) AS [EURO Std. Style:34]
FROM Sales.Orders;
```

CAST

- Converts a value to a specified data type
- Syntax

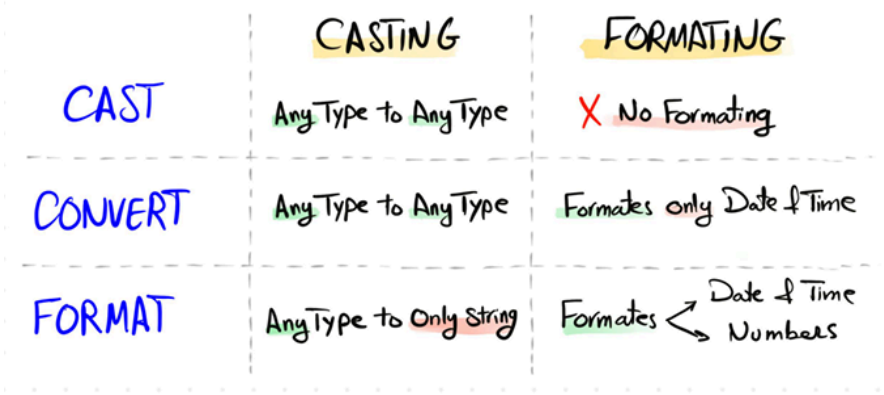
```sql
CAST(value AS data_type)
--EX
CAST('123' AS INT)
CAST('2005-11-06' AS DATE)
```

  - No format can be specified

```sql
SELECT
CAST('123' AS INT) AS [String to Int CAST],
CAST(123 AS VARCHAR) AS [Int to String CAST],
CAST('2005-11-06' AS DATE) AS [String to Date CAST],
CAST('2005-11-06' AS DATETIME2) AS [String to Datetime CAST];
```

```sql
SELECT
    CreationTime,
    CAST(CreationTime AS DATE) AS [Datetime to Date CAST]
FROM Sales.Orders;
```

FORMAT vs CONVERT vs CAST

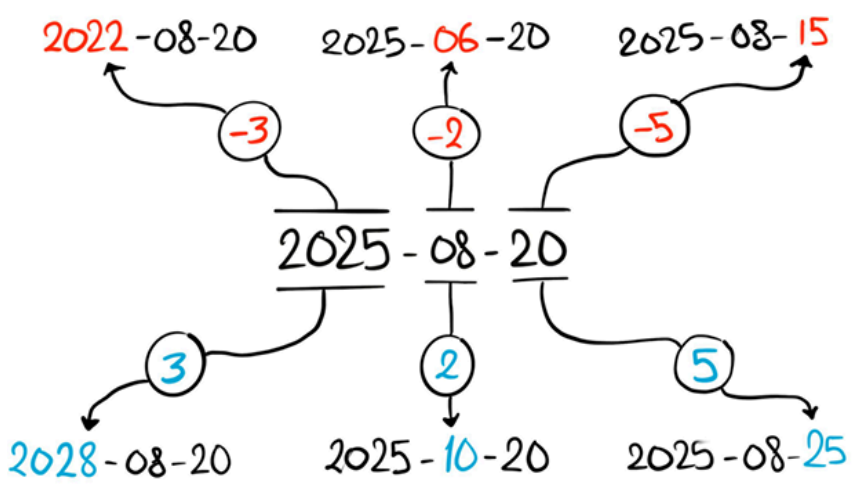


DATEADD

- Adds or subtracts a specific time interval to/from a date
- Syntax

```
DATEADD(part, interval, date)
--EX
DATEADD(year, 2, OrderDate)
DATEADD(month, -4, OrderDate)
```

```
SELECT
    OrderID,
    OrderDate,
    DATEADD(day, -10, OrderDate) AS TenDaysBefore,
    DATEADD(month, 3, OrderDate) AS ThreeMonthsLater,
    DATEADD(year, 2, OrderDate) AS TwoYearsLater
FROM Sales.Orders;
```



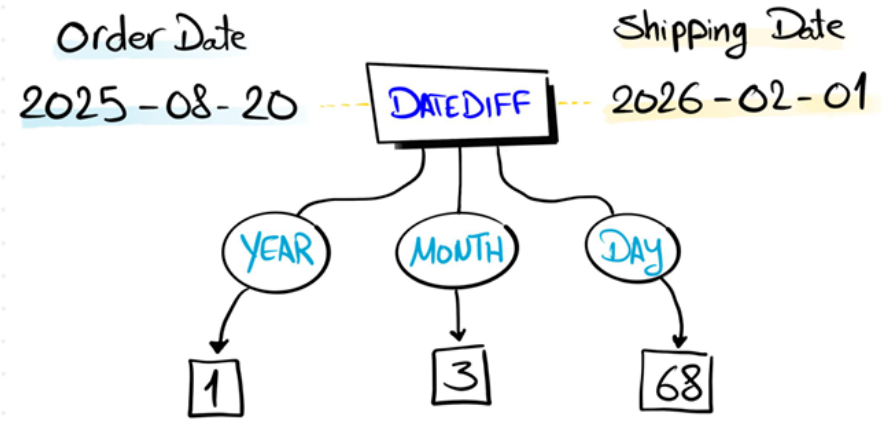
DATEDIFF

- Find the difference between two dates
- Syntax

```
DATEDIFF(part, start_date, end_date)
--EX
DATEDIFF(year, OrderDate, ShipDate)
DATEDIFF(month, OrderDate, ShipDate)
DATEDIFF(day, OrderDate, ShipDate)
```

```
SELECT
    EmployeeID,
    FirstName,
    BirthDate,
```

```
    DATEDIFF(year, BirthDate, GETDATE()) AS Age
FROM Sales.Employees;
```



ISDATE

- Check if a value is a date
  - Returns 1 → if the string is a valid date
  - Returns 0 → if the string is not a valid date
- Syntax

```
ISDATE(value)
--EX
ISDATE('2005-11-06')
ISDATE(2026)
```

```
SELECT
    ISDATE('123') AS DateCheck1,
    ISDATE('2005-11-06') AS DateCheck2,
    ISDATE('06-11-2005') AS DateCheck3,
    ISDATE('2005') AS DateCheck4,
    ISDATE('11') AS DateCheck5;
```

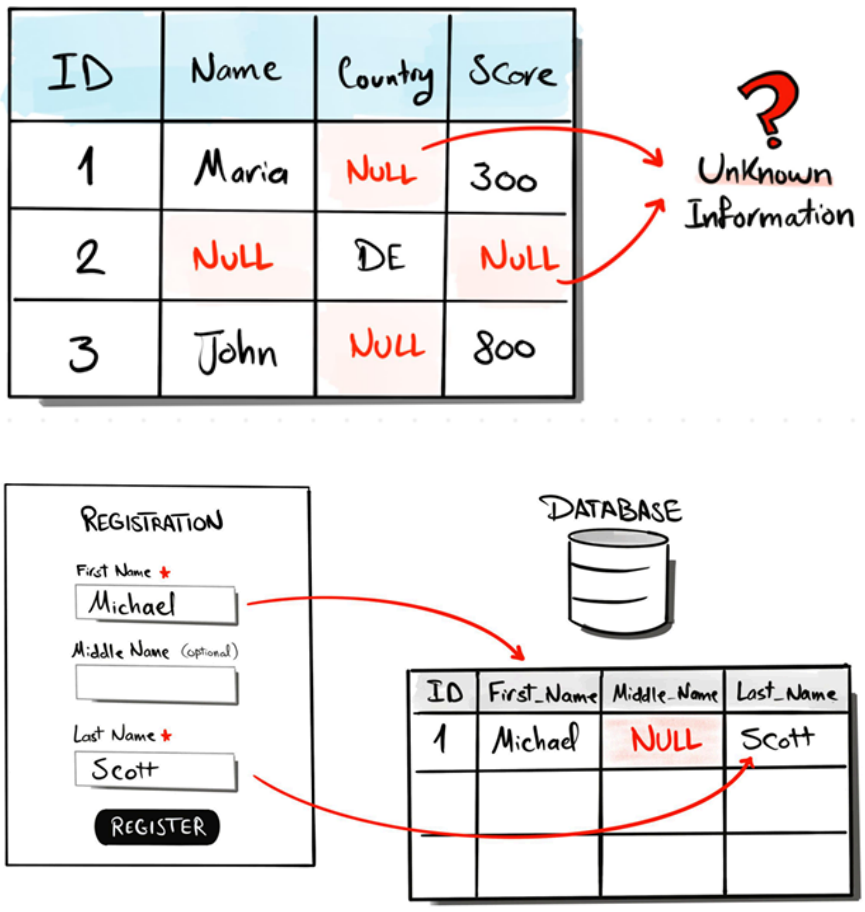
```
SELECT
    --CAST(OrderDate AS DATE) AS OrderDate,
    OrderDate,
    ISDATE(OrderDate),
    CASE WHEN ISDATE(OrderDate) = 1 THEN CAST(OrderDate AS DATE)
        ELSE '9999-09-09'
    END AS NewOrderDate
FROM
(
    SELECT '2025-09-12' AS OrderDate UNION
    SELECT '2025-09-13' UNION
    SELECT '2025-09-15' UNION
    SELECT '2025-09'
)t;
```

Module 11: NULL Functions

Introduction to NULLs

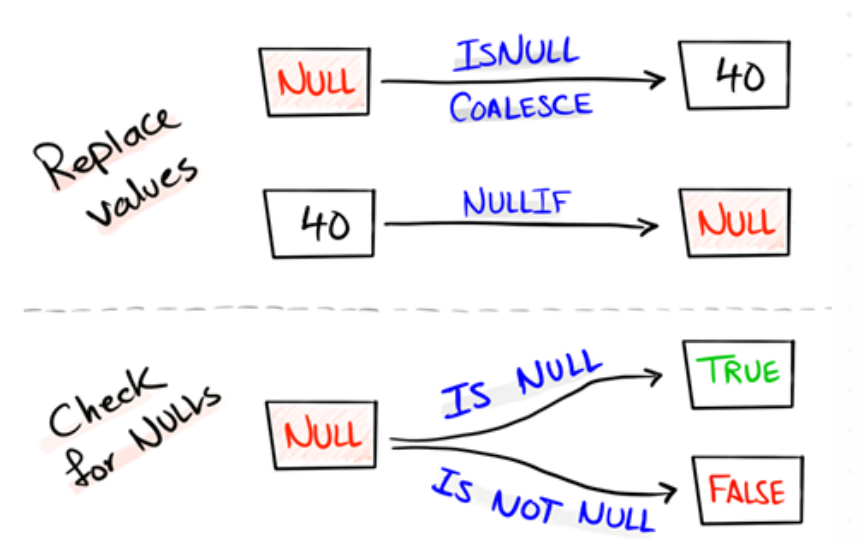
- NULL means nothing, unknown!
- NULL is not equal to anything

- NULL is not zero
- NULL is not empty string
- NULL is not blank space



NULL Functions

- Replace values
  - ISNULL
  - COALESCE
  - NULLIF
- Check for NULLs
  - IS NULL
  - IS NOT NULL

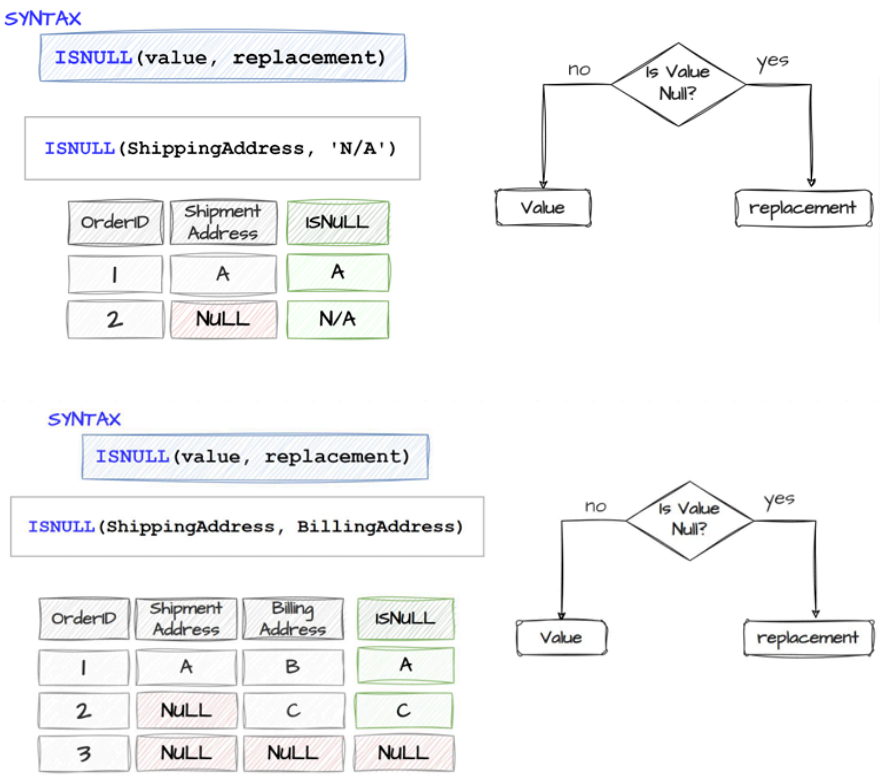


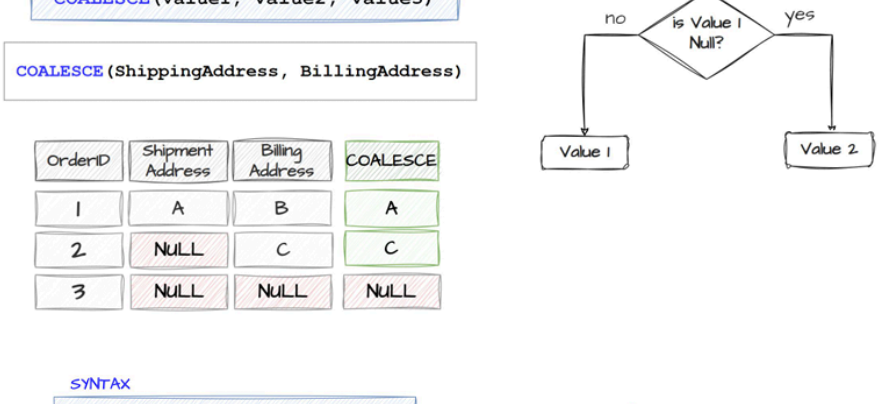
ISNULL

- Replaces ‘NULL’ with a specified value
- Syntax

```
ISNULL(value, replacement_value)
--EX
ISNULL(Shippin_Address, 'unknown')
```

```
--'unknown' = Default Value
ISNULL(Shippin_Address, Billing_Address)
```

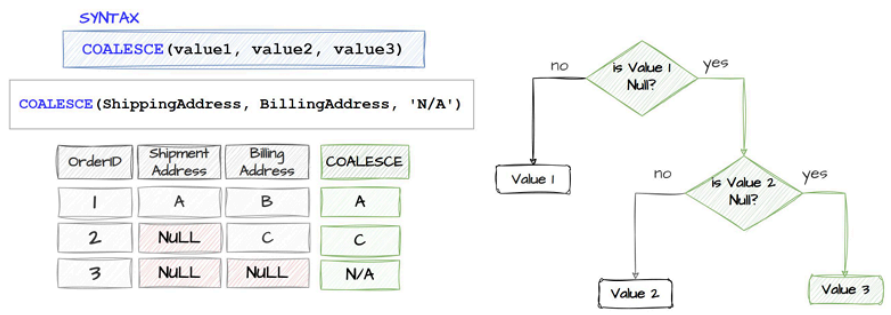




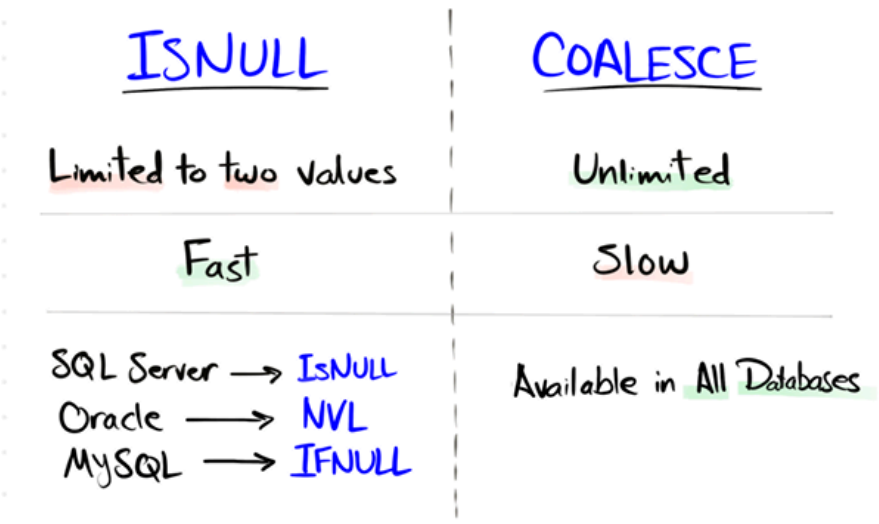
COALESCE

- Returns the first non-NULL value from a list
- Syntax

```
COALESCE(value1, value2, value3, ...)
--EX
COALESCE(Shippin_Address, 'unknown')
COALESCE(Shippin_Address, Billing_Address)
COALESCE(Shippin_Address, Billing_Address, 'unknown')
```

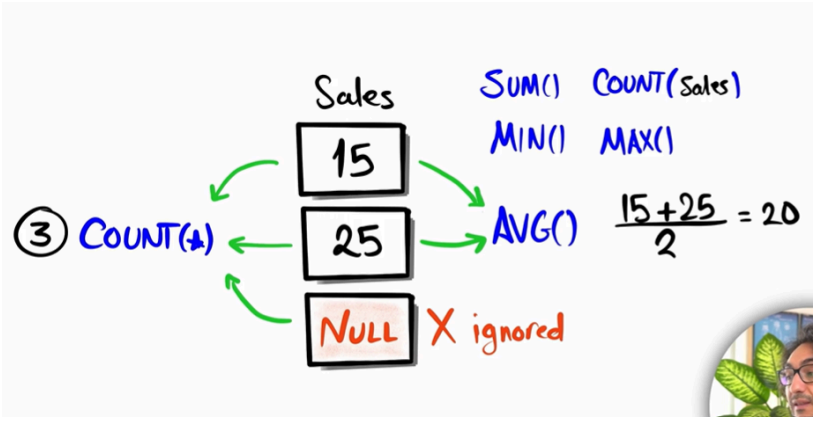



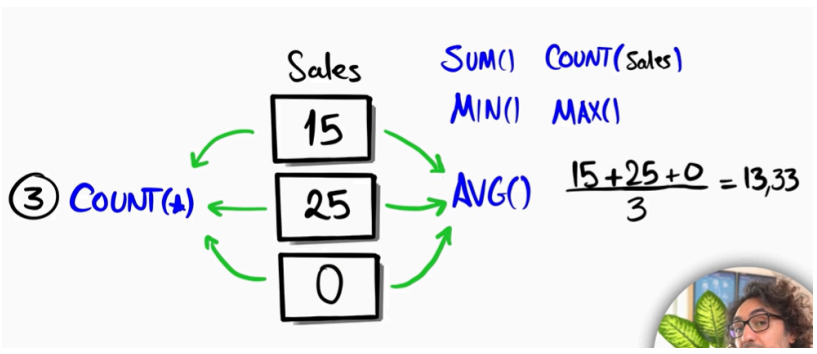
ISNULL vs COALESCE



Handling NULLs Cases (ISNULL & COALESCE)

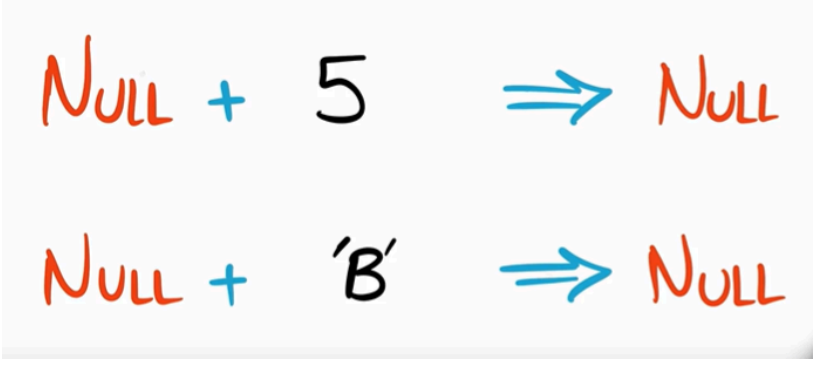
- Data Aggregation
  - Handle the NULL before doing data aggregations






```sql
SELECT
    CustomerID,
    Score,
    COALESCE(Score, 0) AS Score2,
    AVG(Score) OVER () AS AvgScores,
    AVG(COALESCE(Score, 0)) OVER () AS CorrectAvgScores
FROM Sales.Customers;
```

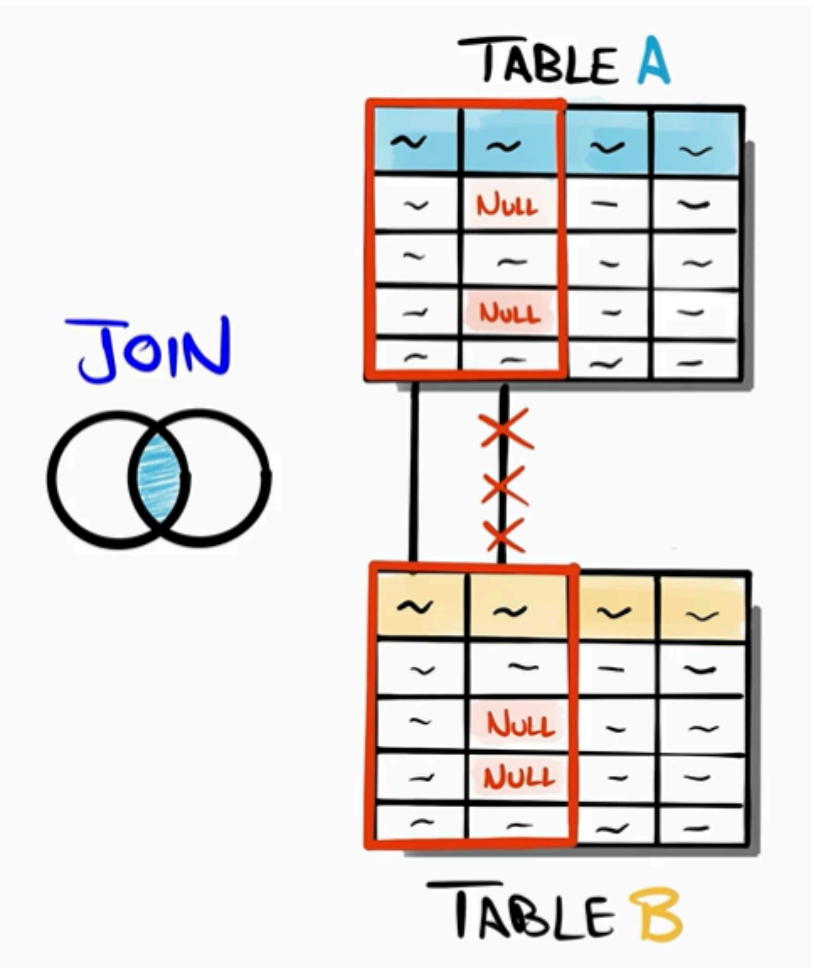
- Mathematical Operations
  - Handle the NULL before doing mathematical operations

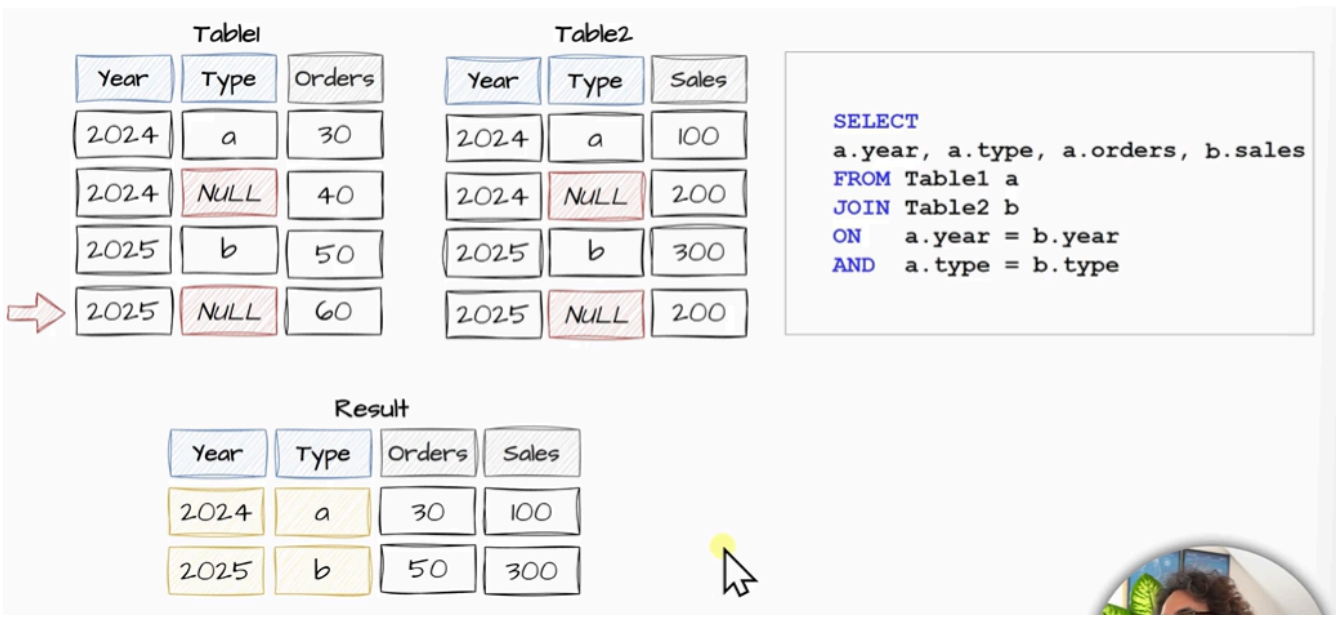


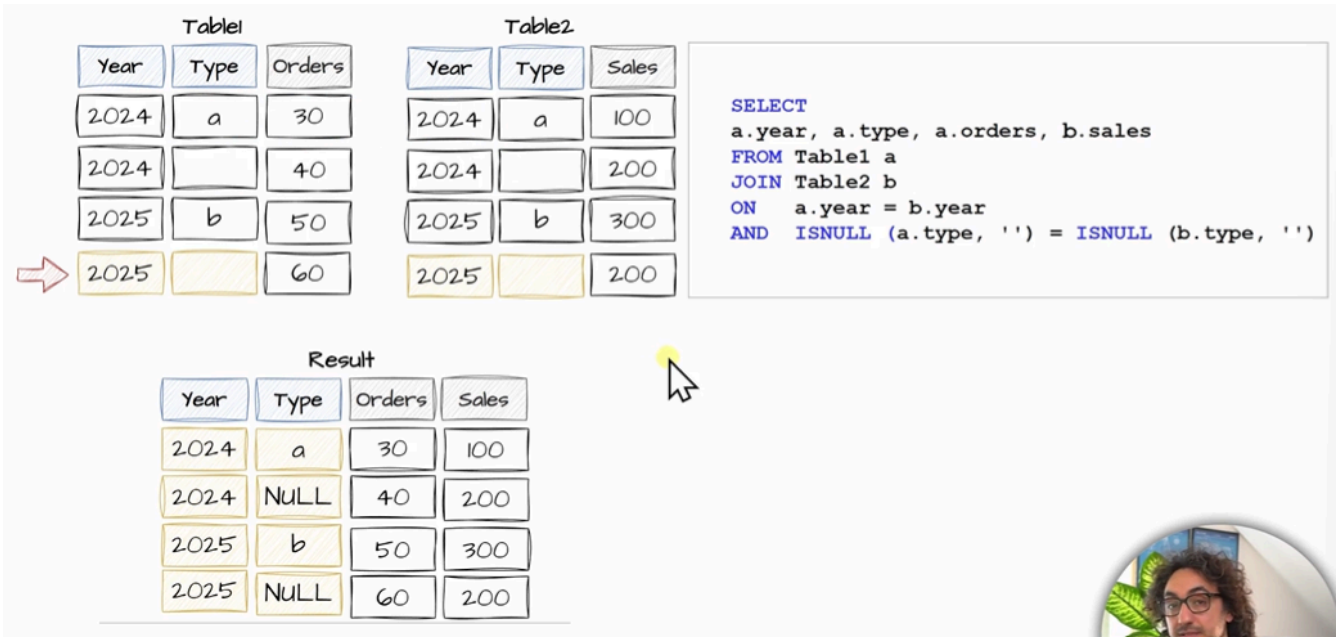

```sql
SELECT
    CustomerID,
```

```
    FirstName,
    LastName,
    FirstName + ' ' + LastName AS FullName1,
    FirstName + ' ' + COALESCE(LastName, '') AS FullName2,
    Score,
    Score + 10 AS NewScore1,
    COALESCE(Score, 0) + 10 AS NewScore2
FROM Sales.Customers;
```

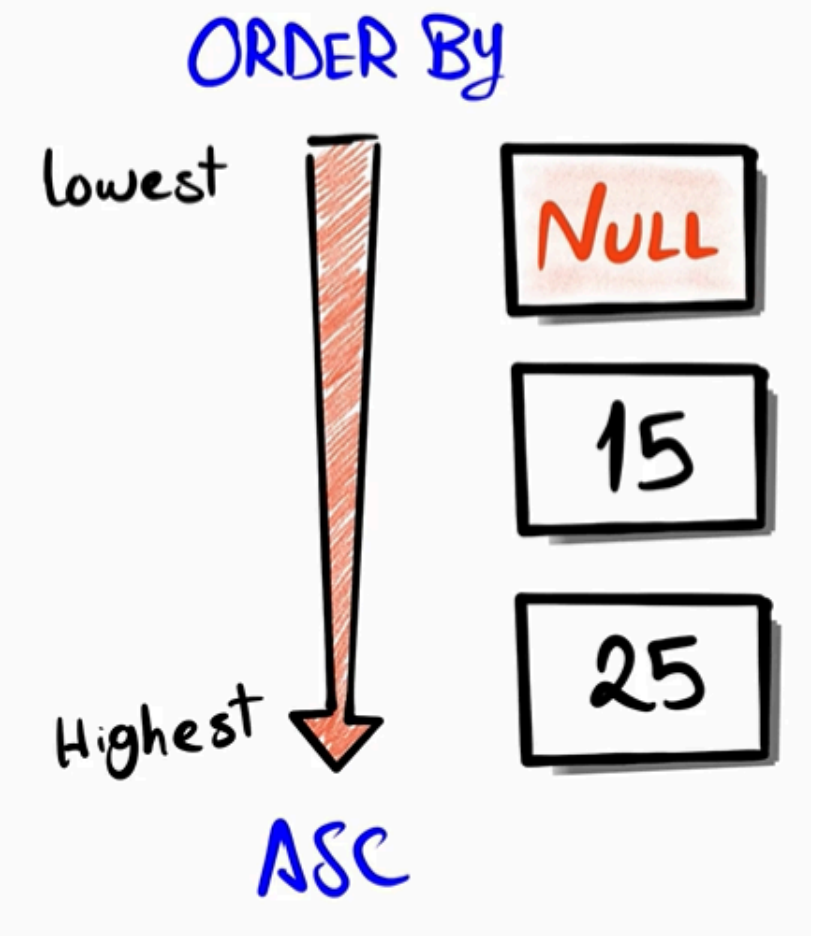
- Joining Data
  - Handle the NULL before joining data

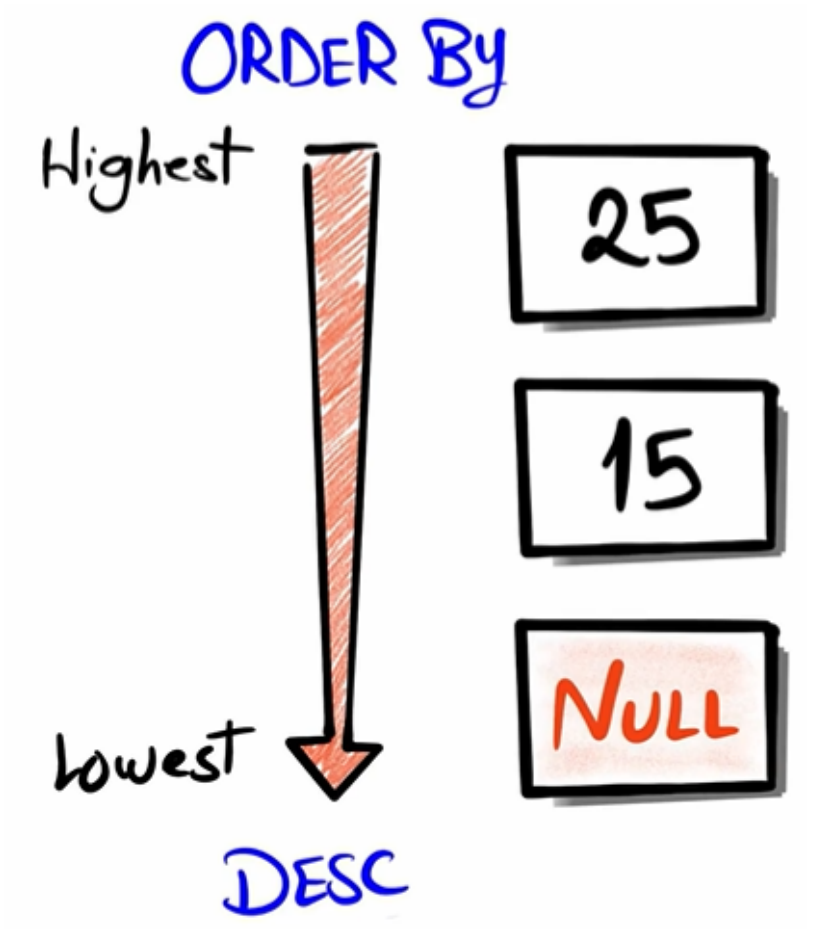






- Sorting Data
  - Handle the NULL before sorting data



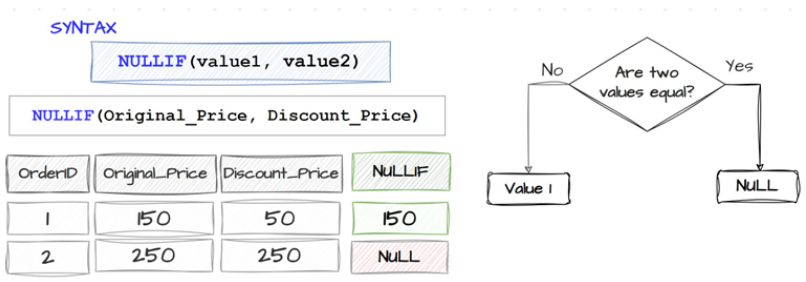


```sql
SELECT
    CustomerID,
    Score
    -- CASE WHEN Score IS NULL THEN 1 ELSE 0 END Flag
FROM Sales.Customers
ORDER BY
    CASE WHEN Score IS NULL THEN 1 ELSE 0 END ASC,
    Score ASC;
```

NULLIF

- Compares two values returns:
  - NULL → if they are equal
  - First Value → if they are not equal
- Syntax

```sql
NULLIF(value1, value2)
--EX
NULLIF(Shippin_Address, 'unknown')
NULLIF(Shippin_Address, Billing_Address)
```



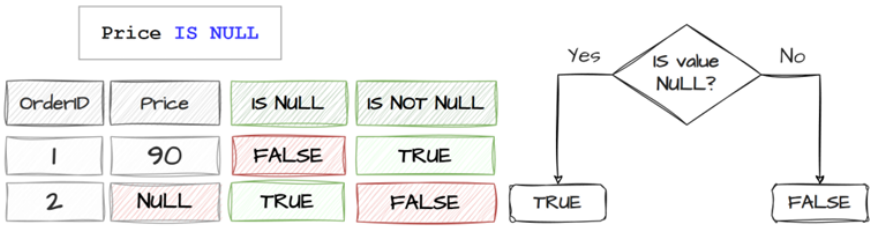
- Use Case
  - Division by Zero
    - Preventing the error of dividing by zero

```sql
SELECT
    OrderID,
    Sales,
    Quantity,
    Sales / NULLIF(Quantity, 0) AS SalePrice
FROM Sales.Orders;
```

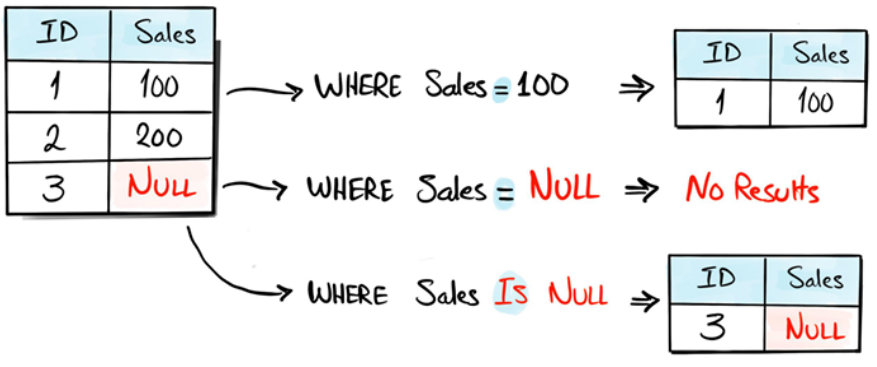
IS NULL

- Returns TRUE → if the value is NULL
- Otherwise → it returns FALSE
- Syntax

```sql
Value IS NULL
--EX
Shipping_Address IS NULL
```



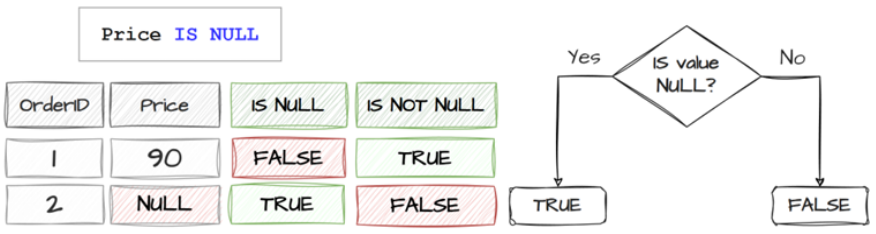
- In SQL, use IS NULL instead of = NULL to correctly filter rows with NULL values



IS NOT NULL

- Returns TRUE → if the value is NOT NULL
- Otherwise → it returns FALSE
- Syntax

```
Value IS NOT NULL
--EX
Shipping_Address IS NOT NULL
```



Use Cases (IS NULL & IS NOT NULL)

- Filtering Data
  - Searching for missing information

```
SELECT
    CustomerID,
    FirstName,
    Score
FROM Sales.Customers
WHERE Score IS NULL;
```

```
SELECT
    CustomerID,
    FirstName,
    Score
FROM Sales.Customers
WHERE Score IS NOT NULL;
```

- ANTI JOINs
  - LEFT ANTI JOIN
  - RIGHT ANTI JOIN

- Finding the unmatched rows between two tables

```sql
SELECT
    c.*,
    o.OrderID
FROM Sales.Customers c
LEFT JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID
WHERE o.CustomerID IS NULL;
```

NULL vs Empty String vs Blank Space

- NULL
  - Means nothing, unknown!
- Empty String ''
  - String value has zero characters
- Blank Space ' '
  - String value has one or more space characters

```sql
WITH Orders AS (
SELECT 1 Id, 'A' Category UNION
SELECT 2, NULL UNION
SELECT 3, '' UNION
SELECT 4, ' '
)
SELECT
    *,
    DATALENGTH(Category) CategoryLen
FROM Orders;
```

	NULL	Empty String	Blank Space
Representation	NULL	''	' '
Meaning	unknown	Known, Empty Value	Known, Space Value
Data Type	Special Marker	String (0)	String (1 or more)
Storage	Very minimal	occupies memory	occupies memory (each space)
Performance	Best	Fast	Slow
Comparison	IS NULL	= ''	= ' '

Handling NULLs - Data Policies

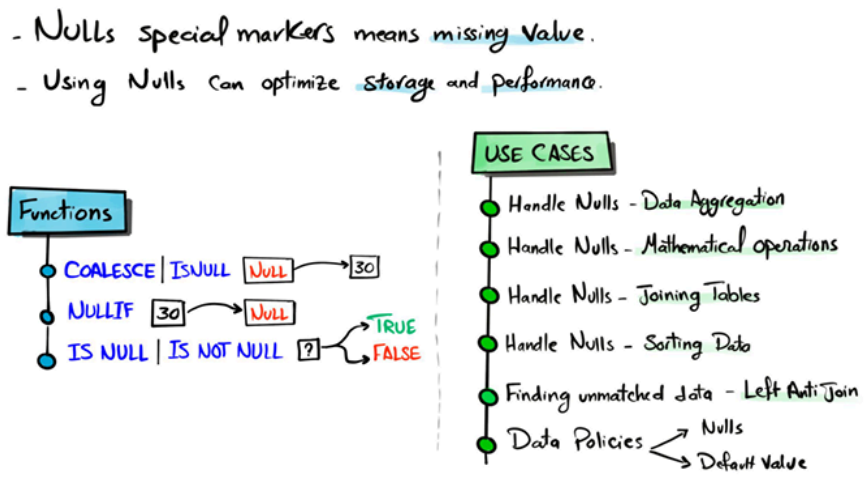
- Set of rules that defines how data should be handled

1. Only use NULLs and empty strings, but avoid blank spaces
   a. TRIM
      i. Remove unwanted leading and trailing spaces from a string
2. Only use NULLs and avoid using empty strings and blank spaces
   a. Replacing empty strings and blanks with NULL during data preparation before inserting into a database to optimize storage and performance
3. Use the default value 'unknown' and avoid using nulls, empty strings and blank spaces

a. Replacing empty strings, blanks and NULL with default value during data preparation before using it in reporting to improve readability and reduce confusion

```sql
WITH Orders AS (
SELECT 1 Id, 'A' Category UNION
SELECT 2, NULL UNION
SELECT 3, '' UNION
SELECT 4, ' '
)
SELECT
    *,
    DATALENGTH(Category) CategoryLen,
    TRIM(Category) Policy1,
    DATALENGTH(TRIM(Category)) PolicyLen1,
    NULLIF(TRIM(Category),'') Policy2,
    COALESCE(NULLIF(TRIM(Category),''),'unknown') Policy3
FROM Orders;
```

Summary

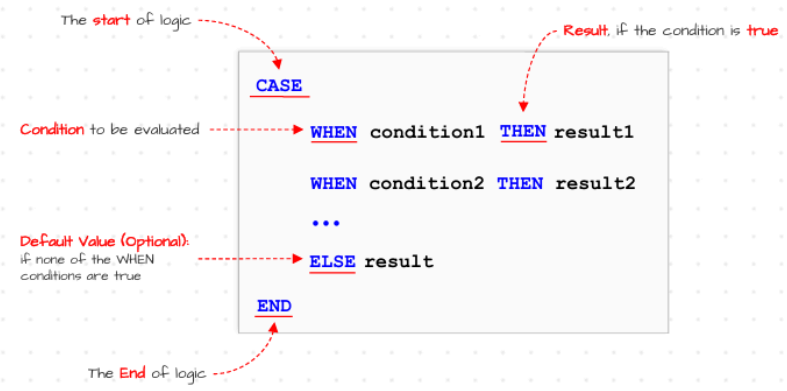


Module 12: CASE WHEN Statement

Introduction to CASE Statements

- Evaluates a list of conditions and returns a value when the first condition is met
- Main purpose → Data Transformation
    - Derive new information
        - Create new columns based on existing data
- Syntax

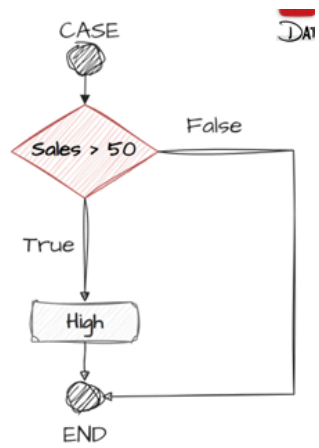
```sql
CASE
            WHEN condition1 THEN result1
            WHEN condition2 THEN result2
            ...
            ELSE default_result
END
```



How It Works?

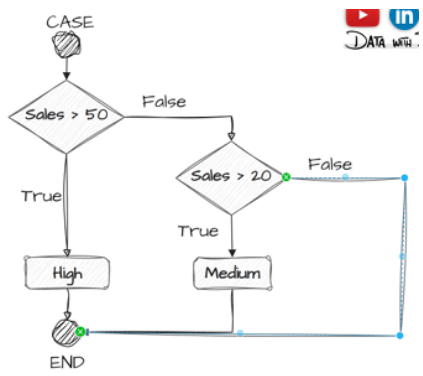
```
CASE
  WHEN Sales > 50 THEN 'High'
END
```

Sales	CASE
60	High
30	NULL
15	NULL
NULL	NULL



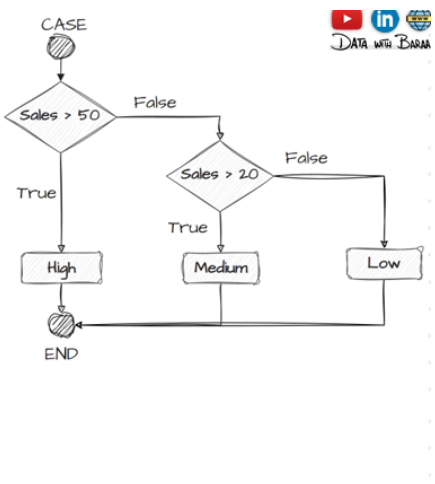

```
CASE
  WHEN Sales > 50 THEN 'High'
  WHEN Sales > 20 THEN 'Medium'
END
```

Sales	CASE
60	High
30	Medium
15	NULL
NULL	NULL




```
CASE
  WHEN Sales > 50 THEN 'High'
  WHEN Sales > 20 THEN 'Medium'
  ELSE 'Low'
END
```

Sales	CASE
60	High
30	Medium
15	Low
NULL	Low



Use Case – Categorizing Data

- Group the data into different categories based on certain conditions

```
SELECT
    Category,
    SUM(Sales) Total_Sales
FROM(
    SELECT
        OrderID,
        Sales,
        CASE
```

```
            WHEN Sales > 50 THEN 'High'
            WHEN Sales > 20 THEN 'Medium'
            ELSE 'Low'
        END Category
    FROM Sales.Orders
)t
GROUP BY Category
ORDER BY Total_Sales DESC;
```

CASE Rules

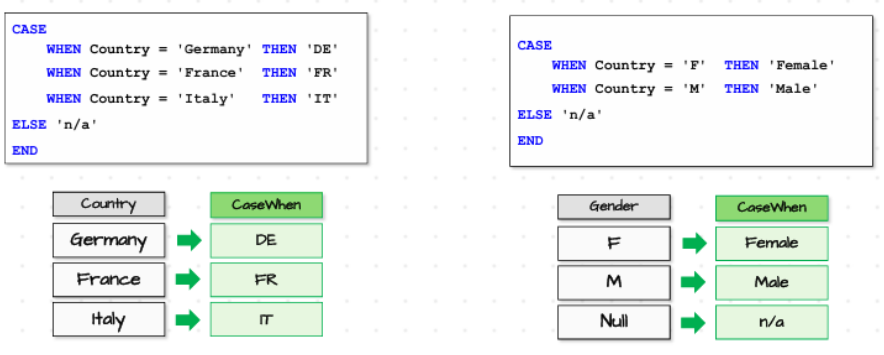
- The data type of the results must be matching
- CASE statement can be used anywhere in the query

Use Case – Mapping Values

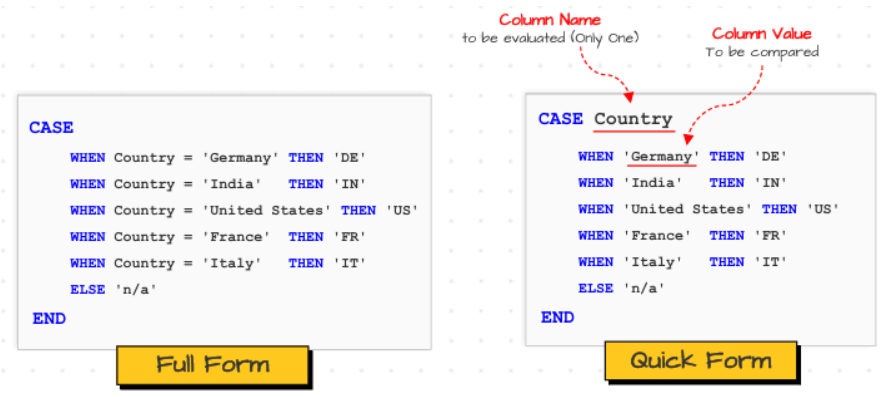
- Transforms the values from one form to another

```
SELECT
      [EmployeeID]
   ,[FirstName]
   ,[LastName]
   ,[Department]
   ,[BirthDate]
   ,CASE Gender
            WHEN 'M' THEN 'Male'
            WHEN 'F' THEN 'Female'
            ELSE 'Not Available'
      END Gender
   ,[Salary]
   ,[ManagerID]
FROM Sales.Employees;
```

```
SELECT
    CustomerID,
    FirstName,
    LastName,
    CASE Country
        WHEN 'Germany' THEN 'DE'
        WHEN 'USA' THEN 'US'
        ELSE 'n/a'
    END Country
FROM Sales.Customers;
```



Quick Form



Use Case – Handling NULLs

- Replace NULLs with a specified value
  - NULLs can lead to inaccurate results, which can lead to wrong decision-making

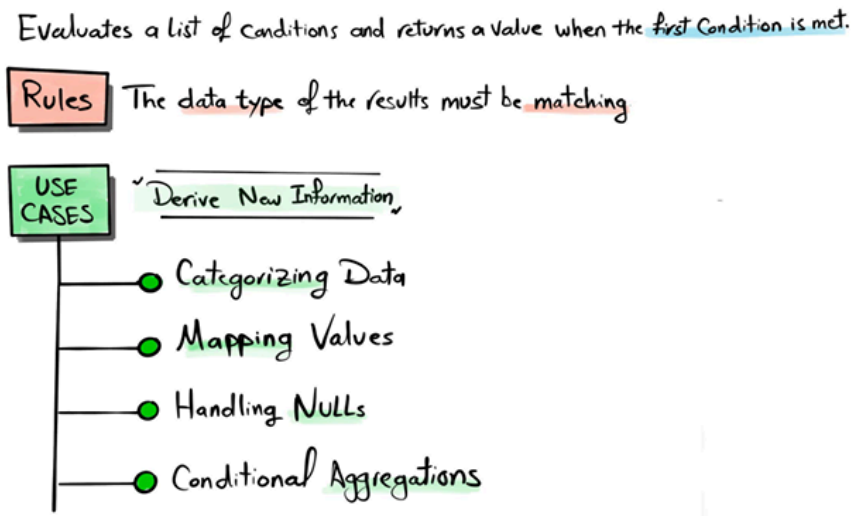
```sql
SELECT
    CustomerID,
    FirstName,
    LastName,
    Score,
    CASE
        WHEN Score IS NULL THEN 0
        ELSE Score
    END ScoreClean,
    AVG(Score) OVER () AvgScore,
    AVG(
        CASE
            WHEN Score IS NULL THEN 0
            ELSE Score
        END
    ) OVER () AvgScoreClean
FROM Sales.Customers;
```

Use Case – Conditional Aggregation

- Apply aggregate functions only on subsets of data that fulfill certain conditions

```sql
SELECT
    CustomerId,
    SUM(Flag) TotalOrdersHigherThan30,
    COUNT(*) TotalOrders
FROM(
    SELECT
        CustomerID,
        CASE
            WHEN Sales > 30 THEN 1
            ELSE 0
        END Flag
    FROM Sales.Orders
)t
GROUP BY CustomerID;
```

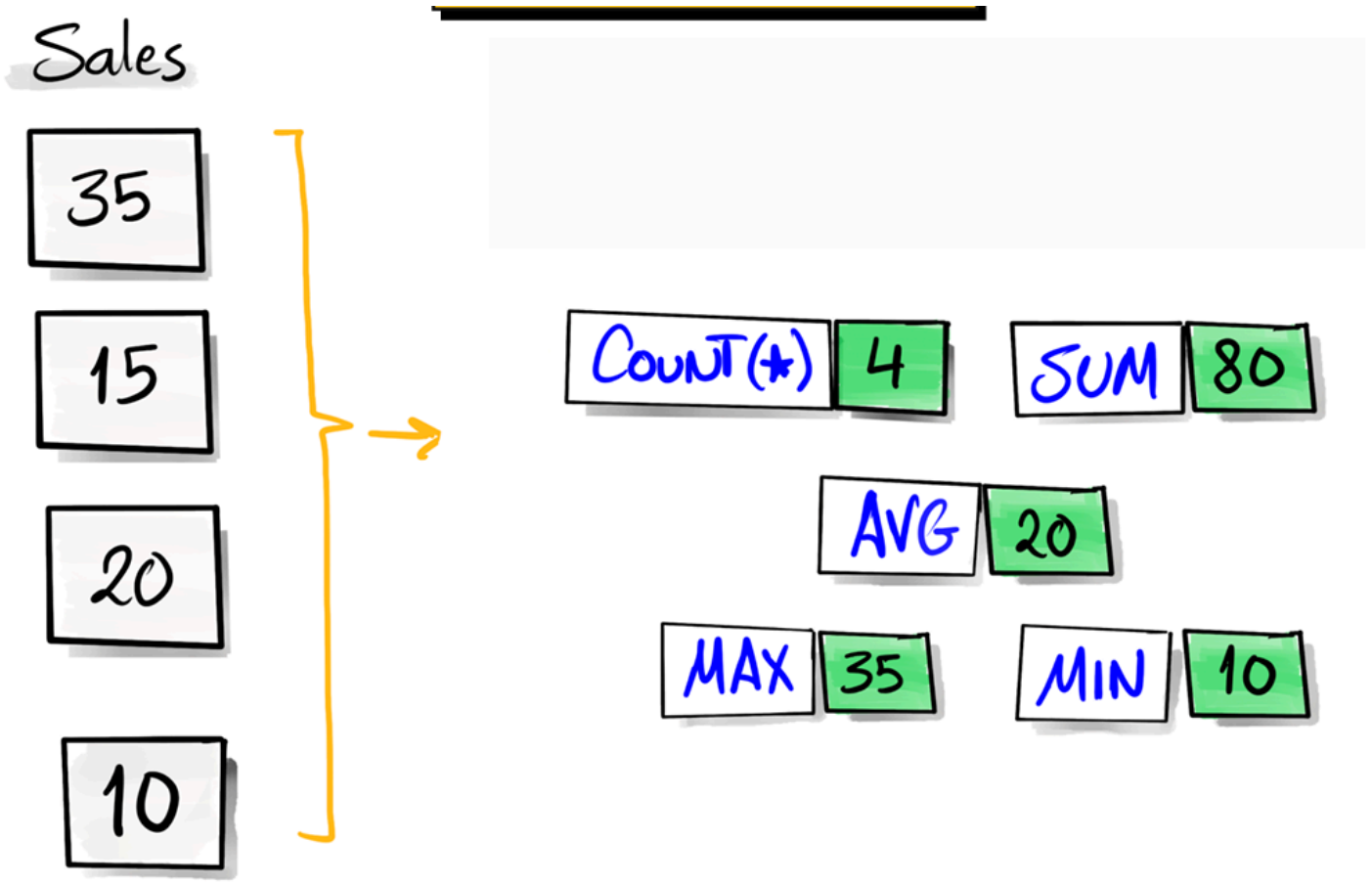
Summary



Module 13: Window Functions Basics

Aggregate Functions

Data Types		
Any Types	COUNT	Counts the number of rows
Only Numbers	SUM	Add up all values in a column
	AVG	Find the average of values
Any Types	MAX	Gets the highest value
	MIN	Gets the lowest value



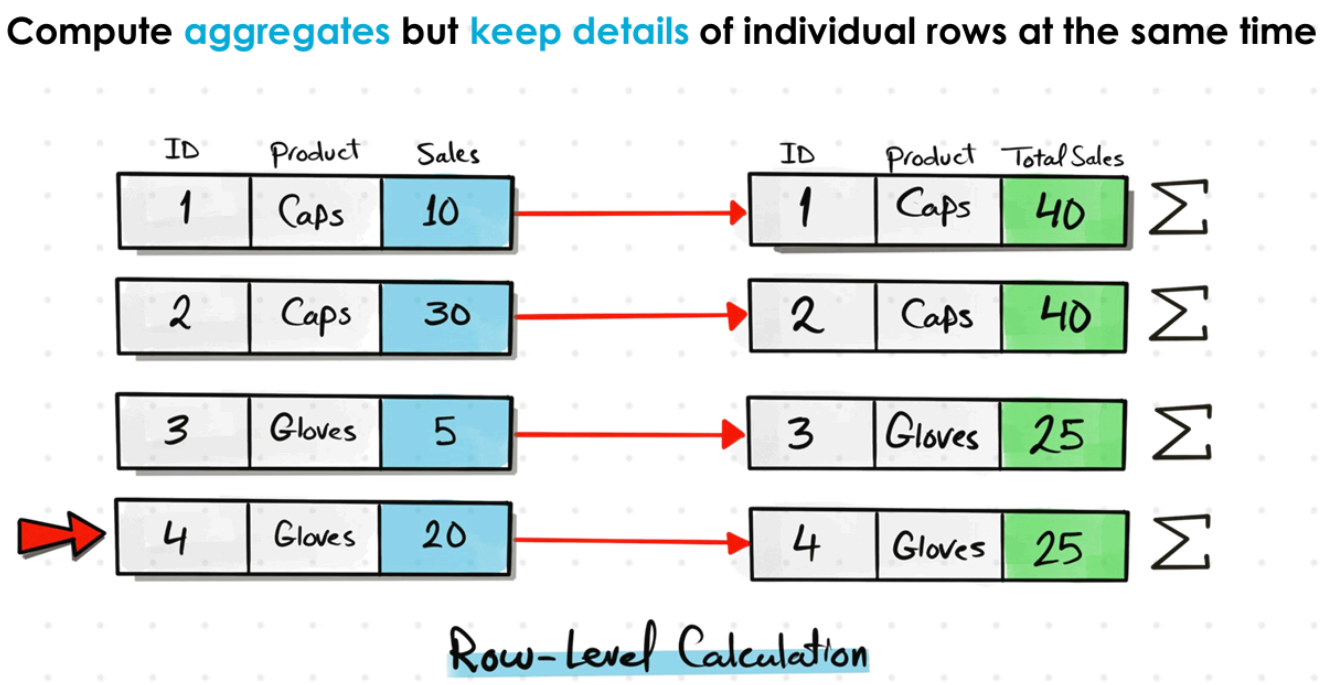

```sql
SELECT
    customer_id,
    COUNT(*) TotalNrOfOrders,
    SUM(sales) TotalSales,
    AVG(sales) AvgSales,
    MAX(sales) HighestSales,
    MIN(sales) LowestSales
FROM dbo.orders
GROUP BY customer_id;
```

```sql
SELECT
    country,
    COUNT(*) TotalNrOfCustomers,
    SUM(score) TotalScores,
    AVG(score) AvgScores,
    MAX(score) HighestScore,
    MIN(score) LowestScore
FROM dbo.customers
GROUP BY country;
```

WINDOW vs GROUP BY

Window Functions

- Perform calculations (aggregation) on a specific subset of data, without losing the level of details of rows
- The granularity stays the same
- Advanced Data Analysis (Aggregations + keep details)
- Result Granularity → Window functions returns a result for each row
- Flexibility of Window → Allows aggregation of data at different granularities within the same query

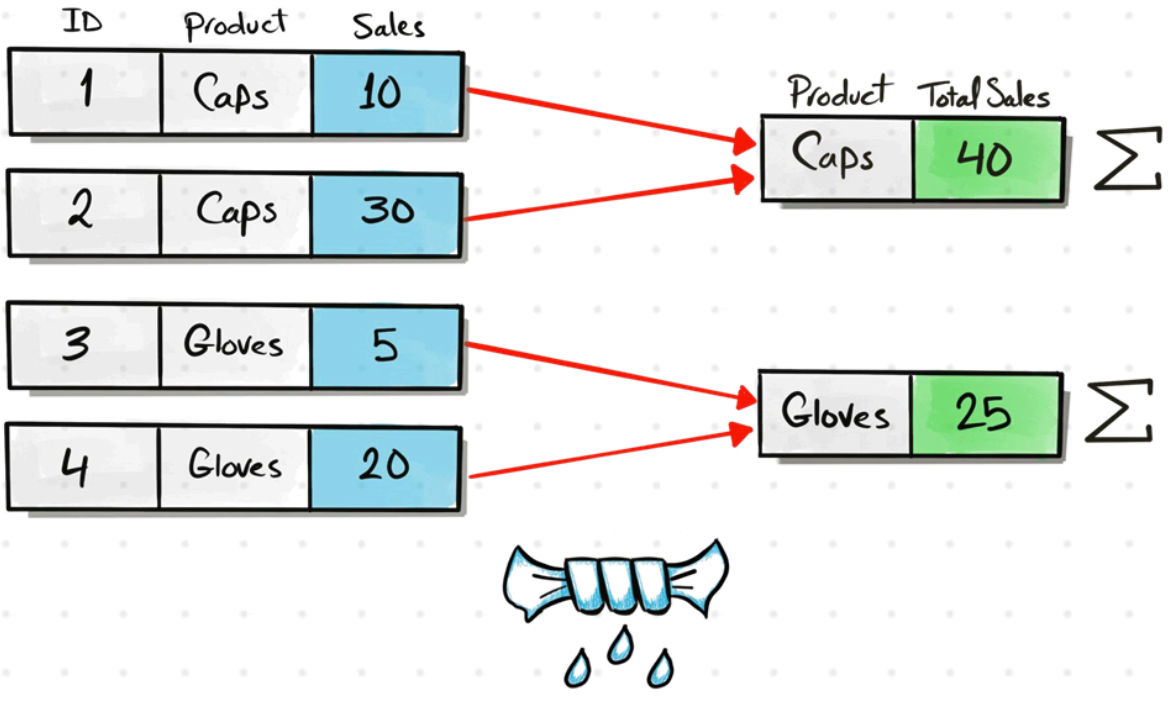



```sql
SELECT
    OrderID,
    OrderDate,
    ProductID,
    SUM(Sales) OVER(PARTITION BY ProductID) TotalSalesByProducts
FROM Sales.Orders;
```

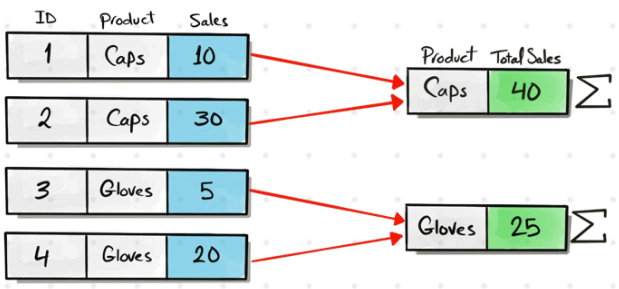
GROUP BY

- Changes the granularity
- Simple Data Analysis (Aggregations)
- Result Granularity → The number of rows in the output is defined by the dimension
- RULE → All columns in SELECT must be included in GROUP BY
- LIMITS → Can’t do aggregations and provide details at the same time

Aggregates and groups rows based on column/s into summary rows



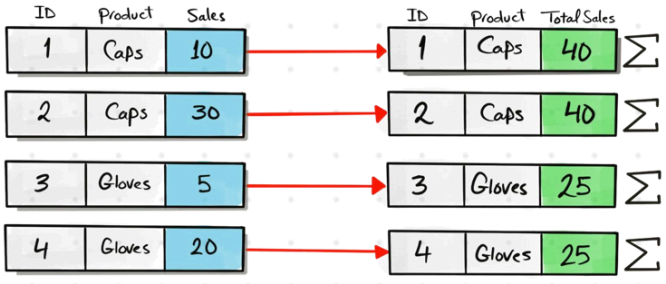
GROUP BY



It collapse groups of rows into a single row

(Group-Level-Calculations)

Window Functions



It doesn't collapse rows into a single row

(Row-Level-Calculations)

GROUP BY Functions

Aggregate Functions	COUNT (*expr*)
	SUM (*expr*)
	MAX (*expr*)
	MIN (*expr*)
	MIN (*expr*)

WINDOW Functions

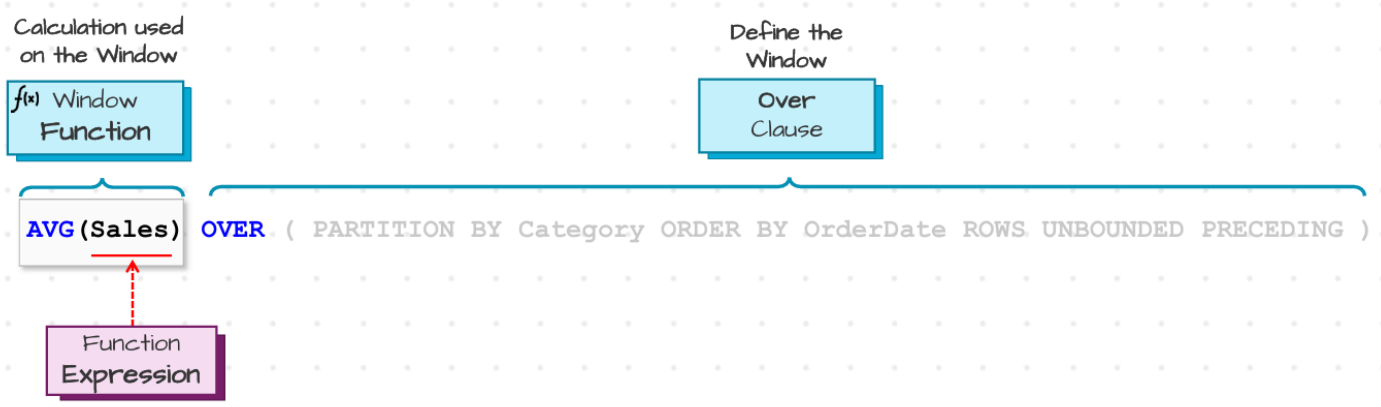
Aggregate Functions	COUNT (*expr*)
	SUM (*expr*)
	MAX (*expr*)
	MIN (*expr*)
	MIN (*expr*)
Rank Functions	ROW_NUMBER ()
	RANK ()
	DENSE_RANK ()
	CUME_DIST ()
	PERCENT_RANK ()
	NTILE (*n*)
Value (Analytics) Functions	LEAD (*expr,offset,default*)
	LAG (*expr,offset,default*)
	FIRST_VALUE (*expr*)
	FIRST_VALUE (*expr*)

ARAA

| Window Functions Basics | GROUP BY vs WINDOW

Syntax of Window Functions

```
AVG(Sales) OVER ( PARTITION BY Category ORDER BY OrderDate ROWS UNBOUNDED PRECEDING )
```

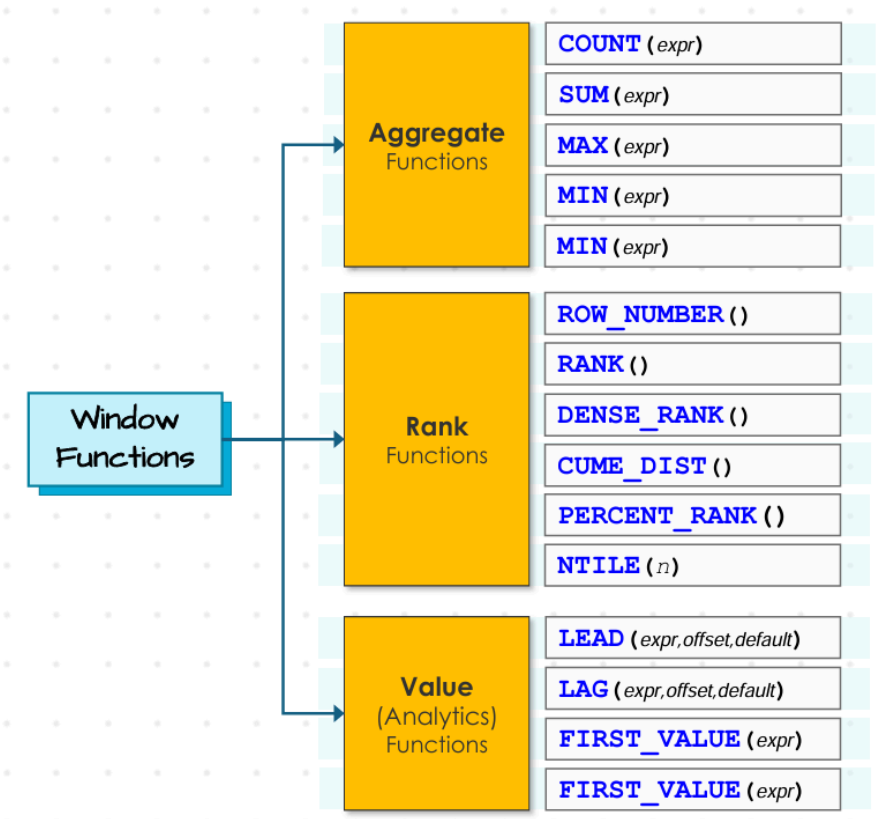


- Window Function
  - AVG(Sales)
  - Perform calculations within a window
  - Function Expression
    - Sales
    - Arguments you pass to a function
- Over Clause
  - OVER(....)
  - Tells SQL that the function used is a window function
  - It defines a window or subset of data

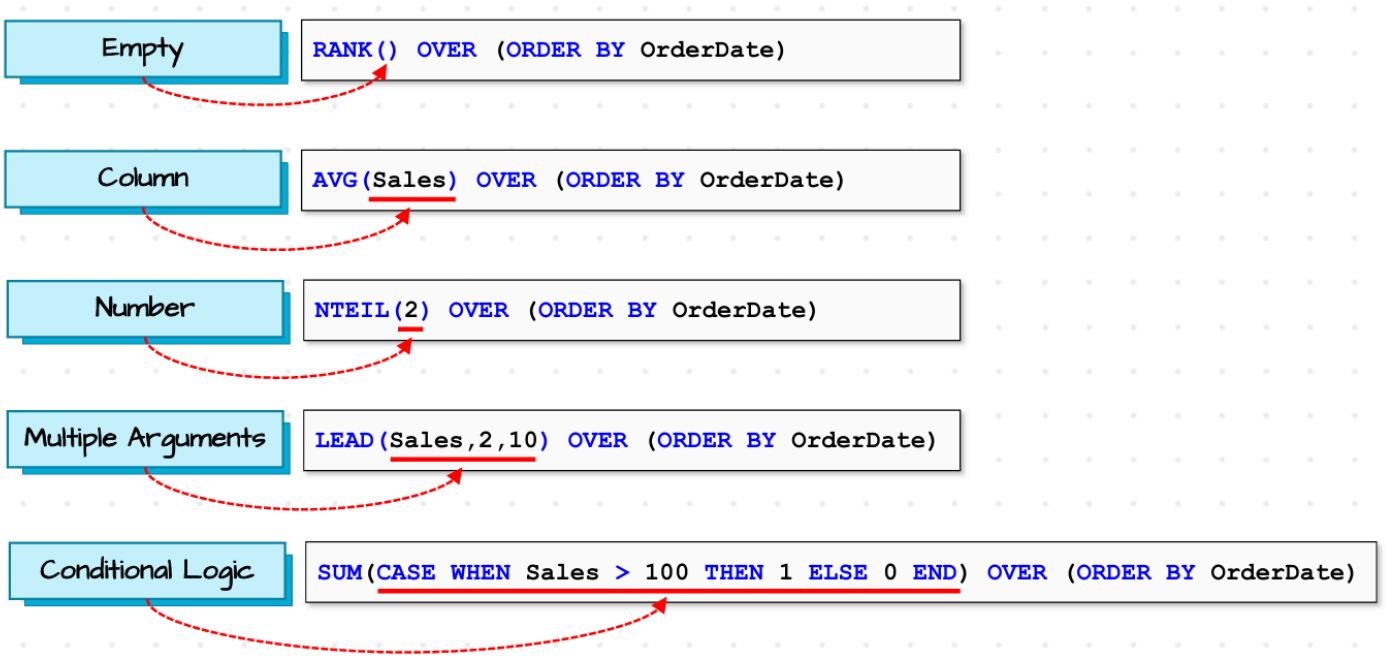
Types of Window Functions

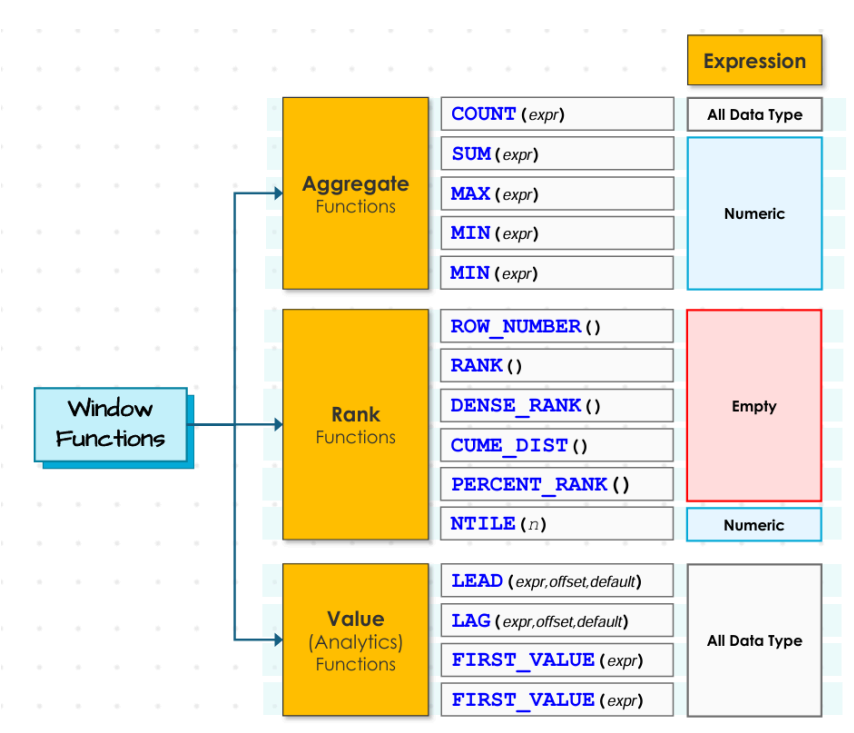
- Aggregate Functions
  - COUNT
  - SUM
  - AVG
  - MAX
  - MIN
- Rank Functions
  - ROW_NUMBER
  - RANK
  - DENSE_RANK
  - CUME_DIST
  - PERCENT_RANK
  - NTILE
- Value (Analytics) Functions
  - LEAD

- LAG
- FIRST_VALUE
- LAST_VALUE

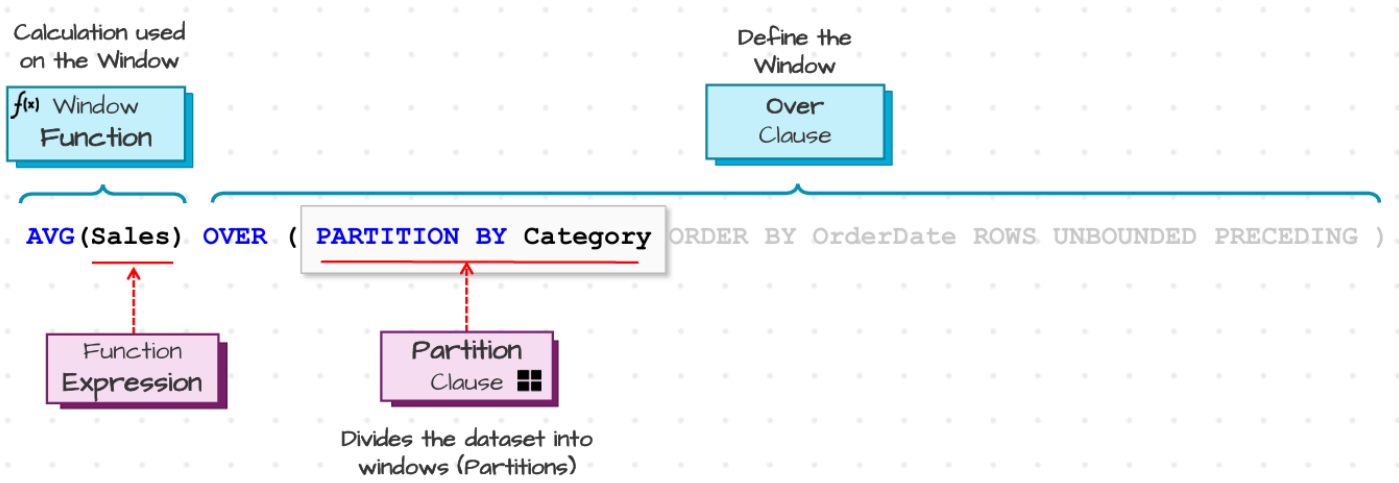


Window Expression





Window PARTITION BY



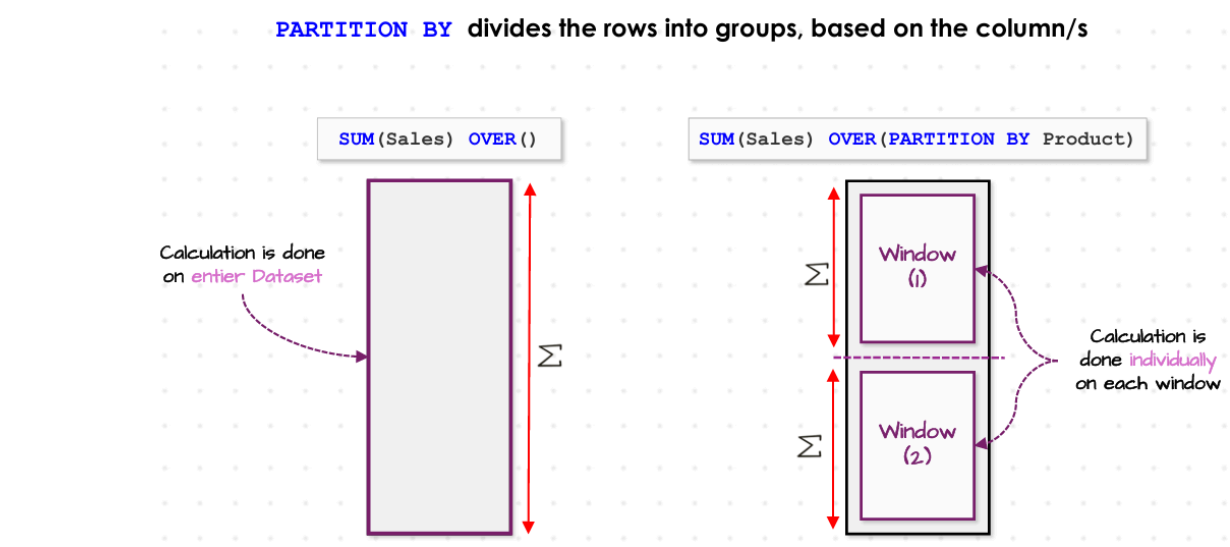
- Divides the result set into partitions (Windows)

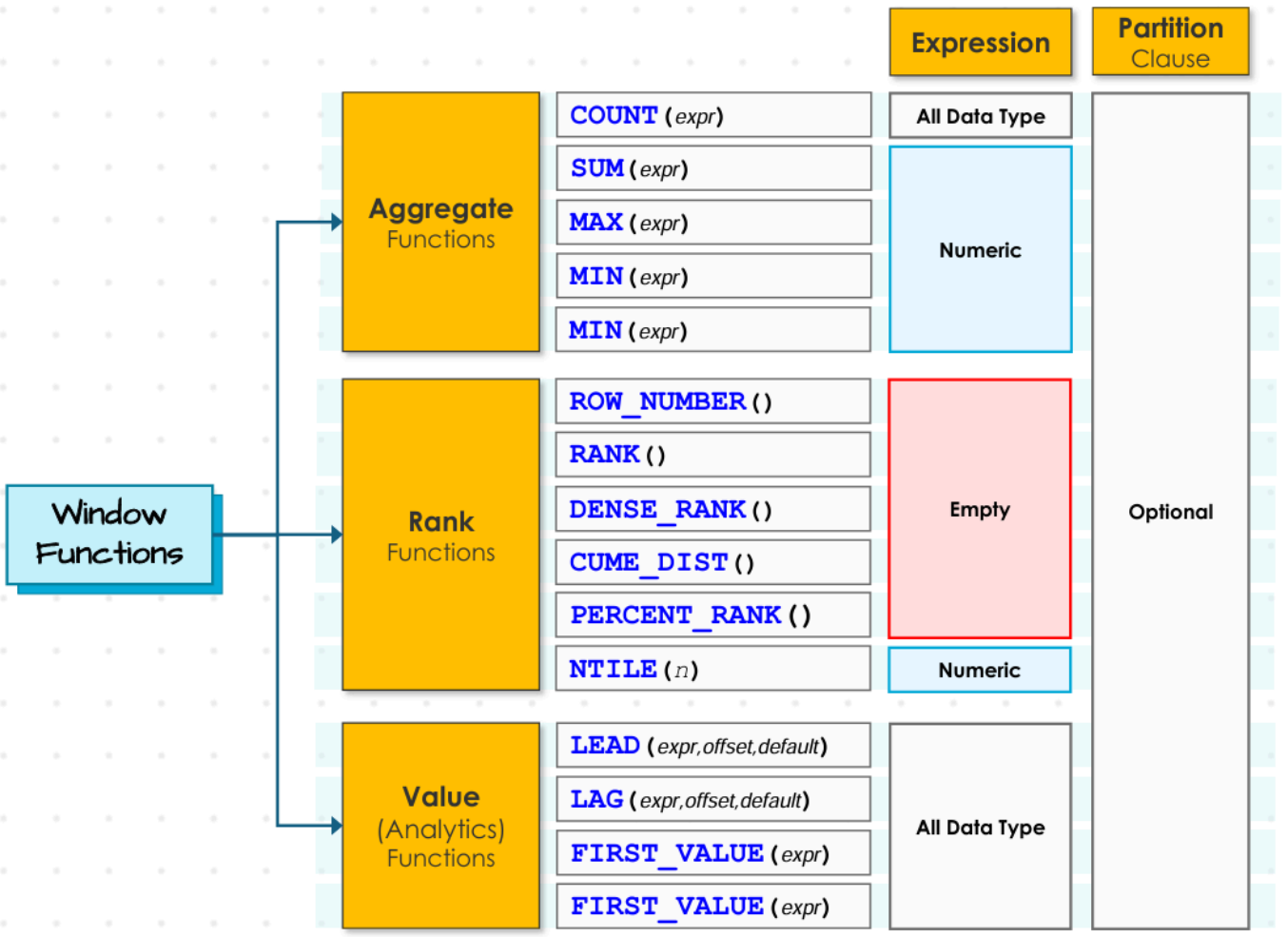
```
SUM(Sales) OVER()
```

- Calculation is done on entire dataset

```
SUM(Sales) OVER(PARTITION BY Product)
```

- Calculation is done individually on each window





- Without PARTITION BY

```sql
SELECT
    OrderID,
    OrderDate,
    SUM(Sales) OVER() TotalSales
FROM Sales.Orders;
```

- PARTITION BY Single Column

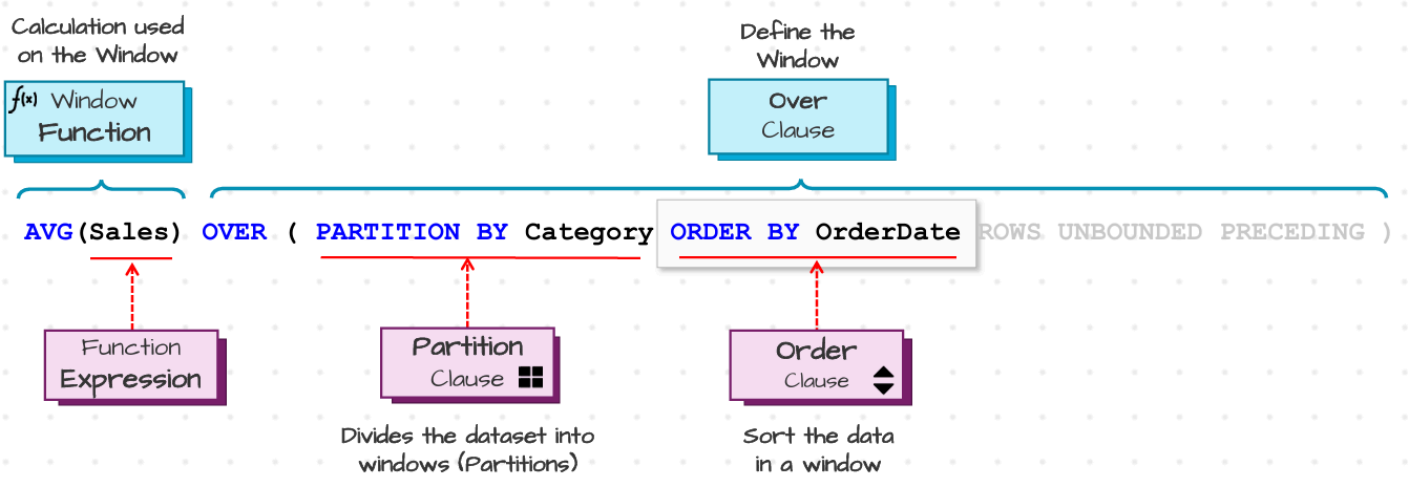
```sql
SELECT
    OrderID,
    OrderDate,
    ProductID,
    SUM(Sales) OVER(PARTITION BY ProductID) TotalSalesByProducts
FROM Sales.Orders;
```

- PARTITION BY Combined-Columns

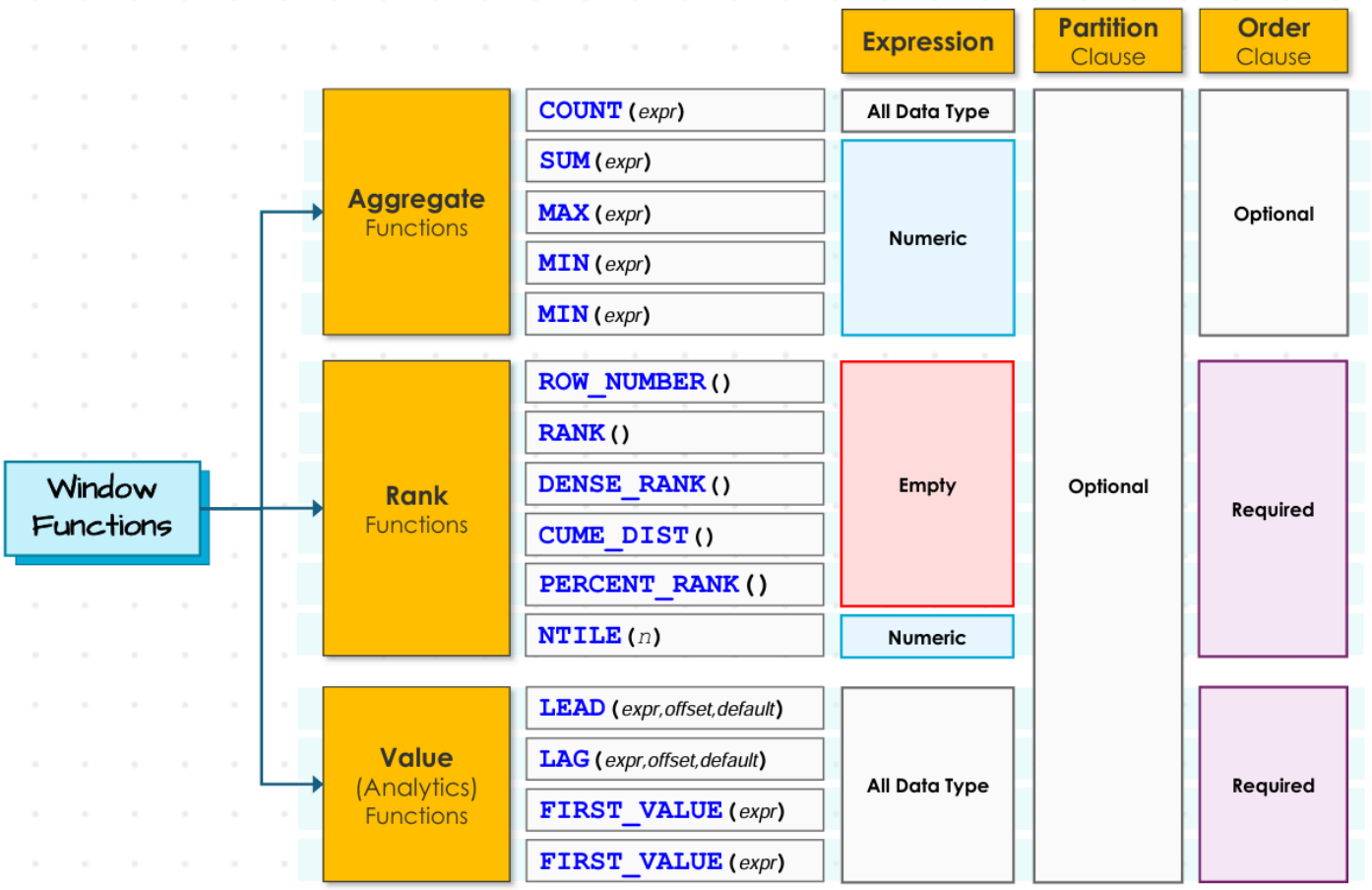
```sql
SELECT
    ProductID,
    OrderStatus,
    SUM(Sales) OVER(PARTITION BY ProductID, OrderStatus) SalesByProductsAndStatus
FROM Sales.Orders;
```

```sql
SELECT
    OrderID,
    OrderDate,
    ProductID,
    OrderStatus,
    Sales,
    SUM(Sales) OVER() TotalSales,
    SUM(Sales) OVER(PARTITION BY ProductID) SalesByProducts,
    SUM(Sales) OVER(PARTITION BY ProductID, OrderStatus) SalesByProductsAndStatus
FROM Sales.Orders;
```

Window ORDER BY

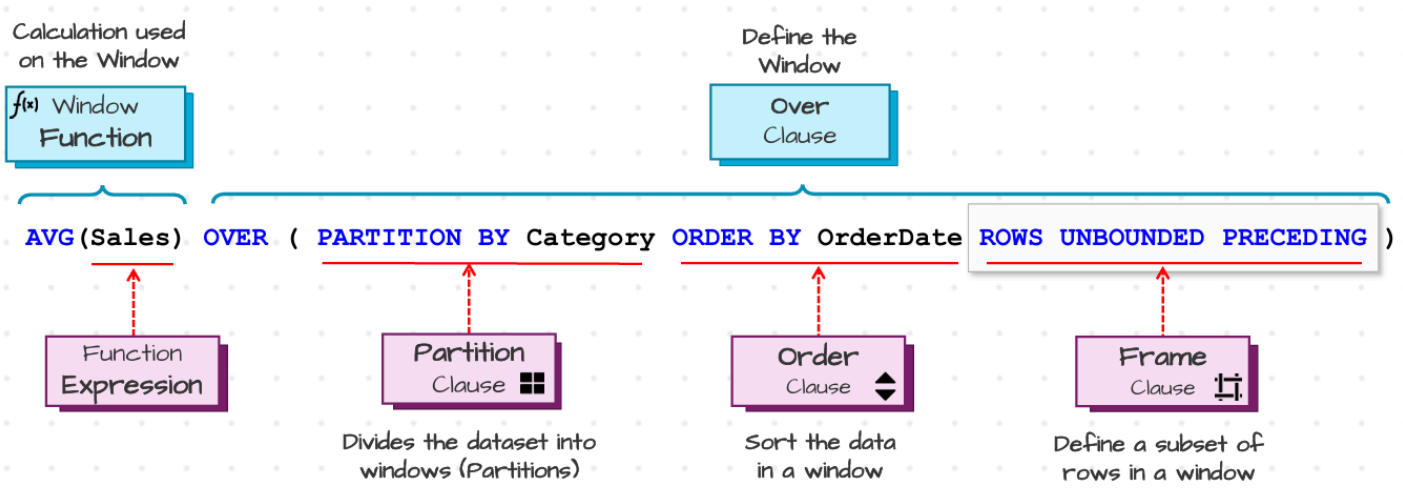


- Sort the data within a window
- Ascending | Descending
- Default Sorting → As default ORDER BY sorts the data in ascending order 'ASC' (from lowest to highest)

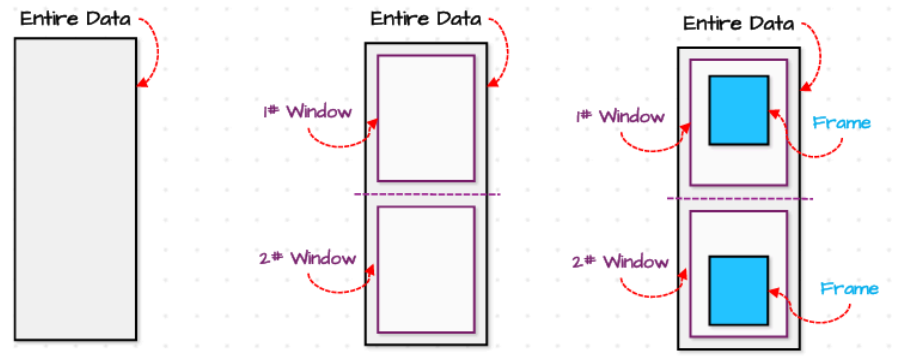


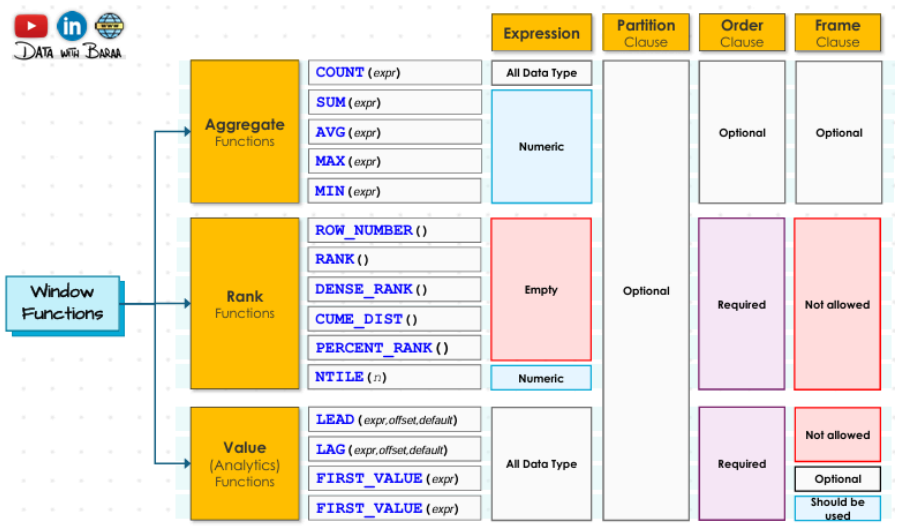

```
SELECT
    OrderID,
    OrderDate,
    Sales,
    RANK() OVER(ORDER BY Sales DESC) RankSales
FROM Sales.Orders;
```

Window Frame



- Defines a subset of rows within each window that is relevant for the calculation
- RULES
  - Frame Clause can only be used together with ORDER BY Clause
  - The Lower Value must be before the Higher Value
- NOTE
  - ORDER BY always uses a FRAME

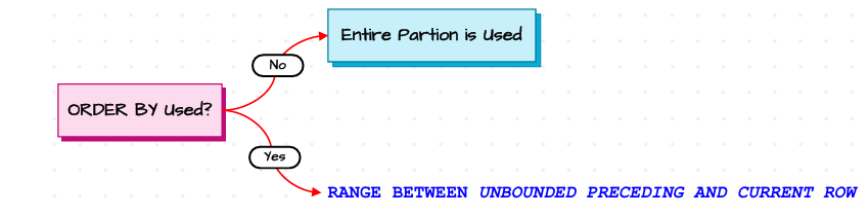


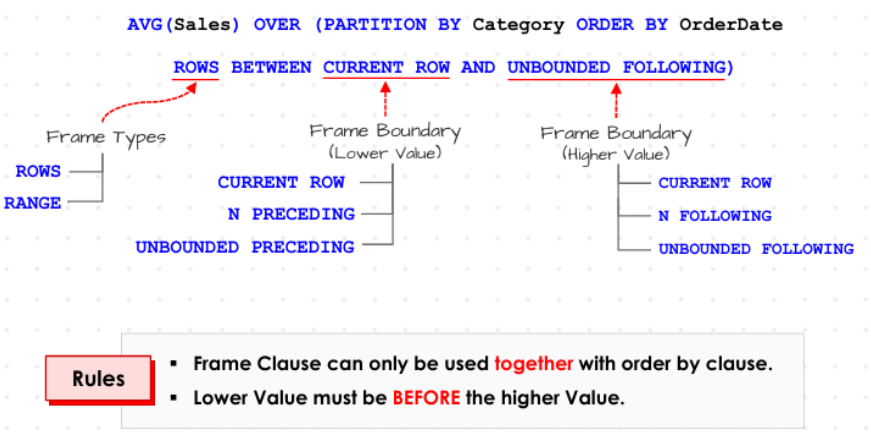


- Syntax
  - Frame Types
    - ROWS
    - RANGE
  - Frame Boundary (Lower Value)
    - CURRENT ROW
    - N PRECEDING → n-th row before the current row
    - UNBOUNDED PRECEDING → the first possible row within a window
  - Frame Boundary (Higher Value)
    - CURRENT ROW
    - N FOLLOWING → n-th row after the current row
    - UNBOUNDED FOLLOWING → the last possible row within a window

- Default Frame
  - SQL uses Default Frame, if ORDER BY is used without FRAME

```
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

ROWS UNBOUNDED PRECEDING
```






```
SELECT
    OrderID,
    OrderDate,
    OrderStatus,
    Sales,
    SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate
    ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING) TotalSales
FROM Sales.Orders;
```

COMPACT FRAME

For only PRECEDING, the CURRENT ROW can be skipped

- NORMAL FORM

```
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
```

- SHORT FORM

```
ROWS 2 PRECEDING
```

```
SELECT
    OrderID,
    OrderDate,
    OrderStatus,
    Sales,
    SUM(Sales) OVER (PARTITION BY OrderStatus ORDER BY OrderDate
    ROWS 2 PRECEDING) TotalSales
FROM Sales.Orders;
```

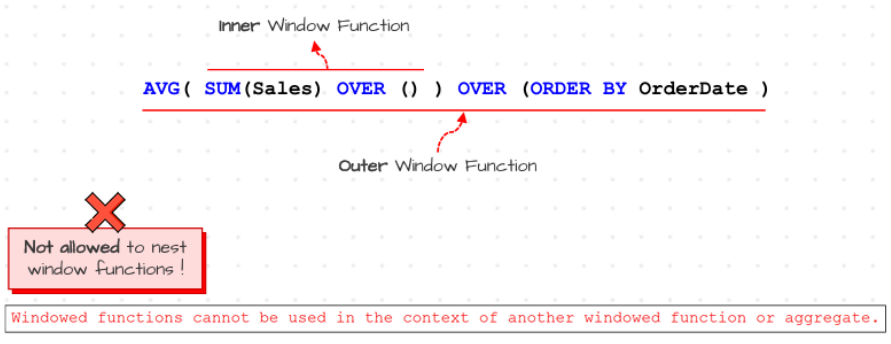
Window Functions Rules

1. Window Functions can be used ONLY in SELECT and ORDER BY Clauses
2. Nesting Window Functions is not allowed
3. SQL execute Window Functions after WHERE Clause
4. Window Function can be used together with GROUP BY in the same query, ONLY if the same columns are used

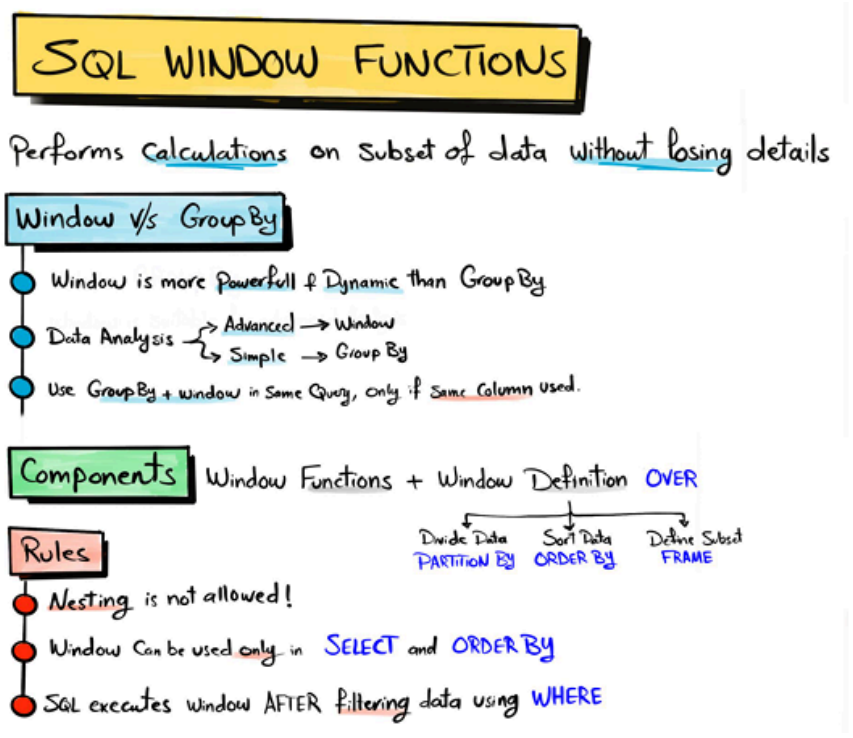
- NOTES
  - Window Functions can’t be used to filter data

```sql
-- 3
SELECT
    OrderID,
    OrderDate,
    OrderStatus,
    ProductID,
    Sales,
    SUM(Sales) OVER (PARTITION BY OrderStatus) TotalSales
FROM Sales.Orders
WHERE ProductID IN (101,102);
```

```sql
-- 4
SELECT
    CustomerID,
    SUM(Sales) TotalSales,
    RANK() OVER (ORDER BY SUM(Sales) DESC) RankCustomers
FROM Sales.Orders
GROUP BY CustomerID;
```



Summary



Module 14: Window Aggregate Functions

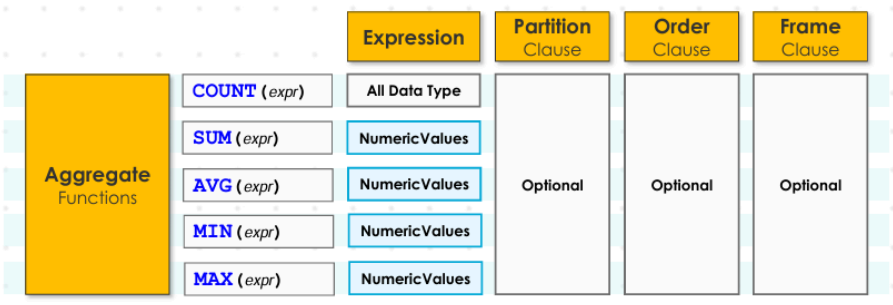
Introduction to Window Aggregate Functions

- Perform calculations on a set of rows and return a single aggregated value for each row
- Aggregate Functions
  - COUNT → Returns the number of Rows within a window
  - SUM → Returns the sum of values within a window
  - AVG → Returns the average of values within a window
  - MIN → Returns the minimum value within a window
  - MAX → Returns the maximum value within a window

Syntax

```
AVG(Sales) OVER (PARTITION BY ProdudtID ORDER BY Sales)
```

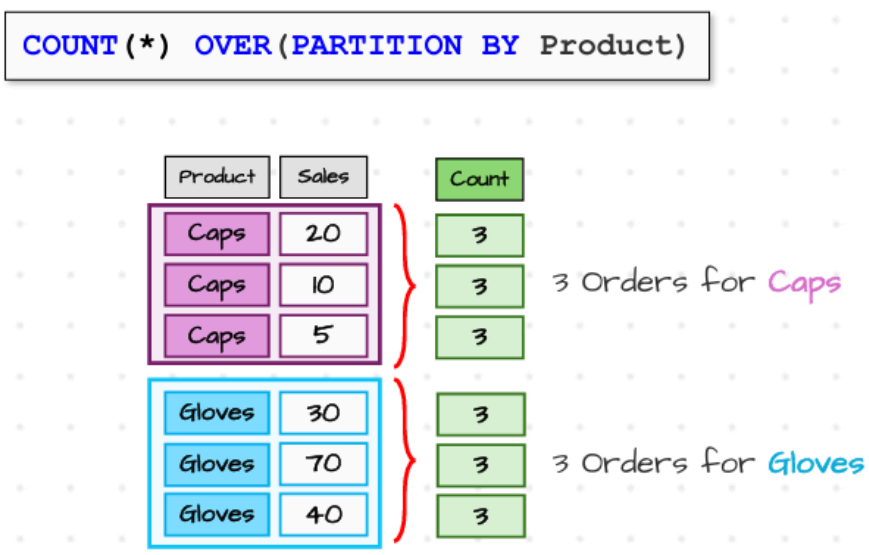
- Sales → Expression is required (Only Numeric Values)
- PARTITION BY → Partition By is optional
- ORDER BY → Order By is optional



COUNT

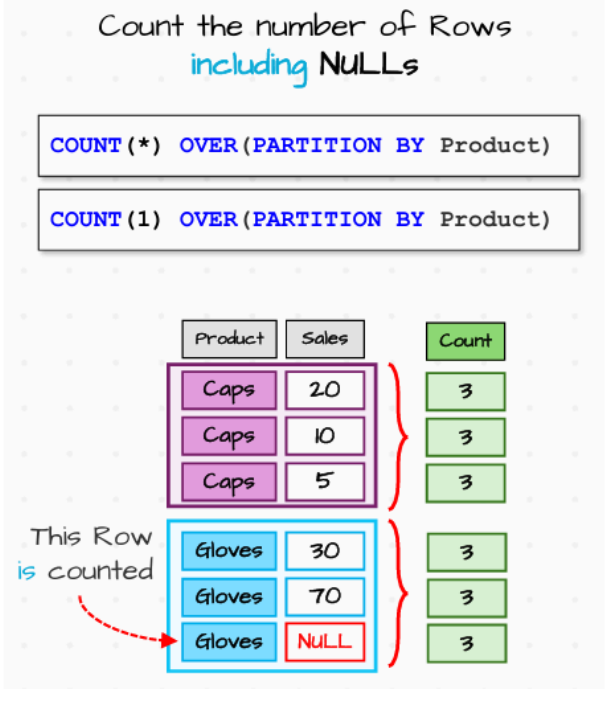
- Returns the number of Rows within a window

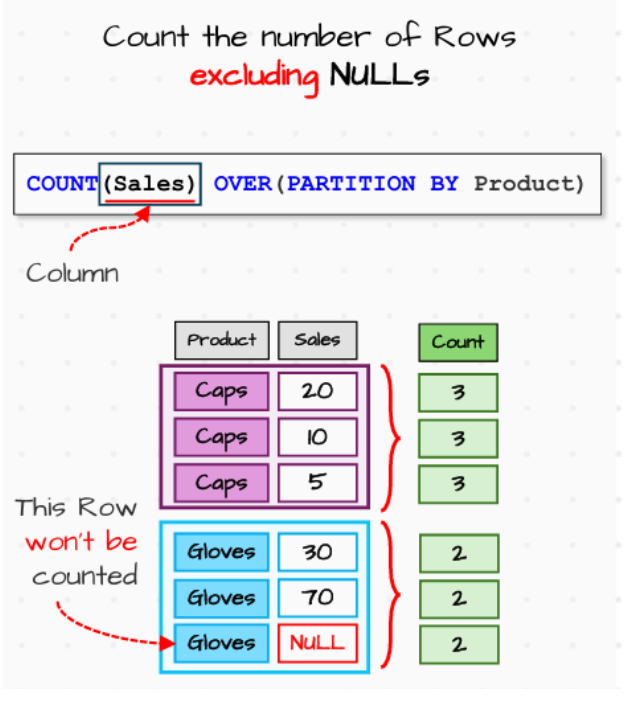
```
COUNT(*) OVER (PARTITION BY Product)
```



- COUNT()
  - Counts the number of values in a column, regardless of their data types
- NOTE
  - Counts the total number of rows, including duplicates, not the unique values
- COUNT(*) | COUNT(1)
  - Counts all the rows in a table, regardless of whether any value is NULL
  - COUNT(1) equals to COUNT(*)

- COUNT(column)
  - Counts the number of non-NULL values in the column





- DATA QUALITY ISSUE
  - Duplicates leads to inaccuracies in analysis
  - COUNT() can be used to identify duplicates
- USE CASES
  1. OVERALL ANALYSIS
     a. Quick summary or snapshot of the entire dataset
  2. TOTAL PER GROUPS
     a. Group-wise analysis, to understand patterns within different categories
  3. QUALITY CHECK: IDENTIFY NULLS
     a. Detecting numbers of NULLS by comparing to the total number of rows
  4. QUALITY CHECK: IDENTIFY DUPLICATES
     a. Identify duplicate rows to improve data quality

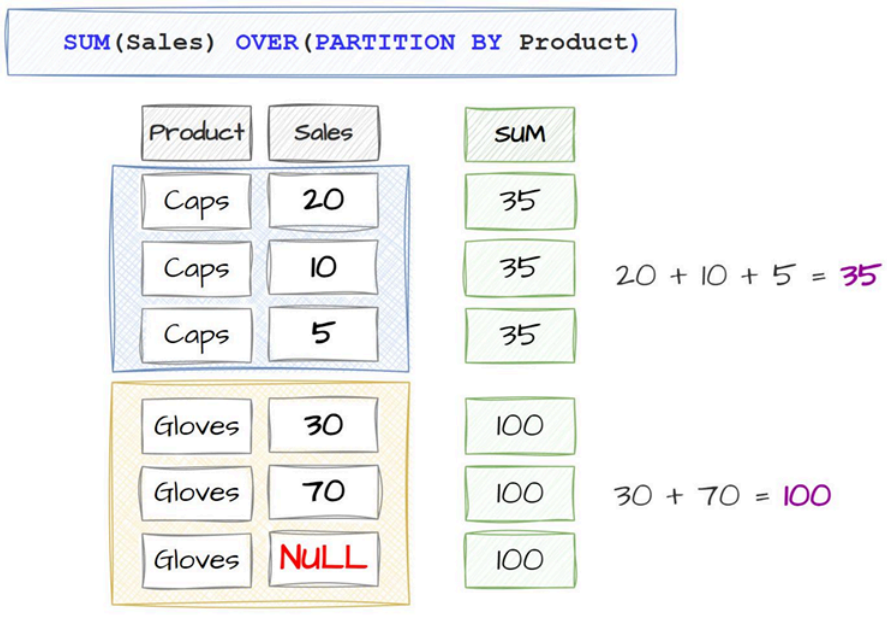
```sql
-- 1
SELECT
    OrderID,
    OrderDate,
    COUNT(*) OVER() TotalNrOfOrders
FROM Sales.Orders;

-- 2
SELECT
    OrderID,
    OrderDate,
    CustomerID,
    COUNT(*) OVER (PARTITION BY CustomerID) OrdersByCustomers
FROM Sales.Orders;

-- 3
SELECT
    *,
    COUNT(*) OVER () TotalNrOfCustomers,
    COUNT(Score) OVER () TotalNrOfScores
FROM Sales.Customers;
```

```sql
-- 4
SELECT
    *
FROM (
    SELECT
        OrderID,
        COUNT(*) OVER (PARTITION BY OrderID) CheckPK
    FROM Sales.OrdersArchive
)t
WHERE CheckPK > 1;
```

SUM

- Returns the sum of values within a window
- RULE
  - SUM() accepts only numbers

```sql
SUM(Sales) OVER (PARTITION BY Product)
```



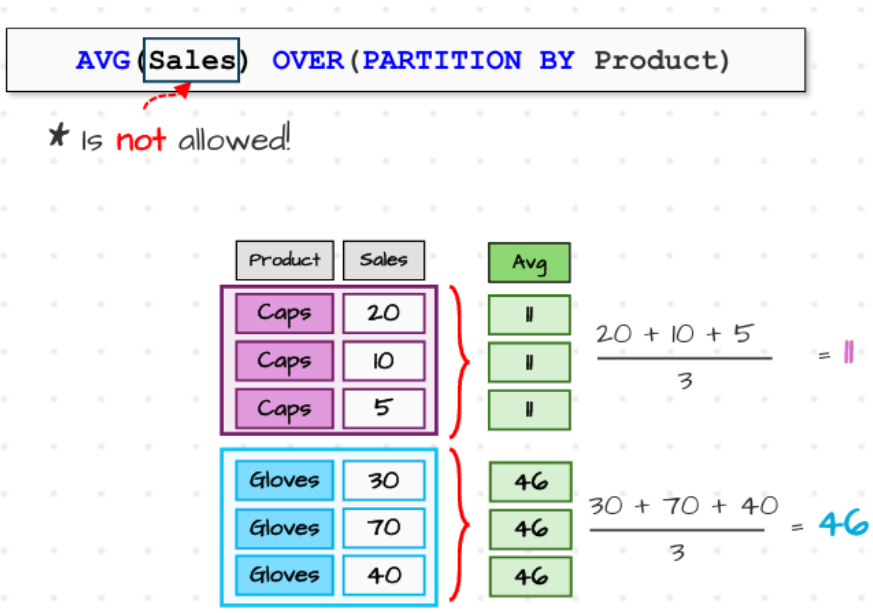
- USE CASES
  1. OVERALL ANALYSIS
     a. Quick summary or snapshot of the entire dataset
  2. TOTAL PER GROUPS
     a. Group-wise analysis, to understand patterns within different categories
  3. PART-TO-WHOLE
     a. Shows the contribution of each data point to the overall dataset

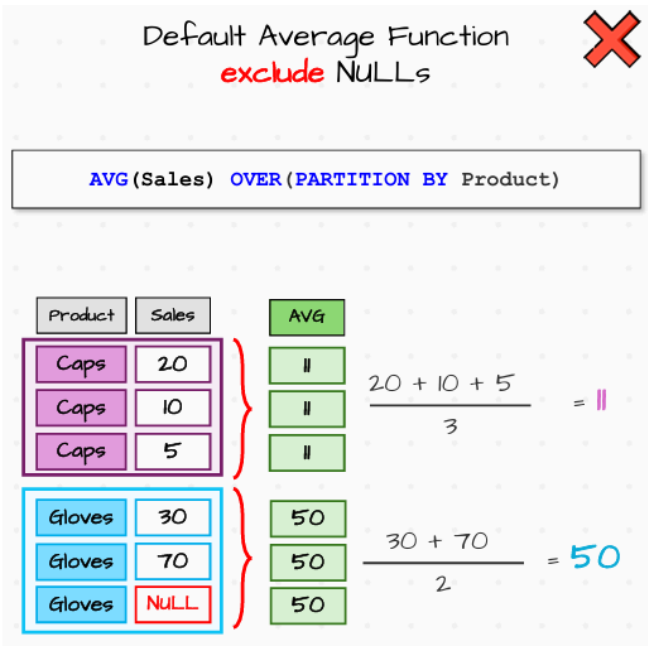
```sql
-- 1
SELECT
    OrderID,
    OrderDate,
    Sales,
    SUM(Sales) OVER () TotalSales,
FROM Sales.Orders;

-- 2
SELECT
    OrderID,
    OrderDate,
```

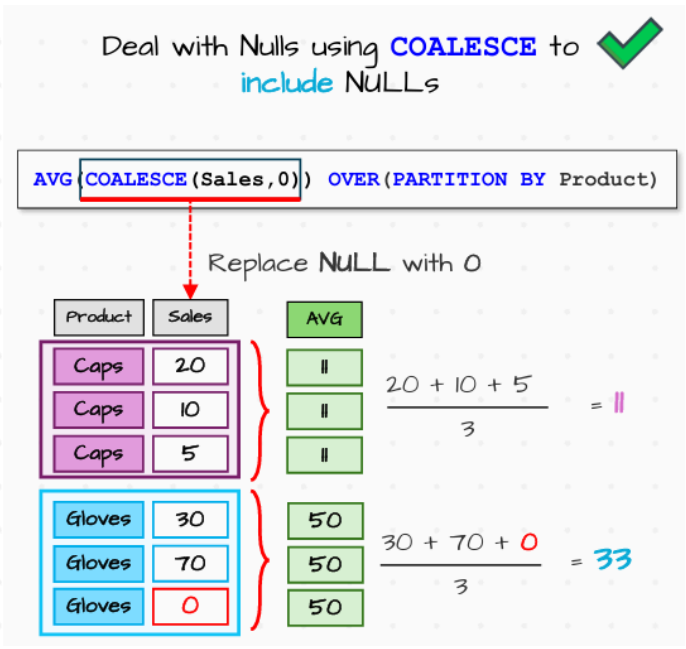
```
    Sales,
    ProductID,
    SUM(Sales) OVER (PARTITION BY ProductID) SalesByProduct
FROM Sales.Orders;

-- 3
SELECT
    OrderID,
    Sales,
    ProductID,
    SUM(Sales) OVER () TotalSales,
    SUM(Sales) OVER (PARTITION BY ProductID) SalesByProducts,
    ROUND((CAST(SUM(Sales) OVER (PARTITION BY ProductID) AS FLOAT) / SUM(Sales) OVER ()) * 10
0,2) PercentageOfTotal
FROM Sales.Orders;
```

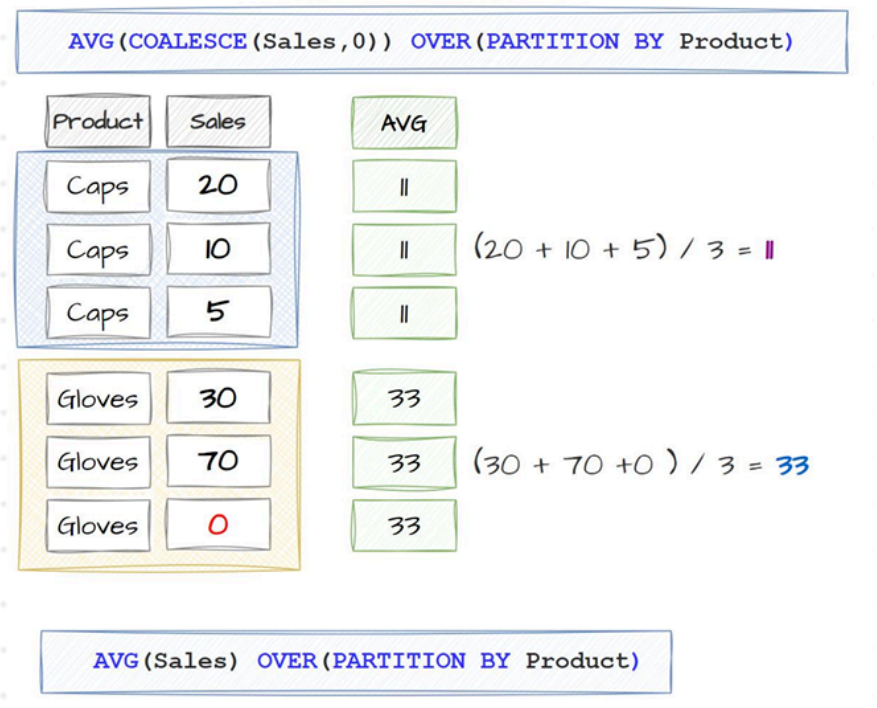
AVG

- Returns the average of values within a window









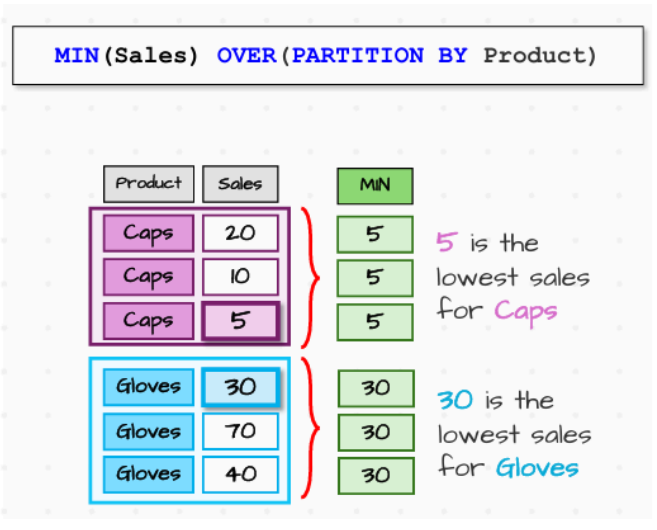
- USE CASES
    1. OVERALL ANALYSIS
        a. Quick summary or snapshot of the entire dataset
    2. TOTAL PER GROUPS
        a. Group-wise analysis, to understand patterns within different categories
    3. HANDLING NULLS
        a. Replaces NULLs to zero
    4. Compare to Average Analysis
        a. Help to evaluate whether a value is above or below the average

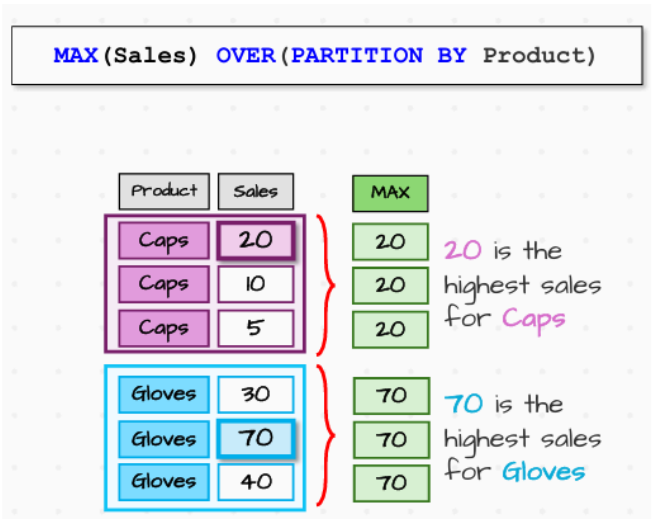
```sql
-- 1
SELECT
    OrderID,
    OrderDate,
    Sales,
    AVG(COALESCE(Sales,0)) OVER () AvgSales AvgSalesByProducts
FROM Sales.Orders;

-- 2
SELECT
    OrderID,
    OrderDate,
    Sales,
    ProductID,
    AVG(COALESCE(Sales,0)) OVER (PARTITION BY ProductID) AvgSalesByProducts
FROM Sales.Orders;

-- 3
SELECT
    CustomerID,
    LastName,
    Score,
    COALESCE(Score,0) UpdatedScore,
    AVG(Score) OVER () AvgScore,
    AVG(COALESCE(Score,0)) OVER () AvgScoreWithoutNulls
FROM Sales.Customers;

-- 4
SELECT
```

```
    *
FROM (
    SELECT
        OrderID,
        ProductID,
        Sales,
        AVG(COALESCE(Sales,0)) OVER () AvgSales
    FROM Sales.Orders
)t
WHERE Sales > AvgSales;
```

MIN / MAX

- MIN
  - Returns the minimum value within a window
- MAX
  - Returns the maximum value within a window





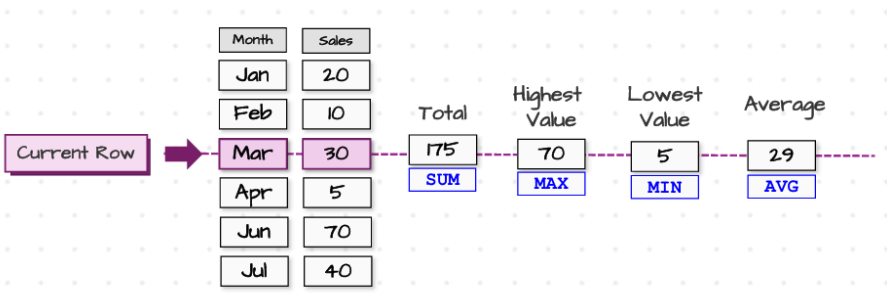
- USE CASES
  1. OVERALL ANALYSIS
     a. Quick summary or snapshot of the entire dataset
  2. TOTAL PER GROUPS
     a. Group-wise analysis, to understand patterns within different categories
  3. Compare to Extremes Analysis
     a. Compare current sales to highest or lowest sales
     b. Help to evaluate how well a value is performing relative to the extremes
     c. Distance from extreme
        i. The lower the deviation, the closer the data point is to the extreme

```
-- 1
SELECT
    OrderID,
    OrderDate,
    Sales,
    MIN(Sales) OVER () LowestSales,
    MAX(Sales) OVER () HighestSales
FROM Sales.Orders;

-- 2
SELECT
    OrderID,
```

```sql
    OrderDate,
    ProductID,
    Sales,
    MIN(Sales) OVER (PARTITION BY ProductID) LowestSalesByProducts,
    MAX(Sales) OVER (PARTITION BY ProductID) HighestSalesByProducts
FROM Sales.Orders;

-- 3
SELECT
    OrderID,
    OrderDate,
    ProductID,
    Sales,
    MIN(Sales) OVER () LowestSales,
    MAX(Sales) OVER () HighestSales,
    Sales - MIN(Sales) OVER () DeviationFromMin,
    MAX(Sales) OVER () - Sales DeviationFromMax
FROM Sales.Orders;
```

Comparison Use Cases

- Compare the current value and aggregated value of window functions
- Part-to-Whole Analysis
  - Compare current sales to total sales
- Compare to Extremes Analysis
  - Compare current sales to highest or lowest sales
- Compare to Average Analysis
  - Help to evaluate whether a value is above or below the average

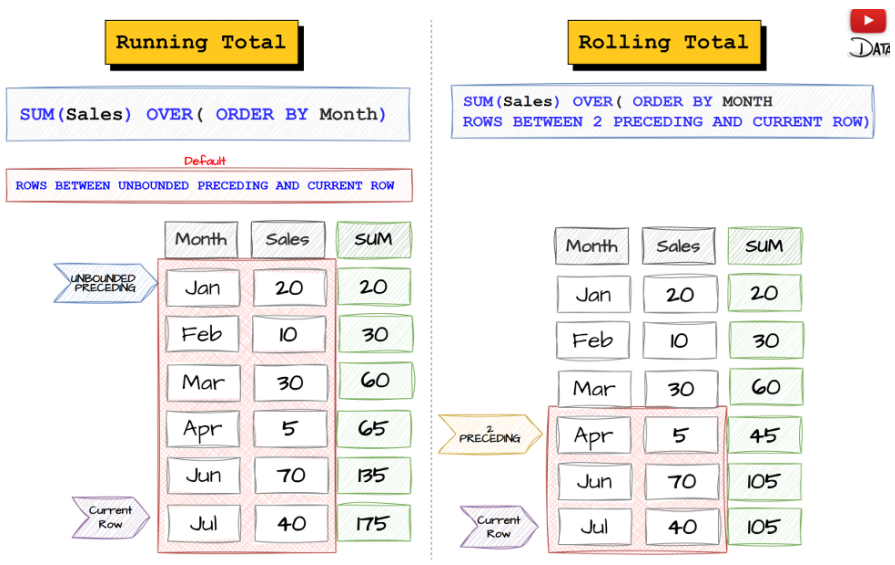


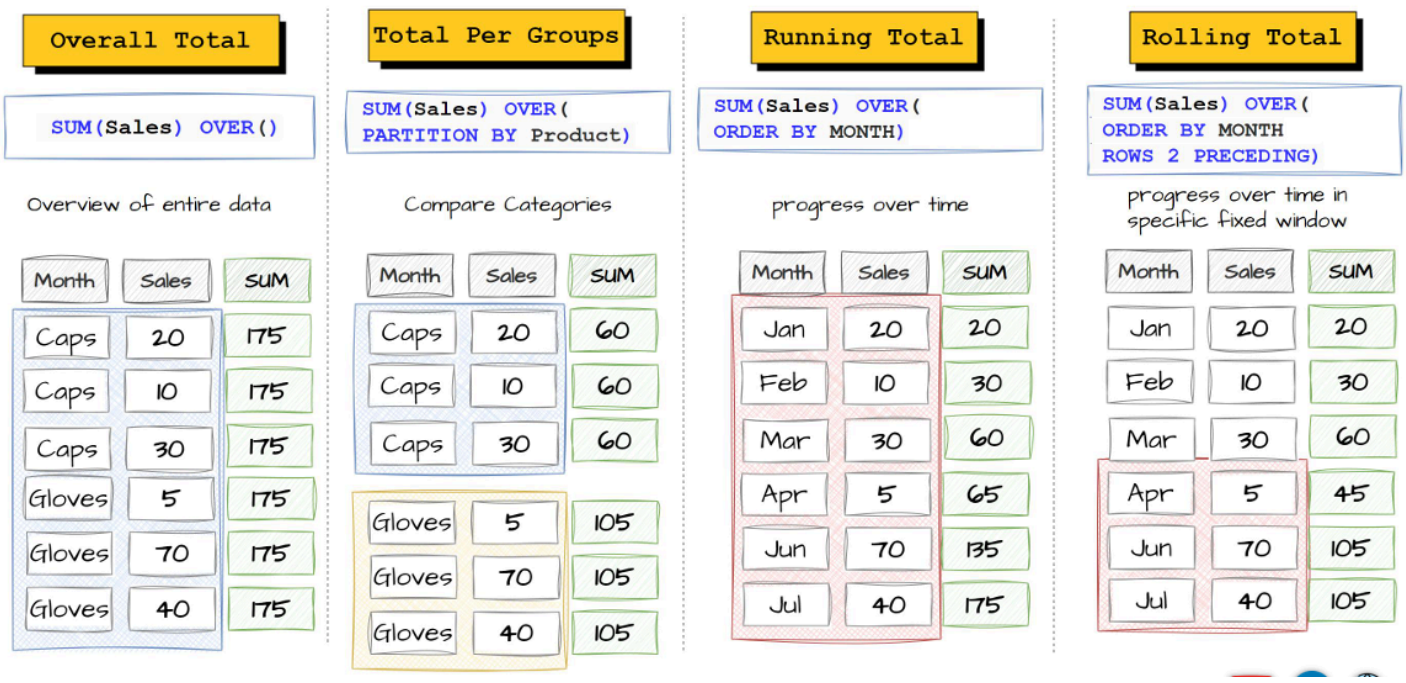
Use Case: Running & Rolling Total

- Concepts
  - Tracking
    - Tracking current sales with target sales
  - Trend Analysis
    - Providing insights into historical patterns
- They aggregate sequence of members, and the aggregation is updated each time a new member is added
- ANALYSIS OVER TIME
  - Sorting dates in ascending order
- Running Total
  - Aggregate all values from the beginning up to the current point without dropping off older data
  - Summarize all values from the first row up to the current row
- Rolling Total

- Aggregate all values within a fixed time window (e.g. 30 days)
- As new data is added, the oldest data point will be dropped
- Rolling / Shifting Window
- Summarize a fixed number of consecutive rows calculated within a moving window

```sql
-- Running Total
SUM(Sales) OVER (ORDER BY Month)

-- Rolling Total
SUM(Sales) OVER (ORDER BY Month
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
```





Use Case: Moving Average

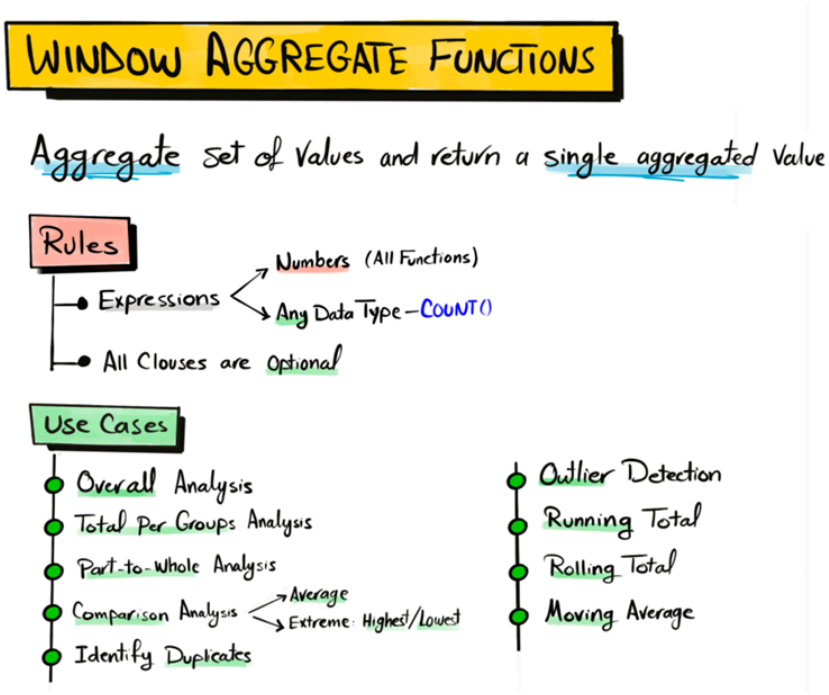
```sql
-- Calculate moving average of sales for each product over time
SELECT
    OrderID,
    ProductID,
    OrderDate,
    Sales,
    AVG(Sales) OVER (PARTITION BY ProductID) AvgSalesByProduct,
    AVG(Sales) OVER (PARTITION BY ProductID ORDER BY OrderDate) MovingAvgSalesByProduct
FROM Sales.Orders;
```

	OrderID	ProductID	OrderDate	Sales	AvgSalesByProduct	MovingAvgSalesByProduct
1	1	101	2025-01-01	10	35	10
2	3	101	2025-01-10	20	35	15
3	8	101	2025-02-18	90	35	40
4	9	101	2025-03-10	20	35	35
5	2	102	2025-01-05	15	35	15
6	7	102	2025-02-15	30	35	22
7	10	102	2025-03-15	60	35	35
8	5	104	2025-02-01	25	37	25
9	6	104	2025-02-05	50	37	37
10	4	105	2025-01-20	60	60	60

```sql
/* Calculate moving average of sales for each product
   over time, including only the next order */
SELECT
    OrderID,
    ProductID,
    OrderDate,
    Sales,
    AVG(Sales) OVER (PARTITION BY ProductID) AvgSalesByProduct,
    AVG(Sales) OVER (PARTITION BY ProductID ORDER BY OrderDate
    ROWS BETWEEN CURRENT ROW AND 1 FOLLOWING) MovingAvgSalesByProduct
FROM Sales.Orders;
```

	OrderID	ProductID	OrderDate	Sales	AvgSalesByProduct	MovingAvgSalesByProduct
1	1	101	2025-01-01	10	35	15
2	3	101	2025-01-10	20	35	55
3	8	101	2025-02-18	90	35	55
4	9	101	2025-03-10	20	35	20
5	2	102	2025-01-05	15	35	22
6	7	102	2025-02-15	30	35	45
7	10	102	2025-03-15	60	35	60
8	5	104	2025-02-01	25	37	37
9	6	104	2025-02-05	50	37	50
10	4	105	2025-01-20	60	60	60

Summary

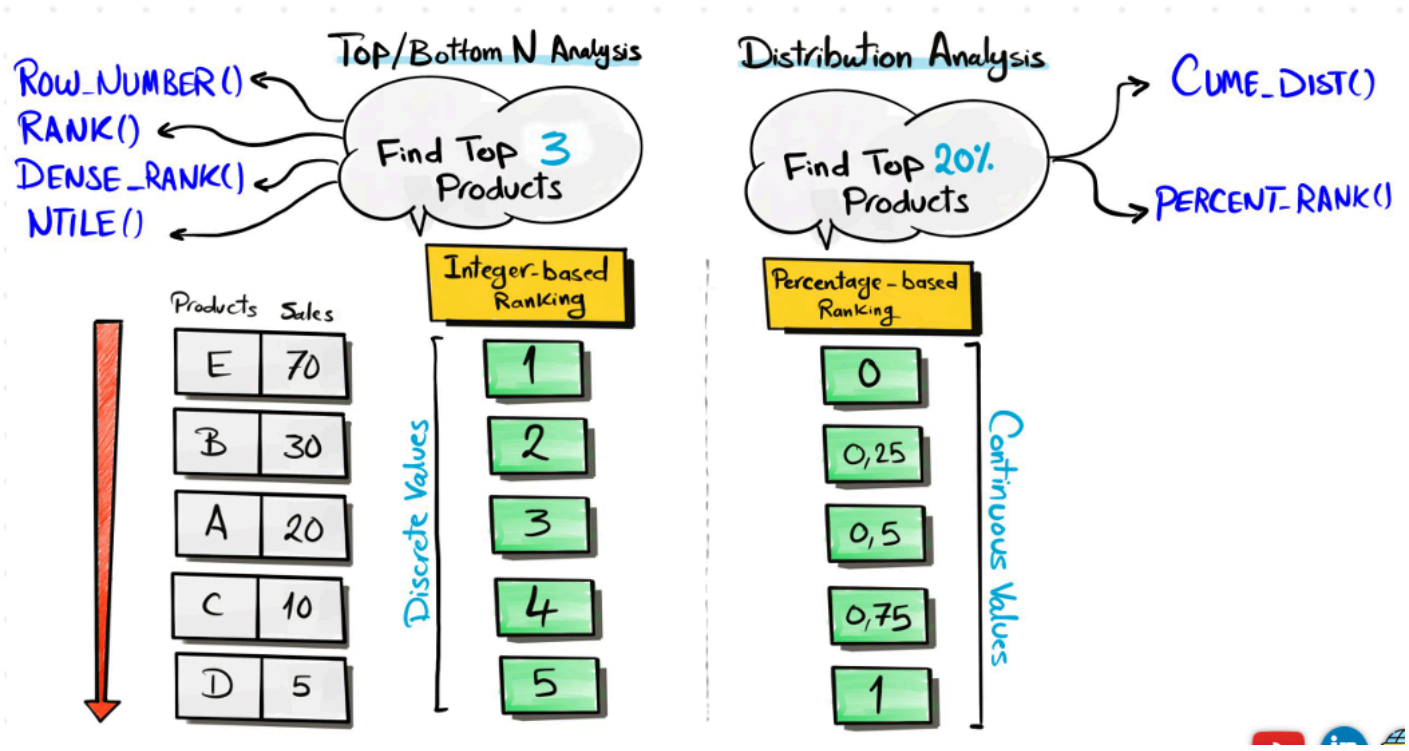


Module 15: Window Ranking Functions

Introduction to Ranking Window Functions

- Assign a rank to each row in a window
- Integer-based Ranking
  - Assign an integer to each row in a window
  - Discrete Values

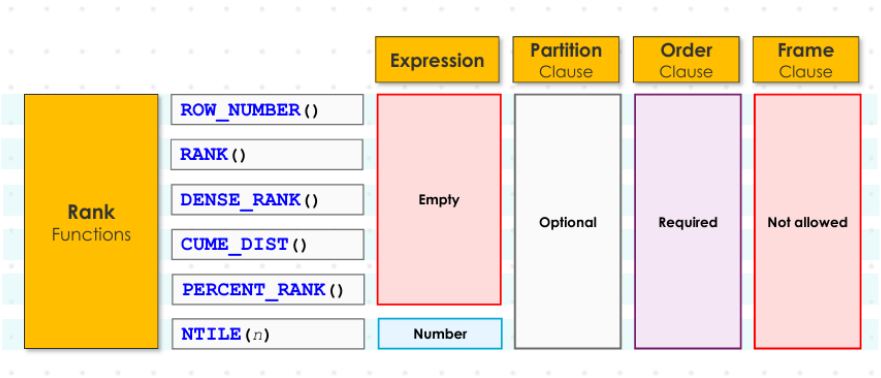
    - Find Top 3 Products
    - Top/Bottom N Analysis
- Percentage-based Ranking
    - Assign a percentage to each row in a window
    - Continuous Values
    - Find Top 20% Products
    - Distribution Analysis
- NOTE
    - SQL sorts data as first step
- Ranking Functions
    - Integer-based Ranking
        - ROW_NUMBER() → Assign a unique number to each in a window
        - RANK() → Assign a rank to each row in a window, with gaps
        - DENSE_RANK() → Assign a rank to each row in a window, without gaps
        - NTILE(n) → Divides the rows into a specified number of approximately equal groups
    - Percentage-based Ranking
        - CUME_DIST() → Calculates the cumulative distribution of a value within a set of values
        - PERCENT_RANK() → Returns the percentile ranking number of a row



Syntax

```
RANK() OVER (PARTITION BY ProductID ORDER BY Sales)
```

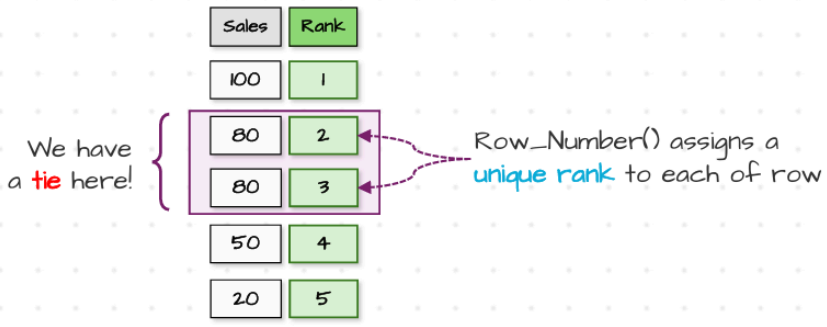
- ( ) → Expression must be empty
- PARTITION BY → Partition By is optional
- ORDER BY → Order By is required



ROW_NUMBER

- Assign a unique number to each in a window
- It doesn't handle ties
- Unique Ranking without gaps/skipping

```
ROW_NUMBER() OVER (ORDER BY Sales DESC)
```

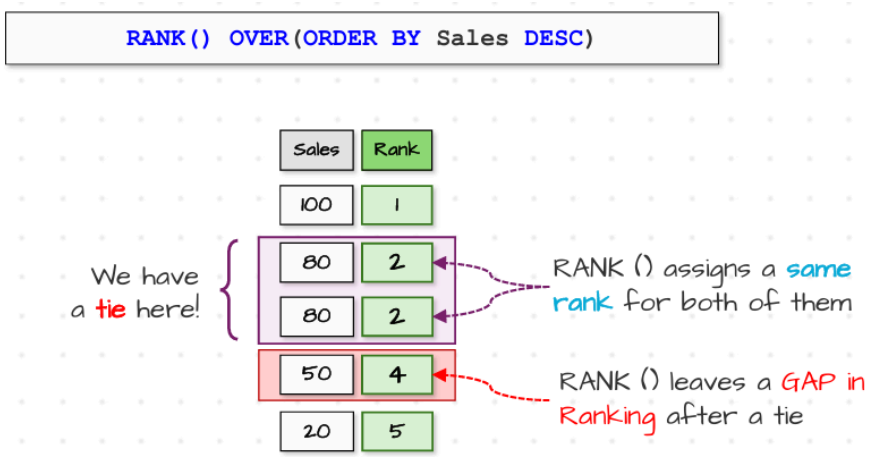



```sql
SELECT
    OrderID,
    ProductID,
    Sales,
    ROW_NUMBER() OVER (ORDER BY Sales DESC) SalesRank_RowNumber
FROM Sales.Orders;
```

	OrderID	ProductID	Sales	SalesRank_RowNumber
1	8	101	90	1
2	4	105	60	2
3	10	102	60	3
4	6	104	50	4
5	7	102	30	5
6	5	104	25	6
7	9	101	20	7
8	3	101	20	8
9	2	102	15	9
10	1	101	10	10

RANK

- Assign a rank to each row in a window, with gaps
- It handles ties
- It leaves gaps in ranking

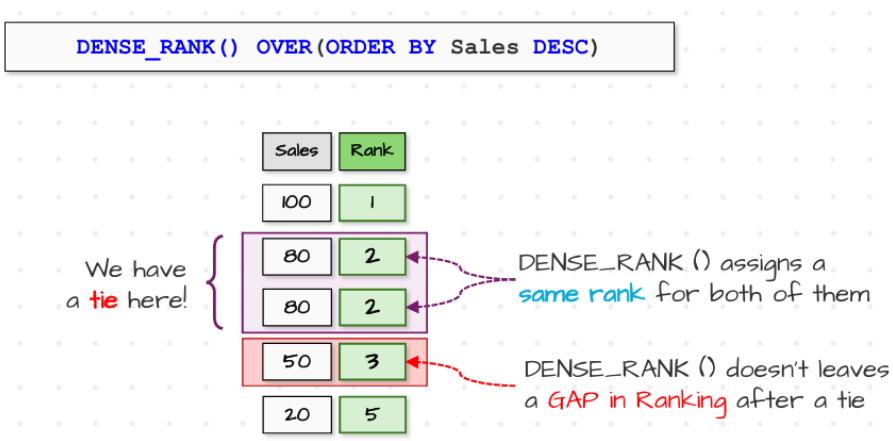



```sql
SELECT
    OrderID,
    ProductID,
    Sales,
    RANK() OVER (ORDER BY Sales DESC) SalesRank_Rank
FROM Sales.Orders;
```

	OrderID	ProductID	Sales	SalesRank_Rank
1	8	101	90	1
2	4	105	60	2
3	10	102	60	2
4	6	104	50	4
5	7	102	30	5
6	5	104	25	6
7	9	101	20	7
8	3	101	20	7
9	2	102	15	9
10	1	101	10	10

DENSE_RANK

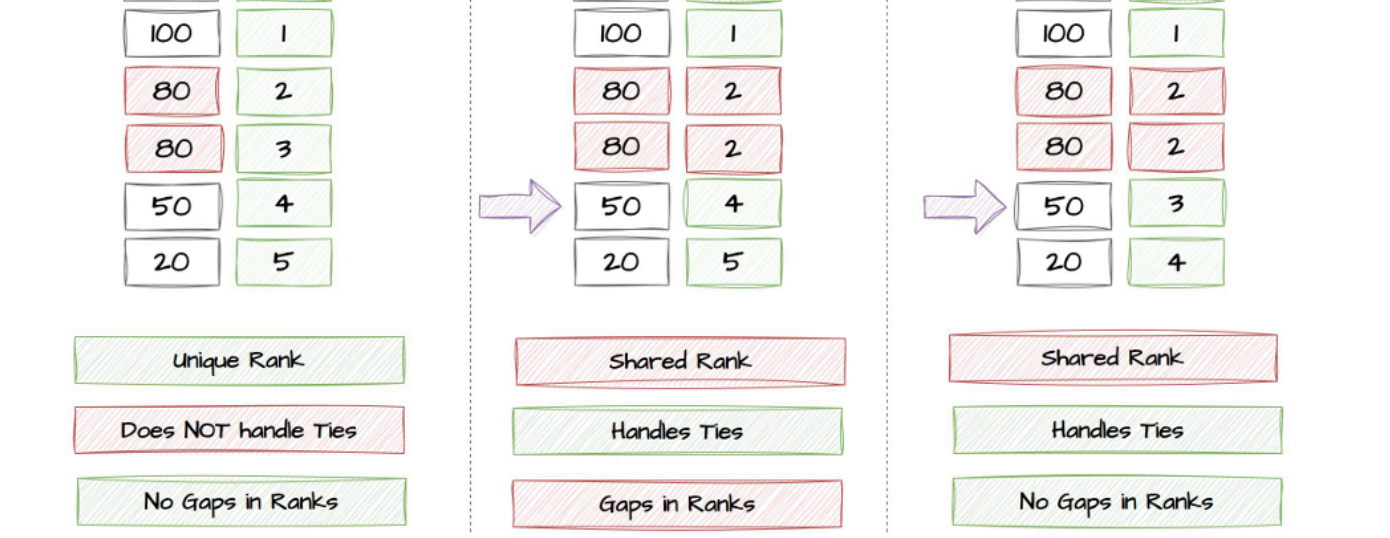
- Assign a rank to each row in a window, without gaps
- It handles ties
- It doesn’t leave gaps in ranking

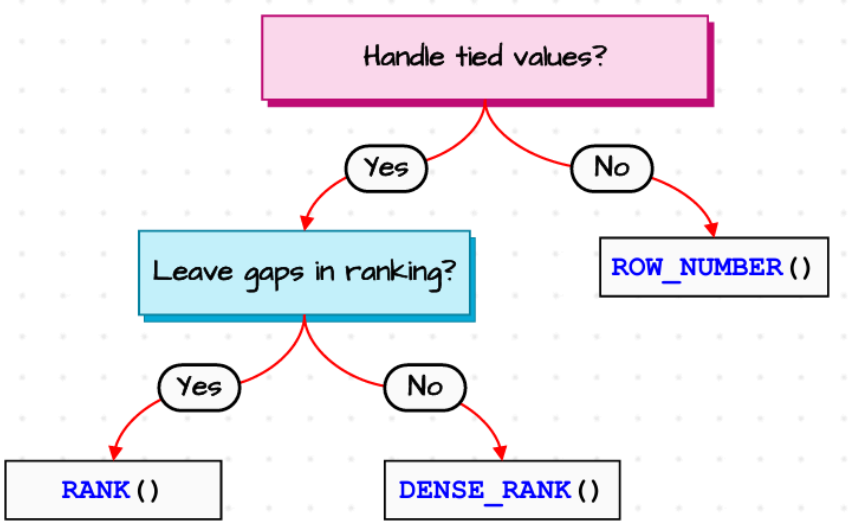



```sql
SELECT
    OrderID,
    ProductID,
    Sales,
    DENSE_RANK() OVER (ORDER BY Sales DESC) SalesRank_DenseRank
FROM Sales.Orders;
```

	OrderID	ProductID	Sales	SalesRank_DenseRank
1	8	101	90	1
2	4	105	60	2
3	10	102	60	2
4	6	104	50	3
5	7	102	30	4
6	5	104	25	5
7	9	101	20	6
8	3	101	20	6
9	2	102	15	7
10	1	101	10	8

ROW_NUMBER vs RANK vs DENSE_RANK






```sql
SELECT
    OrderID,
    ProductID,
    Sales,
    ROW_NUMBER() OVER (ORDER BY Sales DESC) SalesRank_RowNumber,
    RANK()       OVER (ORDER BY Sales DESC) SalesRank_Rank,
    DENSE_RANK() OVER (ORDER BY Sales DESC) SalesRank_DenseRank
FROM Sales.Orders;
```

	OrderID	ProductID	Sales	SalesRank_RowNumber	SalesRank_Rank	SalesRank_DenseRank
1	8	101	90	1	1	1
2	4	105	60	2	2	2
3	10	102	60	3	2	2
4	6	104	50	4	4	3
5	7	102	30	5	5	4
6	5	104	25	6	6	5
7	9	101	20	7	7	6
8	3	101	20	8	7	6
9	2	102	15	9	9	7
10	1	101	10	10	10	8

ROW_NUMBER USE CASE: TOP-N ANALYSIS

- Analysis the top performers to do targeted marketing

```sql
-- Find the top highest sales for each product
SELECT
    *
FROM (
    SELECT
        OrderID,
        ProductID,
        Sales,
        ROW_NUMBER() OVER (PARTITION BY ProductID ORDER BY Sales DESC) SalesRank
    FROM Sales.Orders
)t
WHERE SalesRank = 1;
```

	OrderID	ProductID	Sales	SalesRank
1	8	101	90	1
2	10	102	60	1
3	6	104	50	1
4	4	105	60	1

ROW_NUMBER USE CASE: BOTTOM-N ANALYSIS

- Help analysis the underperformance to manage risks and to do optimizations

```sql
-- Find the lowest 2 customers based on their total sales
SELECT
    *
FROM (
    SELECT
        CustomerID,
        SUM(Sales) TotalSales,
        ROW_NUMBER() OVER (ORDER BY SUM(Sales)) RankCustomers
    FROM Sales.Orders
    GROUP BY CustomerID
)t
WHERE RankCustomers <= 2;
```

	CustomerID	TotalSales	SalesRank
1	2	55	1
2	4	90	2

ROW_NUMBER USE CASE: GENERATE UNIQUE IDS

- Help to assign unique identifier for each row to help paginating
- Paginating
    - The process of breaking down a large data into smaller, more manageable chunks

```sql
-- Assign unique IDs to the rows of the 'OrdersArchive' table
SELECT
    ROW_NUMBER() OVER (ORDER BY OrderDate) NewOrderID,
    *
FROM Sales.OrdersArchive;
```

	NewOrderID	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	S
1	1	1	101	2	3	2024-04-01	2024-04-05	Shipped	1
2	2	2	102	3	3	2024-04-05	2024-04-10	Shipped	4
3	3	3	101	1	4	2024-04-10	2024-04-25	Shipped	7
4	4	4	105	1	3	2024-04-20	2024-04-25	Shipped	9
5	5	4	105	1	3	2024-04-20	2024-04-25	Delivered	9
6	6	5	104	2	5	2024-05-01	2024-05-05	Shipped	3
7	7	6	104	3	5	2024-05-05	2024-05-10	Delivered	5
8	8	6	104	3	5	2024-05-05	2024-05-10	Delivered	5
9	9	6	101	3	5	2024-05-05	2024-05-10	Delivered	5
10	10	7	102	3	5	2024-06-15	2024-06-20	Shipped	1

ROW_NUMBER USE CASE: IDENTIFY DUPLICATES

- Identify and remove duplicate rows to improve data quality

```sql
/* Identify duplicate rows in the table 'OrdersArchive'
   and return a clean result without any duplicates */
SELECT
    *
FROM (
    SELECT
        ROW_NUMBER() OVER (PARTITION BY OrderID
        ORDER BY CreationTime DESC) DuplicateOrder,
        *
    FROM Sales.OrdersArchive
)t
WHERE DuplicateOrder = 1;
```

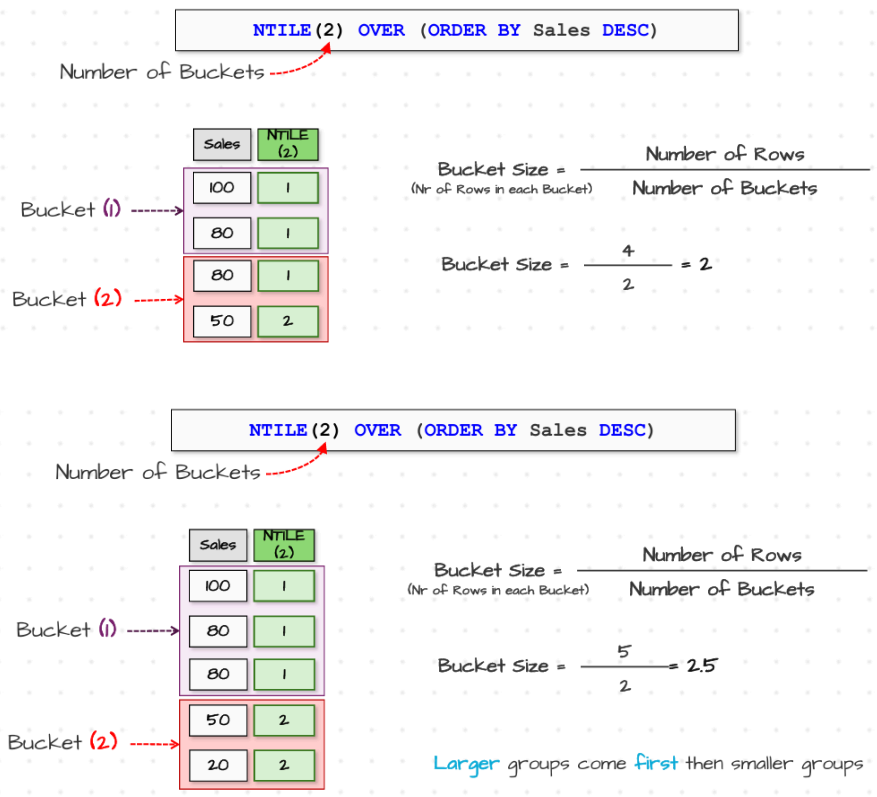
	DuplicateOrder	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Quantity	Sales	CreationTime
1	1	1	101	2	3	2024-04-01	2024-04-05	Shipped	123 Main St	456 Billing St	1	10	2024-04-01 12:34:56.0000000
2	1	2	102	3	3	2024-04-05	2024-04-10	Shipped	456 Elm St	789 Billing St	1	15	2024-04-05 23:22:04.0000000
3	1	3	101	1	4	2024-04-10	2024-04-25	Shipped	789 Maple St	789 Maple St	2	20	2024-04-10 18:24:08.0000000
4	1	4	105	1	3	2024-04-20	2024-04-25	Delivered	987 Victory Lane		2	60	2024-04-20 14:50:33.0000000
5	1	5	104	2	5	2024-05-01	2024-05-05	Shipped	345 Oak St	678 Pine St	1	25	2024-05-01 14:02:41.0000000
6	1	6	101	3	5	2024-05-05	2024-05-10	Delivered	543 Belmont Rd.	3768 Door Way	2	50	2024-05-12 20:36:55.0000000
7	1	7	102	3	5	2024-06-15	2024-06-20	Shipped	111 Main St	222 Billing St	0	60	2024-06-16 23:25:15.0000000

NTILE

- Divides the rows into a specified number of approximately equal groups (Buckets)
- Data Engineer
  - Equalizing Load Processing
- Data Analyst
  - Data Segmentation

$$Bucket\ Size\ =\ \frac{Number\ of\ Rows}{Number\ of\ Buckets}$$

- Bucket Size → Nr of Rows in each Bucket
- RULE
  - Larger groups come first then smaller groups




```sql
SELECT
    OrderID,
    Sales,
    NTILE(1) OVER (ORDER BY Sales DESC) OneBucket,
    NTILE(2) OVER (ORDER BY Sales DESC) TwoBuckets,
    NTILE(3) OVER (ORDER BY Sales DESC) ThreeBuckets,
    NTILE(4) OVER (ORDER BY Sales DESC) FourBuckets
FROM Sales.Orders;
```

NTILE(1)

$$\frac{10}{1} = 10$$

NTILE(2)

$$\frac{10}{2} = 5$$

NTILE(3)

$$\frac{10}{3} = 3.33$$

NTILE(4)

$$\frac{10}{4} = 2.5$$

	OrderID	Sales	OneBucket	TwoBuckets	ThreeBuckets	FourBuckets
1	8	90	1	1	1	1
2	4	60	1	1	1	1
3	10	60	1	1	1	1
4	6	50	1	1	1	2
5	7	30	1	1	2	2
6	5	25	1	2	2	2
7	9	20	1	2	2	3
8	3	20	1	2	3	3
9	2	15	1	2	3	4
10	1	10	1	2	3	4

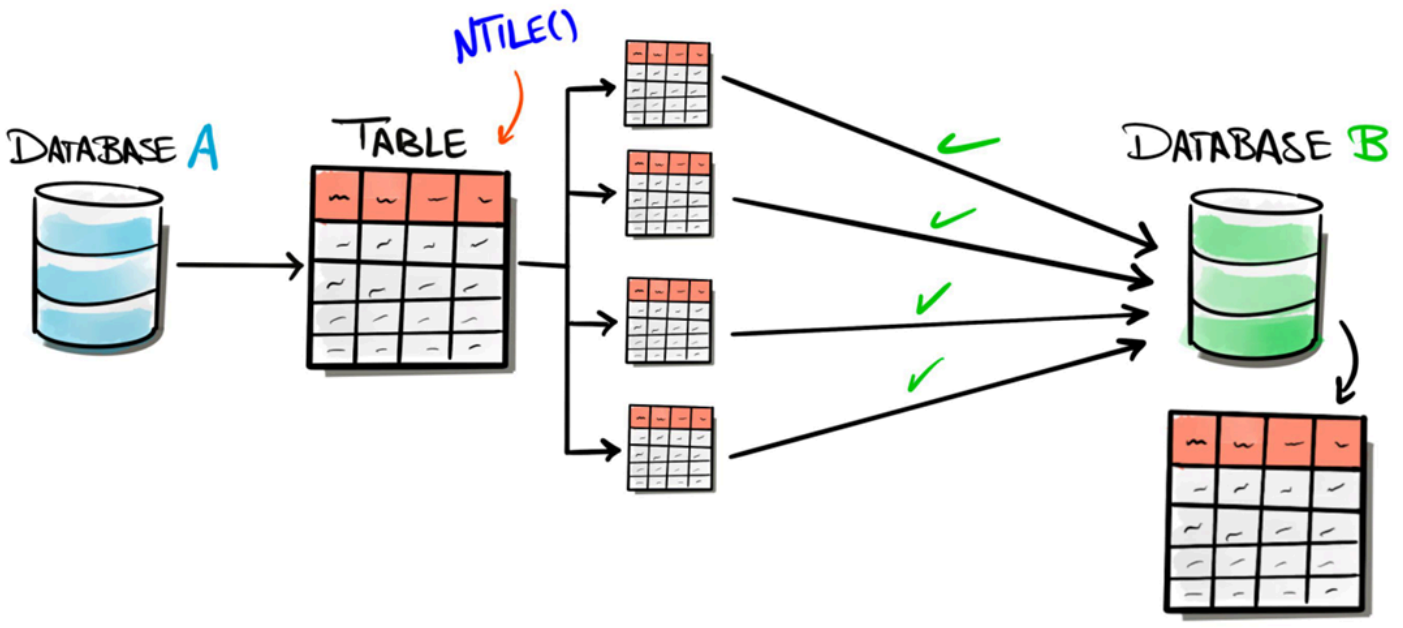
NTILE USE CASE: DATA SEGMENTATION

- Divides a dataset into distinct subsets based on certain criteria

```sql
-- Segment all orders into 3 categories: high, medium and low sales
SELECT
    OrderID,
    Sales,
    CASE (NTILE(3) OVER (ORDER BY Sales DESC))
        WHEN 1 THEN 'High'
        WHEN 2 THEN 'Medium'
        WHEN 3 THEN 'Low'
    END SalesSegmentation
FROM Sales.Orders;
```

	OrderID	Sales	SalesSegmentation
1	8	90	High
2	4	60	High
3	10	60	High
4	6	50	High
5	7	30	Medium
6	5	25	Medium
7	9	20	Medium
8	3	20	Low
9	2	15	Low
10	1	10	Low

NTILE USE CASE: EQUALIZING LOAD




```sql
-- In order to export the data, divide the orders into 2 groups
SELECT
    NTILE(2) OVER (ORDER BY OrderID) GroupID,
    *
FROM Sales.Orders;
```

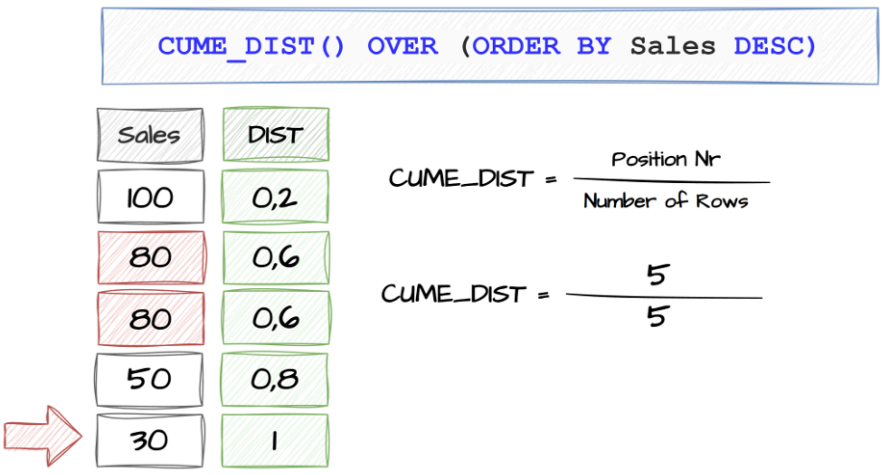
	GroupID	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	Shi
1	1	1	101	2	3	2025-01-01	2025-01-05	Delivered	983
2	1	2	102	3	3	2025-01-05	2025-01-10	Shipped	252
3	1	3	101	1	5	2025-01-10	2025-01-25	Delivered	813
4	1	4	105	1	3	2025-01-20	2025-01-25	Shipped	572
5	1	5	104	2	5	2025-02-01	2025-02-05	Delivered	NU
6	2	6	104	3	5	2025-02-05	2025-02-10	Delivered	173
7	2	7	102	1	1	2025-02-15	2025-02-27	Delivered	133
8	2	8	101	4	3	2025-02-18	2025-02-27	Shipped	294
9	2	9	101	2	3	2025-03-10	2025-03-15	Shipped	373
10	2	10	102	3	5	2025-03-15	2025-03-20	Shipped	NU

CUME_DIST

- Cumulative Distribution
- Calculates the distribution of data points within a window

$$CUME_DIST(X) = \frac{Position\ Nr}{Number\ of\ Rows}$$

- Tie Rule
  - The position of the last occurrence of the same value
- Inclusive
  - The current row is included




```sql
-- Find the products that fall within the highest 40% of the prices
SELECT
    *,
    CONCAT(DistRank * 100,'%') DistRankPerc
FROM(
    SELECT
        Product,
        Price,
        CUME_DIST() OVER(ORDER BY Price DESC) DistRank
    FROM Sales.Products
)t
WHERE DistRank <= 0.4;
```

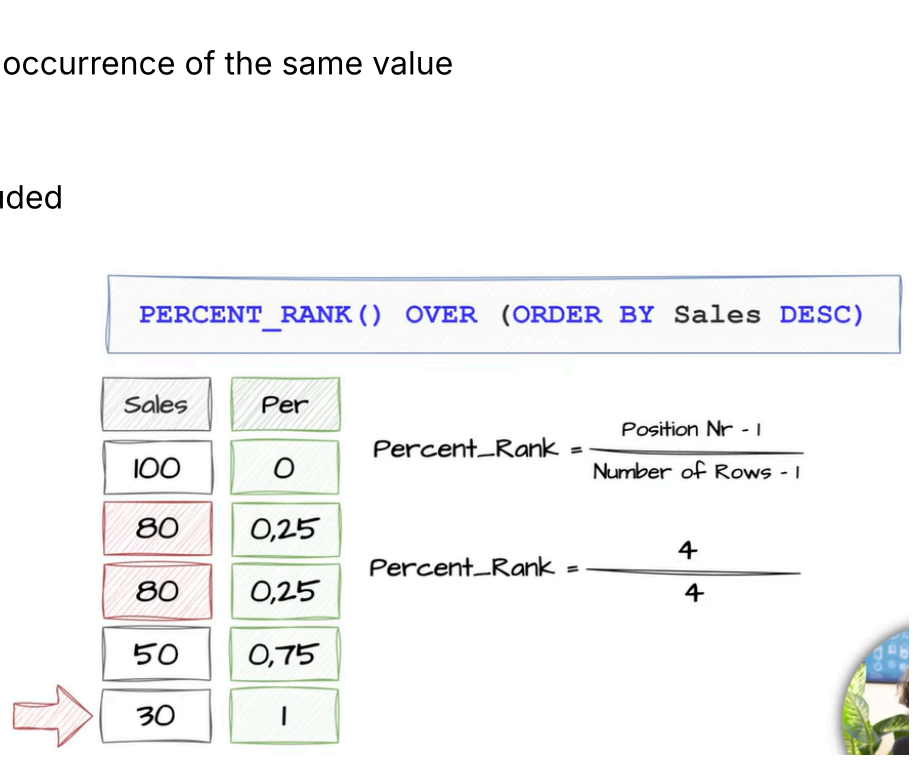
	Product	Price	DistRank	DistRankPerc
1	Gloves	30	0.2	20%
2	Caps	25	0.4	40%

PERCENT_RANK

- Calculates the relative position of each row

$$PERCENT_RANK(X) = \frac{Position\ Nr\ -\ 1}{Number\ of\ Rows\ -\ 1}$$

- Tie Rule
  - The position of the first occurrence of the same value
- Exclusive
  - The current row is excluded




```sql
-- Find the products that fall within the highest 40% of the prices

SELECT
    *,
```

```
    CONCAT(PercentRank * 100,'%') PercentRankPerc
FROM(
    SELECT
        Product,
        Price,
        PERCENT_RANK() OVER(ORDER BY Price DESC) PercentRank
    FROM Sales.Products
)t
WHERE PercentRank <= 0.4;
```

	Product	Price	PercentRank	PercentRankPerc
1	Gloves	30	0	0%
2	Caps	25	0.25	25%

CUME_DIST vs PERCENT_RANK

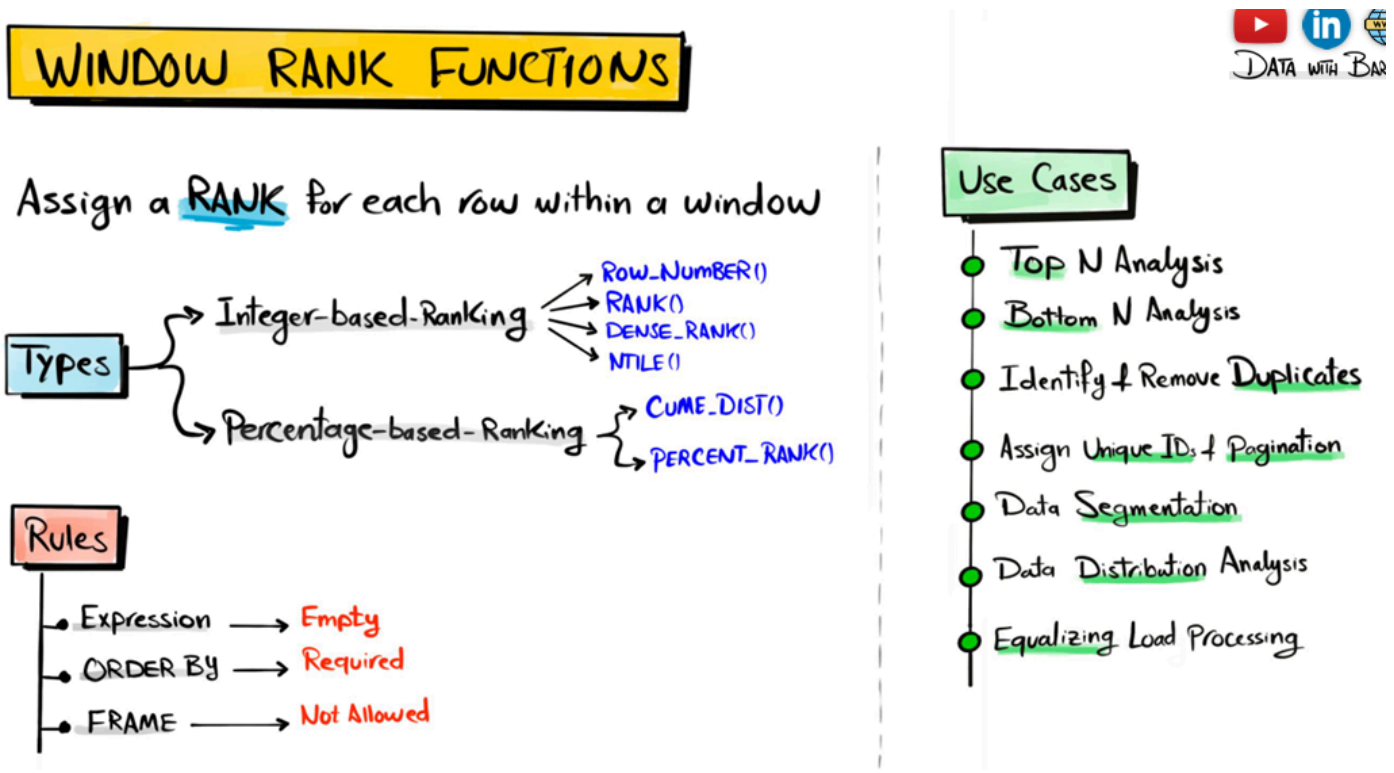
```
CUME_DIST() OVER (ORDER BY Sales DESC)
PERCENT_RANK() OVER (ORDER BY Sales DESC)
```

Sales	DIST	Per
100	0,2	0
80	0,6	0,25
80	0,6	0,25
50	0,8	0,75
30	1	1

$$\text{CUME_DIST} = \frac{\text{Position Nr}}{\text{Number of Rows}}$$

$$\text{Percent_Rank} = \frac{\text{Position Nr} - 1}{\text{Number of Rows} - 1}$$

Summary

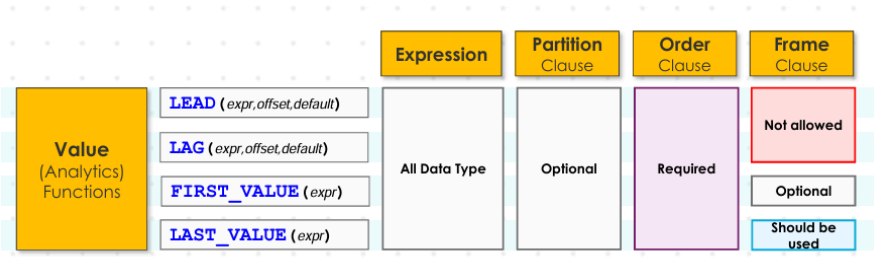


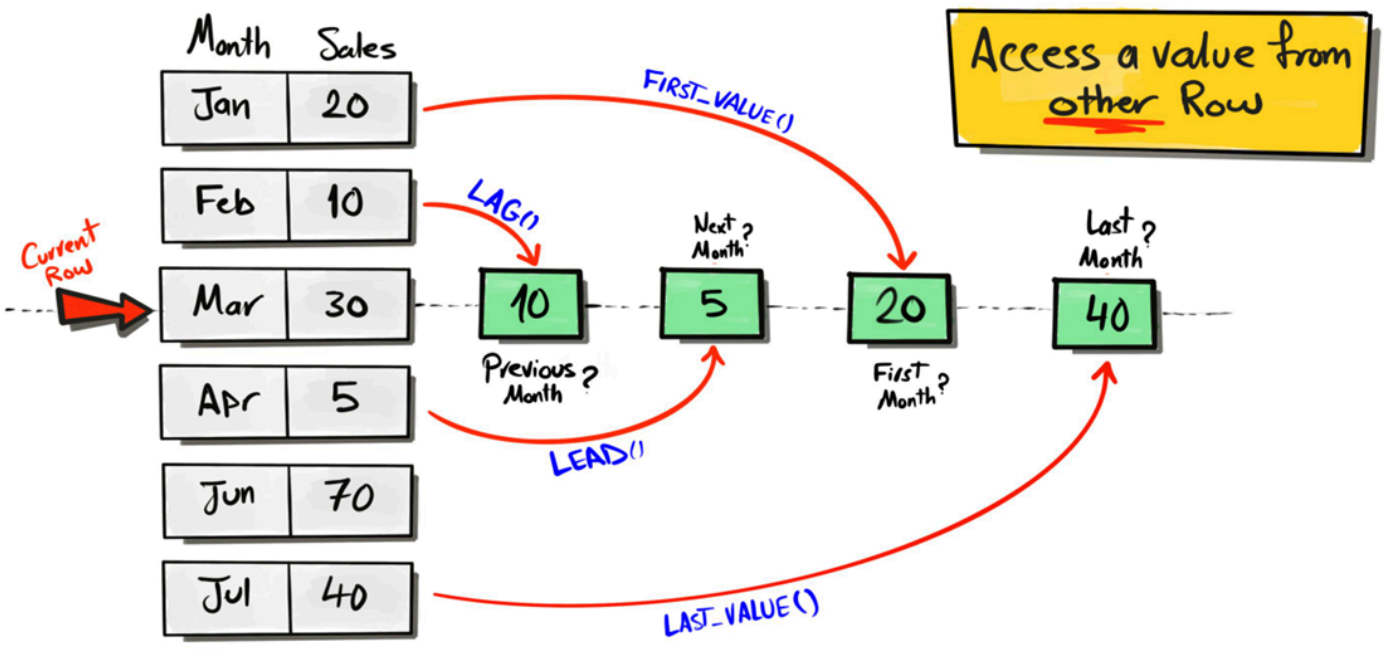
Module 16: Window Value Functions

Introduction to Window Value Functions

- Return a specific value in a window to be compared with the value of current row
- Value Functions
  - LEAD → Returns the value from a subsequent row

- LAG → Returns the value from a previews row
- FIRST_VALUE → Returns the first value in a window
- LAST_VALUE → Returns the last value in a window



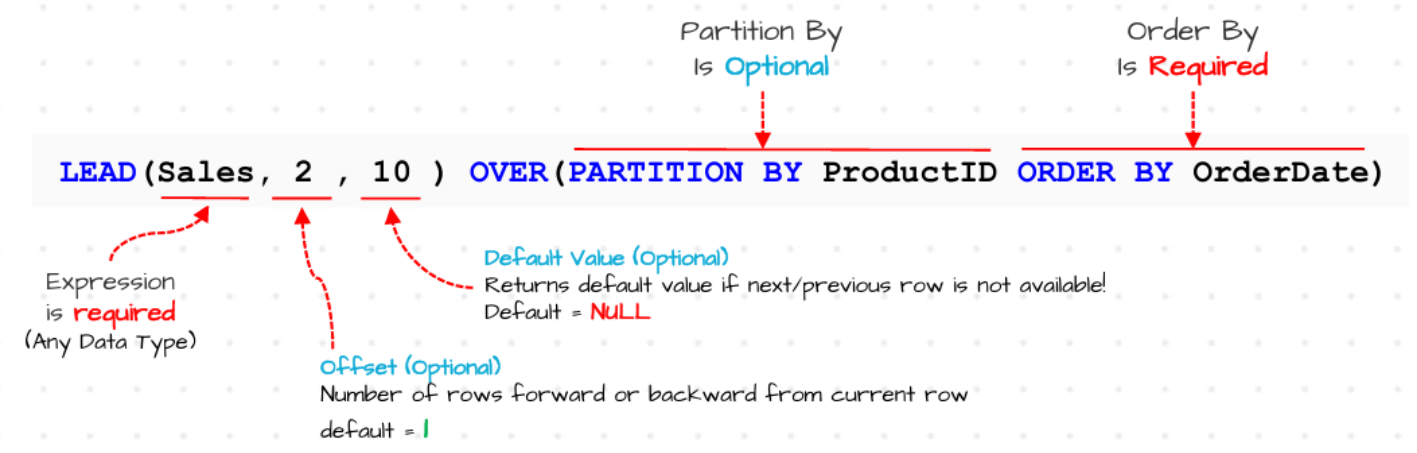


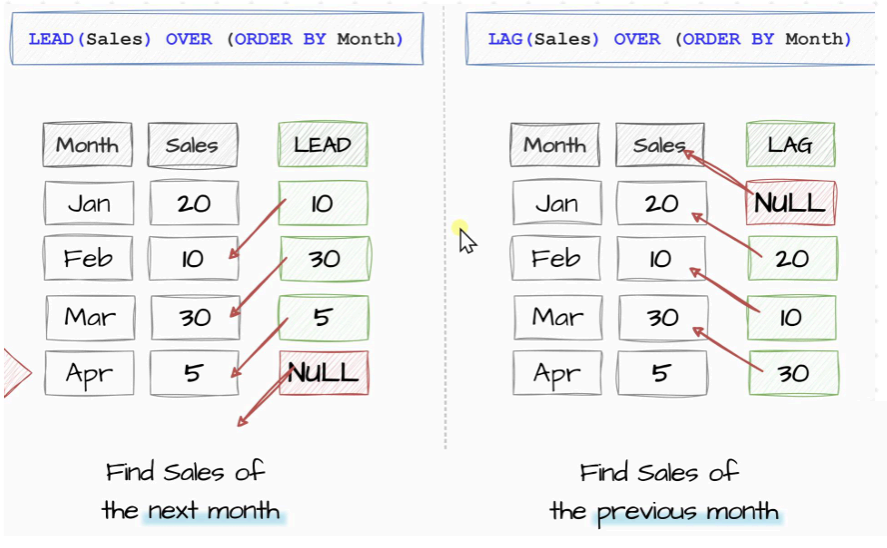
LEAD / LAG

- LEAD
  - Access a value from the next row within a window
- LAG
  - Access a value from the previous row within a window

```
LEAD(Sales, 2, 10) OVER (PARTITION BY ProductID ORDER BY OrderDate))
```

- Sales → Expression is required (any data type)
- 2 → Offset (optional), default = 1
- 10 → Default Value (optional), default = NULL
- PARTITION BY → Partition By is optional
- ORDER BY → Order By is required





LEAD/LAG USE CASE: Month-Over-Month (MoM) ANALYSIS

- TIME SERIES ANALYSIS
  - The process of analyzing the data to understand patterns, trends, and behaviors over time
- Year-over-Year (YoY)
  - Analyze the overall growth or decline of the business's performance over time
- Month-over-Month (MoM)
  - Analyze short-term trends and discover patterns in seasonality

```sql
/* Analyze the month-over-month (MoM) performance by finding
   the percentage change in sales between the current
   and previous month */
SELECT
    *,
    CurrentMonthSales - PreviousMonthSales MoM_Change,
    ROUND(((CAST(CurrentMonthSales AS FLOAT) - PreviousMonthSales) / PreviousMonthSales * 10
0),1) MoM_Percentage
FROM(
    SELECT
        MONTH(OrderDate) OrderMonth,
        SUM(Sales) CurrentMonthSales,
        LAG(SUM(Sales)) OVER(ORDER BY MONTH(OrderDate)) PreviousMonthSales
    FROM Sales.Orders
    GROUP BY MONTH(OrderDate)
)t;
```

	OrderMonth	CurrentMonthSales	PreviousMonthSales	MoM_Change	MoM_Percentage
1	1	105	NULL	NULL	NULL
2	2	195	105	90	85.7
3	3	80	195	-115	-59

LEAD/LAG USE CASE: CUSTOMER RETENTION ANALYSIS

- Measure customer's behavior and loyalty to help businesses build strong relationships with customers

```sql
/* In order to analyze customer loyalty, rank customers
   based on the average days between their orders */
SELECT
    CustomerID,
    AVG(DaysUntilNextOrder) AvgDays,
    RANK() OVER(ORDER BY COALESCE(AVG(DaysUntilNextOrder),9999999)) CustomerRank
FROM(
    SELECT
        OrderID,
        CustomerID,
```

```
        OrderDate CurrentOrder,
        LEAD(OrderDate) OVER(PARTITION BY CustomerID ORDER BY OrderDate) NextOrder,
        DATEDIFF(day,OrderDate,LEAD(OrderDate) OVER(PARTITION BY CustomerID ORDER BY OrderDat
e)) DaysUntilNextOrder
    FROM Sales.Orders
)t
GROUP BY CustomerID;
```

	CustomerID	AvgDays	CustomerRank
1	1	18	1
2	2	34	2
3	3	34	2
4	4	NULL	4

LEAD/LAG USE CASE: TIME GAP ANALYSIS

```
-- Find the average shipping duration in days for each month
SELECT
    MONTH(OrderDate) OrderMonth,
    AVG(DATEDIFF(day,OrderDate,ShipDate)) AvgShippingDuration
FROM Sales.Orders
GROUP BY MONTH(OrderDate);
```

	OrderMonth	AvgShippingDuration
1	1	7
2	2	7
3	3	5

```
-- Find the number of days between each order and previous order
SELECT
    OrderID,
    OrderDate CurrentOrderDate,
    LAG(OrderDate) OVER(ORDER BY OrderDate) PerviousOrderDate,
    DATEDIFF(day,LAG(OrderDate) OVER(ORDER BY OrderDate),OrderDate) DaysDiff
FROM Sales.Orders;
```

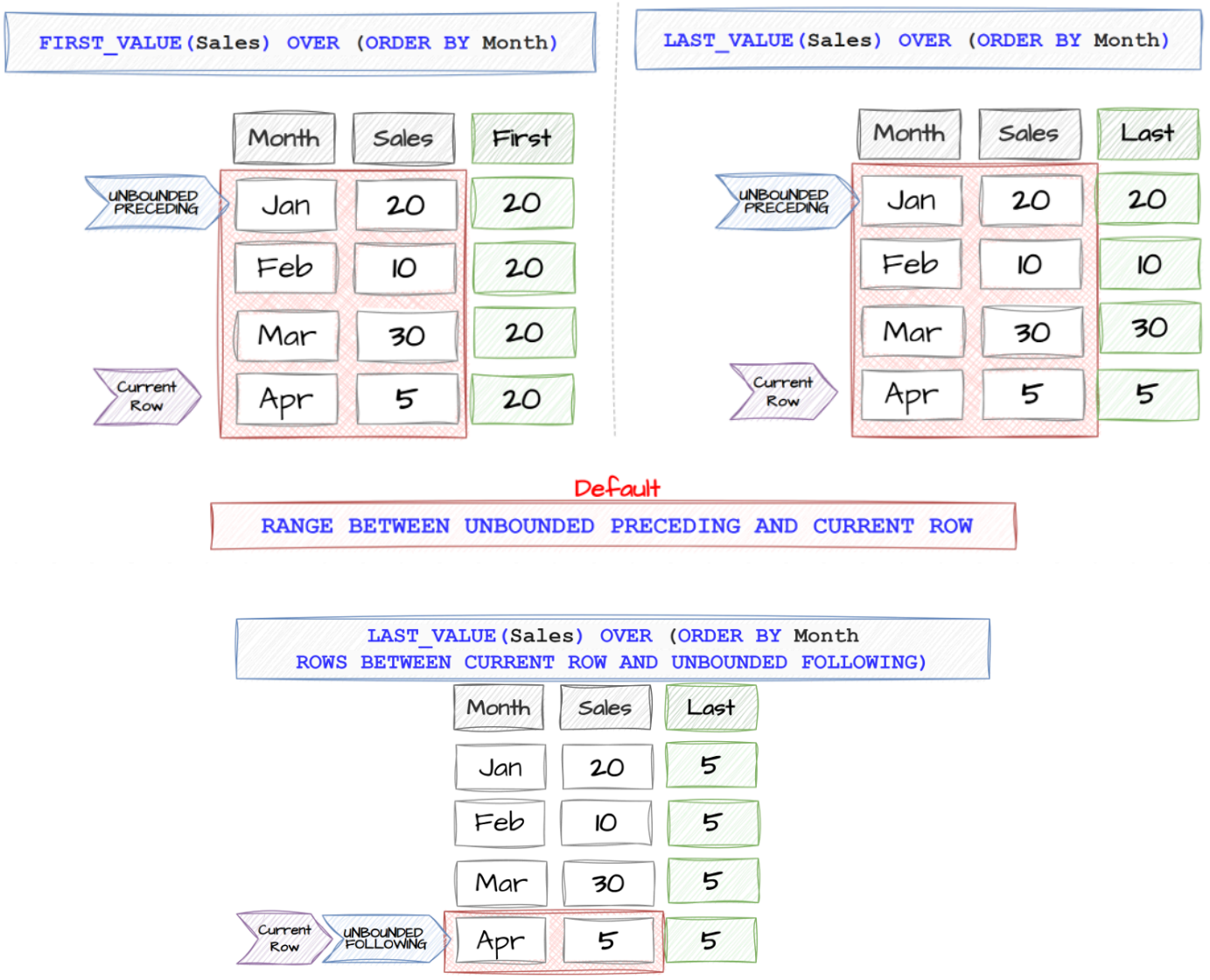
	OrderID	CurrentOrderDate	PerviousOrderDate	DaysDiff
1	1	2025-01-01	NULL	NULL
2	2	2025-01-05	2025-01-01	4
3	3	2025-01-10	2025-01-05	5
4	4	2025-01-20	2025-01-10	10
5	5	2025-02-01	2025-01-20	12
6	6	2025-02-05	2025-02-01	4
7	7	2025-02-15	2025-02-05	10
8	8	2025-02-18	2025-02-15	3
9	9	2025-03-10	2025-02-18	20
10	10	2025-03-15	2025-03-10	5

FIRST_VALUE / LAST_VALUE

- FIRST_VALUE
  - Access a value from the first row within a window
- LAST_VALUE
  - Access a value from the last row within a window

```
-- FIRST_VALUE
FIRST_VALUE(Sales) OVER (ORDER BY Month)
```

```sql
-- LAST_VALUE
LAST_VALUE(Sales) OVER (ORDER BY Month ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)
```




```sql
-- Find the lowest and highest sales for each product
SELECT
    OrderID,
    ProductID,
    Sales,
    FIRST_VALUE(Sales) OVER(PARTITION BY ProductID
    ORDER BY Sales) LowestSales,
    MIN(Sales) OVER(PARTITION BY ProductID) LowestSales2,
    LAST_VALUE(Sales) OVER(PARTITION BY ProductID
    ORDER BY Sales
    ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING) HighestSales,
    FIRST_VALUE(Sales) OVER(PARTITION BY ProductID
    ORDER BY Sales DESC) HighestSales2,
    MAX(Sales) OVER(PARTITION BY ProductID) HighestSales3
FROM Sales.Orders;
```

	OrderID	ProductID	Sales	LowestSales	LowestSales2	HighestSales	HighestSales2	HighestSales3
1	8	101	90	10	10	90	90	90
2	3	101	20	10	10	90	90	90
3	9	101	20	10	10	90	90	90
4	1	101	10	10	10	90	90	90
5	10	102	60	15	15	60	60	60
6	7	102	30	15	15	60	60	60
7	2	102	15	15	15	60	60	60
8	6	104	50	25	25	50	50	50
9	5	104	25	25	25	50	50	50
10	4	105	60	60	60	60	60	60

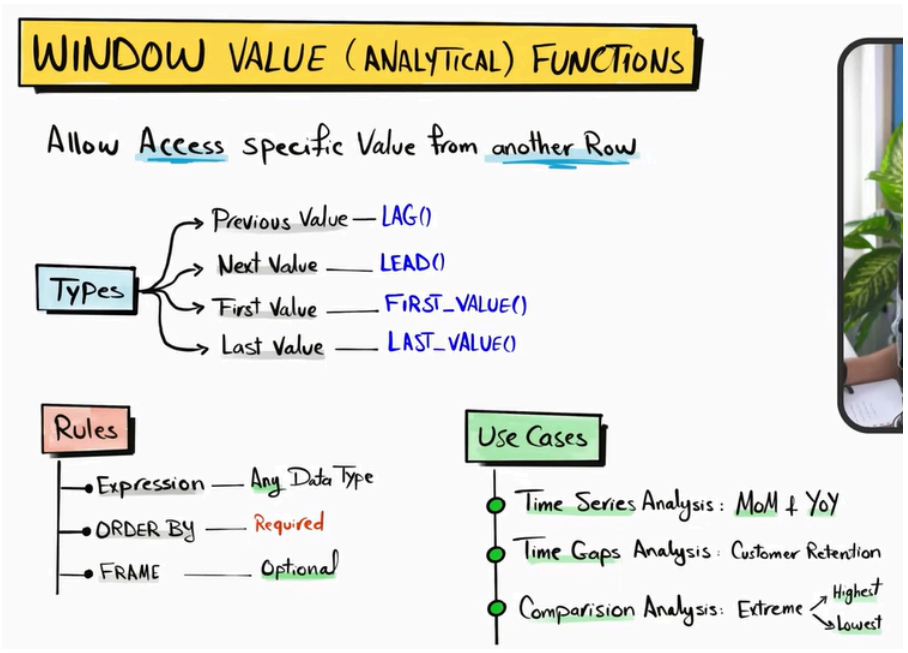
FIRST_VALUE/LAST_VALUE USE CASE: COMPARE TO EXTREMES

```sql
-- Find the difference between the highest and lowest sales
SELECT
```

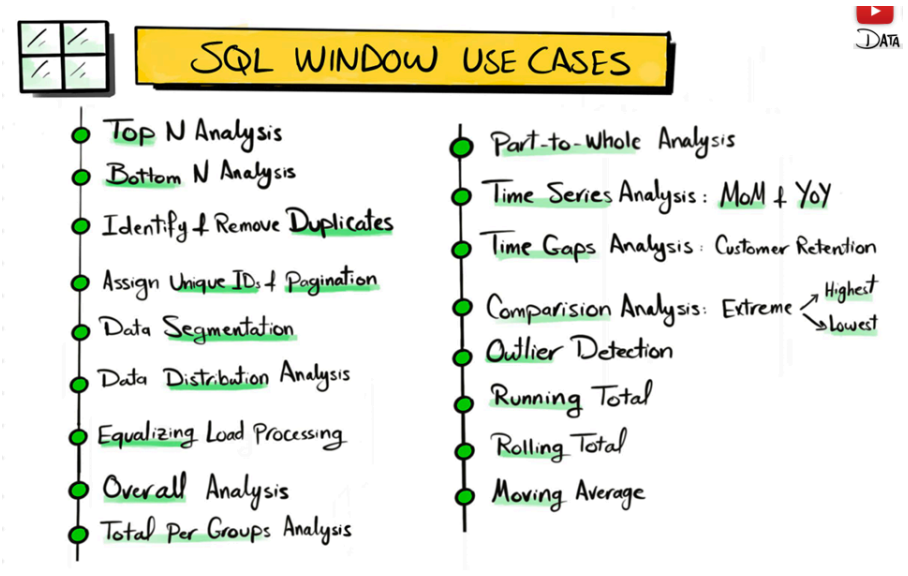
```
    *,
    Sales - LowestSales SalesDifference1,
    HighestSales - Sales SalesDifference2
FROM(
    SELECT
        OrderID,
        ProductID,
        Sales,
        FIRST_VALUE(Sales) OVER(PARTITION BY ProductID
        ORDER BY Sales) LowestSales,
        FIRST_VALUE(Sales) OVER(PARTITION BY ProductID
        ORDER BY Sales DESC) HighestSales
    FROM Sales.Orders
)t;
```

	OrderID	ProductID	Sales	LowestSales	HighestSales	SalesDifference1	SalesDifference2
1	8	101	90	10	90	80	0
2	3	101	20	10	90	10	70
3	9	101	20	10	90	10	70
4	1	101	10	10	90	0	80
5	10	102	60	15	60	45	0
6	7	102	30	15	60	15	30
7	2	102	15	15	60	0	45
8	6	104	50	25	50	25	0
9	5	104	25	25	50	0	25
10	4	105	60	60	60	0	0

Summary



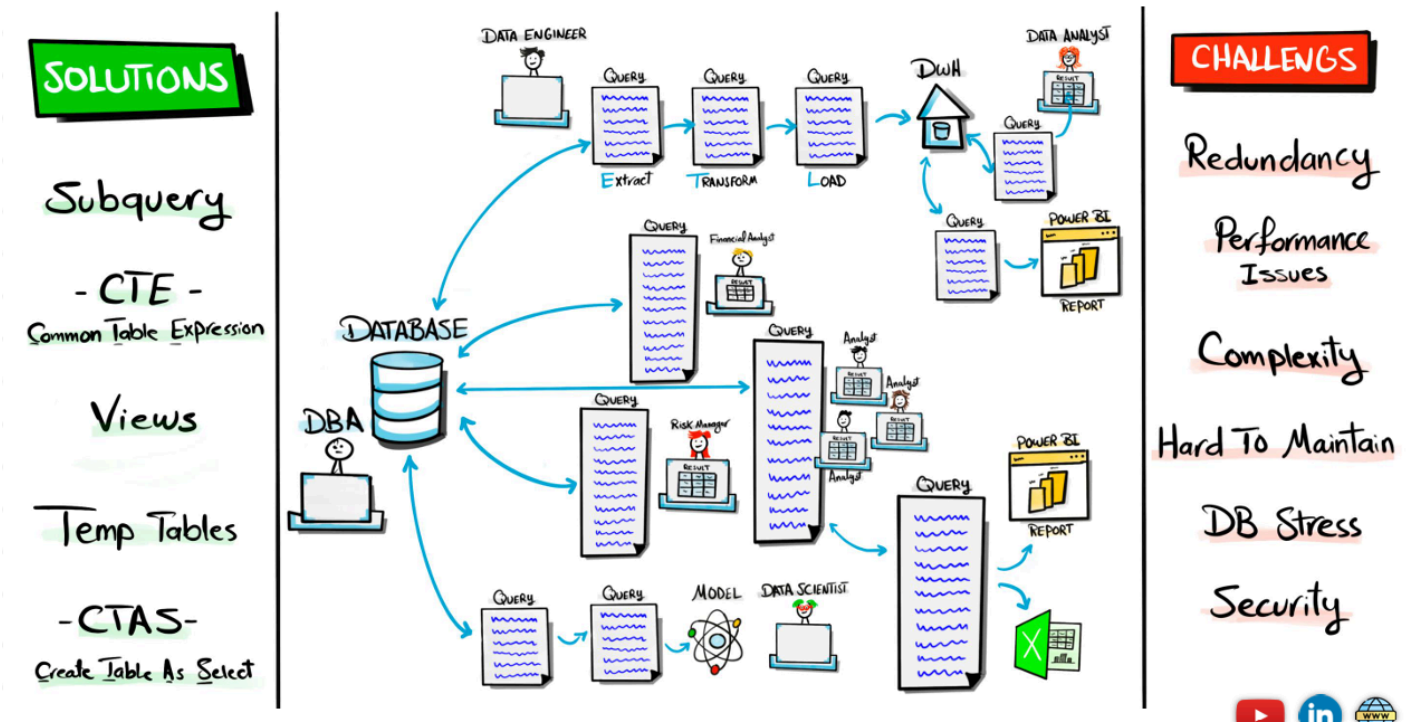
SQL WINDOW USE CASES



Module 17: Subqueries

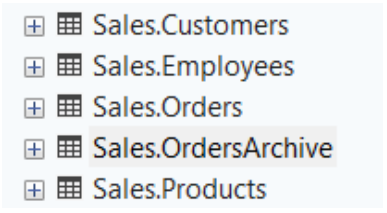
Advanced SQL Techniques

- Data Warehouse
  - A special database that collects and integrates data from different sources to enable analytics and support decision-making



Simplified Database Architecture

- Database Engine
  - It is the brain of the database, executing multiple operations such as storing, retrieving, and managing data within the database
- Disk Storage
  - Long-term memory, where data is stored permanently
  - + Capacity: can hold a large amount of data
  - - Speed: slow to read and to write
  - Types of storage areas
    - USER DATA STORAGE
      - It’s the main content of the database
      - This is where the actual data that users care about is stored

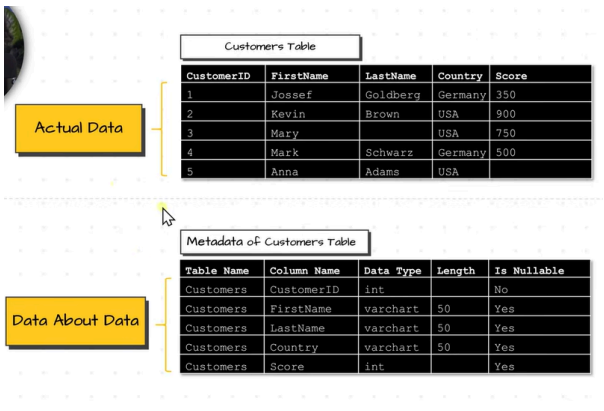



```sql
SELECT
    *
FROM Sales.Orders;
```

	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Quantity	Sales	CreationTime
1	1	101	2	3	2025-01-01	2025-01-05	Delivered	9833 Mt. Dias Blv.	1226 Shoe St.	1	10	2025-01-01 12:34:56.0000000
2	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	NULL	1	15	2025-01-05 23:22:04.0000000
3	3	101	1	5	2025-01-10	2025-01-25	Delivered	8157 W. Book	8157 W. Book	2	20	2025-01-10 18:24:08.0000000
4	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane		2	60	2025-01-20 05:50:33.0000000
5	5	104	2	5	2025-02-01	2025-02-05	Delivered	NULL	NULL	1	25	2025-02-01 14:02:41.0000000
6	6	104	3	5	2025-02-05	2025-02-10	Delivered	1792 Belmont Rd.	NULL	2	50	2025-02-06 15:34:57.0000000
7	7	102	1	1	2025-02-15	2025-02-27	Delivered	136 Balboa Court		2	30	2025-02-16 06:22:01.0000000
8	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4311 Clay Rd	3	90	2025-02-18 10:45:22.0000000
9	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way		2	20	2025-03-10 12:59:04.0000000
10	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	NULL	0	60	2025-03-16 23:25:15.0000000

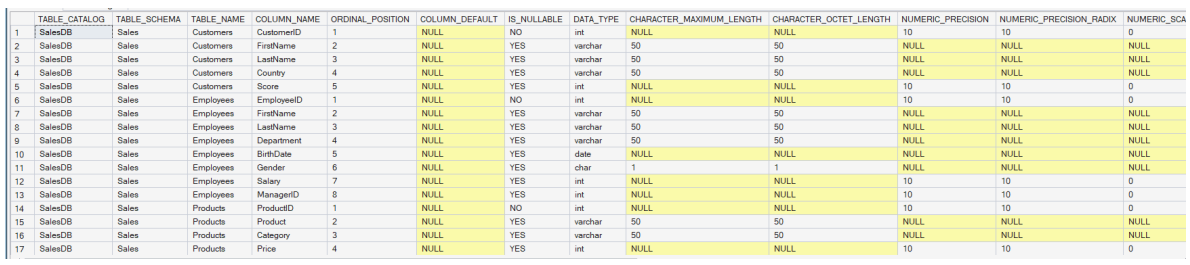
- SYSTEM CATALOG

- Database’s internal storage for its own information
- A blueprint that keeps track of everything about the database itself, not the user data
- It holds the Metadata information about the database
- Metadata
    - Data about Data



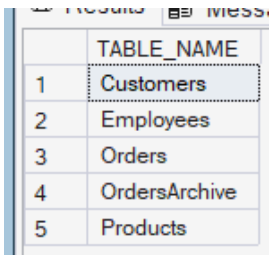
- Information Schema
    - A system-defined schema with built-in views that provide info about the database, like tables and columns

```
SELECT
    *
FROM INFORMATION_SCHEMA.COLUMNS;
```



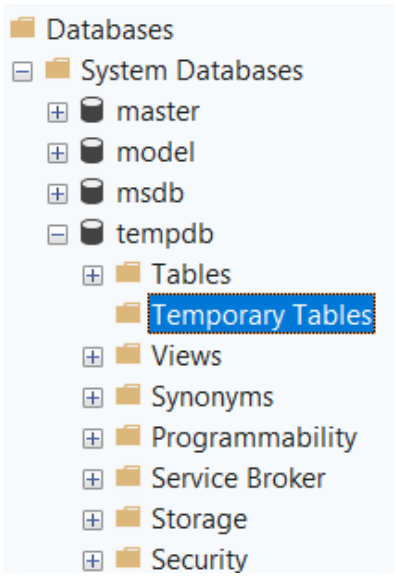
	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	COLUMN_DEFAULT	IS_NULLABLE	DATA_TYPE	CHARACTER_MAXIMUM_LENGTH	CHARACTER_OCTET_LENGTH	NUMERIC_PRECISION	NUMERIC_PRECISION_RADIX	NUMERIC_SCA
1	SalesDB	Sales	Customers	CustomerID	1	NULL	NO	int	NULL	NULL	10	10	0
2	SalesDB	Sales	Customers	FirstName	2	NULL	YES	varchar	50	50	NULL	NULL	NULL
3	SalesDB	Sales	Customers	LastName	3	NULL	YES	varchar	50	50	NULL	NULL	NULL
4	SalesDB	Sales	Customers	Country	4	NULL	YES	varchar	50	50	NULL	NULL	NULL
5	SalesDB	Sales	Customers	Score	5	NULL	YES	int	NULL	NULL	10	10	0
6	SalesDB	Sales	Employees	EmployeeID	1	NULL	NO	int	NULL	NULL	10	10	0
7	SalesDB	Sales	Employees	FirstName	2	NULL	YES	varchar	50	50	NULL	NULL	NULL
8	SalesDB	Sales	Employees	LastName	3	NULL	YES	varchar	50	50	NULL	NULL	NULL
9	SalesDB	Sales	Employees	Department	4	NULL	YES	varchar	50	50	NULL	NULL	NULL
10	SalesDB	Sales	Employees	BirthDate	5	NULL	YES	date	NULL	NULL	NULL	NULL	NULL
11	SalesDB	Sales	Employees	Gender	6	NULL	YES	char	1	1	NULL	NULL	NULL
12	SalesDB	Sales	Employees	Salary	7	NULL	YES	int	NULL	NULL	10	10	0
13	SalesDB	Sales	Employees	ManagerID	8	NULL	YES	int	NULL	NULL	10	10	0
14	SalesDB	Sales	Products	ProductID	1	NULL	NO	int	NULL	NULL	10	10	0
15	SalesDB	Sales	Products	Product	2	NULL	YES	varchar	50	50	NULL	NULL	NULL
16	SalesDB	Sales	Products	Category	3	NULL	YES	varchar	50	50	NULL	NULL	NULL
17	SalesDB	Sales	Products	Price	4	NULL	YES	int	NULL	NULL	10	10	0

```
SELECT
    DISTINCT TABLE_NAME
FROM INFORMATION_SCHEMA.COLUMNS;
```

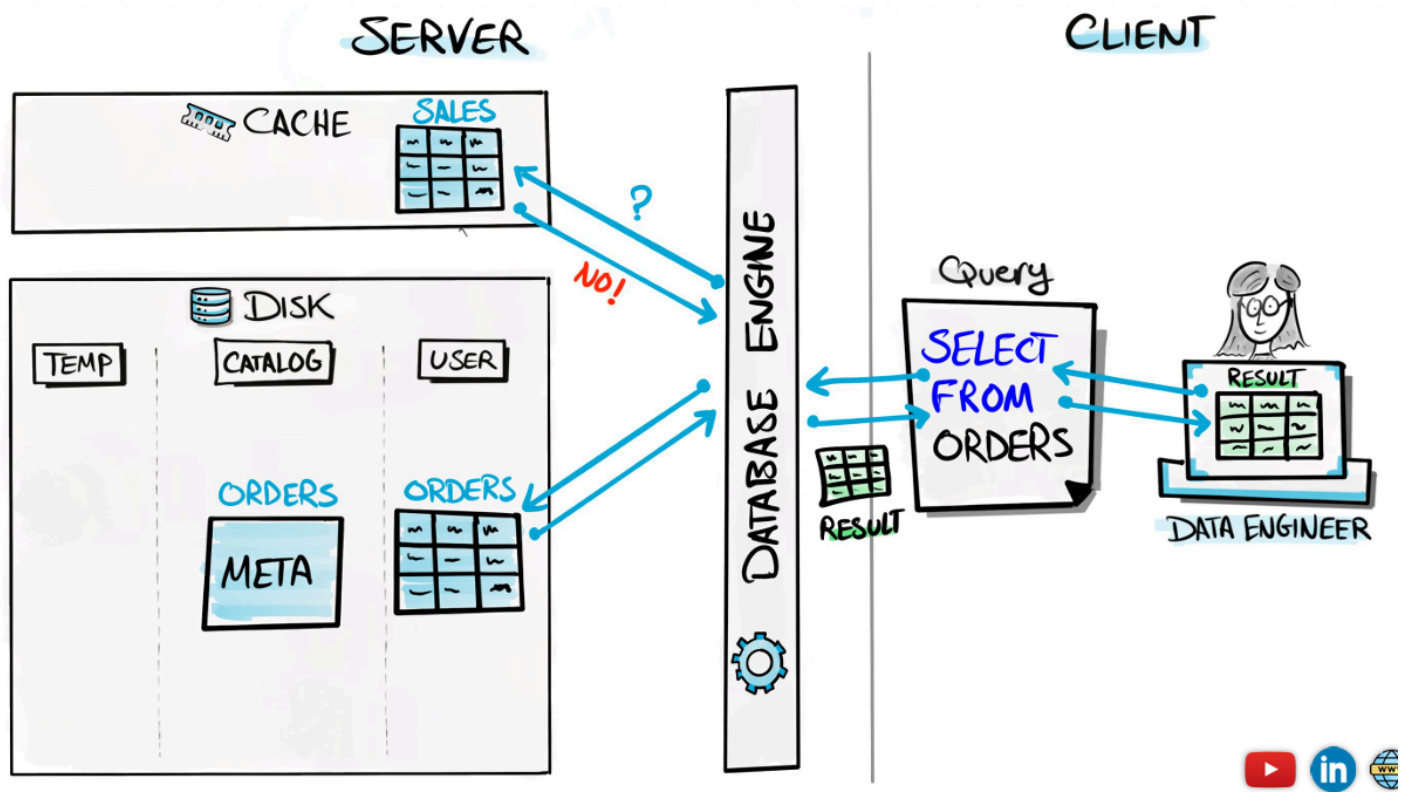


	TABLE_NAME
1	Customers
2	Employees
3	Orders
4	OrdersArchive
5	Products

- TEMPORARY DATA STORAGE
    - Temporary space used by the database for short-term tasks, like processing queries or sorting data
    - Once these tasks are done, the storage is cleared

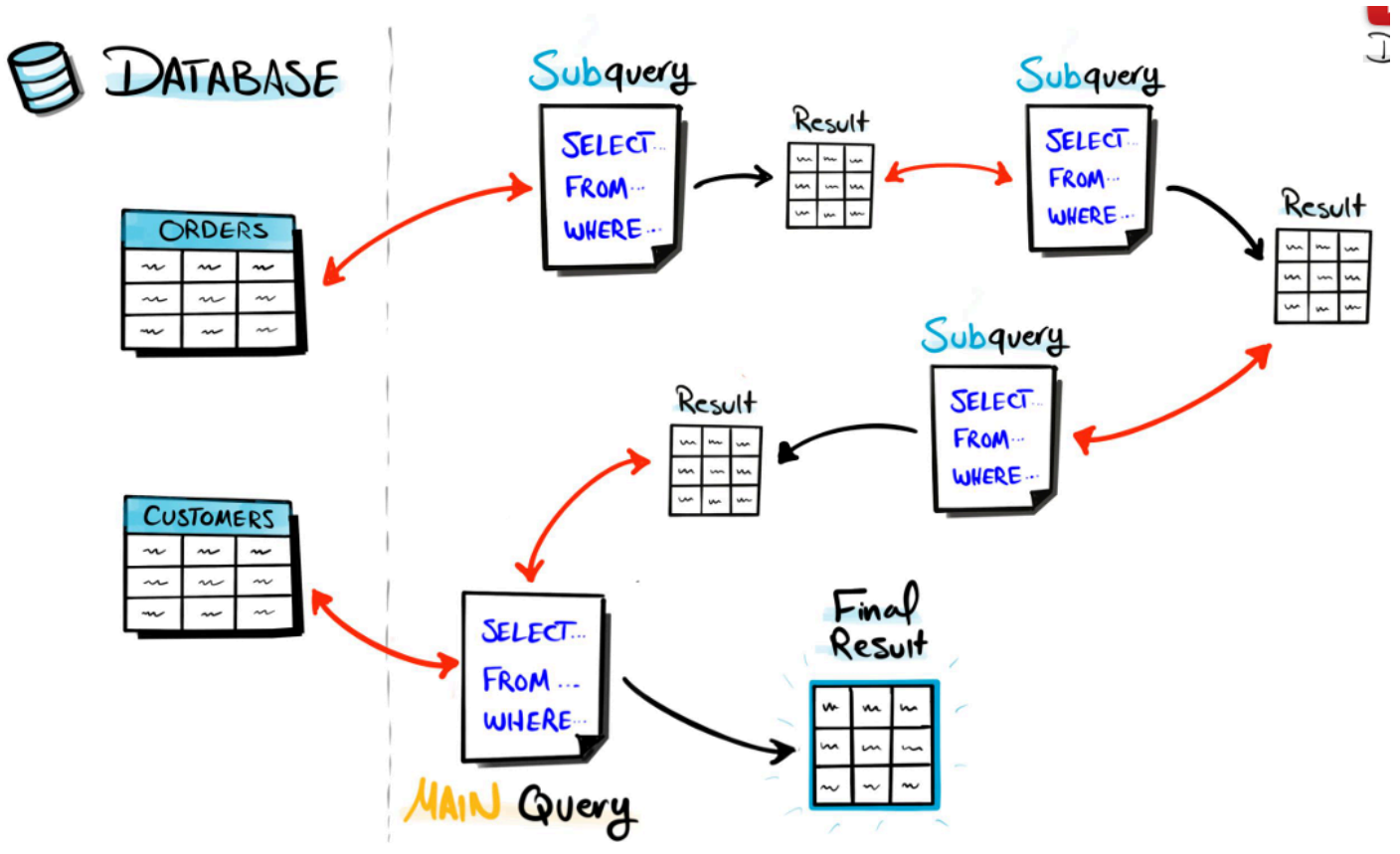


- Cache Storage
  - Fast short-term memory, where data is stored temporarily
  - + Speed: extremely fast to read and to write
  - - Capacity: can hold smaller amount of data

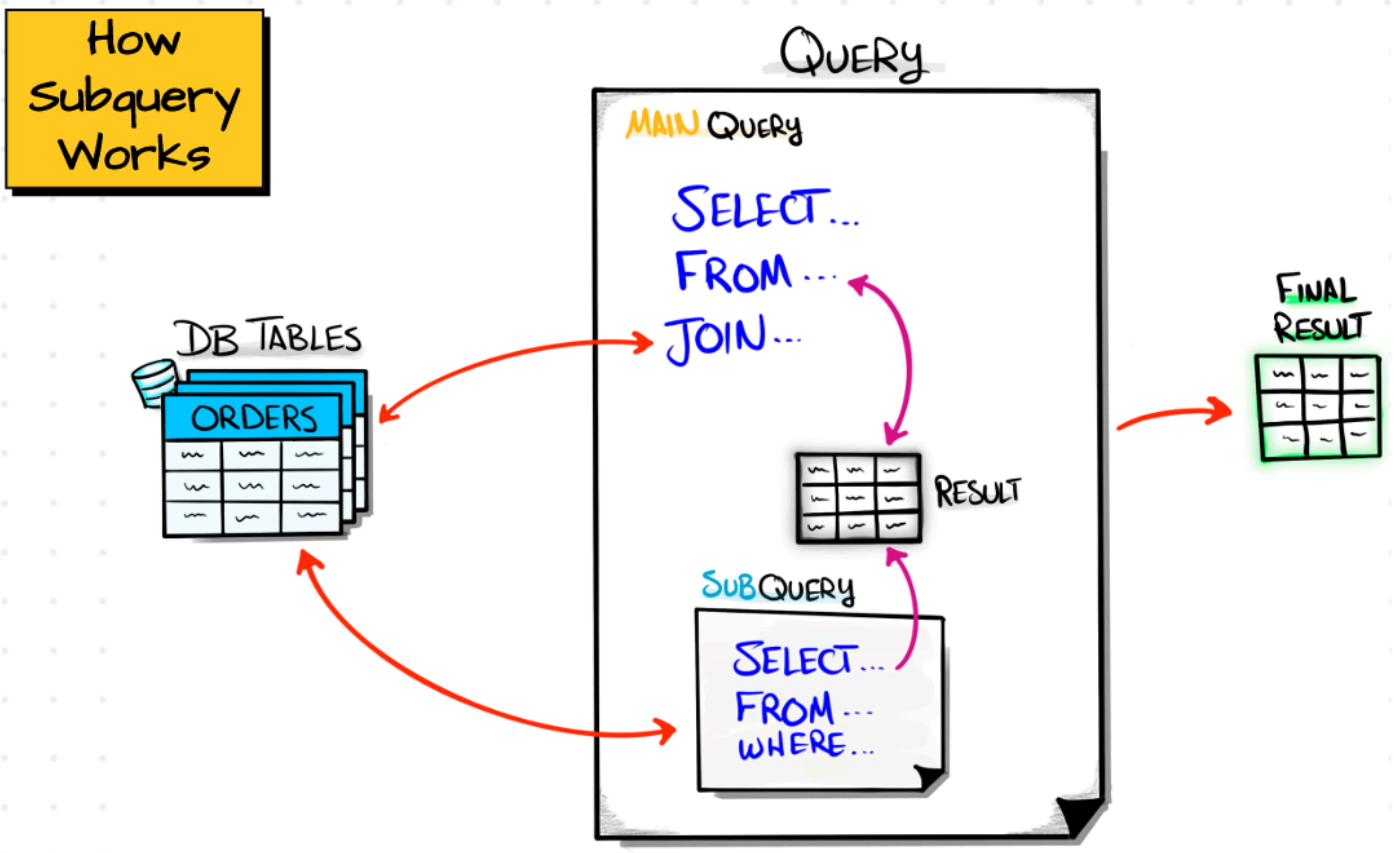


SUBQUERY

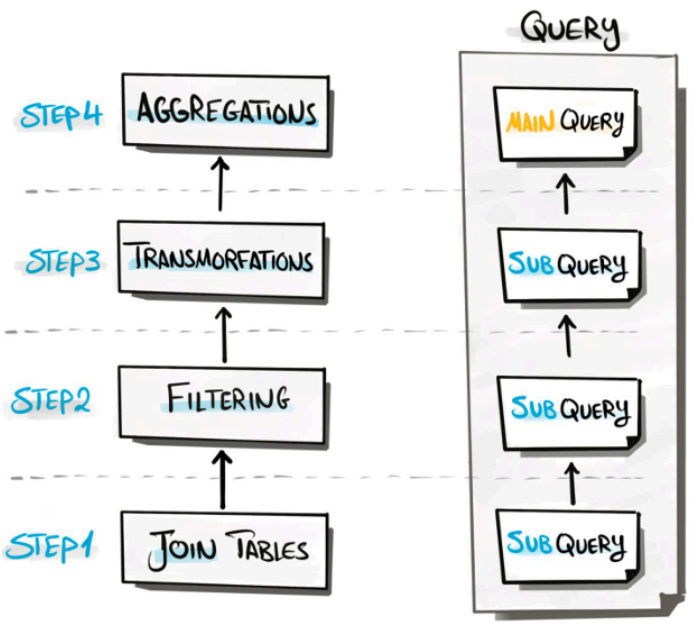
- A query inside another query

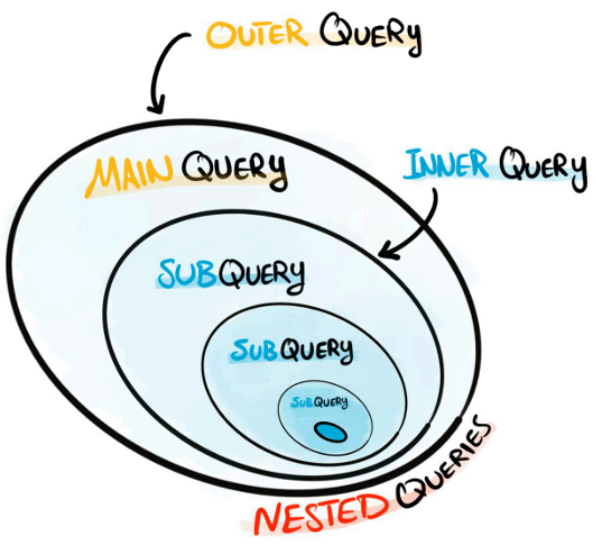


How Subquery Works



Why Subqueries



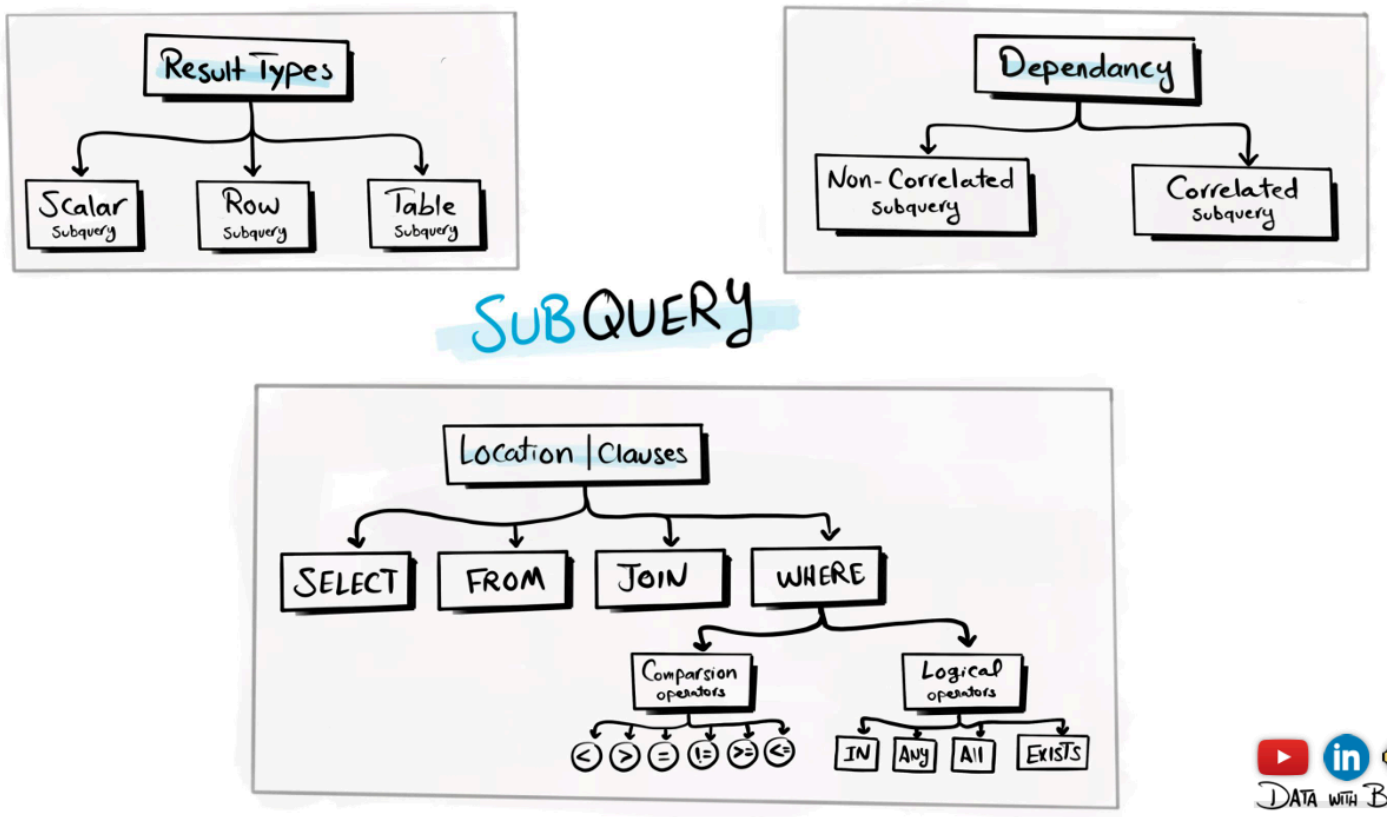


Subquery Categories

- Dependency
  - Non-Correlated Subquery
  - Correlated Subquery
- Result Types
  - Scalar Subquery
  - Row Subquery
  - Table Subquery
- Location | Clauses
  - SELECT
  - FROM
  - JOIN
  - WHERE
    - Comparison Operators

- > < = ≠ ≥ ≤

- Logical Operators
  - IN, ANY, ALL, EXISTS



Subquery Result Types

- Scalar Subquery
  - Single Value

```sql
SELECT
    AVG(Sales) AvgSales
FROM Sales.Orders;
```

	AvgSales
1	38

- Row Subquery
  - Multiple Rows of Single Column

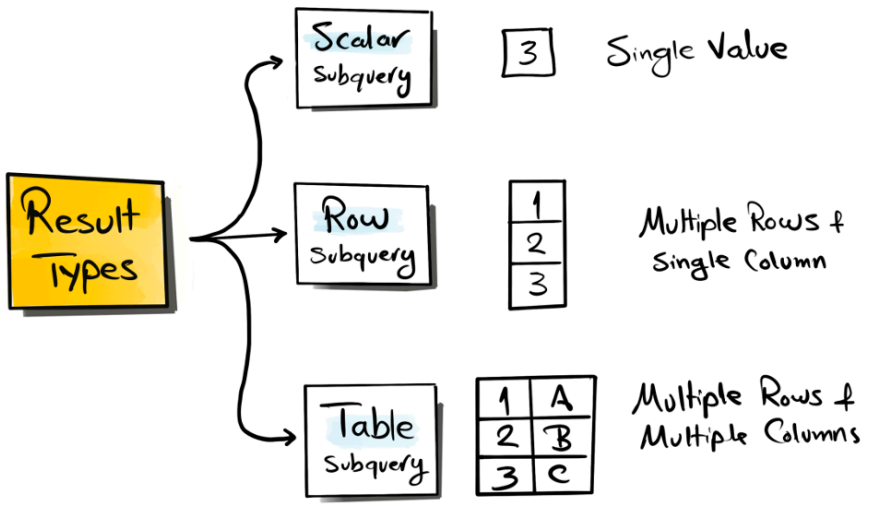
```sql
SELECT
    OrderID
FROM Sales.Orders;
```

	OrderID
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10

- Table Subquery
  - Multiple Rows of Multiple Columns

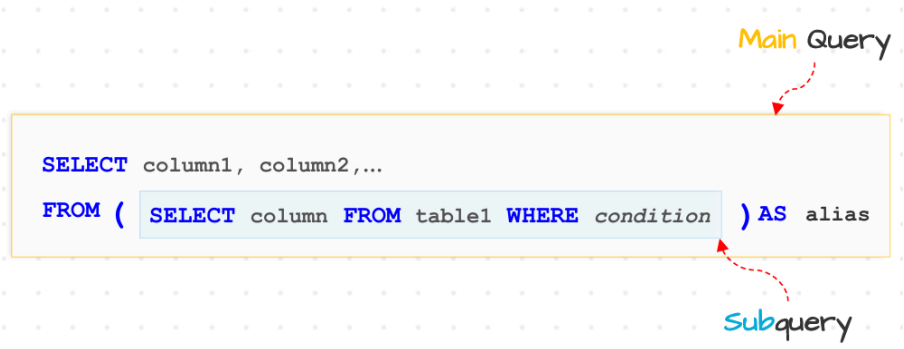
```sql
SELECT
    *
FROM Sales.Orders;
```

	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress
1	1	101	2	3	2025-01-01	2025-01-05	Delivered	9833 Mt. Dias
2	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Co.
3	3	101	1	5	2025-01-10	2025-01-25	Delivered	8157 W. Book
4	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory L
5	5	104	2	5	2025-02-01	2025-02-05	Delivered	NULL
6	6	104	3	5	2025-02-05	2025-02-10	Delivered	1792 Belmor
7	7	102	1	1	2025-02-15	2025-02-27	Delivered	136 Balboa C
8	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine La
9	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door W
10	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL



Subquery in FROM Clause

- Used as temporary table for the main query

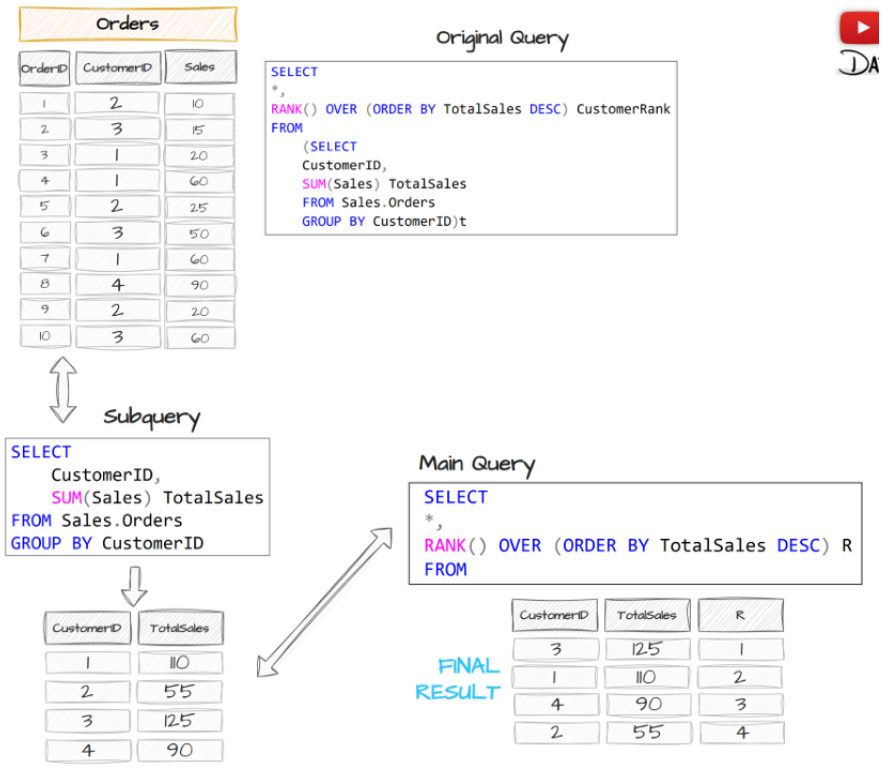



```sql
/* Find the products that have a price higher than
   the average price of all products */

-- Main Query
SELECT
    *
FROM(
    -- Subquery
    SELECT
        ProductID,
        Price,
        AVG(Price) OVER() AvgPrice
    FROM Sales.Products
)t
WHERE Price > AvgPrice;
```

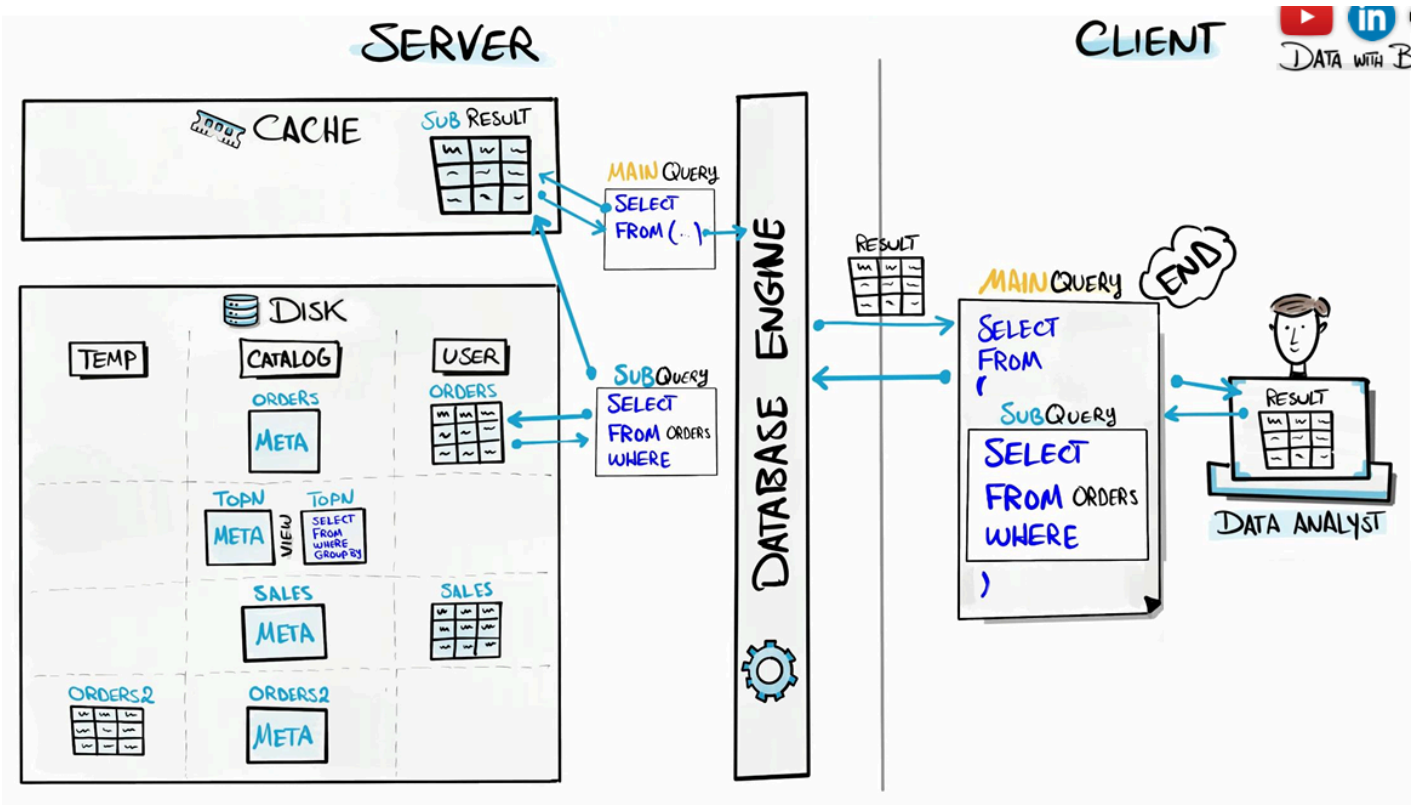
	ProductID	Price	AvgPrice
1	104	25	20
2	105	30	20

```sql
-- Rank customers based on their total amount of sales

-- Main Query
SELECT
    RANK() OVER (ORDER BY TotalSales DESC) CustomerRank,
    *
FROM (
    -- Subquery
    SELECT
        CustomerID,
        SUM(Sales) TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
)t;
```

	CustomerRank	CustomerID	TotalSales
1	1	3	125
2	2	1	110
3	3	4	90
4	4	2	55

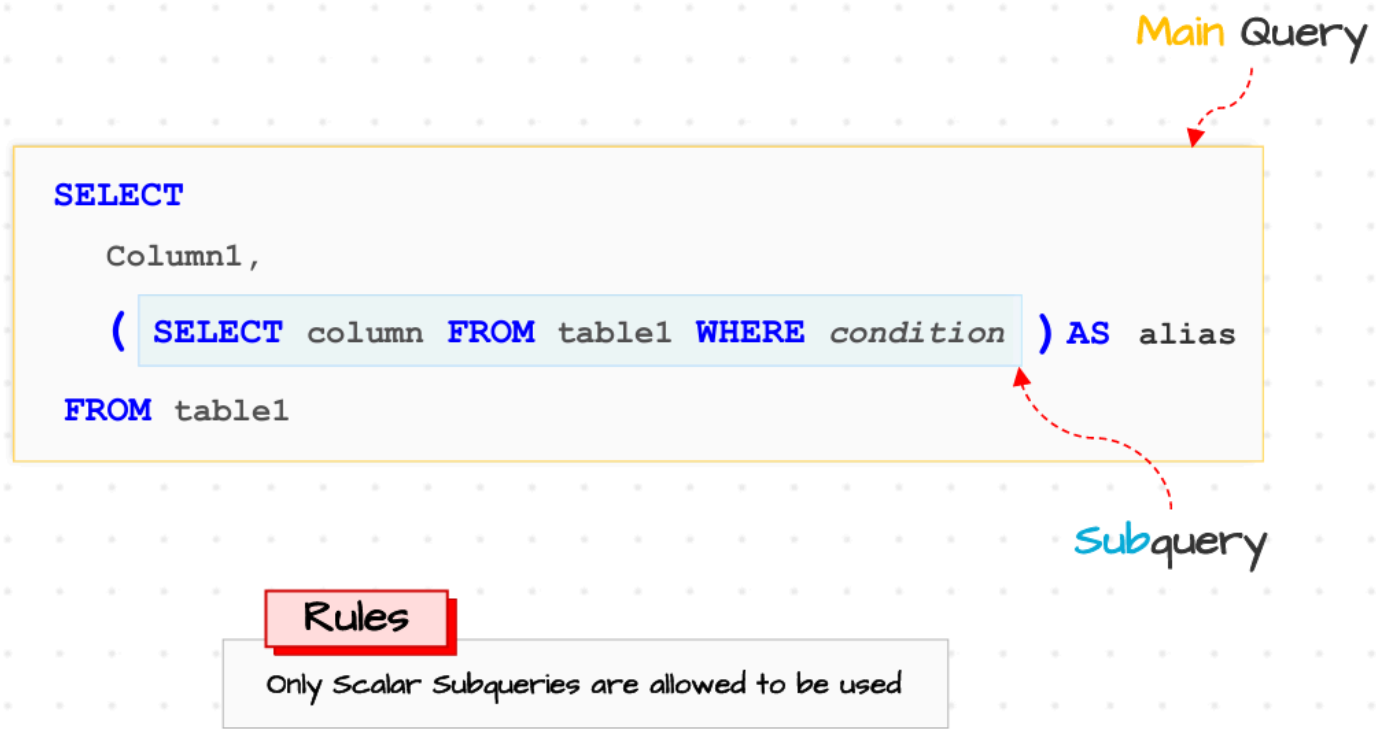


How Database Execute Subqueries



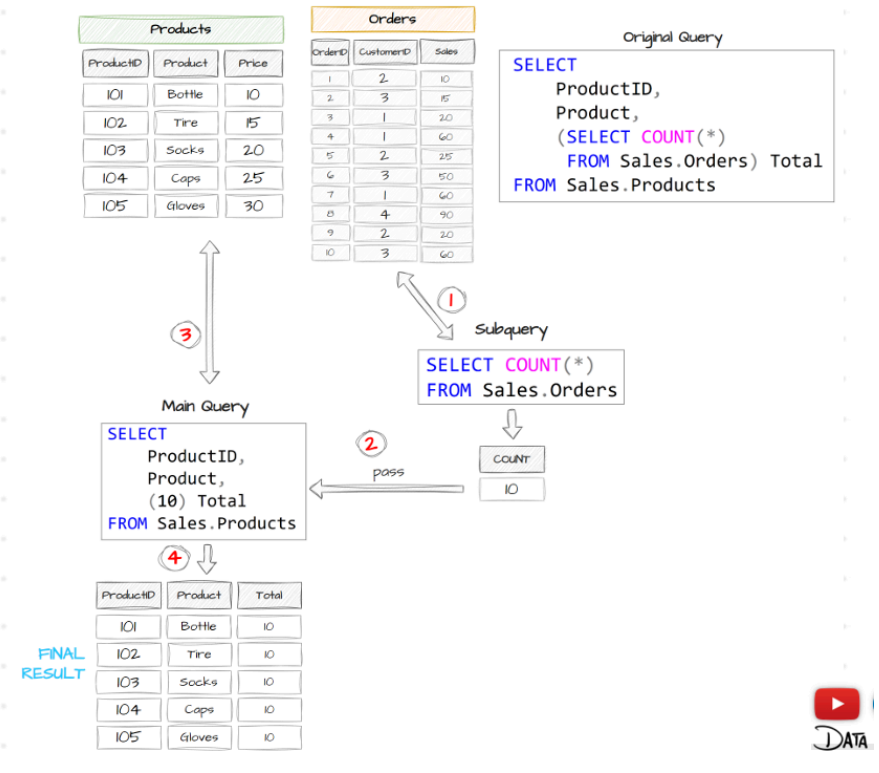
Subquery in SELECT Clause

- Used to aggregate data side by side with the main query's data, allowing for direct comparison
- RULE
  - Only scalar subqueries are allowed to be used




```sql
-- Show the product IDs, names, prices and total number of orders

-- Main Query
SELECT
    ProductID,
    Product,
    Price,
    -- Subquery
    (SELECT
        COUNT(*) TotalOrders
    FROM Sales.Orders) TotalOrders
FROM Sales.Products;
```

	ProductID	Product	Price	TotalOrders
1	101	Bottle	10	10
2	102	Tire	15	10
3	103	Socks	20	10
4	104	Caps	25	10
5	105	Gloves	30	10



Subquery in JOIN Clause

- Used to prepare the data (filtering or aggregation) before joining it with other tables

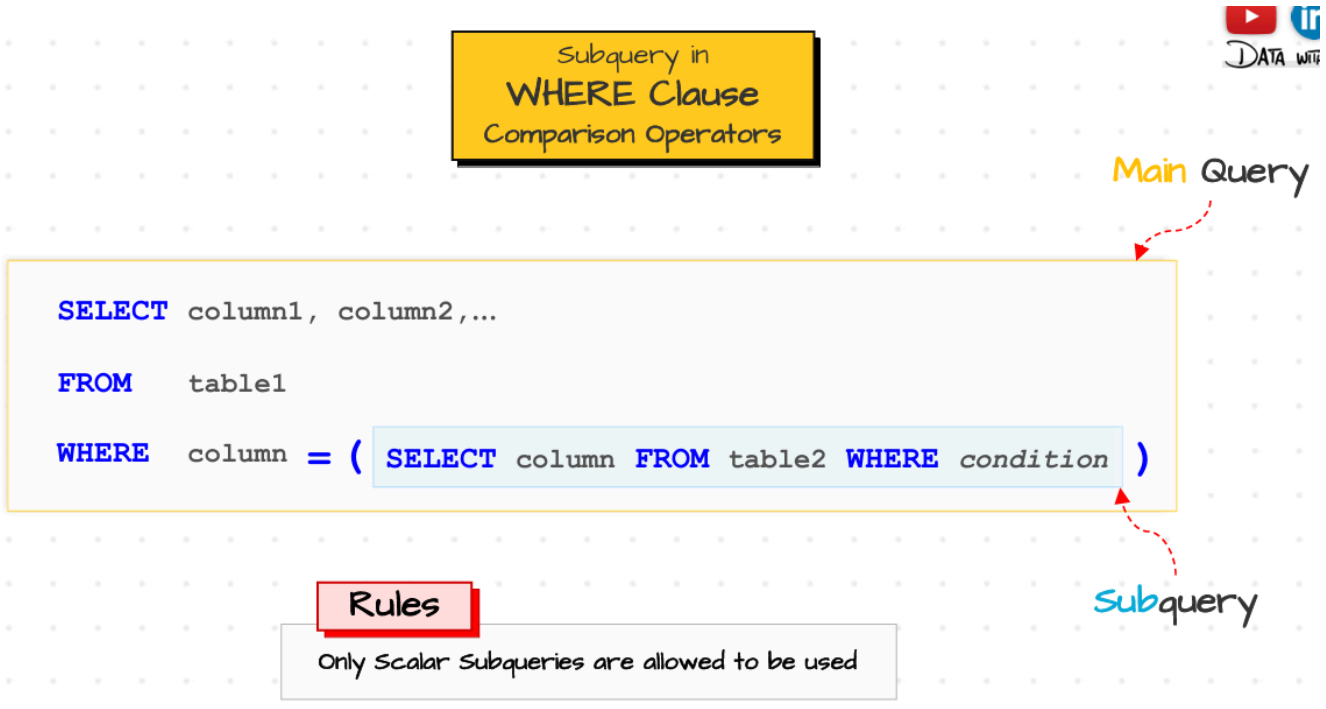
```sql
-- Show all customer details and find the total orders for each customer

-- Main Query
SELECT
    c.*,
    o.TotalOrdersByCustomer
FROM Sales.Customers c
LEFT JOIN (
    -- Subquery
    SELECT
        CustomerID,
        COUNT(*) TotalOrdersByCustomer
    FROM Sales.Orders
    GROUP BY CustomerID
) o
ON c.CustomerID = o.CustomerID;
```

	CustomerID	FirstName	LastName	Country	Score	TotalOrdersByCustomer
1	1	Jossef	Goldberg	Germany	350	3
2	2	Kevin	Brown	USA	900	3
3	3	Mary	NULL	USA	750	3
4	4	Mark	Schwarz	Germany	500	1
5	5	Anna	Adams	USA	NULL	NULL

Subquery in WHERE Clause

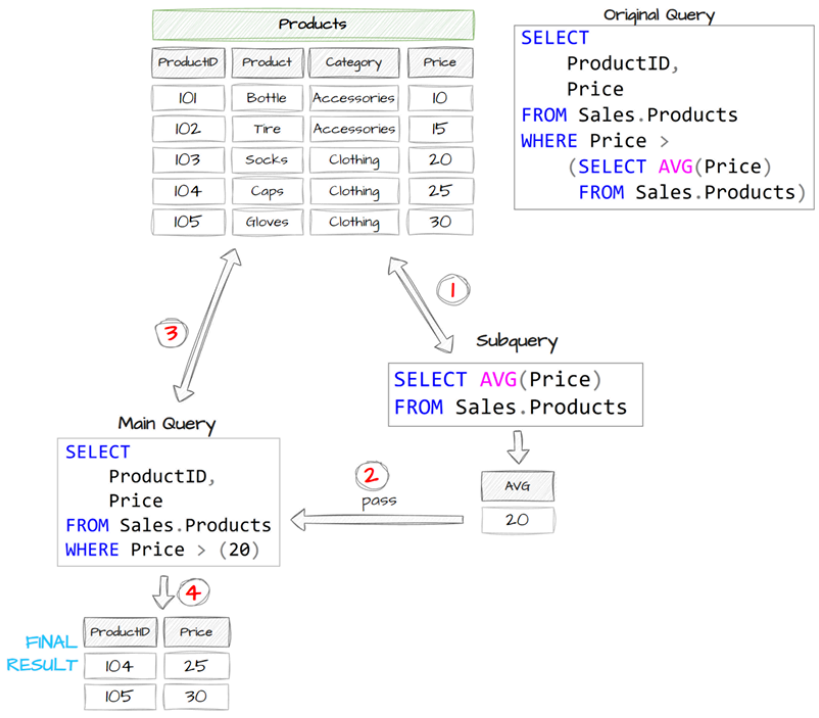
- Used for complex filtering logic and makes query more flexible and dynamic
- Comparison Operators
  - > < = ≠ ≥ ≤
  - Used to filter data by comparing two values
  - RULE
    - Only scalar subqueries are allowed to be used

=	Equal	`WHERE Sales = (SELECT AVG(Sales) FROM ORDERS)`
!= <>	Not Equal	`WHERE Sales != (SELECT AVG(Sales) FROM ORDERS)`
>	Greater than	`WHERE Sales > (SELECT AVG(Sales) FROM ORDERS)`
<	Less than	`WHERE Sales < (SELECT AVG(Sales) FROM ORDERS)`
>=	Greater than or equal to	`WHERE Sales >= (SELECT AVG(Sales) FROM ORDERS)`
<=	Less than or equal to	`WHERE Sales <= (SELECT AVG(Sales) FROM ORDERS)`




```sql
/* Find the products that have a price higher than
   the average price of all products */

-- Main Query
SELECT
    ProductID,
    Price,
(SELECT AVG(Price) FROM Sales.Products) AvgPrice
FROM Sales.Products
-- Subquery
WHERE Price > (SELECT AVG(Price) FROM Sales.Products);
```

	ProductID	Price	AvgPrice
1	104	25	20
2	105	30	20

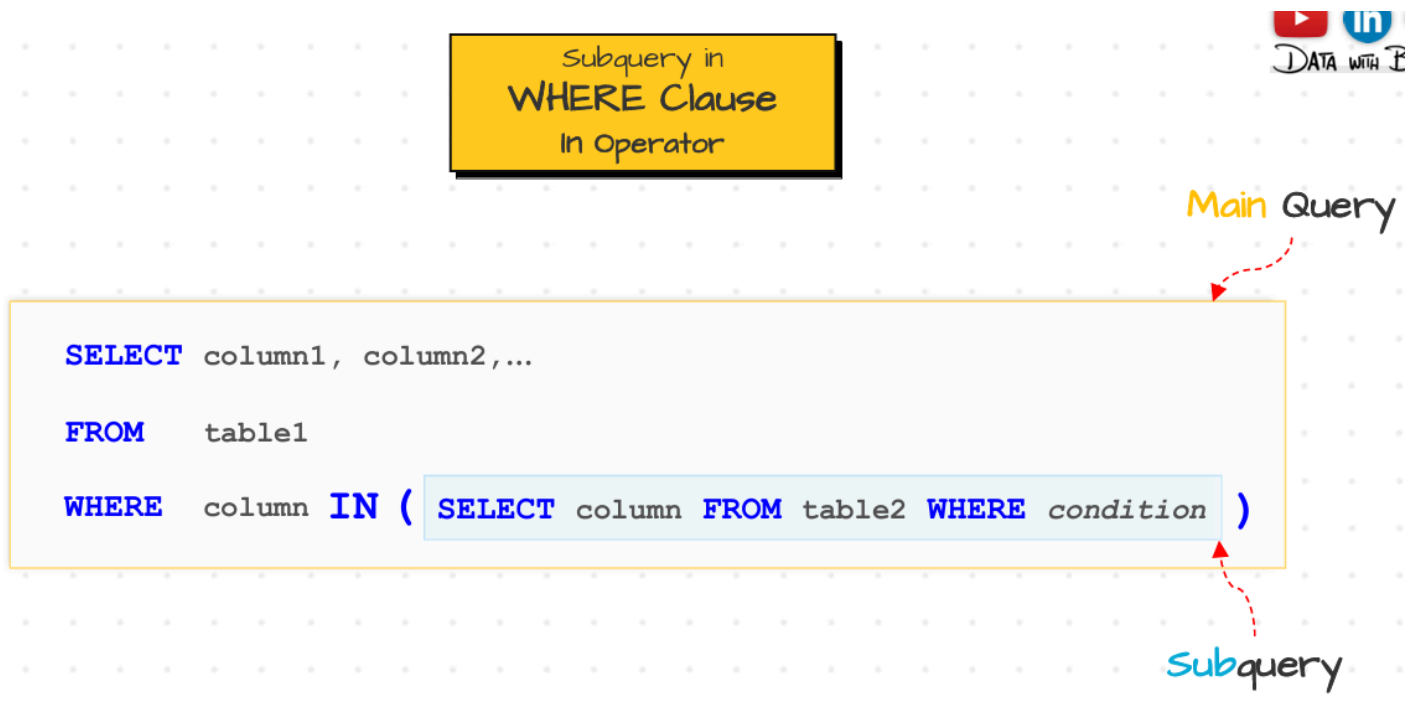


- Logical Operators
  - IN, ANY, ALL, EXISTS
  - Filtering (List of Values)

IN	Checks if a value matches any value in a list	`WHERE Sales IN (SELECT …)`
NOT IN	Checks if a value does not matches any value in a list	`WHERE Sales NOT IN (SELECT …)`
EXISTS	Checks if subquery returns any rows	`WHERE EXISTS (SELECT …)`
NOT EXISTS	Checks if subquery returns no rows	`WHERE NOT EXISTS (SELECT …)`
ANY	Returns true if a value matches any value in a list.	`WHERE Sales < ANY (SELECT …)`
ALL	Returns true if a value matches all values in a list.	`WHERE Sales > ALL (SELECT …)`

Subquery using IN Operator

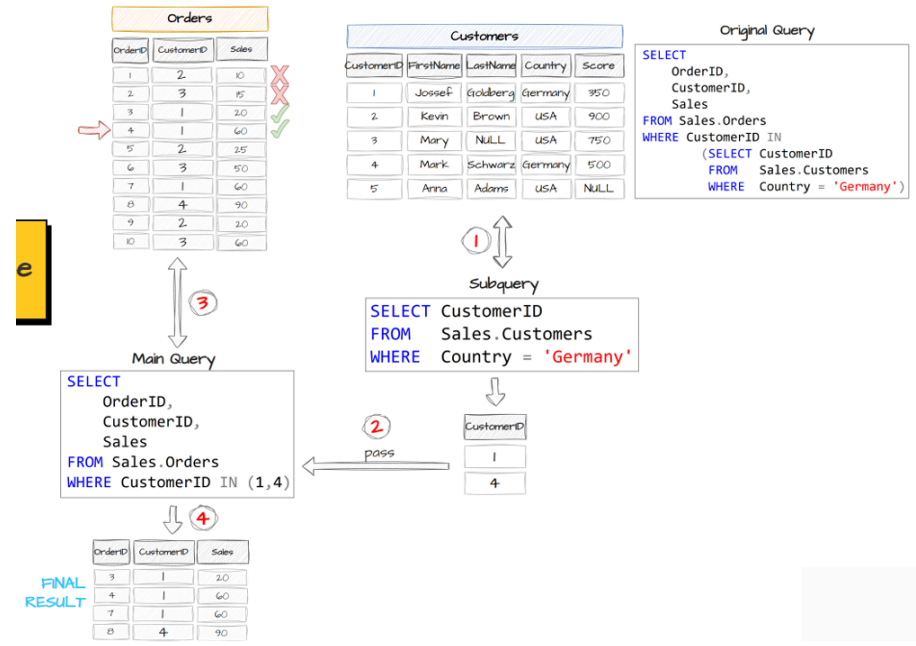
- Checks whether a value matches any value from a list




```
-- Show the details of orders made by customers in Germany

-- Main Query
SELECT
    OrderID,
    OrderDate,
    CustomerID
```

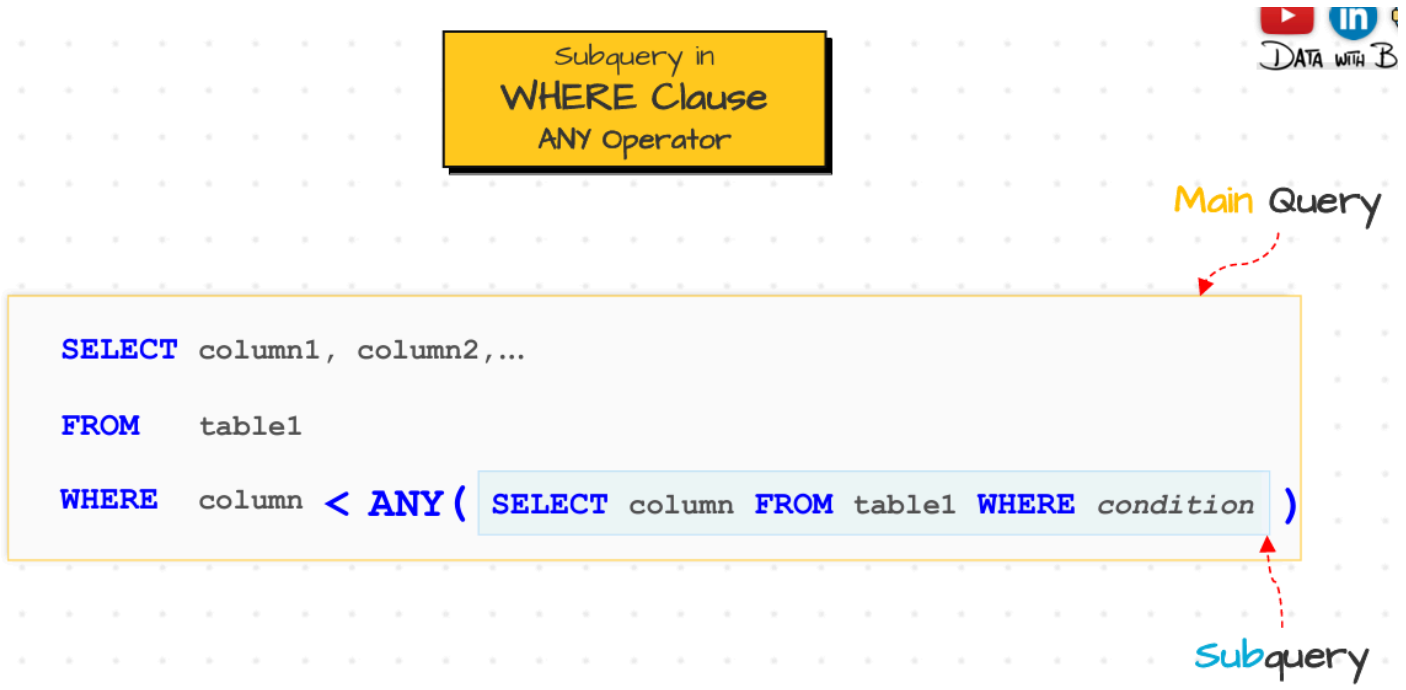
```sql
FROM Sales.Orders
-- Subquery
WHERE CustomerID IN (SELECT
                         CustomerID
                     FROM Sales.Customers
                     WHERE Country = 'Germany');
```

	OrderID	OrderDate	CustomerID
1	3	2025-01-10	1
2	4	2025-01-20	1
3	7	2025-02-15	1
4	8	2025-02-18	4



Subquery using ALL & ANY Operator

- ANY Operator
  - Checks if a value matches ANY value within a list
  - Used to check if a value is true for AT LEAST one of the values in a list




```sql
/* Find female employees whose salaries are greater than
   the salaries of any male employees */

-- Main Query
SELECT
    EmployeeID,
    FirstName,
    Gender,
```

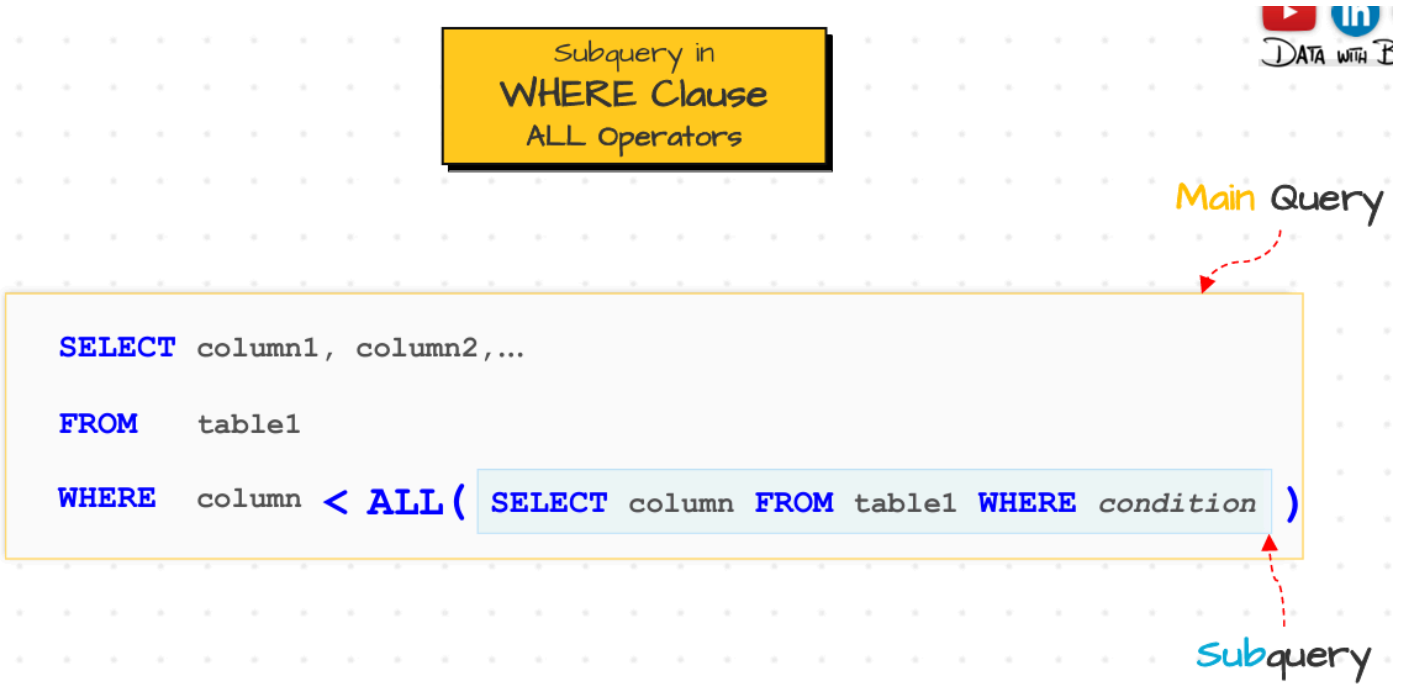
```
    Salary
FROM Sales.Employees
WHERE Gender = 'F' AND
-- Subquery
Salary > ANY (SELECT
                                Salary
                            FROM Sales.Employees
                            WHERE Gender = 'M');
```

	EmployeeID	FirstName	Gender	Salary
1	3	Mary	F	75000

- ALL Operator
  - Checks if a value matches ALL values within a list

```
/* Find female employees whose salaries are greater than
   the salaries of all male employees */

-- Main Query
SELECT
    EmployeeID,
    FirstName,
    Gender,
    Salary
FROM Sales.Employees
WHERE Gender = 'F' AND
-- Subquery
Salary > ALL (SELECT
                                Salary
                            FROM Sales.Employees
                            WHERE Gender = 'M');
```

EmployeeID	FirstName	Gender	Salary



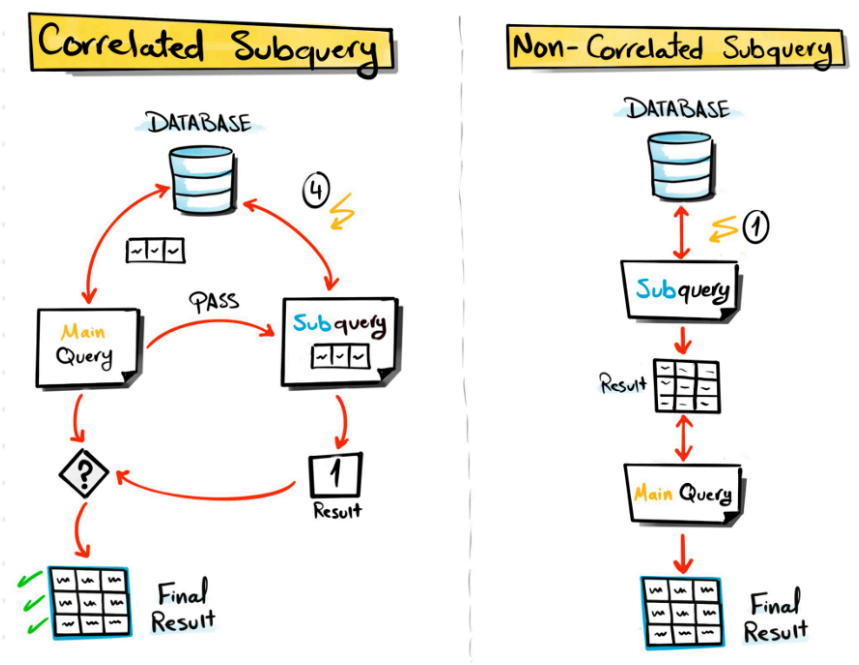
Non-Correlated | Correlated Subqueries

- Non-Correlated Subquery
  - A Subquery that can run independently from the Main Query

- Correlated Subquery
  - A Subquery that relays on values from the Main Query

```sql
-- Show all customer details and find the total orders for each customer
SELECT
    *,
    (SELECT COUNT(*) FROM Sales.Orders o WHERE o.CustomerID = c.CustomerID) TotalOrders
FROM Sales.Customers c;
```

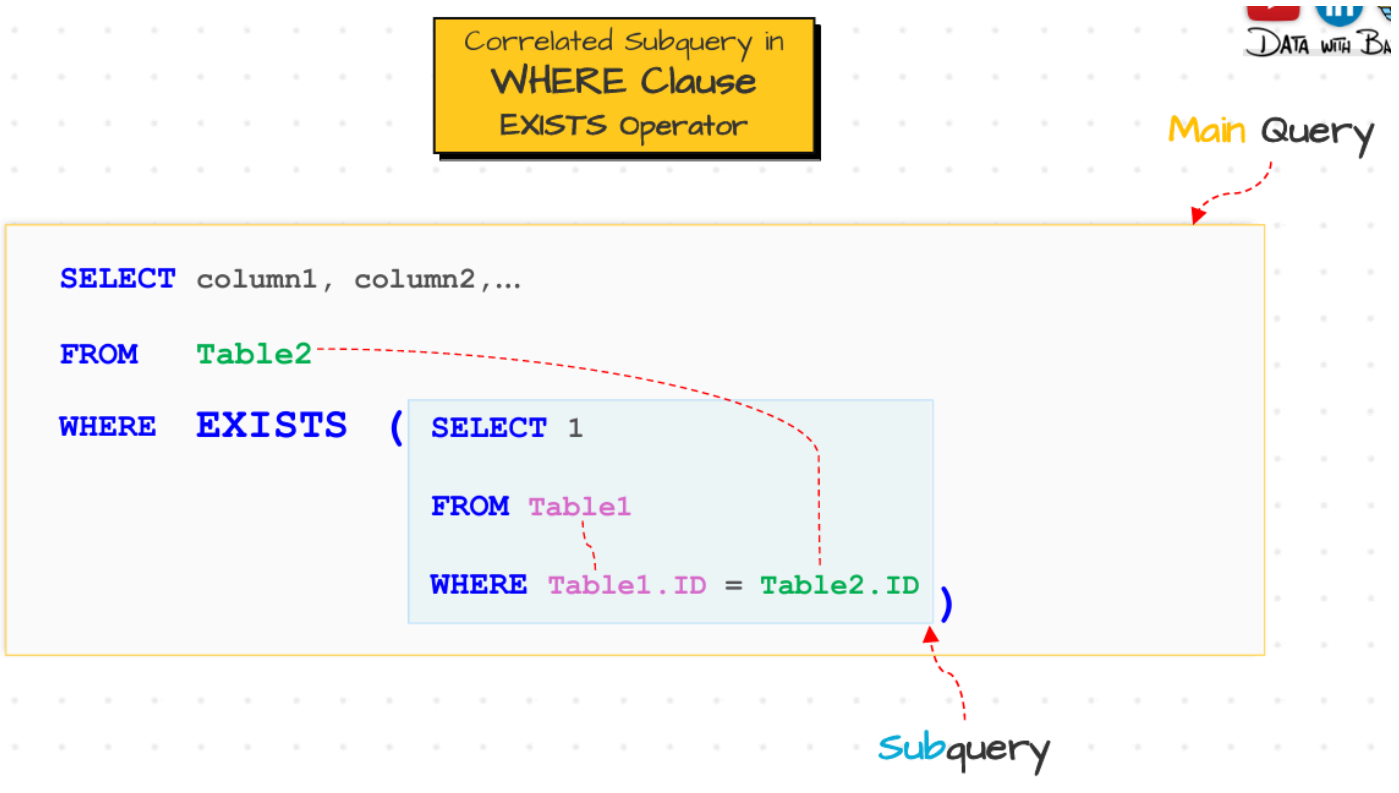
	CustomerID	FirstName	LastName	Country	Score	TotalOrders
1	1	Jossef	Goldberg	Germany	350	3
2	2	Kevin	Brown	USA	900	3
3	3	Mary	NULL	USA	750	3
4	4	Mark	Schwarz	Germany	500	1
5	5	Anna	Adams	USA	NULL	0

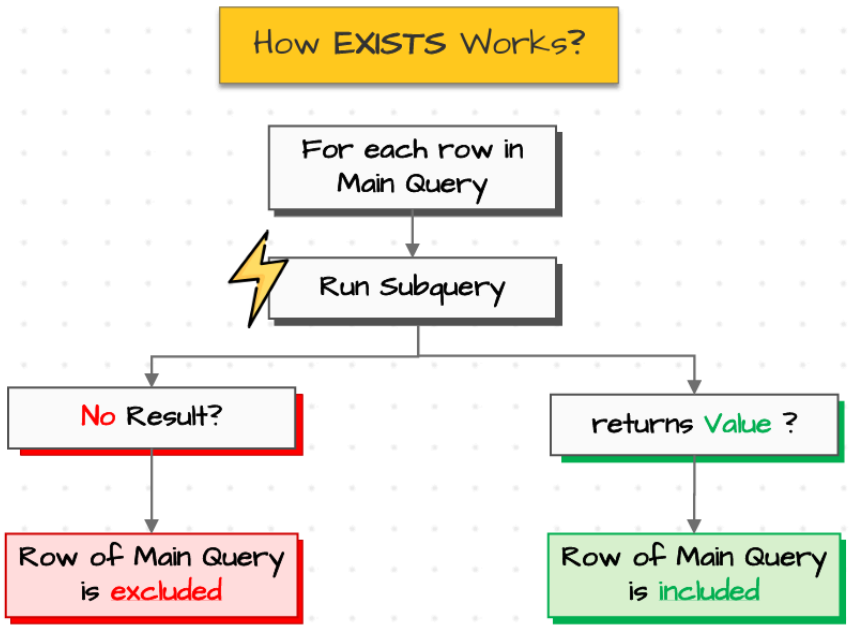


	Non-Correlated Subquery	Correlated Subquery
Definition	Subquery is **independent** of the main query	Subquery is **dependent** of the main query
Execution	Executed **once** and its result is used by the main query **Can** be executed on its Own	Executed for **each row** processed by the main query **Can't be** executed on its Own.
Easy to use	**Easier** to read	**Harder** to read and more **complex**
Performance	Executed **only once** leads to **better** Performance	Executed multiple times leads to **bad** Performance
Usage	Static Comparisons, Filtering with Constants	Row-by-Row Comparisons, Dynamic Filtering

Subquery using EXISTS Operator

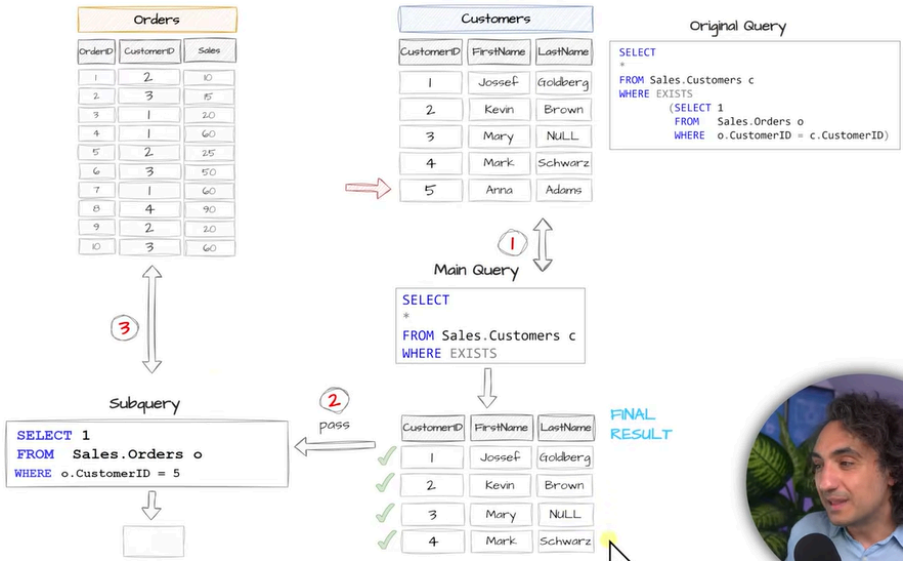
- Check if a subquery returns any rows






```sql
-- Show the details of orders made by customers in Germany
SELECT
    OrderID,
    OrderDate,
    CustomerID
FROM Sales.Orders o
WHERE EXISTS (SELECT 1
              FROM Sales.Customers c
              WHERE c.Country = 'Germany' AND
              c.CustomerID = o.CustomerID);
```

	OrderID	OrderDate	CustomerID
1	3	2025-01-10	1
2	4	2025-01-20	1
3	7	2025-02-15	1
4	8	2025-02-18	4



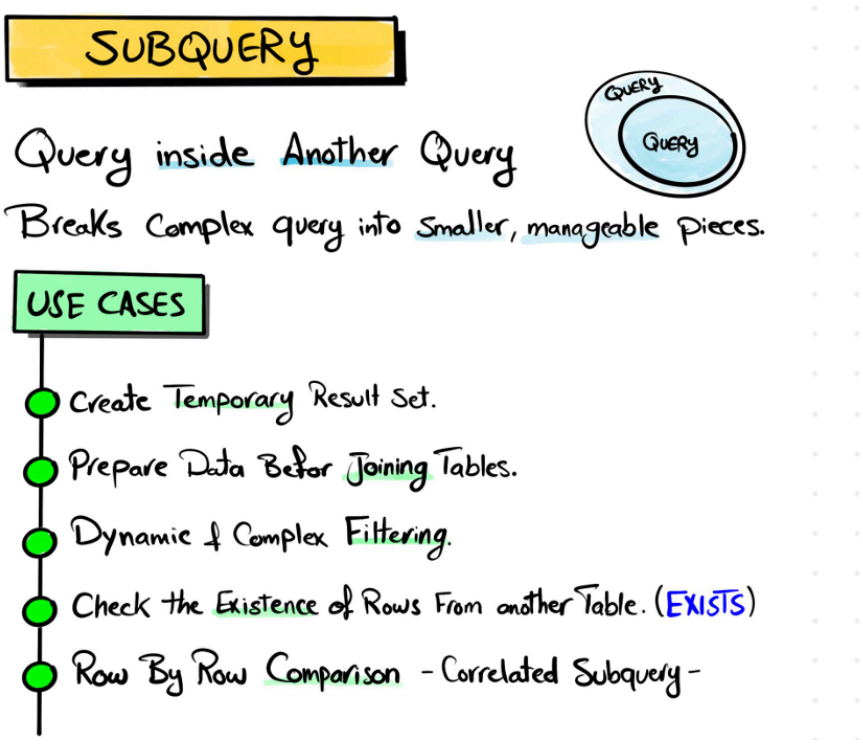
Subquery vs JOINs

	JOINS	SUBQUERIES

Syntax	`SELECT o.*`
	`FROM Orders o`
	`JOIN Customers c`
	`ON c.CustomerID = o.CustomerID`
	`AND c.Country = 'USA'`
		`SELECT *`
		`FROM Orders`
		`WHERE CustomerID IN`
		`(SELECT CustomerID`
		`FROM Customers`
		`WHERE Country = 'USA')`

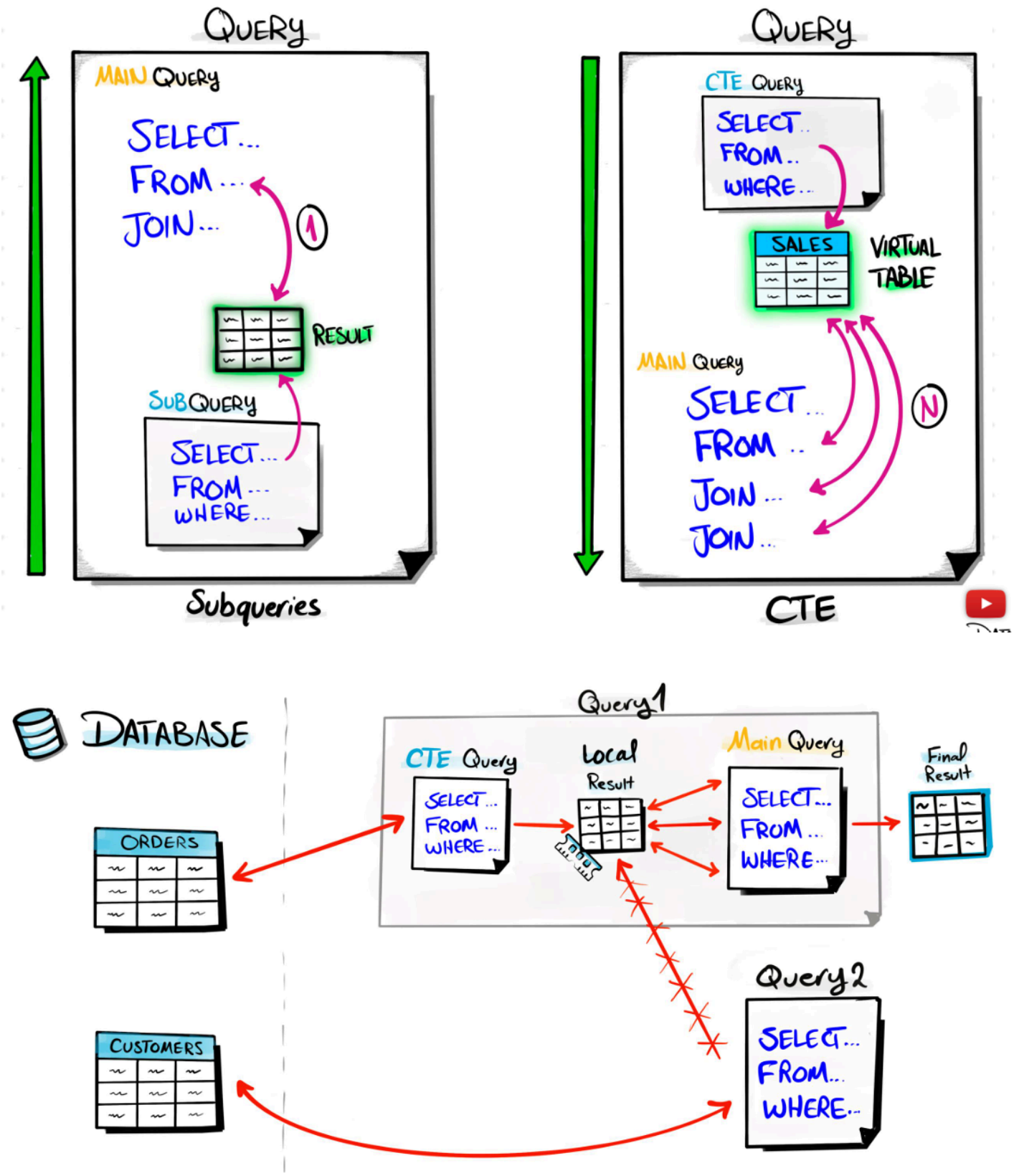
Readability	Not easy to read & maintain	easy to read & maintain
Performance	Fast	Slow
Duplicate	May lead to duplicate	Safer no risk to have duplicates
Best Practices	Useful with Larg tables	Useful with small tables

Summary

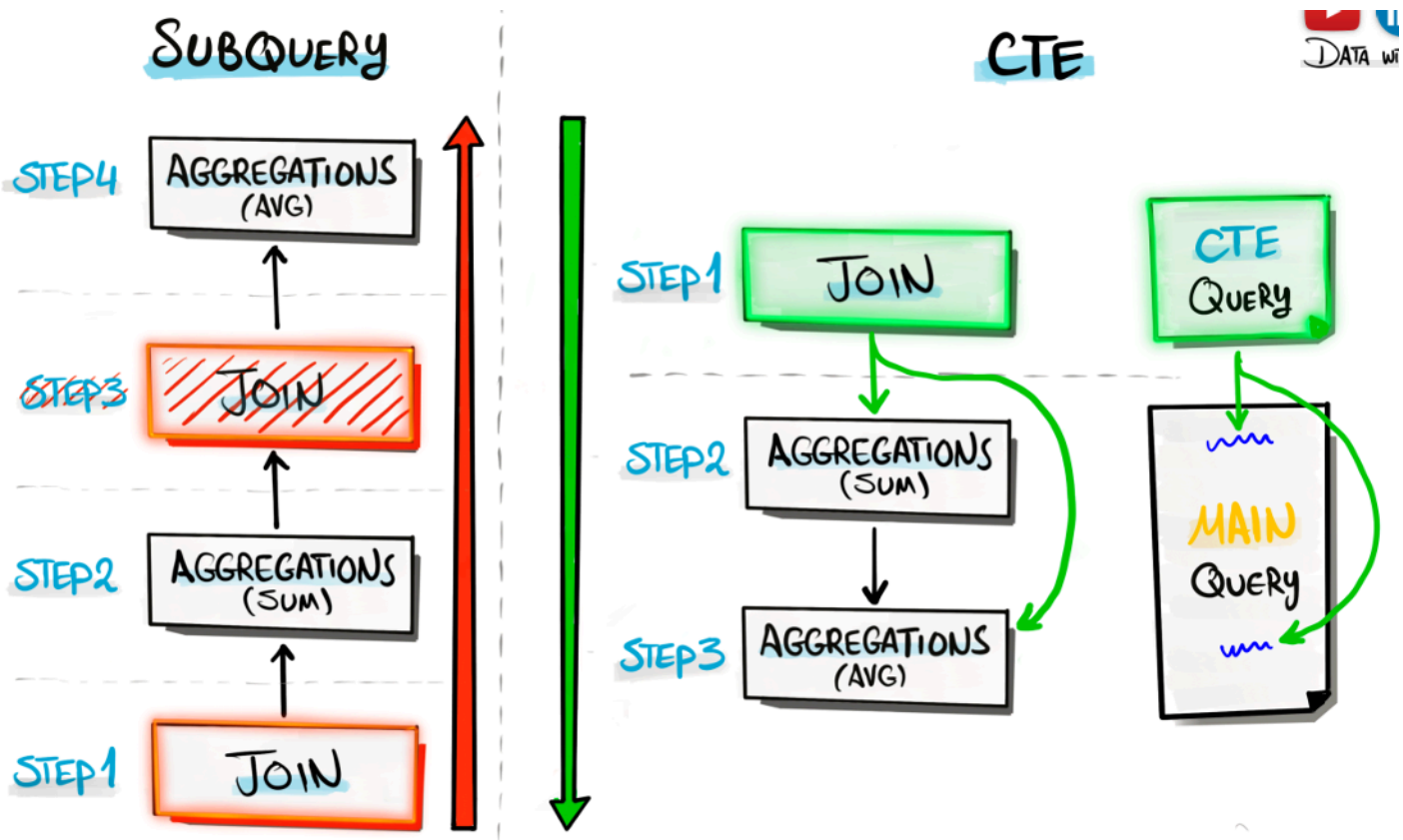


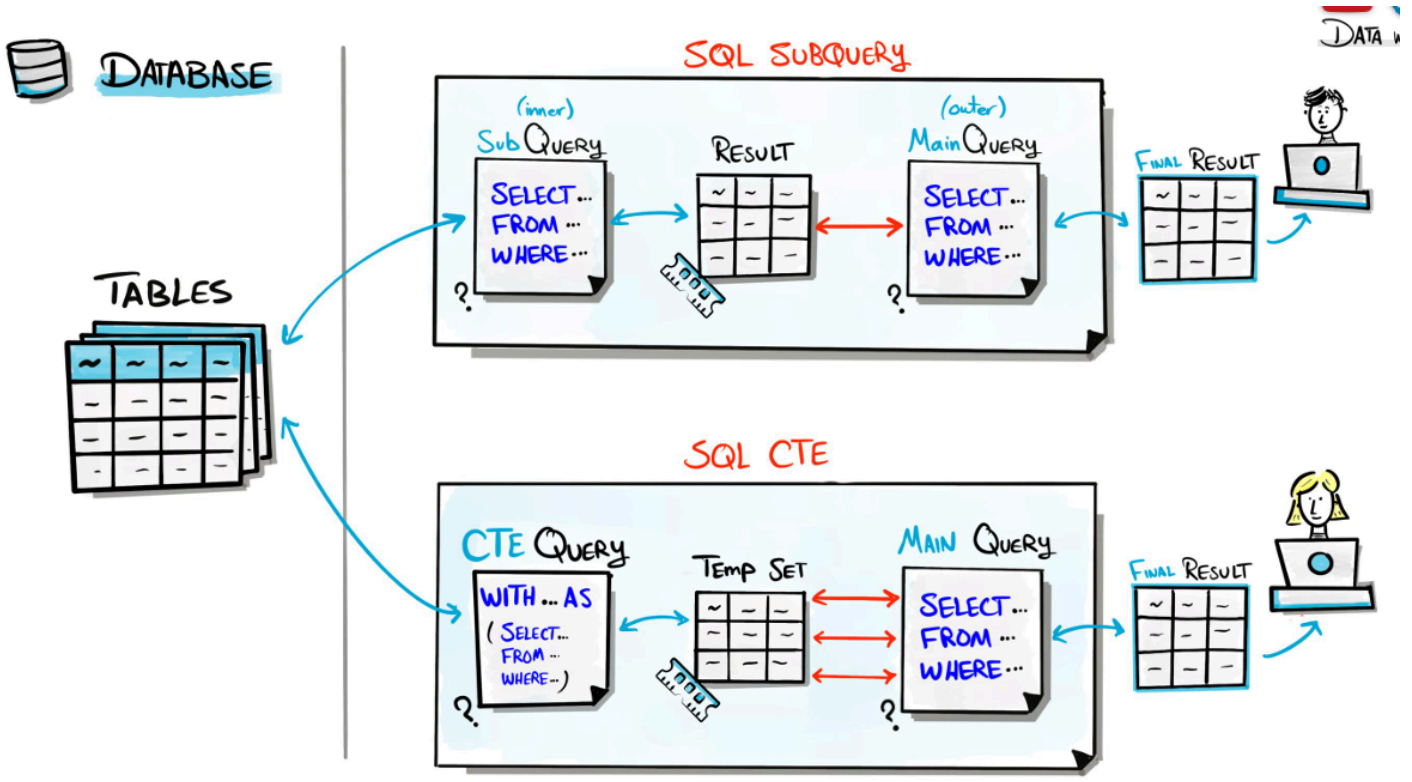
Module 18: Common Table Expression (CTE)

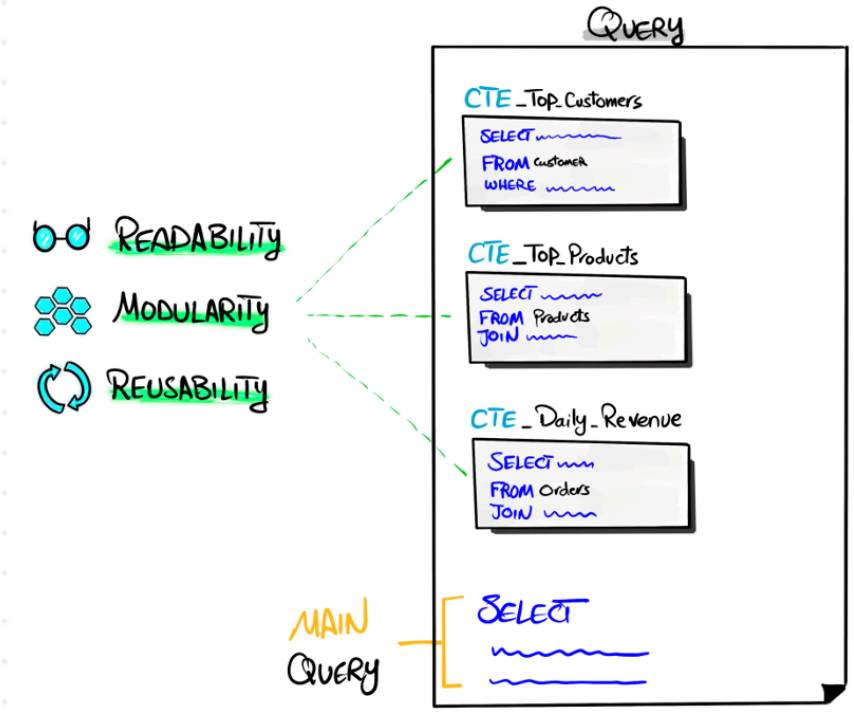
- Temporary, named result set (virtual table), that can be used multiple times within your query to simplify and organize complex query
- CTE-RULE
  - You cannot use ORDER BY directly within the CTE


QUERY
MAIN QUERY
SELECT...
FROM...
JOIN...
1
RESULT
SUBQUERY
SELECT...
FROM...
WHERE...
Subqueries
QUERY
CTE QUERY
SELECT..
FROM..
WHERE..
SALES
VIRTUAL TABLE
MAIN QUERY
SELECT...
FROM..
JOIN...
JOIN...
N
CTE
DATABASE
Query1
CTE Query
SELECT...
FROM...
WHERE...
Local Result
Main Query
SELECT...
FROM...
WHERE...
Final Result
ORDERS
CUSTOMERS
Query2
SELECT...
FROM..
WHERE..

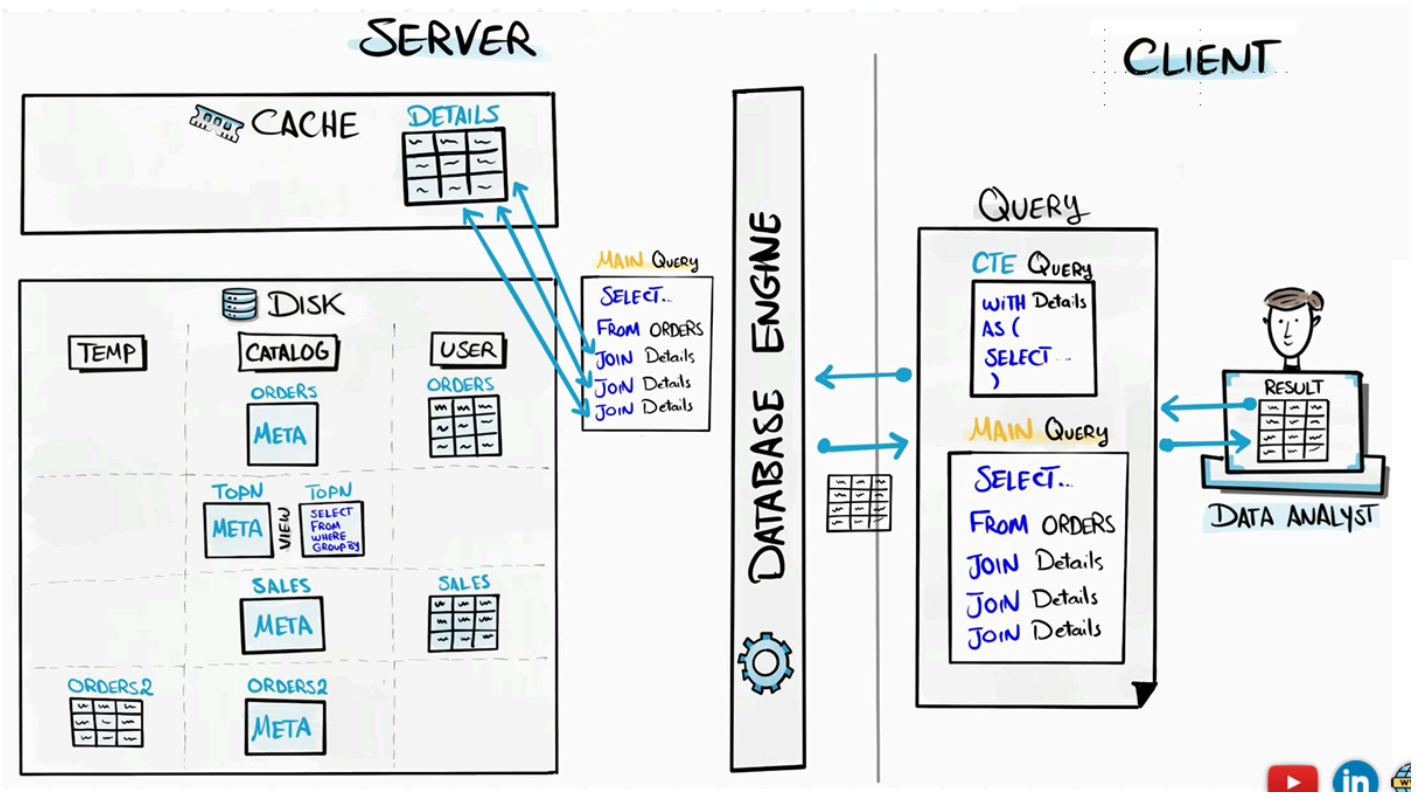
Why CTE ?


SUBQUERY
STEP4
AGGREGATIONS (AVG)
STEP3
JOIN
STEP2
AGGREGATIONS (SUM)
STEP1
JOIN
CTE
STEP1
JOIN
STEP2
AGGREGATIONS (SUM)
STEP3
AGGREGATIONS (AVG)
CTE Query
MAIN Query





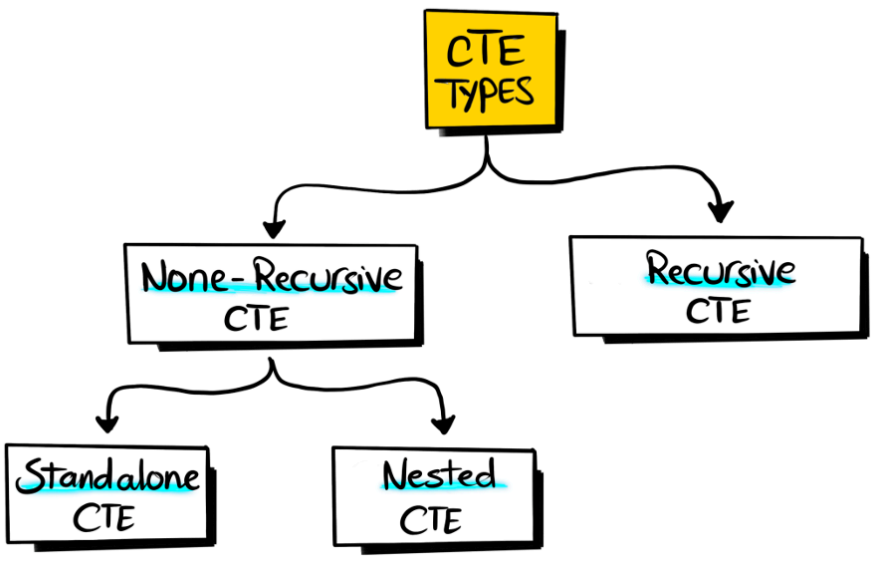
How Database Executes CTE



CTE Types

- None-Recursive CTE
  - Standalone CTE
  - Nested CTE

- Recursive CTE

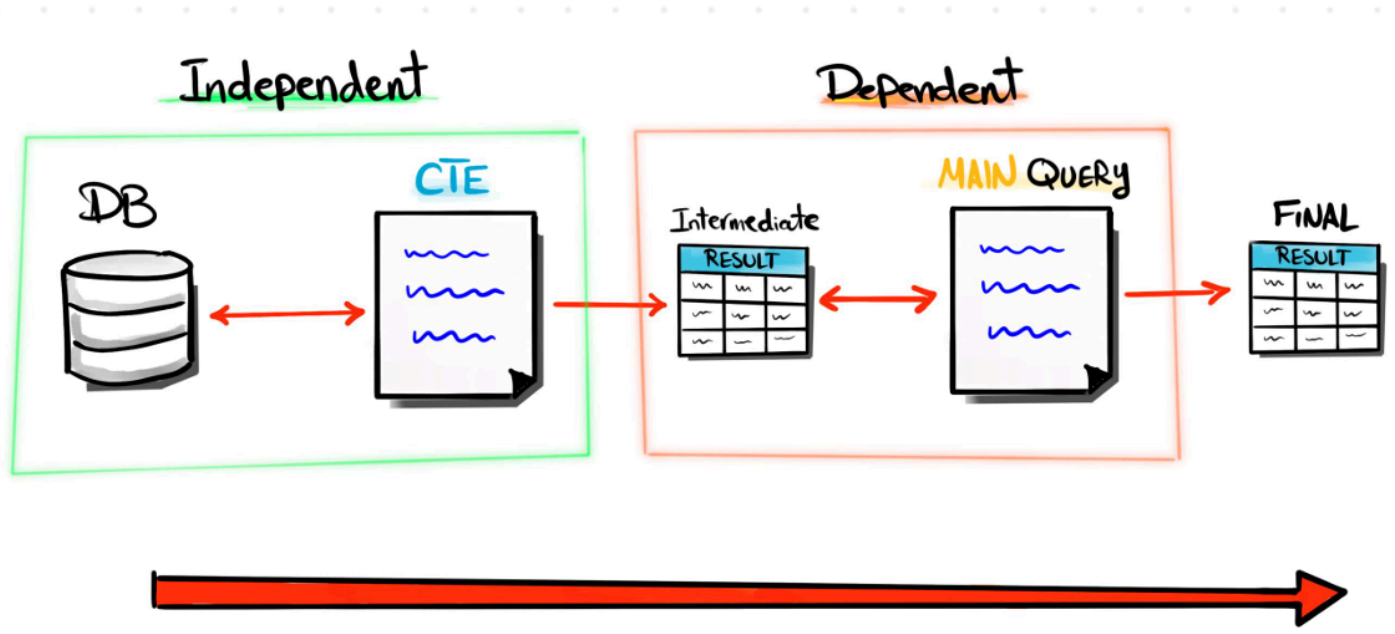


Non-Recursive CTE

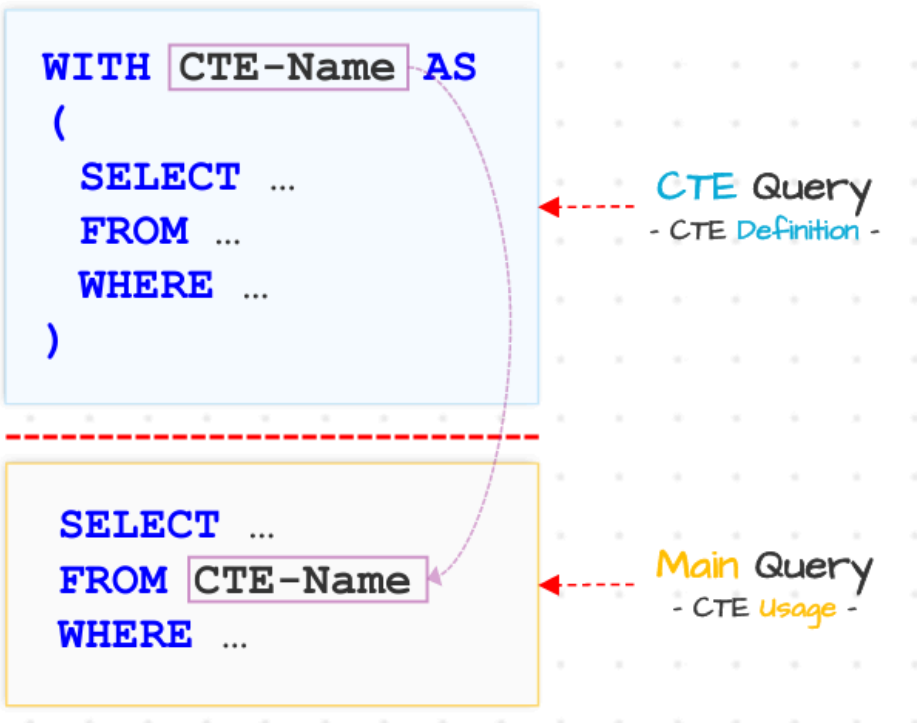
- It is executed only once without any repetition

Standalone CTE

- Defined and Used independently
- Runs independently as it’s self-contained and doesn’t rely on other CTEs or queries



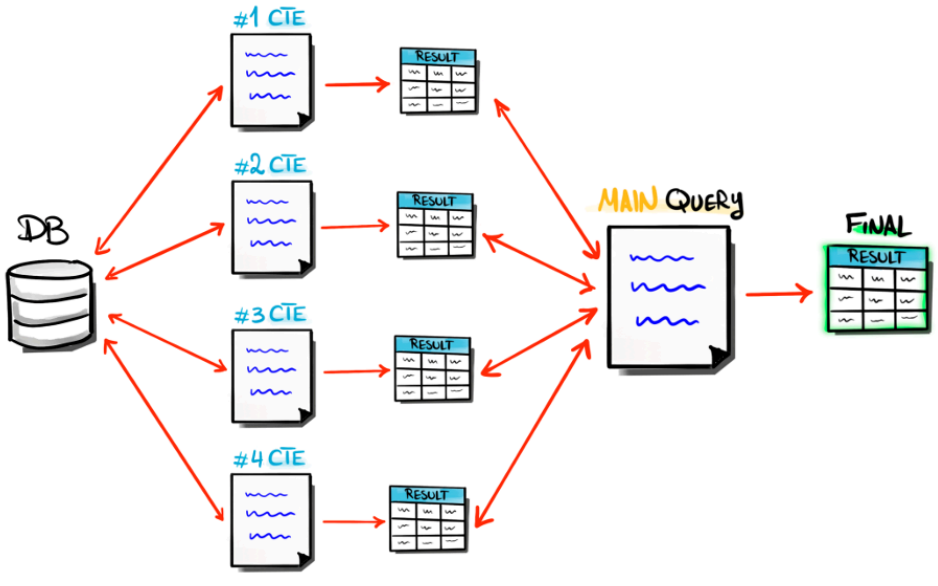
- Syntax



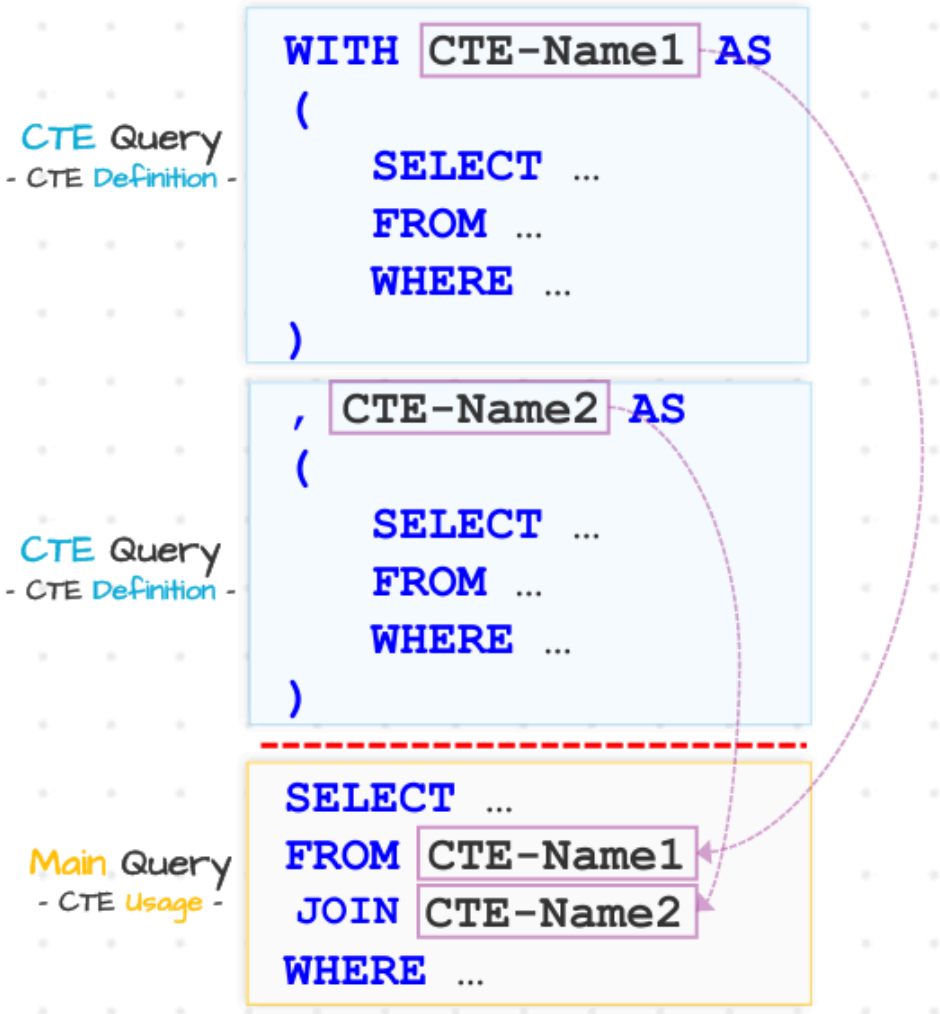
```sql
-- Step1: Find the total sales per customer
WITH CTE_TotalSales AS
(
    SELECT
        CustomerID,
        SUM(Sales) TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
-- Main Query
SELECT
    c.CustomerID,
    c.FirstName,
    cts.TotalSales
FROM Sales.Customers c
LEFT JOIN CTE_TotalSales cts
ON c.CustomerID = cts.CustomerID;
```

	CustomerID	FirstName	TotalSales
1	1	Jossef	110
2	2	Kevin	55
3	3	Mary	125
4	4	Mark	90
5	5	Anna	NULL

Multiple Standalone CTE



- Syntax

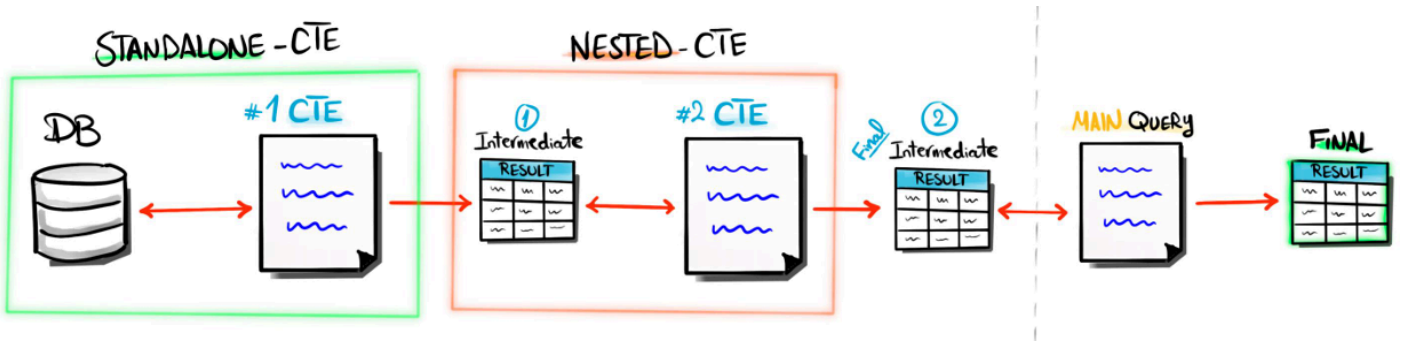



```sql
-- Step1: Find the total sales per customer
WITH CTE_TotalSales AS
(
    SELECT
        CustomerID,
        SUM(Sales) TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
-- Step2: Find the last order date for each customer
, CTE_LastOrder AS
(
    SELECT
        CustomerID,
        MAX(OrderDate) LastOrderDate
    FROM Sales.Orders
    GROUP BY CustomerID
)
-- Main Query
SELECT
    c.CustomerID,
    c.FirstName,
    cts.TotalSales,
    clo.LastOrderDate
FROM Sales.Customers c
LEFT JOIN CTE_TotalSales cts
ON c.CustomerID = cts.CustomerID
LEFT JOIN CTE_LastOrder clo
ON c.CustomerID = clo.CustomerID;
```

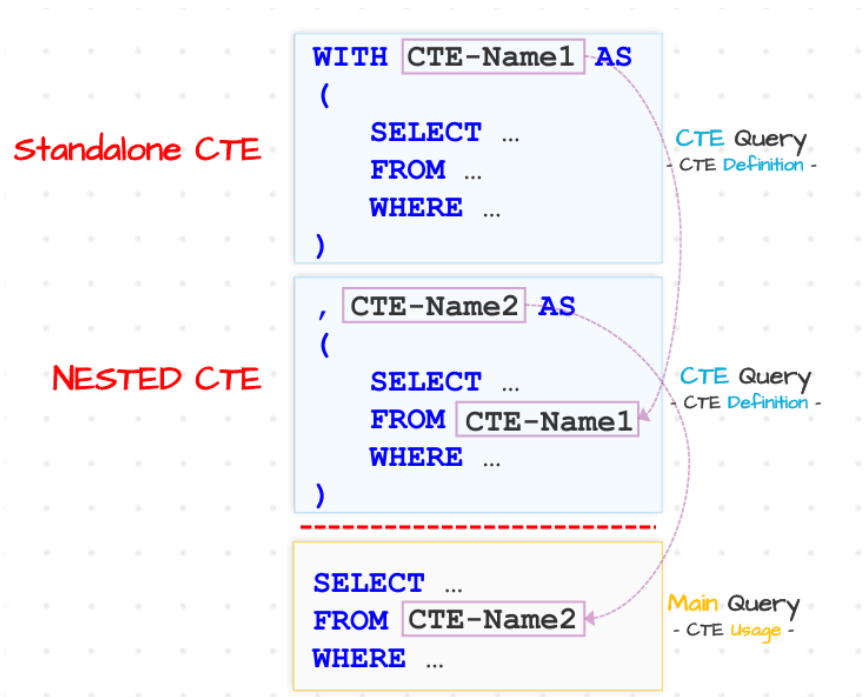
	CustomerID	FirstName	TotalSales	LastOrderDate
1	1	Jossef	110	2025-02-15
2	2	Kevin	55	2025-03-10
3	3	Mary	125	2025-03-15
4	4	Mark	90	2025-02-18
5	5	Anna	NULL	NULL

Nested CTE

- CTE inside another CTE
- A nested CTE uses the result of another CTE, so it can't run independently



- Syntax




```sql
-- Step1: Find the total sales per customer (Standalone CTE)
WITH CTE_TotalSales AS
(
    SELECT
        CustomerID,
        SUM(Sales) TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
-- Step2: Find the last order date for each customer (Standalone CTE)
, CTE_LastOrder AS
(
    SELECT
        CustomerID,
        MAX(OrderDate) LastOrderDate
    FROM Sales.Orders
    GROUP BY CustomerID
)
-- Step3: Rank customers based on total sales per customer (Nested CTE)
, CTE_CustomerRank AS
```

```
(
    SELECT
        CustomerID,
        RANK() OVER (ORDER BY TotalSales DESC) CustomerRank
    FROM CTE_TotalSales
)
-- Step4: Segment customers based on thier total sales (Nested CTE)
, CTE_CustomerSegment AS
(
    SELECT
        CustomerID,
        CASE
            WHEN TotalSales > 100 THEN 'High'
            WHEN TotalSales > 60 THEN 'Medium'
            ELSE 'Low'
        END CustomerSegment
    FROM CTE_TotalSales
)
-- Main Query
SELECT
    c.CustomerID,
    c.FirstName,
    cts.TotalSales,
    clo.LastOrderDate,
    ccr.CustomerRank,
    ccs.CustomerSegment
FROM Sales.Customers c
LEFT JOIN CTE_TotalSales cts
ON c.CustomerID = cts.CustomerID
LEFT JOIN CTE_LastOrder clo
ON c.CustomerID = clo.CustomerID
LEFT JOIN CTE_CustomerRank ccr
ON c.CustomerID = ccr.CustomerID
LEFT JOIN CTE_CustomerSegment ccs
ON c.CustomerID = ccs.CustomerID;
```

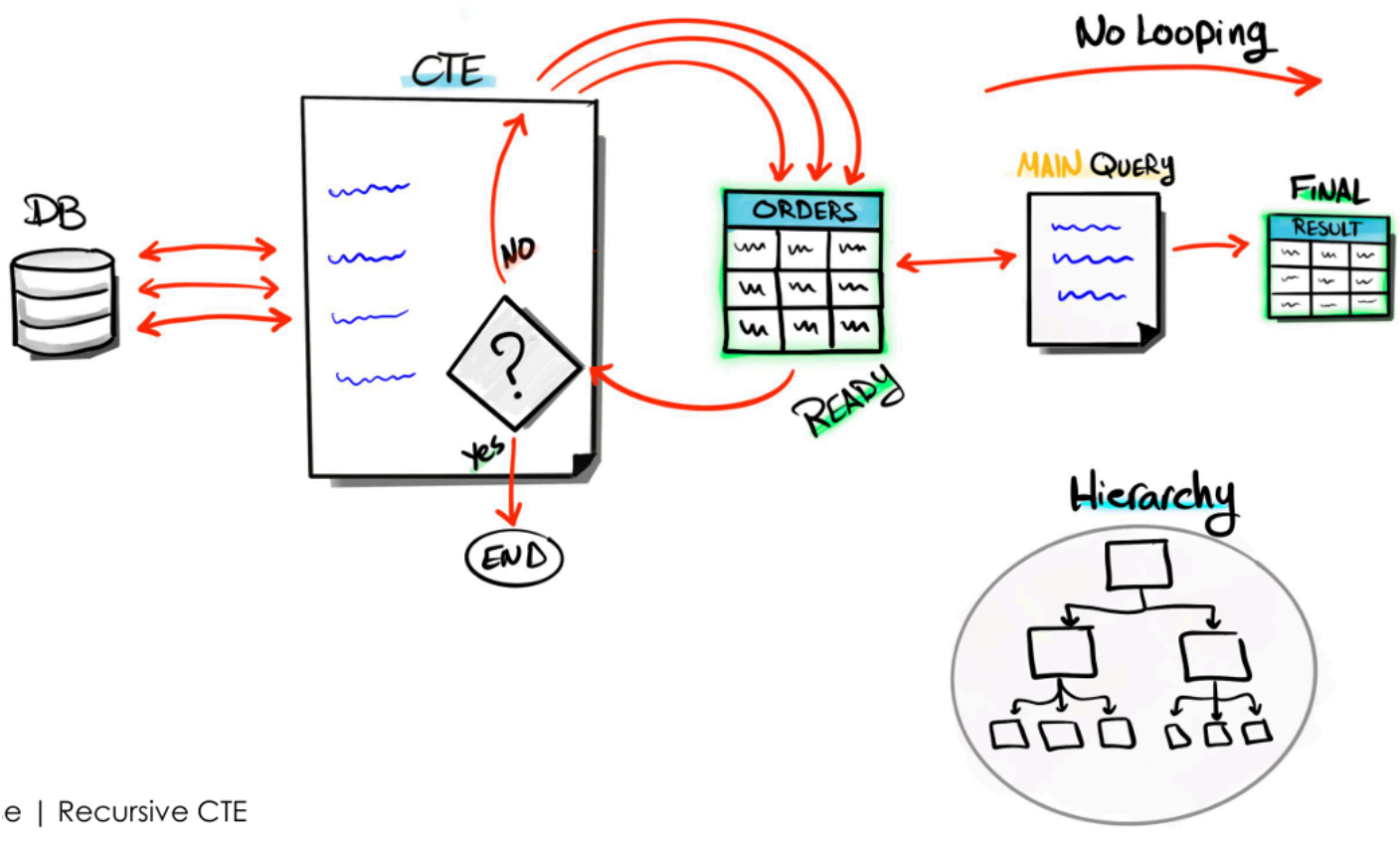
	CustomerID	FirstName	TotalSales	LastOrderDate	CustomerRank	CustomerSegment
1	1	Jossef	110	2025-02-15	2	High
2	2	Kevin	55	2025-03-10	4	Low
3	3	Mary	125	2025-03-15	1	High
4	4	Mark	90	2025-02-18	3	Medium
5	5	Anna	NULL	NULL	NULL	NULL

CTE Best Practices

- Don’t use more than 5 CTEs in one query; otherwise, your code will be hard to understand and maintain
- Rethink and refactor your CTEs before starting a new one

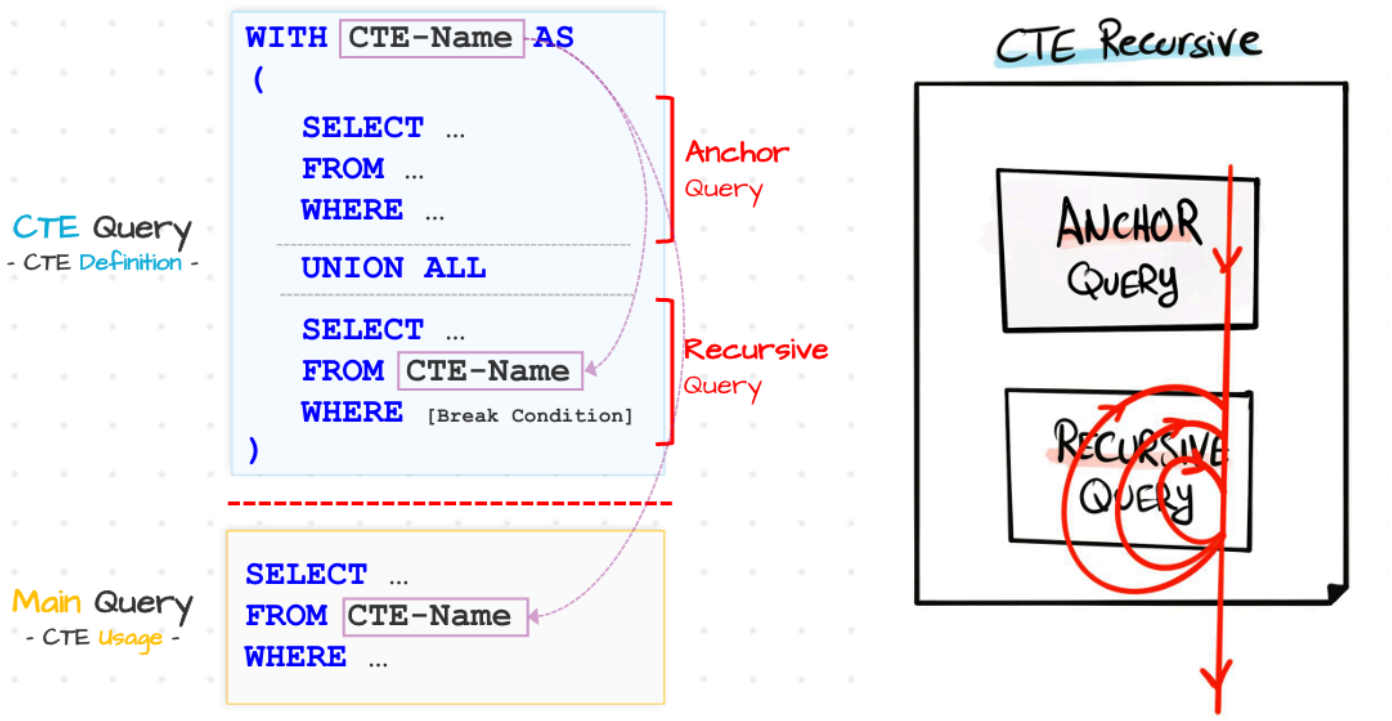
Recursive CTE

- Self-referencing query that repeatedly processes data until a specific condition is met



e | Recursive CTE

- Syntax




```sql
-- Generate a sequence of numbers from 1 to 20
WITH CTE_SequenceNumbers AS
(
    -- Anchor Query
    SELECT
        1 Number
    UNION ALL
    -- Recursive Query
    SELECT
        Number + 1
    FROM CTE_SequenceNumbers
    WHERE Number < 20
)
-- Main Query
SELECT
    Number
FROM CTE_SequenceNumbers;
```

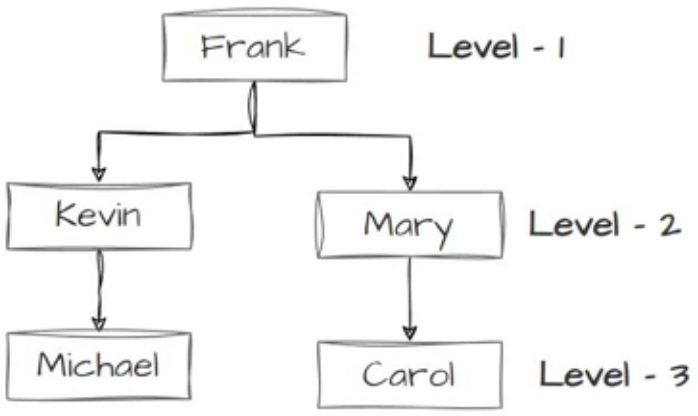
	Number
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20

- Option in Main Query

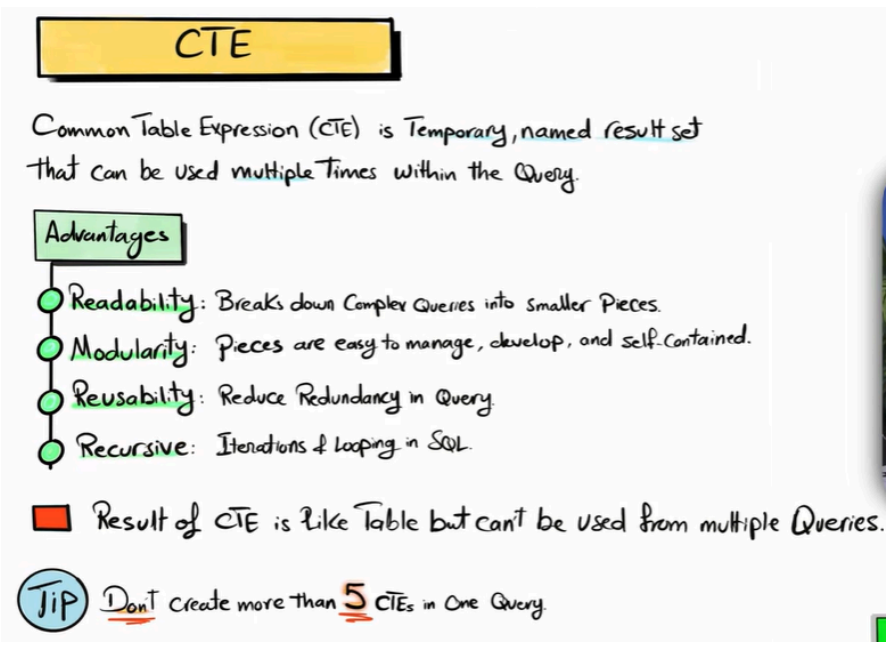
```
OPTION (MAXRECURSION 1000)
```

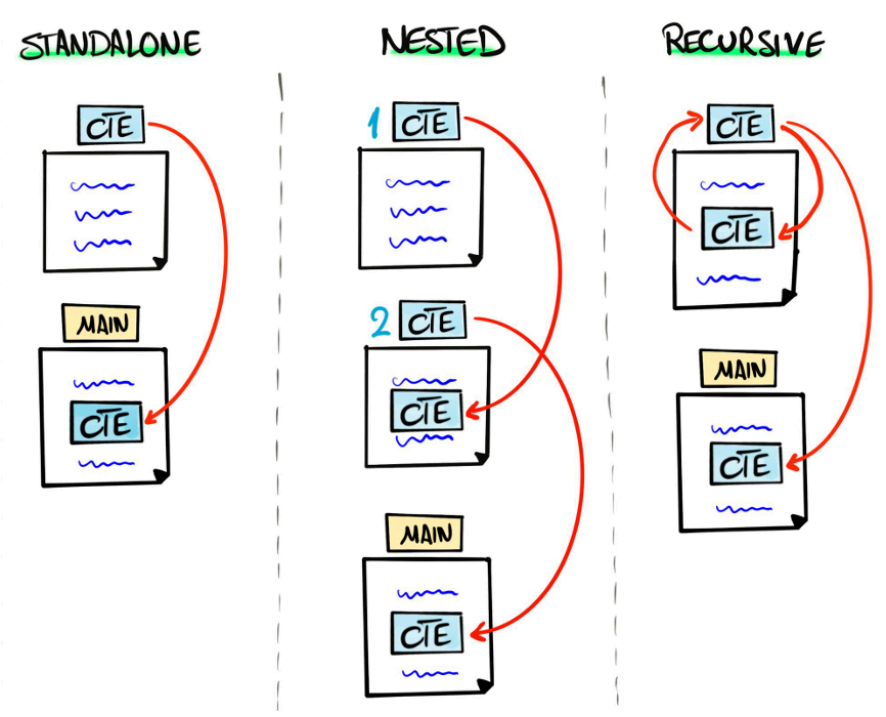
```
/* Show the employee hierarchy by displaying each employee's
   level within the organization */
WITH CTE_EmployeeHierarchy AS
(
    -- Anchor Query
    SELECT
        EmployeeID,
        FirstName,
        ManagerID,
        1 Level
    FROM Sales.Employees
    WHERE ManagerID IS NULL
    UNION ALL
    -- Recursive Query
    SELECT
        e.EmployeeID,
        e.FirstName,
        e.ManagerID,
        ceh.Level + 1 Level
    FROM Sales.Employees e
    INNER JOIN CTE_EmployeeHierarchy ceh
    ON e.ManagerID = ceh.EmployeeID
)
-- Main Query
SELECT
    *
FROM CTE_EmployeeHierarchy;
```

	EmployeeID	FirstName	ManagerID	Level
1	1	Frank	NULL	1
2	2	Kevin	1	2
3	3	Mary	1	2
4	5	Carol	3	3
5	4	Michael	2	3



Summary



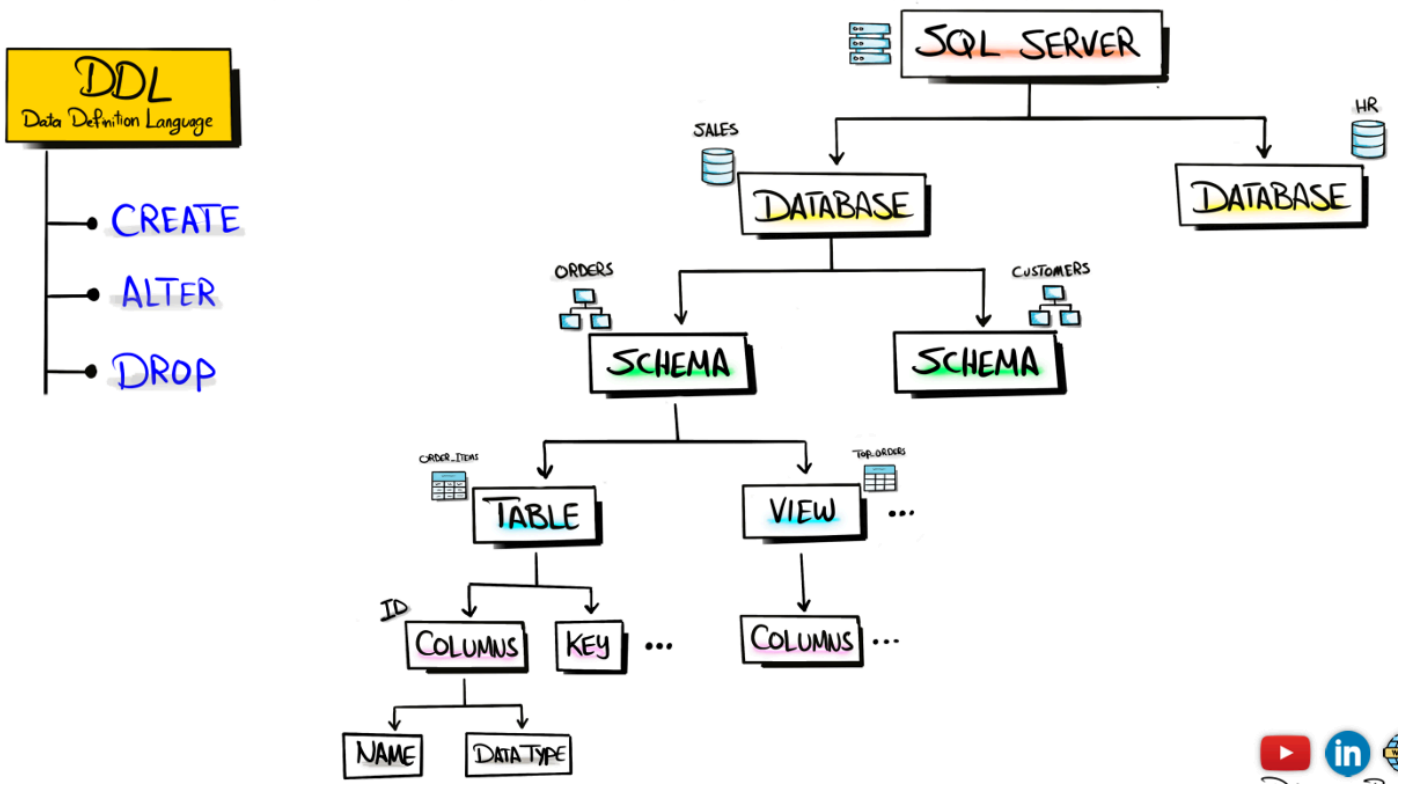


Module 19: Views

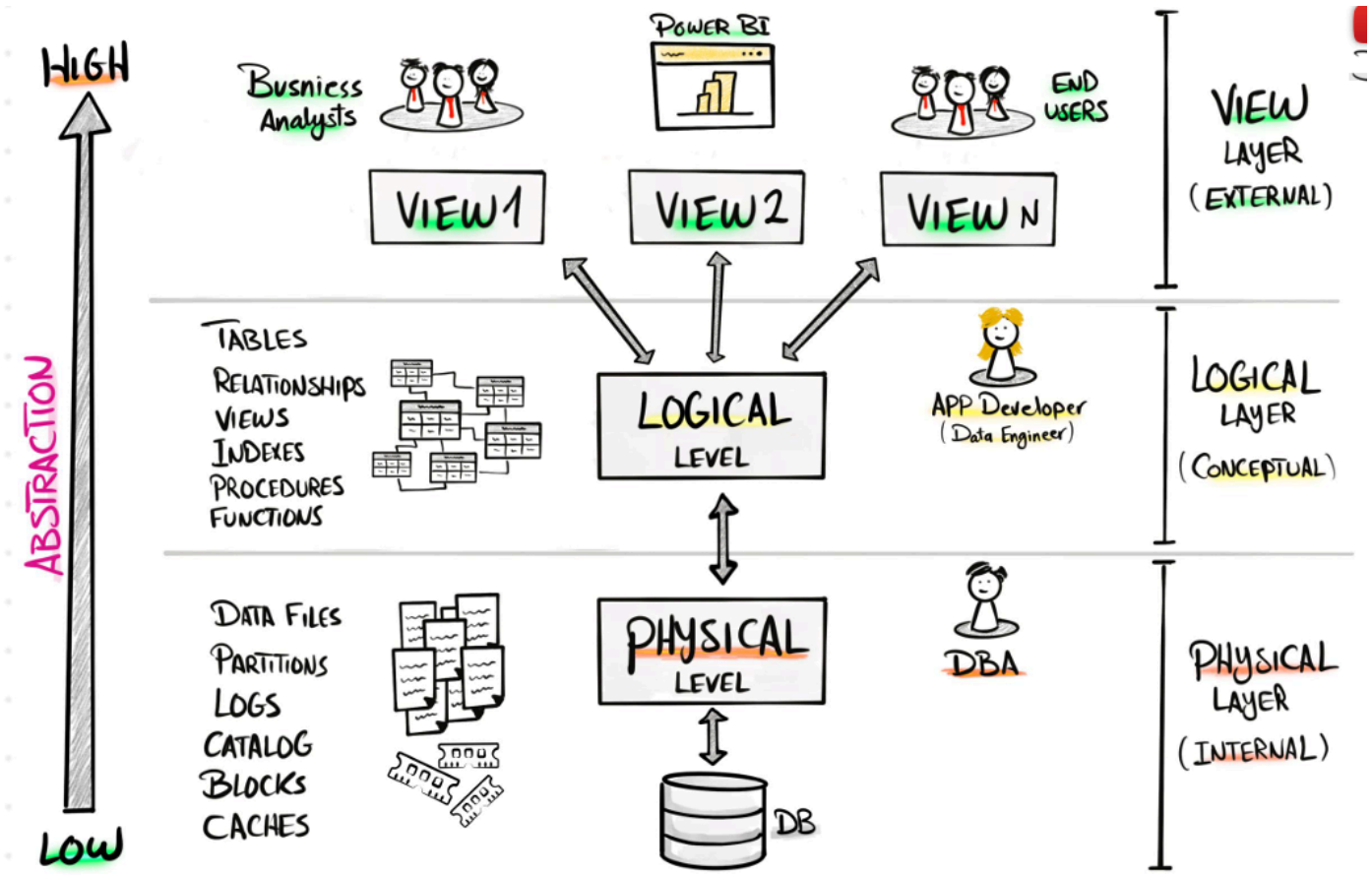
Database Structure

- Database Server
  - Stores, manages, and provides access to databases for users or applications
- Database
  - Collection of information that is stores=d in a structured way
- Schema
  - Logical layer that groups related objects together
- Table
  - A place where data is stored and organized into rows and columns

- View
  - A virtual table that shows data without storing it physically
- DDL (Data Definition Language)
  - A set of commands that allows us to define and manage the structure of a database

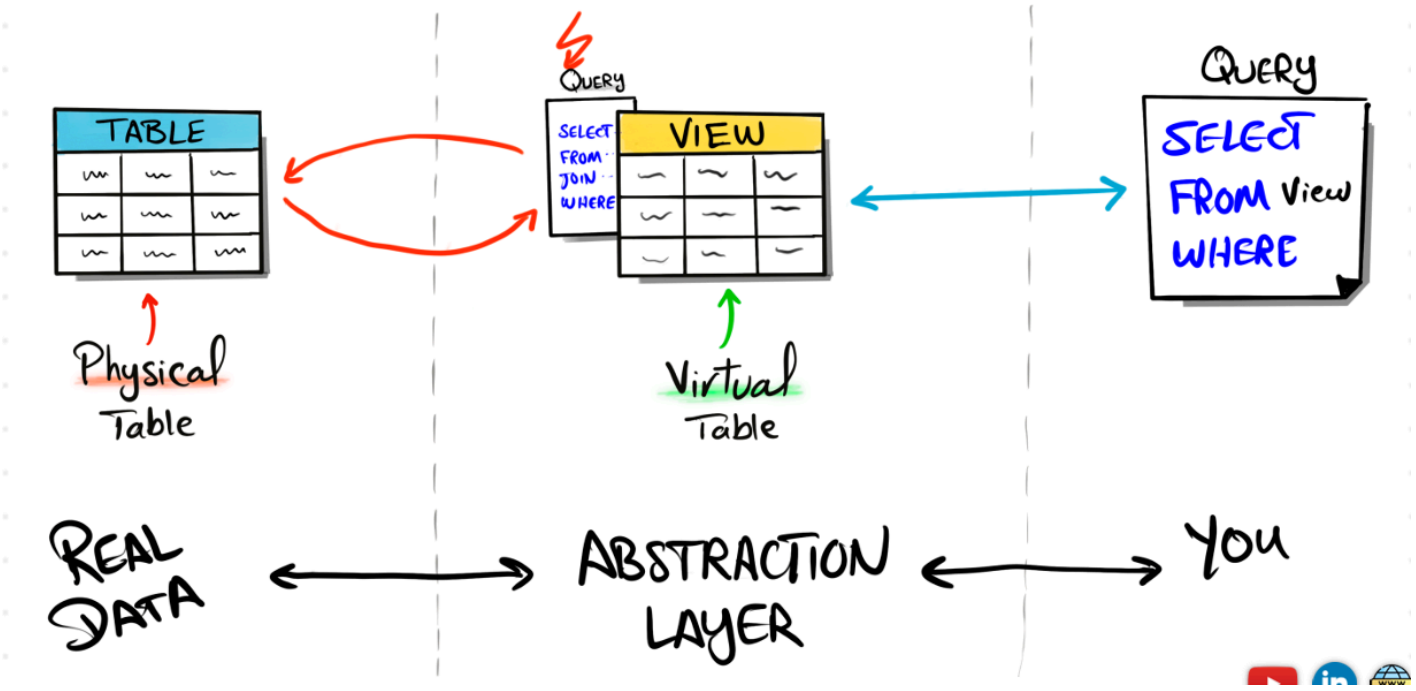


3 Levels Architecture



What is View?

- Virtual table based on the result set of a query, without storing the data in database
- Views are persisted SQL queries in the database

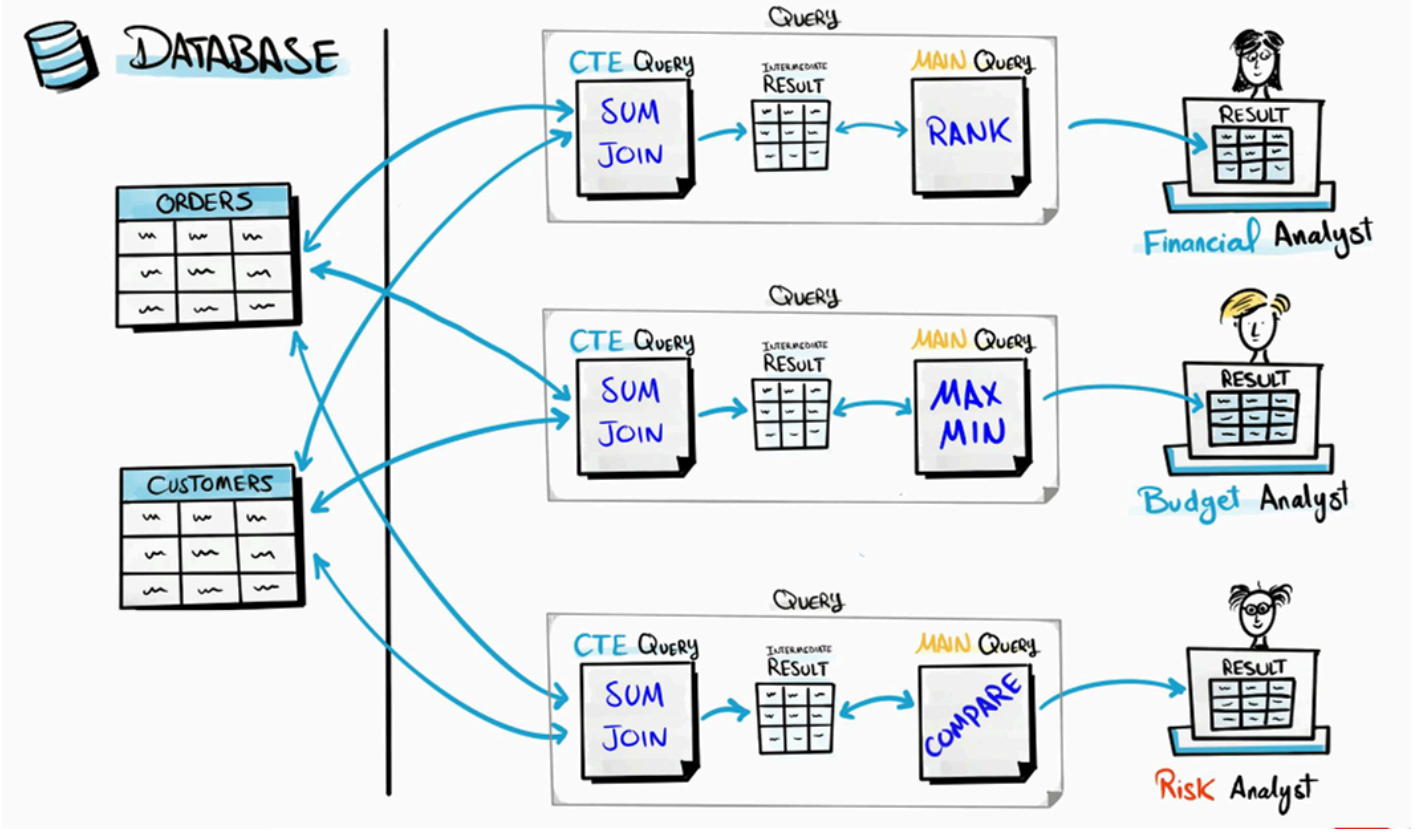


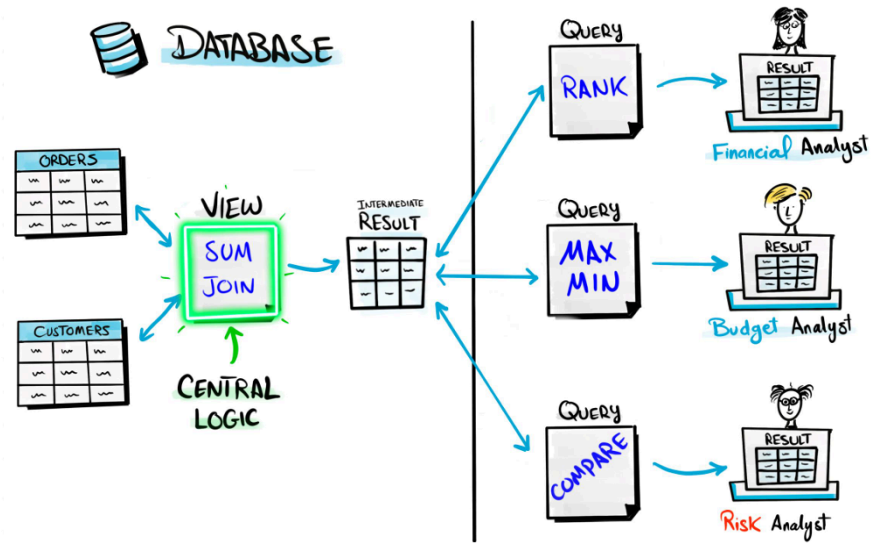
VIEW vs TABLE

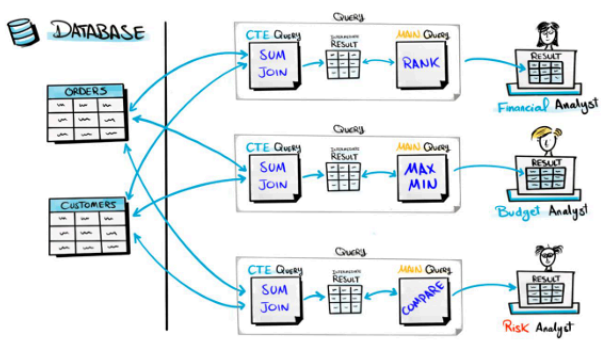
VIEW	TABLE
No Persistance	Persisted Data
Easy to Maintain	Hard to Maintain
Slow Response	Fast Response
Read	Read / Write
Flexible	Hard to Change

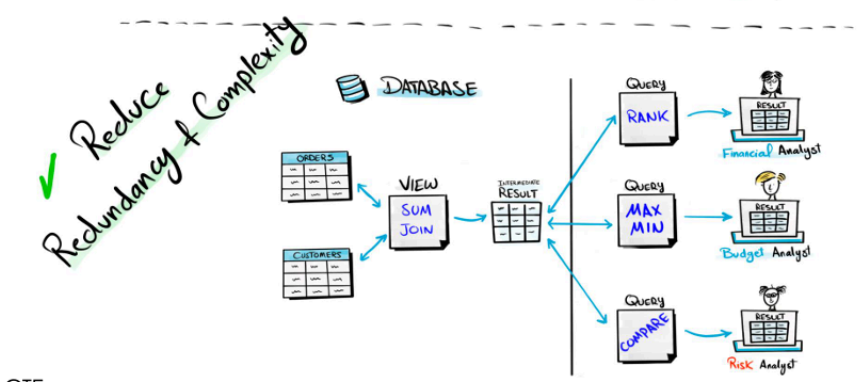
VIEWS USE CASE: Central Complex Query Logic

- Store central, complex query logic in the database for access by multiple queries, reducing project complexity

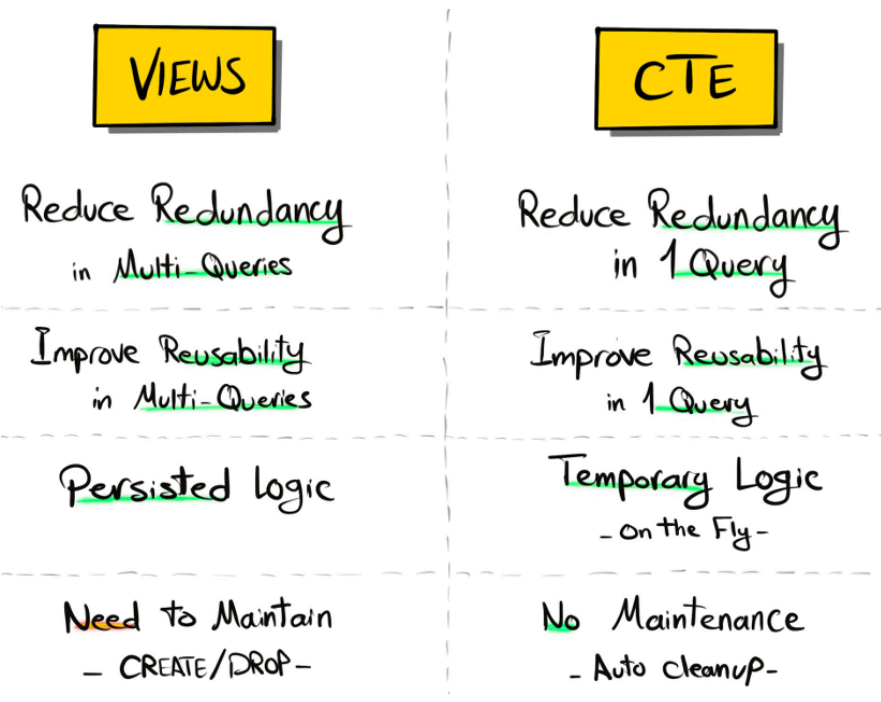






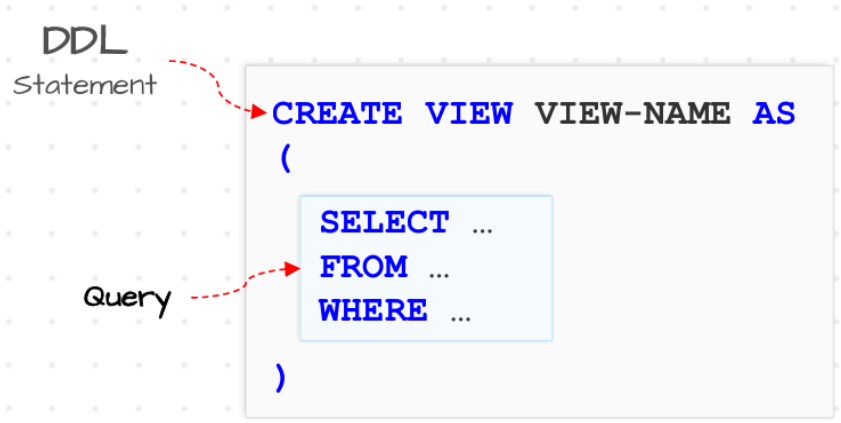


VIEW vs CTE

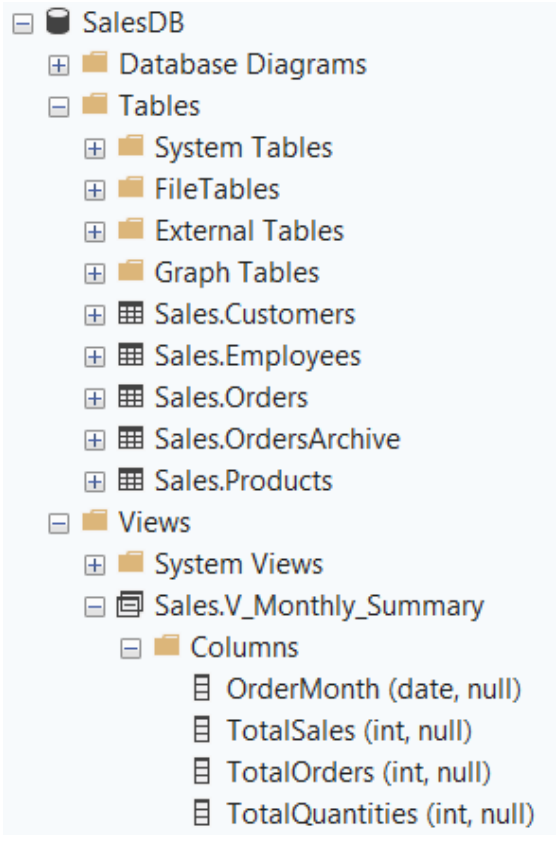


CREATE View

- Syntax




```sql
CREATE VIEW Sales.V_Monthly_Summary AS
(
    SELECT
        DATETRUNC(month, OrderDate) OrderMonth,
        SUM(Sales) TotalSales,
        COUNT(OrderID) TotalOrders,
        SUM(Quantity) TotalQuantities
    FROM Sales.Orders
    GROUP BY DATETRUNC(month, OrderDate)
);
```




```sql
-- Finding the running total of sales for each month
WITH CTE_Monthly_Summary AS
(
    SELECT
        DATETRUNC(month, OrderDate) OrderMonth,
        SUM(Sales) TotalSales,
        COUNT(OrderID) TotalOrders,
        SUM(Quantity) TotalQuantities
    FROM Sales.Orders
    GROUP BY DATETRUNC(month, OrderDate)
)
SELECT
    OrderMonth,
    SUM(TotalSales) OVER(ORDER BY OrderMonth) RunningTotal,
    TotalSales
FROM CTE_Monthly_Summary;
```

```sql
-- OR
SELECT
    OrderMonth,
    SUM(TotalSales) OVER(ORDER BY OrderMonth) RunningTotal,
    TotalSales
FROM Sales.V_Monthly_Summary;
```

	OrderMonth	RunningTotal	TotalSales
1	2025-01-01	105	105
2	2025-02-01	300	195
3	2025-03-01	380	80

- NOTE
  - If a table or view is created without specifying a schema, it defaults to the DBO

DROP View

- Syntax

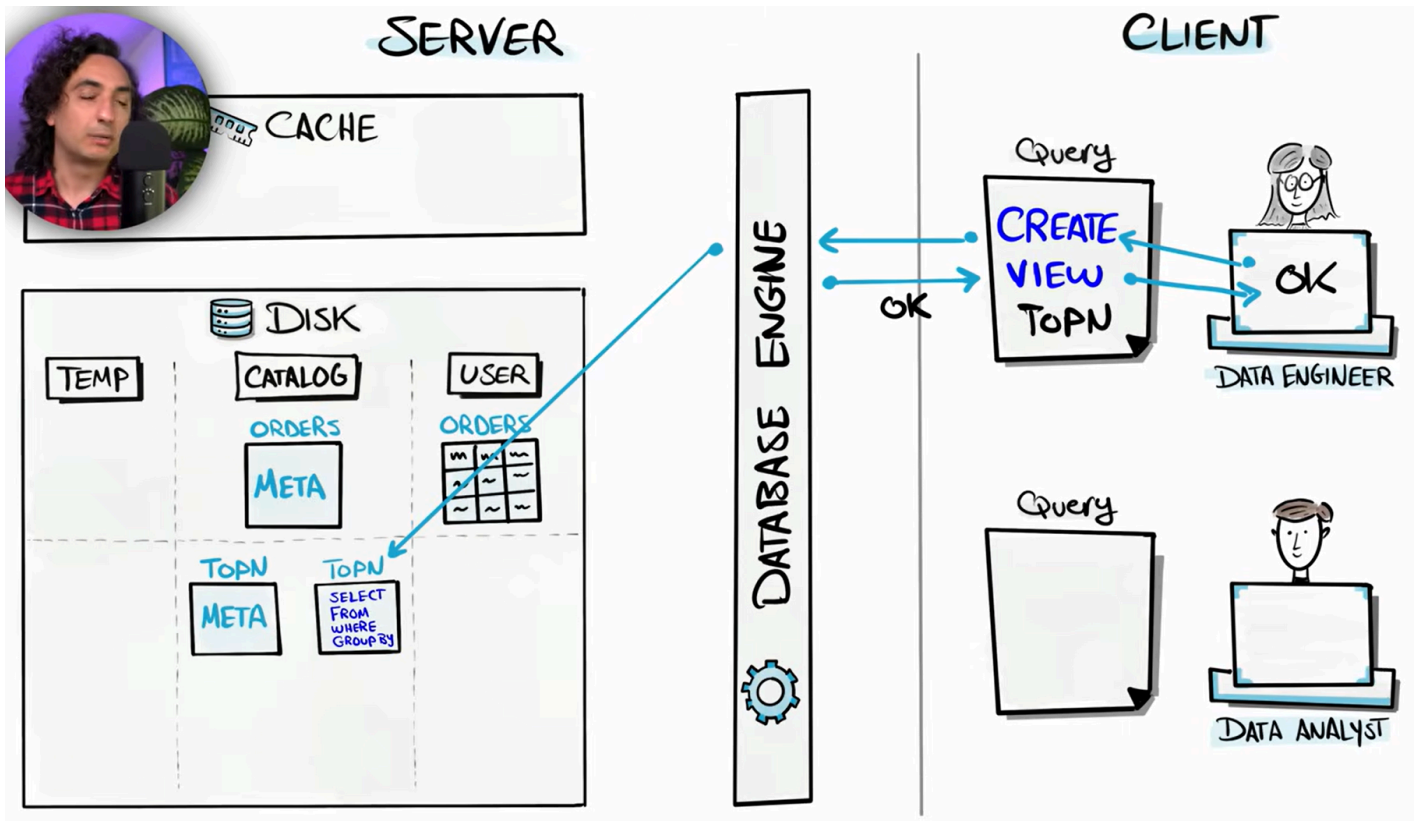
```sql
DROP VIEW V_Monthly_Summary;
```

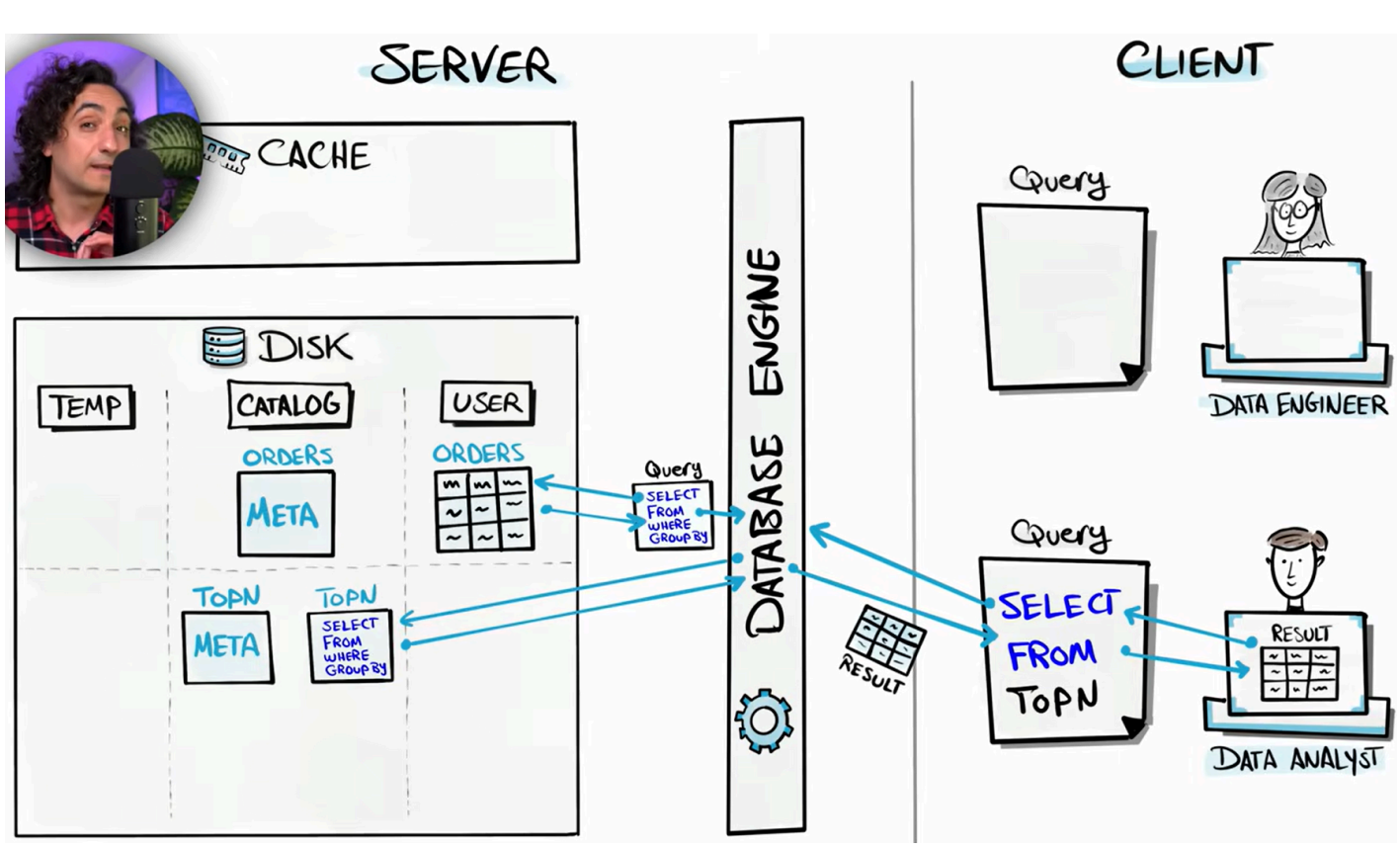
ALTER View

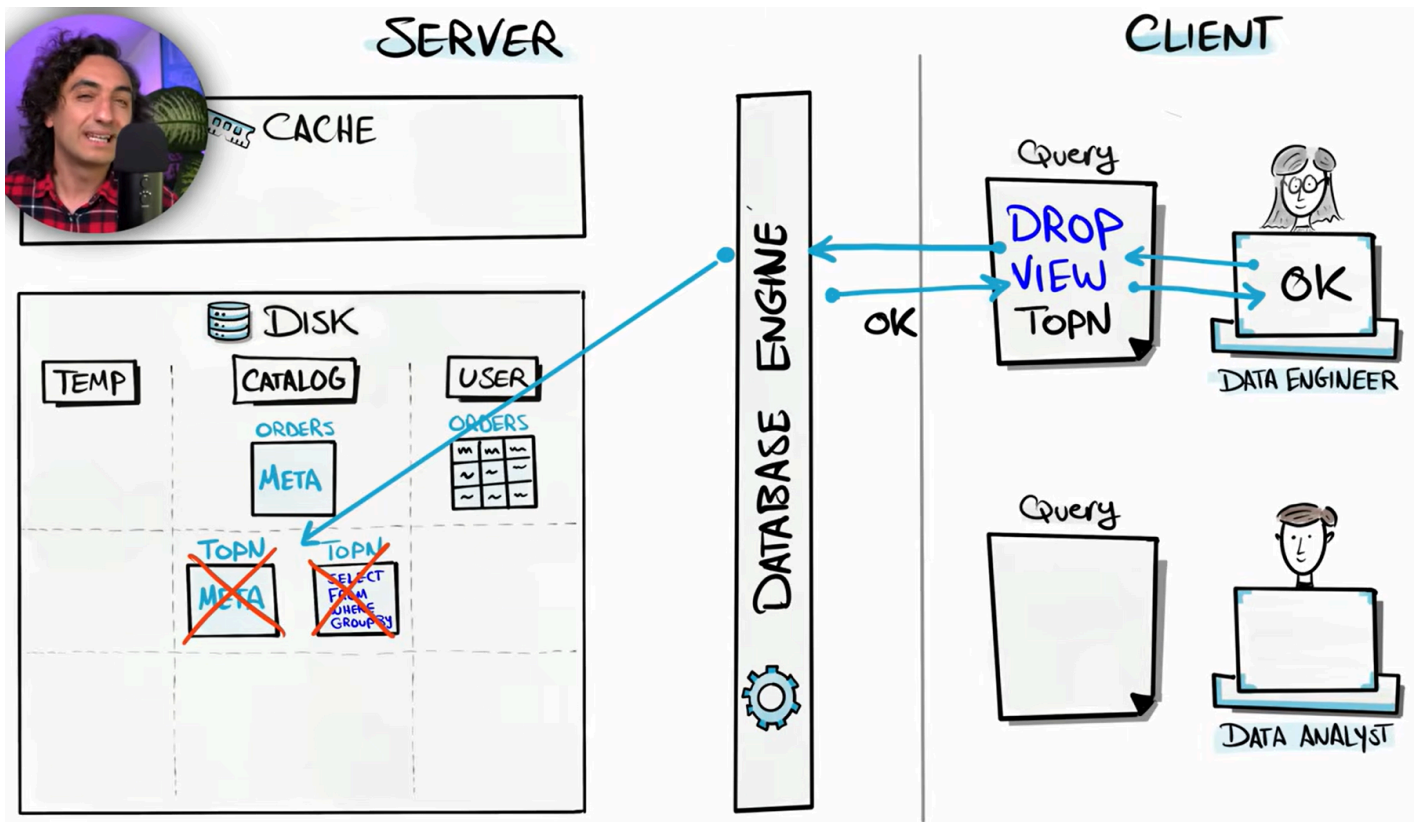
- T-SQL
  - Transact-SQL is an extension of SQL that adds programming features
- Syntax

```sql
IF OBJECT_ID('Sales.V_Monthly_Summary', 'V') IS NOT NULL
    DROP VIEW Sales.V_Monthly_Summary;
GO
CREATE VIEW Sales.V_Monthly_Summary AS
(
    SELECT
        DATETRUNC(month, OrderDate) OrderMonth,
        SUM(Sales) TotalSales,
        COUNT(OrderID) TotalOrders
    FROM Sales.Orders
    GROUP BY DATETRUNC(month, OrderDate)
);
```

How Database Executes Views

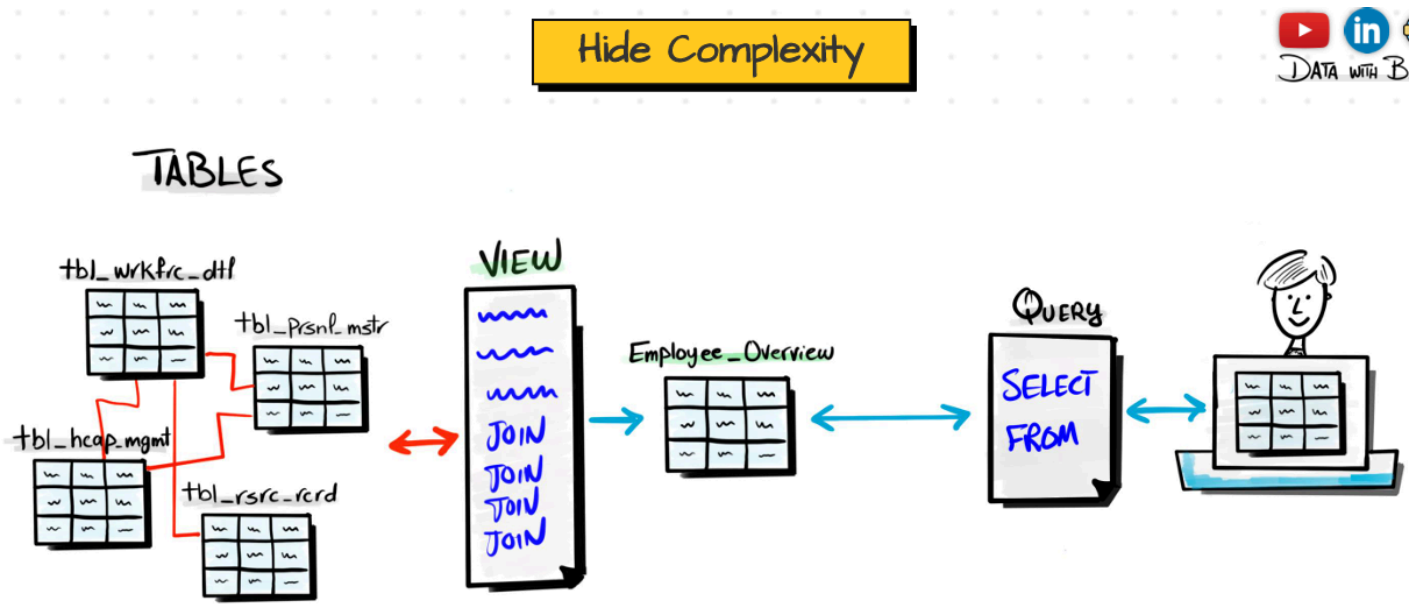






VIEWS USE CASE: Hide Complexity

- Views can be used to hide the complexity of database tables and offers users more friendly and easy-to-consume objects



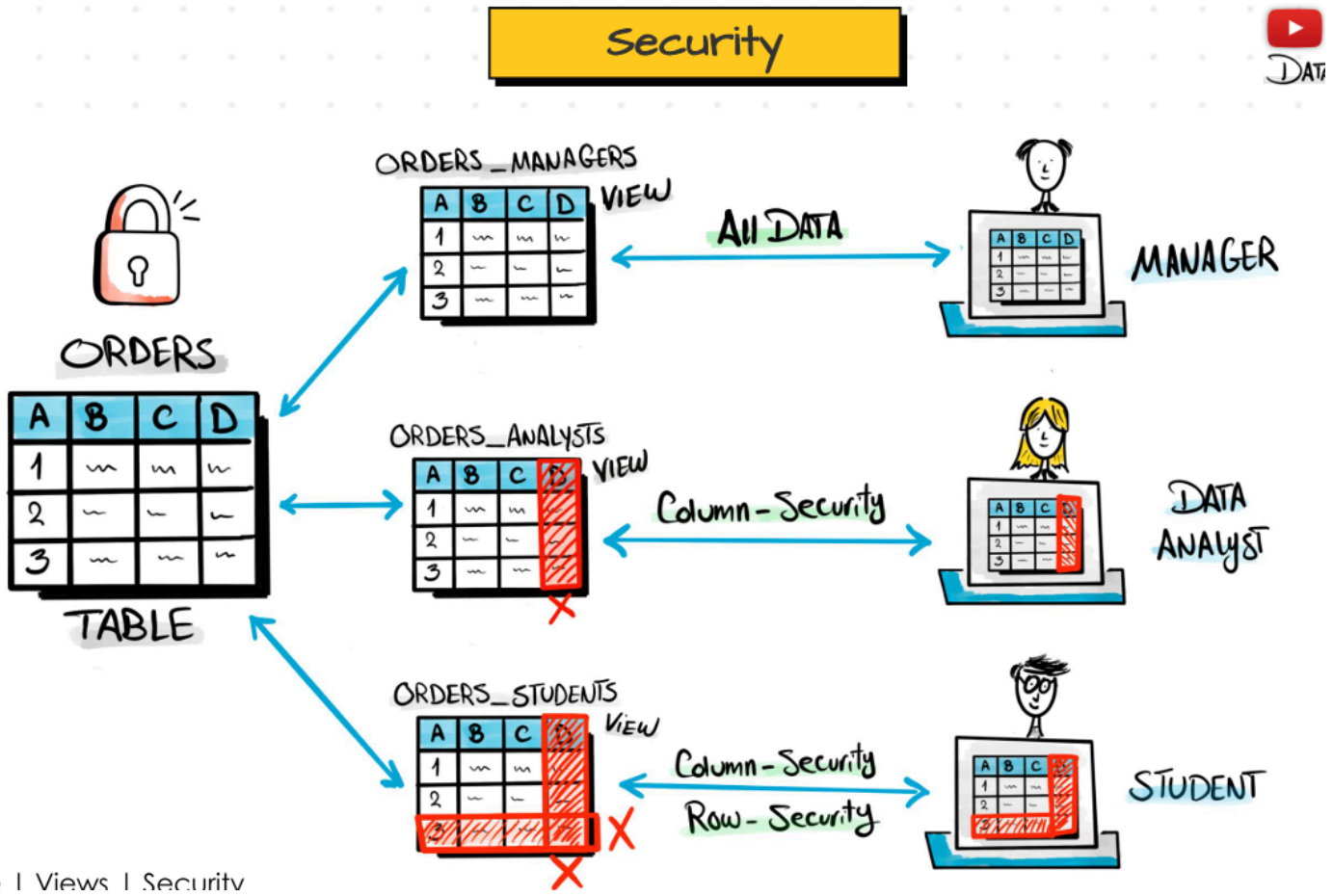

```sql
/* Hide Complexity: Provide a view that combines details from
   orders, products, consumers, and employees */
CREATE VIEW Sales.V_Order_Details AS
(
    SELECT
        o.OrderID,
        o.OrderDate,
        p.Product,
        p.Category,
        COALESCE(c.FirstName,'') + ' ' + COALESCE(c.LastName,'') CustomerName,
        c.Country CustomerCountry,
        COALESCE(e.FirstName,'') + ' ' + COALESCE(e.LastName,'') EmployeeName,
        e.Department,
        o.Sales,
        o.Quantity
    FROM Sales.Orders o
    LEFT JOIN Sales.Products p
    ON o.ProductID = p.ProductID
    LEFT JOIN Sales.Customers c
    ON o.CustomerID = c.CustomerID
    LEFT JOIN Sales.Employees e
    ON o.SalesPersonID = e.EmployeeID
);
```

```sql
SELECT
    *
FROM Sales.V_Order_Details;
```

	OrderID	OrderDate	Product	Category	CustomerName	CustomerCountry	EmployeeName	Department	Sales	Quantity
1	1	2025-01-01	Bottle	Accessories	Kevin Brown	USA	Mary	Sales	10	1
2	2	2025-01-05	Tire	Accessories	Mary	USA	Mary	Sales	15	1
3	3	2025-01-10	Bottle	Accessories	Jossef Goldberg	Germany	Carol Baker	Sales	20	2
4	4	2025-01-20	Gloves	Clothing	Jossef Goldberg	Germany	Mary	Sales	60	2
5	5	2025-02-01	Caps	Clothing	Kevin Brown	USA	Carol Baker	Sales	25	1
6	6	2025-02-05	Caps	Clothing	Mary	USA	Carol Baker	Sales	50	2
7	7	2025-02-15	Tire	Accessories	Jossef Goldberg	Germany	Frank Lee	Marketing	30	2
8	8	2025-02-18	Bottle	Accessories	Mark Schwarz	Germany	Mary	Sales	90	3
9	9	2025-03-10	Bottle	Accessories	Kevin Brown	USA	Mary	Sales	20	2
10	10	2025-03-15	Tire	Accessories	Mary	USA	Carol Baker	Sales	60	0

VIEWS USE CASE: Data Security

- Use views to enforce security and protect sensitive data, by hiding columns and/or rows from tables

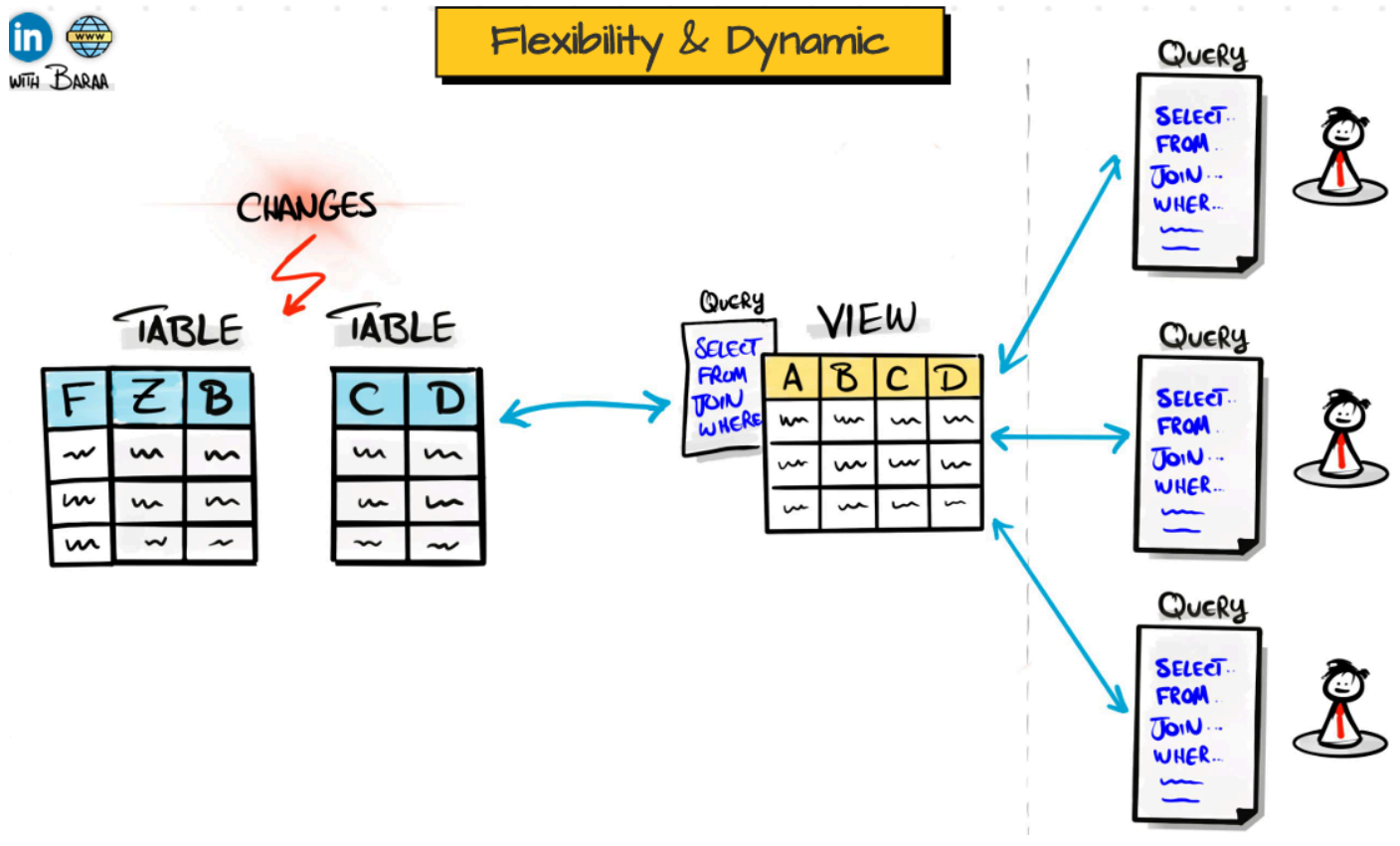



```sql
/* Data Security: Provide a view for the EU Sales Team
   that combines details from all tables and excludes
   data related to the USA */
CREATE VIEW Sales.V_Order_Details_EU AS
(
    SELECT
        o.OrderID,
        o.OrderDate,
        p.Product,
        p.Category,
        COALESCE(c.FirstName,'') + ' ' + COALESCE(c.LastName,'') CustomerName,
        c.Country CustomerCountry,
        COALESCE(e.FirstName,'') + ' ' + COALESCE(e.LastName,'') EmployeeName,
        e.Department,
        o.Sales,
        o.Quantity
    FROM Sales.Orders o
    LEFT JOIN Sales.Products p
    ON o.ProductID = p.ProductID
    LEFT JOIN Sales.Customers c
    ON o.CustomerID = c.CustomerID
    LEFT JOIN Sales.Employees e
    ON o.SalesPersonID = e.EmployeeID
    WHERE c.Country != 'USA'
);
```

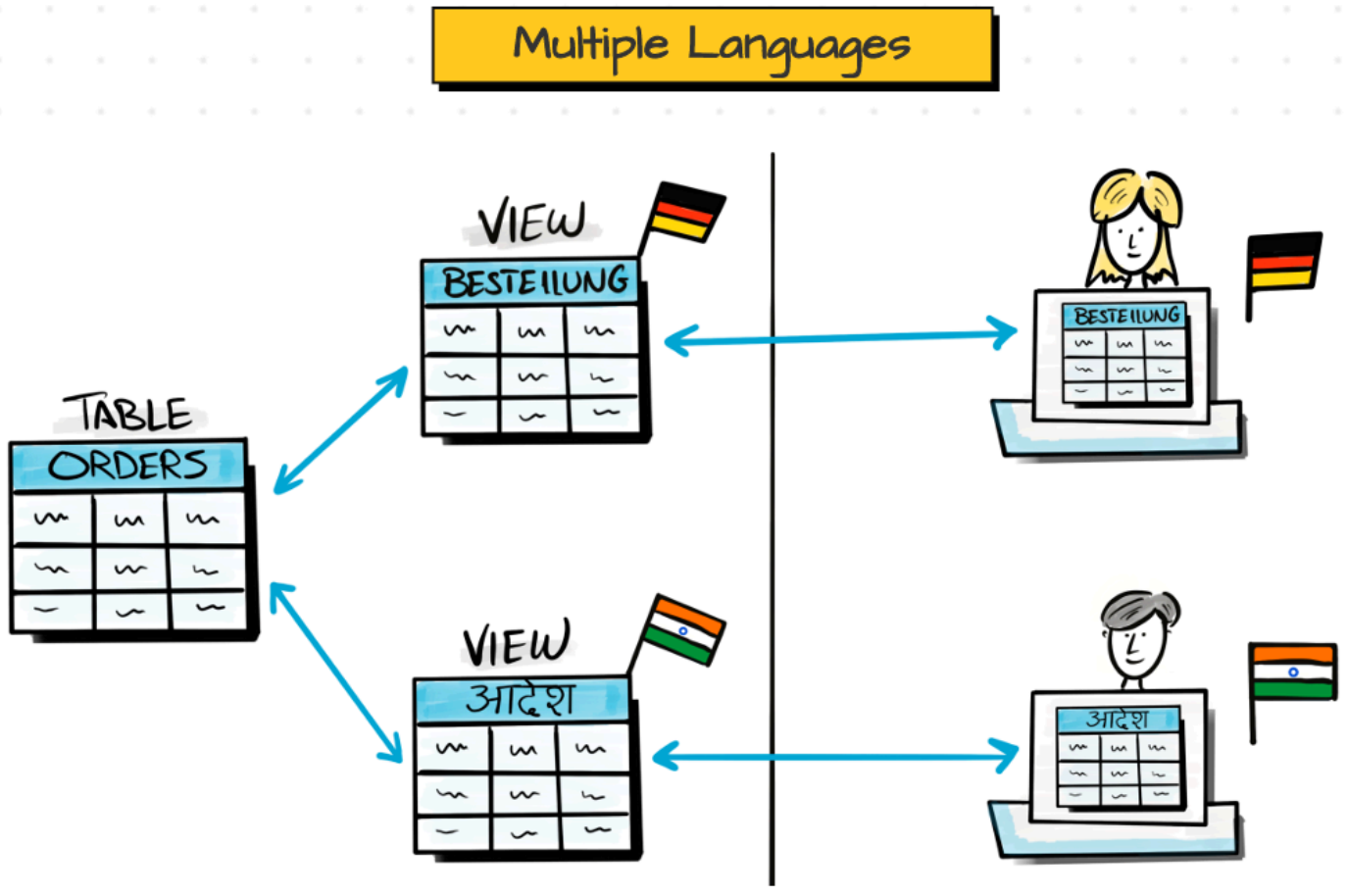
```sql
SELECT
    *
FROM Sales.V_Order_Details_EU;
```

	OrderID	OrderDate	Product	Category	CustomerName	CustomerCountry	EmployeeName	Department	Sales	Quantity
1	3	2025-01-10	Bottle	Accessories	Jossef Goldberg	Germany	Carol Baker	Sales	20	2
2	4	2025-01-20	Gloves	Clothing	Jossef Goldberg	Germany	Mary	Sales	60	2
3	7	2025-02-15	Tire	Accessories	Jossef Goldberg	Germany	Frank Lee	Marketing	30	2
4	8	2025-02-18	Bottle	Accessories	Mark Schwarz	Germany	Mary	Sales	90	3

VIEWS USE CASE: Flexibility & Dynamic

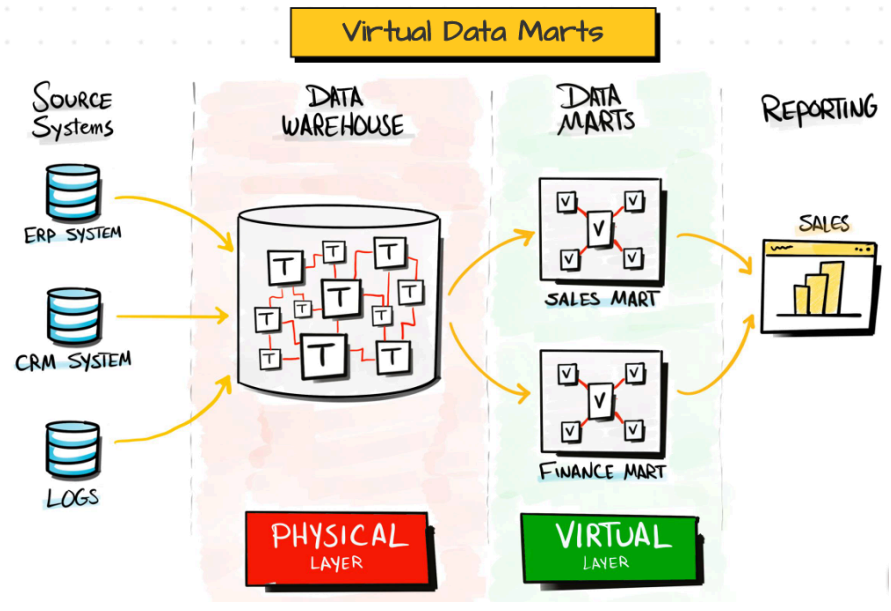


VIEWS USE CASE: Multi-Language

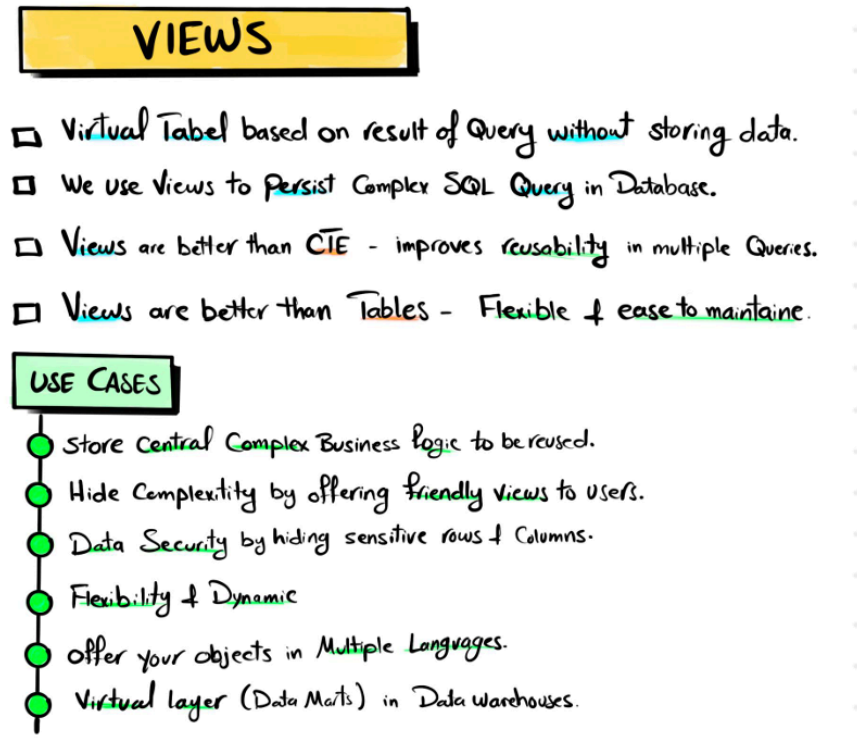


VIEWS USE CASE: Virtual Data Marts in Data Warehouse

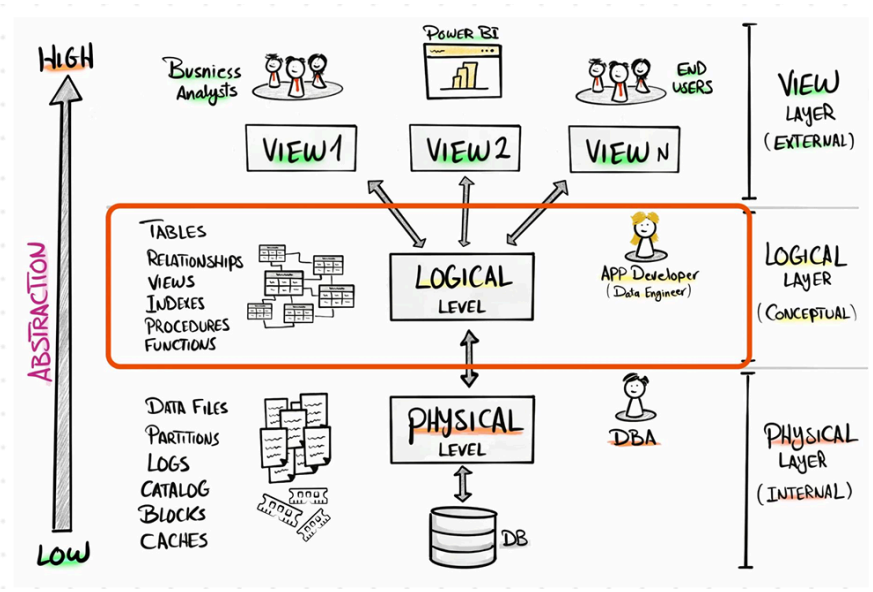
- Views can be used as Data Marts in Data Warehouse System because they provide a flexible and efficient way to present data



Summary



Module 20: CTAS & TEMP Tables



Database Table

- A table is a structured collection of data, similar to a spreadsheet or grid (Excel)

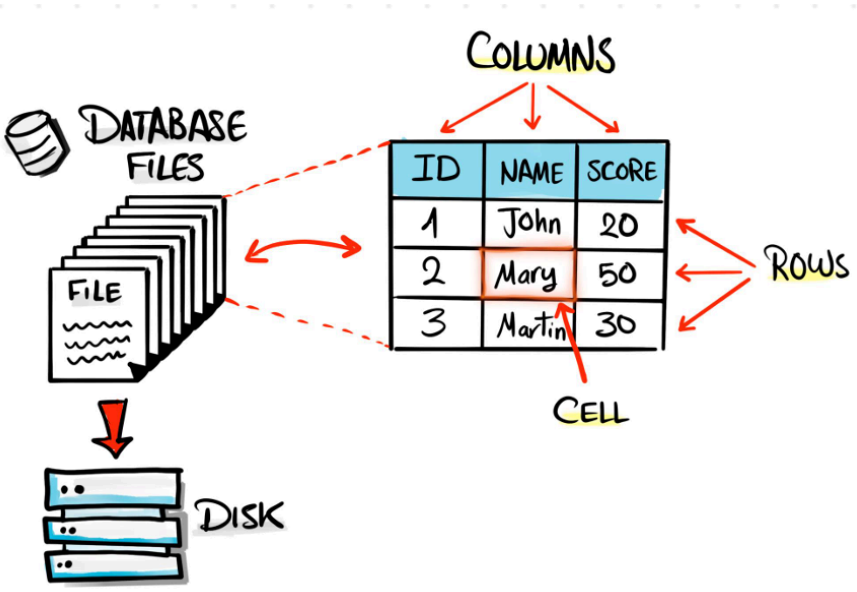
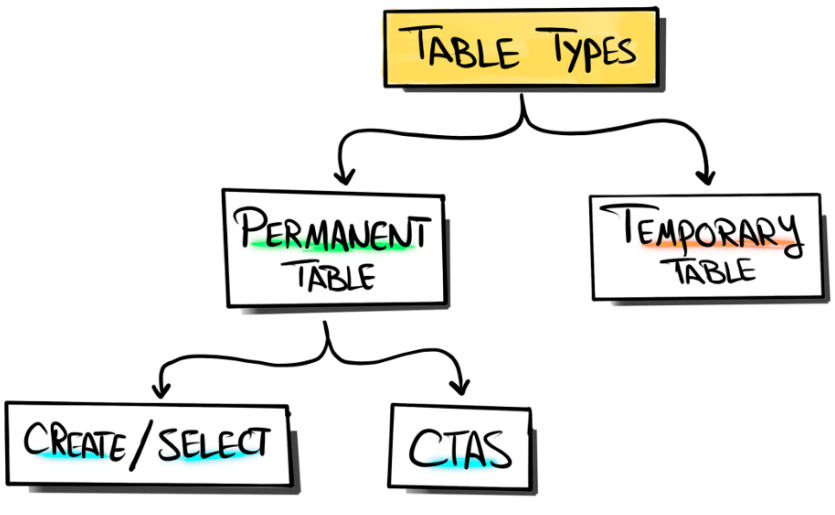
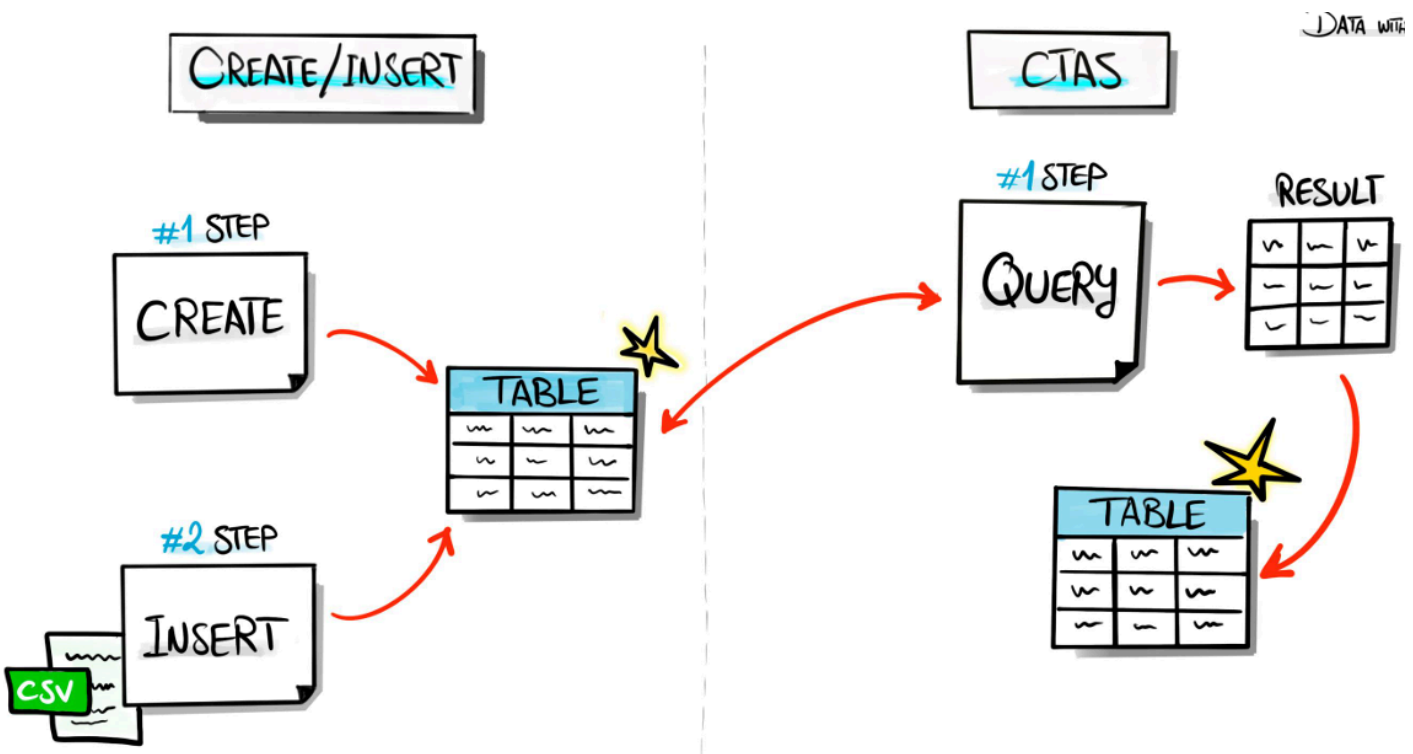


Table Types

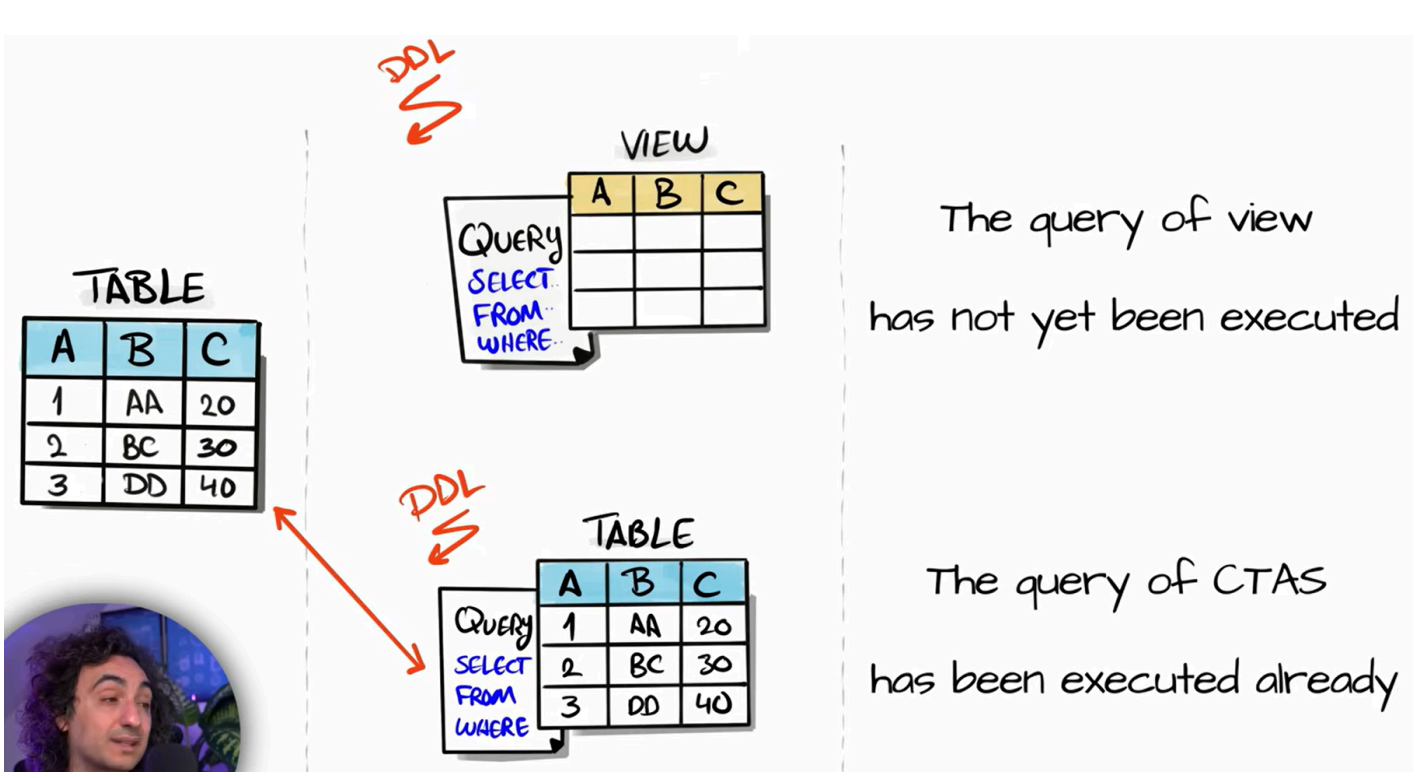


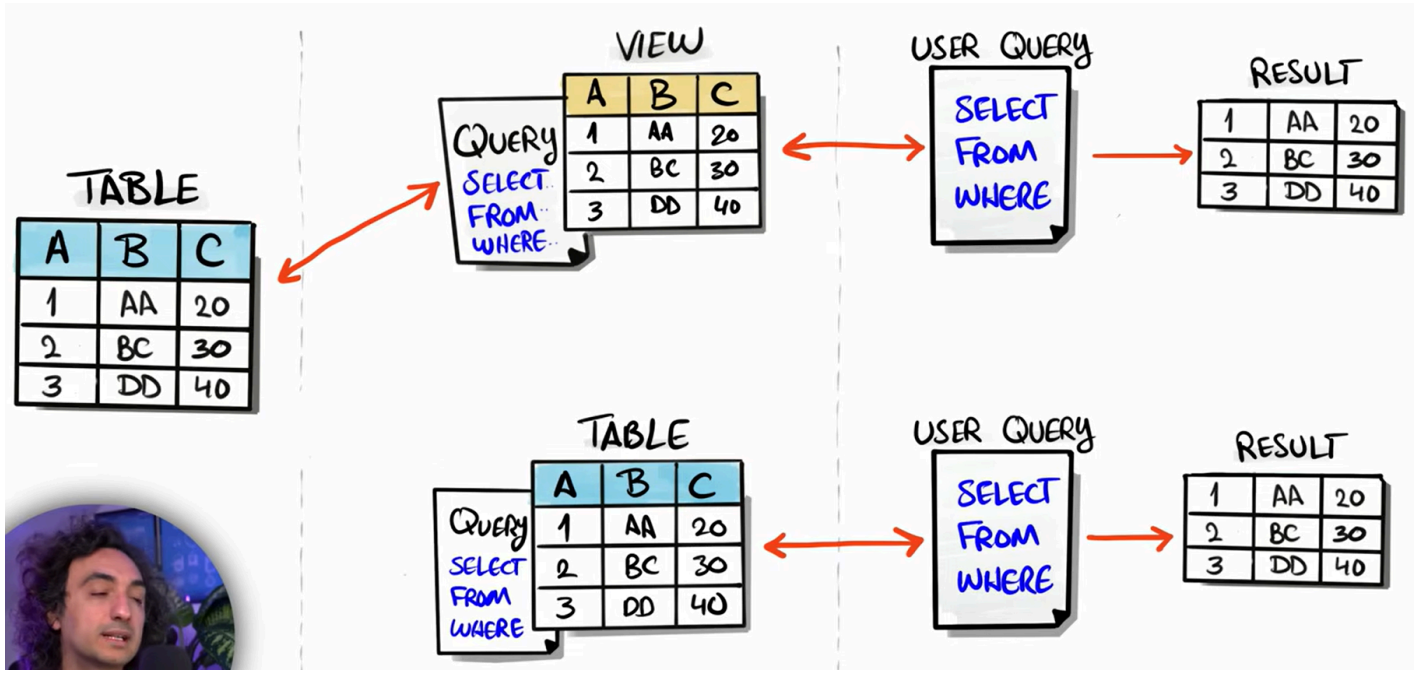
CTAS vs CREATE/INSERT

- CREATE/INSERT
  1. Create: Define the structure of table
  2. Insert: Insert data into the table
- CTAS (CREATE TABLE AS SELECT)
  - Create a new table based on the result of an SQL query

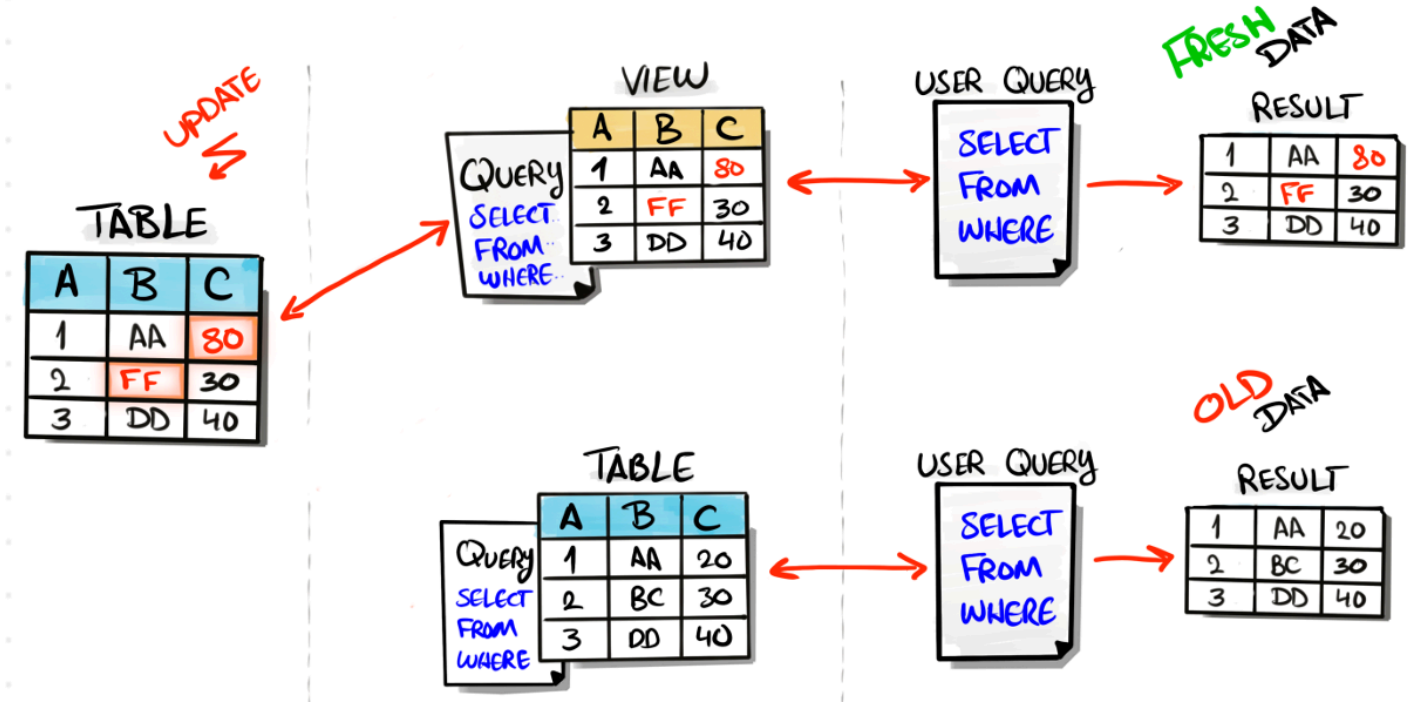


CTAS vs VIEWS

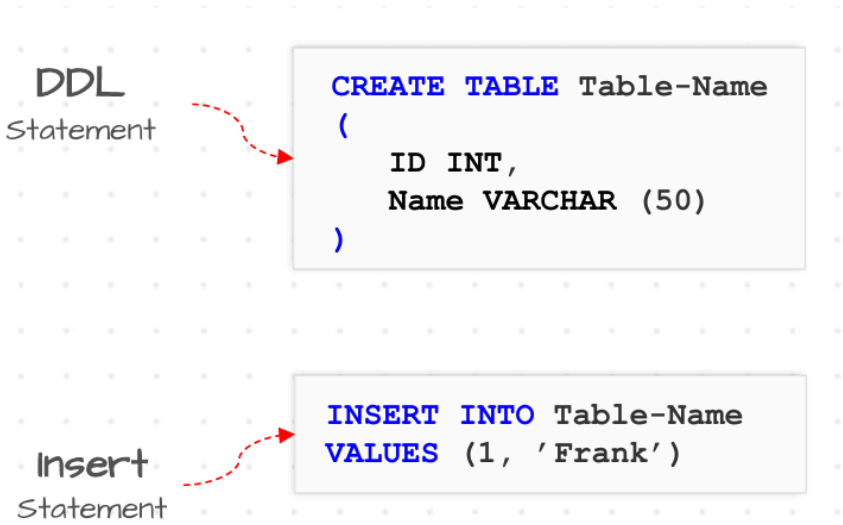




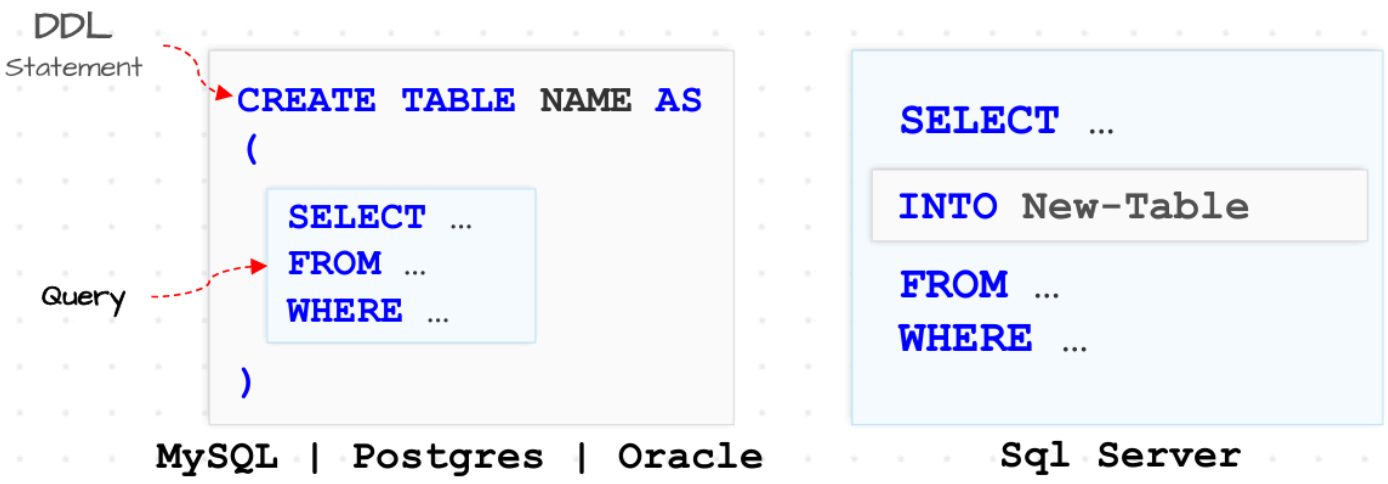
- Querying Views is slower than querying CTAS tables



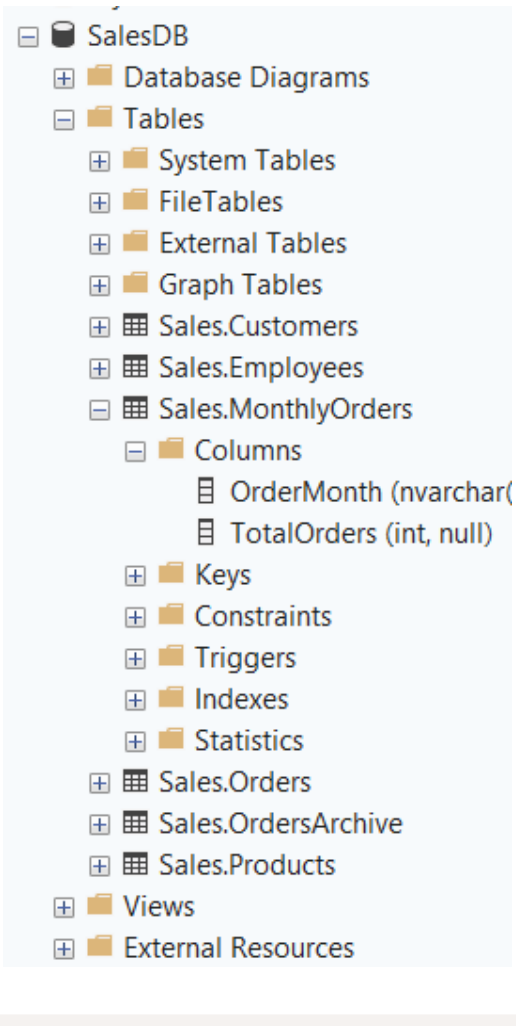
CREATE / INSERT Syntax



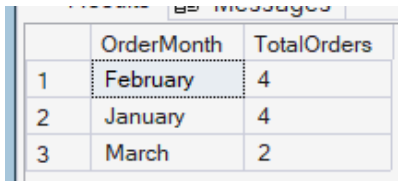
CTAS Syntax




```sql
SELECT
    DATENAME(month, OrderDate) OrderMonth,
    COUNT(OrderID) TotalOrders
INTO Sales.MonthlyOrders
FROM Sales.Orders
GROUP BY DATENAME(month, OrderDate);
```

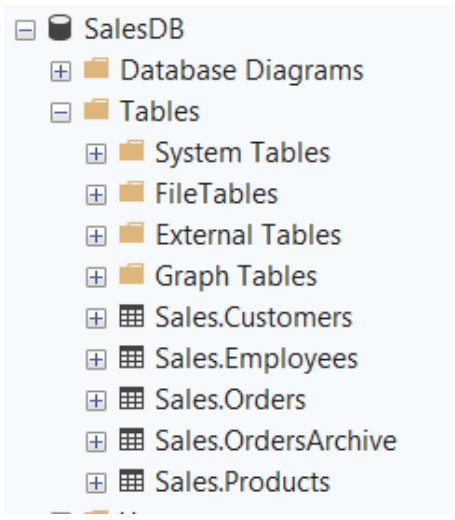



```sql
SELECT
    *
FROM Sales.MonthlyOrders;
```



	OrderMonth	TotalOrders
1	February	4
2	January	4
3	March	2

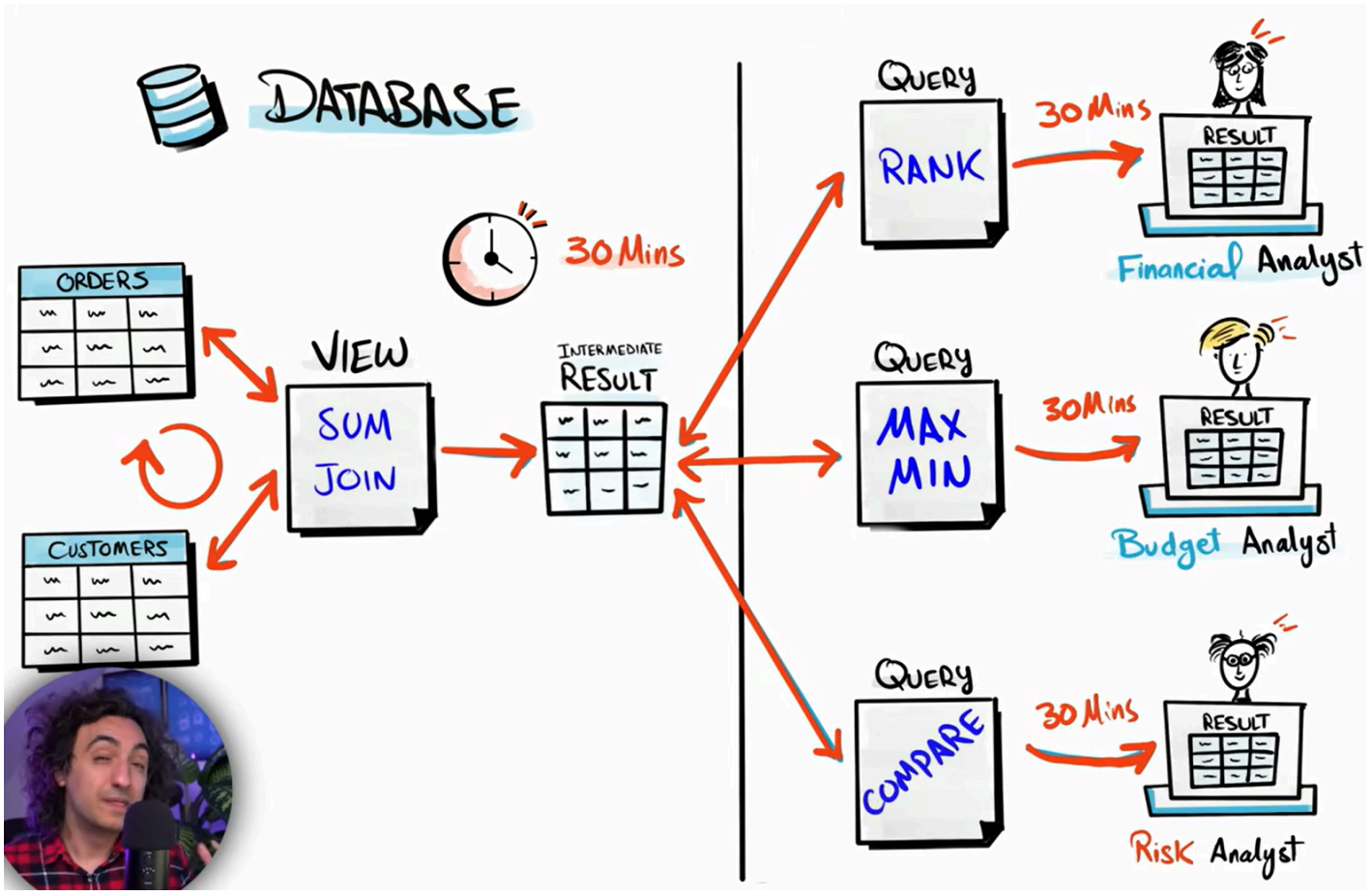
```sql
DROP TABLE Sales.MonthlyOrders;
```

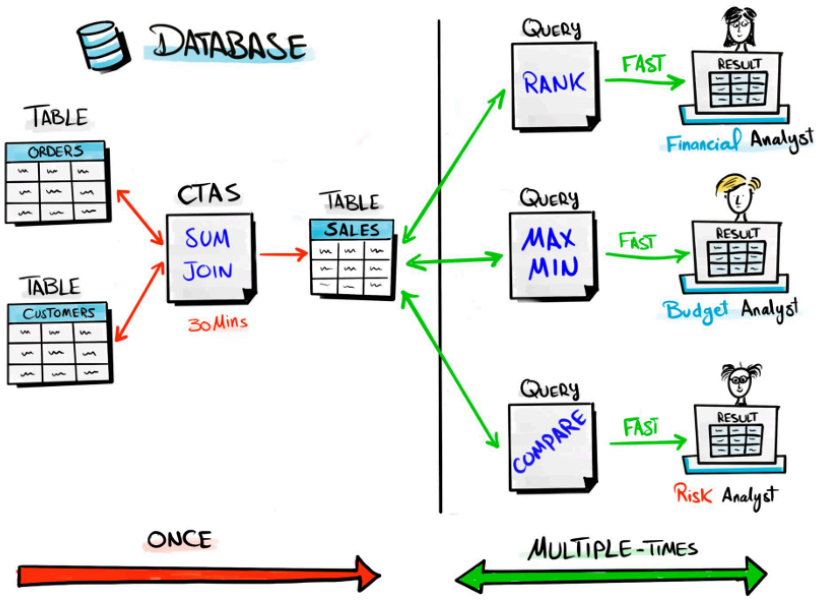


How to Refresh CTAS

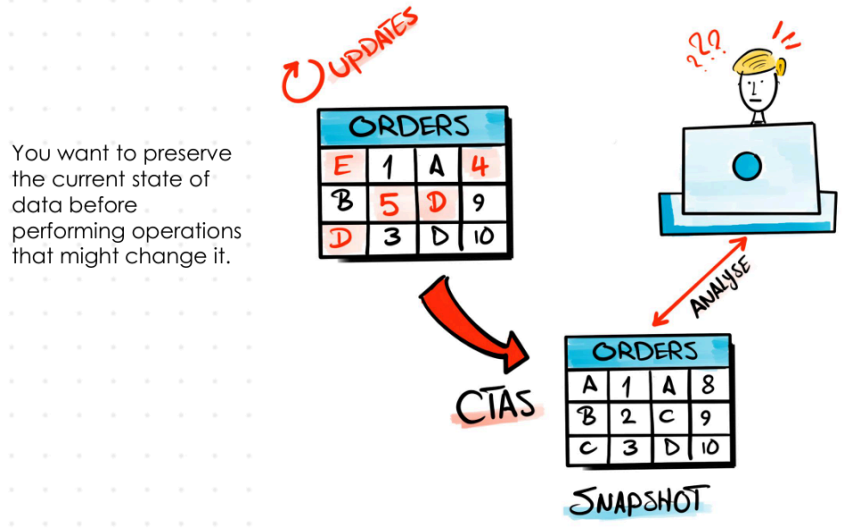
```
IF OBJECT_ID('Sales.MonthlyOrders', 'U') IS NOT NULL
    DROP TABLE Sales.MonthlyOrders;
GO
SELECT
    DATENAME(month, OrderDate) OrderMonth,
    COUNT(OrderID) TotalOrders
INTO Sales.MonthlyOrders
FROM Sales.Orders
GROUP BY DATENAME(month, OrderDate);
```

CTAS USE CASE: Optimize Performance



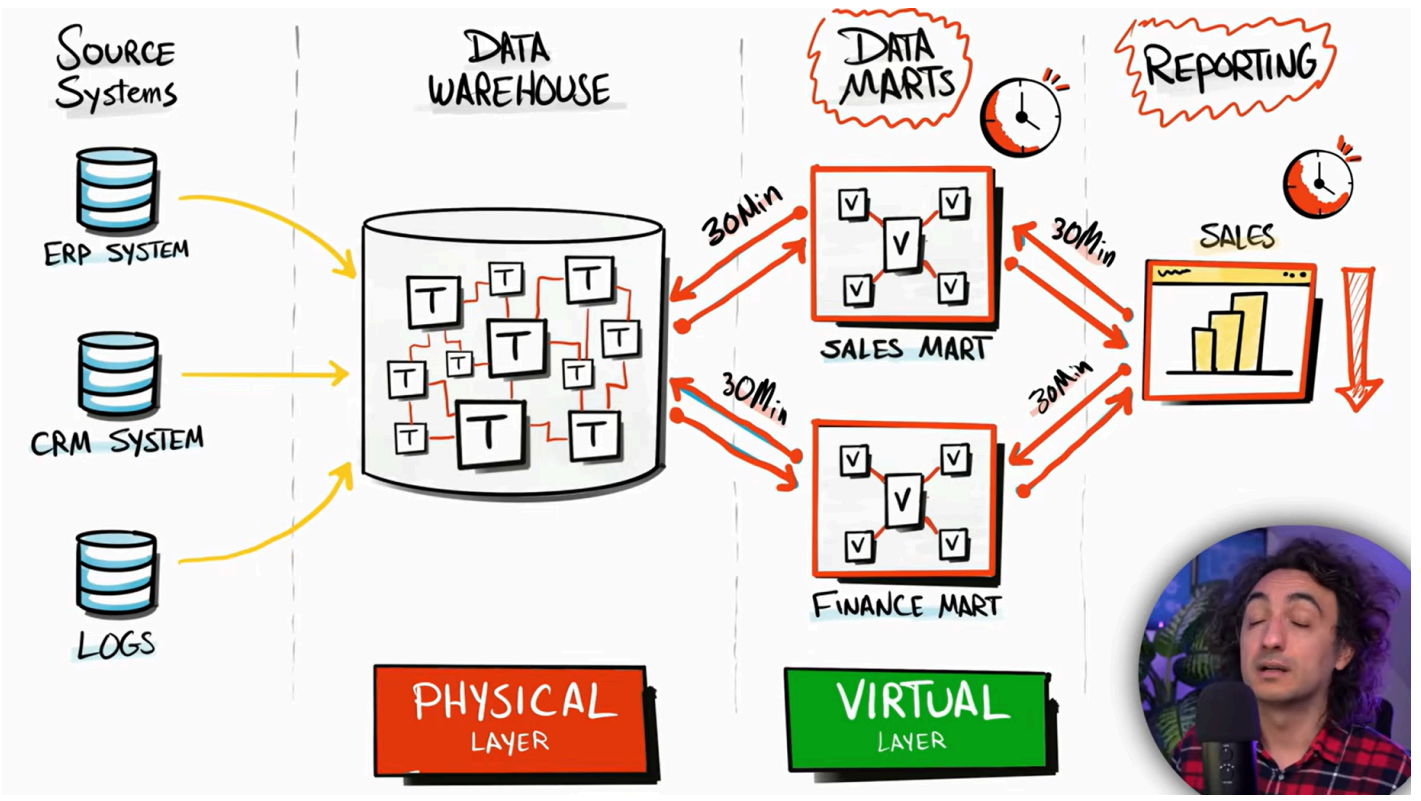


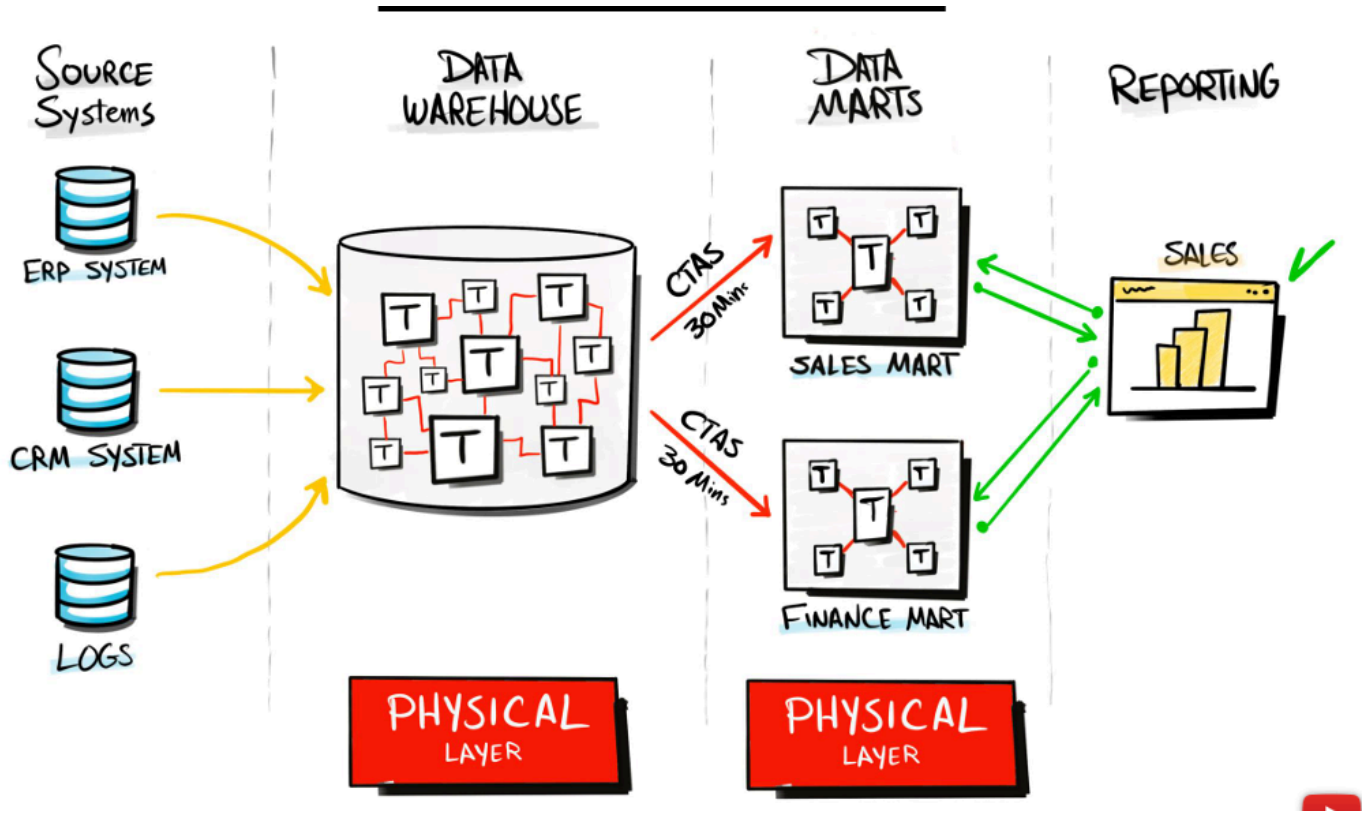
CTAS USE CASE: Creating a Snapshot



CTAS USE CASE: Physical Data Marts in Data Warehouse

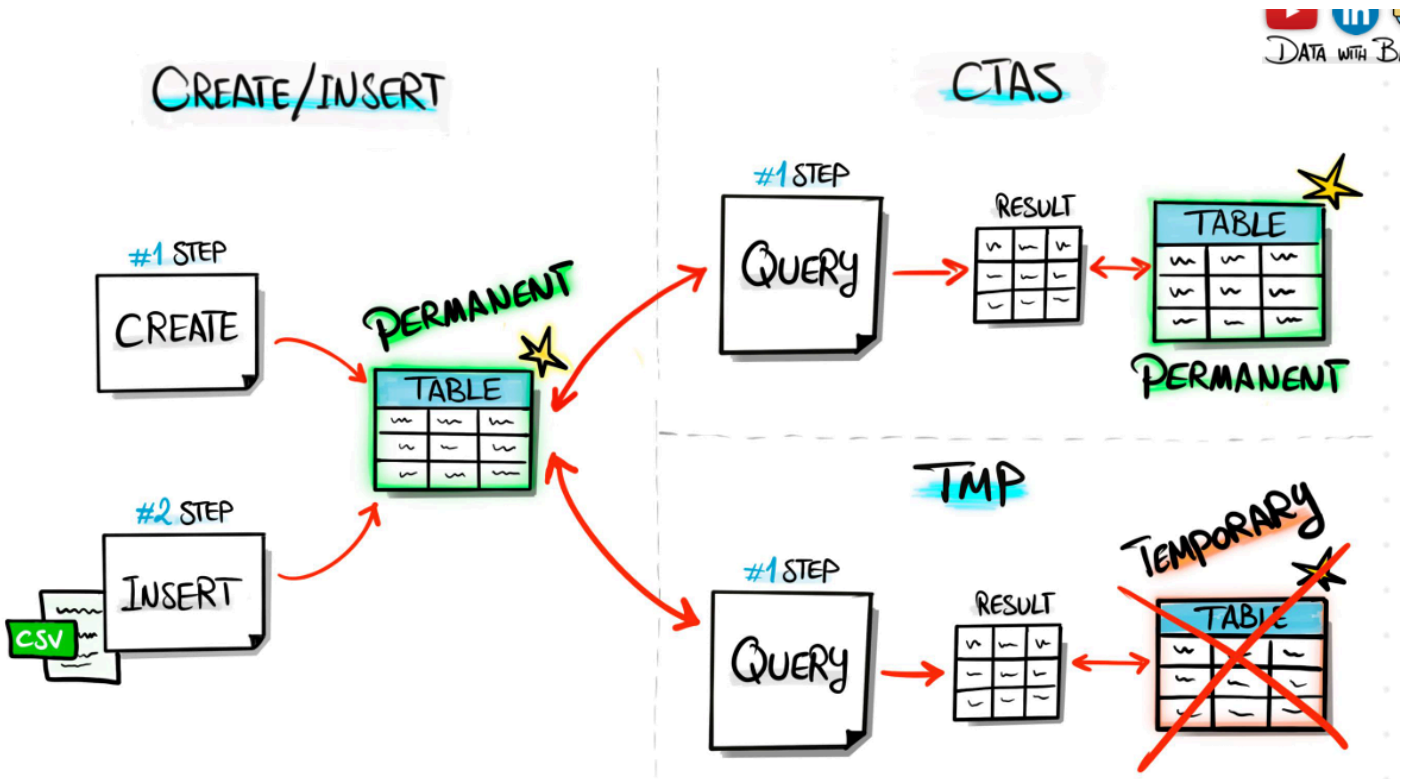
- Persisting the Data Marts of a DWH improves the speed of data retrieval compared to using views





Temporary Tables

- Stores intermediate results in temporary storage within the database during the session
- The database will drop all temporary tables once the session ends
- Session
  - The time between connecting to and disconnecting from the database



Temporary Tables Syntax

```sql
SELECT ...
INTO #New-Table
FROM ...
WHERE ...
```

Sql Server

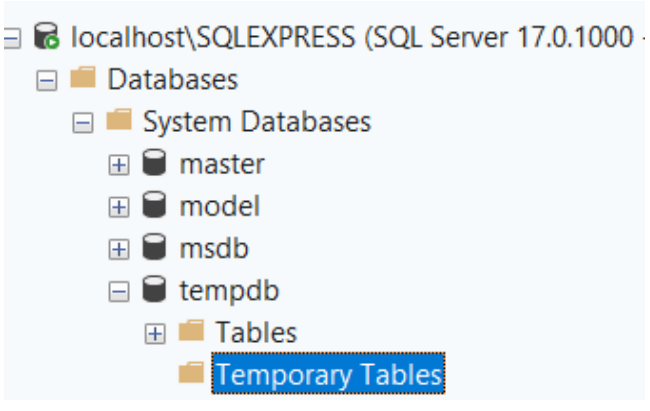
PERMENANT
CREATE TABLE

```sql
CREATE TABLE TABLE-NAME AS
(
  SELECT ...
  FROM ...
  WHERE ...
)
```

TEMPORARY
TABLE

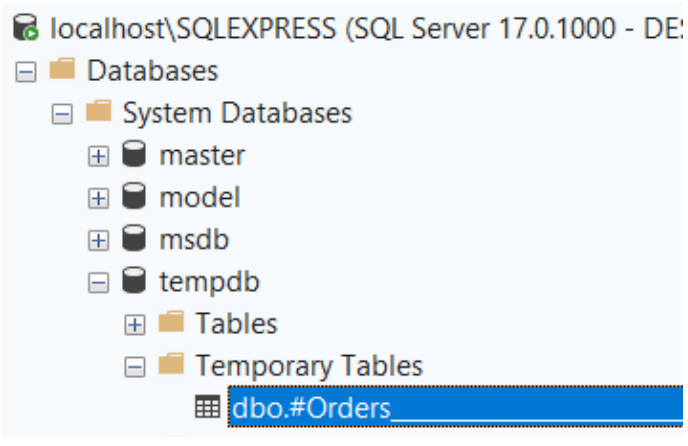
```sql
CREATE TEMPORARY TABLE TABLE-NAME AS
(
    SELECT ...
    FROM ...
    WHERE ...
)
```

MySQL | Postgres | Oracle



Example

```sql
SELECT
    *
INTO #Orders
FROM Sales.Orders;
```



```sql
SELECT * FROM #Orders;
```

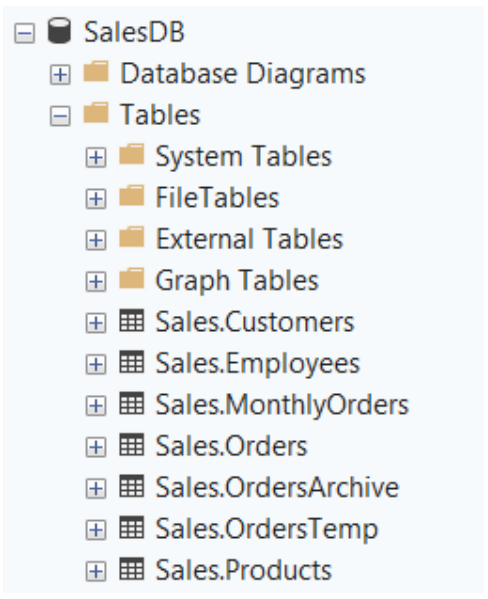
	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Quantity	Sales	CreationTime
1	1	101	2	3	2025-01-01	2025-01-05	Delivered	9833 Mt. Dias Blv.	1226 Shoe St.	1	10	2025-01-01 12:34:56.0000000
2	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	NULL	1	15	2025-01-05 23:22:04.0000000
3	3	101	1	5	2025-01-10	2025-01-25	Delivered	8157 W. Book	8157 W. Book	2	20	2025-01-10 18:24:08.0000000
4	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane		2	60	2025-01-20 05:50:33.0000000
5	5	104	2	5	2025-02-01	2025-02-05	Delivered	NULL	NULL	1	25	2025-02-01 14:02:41.0000000
6	6	104	3	5	2025-02-05	2025-02-10	Delivered	1792 Belmont Rd.	NULL	2	50	2025-02-06 15:34:57.0000000
7	7	102	1	1	2025-02-15	2025-02-27	Delivered	136 Balboa Court		2	30	2025-02-16 06:22:01.0000000
8	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4311 Clay Rd	3	90	2025-02-18 10:45:22.0000000
9	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way		2	20	2025-03-10 12:59:04.0000000
10	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	NULL	0	60	2025-03-16 23:25:15.0000000

```sql
DELETE FROM #Orders
WHERE OrderStatus = 'Delivered';
```

	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Quantity	Sales	CreationTime
1	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	NULL	1	15	2025-01-05 23:22:04.0000000
2	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane		2	60	2025-01-20 05:50:33.0000000
3	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4311 Clay Rd	3	90	2025-02-18 10:45:22.0000000
4	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way		2	20	2025-03-10 12:59:04.0000000
5	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	NULL	0	60	2025-03-16 23:25:15.0000000

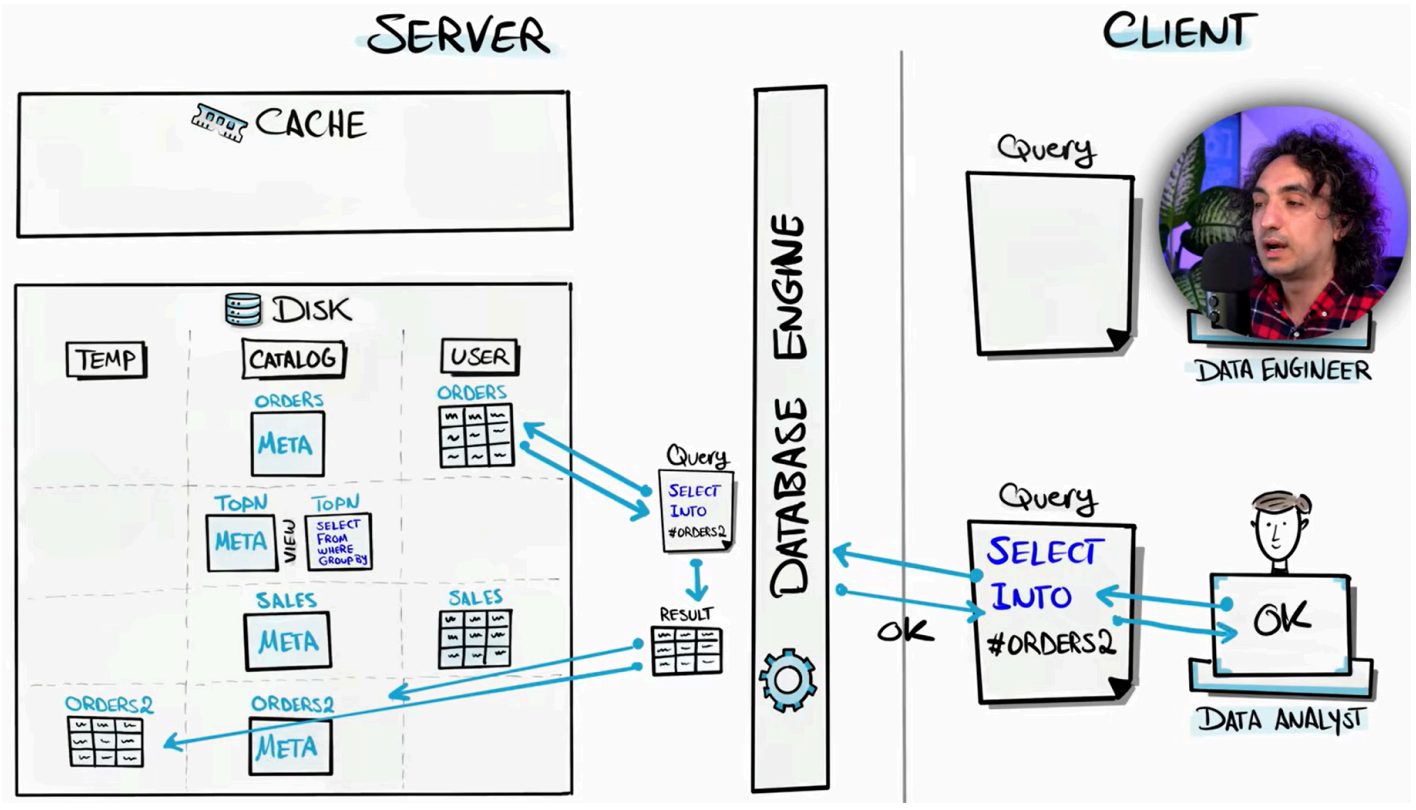
```sql
SELECT
    *
INTO Sales.OrdersTemp
FROM #Orders;
```

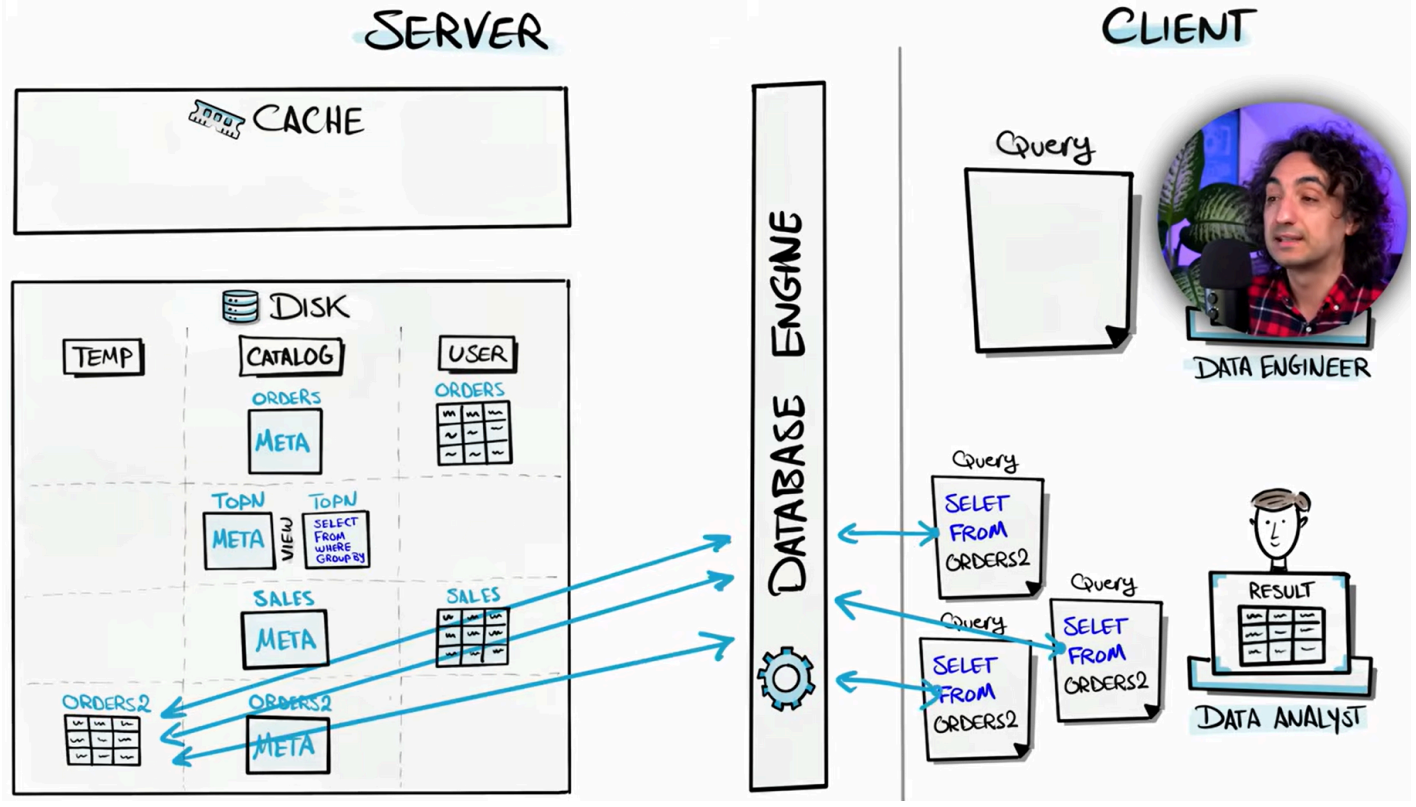
	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Quantity	Sales	CreationTime
1	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	NULL	1	15	2025-01-05 23:22:04.0000000
2	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane		2	60	2025-01-20 05:50:33.0000000
3	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4311 Clay Rd	3	90	2025-02-18 10:45:22.0000000
4	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way		2	20	2025-03-10 12:59:04.0000000
5	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	NULL	0	60	2025-03-16 23:25:15.0000000

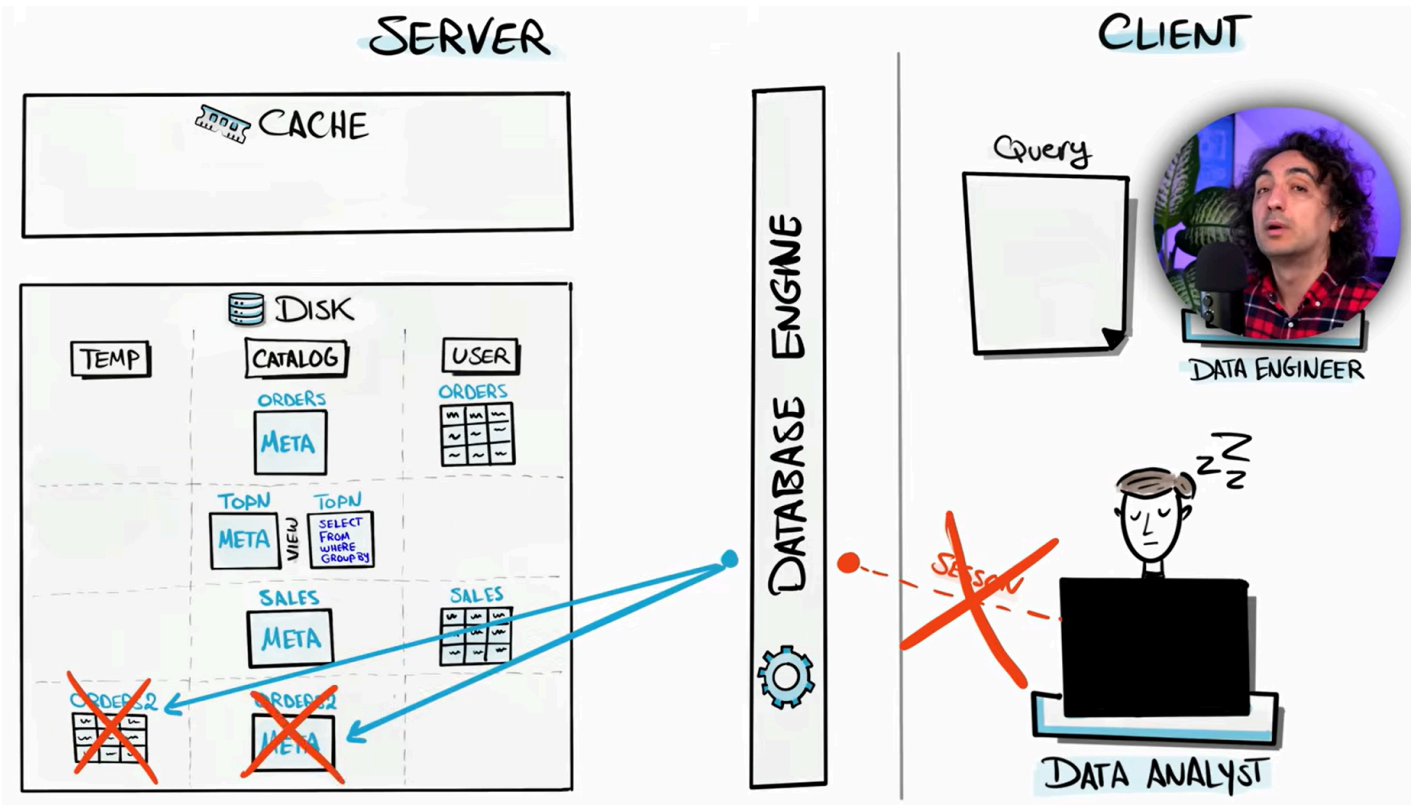


1. Load Data to TEMP Table
2. Transform Data in TEMP Table
3. Load TEMP Table into Permanent Table

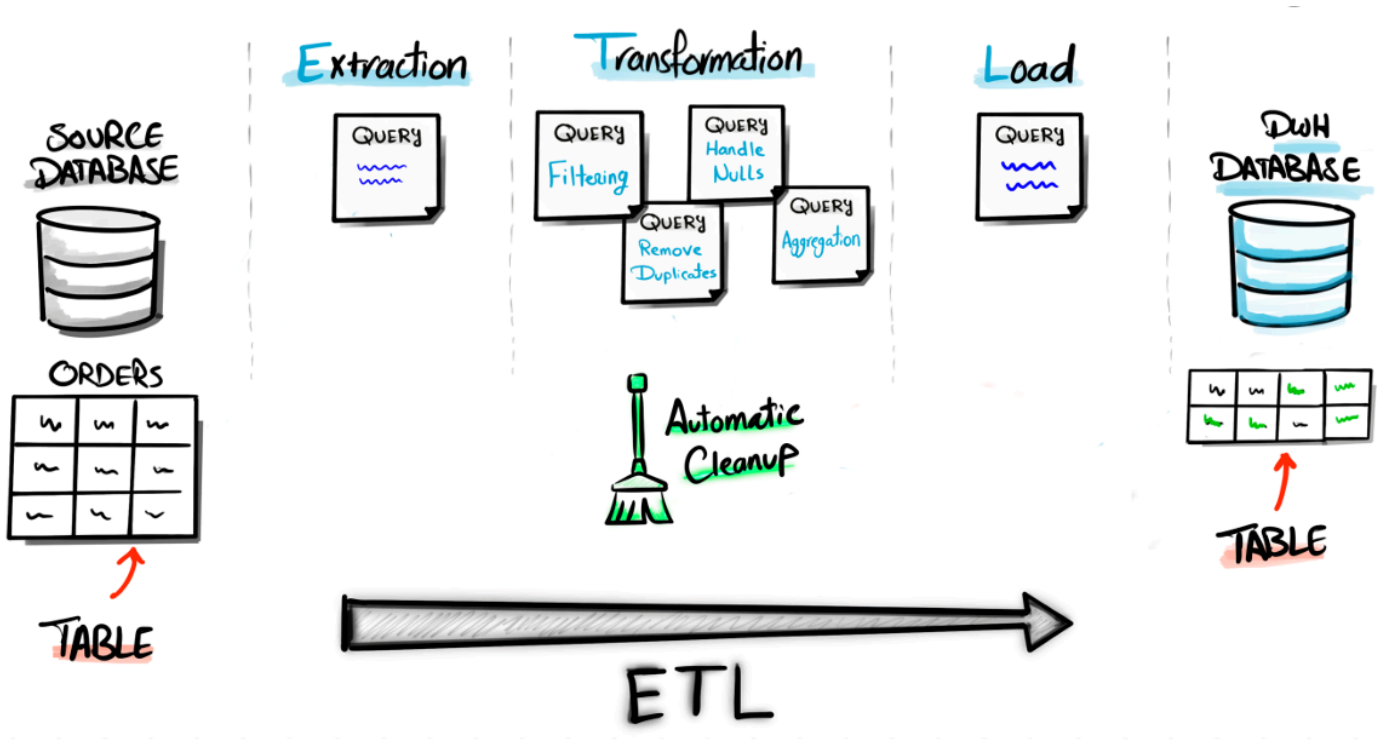
How Database Execute TEMP Tables



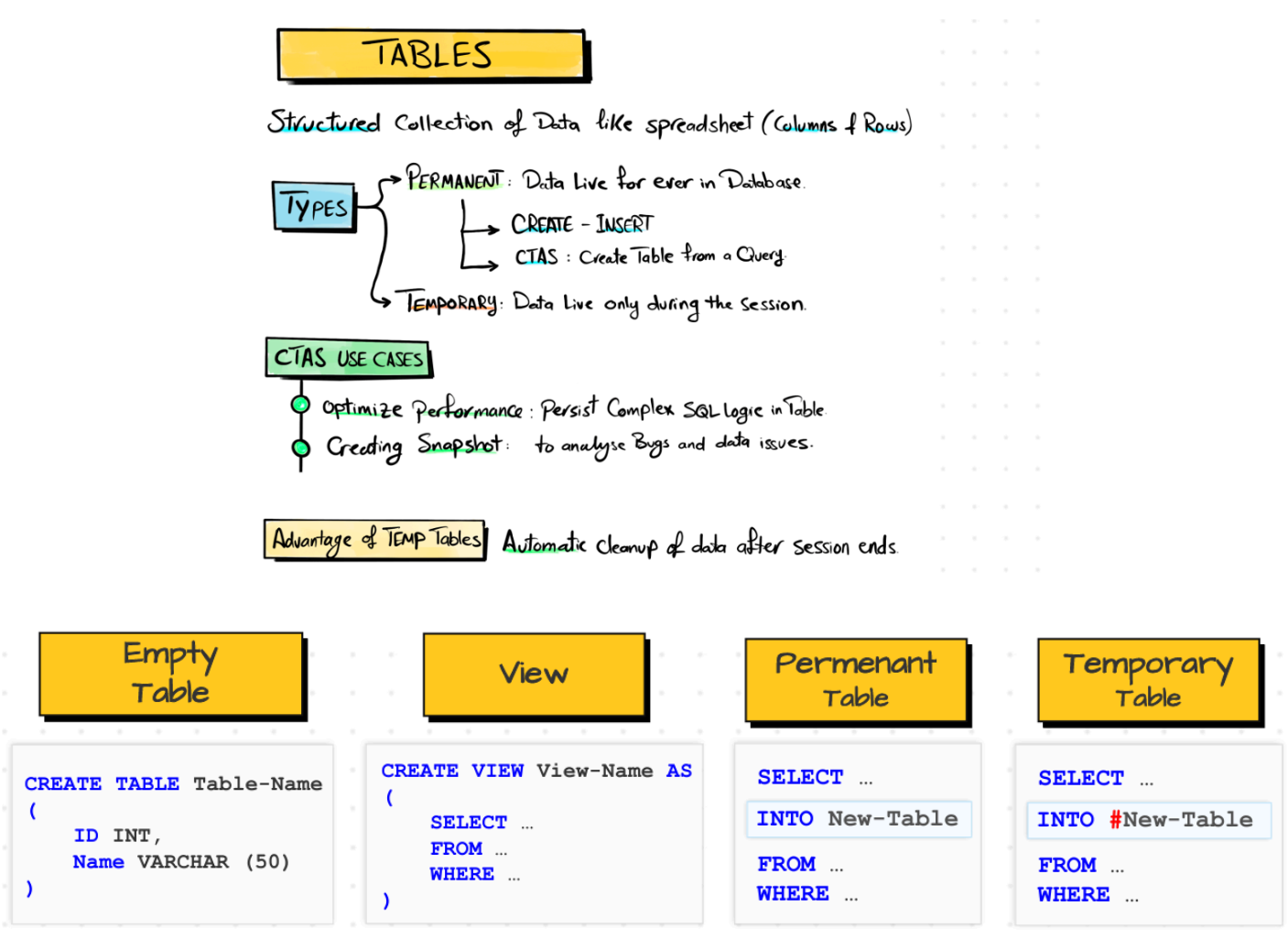




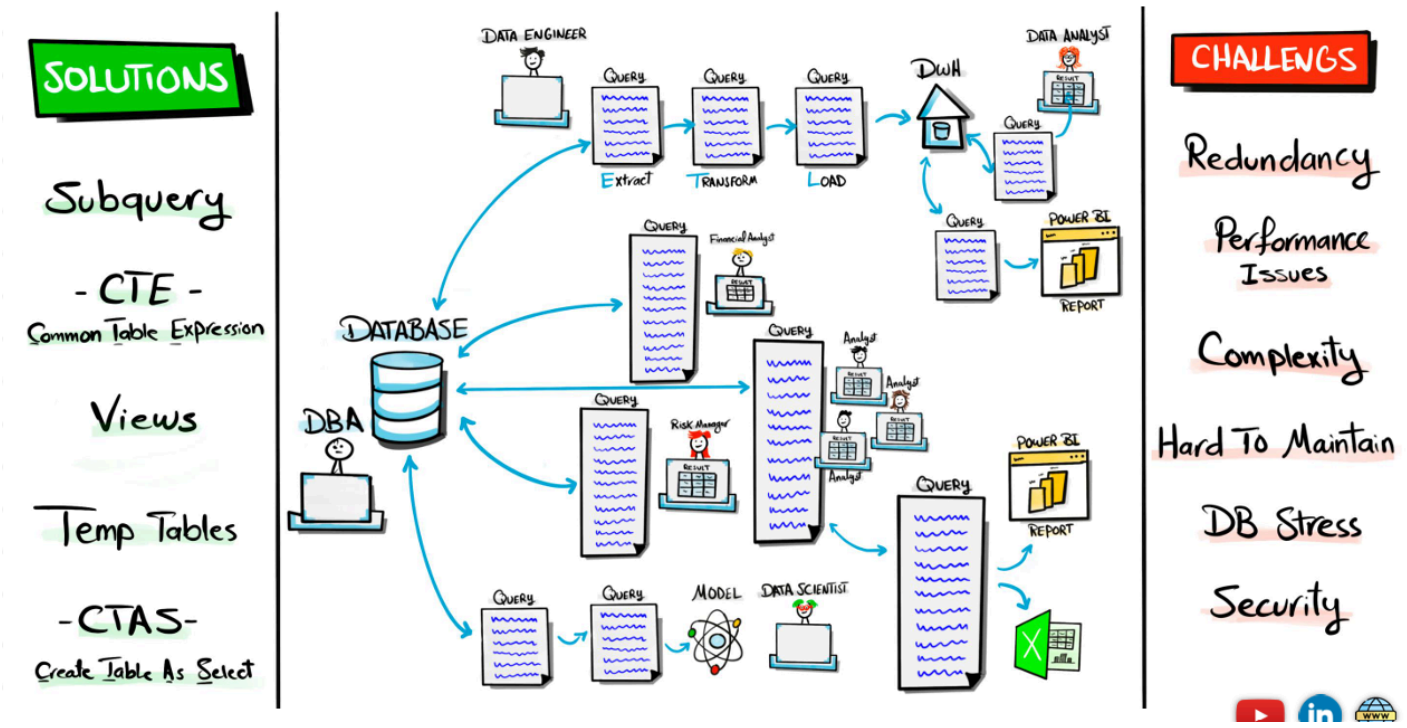
TEMP USE CASE: Intermediate Results

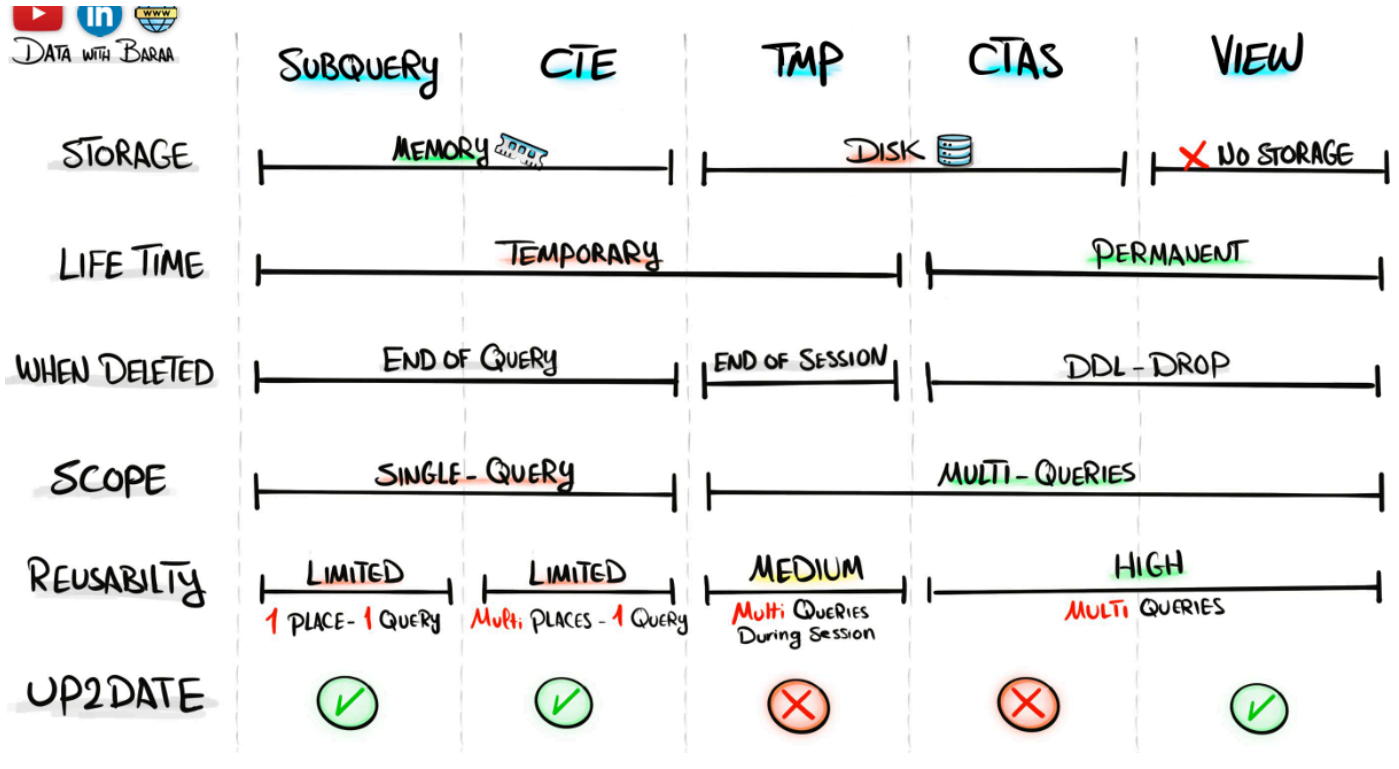


SQL Tables Summary

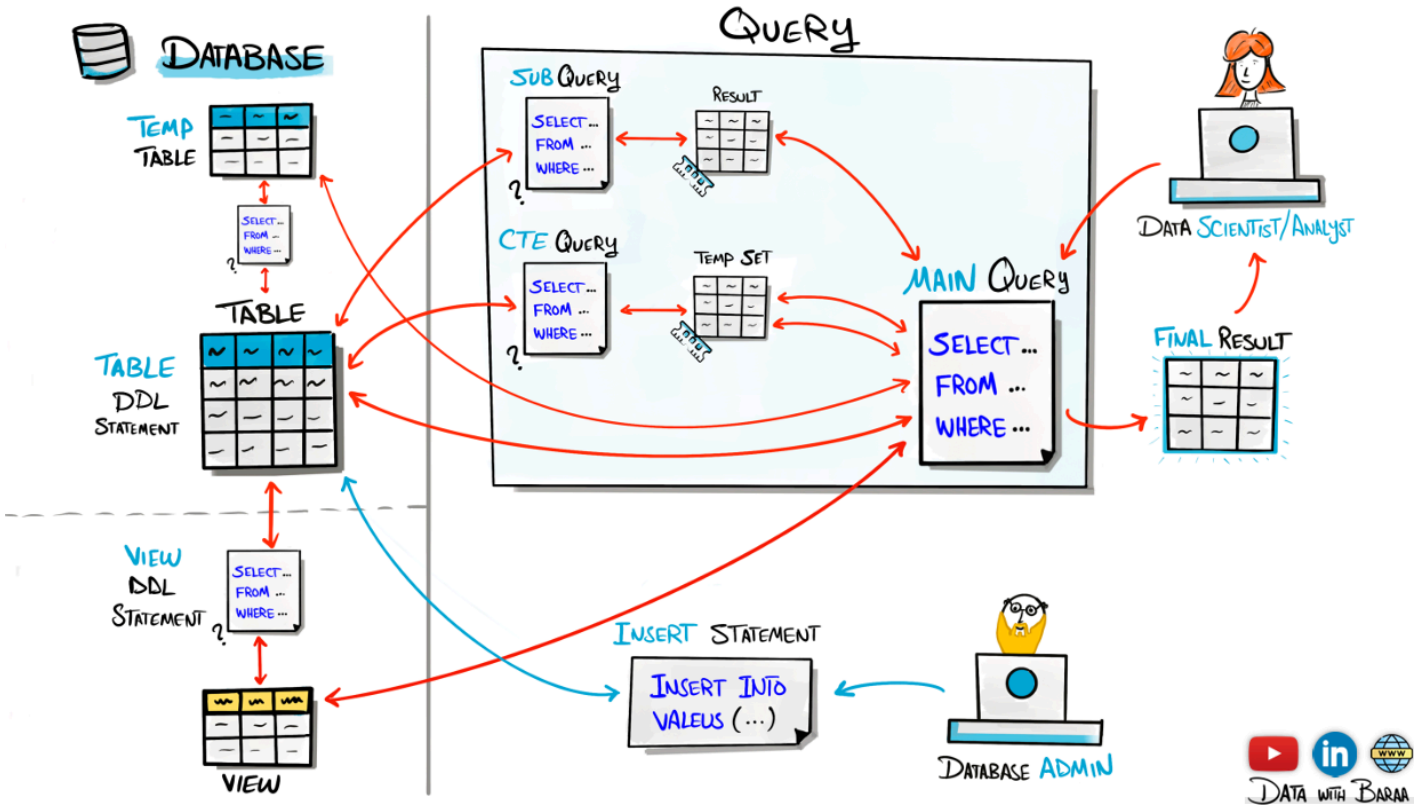


Compare Advanced SQL Techniques

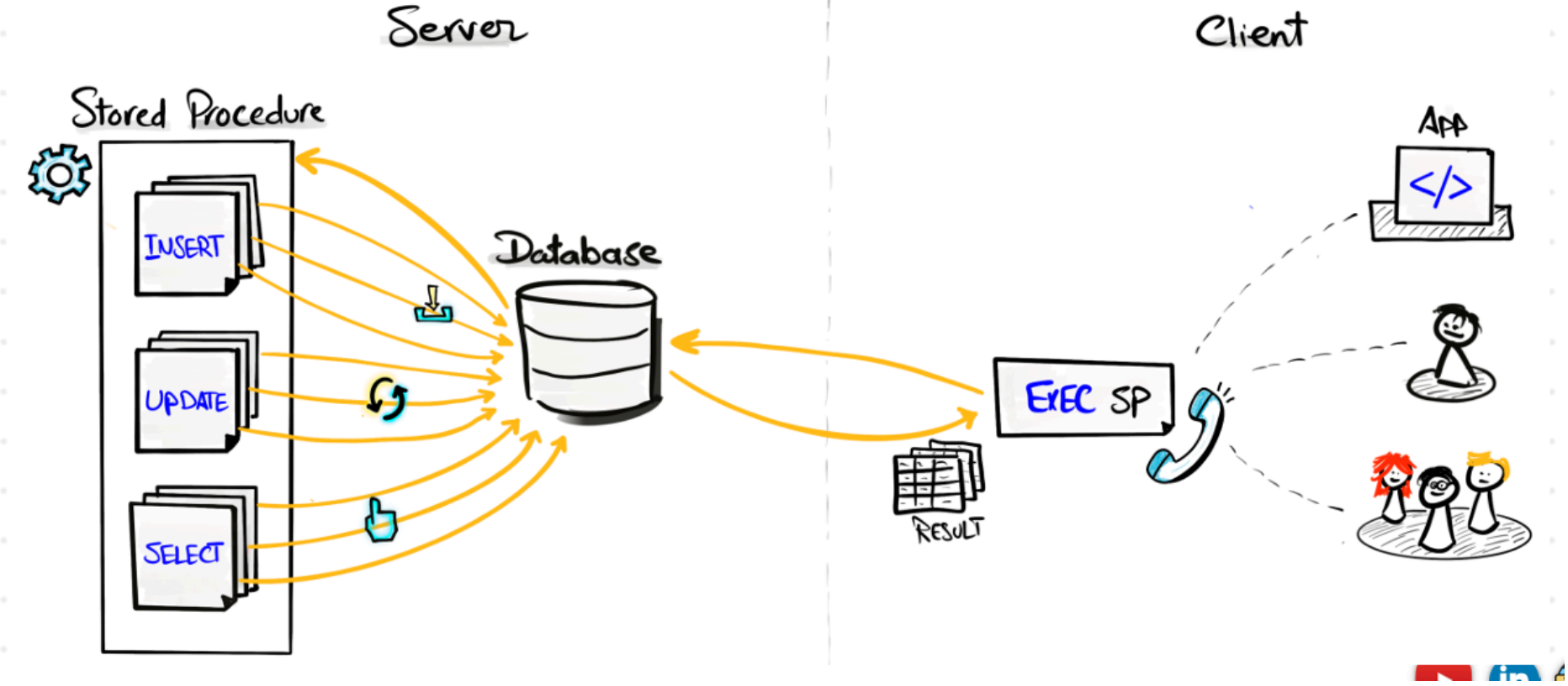




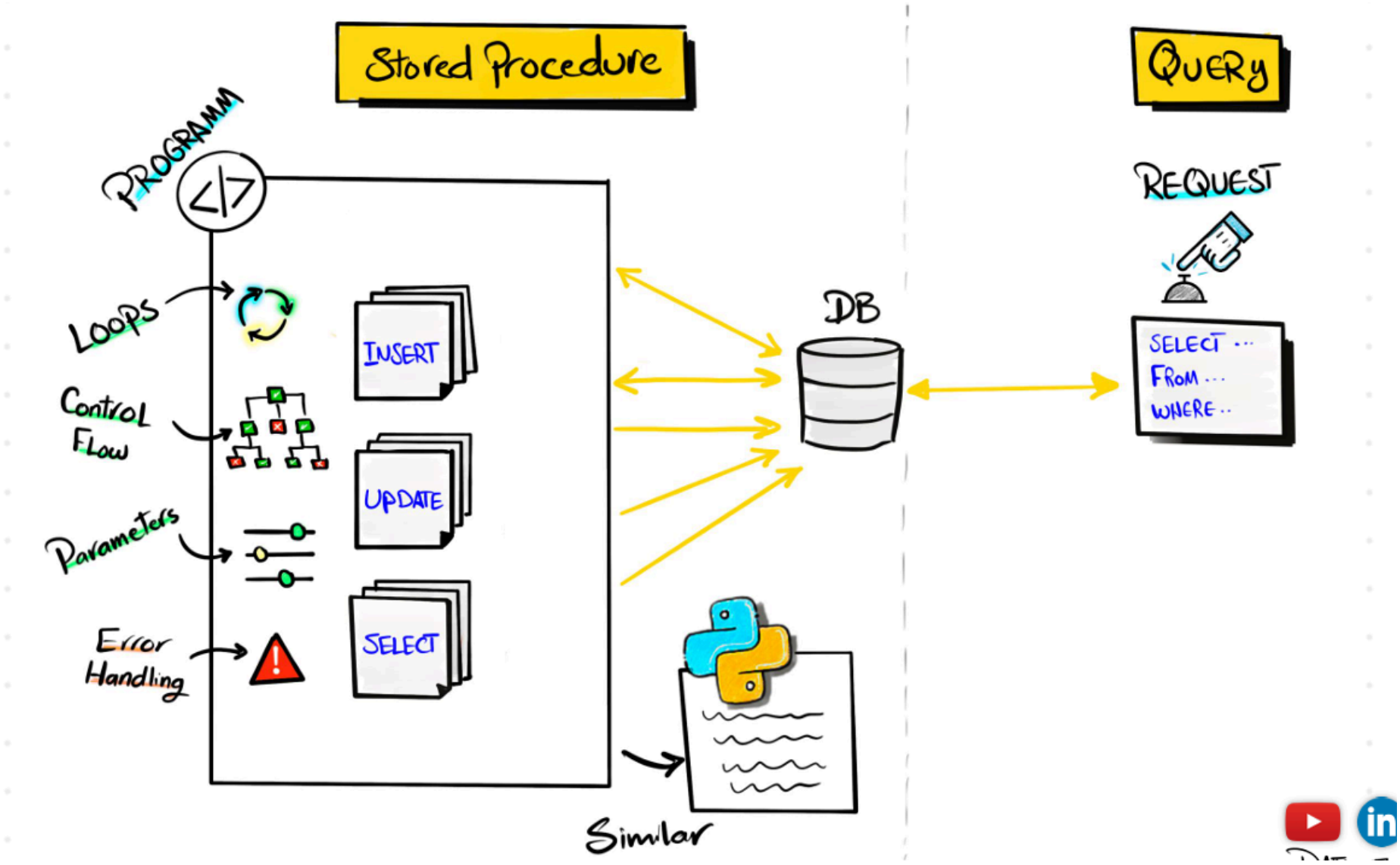
SQL Project Big Picture



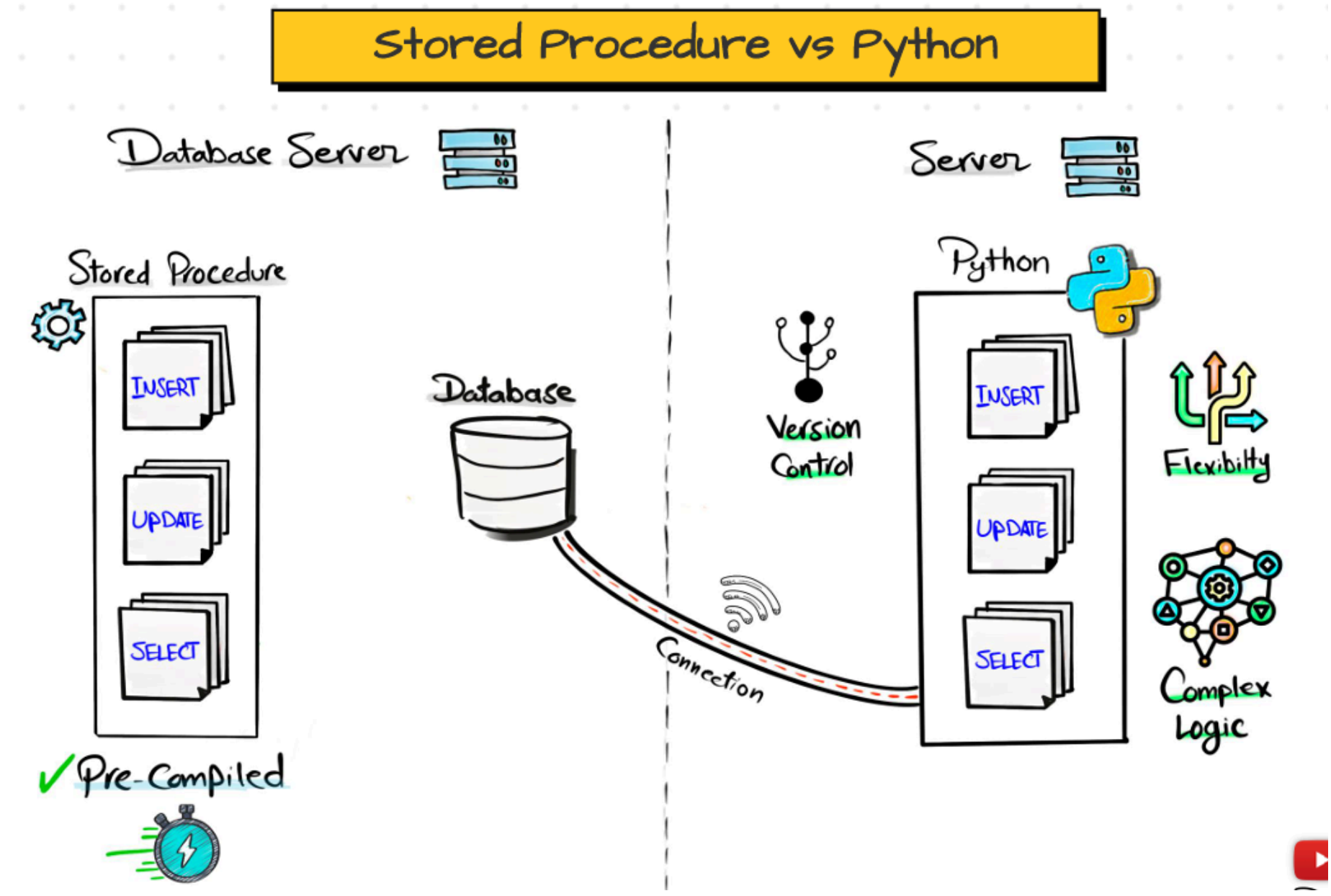
Module 21: Stored Procedures



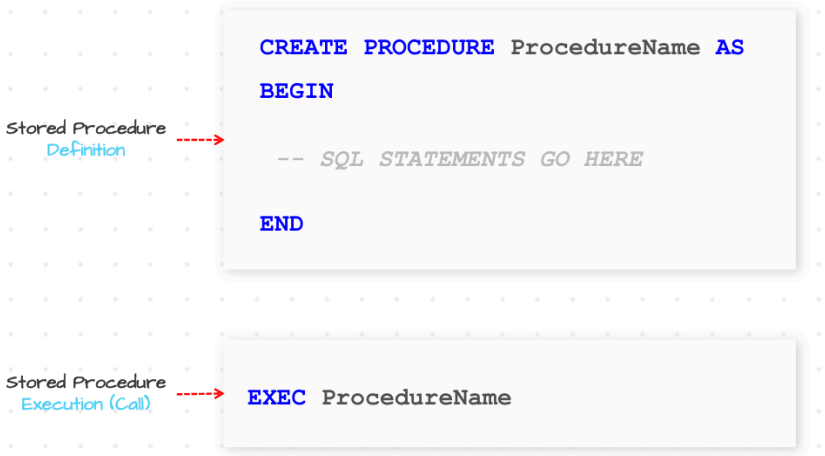
Stored Procedure vs Query



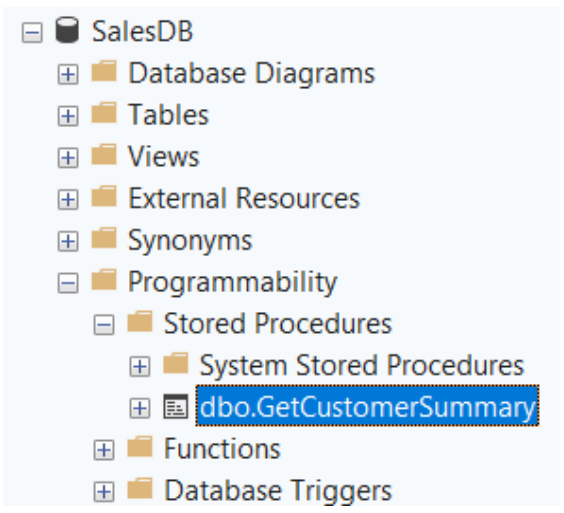
Stored Procedure vs Coding with Python

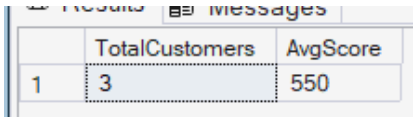


Stored Procedure Syntax




```sql
-- Step1: Write a Query
-- For US Customers, find the total number of customers and the average score
SELECT
    COUNT(CustomerID) TotalCustomers,
    AVG(COALESCE(Score,0)) AvgScore
FROM Sales.Customers
WHERE Country = 'USA';

-- Step2: Turning the query int a stored procedure
CREATE PROCEDURE GetCustomerSummary AS
BEGIN
    SELECT
        COUNT(CustomerID) TotalCustomers,
        AVG(COALESCE(Score,0)) AvgScore
    FROM Sales.Customers
    WHERE Country = 'USA'
END;

-- Step3: Execute the stored procedure
EXEC GetCustomerSummary;
```





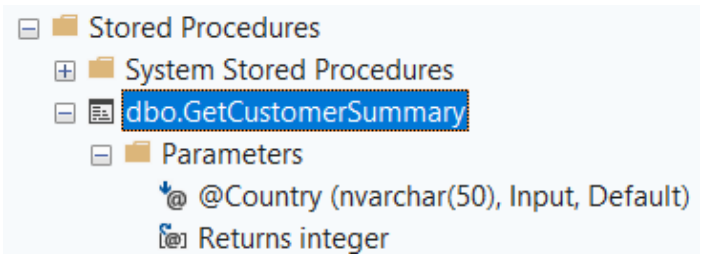
	TotalCustomers	AvgScore
1	3	550

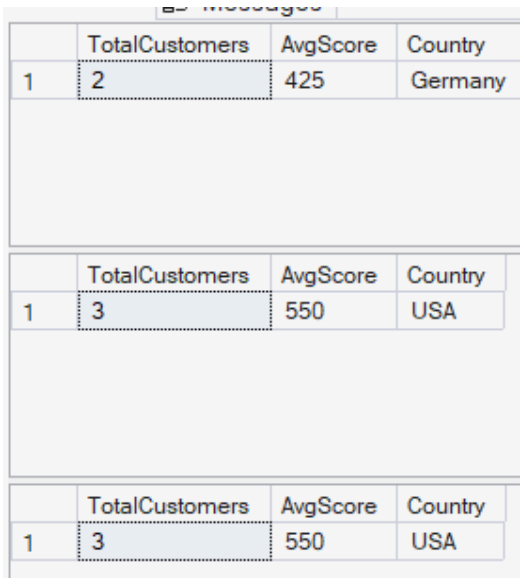
Stored Procedure Parameters

- Placeholders used to pass values as input from the caller to the procedure, allowing dynamic data to be processed
- AVOID REPETETION
  - If you notice repeated code in your project, it’s a sign that your code can be improved

```sql
-- Step1: Define the parameter (Name, Datatype, Default Value)
-- Step2: Use the parameter
-- Step3: Pass the parameter's value at execution
```

```sql
ALTER PROCEDURE GetCustomerSummary @Country NVARCHAR(50) = 'USA'
AS
BEGIN
    SELECT
        COUNT(CustomerID) TotalCustomers,
        AVG(COALESCE(Score,0)) AvgScore,
        @Country Country
    FROM Sales.Customers
    WHERE Country = @Country
END;

EXEC GetCustomerSummary @Country = 'Germany';
EXEC GetCustomerSummary @Country = 'USA';
EXEC GetCustomerSummary;
```





	TotalCustomers	AvgScore	Country
1	2	425	Germany

	TotalCustomers	AvgScore	Country
1	3	550	USA

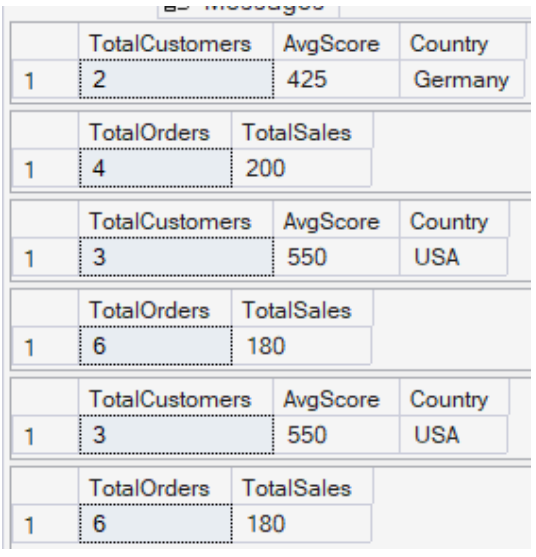
	TotalCustomers	AvgScore	Country
1	3	550	USA

Stored Procedure Multi-Queries

- TIP
  - Add a semicolon (;) at the end of each SQL statement

```sql
CREATE PROCEDURE GetCustomerOrdersSummary @Country NVARCHAR(50) = 'USA'
AS
BEGIN
    SELECT
        COUNT(CustomerID) TotalCustomers,
        AVG(COALESCE(Score,0)) AvgScore,
        @Country Country
    FROM Sales.Customers
    WHERE Country = @Country;

    SELECT
        COUNT(OrderID) TotalOrders,
        SUM(Sales) TotalSales
    FROM Sales.Orders o
    JOIN Sales.Customers c
    ON c.CustomerID = o.CustomerID
    WHERE c.Country = @Country;
END;
```

```sql
EXEC GetCustomerOrdersSummary @Country = 'Germany';
EXEC GetCustomerOrdersSummary @Country = 'USA';
EXEC GetCustomerOrdersSummary;
```



	TotalCustomers	AvgScore	Country
1	2	425	Germany

	TotalOrders	TotalSales
1	4	200

	TotalCustomers	AvgScore	Country
1	3	550	USA

	TotalOrders	TotalSales
1	6	180

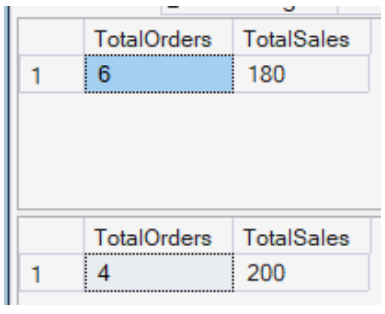
	TotalCustomers	AvgScore	Country
1	3	550	USA

	TotalOrders	TotalSales
1	6	180

Stored Procedure Variables

- Placeholders used to store values to be used later in the procedure
- Variables temporarily store and manipulate data during its execution

```sql
-- Step1: Declare variables
-- Step2: Assign values to variables
ALTER PROCEDURE GetCustomerOrdersSummary @Country NVARCHAR(50) = 'USA'
AS
BEGIN
    DECLARE @TotalCustomers INT, @AvgScore FLOAT;

    SELECT
        @TotalCustomers = COUNT(CustomerID),
        @AvgScore = AVG(COALESCE(Score,0))
    FROM Sales.Customers
    WHERE Country = @Country;

    PRINT 'Total Customers from ' + @Country + ': ' + CAST(@TotalCustomers AS NVARCHAR);
    PRINT 'Average Score from ' + @Country + ': ' + CAST(@AvgScore AS NVARCHAR);

    SELECT
        COUNT(OrderID) TotalOrders,
        SUM(Sales) TotalSales
    FROM Sales.Orders o
    JOIN Sales.Customers c
    ON c.CustomerID = o.CustomerID
    WHERE c.Country = @Country;
END;

EXEC GetCustomerOrdersSummary;
EXEC GetCustomerOrdersSummary @Country = 'Germany';
```



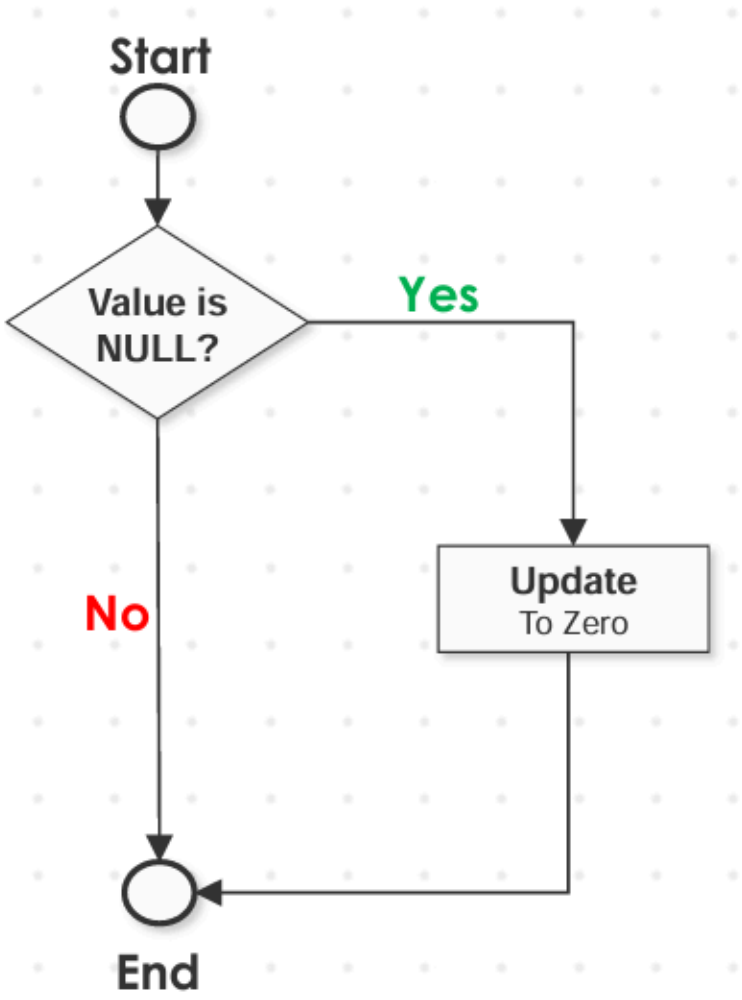
	TotalOrders	TotalSales
1	6	180

	TotalOrders	TotalSales
1	4	200

```
Results  Messages
    Total Customers from USA: 3
    Average Score from USA: 550

    (1 row affected)
    Total Customers from Germany: 2
    Average Score from Germany: 425

    (1 row affected)
```

Stored Procedure Control Flow (IF ELSE)

- Handling nulls before aggregating to ensure accurate results




```sql
ALTER PROCEDURE GetCustomerOrdersSummary @Country NVARCHAR(50) = 'USA'
AS
BEGIN
    DECLARE @TotalCustomers INT, @AvgScore FLOAT;

    -- Prepare & Cleanup Data
    IF EXISTS (SELECT 1 FROM Sales.Customers WHERE Score IS NULL AND Country = @Country)
    BEGIN
        PRINT 'Updating NULL Scores to 0';
        UPDATE Sales.Customers
        SET Score = 0
        WHERE Score IS NULL AND Country = @Country;
    END

    ELSE
    BEGIN
        PRINT 'No NULL Scores found';
    END;

    -- Generating Reports
    SELECT
        @TotalCustomers = COUNT(CustomerID),
        @AvgScore = AVG(Score)
    FROM Sales.Customers
```

```sql
    WHERE Country = @Country;

    PRINT 'Total Customers from ' + @Country + ': ' + CAST(@TotalCustomers AS NVARCHAR);
    PRINT 'Average Score from ' + @Country + ': ' + CAST(@AvgScore AS NVARCHAR);

    SELECT
        COUNT(OrderID) TotalOrders,
        SUM(Sales) TotalSales
    FROM Sales.Orders o
    JOIN Sales.Customers c
    ON c.CustomerID = o.CustomerID
    WHERE c.Country = @Country;
END;

EXEC GetCustomerOrdersSummary;
EXEC GetCustomerOrdersSummary@Country = 'Germany';
```

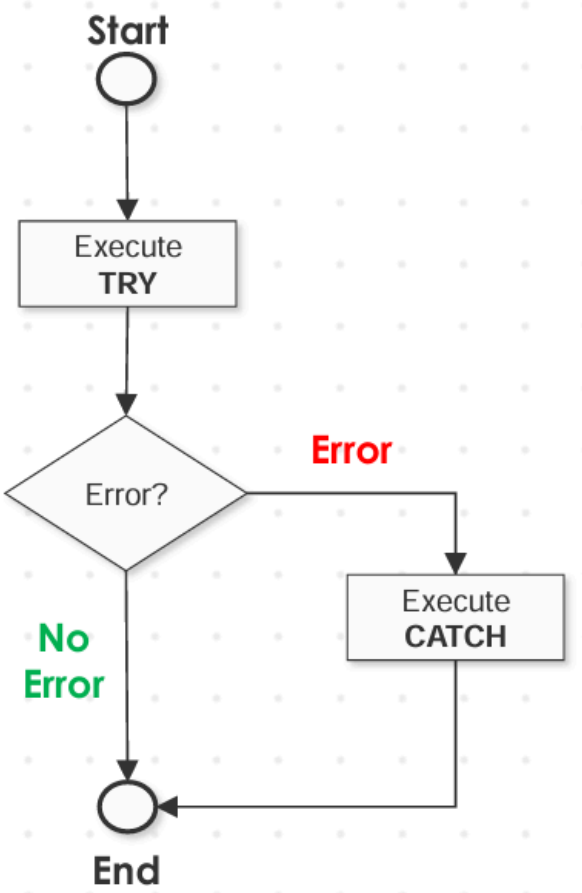
Results | Messages

```
Updating NULL Scores to 0

(1 row affected)
Total Customers from USA: 3
Average Score from USA: 550

(1 row affected)
No NULL Scores found
Total Customers from Germany: 2
Average Score from Germany: 425

(1 row affected)
```

Stored Procedure Error Handling (TRY & CATCH)

```sql
BEGIN TRY

-- SQL statements that might cause an error

END TRY
```

```sql
BEGIN CATCH

-- SQL statements To Handle The Error

END CATCH
```




```sql
ALTER PROCEDURE GetCustomerOrdersSummary @Country NVARCHAR(50) = 'USA'
AS
BEGIN
    BEGIN TRY
        DECLARE @TotalCustomers INT, @AvgScore FLOAT;

        -- Prepare & Cleanup Data
```

```sql
        IF EXISTS (SELECT 1 FROM Sales.Customers WHERE Score IS NULL AND Country = @Country)
        BEGIN
            PRINT 'Updating NULL Scores to 0';
            UPDATE Sales.Customers
            SET Score = 0
            WHERE Score IS NULL AND Country = @Country;
        END

        ELSE
        BEGIN
            PRINT 'No NULL Scores found';
        END;

        -- Generating Reports
        SELECT
            @TotalCustomers = COUNT(CustomerID),
            @AvgScore = AVG(Score)
        FROM Sales.Customers
        WHERE Country = @Country;

        PRINT 'Total Customers from ' + @Country + ': ' + CAST(@TotalCustomers AS NVARCHAR);
        PRINT 'Average Score from ' + @Country + ': ' + CAST(@AvgScore AS NVARCHAR);

        SELECT
            COUNT(OrderID) TotalOrders,
            SUM(Sales) TotalSales,
            1/0    /* -> ERROR */
        FROM Sales.Orders o
        JOIN Sales.Customers c
        ON c.CustomerID = o.CustomerID
        WHERE c.Country = @Country;
    END TRY
    BEGIN CATCH
        PRINT 'An error occured.';
        PRINT 'Error Message: ' + ERROR_MESSAGE();
        PRINT 'Error Number: ' + CAST(ERROR_NUMBER() AS NVARCHAR);
        PRINT 'Error Line: ' + CAST(ERROR_LINE() AS NVARCHAR);
        PRINT 'Error Procedure: ' + ERROR_PROCEDURE();
    END CATCH;
END;

EXEC GetCustomerOrdersSummary;
```

Results Messages

```
No NULL Scores found
Total Customers from USA: 3
Average Score from USA: 550

(0 rows affected)
An error occured.
Error Message: Divide by zero error encountered.
Error Number: 8134
Error Line: 32
Error Procedure: GetCustomerOrdersSummary
```

Styling Stored Procedure

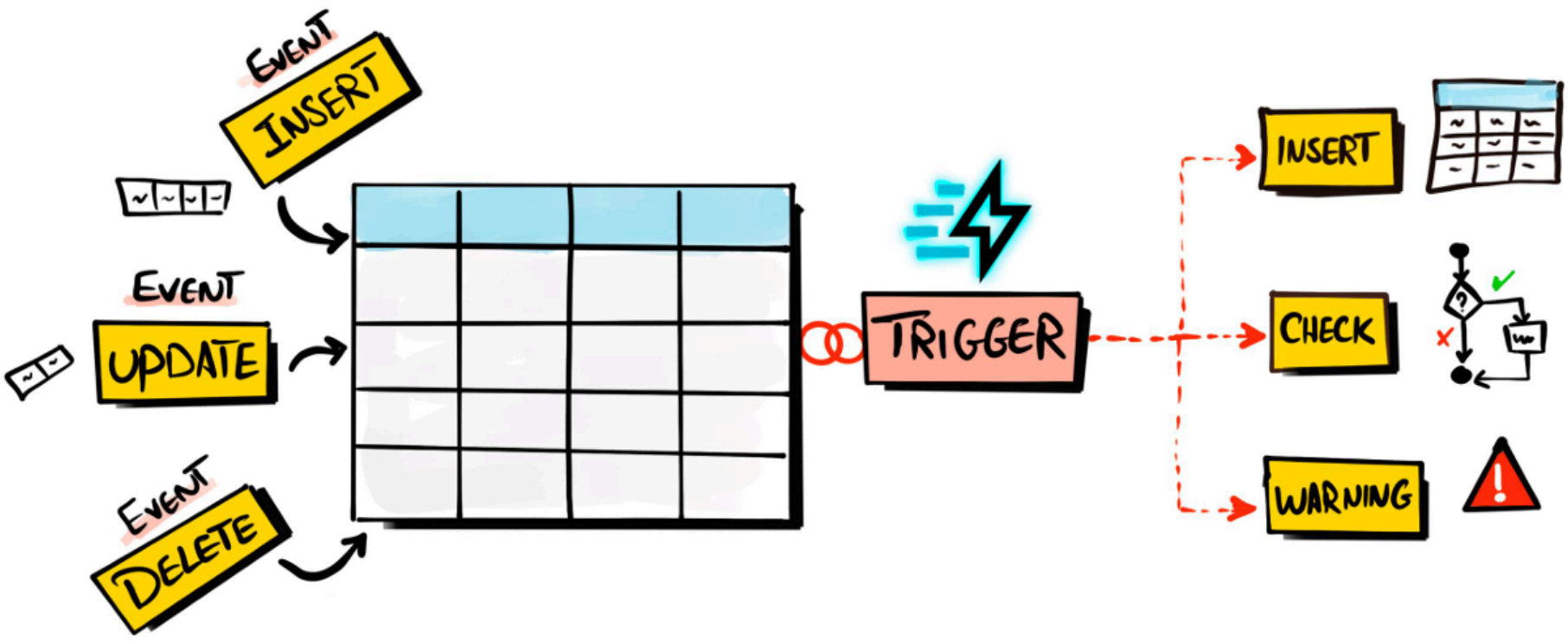
- Use TABS inside BEGIN and END blocks
- Include comments to explain the purpose of code sections

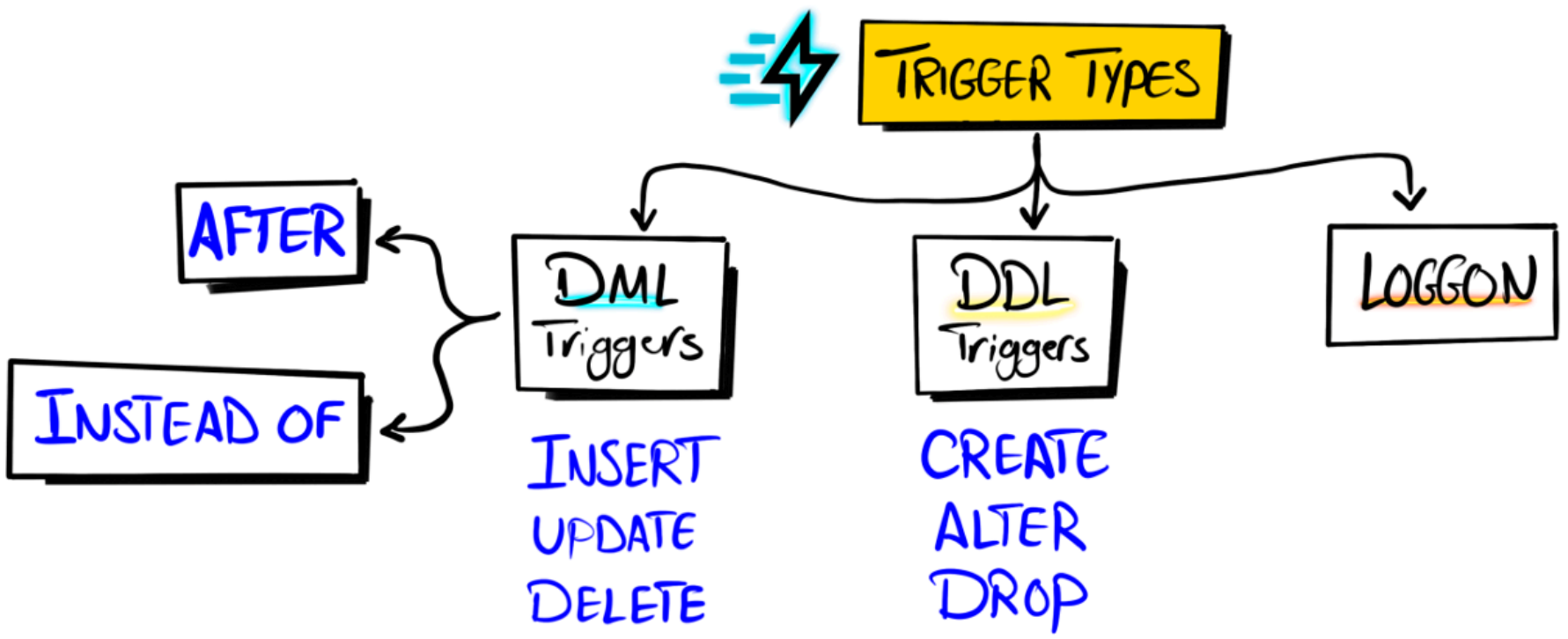
```sql
ALTER PROCEDURE GetCustomerOrdersSummary @Country NVARCHAR(50) = 'USA'
AS
BEGIN
    BEGIN TRY
        DECLARE @TotalCustomers INT, @AvgScore FLOAT;

        -- ============================
        -- Step1: Prepare & Cleanup Data
        -- ============================
        IF EXISTS (SELECT 1 FROM Sales.Customers WHERE Score IS NULL AND Country = @Country)
        BEGIN
            PRINT 'Updating NULL Scores to 0';
            UPDATE Sales.Customers
            SET Score = 0
            WHERE Score IS NULL AND Country = @Country;
        END

        ELSE
        BEGIN
            PRINT 'No NULL Scores found';
        END;

        -- ================================
        -- Step2: Generating Summary Reports
        -- ================================
        -- Calculate total customers and average score for specific country
        SELECT
            @TotalCustomers = COUNT(CustomerID),
            @AvgScore = AVG(Score)
        FROM Sales.Customers
        WHERE Country = @Country;

        PRINT 'Total Customers from ' + @Country + ': ' + CAST(@TotalCustomers AS NVARCHAR);
        PRINT 'Average Score from ' + @Country + ': ' + CAST(@AvgScore AS NVARCHAR);

        -- Calculate total orders and total sales for specific country
        SELECT
            COUNT(OrderID) TotalOrders,
            SUM(Sales) TotalSales
        FROM Sales.Orders o
            JOIN Sales.Customers c
            ON c.CustomerID = o.CustomerID
            WHERE c.Country = @Country;
    END TRY
    BEGIN CATCH
        -- ==============
        -- Error Handling
        -- ==============
        PRINT 'An error occured.';
        PRINT 'Error Message: ' + ERROR_MESSAGE();
        PRINT 'Error Number: ' + CAST(ERROR_NUMBER() AS NVARCHAR);
        PRINT 'Error Line: ' + CAST(ERROR_LINE() AS NVARCHAR);
        PRINT 'Error Procedure: ' + ERROR_PROCEDURE();
    END CATCH;
END;
```

Module 22: Triggers

What are Triggers

- Special stored procedure (set of statements) that automatically runs in response to a specific event on a table or view





- AFTER
  - Runs AFTER event
- INSTEAD OF
  - Runs DURING event

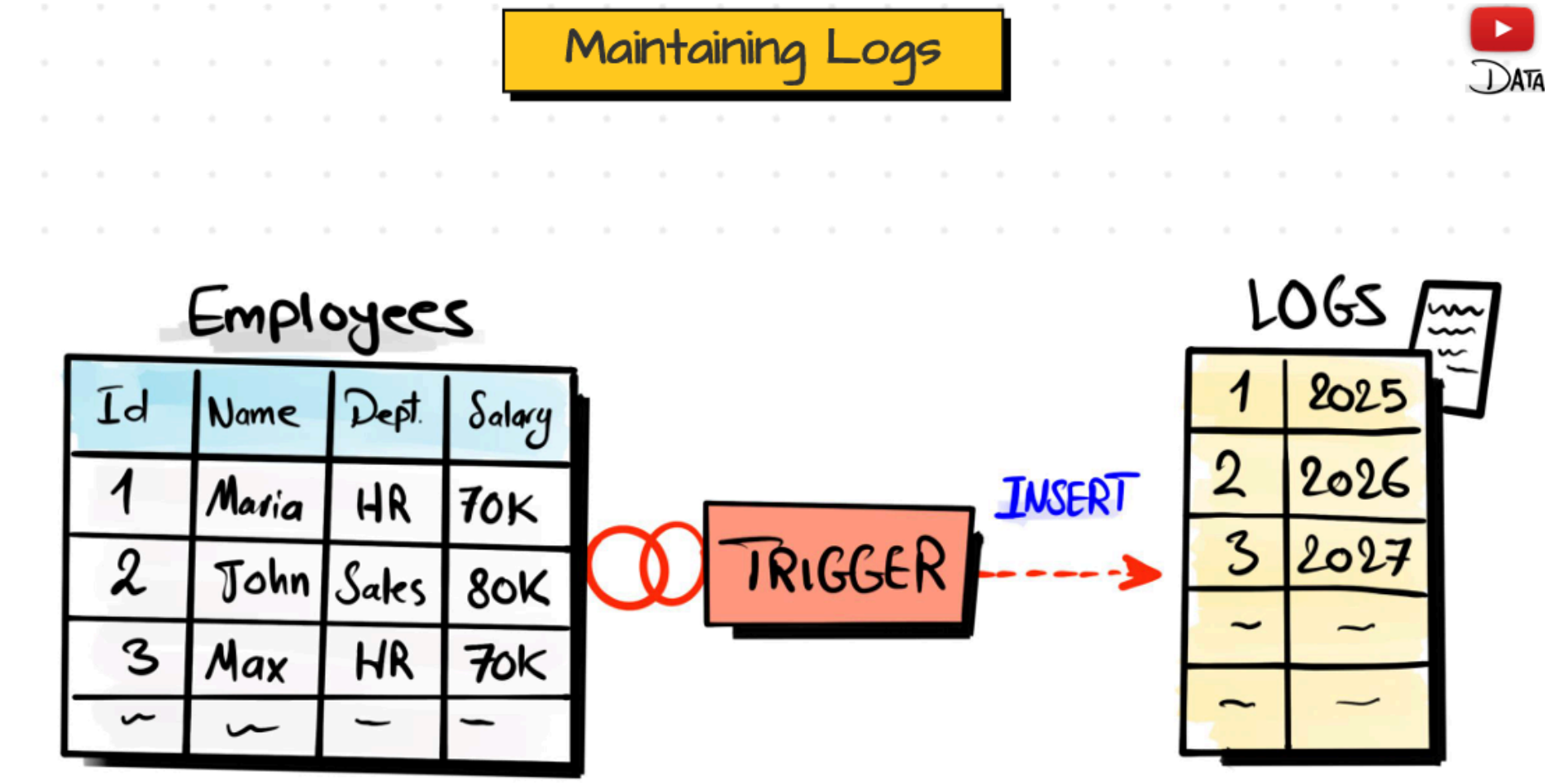
Triggers Syntax

```
CREATE TRIGGER TriggerName ON TableName
AFTER INSERT, UPDATE, DELETE
BEGIN
    -- SQL STATEMENTS GO HERE
END
```

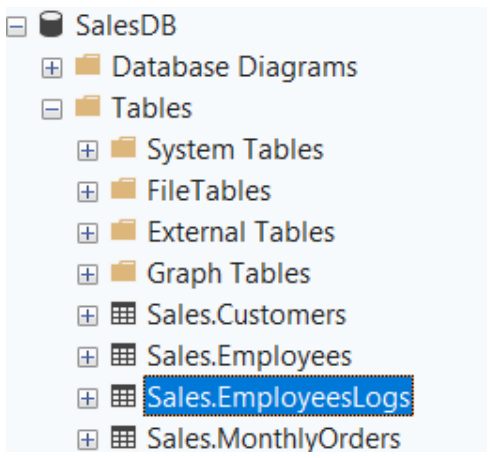
WHEN ----->

WHAT ----->

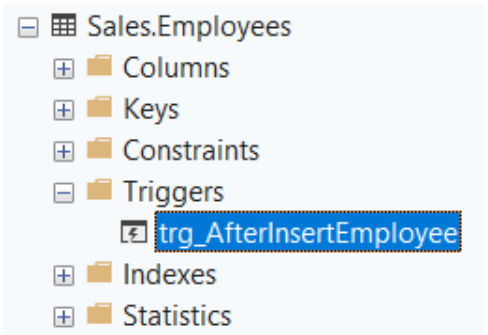
TRIGGERS USE CASE: LOGGING




```sql
-- Step1: Create Log Table
CREATE TABLE Sales.EmployeesLogs (
    LogID INT IDENTITY(1,1) PRIMARY KEY,
    EmployeeID INT,
    LogMessage VARCHAR(255),
    LogDate DATE
);
```




```sql
-- Step2: Create Trigger on Employees Table
CREATE TRIGGER trg_AfterInsertEmployee ON Sales.Employees
AFTER INSERT
AS
BEGIN
    INSERT INTO Sales.EmployeesLogs (EmployeeID, LogMessage, LogDate)
    SELECT
        EmployeeID,
        'New Employee Added = ' + CAST(EmployeeID AS VARCHAR),
        GETDATE()
    FROM INSERTED
END;
```



- INSERTED
  - Virtual table that holds a copy of the rows that are being inserted onto the target table

```
-- Step3: Insert New Data into Employees
SELECT * FROM Sales.EmployeesLogs;

INSERT INTO Sales.Employees
VALUES
(6, 'Maria', 'Doe', 'HR', '1988-01-12', 'F', 80000, 3);

SELECT * FROM Sales.Employees;

SELECT * FROM Sales.EmployeesLogs;
```

	LogID	EmployeeID	LogMessage	LogDate

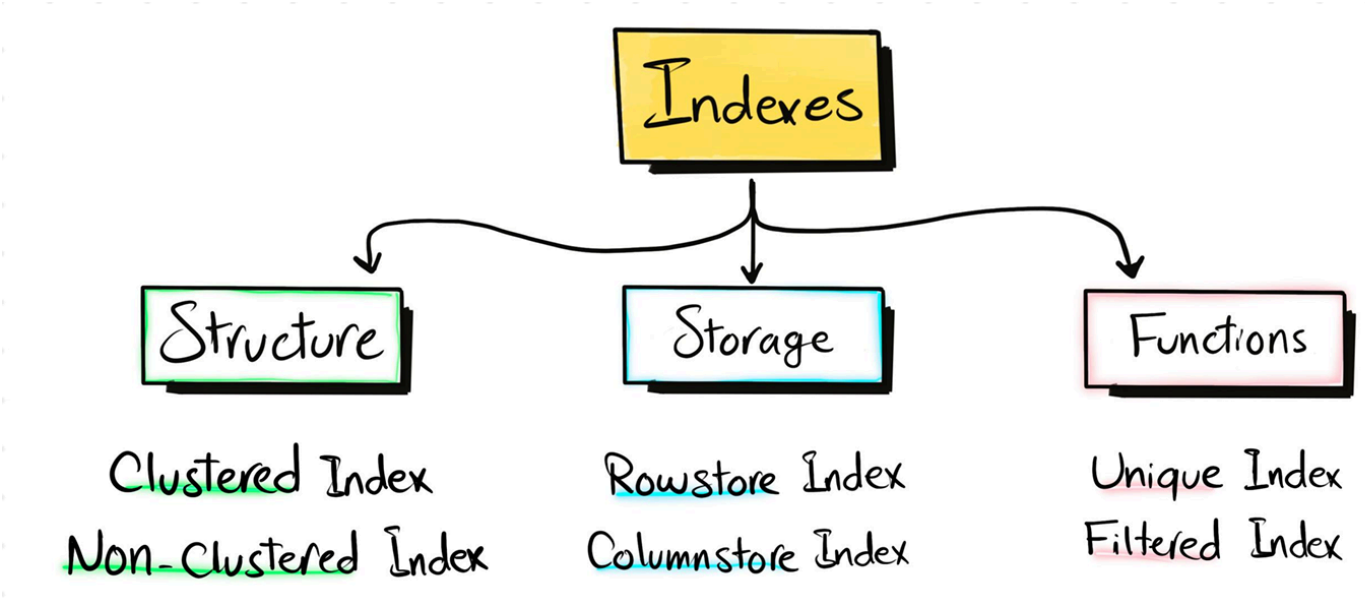
	EmployeeID	FirstName	LastName	Department	BirthDate	Gender	Salary	ManagerID
1	1	Frank	Lee	Marketing	1988-12-05	M	55000	NULL
2	2	Kevin	Brown	Marketing	1972-11-25	M	65000	1
3	3	Mary	NULL	Sales	1986-01-05	F	75000	1
4	4	Michael	Ray	Sales	1977-02-10	M	90000	2
5	5	Carol	Baker	Sales	1982-02-11	F	55000	3
6	6	Maria	Doe	HR	1988-01-12	F	80000	3

	LogID	EmployeeID	LogMessage	LogDate
1	1	6	New Employee Added = 6	2026-10-24

Module 23: Indexes

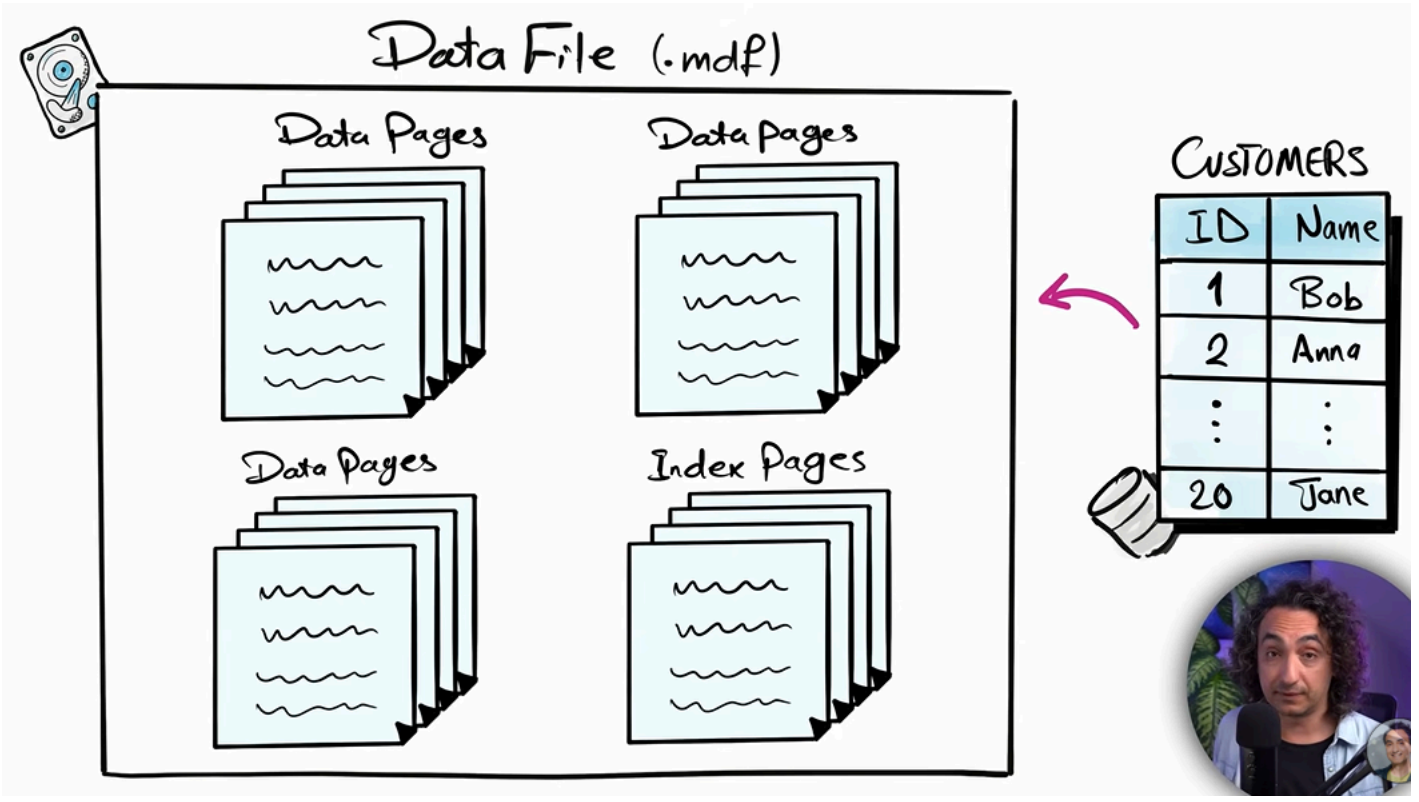
- Data structure that provides quick access to data, optimizing the speed of your queries

Index Types

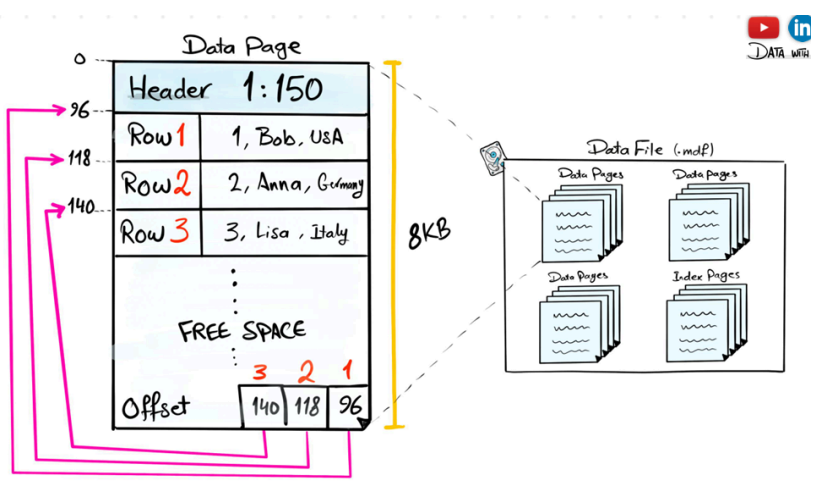


- Trade-off
  - Some Indexes are better for reading, others for writing performance

Database Tables Without Indexes

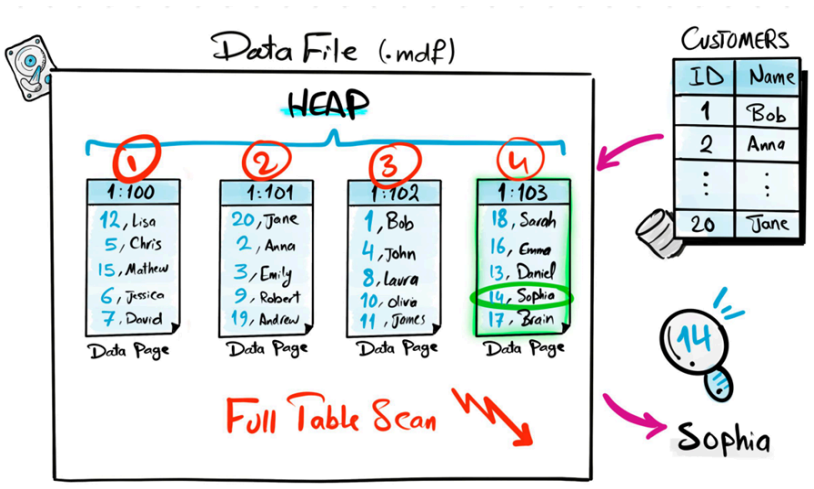


- Page
  - the smallest unit of data storage in a database (8kb)
  - It stores anything (Data, Metadata, Indexes, etc.)
  - Types
    - Data Page
    - Index Page



Heap Structure

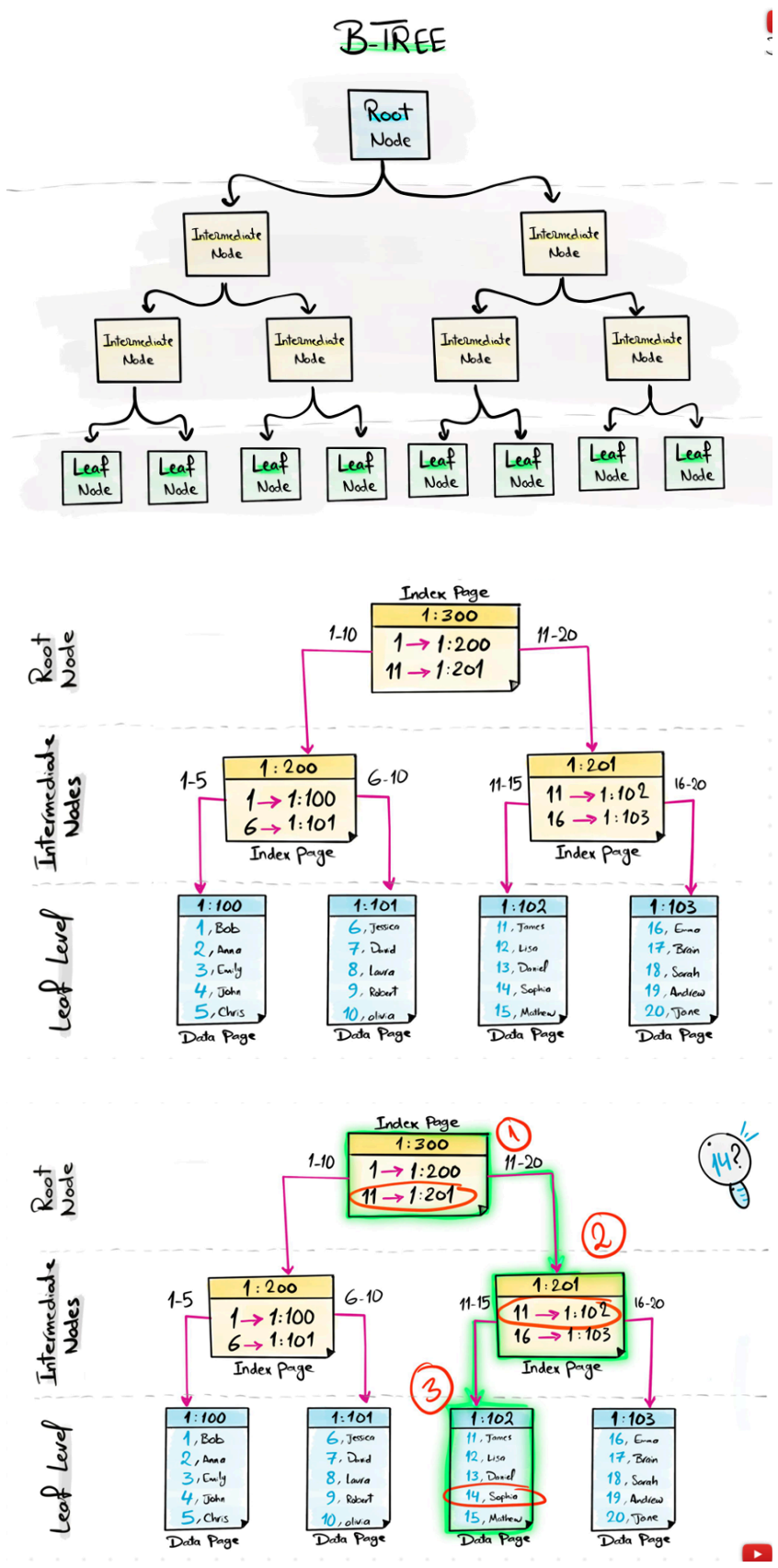
- HEAP = Table without Clustered Index
- Fast Write, Slow Read



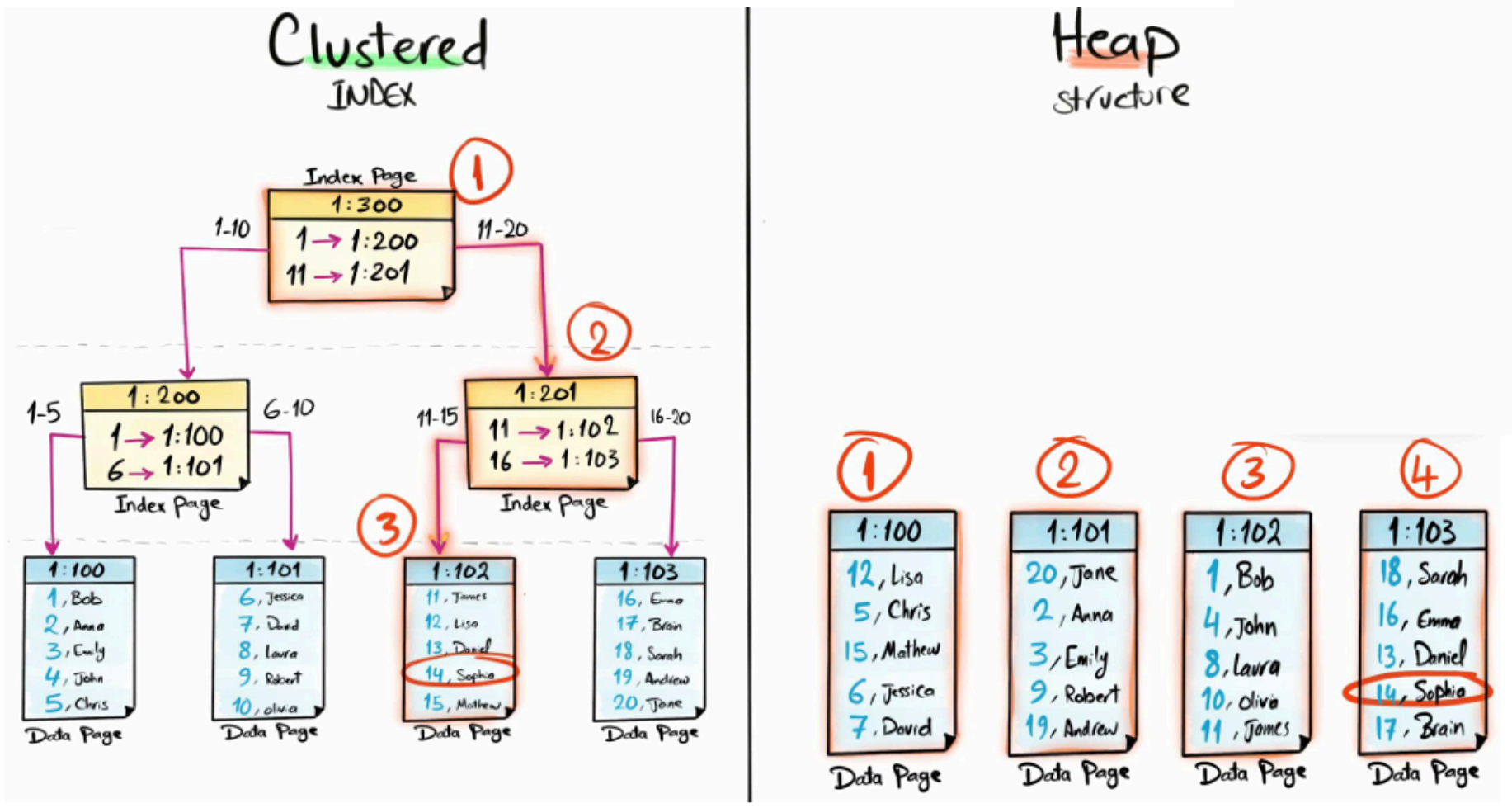
- Full Table Scan
  - Scans the entire table page by page and row by row, searching for data

Clustered Index

- A clustered index will sort all the data based on column ID in each data page
- B-Tree (Balance Tree)
  - Hierarchical structure storing data at leaves, to help quickly locate data
- Index Page
  - It stores key values (Pointers) to another page
  - It doesn’t store the actual rows

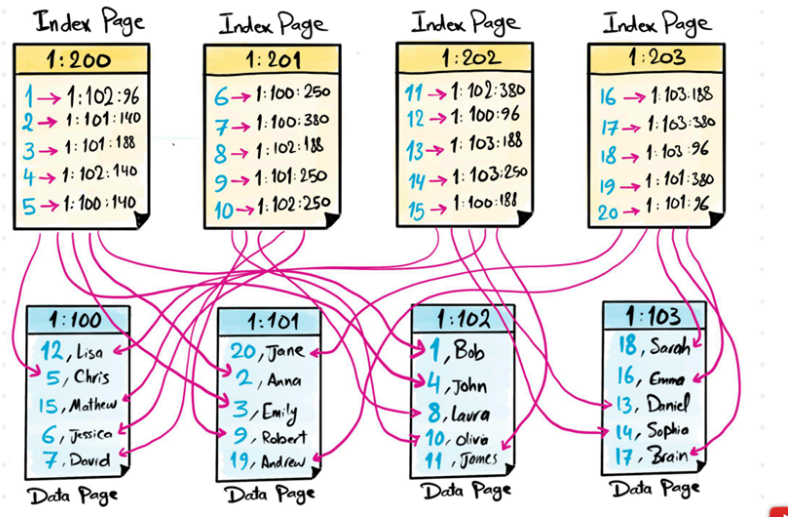


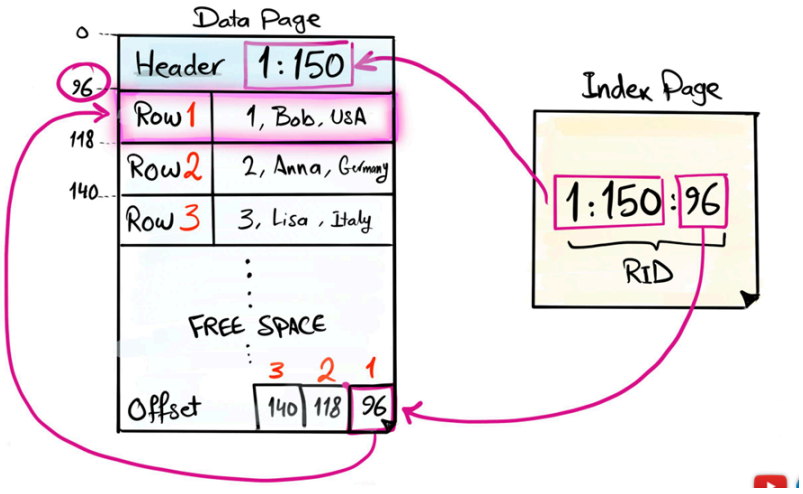
Clustered Index vs Heap Structure

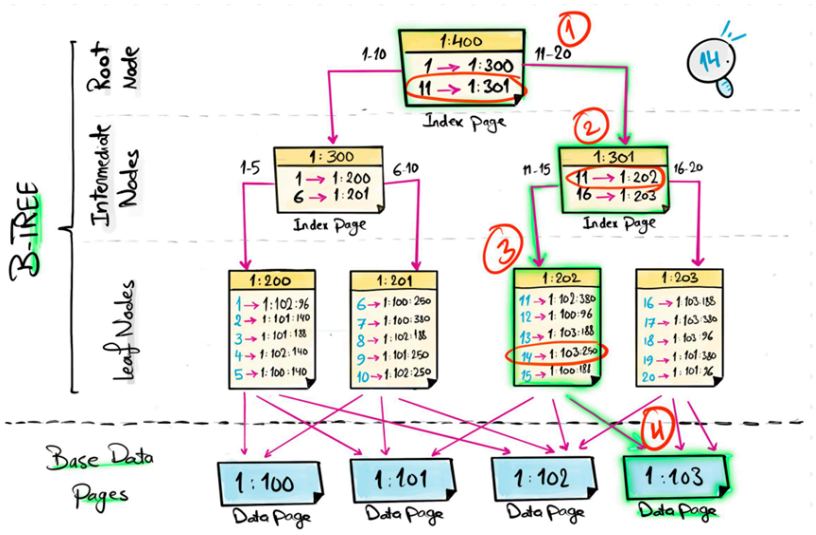


Non-Clustered Index

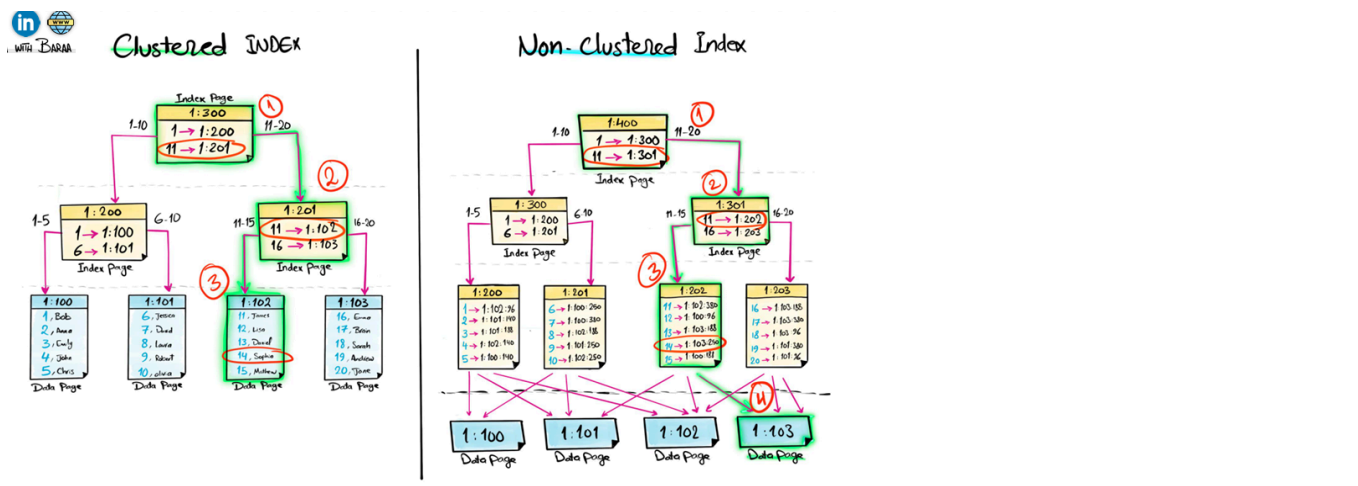
- A non-clustered index won't reorganize or change anything on the data page

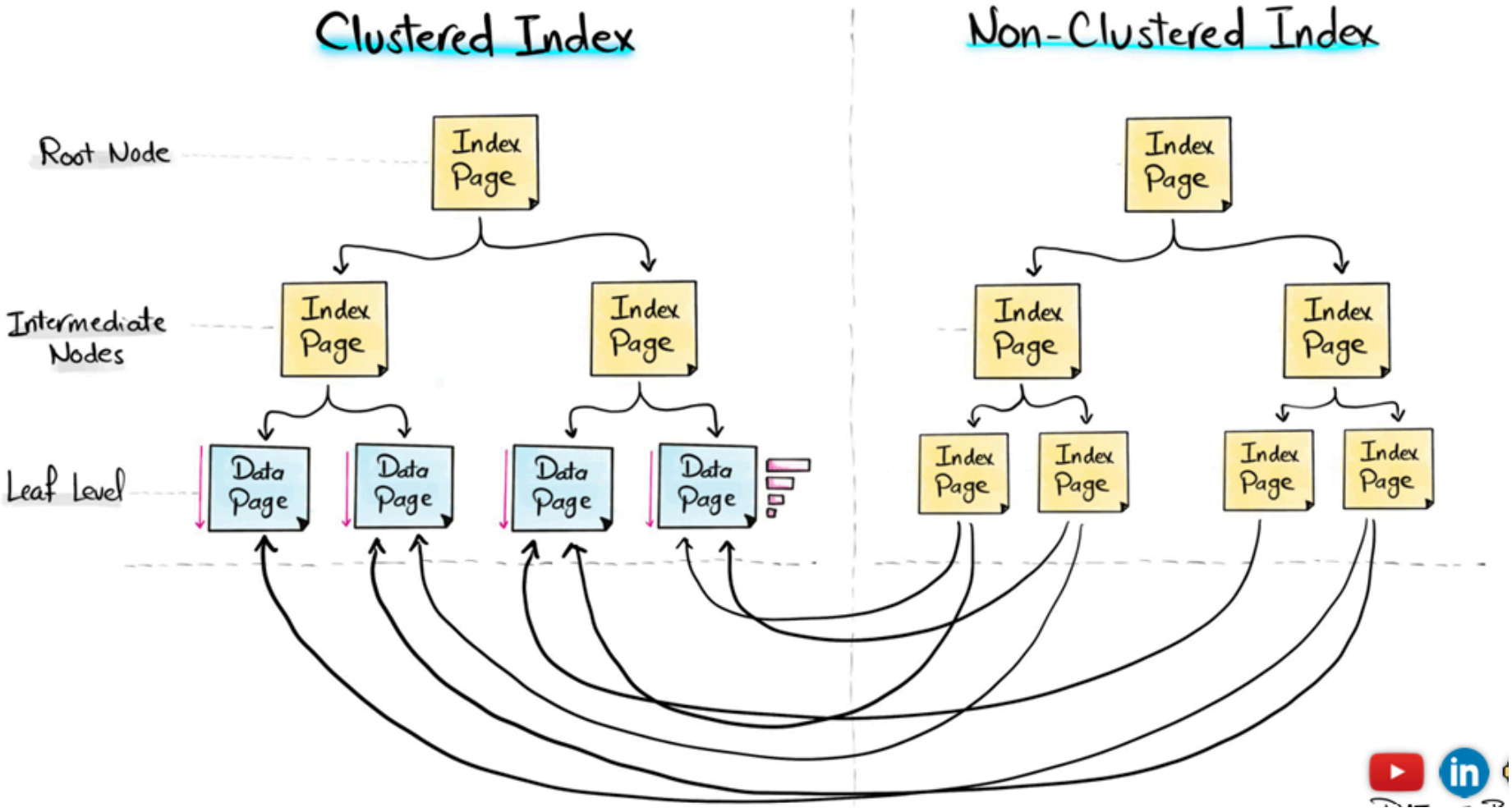






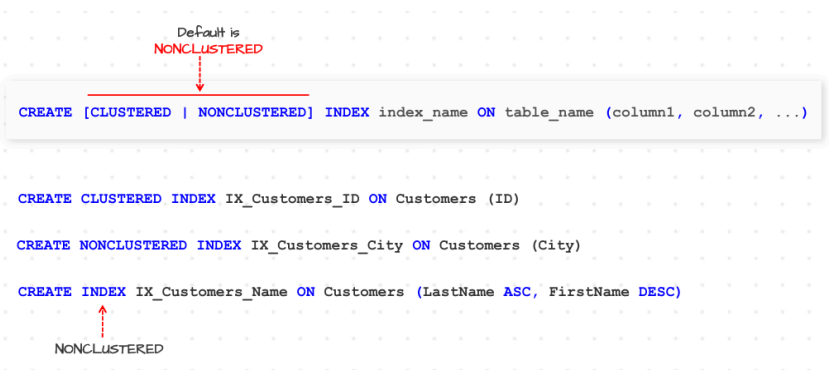
Clustered vs Non-Clustered Index



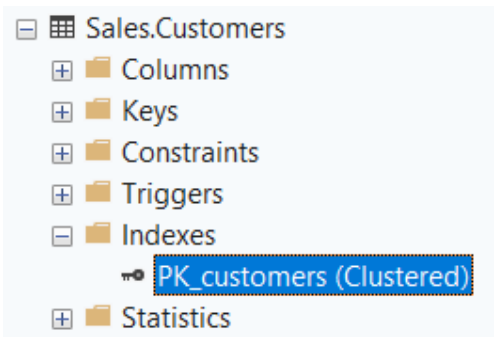


	Clustered Index	Non-Clustered Index
Definition	Physically sorts and stores rows	Separate structure with pointers to the data
Numebr of Indexes	One Index per Table	Multiple indexes are allowed
Read Performance	Faster	Slower
Write Performance	Slower, due to potential data row reordering	Faster, since physical data order is unaffected
Storage Efficiency	More storage-efficient	Requires additional storage space
Use Case	• Unique Column • Not frequently modified Column • Improve range query performance	• Columns frequently used in search conditions and joins • Exact match queries

Clustered & Non-Clustered Index Syntax

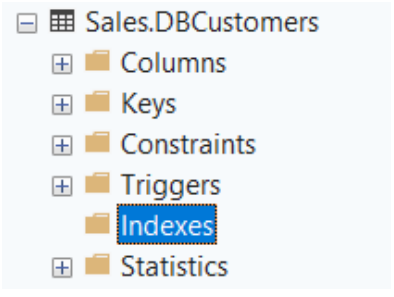


- Composite Index
  - An index with multiple columns
- INFO
  - A primary key (PK) automatically creates a clustered index by default

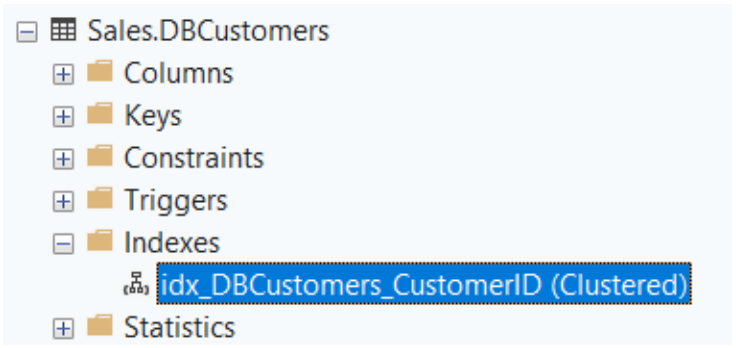


- RULE
  - Only ONE clustered index can be created per table

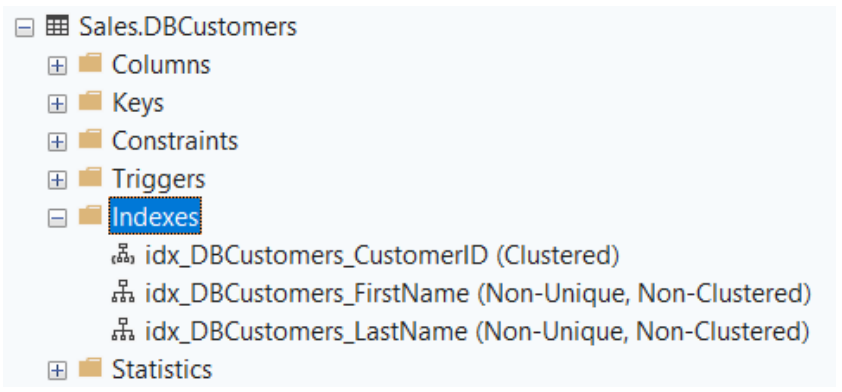
```sql
SELECT *
INTO Sales.DBCustomers
FROM Sales.Customers;
```




```sql
CREATE CLUSTERED INDEX idx_DBCustomers_CustomerID
ON Sales.DBCustomers (CustomerID);
```




```sql
SELECT *
FROM Sales.DBCustomers
WHERE LastName = 'Brown';

CREATE NONCLUSTERED INDEX idx_DBCustomers_LastName
ON Sales.DBCustomers(LastName);

CREATE INDEX idx_DBCustomers_FirstName
ON Sales.DBCustomers (FirstName);
```



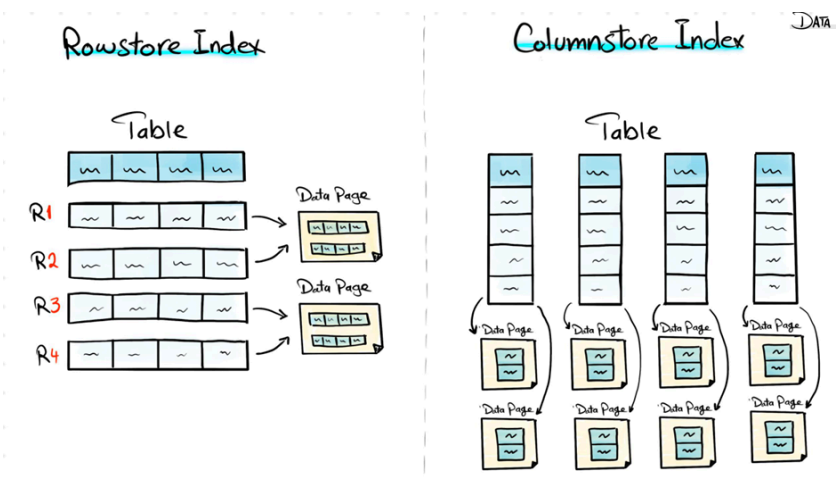
Composite Index

- An index with multiple columns
- RULE
  - The columns of index order must match the order in your query
- Leftmost Prefix Rule
  - Index works only if your query filters starts from the first column in the index and follow its order
- A,B,C,D
  - Index will be used
    - A
    - A,B
    - A,B, C
    - A,B,C,D
  - Index won’t be used
    - B
    - A,C
    - A,B,D

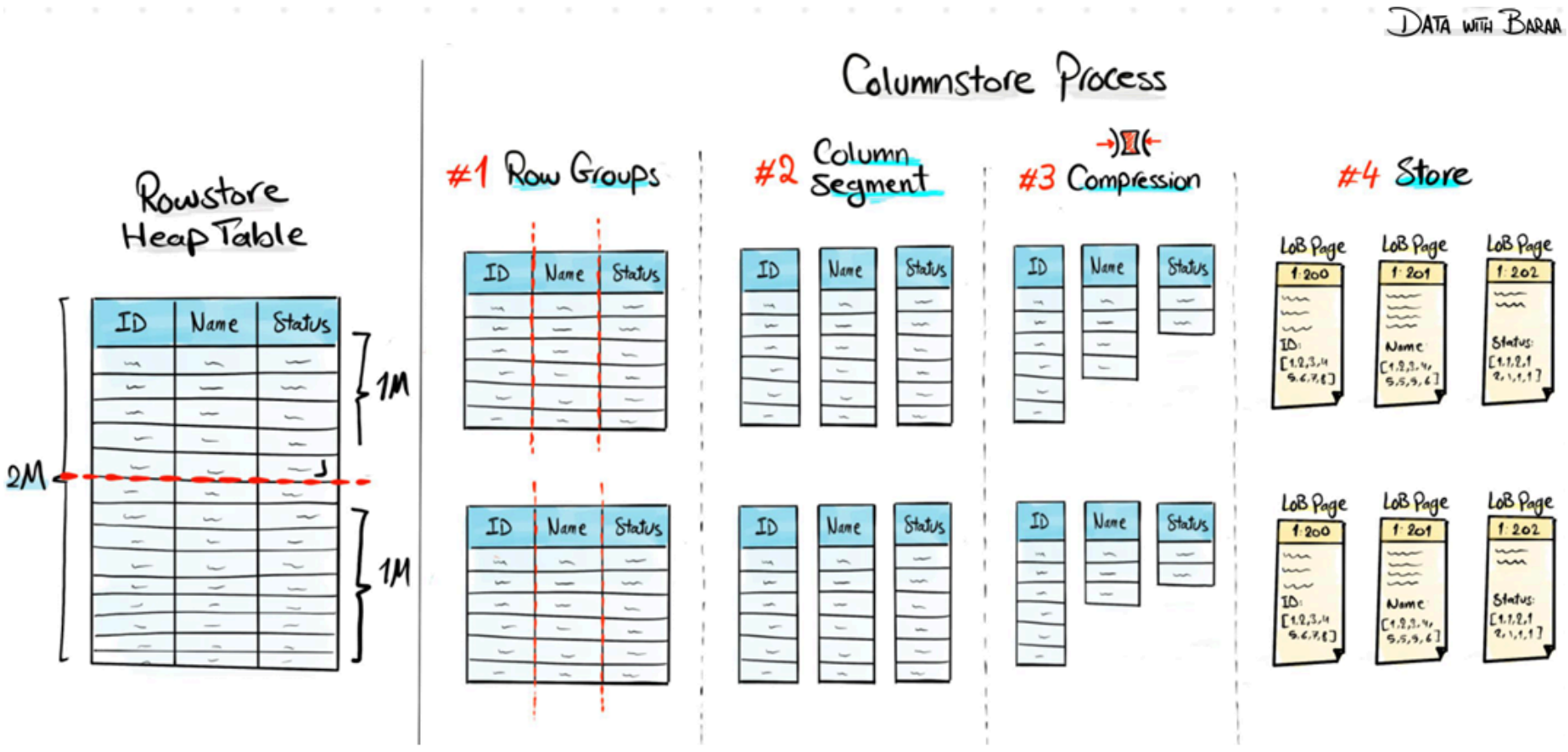
```sql
-- Composite Index
SELECT *
FROM Sales.DBCustomers
WHERE Country = 'USA' AND Score > 500;

CREATE INDEX idx_DBCustomers_CountryScore
```

```sql
ON Sales.DBCustomers (Country, Score);

-- Leftmost Prefix Rule
-- Will Use Index
SELECT *
FROM Sales.DBCustomers
WHERE Country = 'USA';
-- Won't Use Index
SELECT *
FROM Sales.DBCustomers
WHERE Score > 500;
```

Columnstore & Rowstore Index



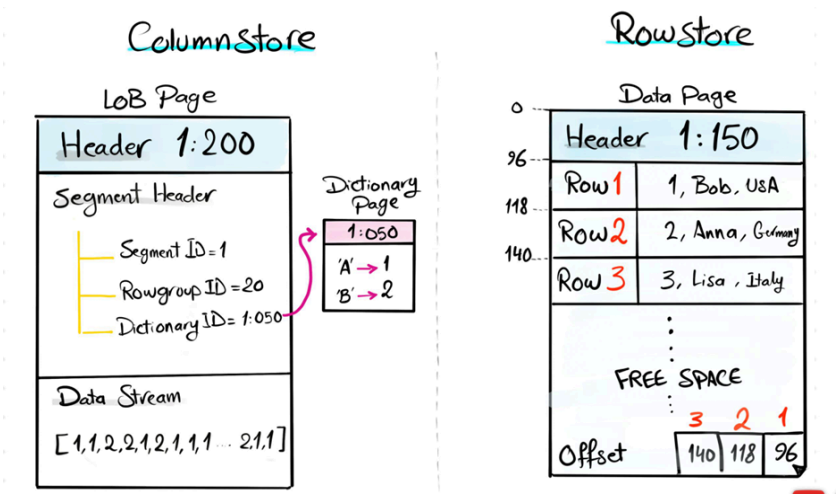
How SQL Builds Columnstore (The Process)



1. **Row Groups**
    a. The database engine first horizontally divides the massive table into smaller, manageable chunks called Row Groups
2. Column Segment
    a. Within each Row Group, the data is vertically partitioned. Instead of keeping entire rows intact, each individual column (ID, Name, Status) is separated into its own distinct unit known as a Column Segment
3. Compression

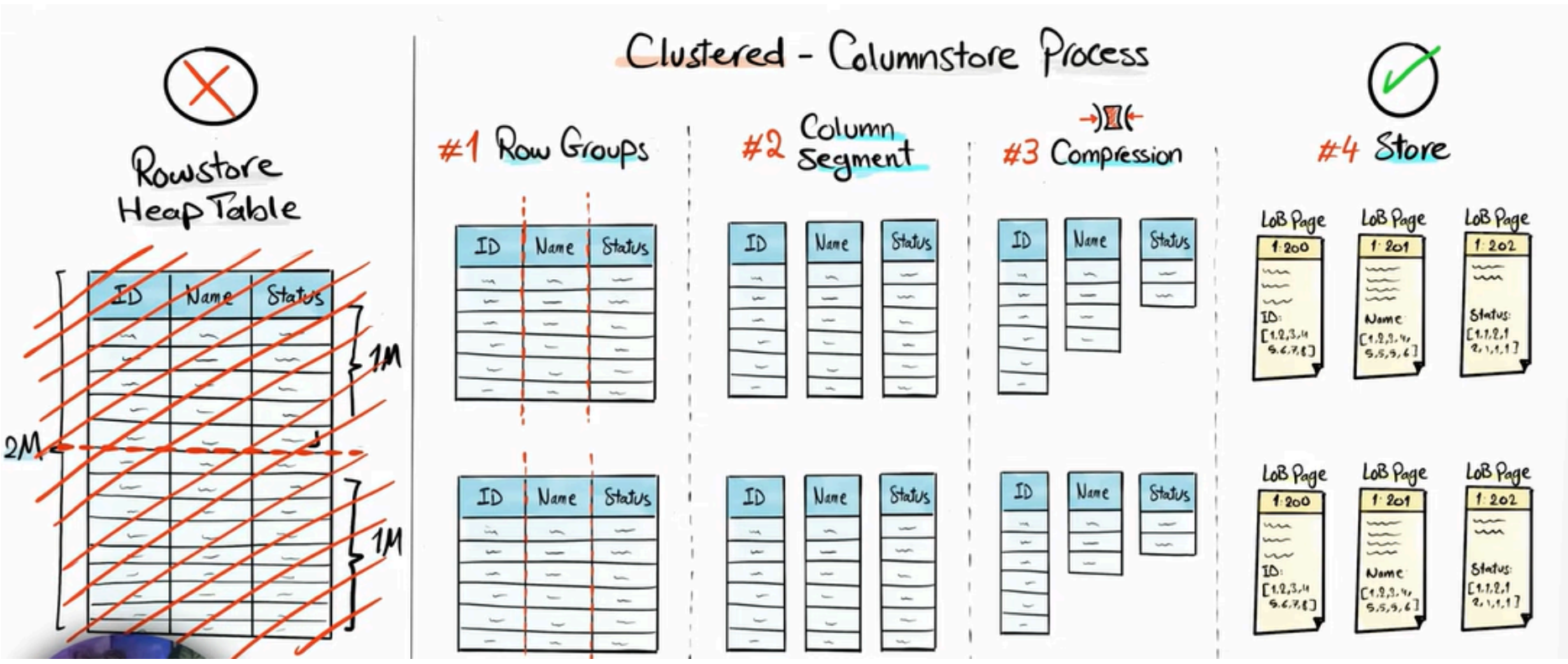
a. Each resulting Column Segment is compressed independently. The diagram visually represents this by shrinking the segments, demonstrating how grouping identical data types together - especially repetitive data like a "Status" column - allows for highly efficient size reduction

4. Store
    a. The highly compressed column segments are finally written to physical storage as Large Object (LoB) Pages. Because each column is stored in its own dedicated page (e.g., ID in page 1:200, Name in 1:201), the database engine can execute queries by reading only the specific columns requested, bypassing irrelevant data entirely

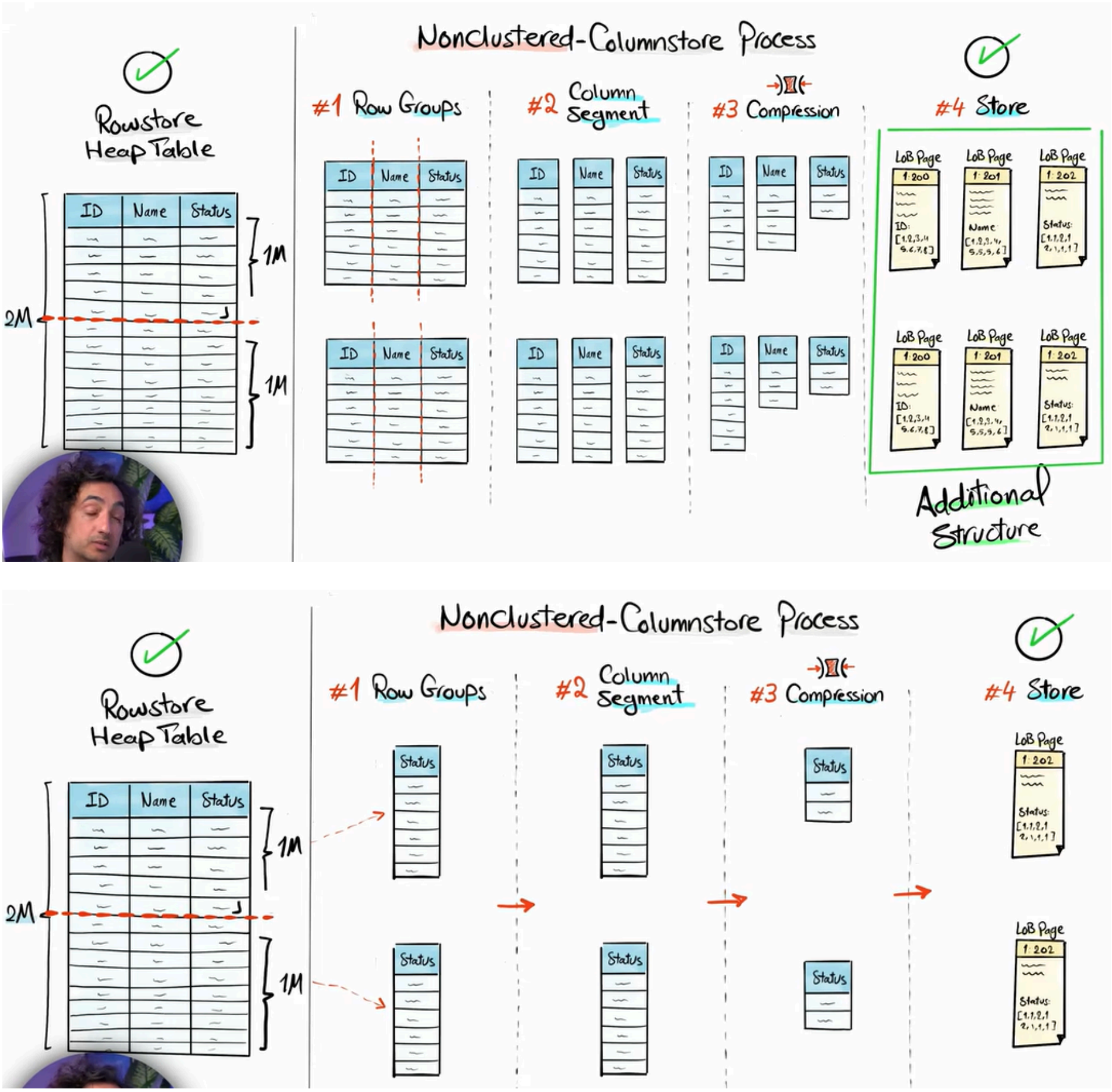


Clustered Columnstore Index vs Non-Clustered Columnstore Index

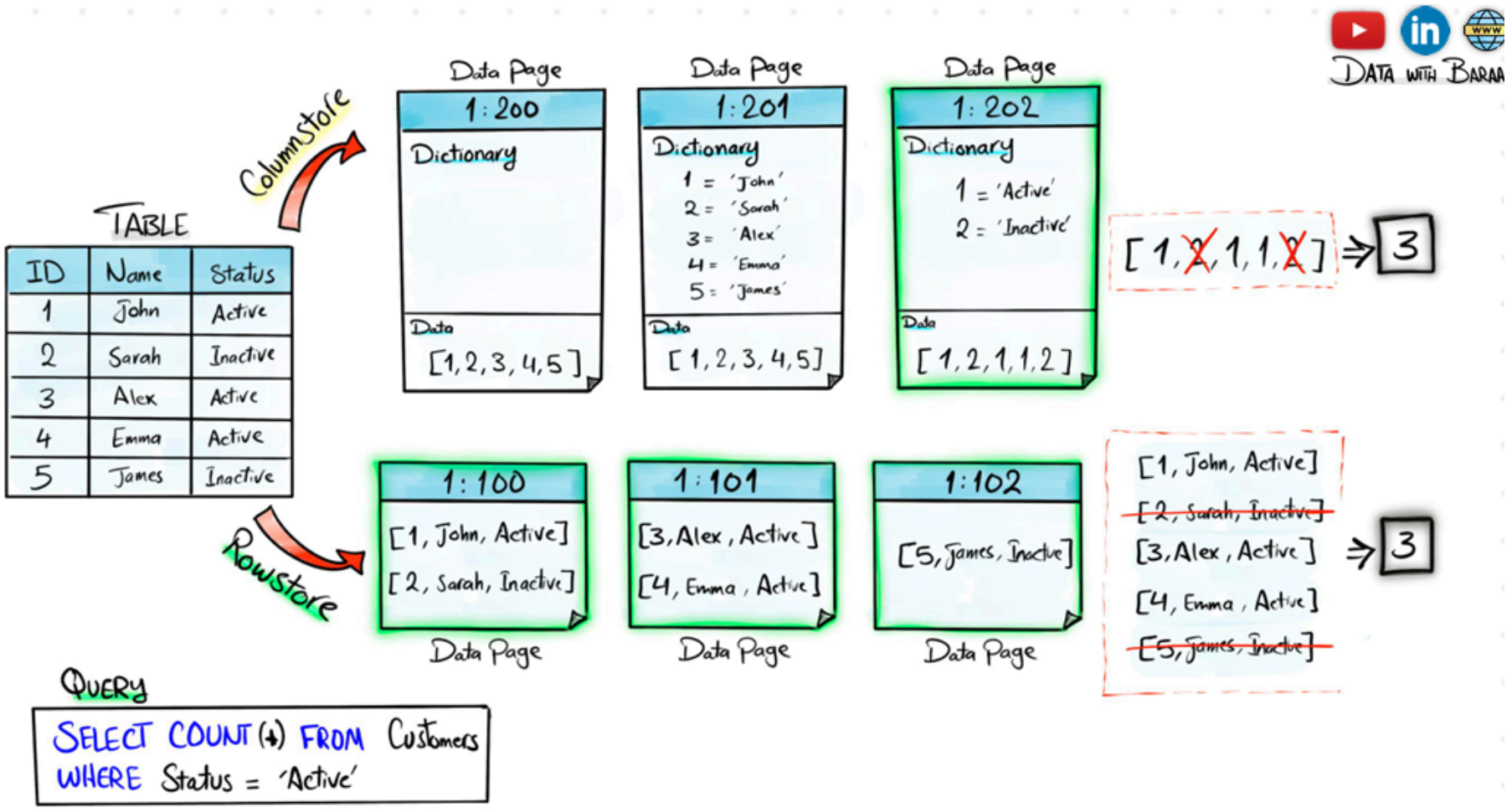
- Clustered Columnstore Index (CCI)
    - Physically replaces the entire table to store all data in a highly compressed columnar format, making it the primary storage method for massive data warehouse analytics (OLAP)



- Non-Clustered Columnstore Index (NCCI)
    - A secondary index built on top of a traditional row-based table, allowing you to run fast analytical queries directly on an active transactional database (OLTP) without disrupting normal day-to-day operations



Columnstore vs Rowstore Index



	Rowstore Index	Columnstore Index
Definition	Organizes and stores data **row by row**	Organizes and stores data **column by column**
Storage Efficiency	**Less efficient in storage**	**Highly efficient with Compression**
Read/Write Optimization	**Fair speed for read & write operations**	**Fast read performance** **Slow write performance**
I/O Efficiency	**Lower (retrieves all columns)**	**Higher (retrieves specific columns)**
Best for ..	**OLTP (Transactional)** commerce, banking, Financial systems, order processing	**OLAP (Analytical)** Data Warehouse, Business intelligence, Reporting, Analytics
Use Case	**- High-frequency transaction applications** **- Quick access to complete records**	**- Big Data Analytics** **- Scanning of large datasets** **- Fast aggregation**

Columnstore Index Syntax

Columnstore Index
Syntax

Default is ROWSTORE

```
CREATE [CLUSTERED | NONCLUSTERED] [COLUMNSTORE] INDEX index_name
ON table_name (column1, column2, ...)
```

Rowstore

```
CREATE NONCLUSTERED INDEX IX_Customers_Country ON Customers (Country)

CREATE CLUSTERED INDEX IX_Customers_ID ON Customers (ID)
```

Columnstore

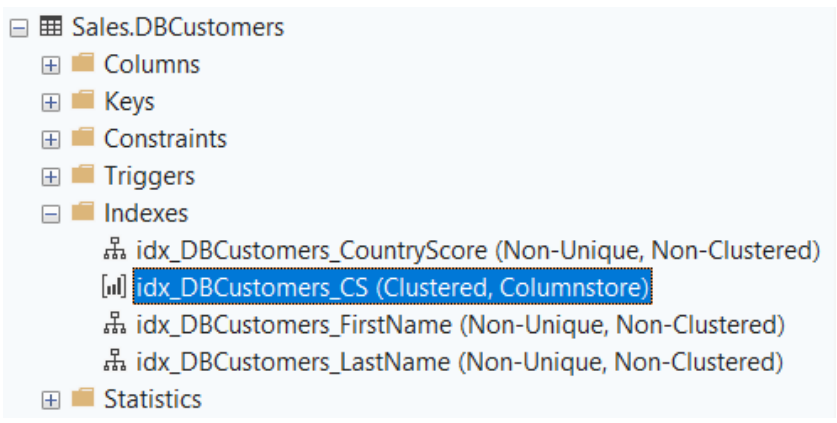
```
CREATE NONCLUSTERED COLUMNSTORE INDEX IX_Customers_Country ON Customers (Country)

CREATE CLUSTERED COLUMNSTORE INDEX IX_Customers ON Customers
```

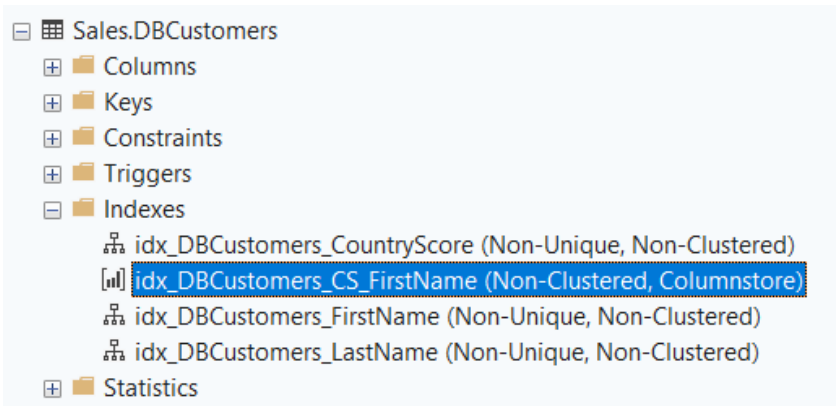
✖ NOT ALLOWED TO USE COLUMNS

Rules

- You can't specific columns in Clustered Index Columnstore

```
-- Clustered Columnstore Index
CREATE CLUSTERED COLUMNSTORE INDEX idx_DBCustomers_CS
ON Sales.DBCustomers;

-- Non-Clustered Columnstore Index
CREATE NONCLUSTERED COLUMNSTORE INDEX idx_DBCustomers_CS_FirstName
ON Sales.DBCustomers (FirstName);
```

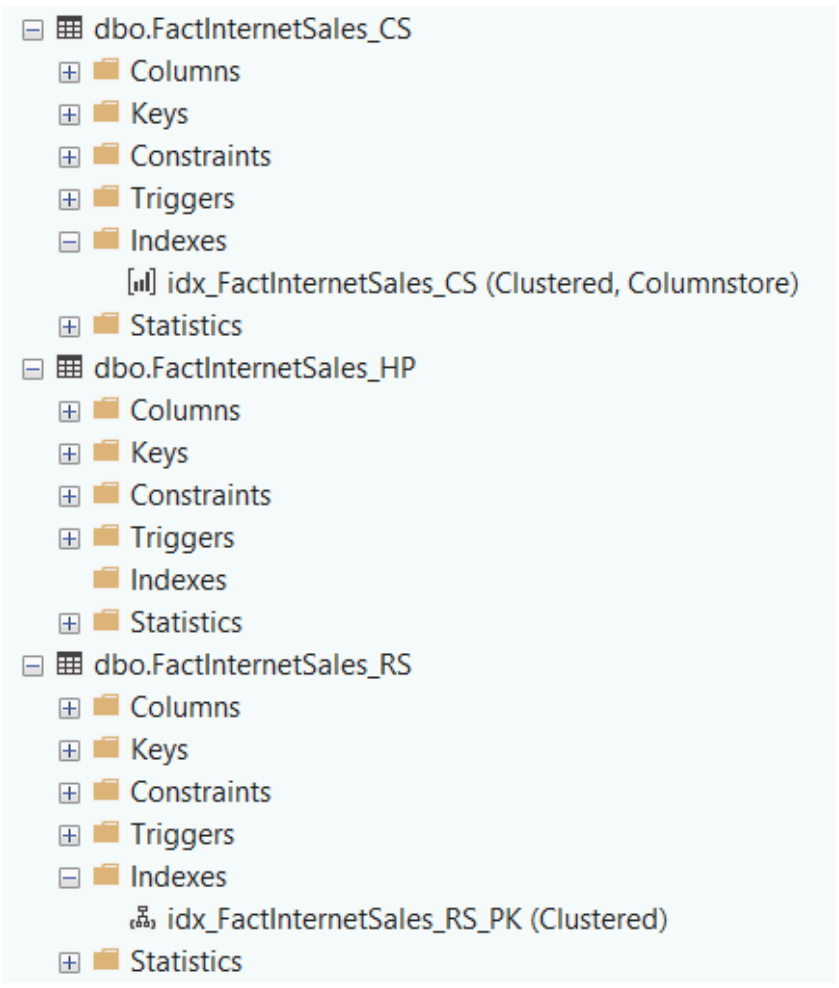




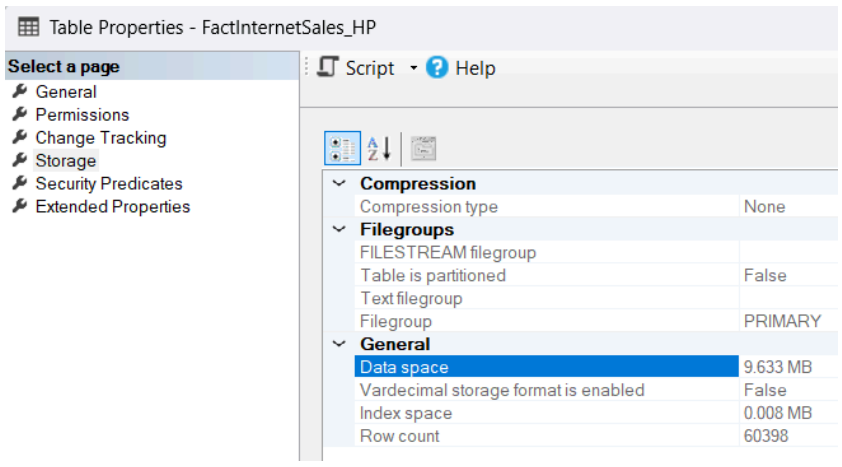
Comparison (HEAP, COLUMNSTORE & ROWSTORE)

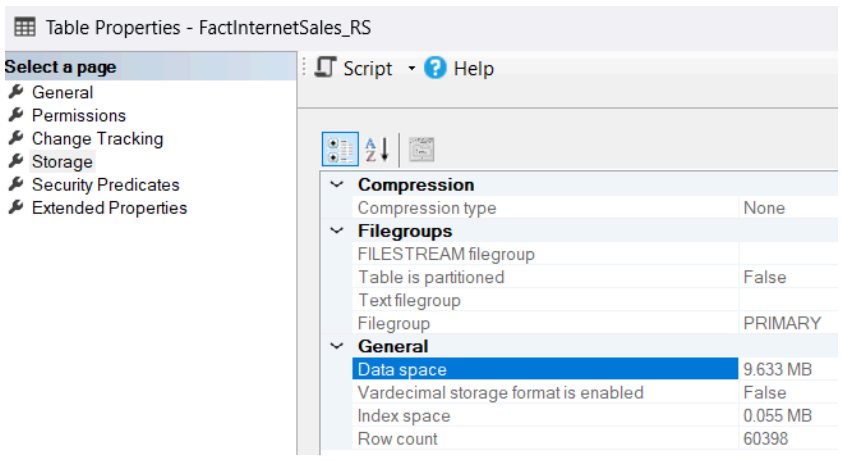
```
-- Comparison
USE AdventureWorksDW2022;

-- HEAP
```

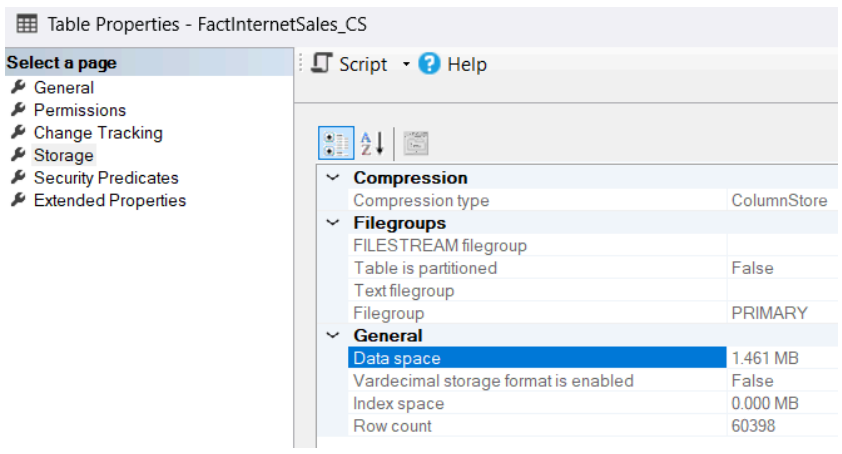
```sql
SELECT *
INTO FactInternetSales_HP
FROM FactInternetSales;

-- ROWSTORE
SELECT *
INTO FactInternetSales_RS
FROM FactInternetSales;

CREATE CLUSTERED INDEX idx_FactInternetSales_RS_PK
ON FactInternetSales_RS (SalesOrderNumber, SalesOrderLineNumber);

-- COLUMNSTORE
SELECT *
INTO FactInternetSales_CS
FROM FactInternetSales;

CREATE CLUSTERED COLUMNSTORE INDEX idx_FactInternetSales_CS
ON FactInternetSales_CS;
```



- Storage
  - Heap → 9.641 MB
  - RowStore → 9.688 MB
  - ColumnStore → 1.461 MB
- Storage Efficiency
  1. ColumnStore Index
  2. Heap Table
  3. RowStore Clustered Index

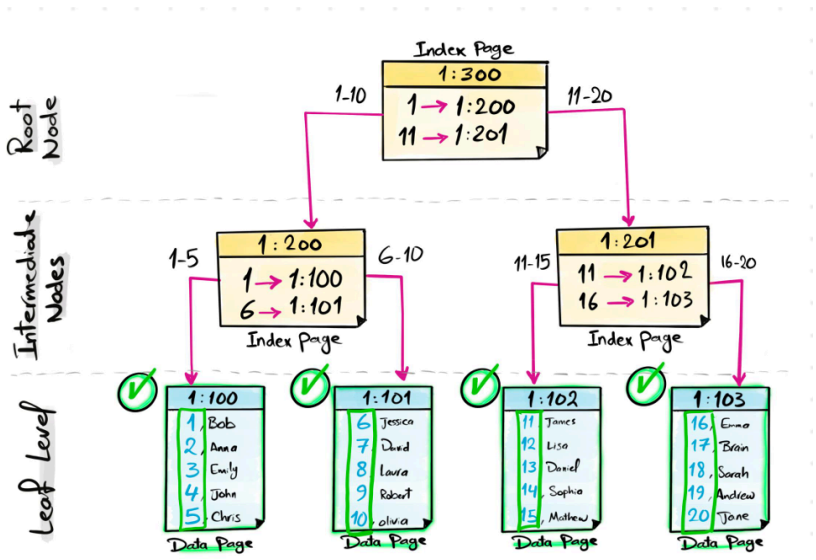






Unique Index

- Ensures no duplicate values exist in specific column
- Benefits
  - Enforce uniqueness
  - Slightly increase query performance
- Performance
  - Writing to an unique index is slower than non-unique
  - Reading from an unique index is faster than non-unique



Unique Index Syntax

Default is NOT Unique

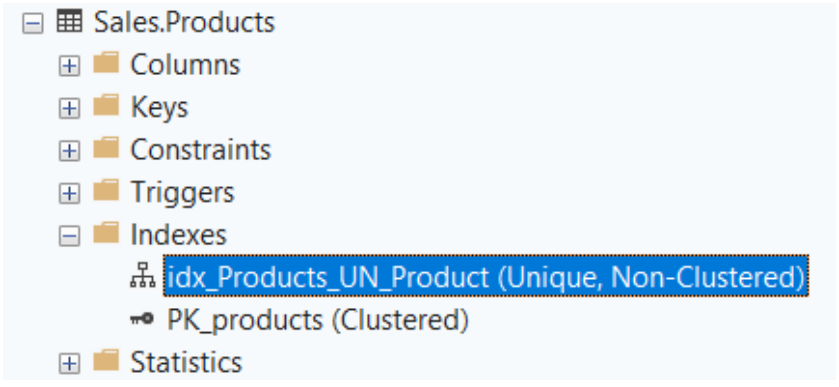
```
CREATE [UNIQUE] [CLUSTERED | NONCLUSTERED] [COLUMNSTORE] INDEX index_name
ON table_name (column1, column2, ...)
```

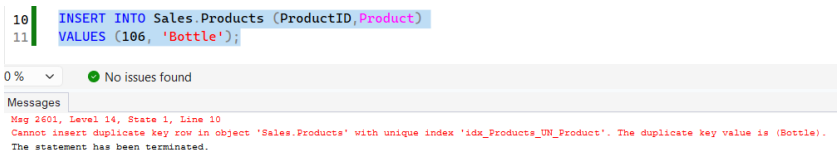
Index Allows Duplicates ------> `CREATE INDEX IX_Customers_Email ON Customers (Email)`

Duplicates are not allowed ------> `CREATE UNIQUE INDEX IX_Customers_Email ON Customers (Email)`

- RULE
  - Duplicates in the columns will prevent creating a unique index

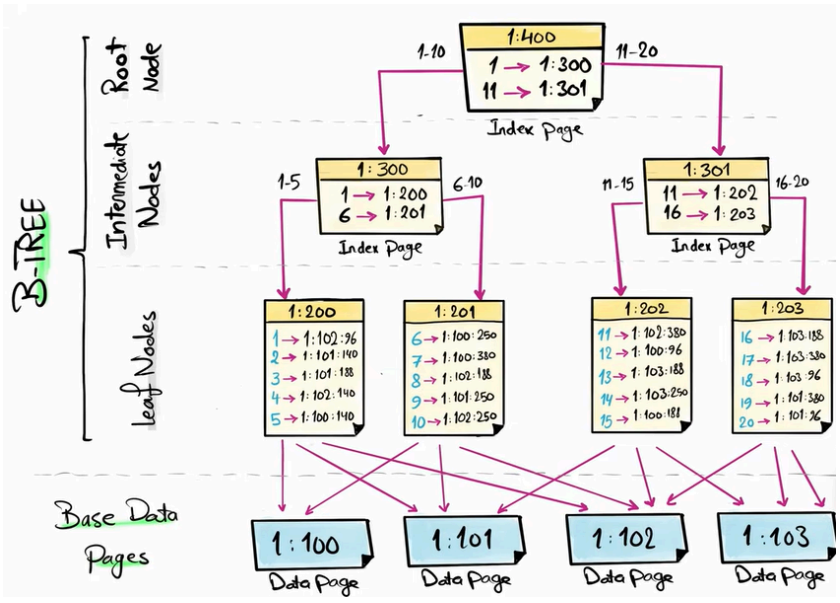
```
CREATE UNIQUE NONCLUSTERED INDEX idx_Products_UN_Product
ON Sales.Products (Product);
```

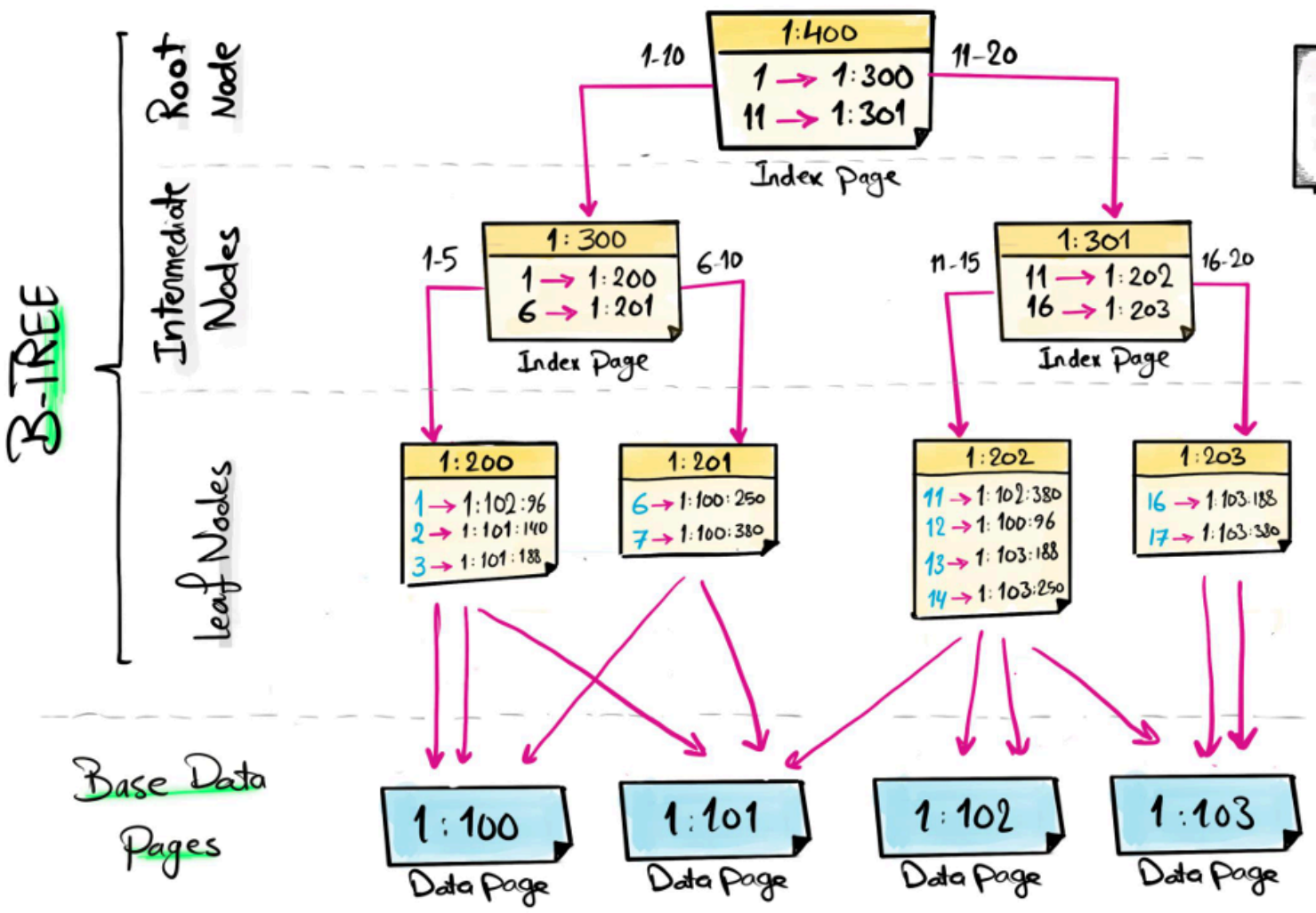




Filtered Index

- An index that includes only rows meeting the specified conditions
- Benefits
  - Targeted Optimization
  - Reduce storage: Less data in the index





Filtered Index Syntax

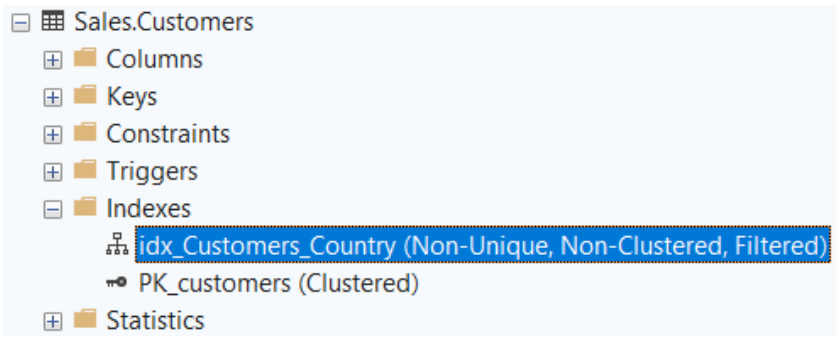
Filtered Index

```
CREATE [UNIQUE] [NONCLUSTERED] INDEX index_name
ON table_name (column1, column2, ...)
WHERE [Condition]
```

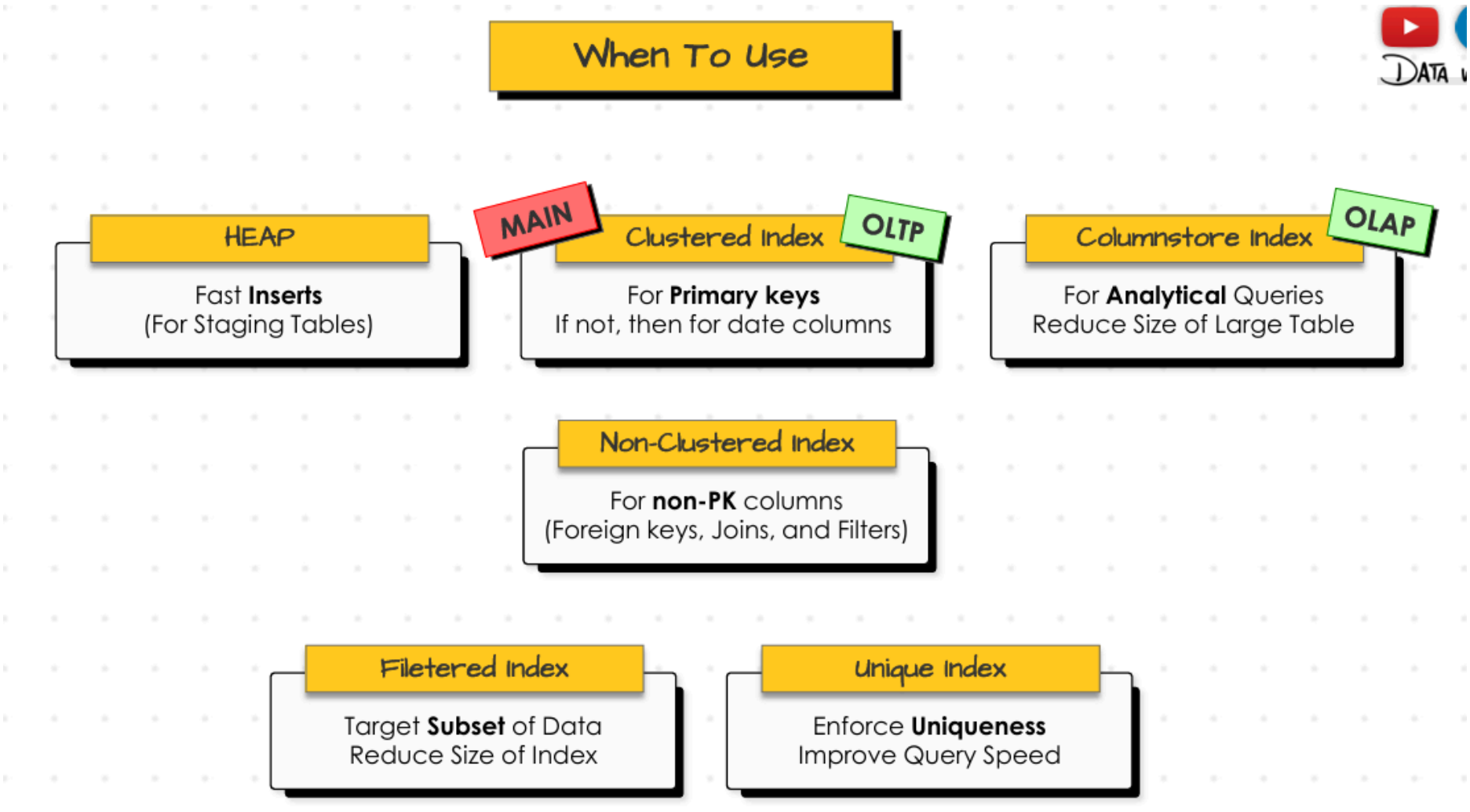
Rules

- You cannot create a filtered index on a clustered index.
- You cannot create a filtered index on a columnstore index.

```sql
CREATE NONCLUSTERED INDEX idx_Customers_Country
ON Sales.Customers (Country)
WHERE Country = 'USA';
```



Choose the Right Index



Index Management & Monitoring

Index Management

1. Monitor Index Usage
2. Monitor Missing Indexes
3. Monitor Duplicate Indexes
4. Update Statistics
5. Monitor Fragmentations

Monitor Index Usage

```sql
-- Quick view of indexes on a specific table
sp_helpindex 'Sales.DBCustomers';
```

	index_name	index_description	index_keys
1	idx_DBCustomers_CountryScore	nonclustered located on PRIMARY	Country, Score
2	idx_DBCustomers_CS_FirstName	nonclustered, columnstore located on PRIMARY	NULL
3	idx_DBCustomers_FirstName	nonclustered located on PRIMARY	FirstName
4	idx_DBCustomers_LastName	nonclustered located on PRIMARY	LastName

- 'Sys' System Schema
  - Contains metadata about database tables, views, indexes ..etc
- Dynamic Management View (DMV)
  - Provides real-time insights into Database performance and system health

```sql
-- Detailed index usage statistics (identifying unused indexes)
SELECT
    tbl.name AS TableName,
    idx.name AS IndexName,
    idx.type_desc AS IndexType,
    idx.is_primary_key AS IsPrimaryKey,
    idx.is_unique AS IsUnique,
    idx.is_disabled AS IsDisabled,
    s.user_seeks AS UserSeeks,
    s.user_scans AS UserScans,
    s.user_lookups AS UserLookups,
    s.user_updates AS UserUpdates,
    COALESCE(s.last_user_seek,s.last_user_scan) AS LastUpdate
FROM sys.indexes idx
JOIN sys.tables tbl
ON idx.object_id = tbl.object_id
LEFT JOIN sys.dm_db_index_usage_stats s
ON idx.object_id = s.object_id AND idx.index_id = s.index_id
ORDER BY tbl.name, idx.name;
```

	TableName	IndexName	IndexType	IsPrimaryKey	IsUnique	IsDisabled	UserSeeks	UserScans	UserLookups	UserUpdates	LastUpdate
1	Customers	idx_Customers_Country	NONCLUSTERED	0	0	0	NULL	NULL	NULL	NULL	NULL
2	Customers	PK_customers	CLUSTERED	1	1	0	NULL	NULL	NULL	NULL	NULL
3	DBCustomers	NULL	HEAP	0	0	0	NULL	NULL	NULL	NULL	NULL
4	DBCustomers	idx_DBCustomers_CountryScore	NONCLUSTERED	0	0	0	NULL	NULL	NULL	NULL	NULL
5	DBCustomers	idx_DBCustomers_CS_FirstName	NONCLUSTERED COLUMNSTORE	0	0	0	NULL	NULL	NULL	NULL	NULL
6	DBCustomers	idx_DBCustomers_FirstName	NONCLUSTERED	0	0	0	NULL	NULL	NULL	NULL	NULL
7	DBCustomers	idx_DBCustomers_LastName	NONCLUSTERED	0	0	0	NULL	NULL	NULL	NULL	NULL
8	Employees	PK_employees	CLUSTERED	1	1	0	NULL	NULL	NULL	NULL	NULL
9	EmployeesLogs	PK__Employee__5E5499A8B8B72B8E	CLUSTERED	1	1	0	NULL	NULL	NULL	NULL	NULL
10	MonthlyOrders	NULL	HEAP	0	0	0	NULL	NULL	NULL	NULL	NULL
11	Orders	PK_orders	CLUSTERED	1	1	0	0	3	0	0	2026-10-27 04:41:38.960
12	OrdersArchive	NULL	HEAP	0	0	0	NULL	NULL	NULL	NULL	NULL
13	OrdersTemp	NULL	HEAP	0	0	0	NULL	NULL	NULL	NULL	NULL
14	Products	idx_Products_UN_Product	NONCLUSTERED	0	1	0	0	1	0	0	2026-10-27 18:24:10.527
15	Products	PK_products	CLUSTERED	1	1	0	NULL	NULL	NULL	NULL	NULL

Monitor Missing Indexes

```sql
USE AdventureWorksDW2022;
SELECT
    fs.SalesOrderNumber,
    dp.EnglishProductName,
    dp.Color
FROM FactInternetSales fs
INNER JOIN DimProduct dp
ON fs.ProductKey = dp.ProductKey
WHERE dp.Color = 'Black'
AND fs.OrderDateKey BETWEEN 20101229 AND 20101231;

-- Check database recommendations for missing indexes
SELECT * FROM sys.dm_db_missing_index_details;
```

Results Messages

	index_handle	database_id	object_id	equality_columns	inequality_columns	included_columns	statement
1	1	7	1317579732	NULL	[OrderDateKey]	[ProductKey]	[AdventureWorksDW2022].[dbo].[FactInternetSales]

- TIP

- Evaluate the recommendations before creating any index

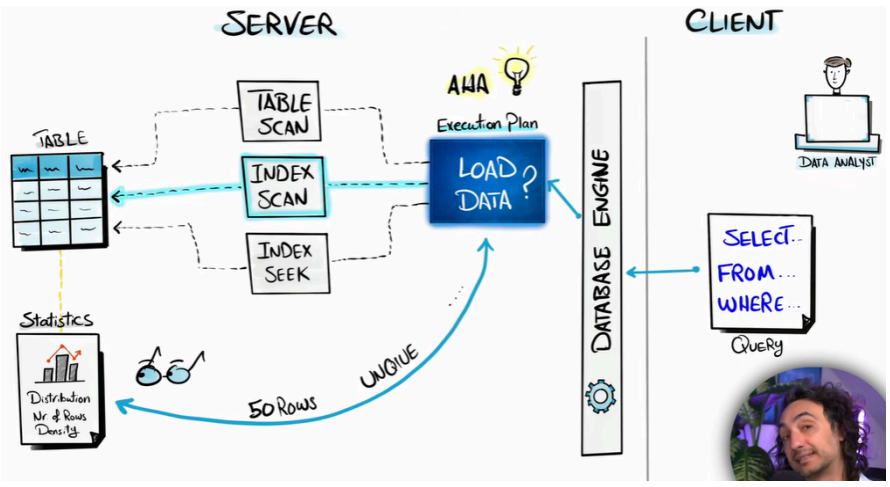
Monitor Duplicate Indexes

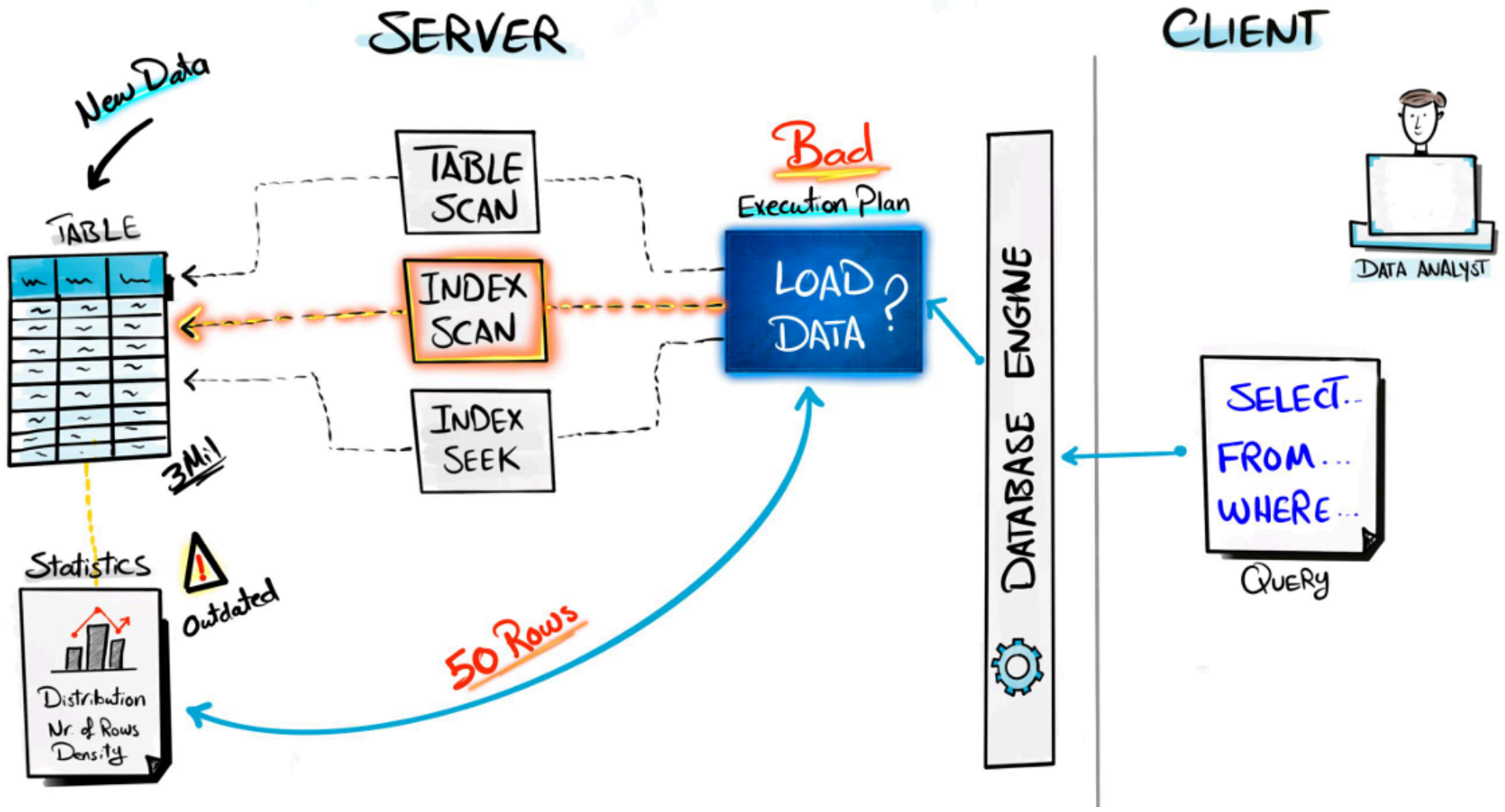
```sql
-- Identify columns involved in multiple indexes
SELECT
    t.name AS TableName,
    c.name AS ColumnName,
    i.name AS IndexName,
    i.type_desc AS IndexType,
    COUNT(*) OVER(PARTITION BY t.name, c.name) AS DuplicateFlag
FROM sys.indexes i
JOIN sys.tables t ON i.object_id = t.object_id
JOIN sys.index_columns ic ON i.object_id = ic.object_id AND i.index_id = ic.index_id
JOIN sys.columns c ON ic.object_id = c.object_id AND ic.column_id = c.column_id
ORDER BY t.name, c.name;
```

	TableName	ColumnName	IndexName	IndexType	DuplicateFlag
1	Customers	Country	idx_Customers_Country	NONCLUSTERED	1
2	Customers	CustomerID	PK_customers	CLUSTERED	1
3	DBCustomers	Country	idx_DBCustomers_CountryScore	NONCLUSTERED	1
4	DBCustomers	FirstName	idx_DBCustomers_FirstName	NONCLUSTERED	2
5	DBCustomers	FirstName	idx_DBCustomers_CS_FirstName	NONCLUSTERED COLUMNSTORE	2
6	DBCustomers	LastName	idx_DBCustomers_LastName	NONCLUSTERED	1
7	DBCustomers	Score	idx_DBCustomers_CountryScore	NONCLUSTERED	1
8	Employees	EmployeeID	PK_employees	CLUSTERED	1
9	EmployeesLogs	LogID	PK__Employee__5E5499A8B8B72B8E	CLUSTERED	1
10	Orders	OrderID	PK_orders	CLUSTERED	1
11	Products	Product	idx_Products_UN_Product	NONCLUSTERED	1
12	Products	ProductID	PK_products	CLUSTERED	1

Update Statistics

1. Weekly job to update statistics on weekends
2. After migrating data






```sql
-- Monitor statistics to see if they are outdated
SELECT
    SCHEMA_NAME(t.schema_id) AS SchemaName,
    t.name AS TableName,
    s.name AS StatisticsName,
    sp.last_updated AS LastUpdated,
    DATEDIFF(day, sp.last_updated, GETDATE()) AS LastUpdatedDay,
    sp.rows AS TotalRows,
    sp.modification_counter AS ModificationsSinceLastUpdate
FROM sys.stats s
JOIN sys.tables t
ON s.object_id = t.object_id
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
ORDER BY sp.modification_counter DESC;
```

	SchemaName	TableName	StatisticsName	LastUpdated	LastUpdatedDay	TotalRows	ModificationsSinceLastUpdate
1	Sales	DBCustomers	_WA_Sys_00000003_14270015	2026-10-24 14:47:46.1400000	4	5	5
2	Sales	DBCustomers	_WA_Sys_00000005_14270015	2026-10-24 14:52:53.2533333	4	5	5
3	Sales	Customers	_WA_Sys_00000005_48CFD27E	2026-09-12 19:22:35.8533333	46	5	3
4	Sales	Products	PK_products	2026-09-02 17:17:35.4266667	56	5	2
5	Sales	Products	_WA_Sys_00000004_4CA06362	2026-10-08 04:23:43.2500000	20	5	2
6	Sales	Products	idx_Products_UN_Product	2026-10-27 05:04:31.3100000	1	5	2
7	Sales	Employees	PK_employees	2026-09-02 17:17:35.4333333	56	5	1
8	Sales	Employees	_WA_Sys_00000002_4AB81AF0	2026-09-02 19:44:23.6000000	56	5	1
9	Sales	Employees	_WA_Sys_00000003_4AB81AF0	2026-09-02 19:44:23.6066667	56	5	1
10	Sales	Employees	_WA_Sys_00000006_4AB81AF0	2026-10-08 04:45:50.4800000	20	5	1
11	Sales	Employees	_WA_Sys_00000007_4AB81AF0	2026-10-08 04:46:14.0133333	20	5	1
12	Sales	Employees	_WA_Sys_00000008_4AB81AF0	2026-10-14 19:07:20.1800000	14	5	1
13	Sales	DBCustomers	idx_DBCustomers_CountryScore	2026-10-27 04:25:53.3066667	1	5	0
14	Sales	Customers	PK_customers	2026-09-02 17:17:35.4200000	56	5	0
15	Sales	Customers	_WA_Sys_00000002_48CFD27E	2026-09-02 19:44:23.6000000	56	5	0
16	Sales	Customers	_WA_Sys_00000003_48CFD27E	2026-09-02 19:44:23.6033333	56	5	0
17	Sales	DBCustomers	idx_DBCustomers_LastName	2026-10-27 04:25:53.3000000	1	5	0
18	Sales	DBCustomers	idx_DBCustomers_FirstName	2026-10-27 04:25:53.3033333	1	5	0
19	Sales	DBCustomers	_WA_Sys_00000004_14270015	2026-10-28 01:40:46.6666667	0	5	0

```sql
-- Update specific statistics for a table
UPDATE STATISTICS Sales.DBCustomers _WA_Sys_00000003_14270015;
```

Results Messages

	SchemaName	TableName	StatisticsName	LastUpdated	LastUpdatedDay	TotalRows	ModificationsSinceLastUpdate
12	Sales	DBCustomers	idx_DBCustomers_CountryScore	2026-10-27 04:25:53.3066667	1	5	0
13	Sales	Customers	PK_customers	2026-09-02 17:17:35.4200000	56	5	0
14	Sales	Customers	_WA_Sys_00000002_48CFD27E	2026-09-02 19:44:23.6000000	56	5	0
15	Sales	Customers	_WA_Sys_00000003_48CFD27E	2026-09-02 19:44:23.6033333	56	5	0
16	Sales	DBCustomers	_WA_Sys_00000003_14270015	2026-10-28 01:42:14.6900000	0	5	0
17	Sales	DBCustomers	idx_DBCustomers_LastName	2026-10-27 04:25:53.3000000	1	5	0

```sql
-- Update all statistics for a specific table
```

```
UPDATE STATISTICS Sales.DBCustomers;
```

Results Messages

	SchemaName	TableName	StatisticsName	LastUpdated	LastUpdatedDay	TotalRows	ModificationsSinceLastUpdate
14	Sales	Customers	_WA_Sys_00000002_48CFD27E	2026-09-02 19:44:23.6000000	56	5	0
15	Sales	Customers	_WA_Sys_00000003_48CFD27E	2026-09-02 19:44:23.6033333	56	5	0
16	Sales	DBCustomers	_WA_Sys_00000003_14270015	2026-10-28 01:43:05.9966667	0	5	0
17	Sales	DBCustomers	idx_DBCustomers_LastName	2026-10-28 01:43:06.0166667	0	5	0
18	Sales	DBCustomers	idx_DBCustomers_FirstName	2026-10-28 01:43:06.0200000	0	5	0
19	Sales	DBCustomers	_WA_Sys_00000004_14270015	2026-10-28 01:43:06.0233333	0	5	0
20	Sales	DBCustomers	_WA_Sys_00000005_14270015	2026-10-28 01:43:06.0266667	0	5	0
21	Sales	DBCustomers	idx_DBCustomers_CountryScore	2026-10-28 01:43:06.0300000	0	5	0

```
-- Update all statistics for the entire database (use with caution on large DBs)
EXEC sp_updatestats;
```

Messages

```
Updating [Sales].[DBCustomers]
    [_WA_Sys_00000003_14270015], update is not necessary...
    [idx_DBCustomers_LastName], update is not necessary...
    [idx_DBCustomers_FirstName], update is not necessary...
    [_WA_Sys_00000004_14270015], update is not necessary...
    [_WA_Sys_00000005_14270015], update is not necessary...
    [idx_DBCustomers_CountryScore], update is not necessary...
    0 index(es)/statistic(s) have been updated, 6 did not require update.

Updating [sys].[db_ledger_blocks]
    [ci_block_id], update is not necessary...
    0 index(es)/statistic(s) have been updated, 1 did not require update.

Updating [sys].[db_ledger_transactions]
    [ci_block_id], update is not necessary...
    [nci_transaction_id], update is not necessary...
    0 index(es)/statistic(s) have been updated, 2 did not require update.

Updating [sys].[backup_metadata_store]
    [IX_backup_metadata_start_date_metadata_uuid], update is not necessary...
    [IX_backup_metadata_type_finish_date], update is not necessary...
    [IX_backup_metadata_first_lsn], update is not necessary...
    [IX_backup_metadata_last_lsn], update is not necessary...
    [IX_backup_metadata_database_guid], update is not necessary...
    0 index(es)/statistic(s) have been updated, 5 did not require update.

Updating [sys].[plan_persist_plan_feedback]
    [plan_feedback_cidx], update is not necessary...
    [plan_feedback_idx1], update is not necessary...
    0 index(es)/statistic(s) have been updated, 2 did not require update.

Updating [sys].[plan_persist_query_variant]
    [plan_persist_query_variant_cidx], update is not necessary...
    [plan_persist_query_variant_idx1], update is not necessary...
    0 index(es)/statistic(s) have been updated, 2 did not require update.

Updating [sys].[ledger_tables_history_internal]
    [ci_ledger_tables_history_internal], update is not necessary...
    0 index(es)/statistic(s) have been updated, 1 did not require update.
```

Messages

```
Updating [Sales].[Customers]
    [PK_customers], update is not necessary...
    [_WA_Sys_00000002_48CFD27E], update is not necessary...
    [_WA_Sys_00000003_48CFD27E], update is not necessary...
    [_WA_Sys_00000005_48CFD27E] has been updated...
    [_WA_Sys_00000004_48CFD27E], update is not necessary...
    [idx_Customers_Country], update is not necessary...
    1 index(es)/statistic(s) have been updated, 5 did not require update.

Updating [Sales].[Employees]
    [PK_employees] has been updated...
    [_WA_Sys_00000002_4AB81AF0] has been updated...
    [_WA_Sys_00000003_4AB81AF0] has been updated...
    [_WA_Sys_00000006_4AB81AF0] has been updated...
    [_WA_Sys_00000007_4AB81AF0] has been updated...
    [_WA_Sys_00000008_4AB81AF0] has been updated...
    6 index(es)/statistic(s) have been updated, 0 did not require update.

Updating [Sales].[Products]
    [PK_products] has been updated...
    [_WA_Sys_00000004_4CA06362] has been updated...
    [idx_Products_UN_Product] has been updated...
    3 index(es)/statistic(s) have been updated, 0 did not require update.

Updating [Sales].[Orders]
    [PK_orders] has been updated...
    [_WA_Sys_00000003_4E88ABD4], update is not necessary...
    [_WA_Sys_00000002_4E88ABD4], update is not necessary...
    [_WA_Sys_00000004_4E88ABD4], update is not necessary...
    [_WA_Sys_0000000C_4E88ABD4], update is not necessary...
    [_WA_Sys_00000005_4E88ABD4], update is not necessary...
    [_WA_Sys_0000000B_4E88ABD4], update is not necessary...
    [_WA_Sys_00000007_4E88ABD4], update is not necessary...
    1 index(es)/statistic(s) have been updated, 7 did not require update.

Updating [Sales].[OrdersArchive]
    [_WA_Sys_00000001_5070F446], update is not necessary...
    [_WA_Sys_00000002_5070F446], update is not necessary...
    [_WA_Sys_00000003_5070F446], update is not necessary...
    [_WA_Sys_00000004_5070F446], update is not necessary...
    [_WA_Sys_00000005_5070F446], update is not necessary...
```

Messages

```
Updating [Sales].[MonthlyOrders]
    0 index(es)/statistic(s) have been updated, 0 did not require update.

Updating [Sales].[OrdersTemp]
    0 index(es)/statistic(s) have been updated, 0 did not require update.

Updating [sys].[queue_messages_1977058079]
    [queue_clustered_index], update is not necessary...
    [queue_secondary_index], update is not necessary...
    0 index(es)/statistic(s) have been updated, 2 did not require update.

Updating [sys].[queue_messages_2009058193]
    [queue_clustered_index], update is not necessary...
    [queue_secondary_index], update is not necessary...
    0 index(es)/statistic(s) have been updated, 2 did not require update.

Updating [sys].[queue_messages_2041058307]
    [queue_clustered_index], update is not necessary...
    [queue_secondary_index], update is not necessary...
    0 index(es)/statistic(s) have been updated, 2 did not require update.

Updating [sys].[filestream_tombstone_2073058421]
    [FSTSClusIdx], update is not necessary...
    [FSTSNCIdx], update is not necessary...
    0 index(es)/statistic(s) have been updated, 2 did not require update.

Updating [sys].[syscommittab]
    [ci_commit_ts], update is not necessary...
    [si_xdes_id], update is not necessary...
    [si_commit_time_ts], update is not necessary...
    0 index(es)/statistic(s) have been updated, 3 did not require update.

Updating [sys].[filetable_updates_2105058535]
    [FFtUpdateIdx], update is not necessary...
    0 index(es)/statistic(s) have been updated, 1 did not require update.

Statistics for all tables have been updated.

Completion time: 2026-10-28T01:45:27.8423336+03:00
```

	SchemaName	TableName	StatisticsName	LastUpdated	LastUpdatedDay	TotalRows	ModificationsSinceLastUpdate
1	Sales	Customers	PK_customers	2026-09-02 17:17:35.4200000	56	5	0
2	Sales	Customers	_WA_Sys_00000002_48CFD27E	2026-09-02 19:44:23.6000000	56	5	0
3	Sales	Customers	_WA_Sys_00000003_48CFD27E	2026-09-02 19:44:23.6033333	56	5	0
4	Sales	Customers	_WA_Sys_00000005_48CFD27E	2026-10-28 01:45:27.7433333	0	5	0
5	Sales	Customers	_WA_Sys_00000004_48CFD27E	2026-10-08 04:34:30.6466667	20	5	0
6	Sales	Customers	idx_Customers_Country	2026-10-27 05:16:36.0633333	1	3	0
7	Sales	Employees	PK_employees	2026-10-28 01:45:27.7600000	0	6	0
8	Sales	Employees	_WA_Sys_00000002_4AB81AF0	2026-10-28 01:45:27.7600000	0	6	0

Monitor Fragmentations

- Fragmentation
  - Unused spaces in data pages
  - Data pages are out of order
- Fragmentation Methods
  - Reorganize
    - Defragments leaf nodes to keep them sorted
    - “Light” Operation
  - Rebuild
    - Recreates Index from Scratch
    - “Heavy” Operation
- avg_fragmentation_in_percent
  - Indicate how out_of_order pages are within the index
  - 0% → means no fragmentation (perfect)
  - 100% → means index is completely fragmented (out of order)

```sql
-- Check index fragmentation levels
SELECT
    t.name AS TableName,
    i.name AS IndexName,
    ips.avg_fragmentation_in_percent AS AvgFragmentation,
    ips.page_count AS PageCount
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
INNER JOIN sys.tables t
ON ips.object_id = t.object_id
```

```
INNER JOIN sys.indexes i
ON ips.object_id = i.object_id AND ips.index_id = i.index_id
ORDER BY ips.avg_fragmentation_in_percent DESC;
```

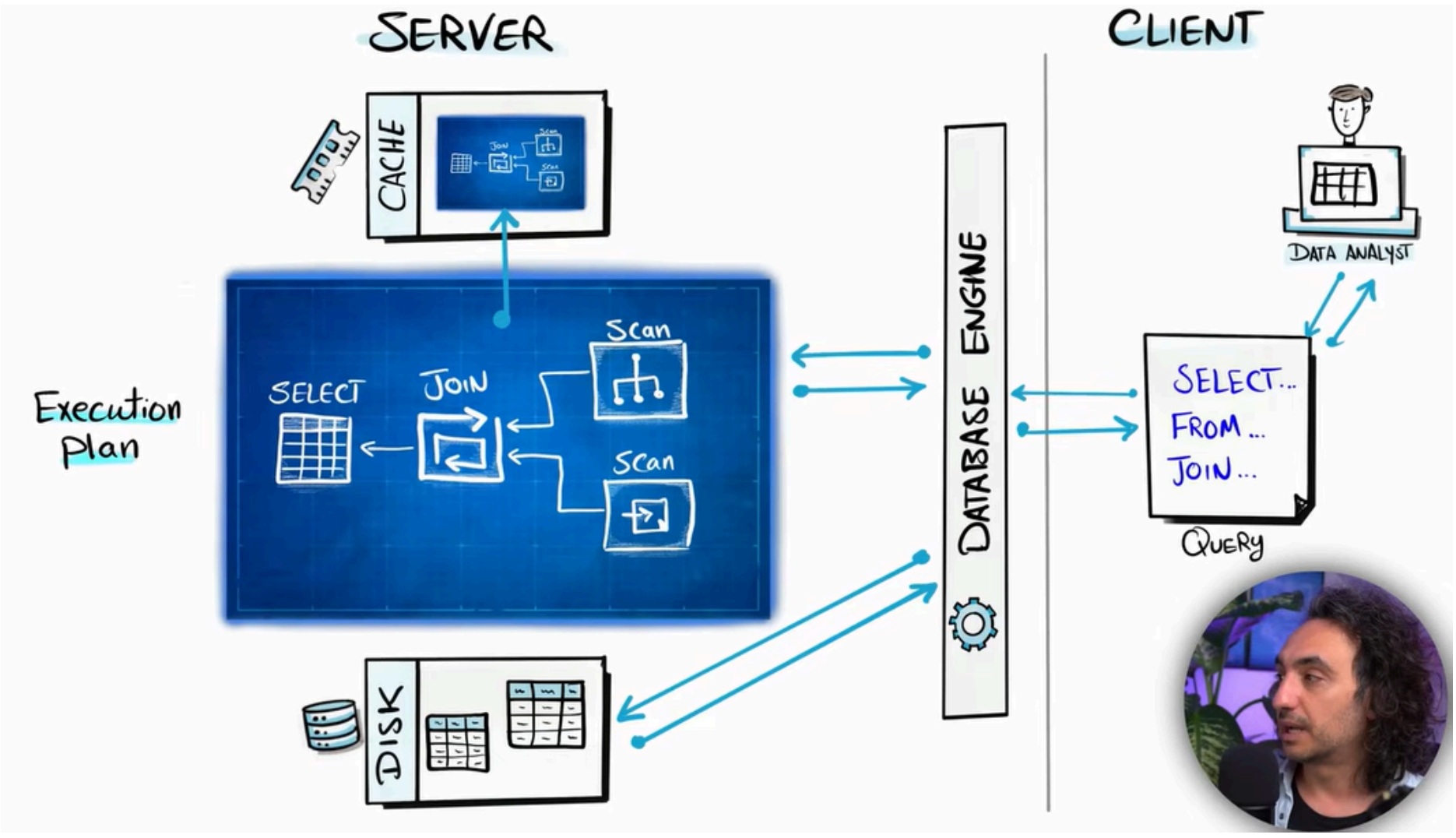
	TableName	IndexName	AvgFragmentation	PageCount
1	EmployeesLogs	PK__Employee__5E5499A8B8B72B8E	0	1
2	DBCustomers	NULL	0	1
3	DBCustomers	idx_DBCustomers_LastName	0	1
4	DBCustomers	idx_DBCustomers_FirstName	0	1
5	DBCustomers	idx_DBCustomers_CountryScore	0	1
6	DBCustomers	idx_DBCustomers_CS_FirstName	0	0
7	DBCustomers	idx_DBCustomers_CS_FirstName	0	0
8	DBCustomers	idx_DBCustomers_CS_FirstName	0	0
9	Customers	PK_customers	0	1
10	Customers	idx_Customers_Country	0	1
11	Employees	PK_employees	0	1
12	Products	PK_products	0	1
13	Products	idx_Products_UN_Product	0	1
14	Orders	PK_orders	0	1
15	OrdersArchive	NULL	0	1
16	MonthlyOrders	NULL	0	1
17	OrdersTemp	NULL	0	1

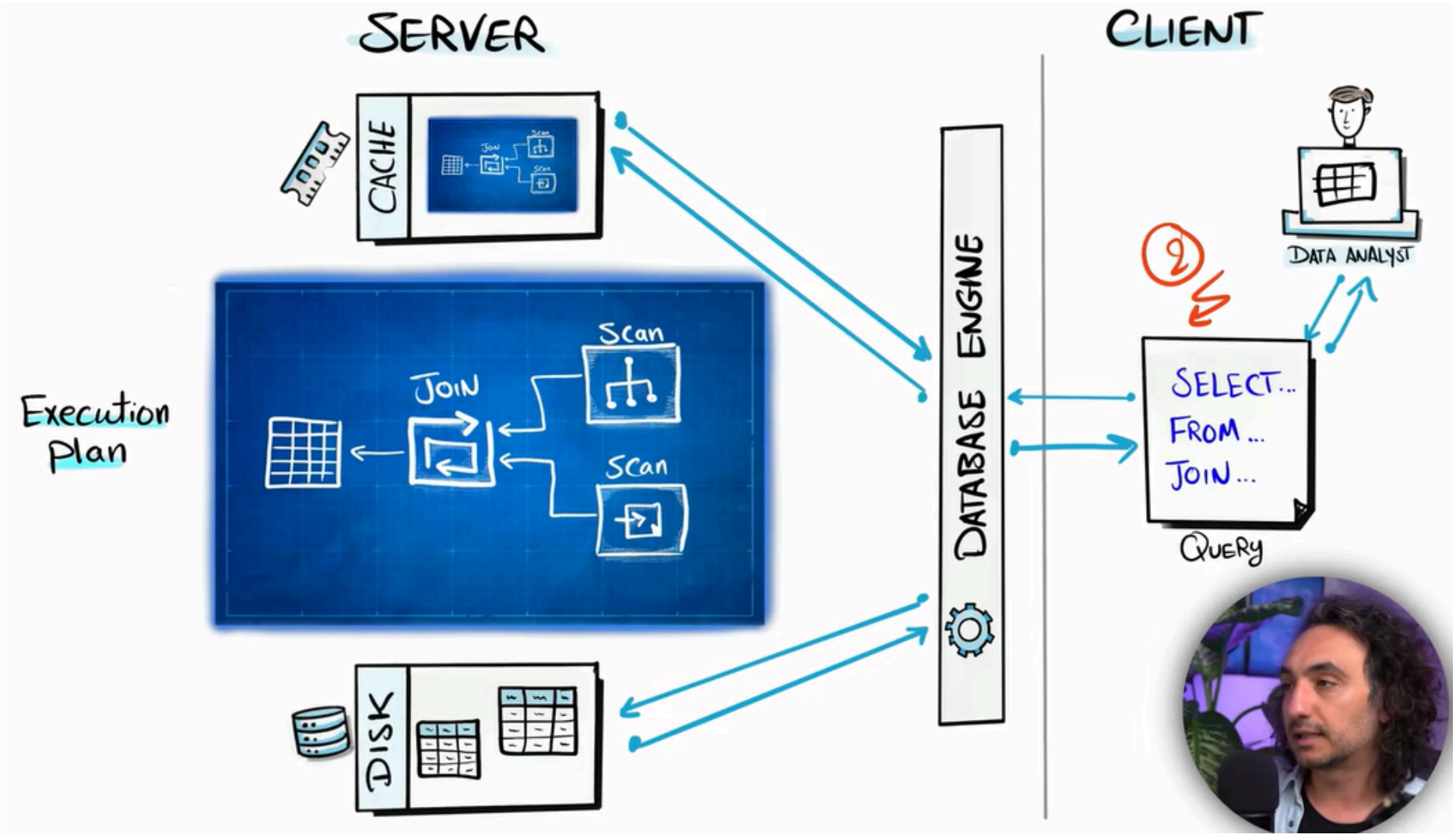
- When to Defragment?
  - <10% → No Action needed
  - 10 - 30% → Reorganize
  - >30% → Rebuild

```
-- Reorganize Index (Lightweight)
ALTER INDEX idx_Customers_Country ON Sales.Customers REORGANIZE;

-- Rebuild Index (Heavyweight)
ALTER INDEX idx_Products_UN_Product ON Sales.Products REBUILD;
```

Execution Plan

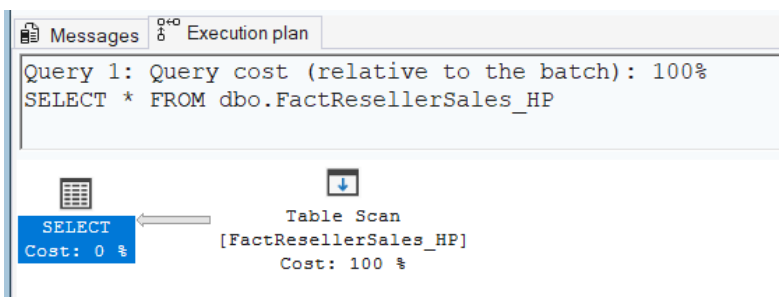
- Roadmap generated by a database on how it will execute your query step by step



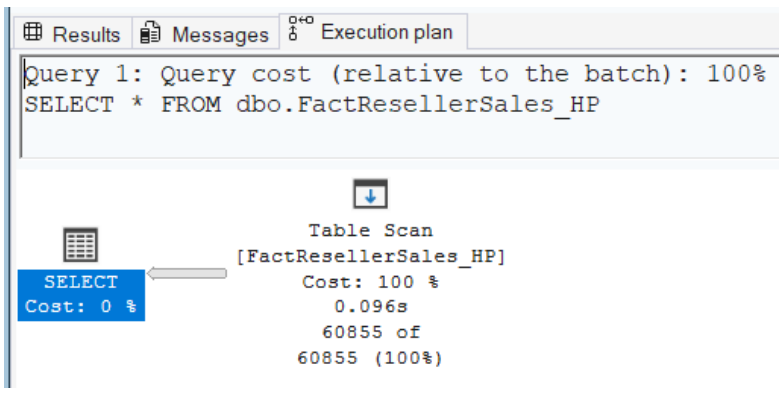


Execution Plan Basics

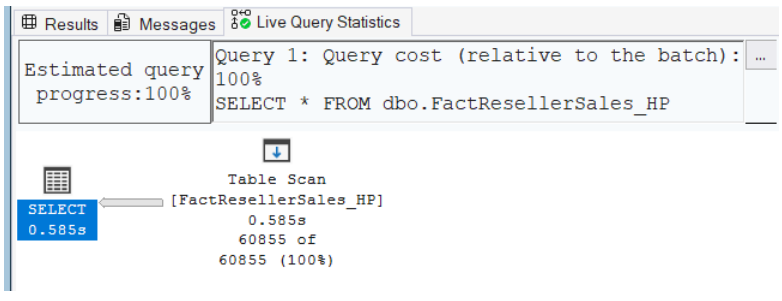
- Estimated Execution Plan
  - Predicts the execution plan without actually running the query



- Actual Execution Plan
  - Shows the execution plan as it occurred after running the query

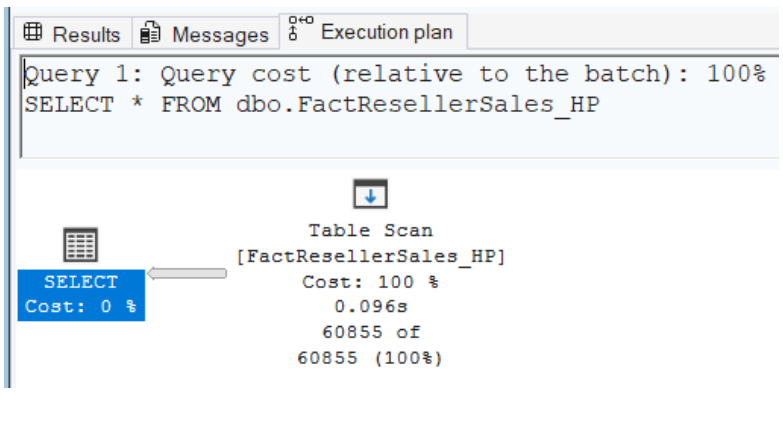


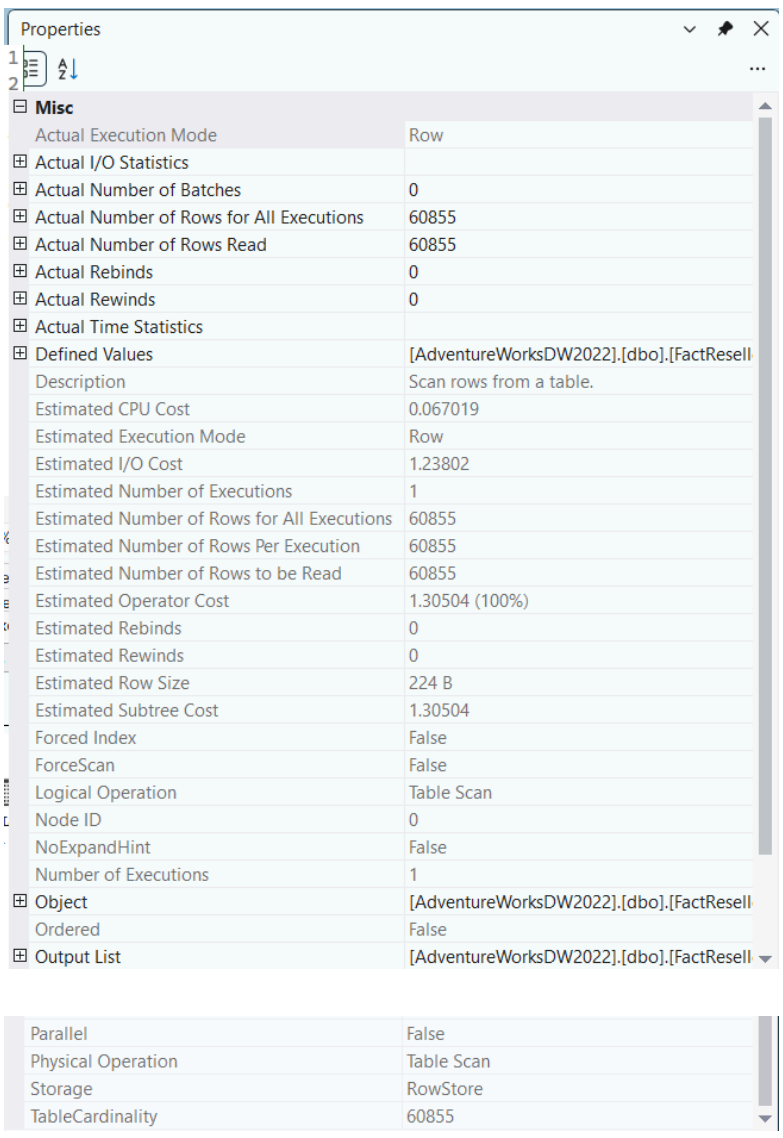
- Live Execution Plan
  - Shows the real-time execution flow as the query runs



- Estimated vs Actual Plans
  - If the predictions don’t match the Actual Execution Plan, this indicate issues like inaccurate statistics or outdated indexes, leading to poor performance
- Table Scan
  - Reads the entire table, page by page and row by row “everything” which can lead to slower query performance on large tables

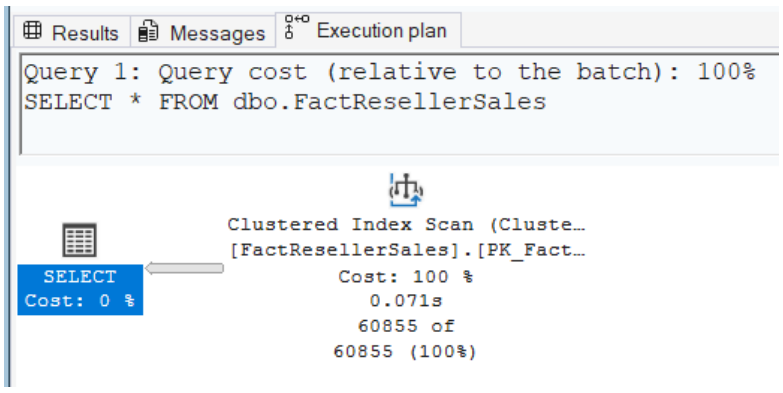
```sql
SELECT *
FROM dbo.FactResellerSales_HP;
```

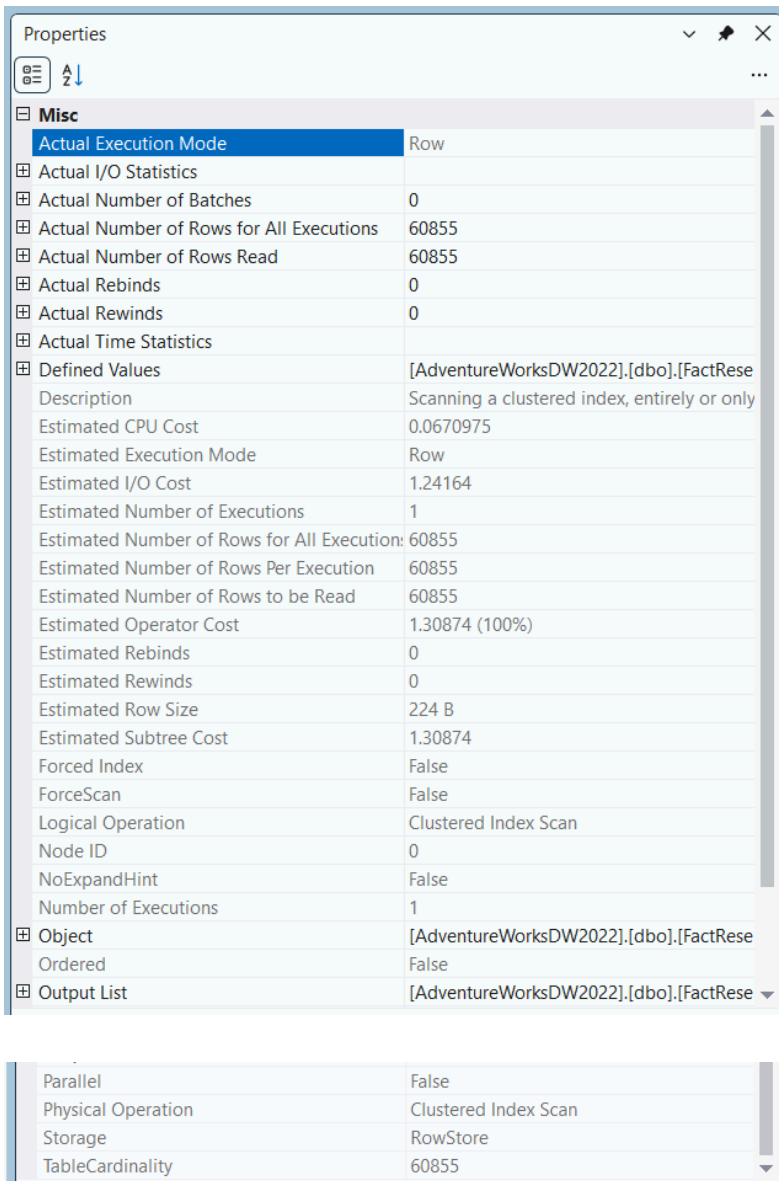




- Index Scan
  - Scans all data in an index to find matching rows

```sql
SELECT *
FROM dbo.FactResellerSales;
```

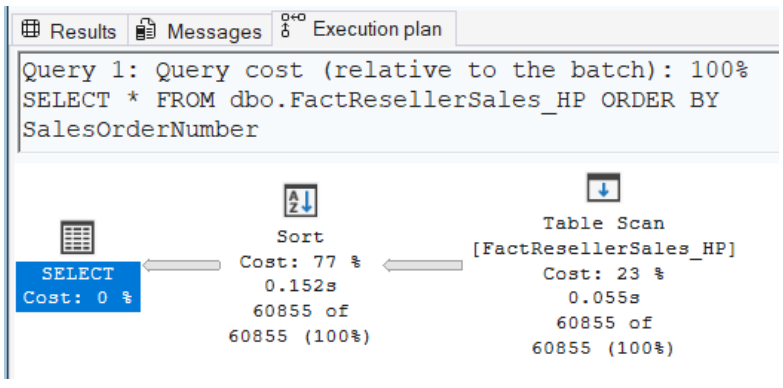




Execution Plan: Heap vs Clustered Index

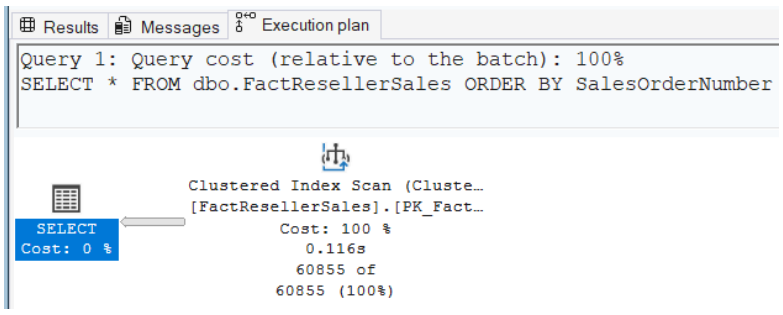
- Heap

```
SELECT *
FROM dbo.FactResellerSales_HP
ORDER BY SalesOrderNumber;
```



- Clustered Index

```
SELECT *
FROM dbo.FactResellerSales
ORDER BY SalesOrderNumber;
```

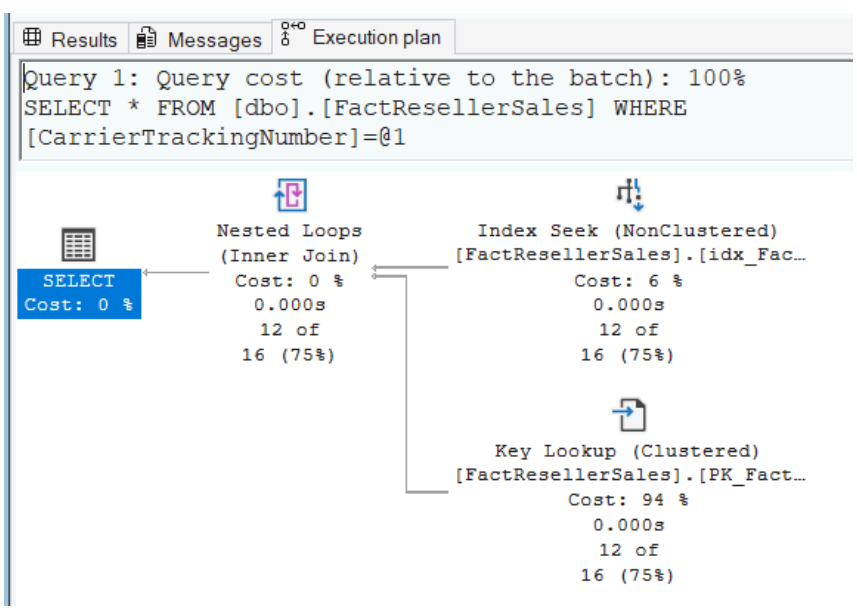


- Sorting Data
  - Heap is slower than Clustered, because Database must perform extra work to sort rows
- TIP
  - After creating a new index, check the execution plan to see if your query uses the index

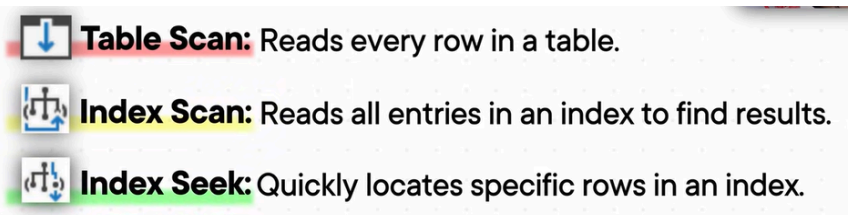
Execution Plan: Non-Clustered Index

- Index Seek
  - A targeted search within an index, retrieving only specific rows

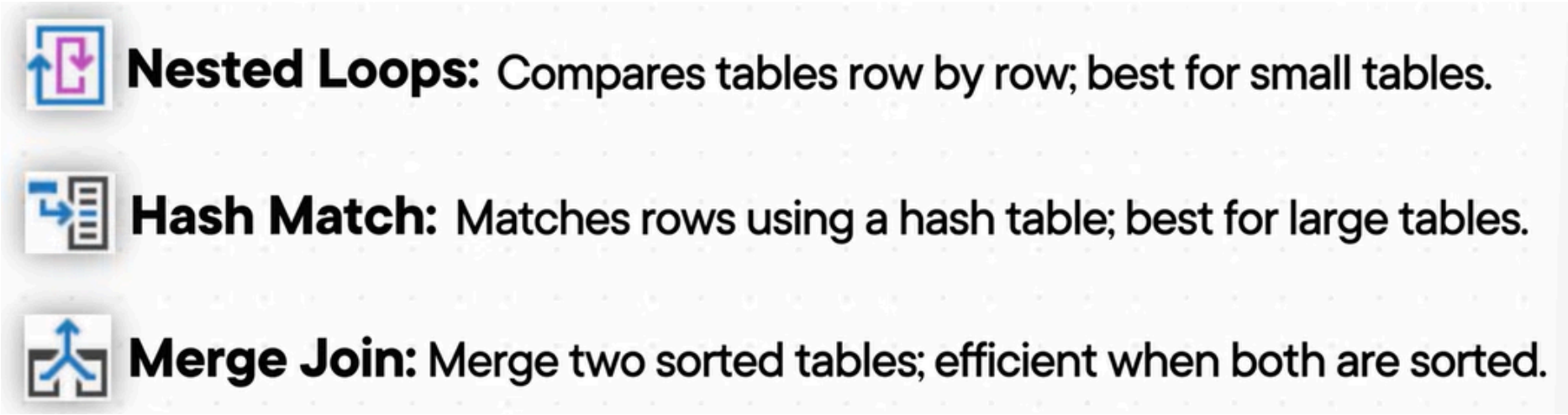
```
CREATE NONCLUSTERED INDEX idx_FactResellerSales_CTA
ON dbo.FactResellerSales (CarrierTrackingNumber);

SELECT *
FROM dbo.FactResellerSales
WHERE CarrierTrackingNumber = '4911-403C-98';
```



Types of Scan



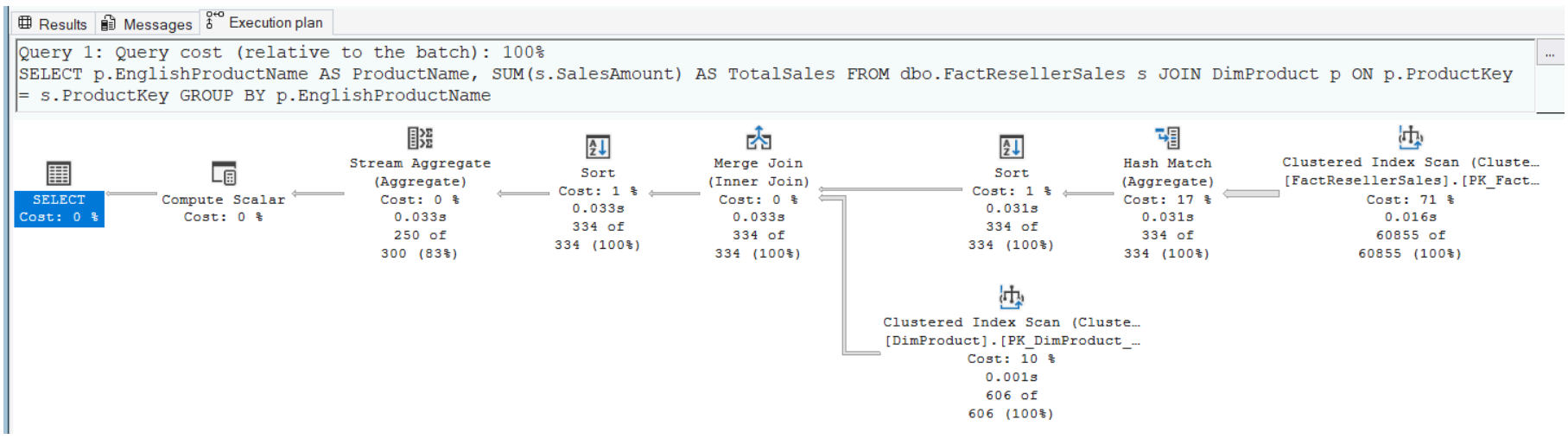
Join Algorithms



Execution Plan: Rowstore vs Columnstore

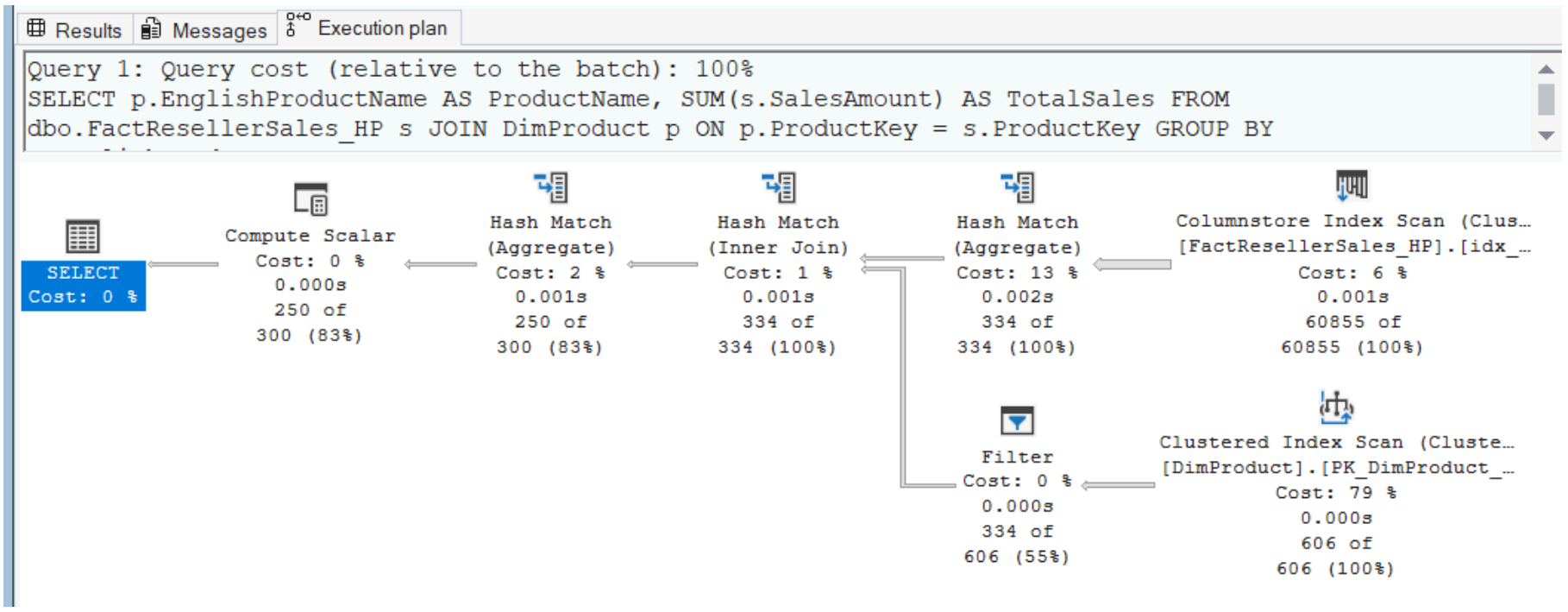
- Rowstore

```
SELECT
    p.EnglishProductName AS ProductName,
    SUM(s.SalesAmount) AS TotalSales
FROM dbo.FactResellerSales s
JOIN DimProduct p
ON p.ProductKey = s.ProductKey
GROUP BY p.EnglishProductName;
```



- Columnstore → High Performance

```
CREATE CLUSTERED COLUMNSTORE INDEX idx_FactResellerSalesHP
ON dbo.FactResellerSales_HP;

SELECT
    p.EnglishProductName AS ProductName,
    SUM(s.SalesAmount) AS TotalSales
FROM dbo.FactResellerSales_HP s
JOIN DimProduct p
ON p.ProductKey = s.ProductKey
GROUP BY p.EnglishProductName;
```



Execution Plan Summary

- Understand how SQL executes your query
- How many resources your query consumes?
- Check if your new indexes are used
- Testing & Experimenting Indexes

SQL Hints

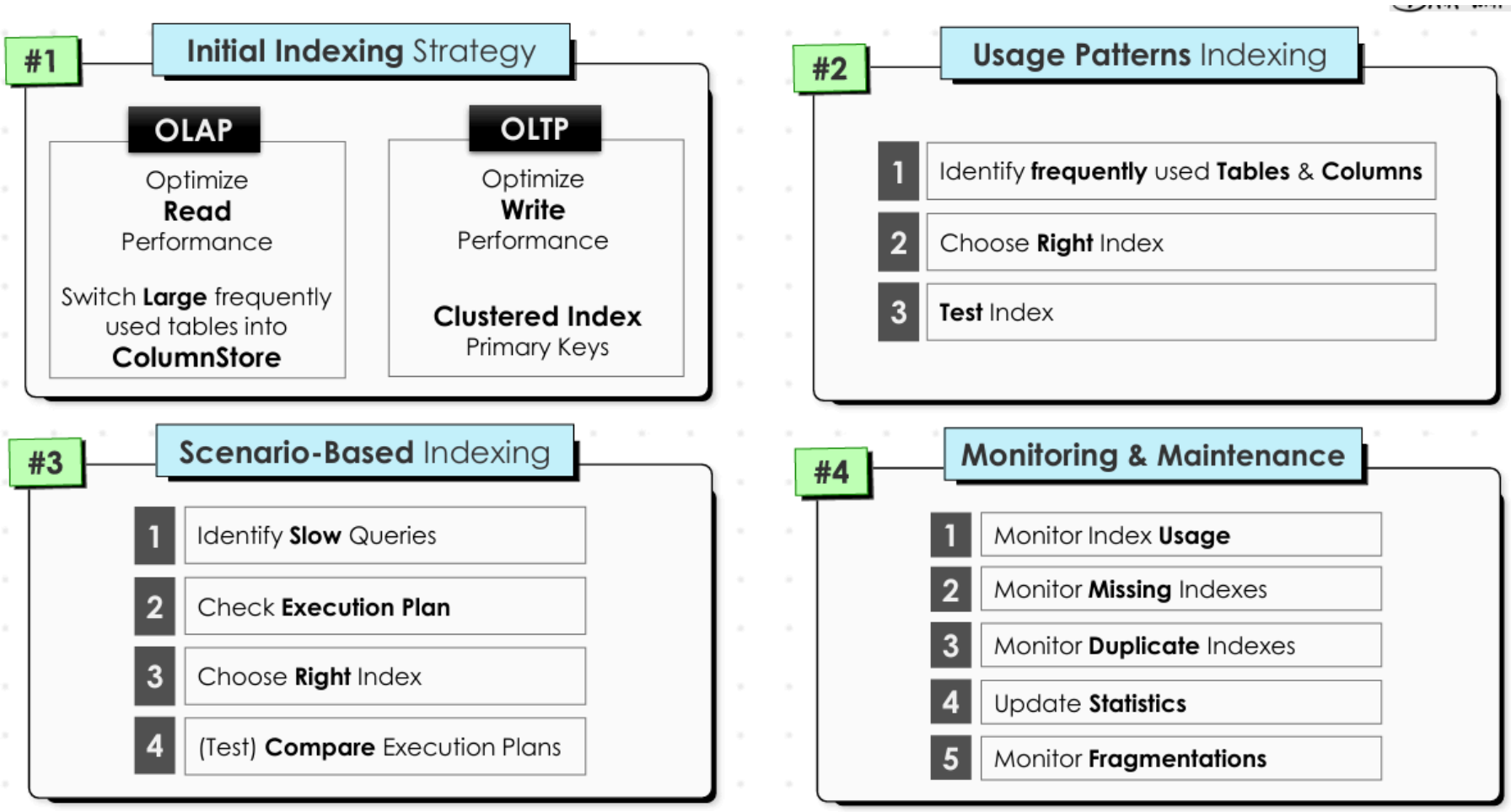
- Commands you add to a query to force the database to run it in a specific way for better performance

```sql
SELECT
    o.Sales,
    c.Country
FROM Sales.Orders o -- WITH (INDEX([PK_orders]))
LEFT JOIN Sales.Customers c -- WITH (FORCESEEK)
ON o.CustomerID = c.CustomerID
-- OPTION (HASH JOIN)
;
```

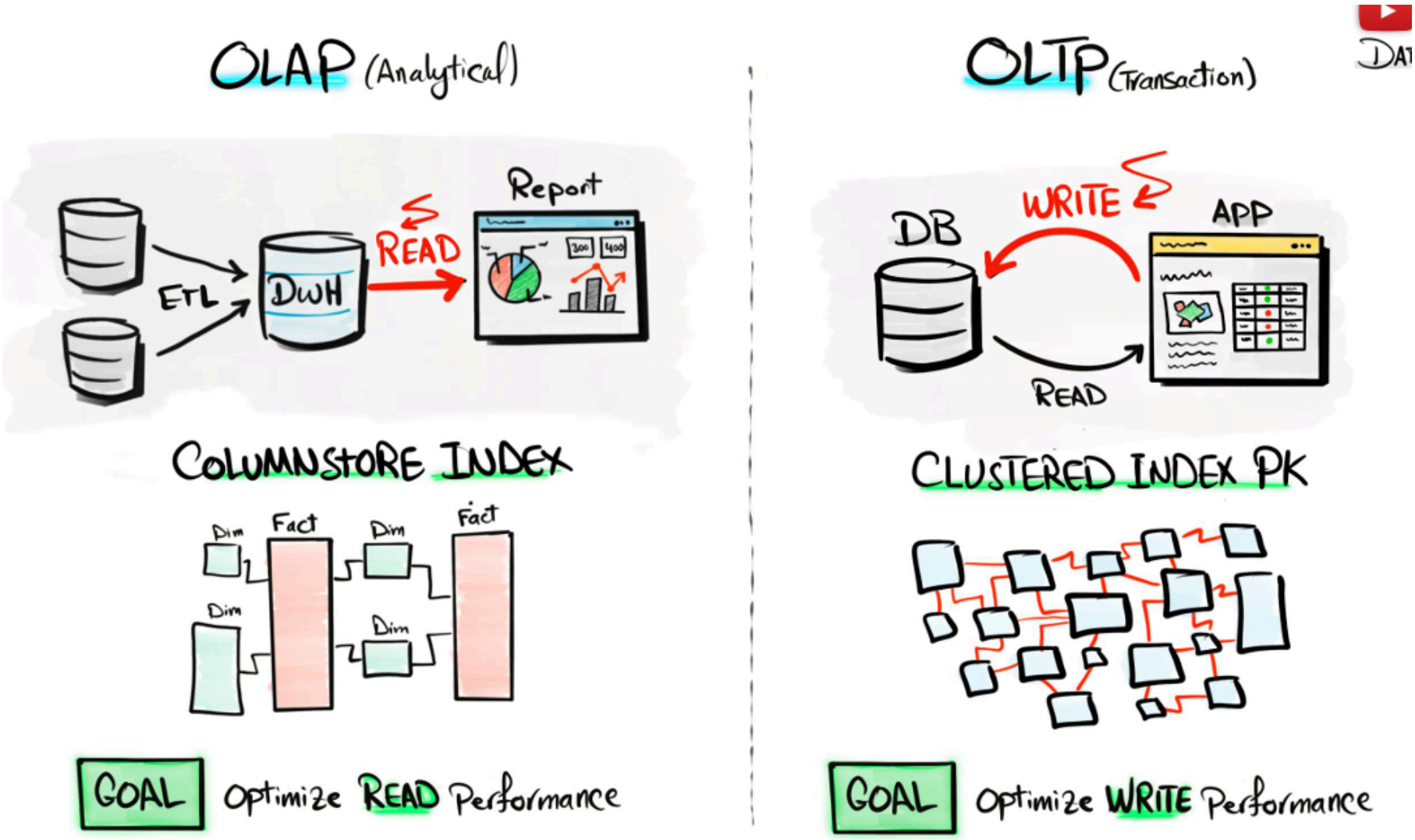
1. Test hints in all project environments (DEV,PROD) as performance may vary
2. Hints are quick fixes (Workaround not Solution), you still have to find the cause and fix it

Indexes Summary (Indexing Strategy)

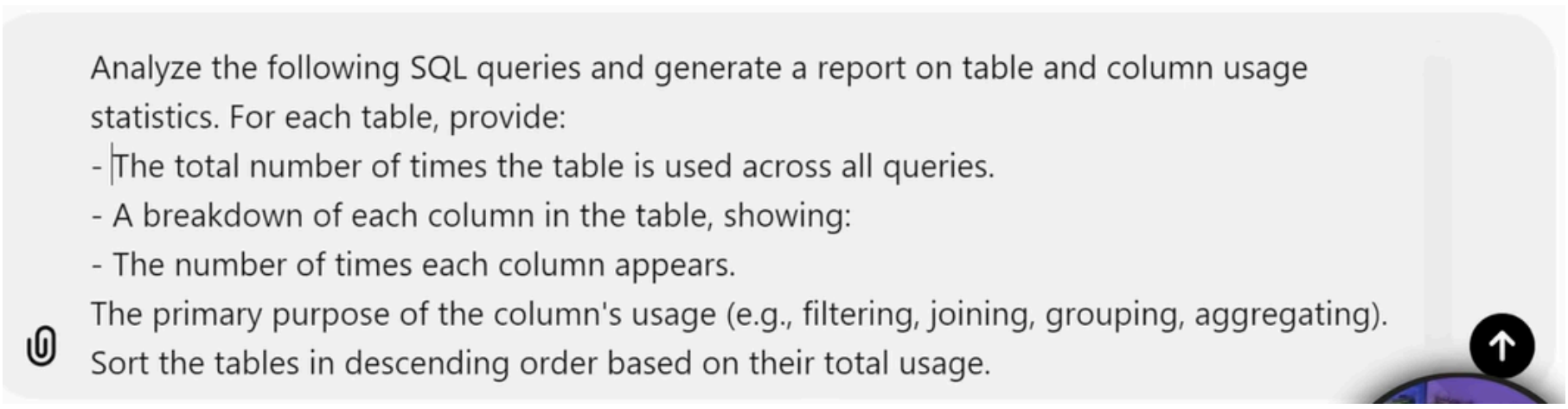
- THE GOLDEN ROLE
  - Avoid Over Indexing
    - Indexes slow down write performance
      - When data is inserted, updated, or deleted, database has to update indexes
    - Too many indexes can confuse execution plan
      - Increase planning time
      - Choose a suboptimal plan
  - Less Is More



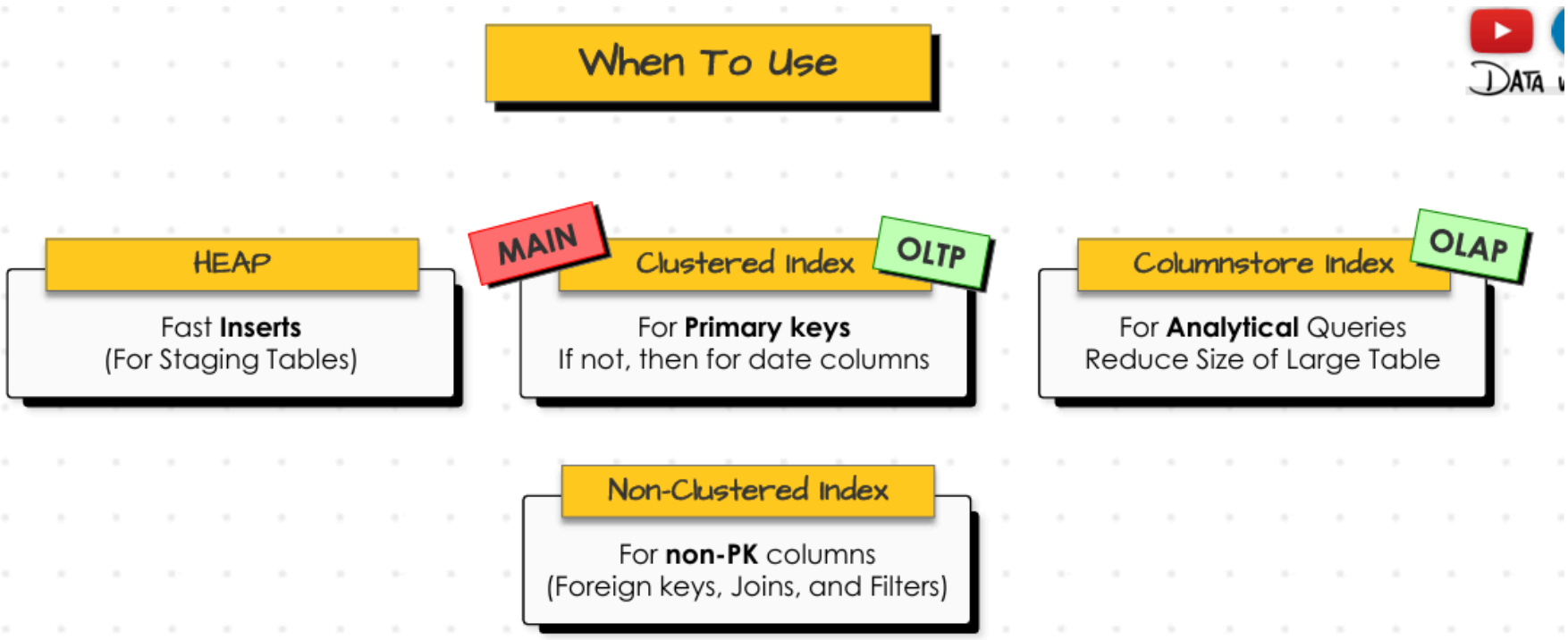
- #1 Phase: Initial Indexing Strategy
  - What is the main goal of our indexing strategy
  - Types of databases
    - OLAP
      - Online Analytical Processing
      - Data Analytics
      - Data Warehouse (DWH)
      - Optimize READ performance
      - Columnstore Index
    - OLTP
      - Online Transaction Processing
      - E-Commerce, Finance, Banking
      - Optimize WRITE performance
      - Clustered Index PK
  - Two Strategies
    - Optimize READ performance
    - Optimize WRITE performance



- #2 Phase: Usage Patterns Indexing
  - Most Important Tables?
  - Most Important Columns?



  - Choose Right Index Type

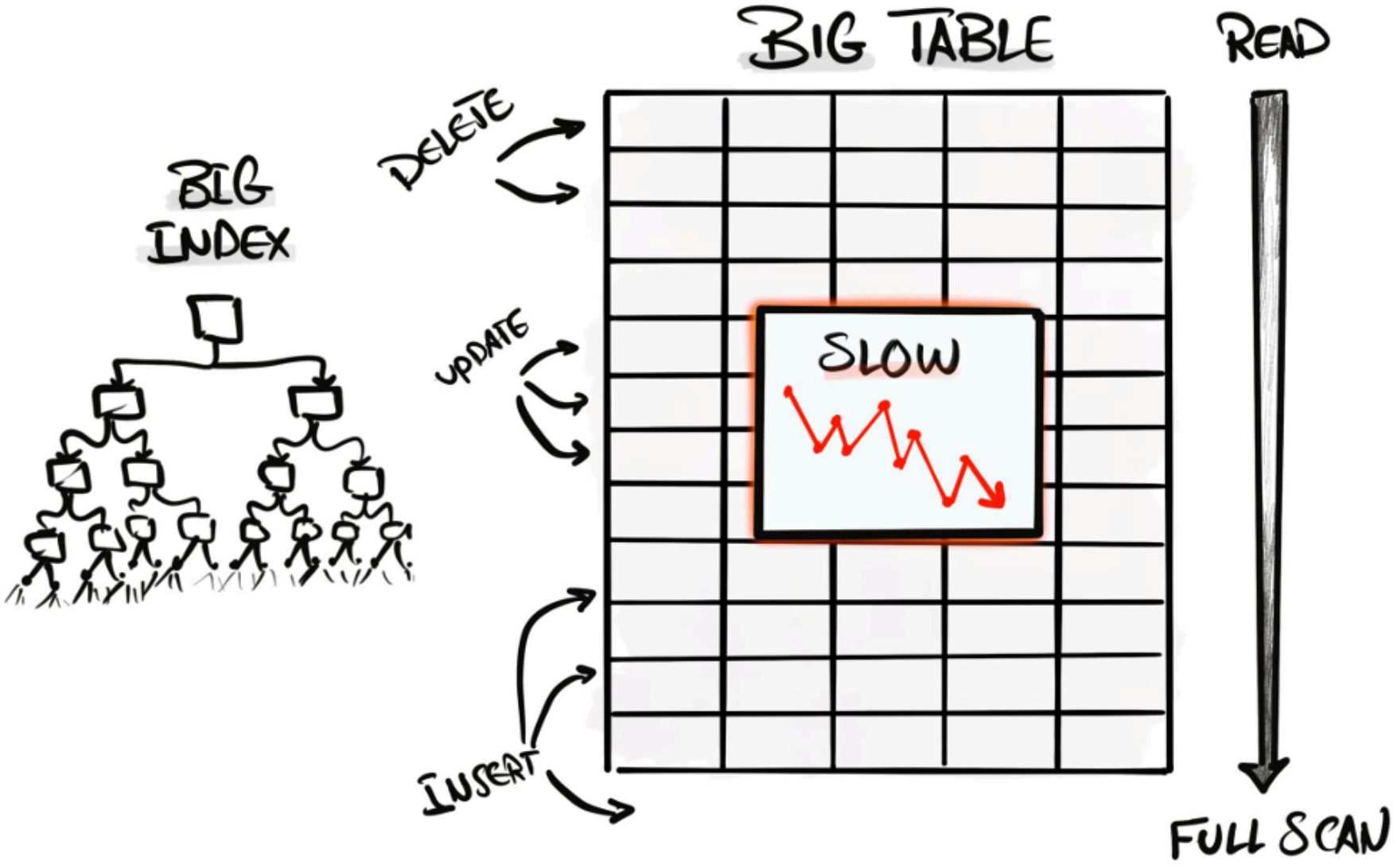


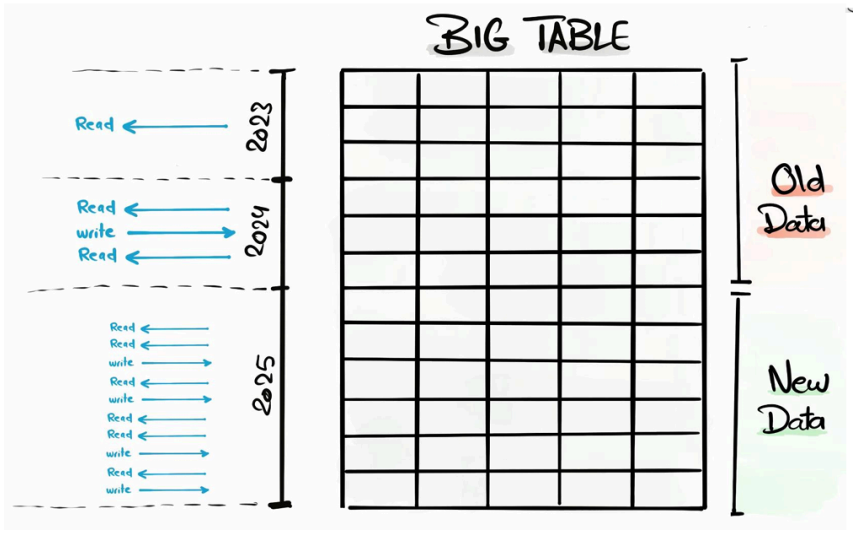
  - Test Index
- #3 Phase: Scenario-Based Indexing
  - Identify Slow Queries
  - Check Execution Plan
  - Choose Right Index Type
  - Test (Compare) Execution Plans
- #4 Phase: Monitoring & Maintenance
  - Monitor Index Usage
  - Monitor Missing Indexes
  - Monitor Duplicate Indexes
  - Update Statistics
  - Monitor Fragmentations

Module 24: Partitions

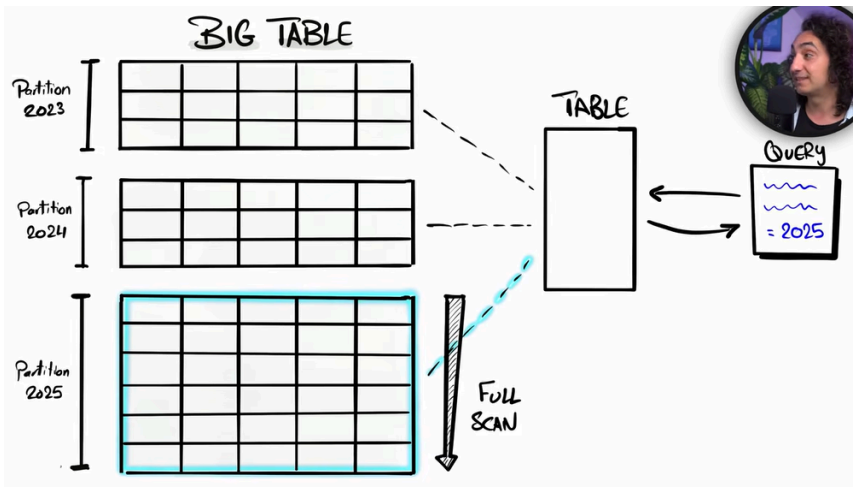
Introduction to Partitions

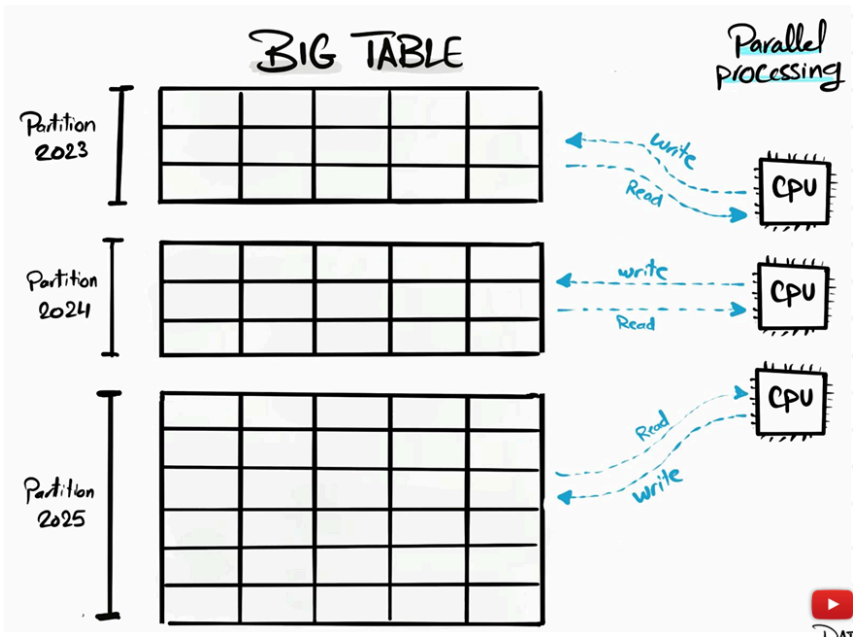
- SQL Partitioning
  - Divides big table into smaller partitions while still being treated as a single logical table

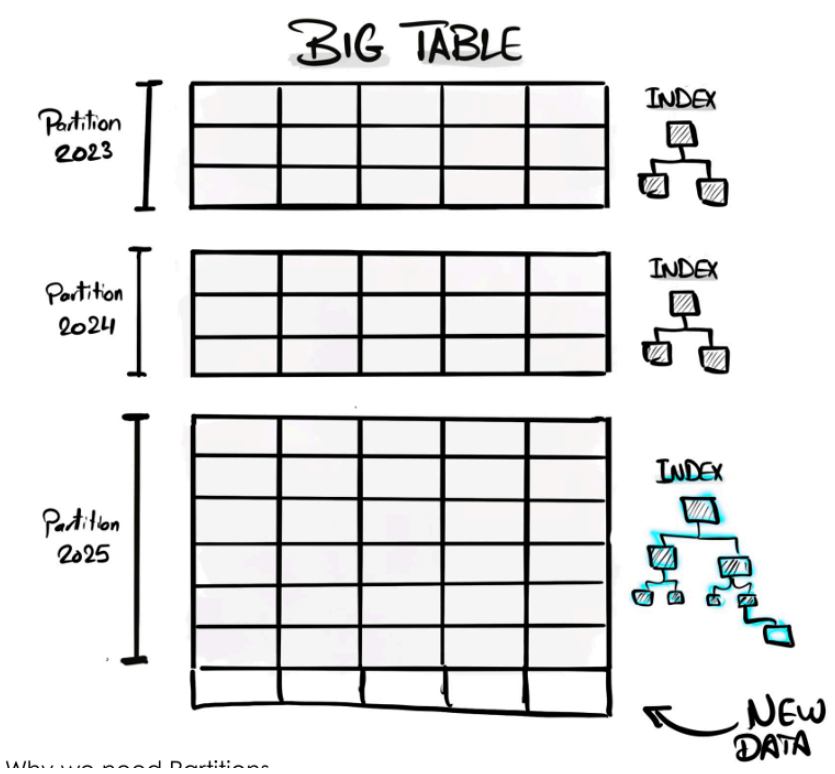

BIG TABLE
READ
BIG INDEX
DELETE
UPDATE
SLOW
INSERT
FULL SCAN


BIG TABLE
Read
2023
Read
write
Read
2024
Read
Read
write
Read
write
Read
Read
write
Read
write
2025
Old Data
New Data

- Benefits


BIG TABLE
Partition 2023
Partition 2024
Partition 2025
TABLE
QUERY
= 2025
FULL SCAN

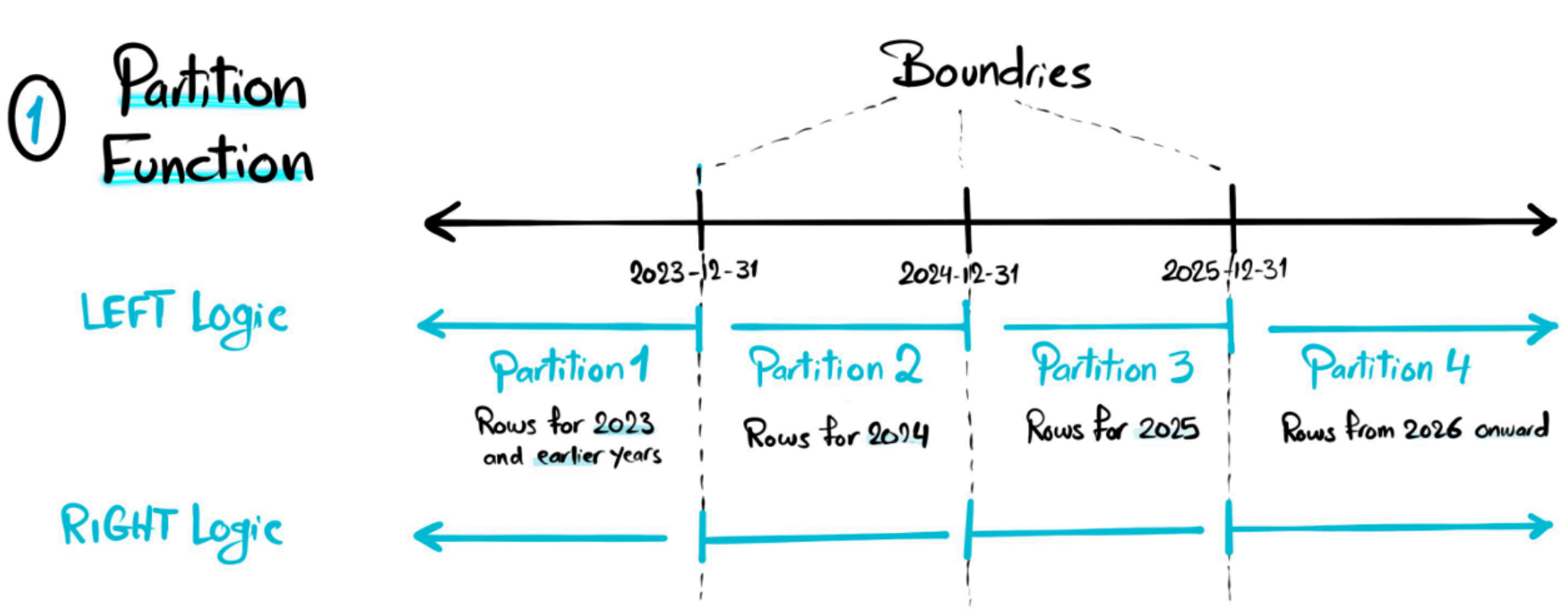




Why we need Partitions

Building Partitions: #1 Create Partition Function

- Partition Function
  - Define the logic on how to divide your data into partitions
  - Based on partition key like (column, region, ..)
  - Range Function



- Syntax

```sql
CREATE PARTITION FUNCTION PartitionName (DataType)
AS PartitionFunctionType (LEFT|RIGHT)
FOR VALUES (Boundaries);

-- EXAMPLE
CREATE PARTITION FUNCTION PartitionByYear (DATE)
AS RANGE LEFT FOR VALUES ('2023-12-31','2024-12-31','2025-12-31');

-- REMOVE
DROP PARTITION FUNCTION PartitionByYear;
```

```sql
-- Query lists all existing partition function
SELECT
    name,
    function_id,
    type,
    type_desc,
    boundary_value_on_right
FROM sys.partition_functions;
```

	name	function_id	type	type_desc	boundary_value_on_right
1	PartitionByYear	65536	R	RANGE	0

Building Partitions: #2 Create Filegroups

- Filegroups
  - Logical container of one or more data files to help organize partitions
- PRIMARY
  - Default filegroup where all objects of database are stored
- Syntax

```sql
ALTER DATABASE DB ADD FILEGROUP FileGroupName;

-- EXAMPLE
ALTER DATABASE SalesDB ADD FILEGROUP FG_2023

-- REMOVE
ALTER DATABASE SalesDB REMOVE FILEGROUP FG_2023;
```

```sql
-- Query lists all existing filegroups
SELECT
    *
FROM sys.filegroups
WHERE type = 'FG';
```

	name	data_space_id	type	type_desc	is_default	is_system	filegroup_guid	log_filegroup_id	is_read_only	is_autogrow_all_files
1	PRIMARY	1	FG	ROWS_FILEGROUP	1	0	NULL	NULL	0	0
2	FG_2023	2	FG	ROWS_FILEGROUP	0	0	5346AA23-F87B-4DFD-BF92-4D5BE4FF6F0E	NULL	0	0
3	FG_2024	3	FG	ROWS_FILEGROUP	0	0	9EB212CC-1788-4FA0-A130-34E3CCA7FC5D	NULL	0	0
4	FG_2025	4	FG	ROWS_FILEGROUP	0	0	17963E5F-E7A1-4E52-BE3C-0DFCEFF36C00	NULL	0	0
5	FG_2026	5	FG	ROWS_FILEGROUP	0	0	4A5B33D9-EB3E-4ED3-931E-8C2582042A23	NULL	0	0

Building Partitions: #3 Create Data Files

- Data Files

    - Physical files where our actual data going to be stored inside it
    - Stored physically in database
    - Assign for each filegroup like one or multiple data files
    - File Format: .ndf (Secondary Data Files)
- Syntax

```
ALTER DATABASE DB ADD FILE
(
    Name = LogicalName,
    FILENAME = 'Complete Path'
) TO FILEGROUP FileGroupName;

-- EXAMPLE
ALTER DATABASE SalesDB ADD FILE
(
    Name = P_2023,
    FILENAME = 'C:\Program Files\Microsoft SQL Server\MSSQL17.SQLEXPRESS\MSSQL\DATA\P_2023.ndf'
) TO FILEGROUP FG_2023;

-- REMOVE
ALTER DATABASE SalesDB REMOVE FILE P_2023;
```

```
-- Query lists all existing data files
SELECT
    fg.name AS FilegroupName,
    mf.name AS LogicalFileName,
    mf.physical_name AS PhysicalFilePath,
    mf.size / 128 AS SizeInMB
FROM sys.filegroups fg
JOIN sys.master_files mf
ON fg.data_space_id = mf.data_space_id
WHERE mf.database_id = DB_ID('SalesDB');
```

	FilegroupName	LogicalFileName	PhysicalFilePath	SizeInMB
1	PRIMARY	SalesDB	C:\Program Files\Microsoft SQL Server\MSSQL17.SQL...	72
2	FG_2023	P_2023	C:\Program Files\Microsoft SQL Server\MSSQL17.SQL...	8
3	FG_2024	P_2024	C:\Program Files\Microsoft SQL Server\MSSQL17.SQL...	8
4	FG_2025	P_2025	C:\Program Files\Microsoft SQL Server\MSSQL17.SQL...	8
5	FG_2026	P_2026	C:\Program Files\Microsoft SQL Server\MSSQL17.SQL...	8

Building Partitions: #4 Create Partition Scheme

- Partition Scheme
    - Connect the partition to the filegroup
- NOTE
    - Sort the filegroups according to the result of the function’s partitions
    - 3 Boundaries = 4 Partitions = 4 Filegroups
- Syntax

```
CREATE PARTITION SCHEME SchemeName
AS PARTITION PartitionName
TO (FilegroupName, FilegroupName, ..);

-- EXAMPLE
CREATE PARTITION SCHEME SchemePartitionByYear
```

```
AS PARTITION PartitionByYear
TO (FG_2023, FG_2024, FG_2025, FG_2026);

-- REMOVE
DROP PARTITION SCHEME SchemePartitionByYear;
```

```
-- Query lists all existing partition schemes
SELECT
    ps.name AS PartitionSchemeName,
    pf.name AS PartitionFunctionName,
    ds.destination_id AS PartitionNumber,
    fg.name AS FilegroupName
FROM sys.partition_schemes ps
JOIN sys.partition_functions pf
ON ps.function_id = pf.function_id
JOIN sys.destination_data_spaces ds
ON ps.data_space_id = ds.partition_scheme_id
JOIN sys.filegroups fg
ON ds.data_space_id = fg.data_space_id;
```

	PartitionSchemeName	PartitionFunctionName	PartitionNumber	FilegroupName
1	SchemePartitionByYear	PartitionByYear	1	FG_2023
2	SchemePartitionByYear	PartitionByYear	2	FG_2024
3	SchemePartitionByYear	PartitionByYear	3	FG_2025
4	SchemePartitionByYear	PartitionByYear	4	FG_2026

Building Partitions: #5 Create Partitioned Table

- Syntax

```
CREATE TABLE TableName
(
    Columns
) ON PartitionSchemeName (Column);

-- EXAMPLE
CREATE TABLE Sales.Orders_Partitioned
(
    OrderID INT,
    OrderDate DATE,
    Sales INT
) ON SchemePartitionByYear (OrderDate);
```

Building Partitions: #6 Insert Data

```
INSERT INTO Sales.Orders_Partitioned VALUES (1, '2023-05-15', 100);
INSERT INTO Sales.Orders_Partitioned VALUES (2, '2024-07-20', 50);
INSERT INTO Sales.Orders_Partitioned VALUES (3, '2025-12-31', 20);
INSERT INTO Sales.Orders_Partitioned VALUES (4, '2026-01-01', 100);
```

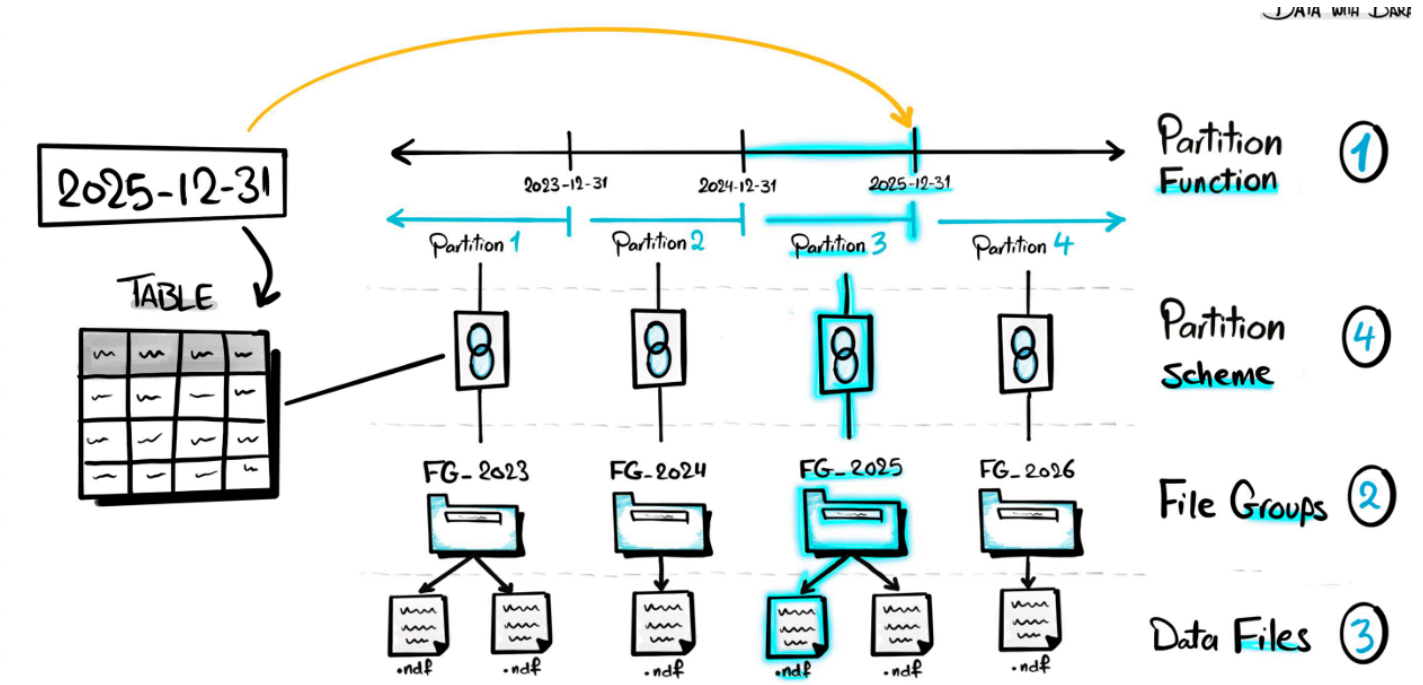
```
SELECT * FROM Sales.Orders_Partitioned
```

	OrderID	OrderDate	Sales
1	1	2023-05-15	100
2	2	2024-07-20	50
3	3	2025-12-31	20
4	4	2026-01-01	100

```sql
-- Query Test
SELECT
    p.partition_number AS PartitionNumber,
    f.name AS PartitionFilegroup,
    p.rows AS NumberOfRows
FROM sys.partitions p
JOIN sys.destination_data_spaces dds
ON p.partition_number = dds.destination_id
JOIN sys.filegroups f
ON dds.data_space_id = f.data_space_id
WHERE OBJECT_NAME(p.object_id) = 'Orders_Partitioned';
```

	PartitionNumber	PartitionFilegroup	NumberOfRows
1	1	FG_2023	1
2	2	FG_2024	1
3	3	FG_2025	1
4	4	FG_2026	1

Everything is Connected



The Performance (Execution Plan)

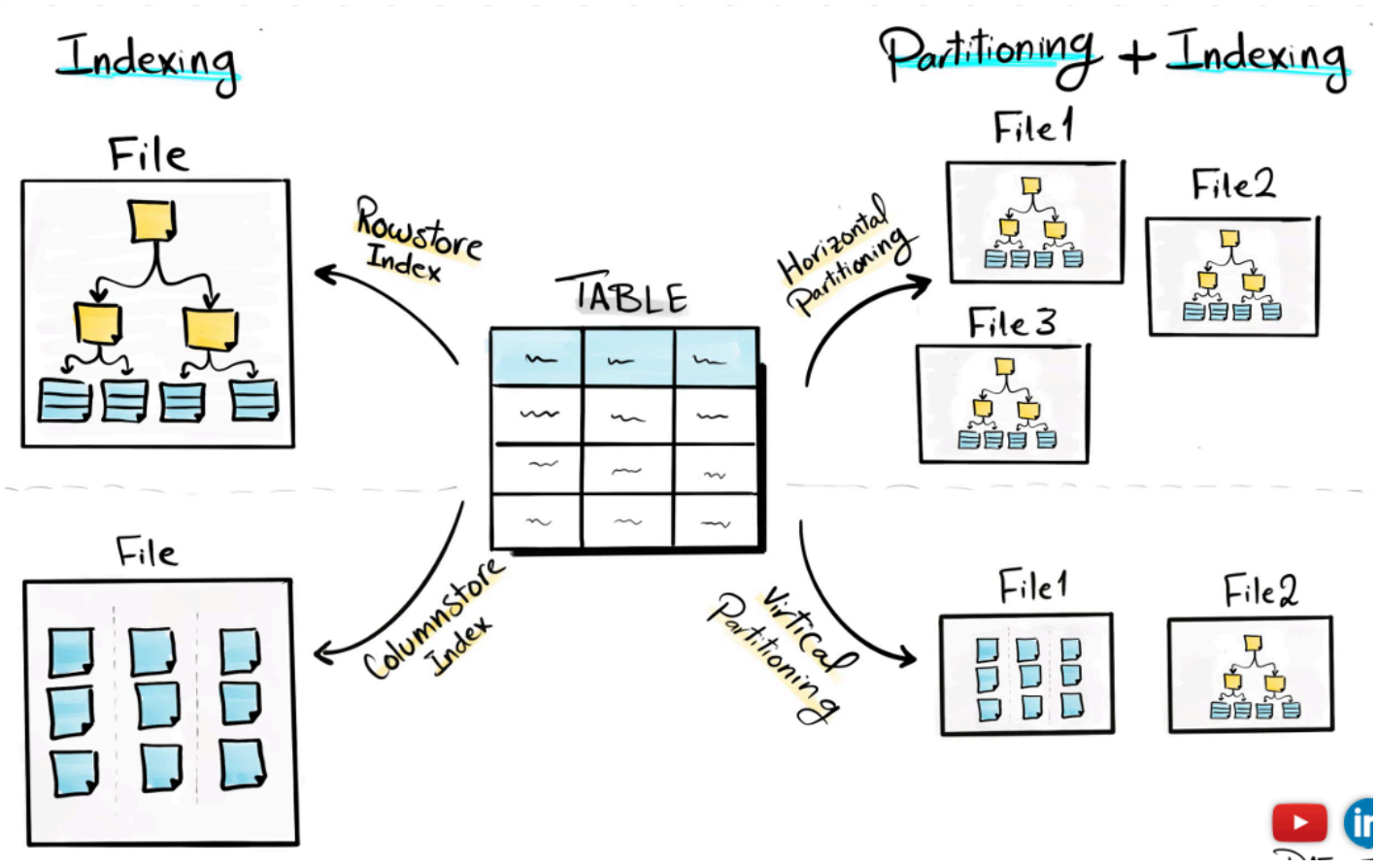
```sql
SELECT *
FROM Sales.Orders_NoPartition
WHERE OrderDate = '2026-01-01';
```

Actual Number of Rows for All Executions	1
Actual Number of Rows Read	4
Actual Rebinds	0

```sql
SELECT *
FROM Sales.Orders_Partitioned
WHERE OrderDate = '2026-01-01';
```

Actual Number of Rows for All Executions	1
Actual Number of Rows Read	1
Actual Partition Count	1

Partitioning & Indexes

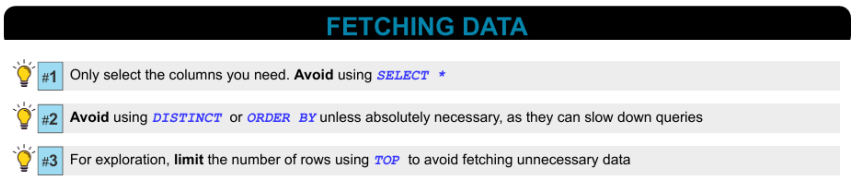


Module 25: Performance Tips

The Golden Rule

- For small-medium tables, the query optimizer may react similarly to different query styles
- Always check the execution plan to confirm performance improvements when optimizing your query
- If there's no improvement, then just focus on readability

Fetching Data



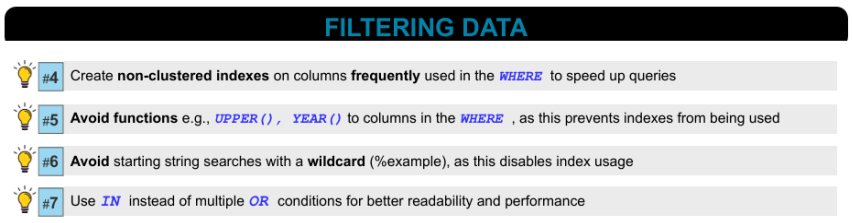

```
-- ======================================================
-- Tip 1: Select Only What You Need
-- ======================================================

-- Bad Practice
SELECT * FROM Sales.Customers;

-- Good Practice
SELECT CustomerID, FirstName, LastName FROM Sales.Customers;
```

```
-- ======================================================
-- Tip 2: Avoid unnecessary DISTINCT & ORDER BY
-- ======================================================
```

```sql
-- Bad Practice
SELECT DISTINCT
    FirstName
FROM Sales.Customers
ORDER BY FirstName;

-- Good Practice
SELECT FirstName
FROM Sales.Customers;
```

```sql
-- ====================================================
-- Tip 3: For Exploration Purpose, Limit Rows
-- ====================================================

-- Bad Practice
SELECT
    OrderID,
    Sales
FROM Sales.Orders;

-- Good Practice
SELECT TOP 10
    OrderID,
    Sales
FROM Sales.Orders;
```

Filtering Data




```sql
-- ====================================================
-- Tip 4: Create nonclustered index on frewuently
--        used columns in WHERE clause
-- ====================================================

-- Bad Practice
SELECT * FROM Sales.Orders WHERE OrderStatus = 'Delivered';

-- Good Practice
CREATE NONCLUSTERED INDEX idx_Orders_OrderStatus
ON Sales.Orders (OrderStatus);

SELECT * FROM Sales.Orders WHERE OrderStatus = 'Delivered';
```

```sql
-- ====================================================
-- Tip 5: Avoid applying functions to columns
--        in WHERE clauses
-- ====================================================

-- Bad Practice
SELECT *
```

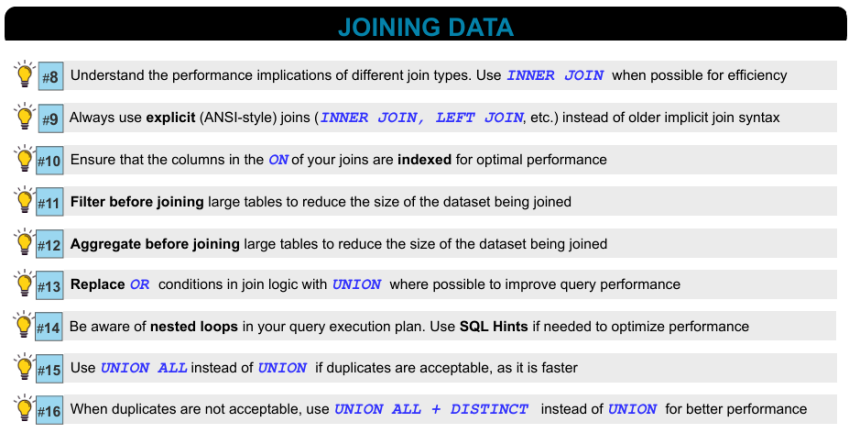
```sql
FROM Sales.Orders
WHERE LOWER(OrderStatus) = 'delivered';

SELECT *
FROM Sales.Customers
WHERE SUBSTRING(FirstName,1,1) = 'A';

SELECT *
FROM Sales.Orders
WHERE YEAR(OrderDate) = 2025;

-- Good Practice
SELECT *
FROM Sales.Orders
WHERE OrderStatus = 'Delivered';

SELECT *
FROM Sales.Customers
WHERE FirstName LIKE 'A%';

SELECT *
FROM Sales.Orders
WHERE OrderDate BETWEEN '2025-01-01' AND '2025-12-31';
```

- NOTE
  - Functions on columns can block index usage

```sql
-- =============================================
-- Tip 6: Avoid leading wilcards as they prevent
--        index usage
-- =============================================

-- Bad Practice
SELECT *
FROM Sales.Customers
WHERE LastName LIKE '%Gold%';

-- Good Practice
SELECT *
FROM Sales.Customers
WHERE LastName LIKE 'Gold%';
```

```sql
-- =============================================
-- Tip 7: Use IN istead of multiple OR conditions
-- =============================================

-- Bad Practice
SELECT *
FROM Sales.Orders
WHERE CustomerID = 1 OR CustomerID = 2 OR CustomerID = 3;

-- Good Practice
SELECT *
FROM Sales.Orders
WHERE CustomerID IN (1, 2, 3);
```

Joining Data




```
-- ======================================================
-- Tip 8: Understand the speed of joins & use
--        INNER JOIN when possible
-- ======================================================

-- Best Performance
SELECT c.FirstName, o.OrderID
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID;

-- Slightly Slower Performance
SELECT c.FirstName, o.OrderID
FROM Sales.Customers c
RIGHT JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID;

SELECT c.FirstName, o.OrderID
FROM Sales.Customers c
LEFT JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID;

-- Worst Performance
SELECT c.FirstName, o.OrderID
FROM Sales.Customers c
FULL OUTER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID;
```

```
-- ======================================================
-- Tip 9: Use explicit join (ANSI Join) instead of
--        implicit join (non-ANSI Join)
-- ======================================================

-- Bad Practice
SELECT c.FirstName, o.OrderID
FROM Sales.Customers c, Sales.Orders o
WHERE c.CustomerID = o.CustomerID;

-- Good Practice
SELECT c.FirstName, o.OrderID
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID;
```

```sql
-- ======================================================
-- Tip 10: Make sure to index the columns used in
--         the ON clause
-- ======================================================

-- Bad Practice
SELECT c.FirstName, o.OrderID
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID;

-- Good Practice
CREATE NONCLUSTERED INDEX idx_Orders_CustomerID
ON Sales.Orders (CustomerID);

SELECT c.FirstName, o.OrderID
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID;
```

```sql
-- ======================================================
-- Tip 11: Filtering Befor Joining (big tables)
-- ======================================================

-- Best Practice For Small-Medium Tables
-- Filter After Join (WHERE)
SELECT c.FirstName, o.OrderID
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID
WHERE o.OrderStatus = 'Delivered';

-- Filter During Join (ON)
SELECT c.FirstName, o.OrderID
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID
AND o.OrderStatus = 'Delivered';

-- Best Practice For Big Tables
-- Filter Befor Join (SUBQUERY or CTE)
SELECT c.FirstName, o.OrderID
FROM Sales.Customers c
INNER JOIN (SELECT OrderID, CustomerID FROM Sales.Orders WHERE OrderStatus = 'Delivered') o
ON c.CustomerID = o.CustomerID;
```

- TIP
  - Try to isolate the preparation step in a CTE or SUBQUERY

```sql
-- ======================================================
-- Tip 12: Aggregate Before Joining (big tables)
-- ======================================================

-- Best Practice For Small-Medium Tables
-- Grouping ans Joining
SELECT c.CustomerID, c.FirstName, COUNT(o.OrderID) AS OrderCount
```

```sql
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.FirstName;

-- Best Practice For Big Tables
-- Pre-aggregated Subquery (or CTE)
SELECT c.CustomerID, c.FirstName, o.OrderCount
FROM Sales.Customers c
INNER JOIN (
    SELECT CustomerID, COUNT(OrderID) AS OrderCount
    FROM Sales.Orders
    GROUP BY CustomerID
) o
ON c.CustomerID = o.CustomerID;

-- Bad Practice
-- Correlated Subquery
SELECT
    CustomerID,
    FirstName,
    (SELECT COUNT(OrderID)
     FROM Sales.Orders o
     WHERE o.CustomerID = c.CustomerID) AS OrderCount
FROM Sales.Customers c;
```

- Correlated Queries are inefficient because SQL execute aggregations for every row

```sql
-- =================================================
-- Tip 13: Use UNION instead of OR in Joins
-- =================================================

-- Bad Practice
SELECT o.OrderID, c.FirstName
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID
OR c.CustomerID = o.SalesPersonID;

-- Best Practice
SELECT o.OrderID, c.FirstName
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID
UNION
SELECT o.OrderID, c.FirstName
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.SalesPersonID;
```

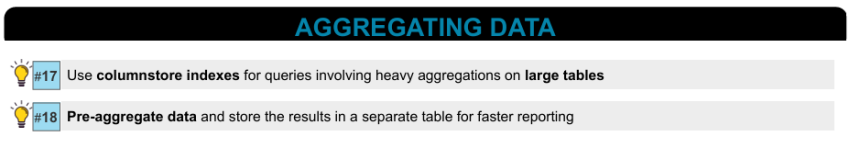
```sql
-- =================================================
-- Tip 14: Check for Nested Loops ans use SQL Hints
-- =================================================

-- Bad Practice
SELECT o.OrderID, c.FirstName
FROM Sales.Customers c
```

```sql
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID;

-- Good Practice for Having Big & Small Tables
SELECT o.OrderID, c.FirstName
FROM Sales.Customers c
INNER JOIN Sales.Orders o
ON c.CustomerID = o.CustomerID
OPTION (HASH JOIN);
```

```sql
-- ====================================================
-- Tip 15: Use UNION ALL instead of using UNION
--         if duplicates are acceptable
-- ====================================================

-- Bad Practice
SELECT CustomerID FROM Sales.Orders
UNION
SELECT CustomerID FROM Sales.OrdersArchive;

-- Best Practice
SELECT CustomerID FROM Sales.Orders
UNION ALL
SELECT CustomerID FROM Sales.OrdersArchive;
```

```sql
-- ====================================================
-- Tip 16: Use UNION ALL + DISTINCT instead of using
--         UNION if duplicates are not acceptable
-- ====================================================

-- Bad Practice
SELECT CustomerID FROM Sales.Orders
UNION
SELECT CustomerID FROM Sales.OrdersArchive;

-- Best Practice
SELECT DISTINCT CustomerID
FROM (
    SELECT CustomerID FROM Sales.Orders
    UNION ALL
    SELECT CustomerID FROM Sales.OrdersArchive
) AS CombinedData;
```

Aggregating Data



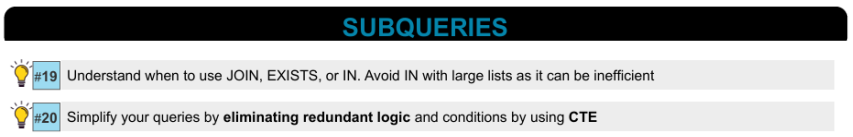

```sql
-- ====================================================
-- Tip 17: Use Columnstore Index for aggregations
--         on large table
-- ====================================================

-- Bad Practice
```

```sql
SELECT CustomerID, COUNT(OrderID) AS OrderCount
FROM Sales.Orders
GROUP BY CustomerID;

-- Best Practice
CREATE CLUSTERED COLUMNSTORE INDEX idx_Orders_Columnstore ON Sales.Orders;

SELECT CustomerID, COUNT(OrderID) AS OrderCount
FROM Sales.Orders
GROUP BY CustomerID;
```

```sql
-- =============================================
-- Tip 18: Pre-Aggregate data and store it in new
--         table for reporting
-- =============================================

-- Bad Practice
SELECT MONTH(OrderDate) AS OrderMonth, SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY MONTH(OrderDate);

-- Good Practice
SELECT MONTH(OrderDate) AS OrderMonth, SUM(Sales) AS TotalSales
INTO Sales.SalesSummary
FROM Sales.Orders
GROUP BY MONTH(OrderDate);

SELECT OrderMonth, TotalSales FROM Sales.SalesSummary;
```

- Don’t forget to update the new table

Subqueries




```sql
-- =============================================
-- Tip 19: Understand when to use JOIN, EXISTS, or IN
-- =============================================

-- JOIN (Best Practice: If the performance equals to EXISTS)
SELECT o.OrderID, o.Sales
FROM Sales.Orders o
INNER JOIN Sales.Customers c
ON o.CustomerID = c.CustomerID
WHERE c.Country = 'USA';

-- EXISTS (Best Practice: Use it for large tables)
SELECT o.OrderID, o.Sales
FROM Sales.Orders o
WHERE EXISTS (
    SELECT 1
    FROM Sales.Customers c
    WHERE c.CustomerID = o.CustomerID
    AND c.Country = 'USA'
```

```sql
);

-- IN (Bad Practice for large tables)
SELECT o.OrderID, o.Sales
FROM Sales.Orders o
WHERE o.CustomerID IN (
    SELECT CustomerID
    FROM Sales.Customers
    WHERE Country = 'USA'
);
```

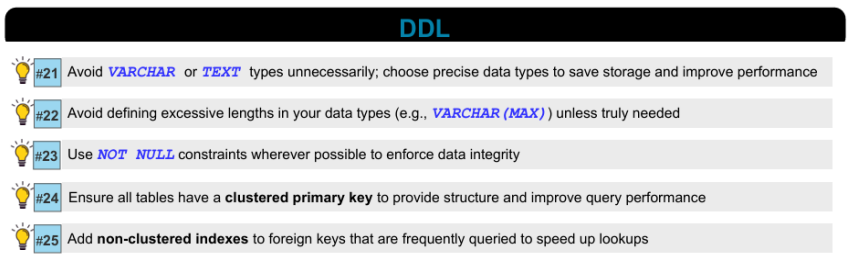
- The IN operator processes and evaluates all rows, it lacks an early exist mechanism
- EXISTS better than JOIN because, it stops at first match and avoid data duplication

```sql
-- ================================================
-- Tip 20: Avoid Redundant Logic in Your Query
-- ================================================

-- Bad Practice
SELECT EmployeeID, FirstName, 'Above Average' Status
FROM Sales.Employees
WHERE Salary > (SELECT AVG(Salary) FROM Sales.Employees)
UNION ALL
SELECT EmployeeID, FirstName, 'Bellow Average' Status
FROM Sales.Employees
WHERE Salary < (SELECT AVG(Salary) FROM Sales.Employees);

-- Best Practice
-- CTE
WITH CTE_EmployeeStatus AS (
    SELECT
        EmployeeID,
        FirstName,
        Salary,
        AVG(Salary) OVER () AS AvgSalary
    FROM Sales.Employees
)
SELECT
    EmployeeID,
    FirstName,
    CASE
        WHEN Salary > AvgSalary THEN 'Above Average'
        WHEN Salary < AvgSalary THEN 'Bellow Average'
        ELSE 'Average'
    END AS Status
FROM CTE_EmployeeStatus;

-- Window Functions
SELECT
    EmployeeID,
    FirstName,
    CASE
        WHEN Salary > AVG(Salary) OVER () THEN 'Above Average'
        WHEN Salary < AVG(Salary) OVER () THEN 'Bellow Average'
        ELSE 'Average'
```

```
    END AS Status
FROM Sales.Employees;
```

- NOTE
  - Spot redundant queries? Review and fix them!

Creating Tables (DDL)




```
-- ================================================
-- Tip 21: Avoid Data Types VARCHAR & TEXT
-- ================================================

-- Bad Practice
CREATE TABLE CustomersInfo (
    CustomerID INT,
    FirstName VARCHAR(MAX),
    LastName TEXT,
    Country VARCHAR(255),
    TotalPurchases FLOAT,
    Score VARCHAR(255),
    BirthDate VARCHAR(255),
    EmployeeID INT,
    CONSTRAINT FK_CustomersInfo_EmployeeID FOREIGN KEY (EmployeeID)
        REFERENCES Sales.Employees(EmployeeID)
);

-- Good Practice
CREATE TABLE CustomersInfo (
    CustomerID INT,
    FirstName VARCHAR(MAX),
    LastName VARCHAR(50), -- Fix
    Country VARCHAR(255),
    TotalPurchases FLOAT,
    Score INT, -- Fix
    BirthDate DATE, -- Fix
    EmployeeID INT,
    CONSTRAINT FK_CustomersInfo_EmployeeID FOREIGN KEY (EmployeeID)
        REFERENCES Sales.Employees(EmployeeID)
);
```

```
-- ================================================
-- Tip 22: Avoid (MAX) unnecessarily large lenghts
--         in data types
-- ================================================

-- Bad Practice
CREATE TABLE CustomersInfo (
    CustomerID INT,
    FirstName VARCHAR(MAX),
```

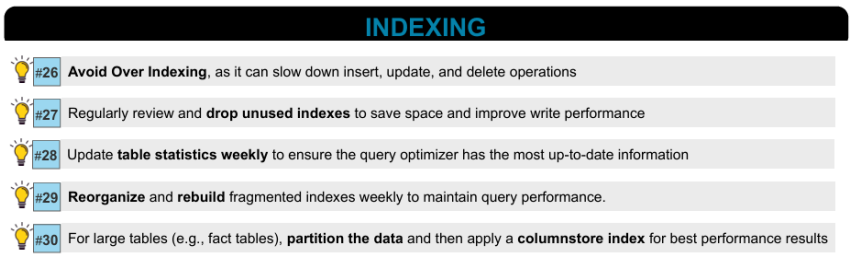
```
    LastName VARCHAR(50),
    Country VARCHAR(255),
    TotalPurchases FLOAT,
    Score INT,
    BirthDate DATE,
    EmployeeID INT,
    CONSTRAINT FK_CustomersInfo_EmployeeID FOREIGN KEY (EmployeeID)
        REFERENCES Sales.Employees(EmployeeID)
);

-- Good Practice
CREATE TABLE CustomersInfo (
    CustomerID INT,
    FirstName VARCHAR(50), -- Fix
    LastName VARCHAR(50),
    Country VARCHAR(50), -- Fix
    TotalPurchases FLOAT,
    Score INT,
    BirthDate DATE,
    EmployeeID INT,
    CONSTRAINT FK_CustomersInfo_EmployeeID FOREIGN KEY (EmployeeID)
        REFERENCES Sales.Employees(EmployeeID)
);
```

```
-- ================================================
-- Tip 23: Use the NOT NULL constraint where
--         applicable
-- ================================================

-- Bad Practice
CREATE TABLE CustomersInfo (
    CustomerID INT,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Country VARCHAR(50),
    TotalPurchases FLOAT,
    Score INT,
    BirthDate DATE,
    EmployeeID INT,
    CONSTRAINT FK_CustomersInfo_EmployeeID FOREIGN KEY (EmployeeID)
        REFERENCES Sales.Employees(EmployeeID)
);

-- Good Practice
CREATE TABLE CustomersInfo (
    CustomerID INT,
    FirstName VARCHAR(50) NOT NULL, -- Fix
    LastName VARCHAR(50) NOT NULL, -- Fix
    Country VARCHAR(50) NOT NULL, -- Fix
    TotalPurchases FLOAT,
    Score INT,
    BirthDate DATE,
    EmployeeID INT,
    CONSTRAINT FK_CustomersInfo_EmployeeID FOREIGN KEY (EmployeeID)
        REFERENCES Sales.Employees(EmployeeID)
);
```

```sql
-- ====================================================
-- Tip 24: Ensure all your tables have a Clustered
--         Primary Key
-- ====================================================

-- Bad Practice
CREATE TABLE CustomersInfo (
    CustomerID INT,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Country VARCHAR(50) NOT NULL,
    TotalPurchases FLOAT,
    Score INT,
    BirthDate DATE,
    EmployeeID INT,
    CONSTRAINT FK_CustomersInfo_EmployeeID FOREIGN KEY (EmployeeID)
        REFERENCES Sales.Employees(EmployeeID)
);

-- Good Practice
CREATE TABLE CustomersInfo (
    CustomerID INT PRIMARY KEY CLUSTERED, -- Fix
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Country VARCHAR(50) NOT NULL,
    TotalPurchases FLOAT,
    Score INT,
    BirthDate DATE,
    EmployeeID INT,
    CONSTRAINT FK_CustomersInfo_EmployeeID FOREIGN KEY (EmployeeID)
        REFERENCES Sales.Employees(EmployeeID)
);
```

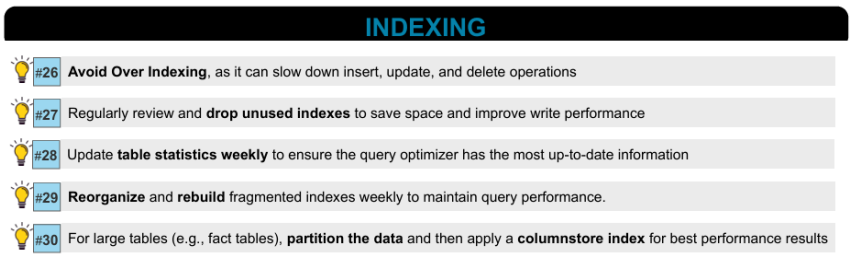
```sql
-- ====================================================
-- Tip 25: Create a non-clustered index for foreign
--         keys that are used frequently
-- ====================================================

-- Bad Practice
CREATE TABLE CustomersInfo (
    CustomerID INT PRIMARY KEY CLUSTERED,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Country VARCHAR(50) NOT NULL,
    TotalPurchases FLOAT,
    Score INT,
    BirthDate DATE,
    EmployeeID INT,
    CONSTRAINT FK_CustomersInfo_EmployeeID FOREIGN KEY (EmployeeID)
        REFERENCES Sales.Employees(EmployeeID)
);

-- Good Practice
CREATE NONCLUSTERED INDEX idx_CustomersInfo_EmployeeID
ON CustomersInfo(EmployeeID); -- Fix
```

Indexing



- Tip 26: Avoid Over Indexing
- Tip 27: Drop unused Indexes
- Tip 28: Update Statistics (Weekly)
- Tip 29: Reorganize & Rebuild Indexes (Weekly)

Partitioning



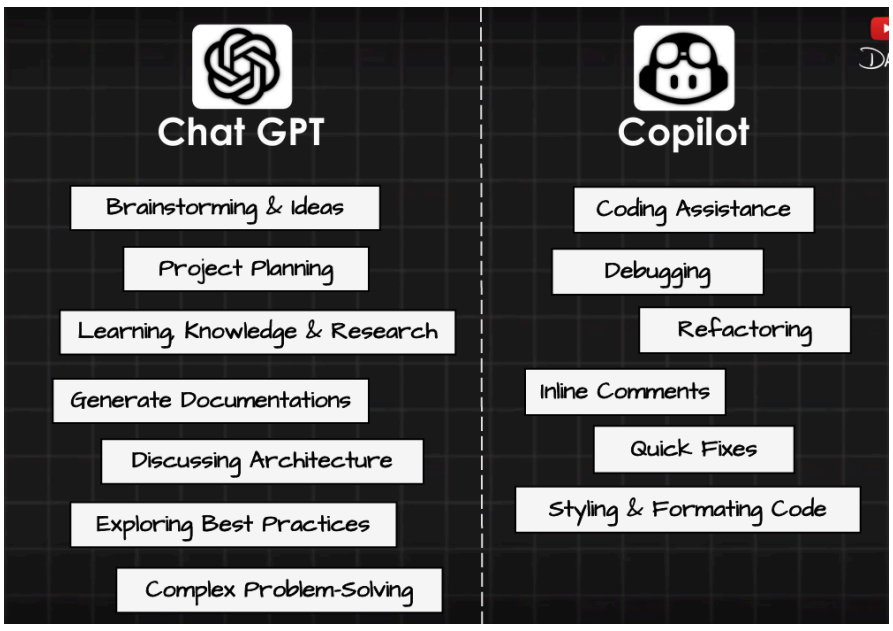
- Tip 30: Partition Large Tables (Facts) to improve performance
  - Next, apply a Columnstore Index for the best results

Summary

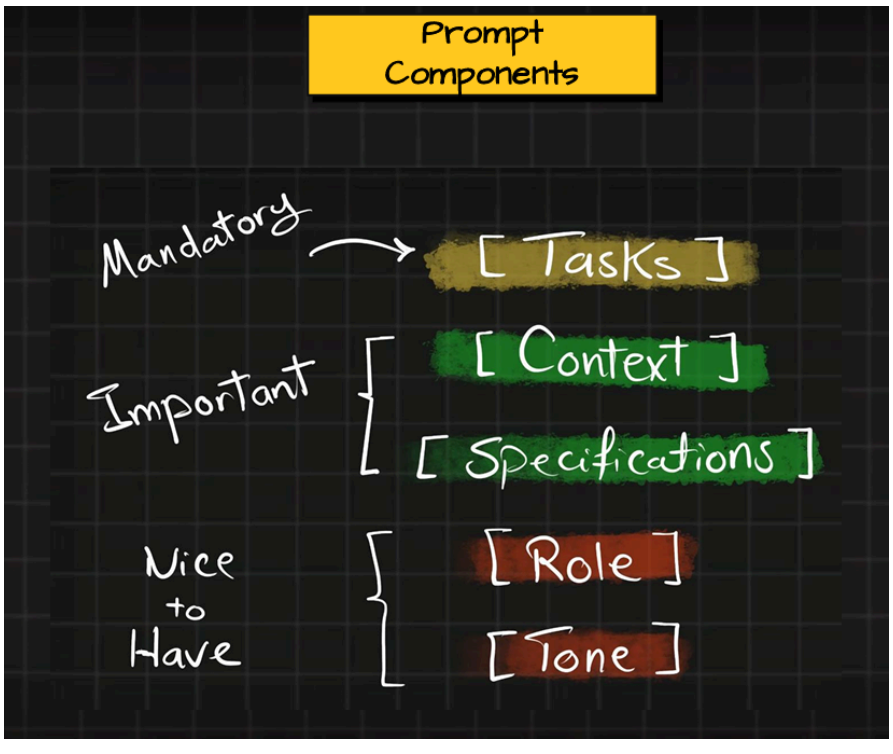
- Focus on Writing Clear Queries
- Optimize performance only when necessary
- Always Test Using Execution Plan

Module 26: AI & SQL

When to use ChatGPT & Copilot



ChatGPT Prompt



Top Prompts For Developers

1. Solve an SQL Task

```
In my SQL Server database, we have two tables:
The first table is 'orders' with the following columns: order_id, sales, customer_id, product_id.
The second table is 'customers' with the following columns: customer_id, first_name, last_name, country.
Do the following:
  Write a query to rank customers based on their sales.
  The result should include the customer's customer_id, full name, country, total sales, and their rank.
  Include comments but avoid commenting on obvious parts.
  Write three different versions of the query to achieve this task.
  Evaluate and explain which version is best in terms of readability and performance.
```

2. Improve the Readability

```
The following SQL Server query is long and hard to understand.
Do the following:
  Improve its readability.
  Remove any redundancy in the query and consolidate it.
  Include comments but avoid commenting on obvious parts.
  Explain each improvement to understand the reasoning behind it.
```

3. Optimize the Performance Query

```
The following SQL Server query is slow.
Do the following:
  Propose optimizations to improve its performance.
  Provide the improved SQL query.
  Explain each improvement to understand the reasoning behind it.
```

4. Optimize Execution Plan

```
The image is the execution plan of SQL Server query.
Do the following:
  Describe the execution plan step by step.
  Identify performance bottlenecks and issues.
  Suggest ways to improve performance and optimize the execution plan.
```

5. Debugging

```
The following SQL Server Query causing this error: [Error Message GOES HERE].
Do the following:
  Explain the error massage.
  Find the root cause of the issue.
  Suggest how to fix it.
```

6. Explain the Result

```
I didn't understand the result of the following SQL Server query.
Do the following:
   Break down how SQL processes the query step by step.
   Explaining each stage and how the result is formed.
```

7. Styling & Formatting

```
The following SQL Server query hard to understand.
Do the following:
   Restyle the code to make it easier to read.
   Align column aliases.
   Keep it compact - do not introduce unnecessary new lines.
   Ensure the formatting follows best practices.
```

8. Documentations & Comments

```
The following SQL Server query lacks comments and documentation.
Do the following:
   Insert a leading comment at the start of the query describing its overall purpose.
   Add comments only where clarification is necessary, avoiding obvious statements.
   Create a separate document explaining the business rules implemented by the query.
   Create another separate document describing how the query works.
```

9. Improve Database DDL

```
The following SQL Server DDL Script has to be optimized.
Do the following:
   Naming: Check the consistency of table/column names, prefixes, standards.
   Data Types: Ensure data types are appropriate and optimized.
   Integrity: Verify the integrity of primary keys and foreign keys.
   Indexes: Check that indexes are sufficient and avoid redundancy.
   Normalization: Ensure proper normalization and avoid redundancy.
```

10. Generate Test Dataset

```
I need dataset for testing the following SQL Server DDL.
Do the following:
   Generate test dataset as Insert statements.
   Dataset should be realstic.
   Keep the dataset small.
   Ensure all primary/foreign key relationships are valid (use matching IDs).
   Dont introduce any Null values.
```

11. Migrating from a DB to another DB

```
Convert the following SQL query from SQL Server to MySQL
```

Top Prompts For Learning

1. Create SQL Course

```
Create a comprehensive SQL course with a detailed roadmap and agenda.
Do the following:
   Start with SQL fundamentals and advance to complex topics.
   Make it beginner-friendly.
   Include topics relevant to data analytics.
   Focus on real-world data analytics use cases and scenarios.
```

2. Understand SQL Concept

```
I want detailed explanation about SQL Window Functions.
Do the following:
   Explain what Window Functions are.
   Give an analogy.
   Describe why we need them and when to use them.
   Explain the syntax.
   Provide simple examples.
   List the top 3 use cases.
```

3. Comparing SQL Concepts

```
I want to understand the differences between SQL Windows and GROUP BY.
Do the following:
   Explain the key differences between the two concepts.
   Describe when to use each concept, with examples.
   Provide the pros and cons of each concept.
   Summarize the comparison in a clear side-by-side table.
```

4. Practice SQL

```
Act as an SQL trainer and help me practice SQL Window Functions.
Do the following:
   Make it interactive Practicing, you provide task and give solution.
   Provide a sample dataset.
   Give SQL tasks that gradually increase in difficulty.
   Act as an SQL Server and show the results of my queries.
   Review my queries, provide feedback, and suggest improvements.
```

5. Prepare for a SQL Interview

```
Act as Interviewer and prepare me for a SQL interview.
Do the following:
   Ask common SQL interview questions.
   Make it interactive Practicing, you provide question and give answer.
   Gradually progress to advanced topics.
   Evaluate my answer and give me a feedback.
```

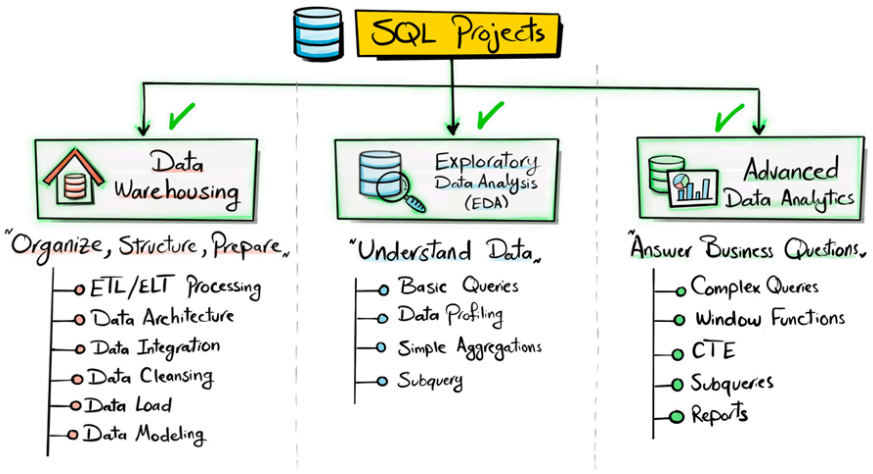
6. Prepare for a SQL Exam

```
Prepare me for a SQL exam.
Do the following:
   Ask common SQL exam questions.
   Make it interactive Practicing, you provide question and give answer.
   Gradually progress to advanced topics.
   Evaluate my answer and give me a feedback.
```

Module 27: SQL Data Warehouse Project

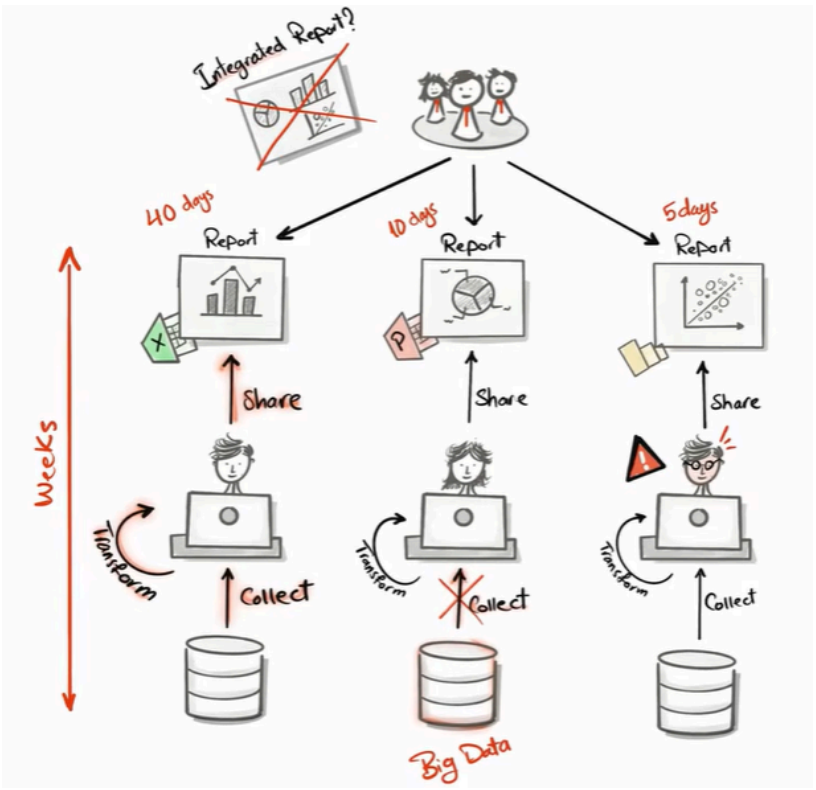
Project 1: Data Warehousing & ETL Architecture

SQL Projects

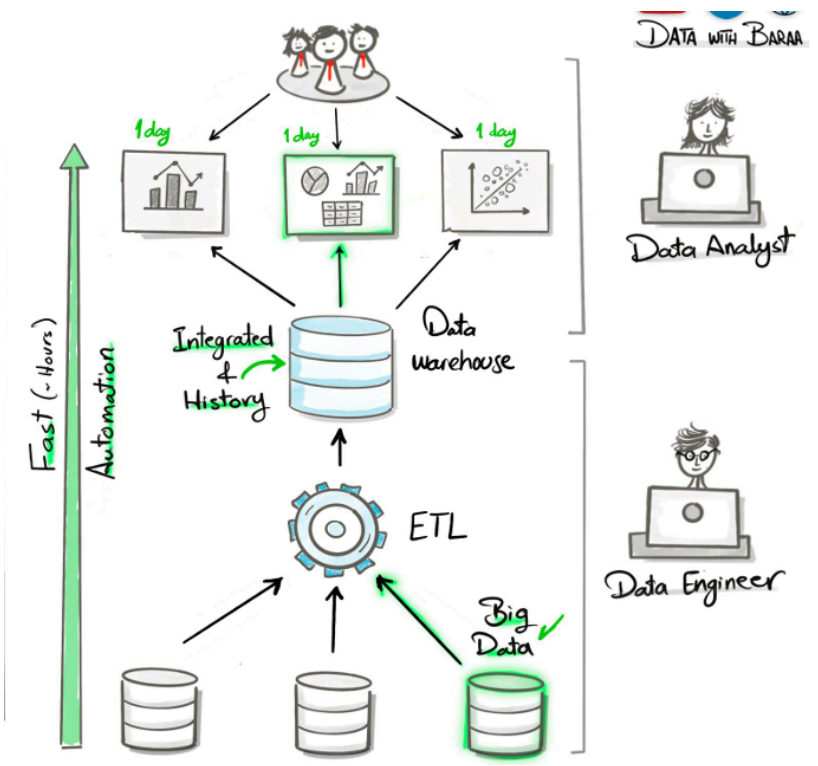


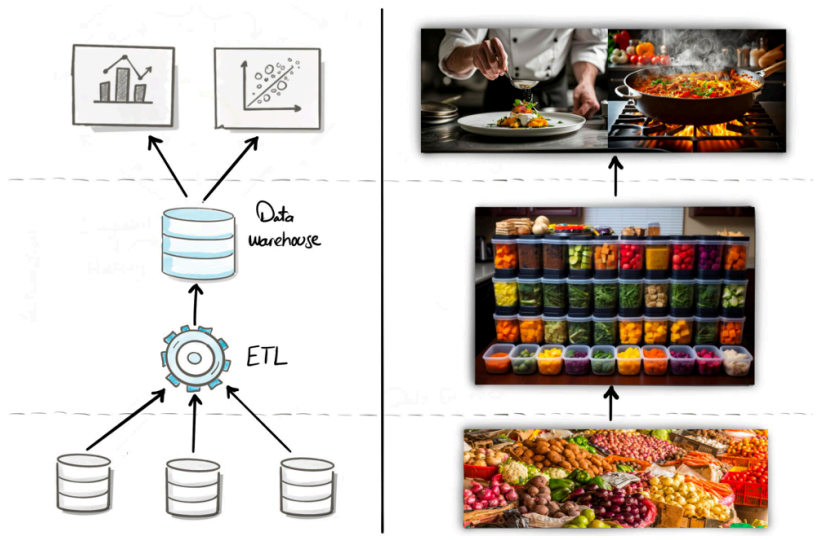
Data Warehouse (DWH)

- A subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management's decision-making process
- Do not have a real data management



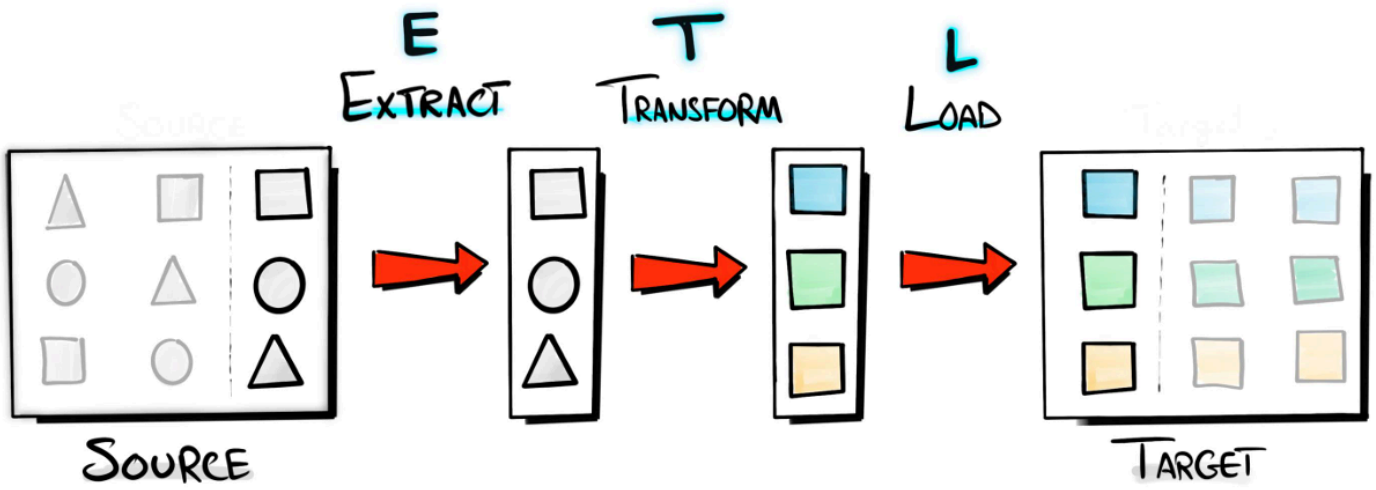
- Have a Data Warehouse

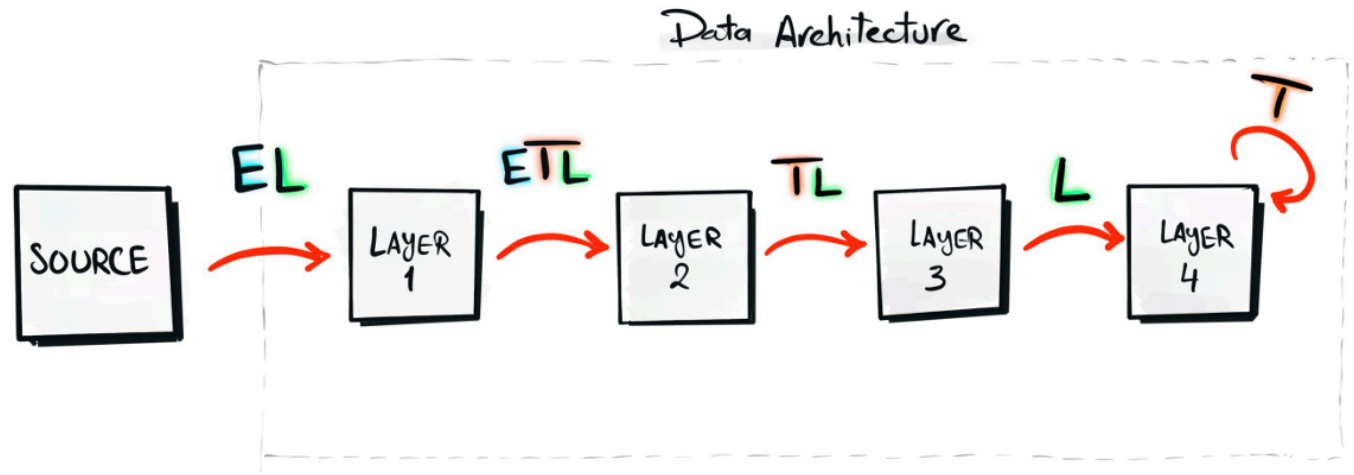


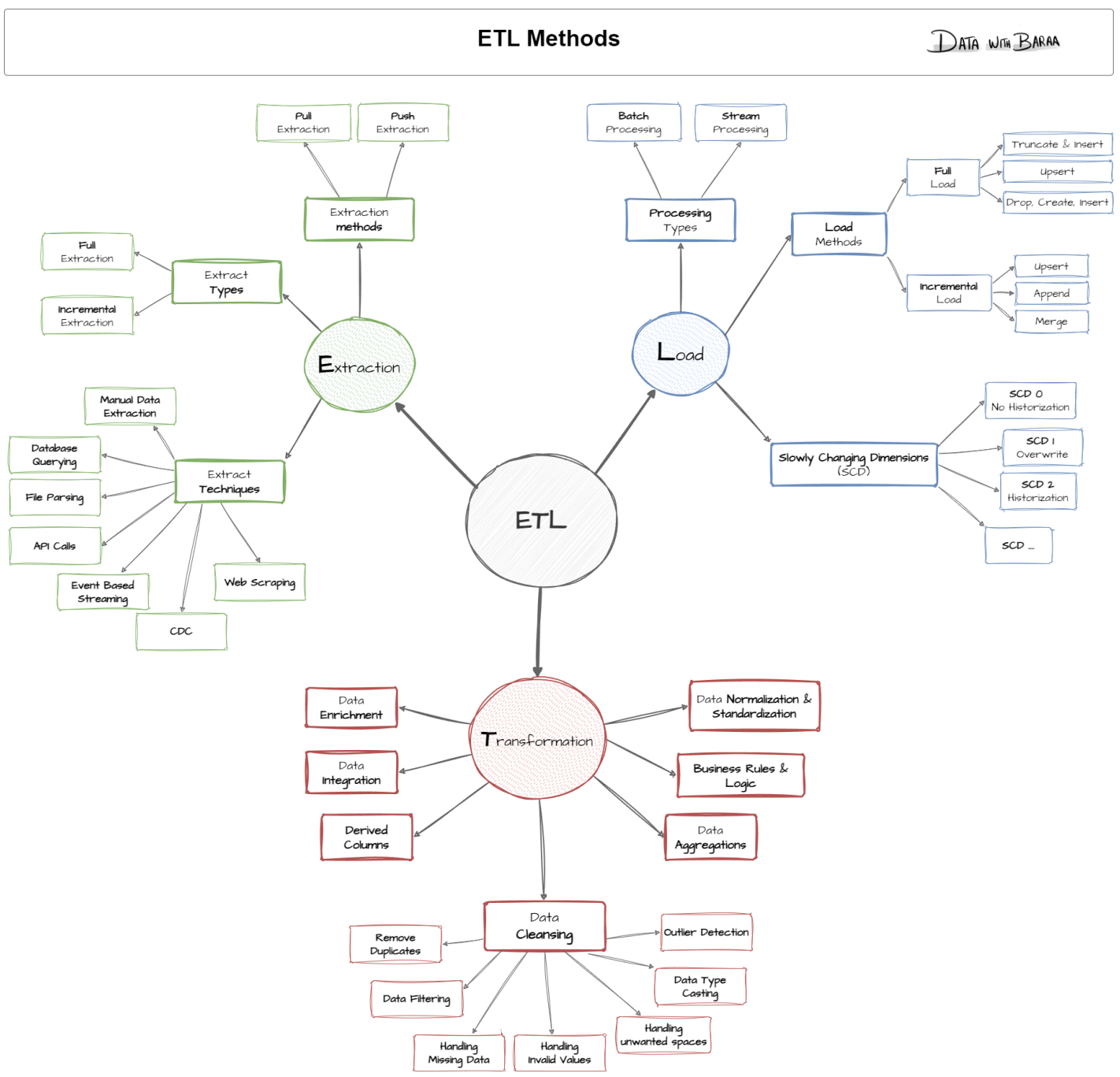


ETL

- Extract Transform Load



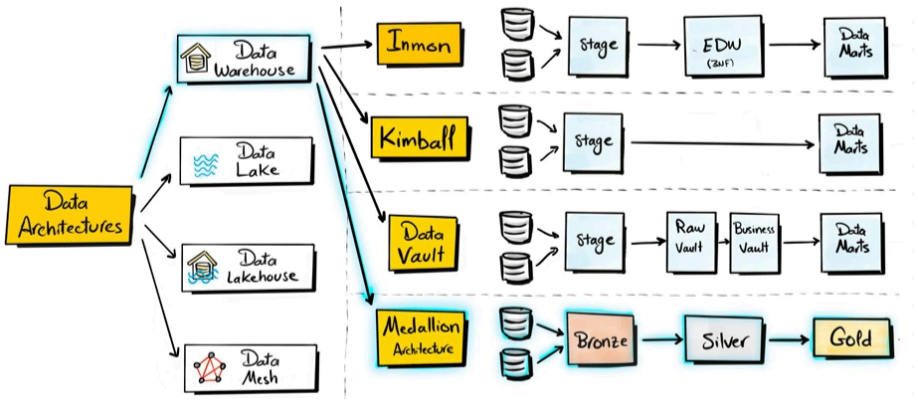




- Which of those types we will be using in our project?
  - Extraction
    - Pull Extraction

        - Full Extraction
        - File Parsing
    - Transform
        - Everything
    - Load
        - Batch Processing
        - Full Load → Truncate & Insert
        - SCD 1 (Overwrite)

Data Architectures

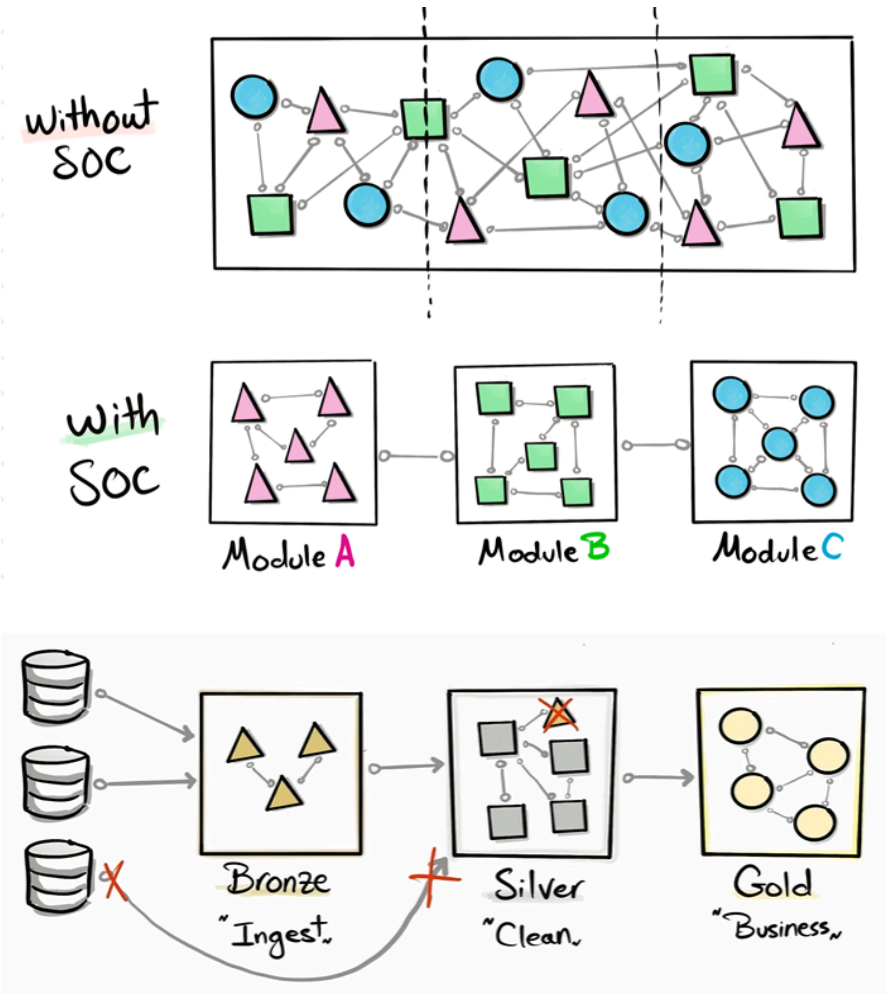


- Data Warehouse
    - It’s very suitable if you have only structured data
    - Your business want to build solid foundations for reporting and business intelligence
- Data Lake
    - This one is way more flexible than data warehouse
    - You can store not only structured data but as well semi and unstructured data
    - We usually use this approach if you have mixed types of data like database tables, logs, images, and videos
    - Your business want to focus not only for reporting but as well on advanced analytics or machine learning
    - But it’s not organized like a data warehouse
    - If it’s too much unorganized, it will turn into data swamp
- Data Lakehouse
    - Mix between data warehouse and data lake
    - Get the flexibility of having different types of data from the data lake
    - But you still want to structure and organize your data like we do in data warehouse
- Data Mesh
    - Instead of having centralized data management system, you will make it decentralized

Design the Layers

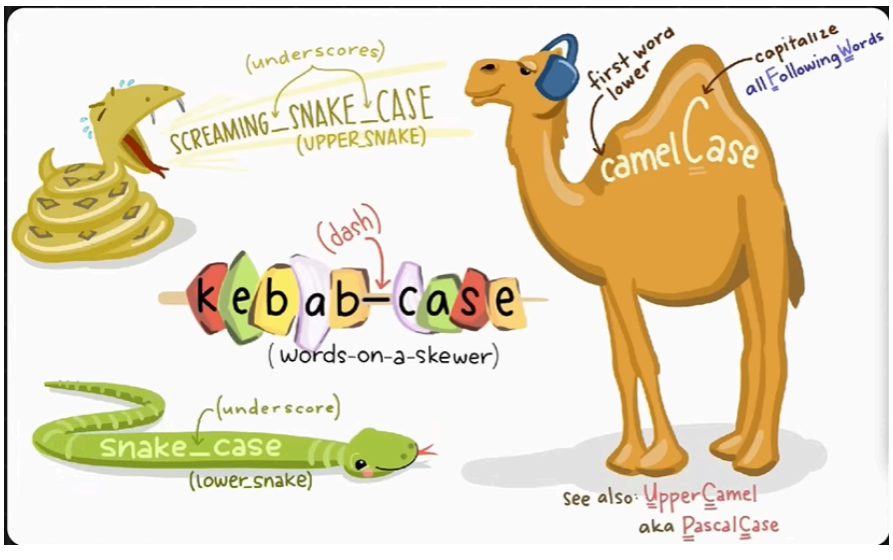
DATA with BARAA	Bronze Layer	Silver Layer	Gold Layer
Definition	Raw, unprocessed data as-is from sources	Clean & standardized data	Business-Ready data
Objective	Traceability & Debugging	(Intermediate Layer) Prepare Data for Analysis	Provide data to be consumed for reporting & Analytics
Object Type	Tables	Tables	Views
Load Method	Full Load (Truncate & Insert)	Full Load (Truncate & Insert)	None
Data Transformation	None (as-is)	- Data **Cleaning** - Data **Standardization** - Data **Normalization** - **Derived Columns** - Data **Enrichment**	- Data **Integration** - Data **Aggregation** - **Business Logic & Rules**
Data Modeling	None (as-is)	None (as-is)	- **Start Schema** - **Aggregated Objects** - **Flat Tables**
Target Audience	- Data Engineers	- **Data Analysts** - **Data Engineers**	- **Data Analysts** - **Business Users**

Separation of Concerns (SOC)

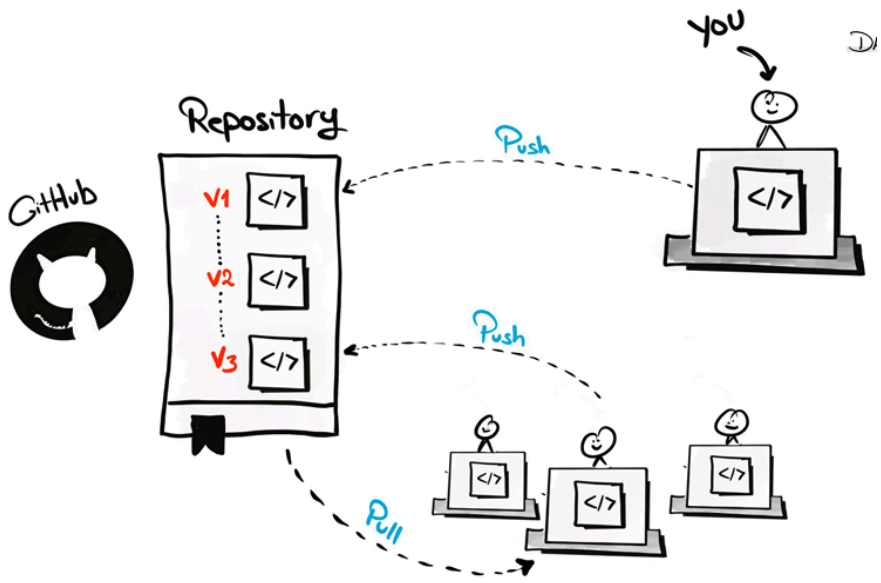


Naming Conventions

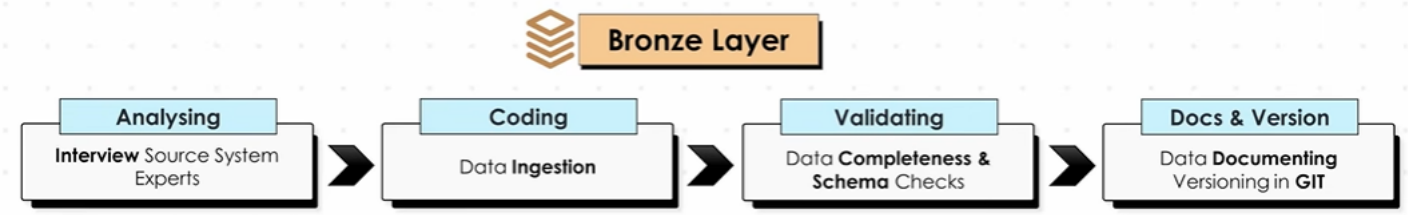
- Set of Rules or Guidelines for naming anything in the project
  - Database
  - Schema
  - Tables
  - Stored Procedures ....



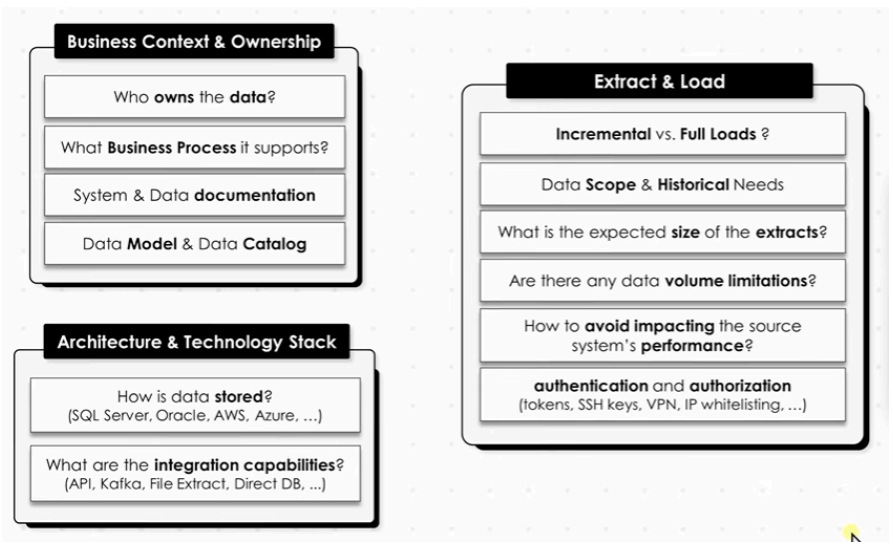
Git



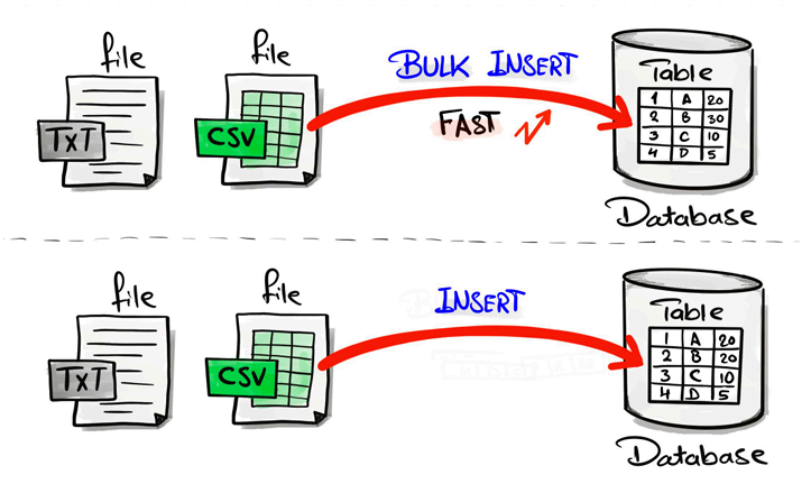
Bronze Layer



- Analyzing



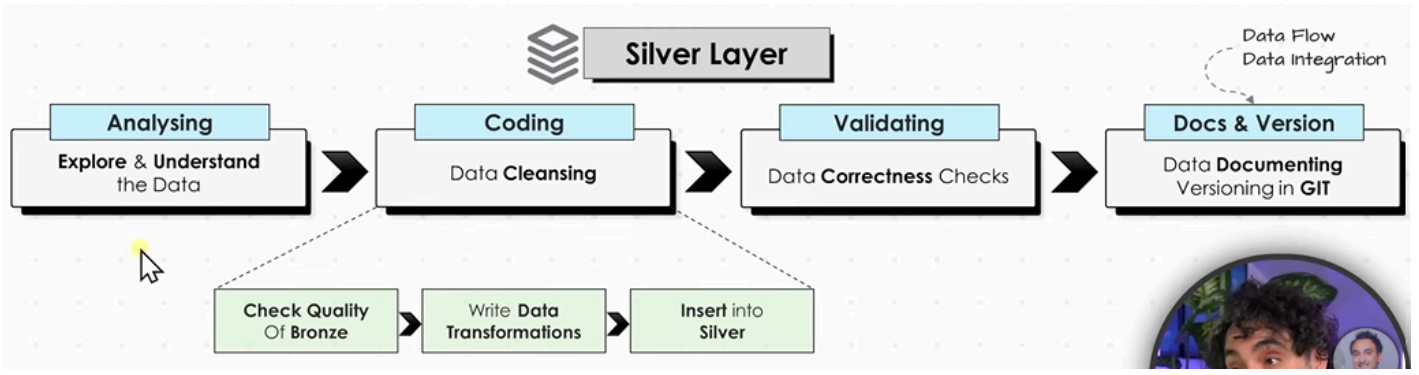
- Coding & Validating
  - Consult the technical experts of the source system to understand its metadata
  - Data Profiling
    - Explore the data to identify column names and data types



  - Quality Check
    - Check that the data has not shifted and is in the correct columns
  - HINT
    - Save frequently used SQL code in stored procedures in database

    - Add PRINTS
        - Add prints to track execution, debug issues, and understand its flow
    - Add TRY...CATCH
        - Ensures error handling, data integrity, and issue logging for easier debugging
        - SQL runs the TRY block, and if it fails, it runs the CATCH block to handle the error
    - Track ETL Duration
        - Helps to identify bottlenecks, optimize performance, monitor trends, and detect issues
- Docs & Version
    - Data Documenting Versioning in GIT

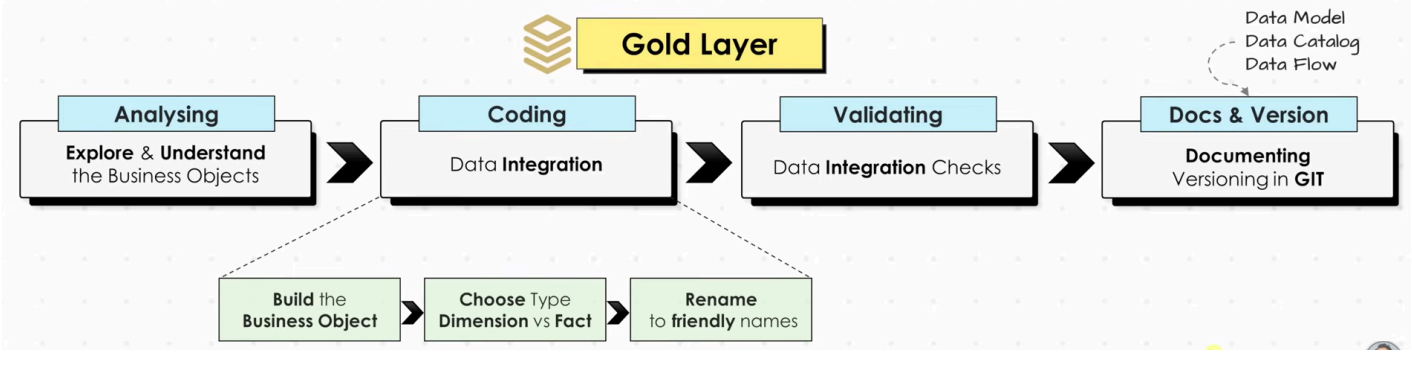
Silver Layer



- Analyzing
    - Explore & Understand the Data
- Coding
    - Data Cleansing
        - Check Quality of Bronze
        - Write Data Transformations
        - Insert into Silver
    - Metadata Columns
        - Extra columns added by data engineers that do not originate from the source data
        - create_date: The record’s load timestamp
        - update_date: The record’s last update timestamp
        - source_system: The origin system of the record
        - file_location: The file source of the record
    - Remove Unwanted spaces
        - Remove unnecessary spaces to ensure data consistency, and uniformity across all records
    - Data Normalization & Standardization
        - Maps coded values to meaningful, user-friendly descriptions
    - Handling Missing Data
        - Fills in the blanks by adding a default value
    - Remove Duplicates
        - Ensure only one record per entity identifying and retaining the most relevant row
- Validating
    - Data Correctness Checks
- Docs & Version

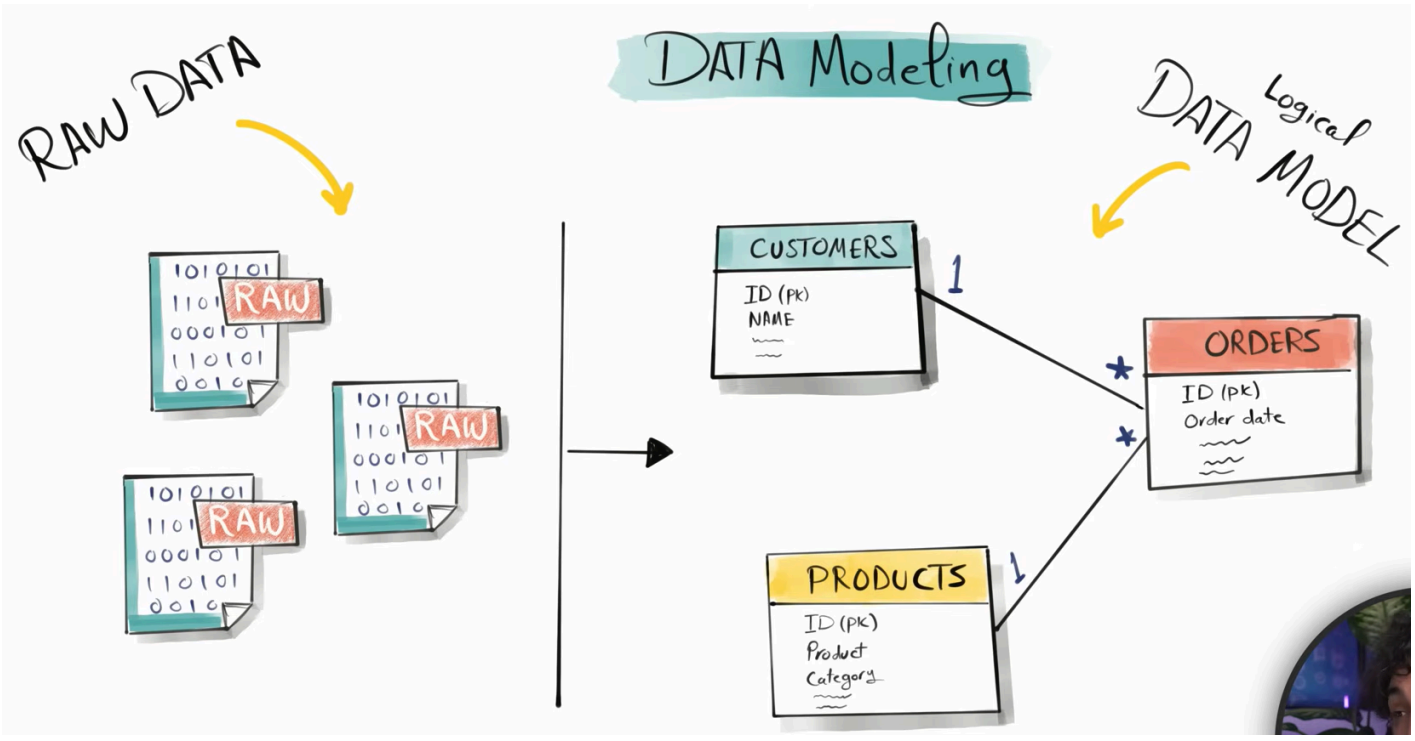
- Data Documenting Versioning in GIT
  - Data Flow Diagram
  - Data Integration Diagram

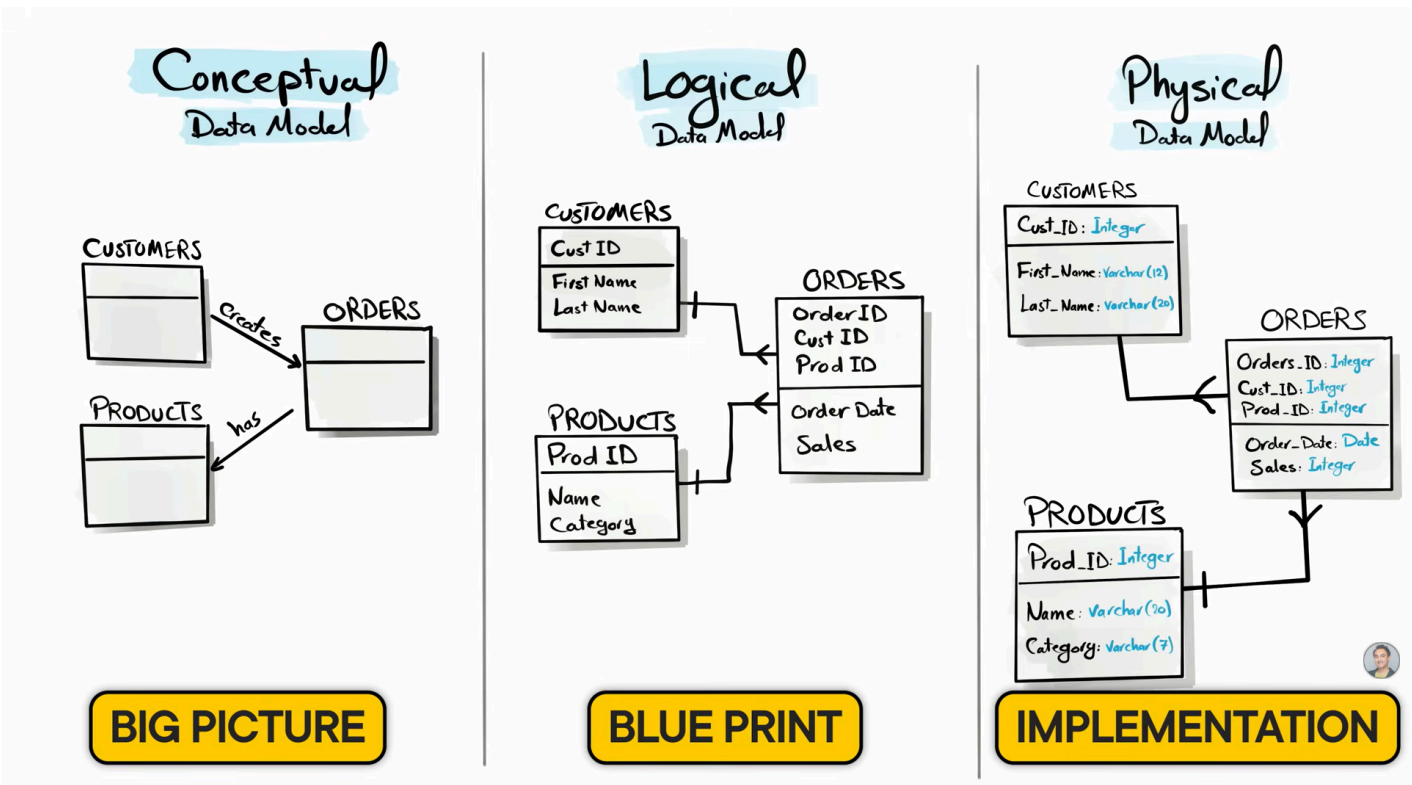
Gold Layer



- Analyzing
  - Explore & Understand the Business Objects
- Coding
  - Data Integration
    - Build the Business Object
    - Choose Type (Dimension vs Fact)
    - Rename to friendly names
- Validating
  - Data Integration Checks
- Docs & Version
  - Documenting Versioning in GIT
    - Data Model Diagram
    - Data Catalog
    - Data Flow Diagram

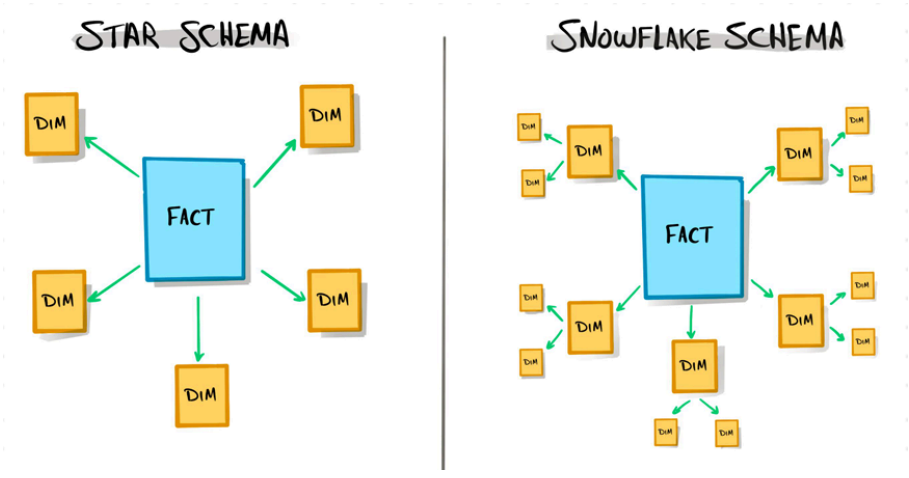
Data Modeling





Theory: Star Schema vs Snowflake Schema

- Optimized for performance
- Flexible
- Ease to understand



- Fact Table
  - Contains transactions, and events
- Dimension Table
  - Contains descriptive information
- Star Schema
  - Pros: Simple & Easy
  - Cons: Big Dimensions
- Snowflake Schema
  - Pros: Large Datasets
  - Cons: More Complex

Theory: Dimensions vs Facts

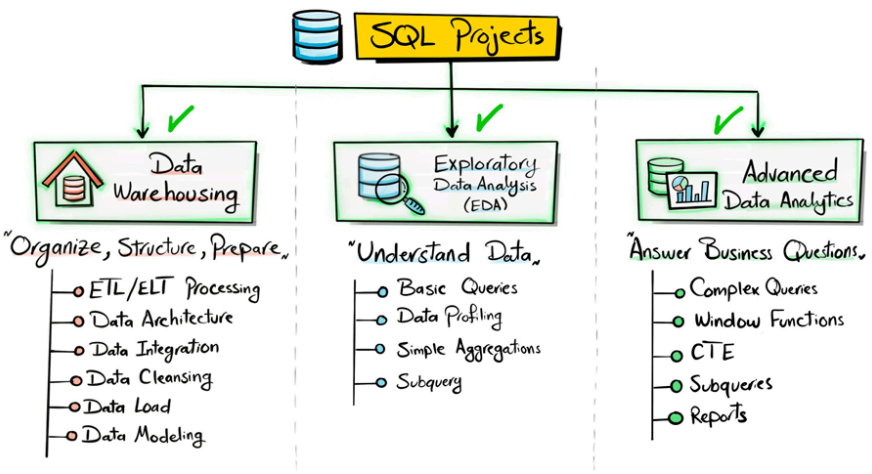
- Dimensions
  - Descriptive information that give context to your data
  - Who? What? Where?
- Facts
  - Quantitative information that represents events

    - Multiple Ids from Multiple Dimensions
    - Date Information
    - Measure & Numbers
  - How much? How many?

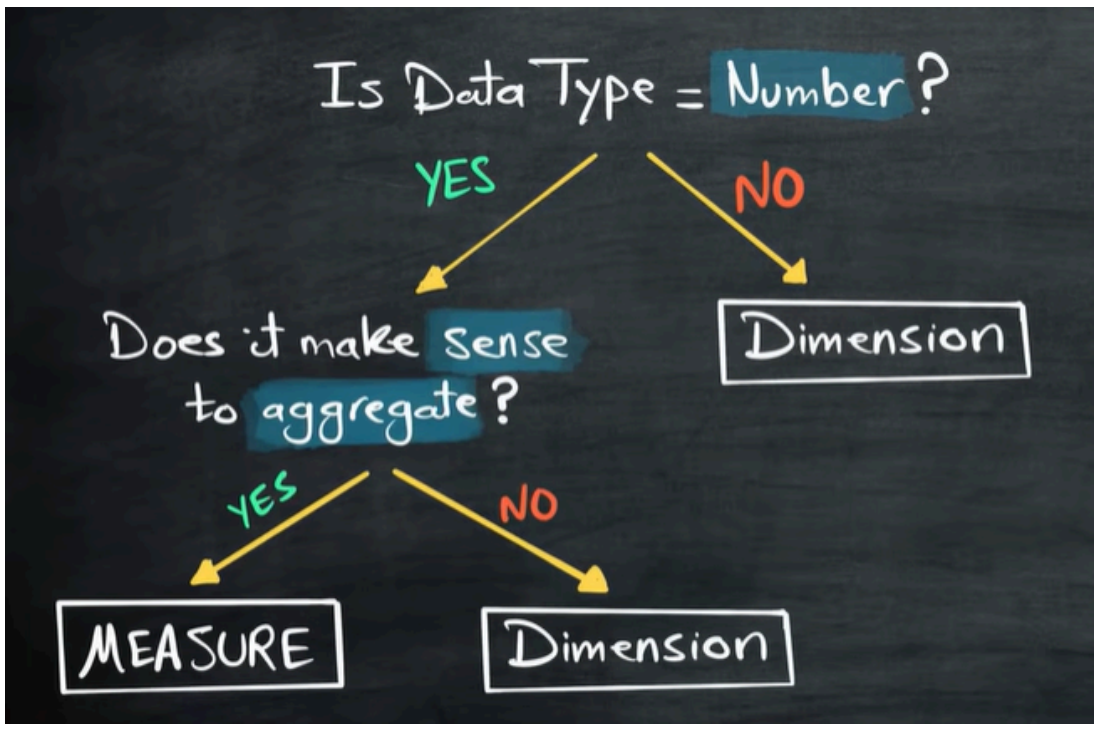
Module 28: Exploratory & Advanced Data Analytics Project

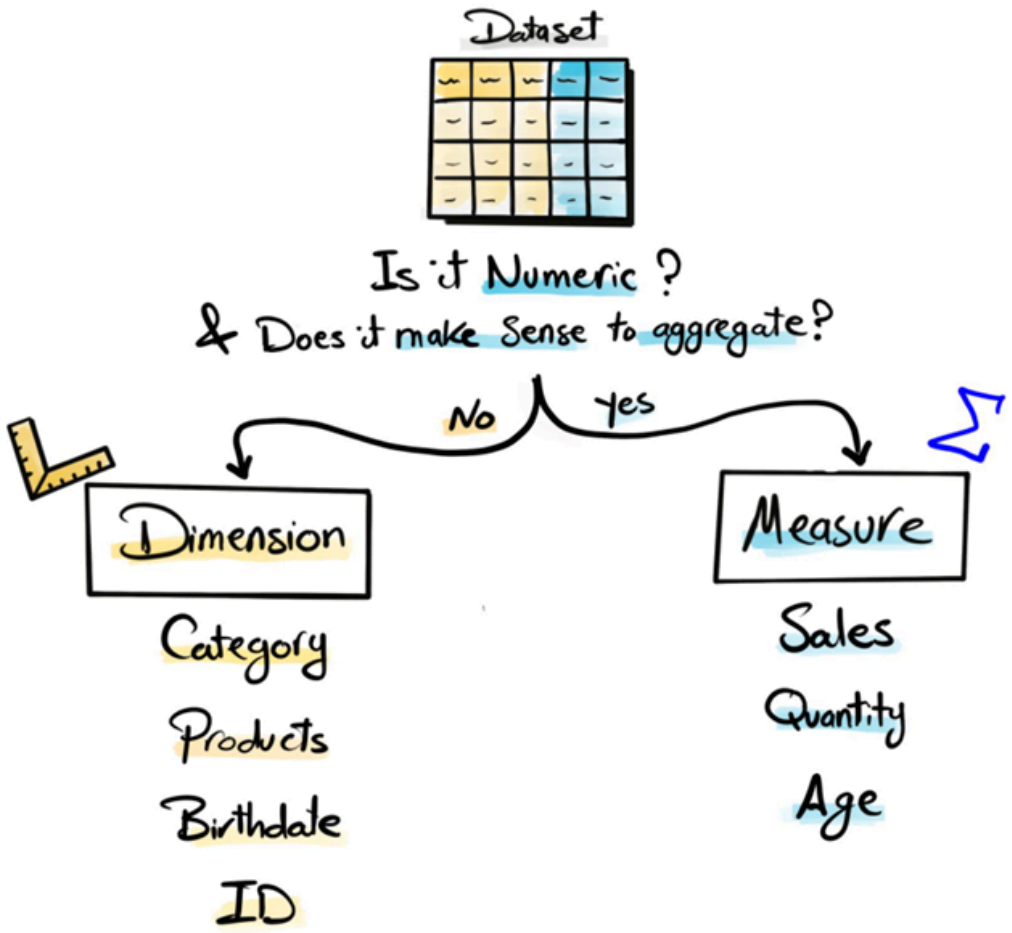
Project 2 & 3: Exploratory & Advanced Data Analytics

SQL Projects

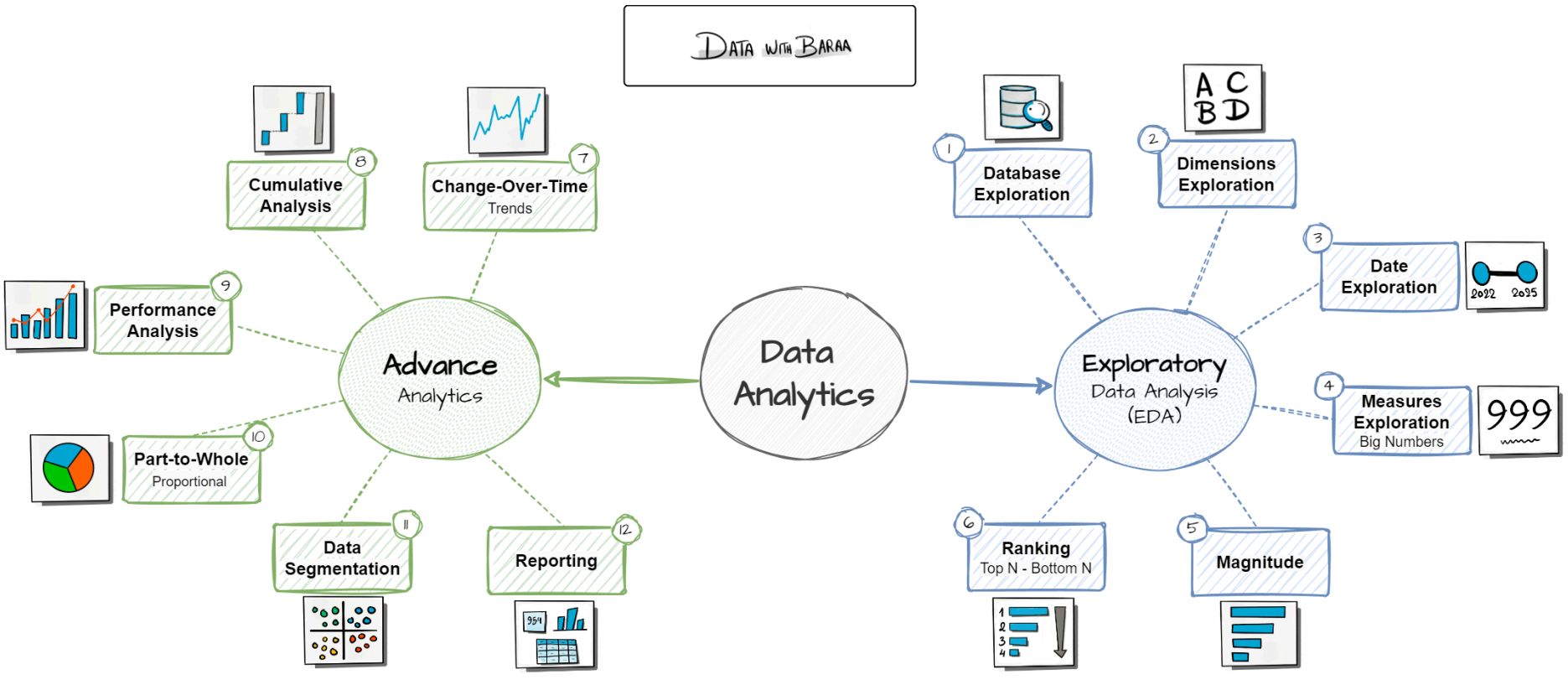


Dimensions vs Measures





Data Analytics Project Roadmap

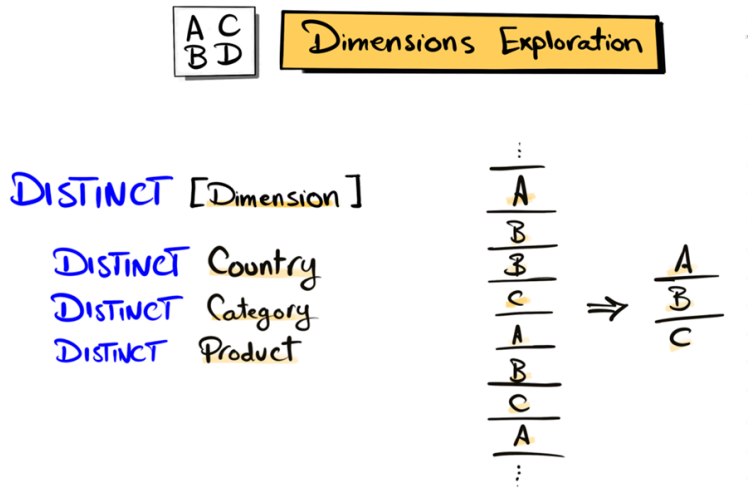


Exploratory Data Analysis (EDA)

Database Exploration

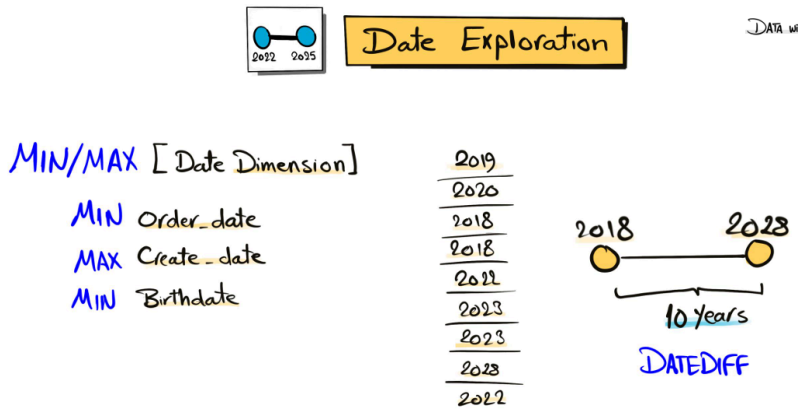
Dimension Exploration

- Identifying the unique values (or categories) in each dimension
- Recognizing how data might be grouped or segmented, which is useful for later analysis



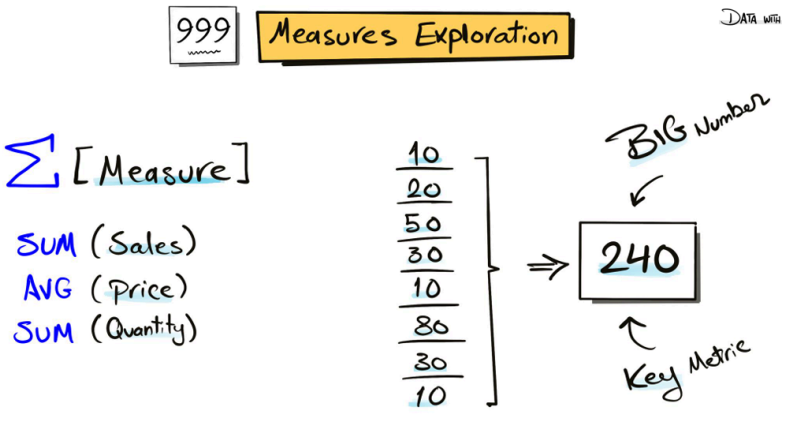
Data Exploration

- Identify the earliest and latest dates (boundaries)
- Understand the scope of data and the timespan



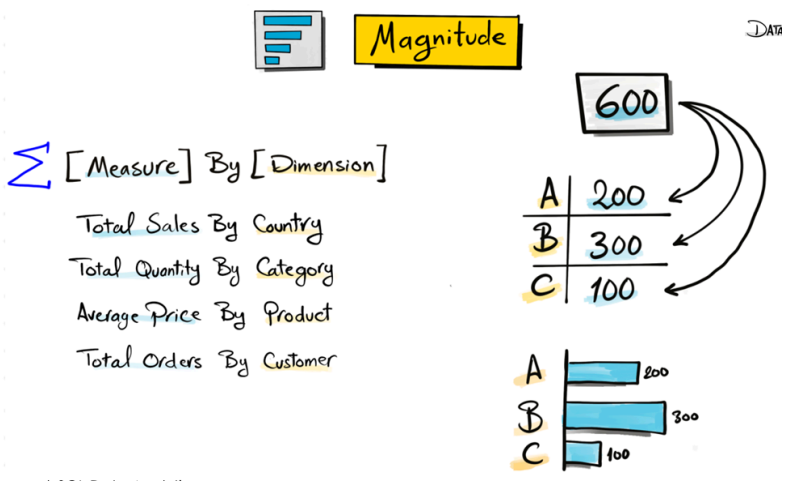
Measure Exploration

- Calculate the key metric of the business (Big Numbers)
- Highest Level of Aggregation | Lowest Level of Details



Magnitude Exploration

- Compare the measure values by categories
- It helps us understand the importance of different categories

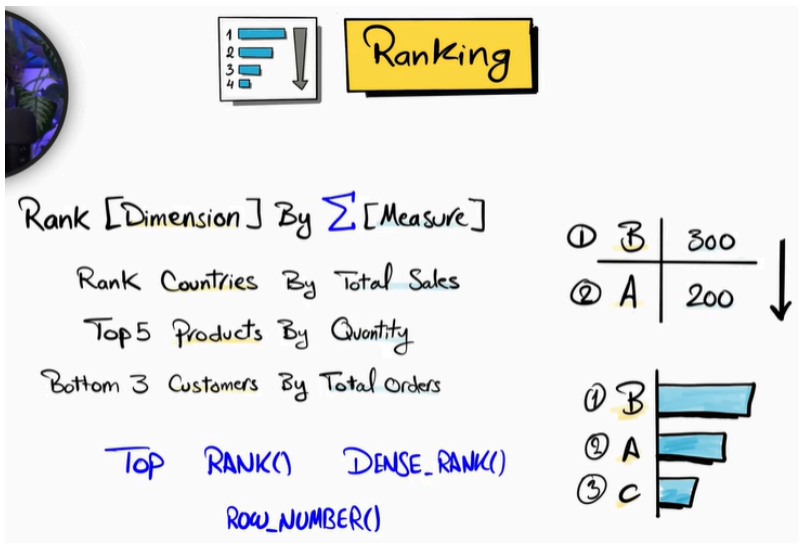


- Low Cardinality Dimension

    - Dimension with few unique values (e.g. Category, Country, Gender, ...)
- High Cardinality Dimension
    - Dimension with large number of unique values (e.g. Product, Customer, Address, ...)

Ranking Analysis

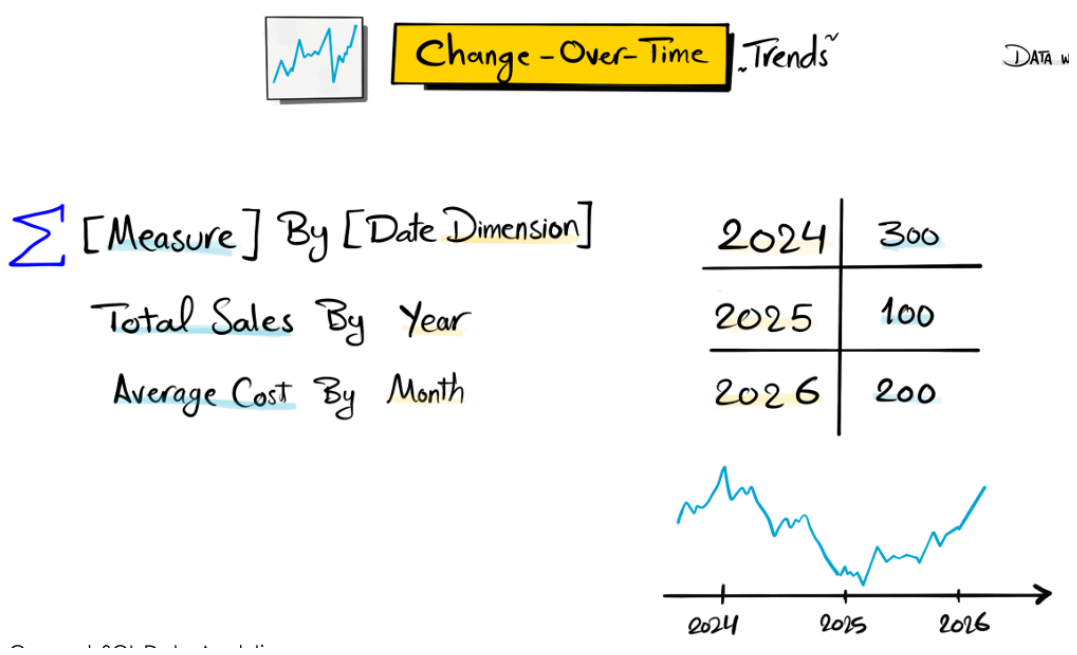
- Order the values of dimensions by measure
- Top N Performers | Bottom N Performers



Advanced Data Analytics

Change Over Time Analysis

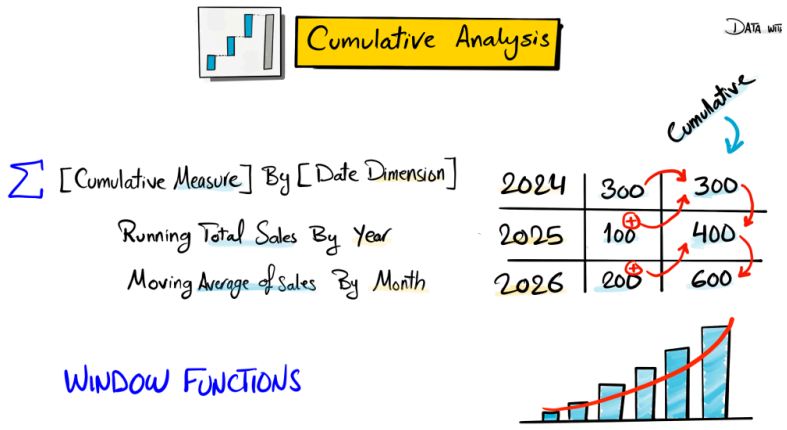
- Analyze how a measure evolves over time
- Helps track trends and identify seasonality in your data



- Changes Over Years
    - A high-level overview insights that helps with strategic decision-making
- Changes Over Months
    - Detailed insight to discover seasonality in your data

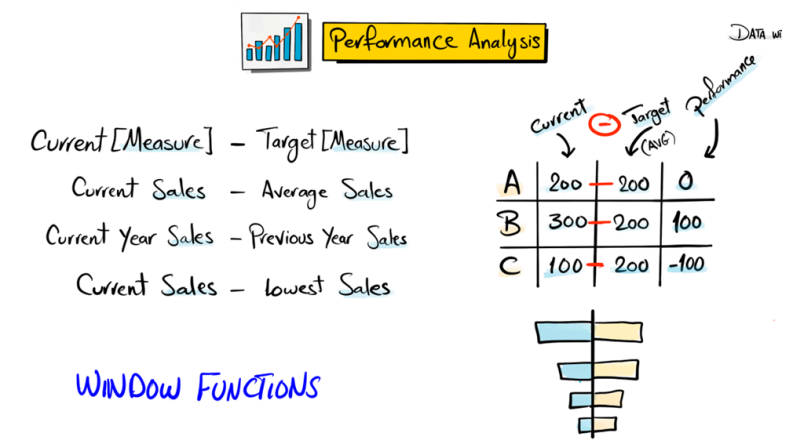
Cumulative Analysis

- Aggregate the data progressively over time
- Helps to understand whether our business is growing or declining



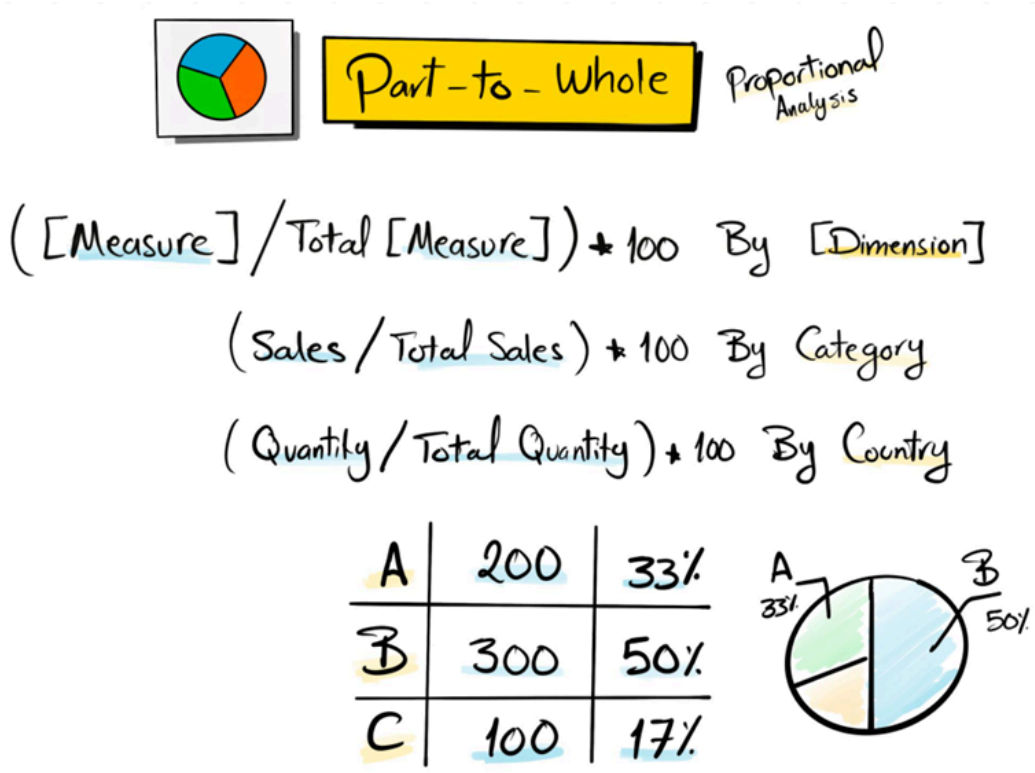
Performance Analysis

- Comparing the current value to a target value
- Helps measure success and compare performance



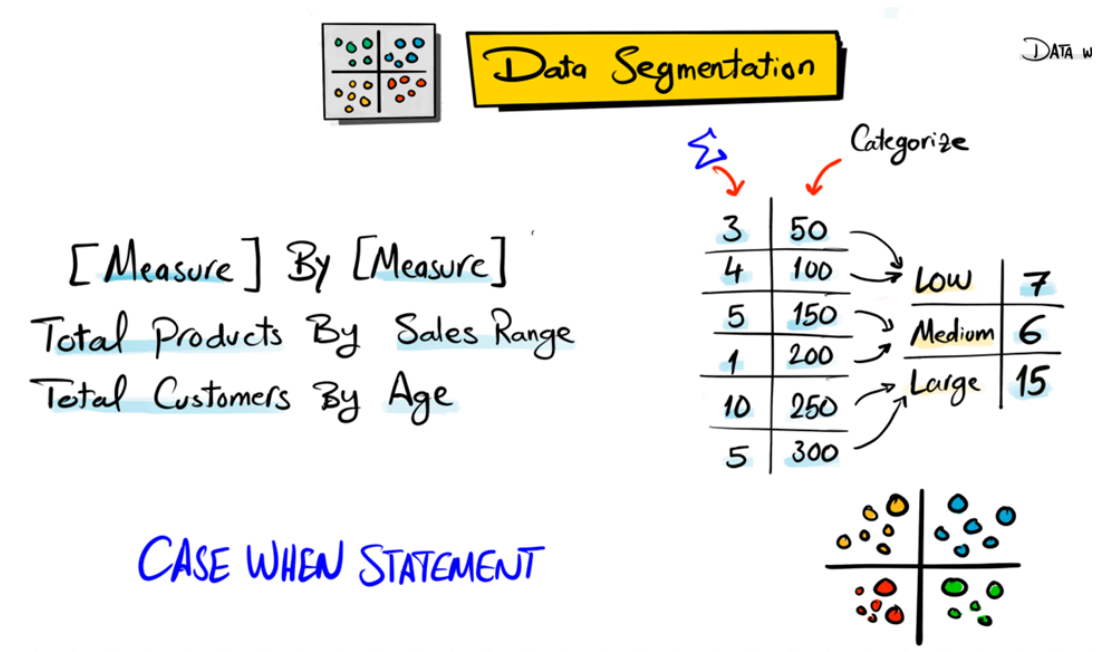
Part to Whole Analysis

- Analyze how an individual part is performing compared to the overall, allowing us to understand which category has the greatest impact on the business



Data Segmentation

- Group the data based on a specific range
- Helps understand the correlation between two measures



Build Report